

Studies in Computational Intelligence 1050

Oscar Castillo
Patricia Melin *Editors*

New Perspectives on Hybrid Intelligent System Design based on Fuzzy Logic, Neural Networks and Metaheuristics

Studies in Computational Intelligence

Volume 1050

Series Editor

Janusz Kacprzyk, Polish Academy of Sciences, Warsaw, Poland

The series “Studies in Computational Intelligence” (SCI) publishes new developments and advances in the various areas of computational intelligence—quickly and with a high quality. The intent is to cover the theory, applications, and design methods of computational intelligence, as embedded in the fields of engineering, computer science, physics and life sciences, as well as the methodologies behind them. The series contains monographs, lecture notes and edited volumes in computational intelligence spanning the areas of neural networks, connectionist systems, genetic algorithms, evolutionary computation, artificial intelligence, cellular automata, self-organizing systems, soft computing, fuzzy systems, and hybrid intelligent systems. Of particular value to both the contributors and the readership are the short publication timeframe and the world-wide distribution, which enable both wide and rapid dissemination of research output.

Indexed by SCOPUS, DBLP, WTI Frankfurt eG, zbMATH, SCImago.

All books published in the series are submitted for consideration in Web of Science.

Oscar Castillo · Patricia Melin
Editors

New Perspectives on Hybrid Intelligent System Design based on Fuzzy Logic, Neural Networks and Metaheuristics

Editors

Oscar Castillo
Division of Graduate Studies and Research
Tijuana Institute of Technology
Tijuana, Baja California, Mexico

Patricia Melin
Division of Graduate Studies and Research
Tijuana Institute of Technology
Tijuana, Baja California, Mexico

ISSN 1860-949X

ISSN 1860-9503 (electronic)

Studies in Computational Intelligence

ISBN 978-3-031-08265-8

ISBN 978-3-031-08266-5 (eBook)

<https://doi.org/10.1007/978-3-031-08266-5>

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2022

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors, and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

Preface

We describe in this book, recent developments on fuzzy logic, neural networks and metaheuristic optimization algorithms, as well as their hybrid combinations, and their application in areas such as intelligent control and robotics, pattern recognition, medical diagnosis, time series prediction and optimization of complex problems. There are papers with the main topics of type-1 and type-2 fuzzy logic, which basically consists of papers that propose new concepts and algorithms based on type-1 and type-2 fuzzy logic and their applications. There are also papers that present theory and practice of metaheuristics in diverse areas of application. There are interesting papers on different applications of fuzzy logic, neural networks and hybrid intelligent systems in medical problems. In addition, we can find papers describing applications of fuzzy logic, neural networks and metaheuristics in robotics problems. Another set of papers are presenting theory and practice of neural networks in different areas of application, including convolutional and deep learning neural networks. There are also a group of papers that present theory and practice of optimization and evolutionary algorithms in different areas of application. Finally, we can find a set of papers describing applications of fuzzy logic, neural networks and metaheuristics in pattern recognition problems. The papers of the book are organized into six parts that group together papers with similar topics or applications.

In conclusion, the edited book comprises papers on diverse aspects of fuzzy logic, neural networks and nature-inspired optimization metaheuristics for designing and implementing hybrid intelligent systems and their application in areas such as intelligent control and robotics, pattern recognition, time series prediction and optimization of complex problems. There are theoretical aspects as well as application papers. We expect that the book will serve as reference for researchers and graduate students working in the computational intelligence area.

Tijuana, Mexico
November 2021

Patricia Melin
Oscar Castillo

About This Book

In this book, recent developments on fuzzy logic, neural networks and optimization algorithms, as well as their hybrid combinations, are presented. In addition, the above-mentioned methods are applied to areas such as intelligent control and robotics, pattern recognition, medical diagnosis, time series prediction and optimization of complex problems. Nowadays, the main topic of the book is highly relevant, as most current intelligent systems and devices in use, utilize some form of intelligent feature to enhance their performance. In addition, on the theoretical side, new and advanced models and algorithms of type-2 fuzzy logic are presented, which will be of great interest to researchers in these areas. Also, new nature-inspired optimization algorithms and innovative neural models are put forward in the manuscript, that are very popular subjects, at this moment. There are contributions on theoretical aspects as well as applications, which make the book very appealing to a wide audience, ranging from researchers to professors and graduate students.

Contents

Neural Networks

Automated Medical Diagnosis and Classification of Skin Diseases Using Efficinetnet-B0 Convolutional Neural Network	3
Harsh Gunwant, Abhisht Joshi, Moolchand Sharma, and Deepak Gupta	
Modular Approach for Neural Networks in Medical Image Classification with Enhanced Fuzzy Integration	21
Sergio Varela-Santos and Patricia Melin	
Clustering and Prediction of Time Series for Traffic Accidents Using a Nested Layered Artificial Neural Network Model	37
Martha Ramirez and Patricia Melin	
Ensemble Recurrent Neural Networks and Their Optimization by Particle Swarm for Complex Time Series Prediction	47
Martha Pulido and Patricia Melin	
Filter Estimation in a Convolutional Neural Network with Type-2 Fuzzy Systems and a Fuzzy Gravitational Search Algorithm	63
Yutzil Poma and Patricia Melin	

Optimization

Artificial Fish Swarm Algorithm for the Optimization of a Benchmark Set of Functions	77
Cinthia Peraza, Patricia Ochoa, Leticia Amador, and Oscar Castillo	
Hierarchical Logistics Methodology for the Routing Planning of the Package Delivery Problem	93
Norberto Castillo-García, Paula Hernández-Hernández, and Edilberto Rodríguez Larkins	

A Novel Distributed Nature-Inspired Algorithm for Solving Optimization Problems 107
J. C. Felix-Saul, Mario García Valdez, and Juan J. Merelo Guervós

Evaluation and Comparison of Brute-Force Search and Constrained Optimization Algorithms to Solve the N-Queens Problem 121
Alfredo Arteaga, Ulises Orozco-Rosas, Oscar Montiel, and Oscar Castillo

Performance Comparative Between Single and Multi-objective Algorithms for the Capacitated Vehicle Routing Problem 141
David Bolaños-Rojas, Luis Chávez, Jorge A. Soria-Alcaraz, Andrés Espinal, and Marco A. Sotelo-Figueroa

Fuzzy Logic

Optimization of a Fuzzy Classifier for Obtaining the Blood Pressure Levels Using the Ant Lion Optimizer 155
Oscar Carvajal, Patricia Melin, Ivette Miramontes, and German Prado-Arechiga

Optimization of Fuzzy-Control Parameters for Path Tracking of a Mobile Robot Using Distributed Genetic Algorithms 167
Alejandra Mancilla, Oscar Castillo, and Mario Garcia-Valdez

A New Fuzzy Approach to Dynamic Adaptation of the Marine Predator Algorithm Parameters in the Optimization of Fuzzy Controllers for Autonomous Mobile Robots 179
Felizardo Cuevas, Oscar Castillo, and Prometeo Cortes-Antonio

Evaluation of Times and Best Solutions of MFO, LSA and PSO Using Parallel Computing, Fuzzy Logic Systems and Migration Blocks Together to Evaluate Benchmark Functions 205
Yunkio Kawano, Fevrier Valdez, and Oscar Castillo

Fuzzy Dynamic Parameter Adaptation in the Mayfly Algorithm: Preliminary Tests for a Parameter Variation Study 223
Enrique Lizarraga, Fevrier Valdez, Oscar Castillo, and Patricia Melin

Optimization: Theory and Applications

Symmetric-Approximation Energy-Based Estimation of Distribution (SEED) Algorithm for Solving Continuous High-Dimensional Global Optimization Problems 243
Valentín Calzada-Ledesma, Juan de Anda-Suárez, Lucero Ortiz-Aguilar, Luis Fernando Villanueva-Jiménez, and Rosa Trasviña-Osorio

Optimization Models and Methods for Bin Packing Problems: A Case Study on Solving 1D-BPP	265
Jessica González-San Martín, Laura Cruz-Reyes, Bernabé Dorronsoro, Marcela Quiroz-Castellanos, Héctor Fraire, Claudia Gómez-Santillán, and Nelson Rangel-Valdez	
CMA Evolution Strategy Applied to Optimize Chemical Molecular Clusters $MxNz$ ($x + y \leq 5$; $M = N$ or $M \leq N$)	281
J. M. Pérez-Rocha, Andrés Espinal, Erik Díaz-Cervantes, J. A. Soria-Alcaraz, M. A. García-Revilla, and M. A. Sotelo-Figueroa	
Specialized Crossover Operator for the Differential Evolution Algorithm Applied to a Car Sequencing Problem with Constraint Smoothing	291
Javier Manzanares, Elvi Sánchez, Héctor Puga, Manuel Ornelas, and Martin Carpio	
A Brave New Algorithm to Maintain the Exploration/Exploitation Balance	305
Cecilia Merelo, Juan J. Merelo, and Mario García-Valdez	
A New Optimization Method Based on the Lotka-Volterra System Equations	317
Hector Carreon and Fevrier Valdez	
Hybrid Intelligent Systems	
A Comparison of Replacement Operators in Heuristics for CSP Problems	335
Lucero Ortiz-Aguilar, Valentín Calzada-Ledesma, Juan de Anda-Suárez, Rogelio Bautista-Sánchez, and Natanael Zapata-Gonzalez	
Synchronisms Using Reinforcement Learning as an Heuristic	355
Omar Zamarrón, Mauricio A. Sanchez, and Antonio Rodríguez-Díaz	
A Mathematical Deduction of Variational Minimum Distance in Gaussian Space and Its Possible Application to Artificial Intelligence	381
Juan de Anda-Suárez, Lucero Ortiz-Aguilar, Valentín Calzada-Ledesma, Luis Fernando Villanueva-Jiménez, Rosa Trasviña-Osorio, and Germán Pérez-Zúñiga	
A Model for Learning Cause-Effect Relationships in Bayesian Networks	395
Jenny Betsabé Vázquez-Aguirre, Nicandro Cruz-Ramírez, and Marcela Quiroz-Castellanos	

Eureka-Universe: A Business Analytics and Business Intelligence System	419
José Fernando Padrón-Tristán, Laura Cruz-Reyes, Rafael Alejandro Espín-Andrade, Carlos Eric Llorente-Peralta, Claudia Guadalupe Gomez-Santillan, Alejandro Castellanos-Alvarez, and Jordan Michelt Aran-Pérez	
Neural Networks and Learning	
Extension of Windowing as a Learning Technique in Artificial Noisy Domains	443
David Martínez-Galicia, Alejandro Guerra-Hernández, Xavier Limón, Nicandro Cruz-Ramírez, and Francisco Grimaldo	
Why Rectified Linear Activation Functions? Why Max-Pooling? A Possible Explanation	459
Julio C. Urenda and Vladik Kreinovich	
Localized Learning: A Possible Alternative to Current Deep Learning Techniques	465
Javier Viaña, Kelly Cohen, Anca Ralescu, Stephan Ralescu, and Vladik Kreinovich	
What is a Reasonable Way to Make Predictions?	475
Leonardo Orea Amador and Vladik Kreinovich	

Neural Networks

Automated Medical Diagnosis and Classification of Skin Diseases Using EfficientNet-B0 Convolutional Neural Network



Harsh Gunwant, Abhisht Joshi, Moolchand Sharma, and Deepak Gupta

Abstract The skin is the biggest and most vulnerable organ in the body, so skin illnesses are common nowadays. However, the significance of skin health is sometimes underestimated. According to one survey, around 1.79% of the world population suffers from skin-related disorders. These disorders can be fatal when they are not given treatment in their early stages. As a result, skin illnesses must be recognized and diagnosed early to avert severe dangers to one's life. However, the patient may be impacted, and they are frequently subjected to extensive examinations to determine the severity of their skin issue. As a result, we must create an expert system capable of detecting illnesses at an early stage. At the moment, just a few computerized methods are available for detecting skin illnesses, but this is an era in which substantial research is being conducted and may be further expanded. In this paper, an expert system was created using the EfficientNet B-0 model, and the model can also be used to assist specialists in more effectively and efficiently identifying and diagnosing various significant skin diseases such as (Eczema, Psoriasis & Lichen Planus, Benign Tumours, Fungal Infections, and Viral Infections). In addition, we conducted a comparison of sequential Convolutional Neural Networks (CNNs), EfficientNet B0, and ResNet50. Through these models, the reasons for recognized skin illness may be defined, and therapy may be offered. The Python programming language was utilized to implement the models. The dataset was obtained via DERMNET. Using EfficientNet-B0 to train the model and predict outcomes at an epoch value of 10, we attained an accuracy of 91.36%.

Keywords Expert system · CNNs · EfficientNet B0 · ResNet50 · Skin diseases · Dermnet

H. Gunwant · M. Sharma (✉) · D. Gupta
Department of Computer Science, MAIT, GGSIPU, New Delhi, India
e-mail: moolchand@mait.ac.in

D. Gupta
e-mail: deepakgupta@mait.ac.in

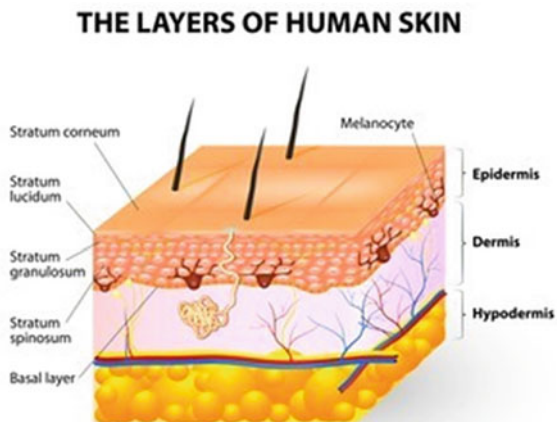
A. Joshi
Department of Information Technology, MAIT, GGSIPU, New Delhi, India

1 Introduction

One of the most important organs in the human body is the skin, and most exposed in the environment. It covers around 1.5–2.5 msq and has roughly 4–5 kg body mass. The sense organ and the most critical body part is the skin. It serves several purposes, including protection, sensation, and regulation. The skin has three layers (as illustrated in Fig. 1): Dermis is thicker than Epidermis and contains all oil and sweat glands, whereas Subcutaneous is beneath the Dermis and offers insulation to our bodies well as acting as a cushion for critical organs. Below is a quick summary of several skin conditions:

- (i) Rash: Any alteration in the look of the skin is a rash. The variations in rashes are from skin irritation and superficial pores, and some may also be the result of a medical condition.
- (ii) Eczema: It is a skin inflammation that results in an itchy rash. An overactive immune system primarily causes it.
- (iii) Dandruff: Psoriasis, eczema, or seborrheic dermatitis causes the scaling of the scalp this is known as dandruff.
- (iv) Dermatitis: It is an umbrella term for skin inflammation. Atopic dermatitis is the most common type.
- (v) Acne: This may be the most frequently occurring skin condition, that usually affects over 80% of the population at some point in their lives.
- (vi) Melanoma: It is the worst kind of skin cancer caused by UV radiation from the sun and other factors. It can be identified with a skin biopsy.
- (vii) Psoriasis: It is an autoimmune disorder that manifests as a variety of different skin rashes. The most common symptom is the appearance of silvery and scaly plaques on the skin.

Fig. 1 Skin anatomy



- (viii) Rosacea: The skin disorder is something that appears on the face as a red rash. Acne-like lesions may be visible on the skin, but their characteristics are not well recognized. Additionally, they should be treated as soon as possible.
- (ix) Basal cell carcinoma: The most prevalent kind of skin cancer is called basal cell carcinoma. Because it develops more slowly than melanoma, it is less hazardous.
- (x) Warts: Virus attacks the skin, causing the body to produce extra growth., resulting in the formation of a wart. Warts can be removed by a physician or they can also be treated at home.
- (xi) Skin abscess (boil or furuncle): local skin infection results in the accumulation of pus underneath the skin. To be treated, some of it must be opened and drained by a physician.
- (xii) Cellulitis: Swelling of the dermis as well as hypodermis tissues. and subcutaneous tissues, most often induced by infection. It manifests as a red, warm, and painful rash on the skin.

Skin illnesses are nowadays more frequent, and one of the most frequent disorders people of all ages suffer. Skin disorders can become harmful and can unless early diagnosed and handled, lead to severe health and financial problems. On the other hand, early detection and diagnosis may prevent a deadly disease. However, owing to the many symptoms, To study pore and skin illness, you need to know a lot about pores and skin. A computerized diagnostic system should thus be seen, as it takes time to diagnose skin illness correctly. In addition, each patient is subject to very costly monitoring costs with a dermatologist [1, 2]. So, we will describe several techniques that can identify skin illnesses through this article, to improve diagnosis effectiveness to firmly established diagnosis methods that will help produce a more trustworthy, more time-efficient, and less costly diagnosis.

We have done a comparative analysis between the different models to better understand and use these models according to the requirements. The main goal of our project is to study the ability of various pre-trained networks to extract features. Although many pre-trained models have emerged as “the best” for knowledge transfer, no consensus has been reached on which model is superior. ResNet50 has been demonstrated to be the best in various studies. EfficientNet has been found to outperform other pre-trained models in various tests. However, in our work, we employ PCA to demonstrate that the EfficientNet pre-trained model has superior feature extraction performance.

In 2019, Tan and Le presented a new model called EfficientNet, which requires the fewest FLOPS. As well as handling routine picture categorization transfer learning tasks, it can reach State-of-the-Art accuracy in the ImageNe competition [3]. The Residual Network is a well-known neural network utilized to serve a wide variety of computer vision tasks [4, 5]. The ResNet enabled us to effectively train massive and dense neural networks with 140+ layers, a significant advance. The critical aspects of the technique proposed are as follows:

- (i) In photographs of different types of body regions of skins, the technology may identify skin problems. It also details the reasons and processes of the skin problem detected, along with a summary of how it is treated.
- (ii) Using an EfficientNet B0, the data set is trained, and results for new images are generated.
- (iii) Five main skin conditions are detected in Eczema, Psoriasis & Litch Planus, Benign Tumour, Viral Infections, and Fungal Infections.
- (iv) Previous methods could not get an accuracy of 91.36%. The approach we've taken has neutralized these difficulties and is more precise than the approaches developed in the past.

A systems expert system contains an inference engine and a knowledge base. Systems that give advice or choices are referred to as expert systems. Python was used to build the expert system. As is the case with most expert systems, a base, an interference engine, and the UI comprises typical components. The paper is aligned parallel to the edge of the table. Various past system implementations and approaches for picture categorization are covered in Sect. 2. Section 3 describes the suggested approach and stages in the system's development. Section 4 details system execution and the outcomes generated. Moreover, last, the final remarks paper is part of the final component.

2 Related Work

Skin-related diseases are ranked fourth as the most common cause of human illness. Skin-related diseases have been growing to a great extent. There is an urgent need to detect skin-related diseases early as they can also become life-threatening. Skin image identification and classification has become a significant research area in the past few years. Muhammad Zubair Asghar presented an online expert system based on Java technology in his article. The expert rules in a tree graph were submitted and inferred using the search-first-progress method. However, advancements in technology have enabled us to design more accurate ways that apply a system based on predefined rules and indicators [1].

M. Shamsul's study proposes the use of feedforward backpropagation artificial neural networks. The system is driven by visual input, namely high-resolution color photographs that have been pre-processed and patient history [2]. Damilola uses a prototyping technique for his solution. The purpose of this system is to merge PSL (Pigmented skin lesion) image data, analysis, matching observations, and conclusions reached by medical specialists into one image [6]. Munirah M. proposes an online system that uses an inference engine that is rule-based and uses forward chaining to search for symptoms associated with a particular skin illness through the use of an online questionnaire [7]. Finally, Ranjan Parekh suggested an automated technique for identifying illness conditions utilizing skin texture photographs and Grey Level Co-occurrence Matrices in his study (GLCM)) [8].

AALC Amarathunga, in his paper, proposed another Expert online system in which one can upload an image, and it can answer some of the questions that will be according to their skin disease condition [9]. This can increase the accuracy of detecting skin disease as it uses both image processing and data mining techniques to evaluate the disease further. Pravin S. Ambad, in his research paper, proposed an imaging system in which the user can upload the image of the skin rash or mole and get real-time updates on whether you should contact the doctor or not [10]. Skin disease is a type of infection or bacterial infection that affects the skin. A new method, described in Nisha Yadav's work, incorporates filtering, segmentation, feature extraction, picture pre-processing, and edge detection to create a new image processing method [11]. Dermatological conditions are one of the most prevalent illnesses on the planet. Sourav Kumar Patnaik used three image recognition architectures in developing a maximum voting system: Inception V3, Inception Resnet V2, and Mobile Net [12].

Anurag Kumar Verma proposed a method that utilizes six machine learning techniques, which incorporates three ensemble approaches to identify unique classes of skin condition utilizing three of these machine learning techniques. Furthermore, in his article, he evaluated the outcomes from several machine learning algorithms using a feature selection step [13]. The Ketut Gede Darma Putra examination showed a skin ailment identification image. Through machine learning and image processing, he was able to obtain this result. K-Means Clustering was used for segmentation, while DWT and Color moments feature extraction was employed for feature extraction [14]. We need to spread awareness about skin disease in rural areas where access to dermatologists is limited. G. RAJASEKARAN, in his research paper, proposed a method that uses image pre-processing and feature extraction in the first step and then machine learning techniques as the next step [15].

They created a scaling model for convolutional neural networks, proposing Mingxing Tan and Quoc V. Le as authors. and the resultant approach was named Efficient Net. Comprehensive research of model scaling and identification found that properly balancing network depth, breadth, and resolution may improve performance. Efficient Net produced superior outcomes [3]. Deevyankar Agarwal, in his paper, proposed an automated diagnosis of covid-19 patients using Efficient Net. A CNN was developed using EfficientNet architecture [16]. Transfer learning helped Jiangchuan Liu to fine-tune the model parameters. A small sample of maize disease datasets (by kind) is more accurately and rapidly diagnosed with more efficient network identification technology [17]. Finally, Nils Dessert suggested using an ensemble of deep learning models, including Efficient Nets, ResNet WSL, and SENet, to classify skin lesions [18].

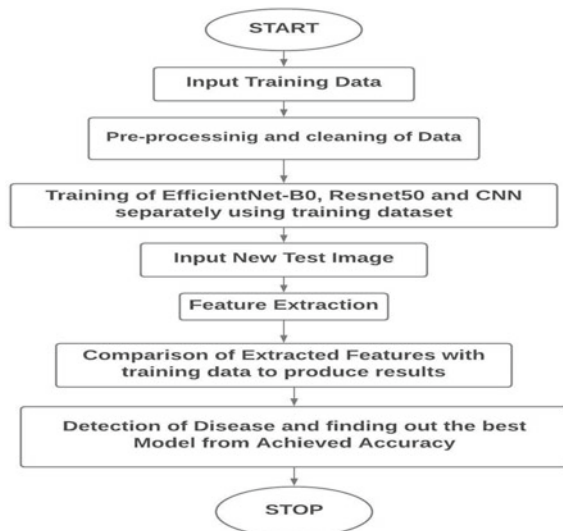
On the DERMNET dataset, we present a system that classifies five distinct types of skin disorders using several pre-processing techniques for image processing and various models such as CNN and its architectures, EfficientNet-B0, and ResNet50. Additionally, a comparison of the three has been conducted.

3 Methodology

This section will explain how the images have been pre-processed into the model for training and the validation to detect the type of skin disease. Also, we will see how the deep learning models are successful for image classification tasks. Since there have been significant advances in Deep Learning architectures called Convolutional Neural Networks, the Deep Learning approaches have shown to be quite successful in picture categorization (CNN). The ImageNet Large Scale Visual Recognition Challenge in 2012 honored AlexNet for the impact of their work in computer vision on the field [19, 20]. The CNN architecture has since then seen continued advancement in the Deep Learning community. CNN designs well-known to the public include VGG, GoogleNet, Resnet, and EfficientNet. While CNN's excel at classifying images according to location invariant characteristics, CNNs can also successfully do image classification tasks.

Furthermore, while training CNN as soon as CNN learns the filters that serve as feature extractors, it automatically optimizes the machine learning models. This filter helps the model learn and discriminate between various types of pictures fed into the model. This study leverages CNN and two of its architectures to classify Skin disease images into five diseases like Benign Tumours, Fungal Infections, Viral Infections, and Eczema, Psoriasis, and Lichen Planus. Below the Fig. 2 shows the flow process used in the paper, and further, we will see the architecture of the different expert systems used for the simulation.

Fig. 2 The flow diagram of the expert system used



3.1 EfficientNet-B0

EfficientNet is a type of ConvNet that is one of the most efficient, effective, and accurate [21]. The study's authors presented a new Convolutional neural network scaling methodology in which to increase accuracy and efficiency, the breadth, resolution, and depth of ConvNets are all scaled equally [3]. Using the new scaling approach, a new set of models termed EfficientNets was built. The eight original EfficientNet models are called EfficientNet-B0 through EfficientNet-B7. Following are scaled variations of the basic model, EfficientNet-B0. The architectural complexity and quantity of model parameters increase as we progress from B0 to B7. Following extensive testing and comparison, it was discovered that EfficientNet-B0 provided the best accuracy and efficiency. Because these models have low FLOPS costs and fewer parameters, EfficientNet was chosen. As a result, they have cutting-edge picture categorization results that can be used on various datasets. Reference [3] quotes the authors as follows: "On ImageNet, our EfficientNetB7 is 84.4% top-1 97.1% Top-5 and $8.4 \times$ smaller and $6.1 \times$ quicker than the best current ConvNet. Our EfficientNets also transfer effectively to CIFAR-100 (91.7%), Flowers (98.8%), and three other transfer learning datasets. The goal was to see if EfficientNets could reproduce its success on the Skin disease dataset, given it had done so well on previous datasets. Furthermore, EfficientNet proved to be an excellent pick. Not only did we outperform Resnet50 and CNN in terms of performance, but our model also contains fewer parameters (5.3 million) than Resnet50 and CNN.

3.1.1 Architecture

A Neural Architecture Search technique was used to create the baseline architecture for EfficientNet-B0, which was optimized for FLOPS and accuracy. The EfficientNet-B0 architecture is shown in Fig. 3.

The first step is the design of the stem network, which is illustrated in Fig. 4.

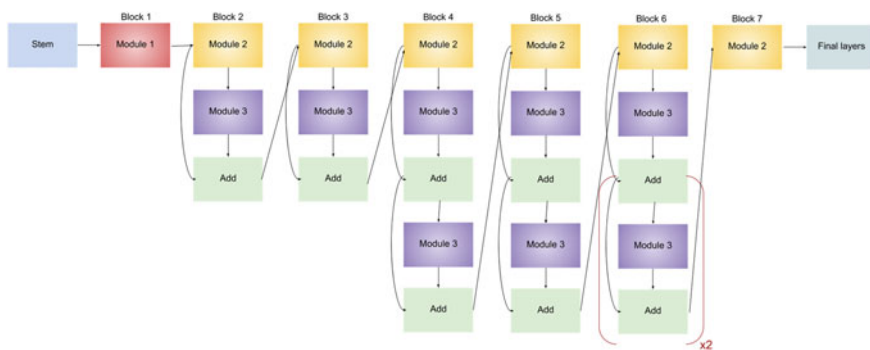


Fig. 3 Architecture of EfficientNet-B0

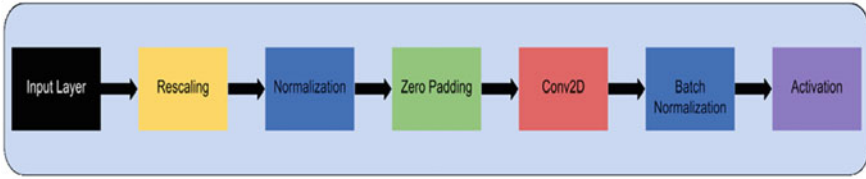


Fig. 4 STEM of each architecture in EfficientNet B0-B7 Model

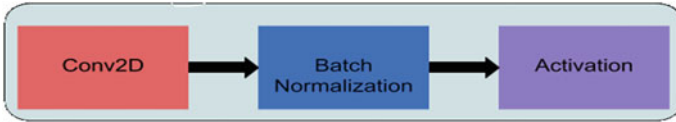


Fig. 5 Final Layer of the architecture in EfficientNet B0 Model

Then, the design of the various architectures begins, which is consistent in all of the EfficientNet models and is represented in Fig. 5.

Next, each of them has seven blocks inside of it. These blocks are further equipped with a variable number of sub-parts when we proceed from EfficientNetB0 to EfficientNetB7, the number of sub-blocks changes EfficientNet-B0 contains 237 layers, whereas EfficientNet-B7 has 813 layers. Do not worry, though. These 5 modules may be used to create all these layers. The sub-blocks are formed by combining more specific modules. This design from Fig. 3 describes the EfficientNet B0 model perfectly.

- (i) **Module 1**—This is the base for the sub-blocks.
- (ii) **Module 2**—This is the first of the seven major blocks and blocks in sub-blocks start here.
- (iii) **Module 3**—This is an interconnected chain of sub-blocks, all of which are connected as a skip to the main block.
- (iv) **Module 4**—This is used for connecting the sub-blocks so that each can have its skip connection.
- (v) **Module 5**—This module connects the two preceding sub-blocks via a skip connection.

The kernel size for convolution operations, as well as the resolution, channels, and layers in EfficientNet-B0, are listed below in Table 1.

3.1.2 Compound Scaling

The breadth, depth, and resolution of ConvNets can be scaled in three dimensions. Earlier attempts to scale ConvNets were solely focused on scaling. One of the three dimensions demonstrates that scaling while balancing the three dimensions of

Table 1 Input parameters for EfficientNet B0 model

Stage i	Operator F_i	Resolution $H_i \times w_i$	No. of Channels c_i	No. of layers L_i
1	Conv3 $\times$ 3	24×224	32	1
2	MBConv1, k3 $\times$ 3	112×112	16	1
3	MBConv6, k3 $\times$ 3	112×112	24	2
4	MBConv6, k5 $\times$ 5	56×56	40	2
5	MBConv6, k3 $\times$ 3	28×28	80	3
6	MBConv6, k5 $\times$ 5	14×14	112	3
7	MBConv6, k5 $\times$ 5	14×14	192	4
8	MBConv6, k3 $\times$ 3	7×7	320	1
9	Conv1 $\times$ 1 & Pooling & FC	7×7	1280	1

breadth, network-depth, and resolution yields more efficient, effective, and accurate models than ever before.

The goal of compound scaling is to: “We should enhance network depth for higher resolution images so that the larger receptive fields can help catch similar features that include more pixels in larger images. If the resolution is increased, we also need to extend the breadth of the network to accommodate larger and more detailed patterns in photographs with a higher resolution. This suggests that we should use many scaling dimensions in tandem, rather than utilizing a single dimension that scales everything uniformly [3].

$$\textbf{depth: } d = \alpha^\phi \quad (1)$$

$$\textbf{width: } W = \beta^\phi \quad (2)$$

$$\textbf{resolution: } r = \gamma^\phi \quad (3)$$

$$\textbf{s.t } \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2, \quad (4)$$

$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$$

In the preceding Eqs. (1), (2), (3), (4), is the compound coefficient, which the user can specify for scale based on the resources available. The constants specify how these resources will be distributed over the ConvNet’s depth, breadth, and resolution. Grid search can also be used to produce, and then a user-specified compound coefficient can be used to scale a baseline model (Fig. 6).

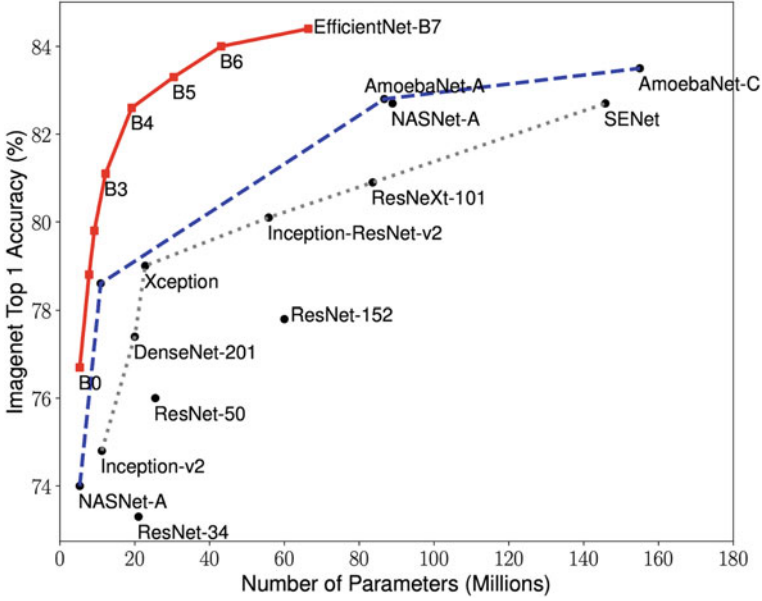
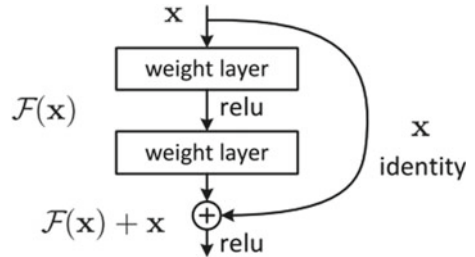


Fig. 6 Model size versus accuracy comparison. EfficientNet-B0

3.2 ResNet50

Kaiming He et al. introduced ResNet. in 2015 [4]. ResNet is a well-known neuronal network used as a support for several tasks in computer vision. ResNet is a residual network. Because of the problem of vanishing gradients, dense neural networks were complicated before ResNet training. ResNets are created to learn residual functions associated with the layer inputs instead of unreferenced functions. Residual nets allow these layers to be utilized as residual mappings, not just as additional ones. Residual blocks are stacked on top of each other to construct a network such as a ResNet-50: a ResNet-50 has fifty layers made up of these residual blocks. A “shortcut connection to identification” can be created by using the techniques described in Fig. 7.

Fig. 7 Residual identity mapping



layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Fig. 8 Architecture of ResNet 50

The ResNet models used previously skipped two layers, but for ResNet50, they skipped three layers, and 1 × 1 convolution layers were added, which can be seen in the ResNet50 Architecture as shown below in Fig. 8.

3.3 CNN

The heart of AlexNet is a convolutional neural network (CNN), an artificial network that mimics significantly the human visual system [19]. Figure 9 demonstrates that Convolutional Neural Network (ConvNet/CNN) constitutes a deep learning method that may use an input picture to distinguish between aspects/objects (learnable weights and biases). Convolutional neural networks are composed of many artificial neuron layers. Artificial neurons are mathematical functions that provide an activation value after adding together several weighted inputs. CNN image classifications capture, evaluate and classify a picture into several groups (E.g., Dog, Cat,

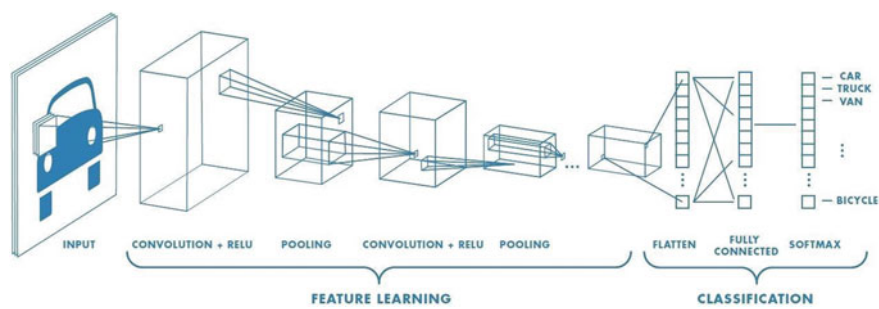


Fig. 9 Numerous convolutional layers in a neural network

Tiger, Lion). Computers see an image input as a pixel array, depending on the picture resolution, with several pixels variable. It will view,

$h \times w \times d$ based on the image resolution.

(w = Width, h = Height, d = Dimension).

Several CNN architectures are used for Image Processing, Feature Extraction, etc. Some of the Architectures are VGG, Inception, MobileNet, ResNet, etc.

3.4 Dataset

The suggested CNN and its architectures make use of a DERMNET dataset. The collection contains skin conditions classified as Eczema, Psoriasis, Lichen Planus, benign tumors, fungal infections, and viral infections. It is pre-classified into five classes. Each of the five categories of skin disorders has several photos ranging in size from 1900 to 2500. These photographs depict skin diseases in various body areas, including the neck, forehead, hands, back, and legs. The acquired dataset is pre-processed using a data augmentation technique that generates multiple copies of each image by rotation, flipping, and resizing operations, increasing the dataset's size and improving training efficiency and effectiveness. In addition, it assists in avoiding underfitting. Figure 10 depicts representative images of skin disease kinds from the dataset, whereas Fig. 11 depicts the dataset's format before pre-processing.

The skin illnesses depicted in the Dataset Sample images (Fig. 10) are as follows:

- (i) Eczema
- (ii) Psoriasis and Lichen Planus
- (iii) Benign Tumor
- (iv) Bacterial Infections
- (v) Fungal Infections

Separating before processing into training and testing datasets allows for more reliability in determining if the model is functioning correctly. The training set consists of 80% of the total images, and the model is being trained using the training set. The model then estimates the optimal weights that best suit the ANN to predict all categories with the least amount of error. Also known as the Validation subset, the remaining 20% of the dataset's images make up the testing dataset. This dataset is employed to guarantee the model's accuracy. Also, the Model is used to predict test images, which are then compared to the desired output to determine the model's efficiency and efficacy.

3.5 Experimental Setup

The system was built on a computing environment employing mobile computers. The system on which our laptop is based on a seventh-generation Intel Core i5 CPU



Fig. 10 Illustrations from several classes

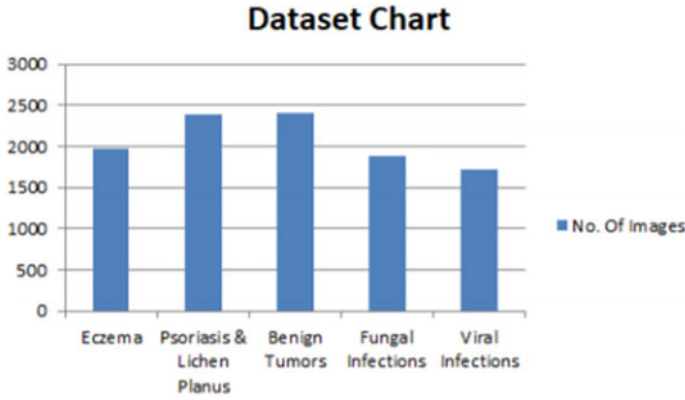


Fig. 11 Pre-processed representation of the dataset

that has a 2.1 GHz clock speed, 3 MB cache memory, and 8 GB of RAM installed on a 1 TB hard drive (ver. 20H2, 64 bit). Python 3.2 with TensorFlow, NumPy, Scikit-learn, and Pandas is available at the Google Collaboratory.

4 Results and Discussion

It is noted that the Convolutional Neural Network, ResNet50, and EfficientNet-B0 models all produce accurate results for all five skin conditions. However, the best results are obtained when one of the CNN designs is used, namely EfficientNet-B0, which has a precision of 91.36%. Therefore, we pre-processed the dataset by conducting data augmentation to enhance its size, trained and tested it using Convolutional Neural Network, ResNet50, and EfficientNet-B0 models. EfficientNet-B0 does classification based on feature extraction. Therefore, no classifier is required. The maximum accuracy is obtained when the batch size is 32 samples, and the epoch is set to 10.

As shown in Fig. 12 and table. Two, it is clear that EfficientNet-B0, with an accuracy of 91.36%, has more accuracy and sensitivity than CNN and ResNet50 when trained on the DERMNET dataset to classify the images into five classes. Sensitivity measures the proportion of people suffering from the disease who got predicted correctly as those suffering from the disease. Sensitivity is also termed Recall. ResNet50 and CNN got an accuracy of 87.49 %and 84.52%, respectively (Table 2).

EfficientNet-B0 had a 224×224 input with ReLu as the activation function, ten epochs, and batch size is kept constant, i.e., 32 also there are 237 layers. The EfficientNet-B0 model is a simple baseline architecture for mobile devices learned on the ImageNet dataset. EfficientNet-B0 has the fewest parameters, around 5.3 million; ResNet50 and CNN have 25.6 million and 60 million, respectively. For all three

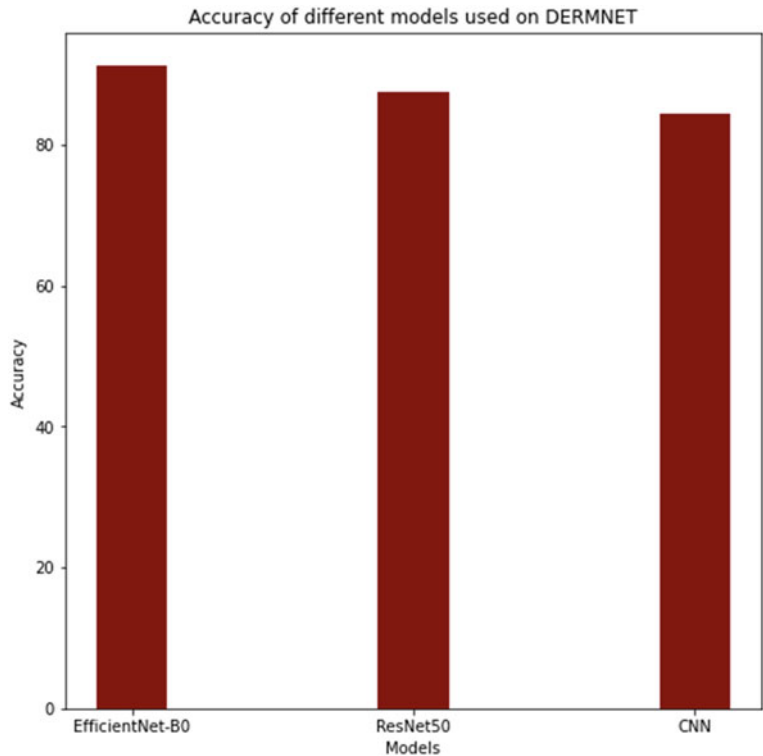


Fig. 12 Accuracy of different models

Table 2 Accuracy and sensitivity for different models

Model	Accuracy (%)	Sensitivity (%)
CNN	84.52	81.75
ResNet50	87.49	85.34
EfficientNet-B0	91.36	92.68

models, the learning rate is set at 0.0001. EfficientNet-B0 outperforms ResNet50 and CNN on this dataset in terms of performance and accuracy. The parameters used in all three models are listed in Table 3 below:

5 Limitation of the Study

The limitation of our study is only about the models that were used. The EfficientNet Model starts from B0 to B7. However, we have used the B0 model in our study due to the machine and space limitation as EfficientNets’ input size grows from 224*224

Table 3 Hyperparameter for the three models

Hyperparameters	Values for ResNet50	Values for EfficientNet-B0	Values for CNN
Batch Size	32	32	32
Epochs	10	10	20
Activation Function	ReLu	ReLu	ReLu
Dropout	0.5	0.5	0.5
No. of layers	50	237	6
Input shape	(150,150)	(224,224)	(64,64)

in B0 to 600*600 in B7. As a result, the complexity of space and time increases. EfficientNetB7 will take a long time to train on a large dataset. Going from 5.3 million in B0 to 66 million in B7, there is an increase in the parameters. So, for using a model above B0, we need a high-performance machine.

6 Conclusion and Future Scope

This paper conducts a comprehensive examination and comparison of three models, ResNet50, EfficientNetB0, and CNN, taking into account the many factors used to forecast skin disease types. Thus, based on the application need, the research assists in determining which model to select solely based on the advantages and disadvantages of each design. Additionally, a comparison of the three models above was conducted to detect and identify the critical five skin illnesses, namely Eczema, Psoriasis and Lichen Planus, Benign Tumors, Fungal Infections, and Viral Infections.

Additionally, it has been observed that using EfficientNet-B0 results in a significant boost in illness detection accuracy (about 4%) when compared to the other two models utilized. In the future, EfficientNet-B0 may be regarded as a safe bet for achieving superior outcomes in image prediction and classification—accuracy attained at a significantly reduced computational cost in these models. Additionally, we may expand the dataset for skin disease categorization. These algorithms/models may be used in a variety of applications, including biomedical and image processing.

References

1. Asghar, M., Asghar, J., Saqib, S., Ahmad, B., Ahmad, S., Ahmad, H.: Diagnosis of skin diseases using online expert system. *Int. J. Comput. Sci. Inf. Secur. (IJCSIS)* **9**(6), 323–325 (2011)
2. Shamsul Arifin, M., Golam Kibria, M., Firoze, A., Ashraful Amini, M., Yan, H.: Dermatological disease diagnosis using color-skin images. In: 2012 International Conference on Machine Learning and Cybernetics, pp. 1675–1680 (2012). <https://doi.org/10.1109/ICMLC.2012.6359626>

3. Tan, M., Le, Q.V.: EfficientNet: rethinking model scaling for convolutional neural networks. International Conference on Machine Learning (2019). [arXiv:1905.11946](https://arxiv.org/abs/1905.11946)
4. Albawi, S., Mohammed, T.A., Al-Zawi, S.: Understanding of a convolutional neural network. 2017 International Conference on Engineering and Technology (ICET), pp. 1–6 (2017). <https://doi.org/10.1109/ICEngTechnol.2017.8308186>
5. Sharma, M., Jain, B., Kargeti, C., Gupta, V., Gupta, D.: Detection and diagnosis of skin diseases using residual neural networks (Resnet). Int. J. Image Graph. (2020). <https://doi.org/10.1142/s0219467821400027>
6. Okuboyejo, D., Olugbara, O., Odunaike, S.: Automating skin disease diagnosis using image classification. In: Lecture Notes in Engineering and Computer Science, Proceedings of the World Congress on Engineering and Computer Science 2013, Vol II WCECS, 23–25 October 2013, San Francisco, USA
7. Mohd Yusof, M., Ab Aziz, R., Fei, C.: The development of online children skin diseases diagnosis system. Int. J. Inf. Educ. Technol. **3**(2), 231–234 (2013). <https://doi.org/10.7763/IJJET.2013.V3.270>
8. Parekh, R., Mittra, A.K.: 2011: Automated detection of skin diseases using texture features. Int. J. Eng. Sci. Technol. **3**(6), 4801–4808 (2011)
9. Amarathunga, A.A.L.C., Ellawala, E.P.W.C., Amalraj, C.R.J., Abeysekara, G.N.: Expert system for diagnosis of skin diseases. Int. J. Scientific Technol. Res. **4**(1), 174–178 (2015). ISSN 2277-8616
10. Ambad, P., Shirsat, A.: An image analysis system to detect skin diseases. IOSR J. VLSI Signal Process. **6**, 17–25 (2016). <https://doi.org/10.9790/4200-0605011725>
11. Yadav, N., Narang, V.K., Shrivastava, U.: Skin diseases detection models using image processing: a survey. Int. J. Comput. Appl. **137**(12), 0975–8887 (2016)
12. Patnaik, S.K., Sidhu, M.S., Gehlot, Y., Sharma, B., Muthu, P.: Automated skin disease identification using deep learning algorithm. Biomei. Pharmacol. J. **11**(3) (2018). <https://doi.org/10.13005/bpj/1507>
13. Verma, A.K., Pal, S., Kumar, S.: Prediction of skin disease using ensemble data mining techniques and feature selection method—a comparative study. Appl. Biochem. Biotechnol. **190**, 341–359 (2020). <https://doi.org/10.1007/s12010-019-03093-z>
14. Putra, K.G.D., Wiastini, N.P.A.O., Wibawa, K.S., Putra, I.M.S.: Identification of skin disease using K-means clustering, discrete wavelet transform, color moments and support vector machine. Int. J. Mach. Learn. Comput. **10**(4), 542–548 (2020). <https://doi.org/10.18178/ijmlc.2020.10.4.970>
15. Rajasekaran, G., Aiswarya, N., Keerthana, R.: Skin disease identification using image processing and machine learning techniques. Int. Res. J. Eng. Technol. (IRJET) **7**(3) (2020) p-ISSN: 2395–0072
16. Marques, G., Agarwal, D., de la Torre Díez, I.: An automated medical diagnosis of COVID-19 through EfficientNet convolutional neural network. Appl. Soft Comput., 106691 (2020). <https://doi.org/10.1016/j.asoc.2020.106691>
17. Liu, J., Wang, M., Bao, L., Li, X.: EfficientNet based recognition of maize diseases by leaf image classification. J. Phys. Conf. Series **1693**, The 2020 3rd International Conference on Computer Information Science and Artificial Intelligence (CISAI) 2020 25–27 September 2020, Inner Mongolia, China (2019)
18. Gessert, N., Nielsen, M., Shaikh, M., Werner, R., Schlaefel, A.: Skin lesion classification using ensembles of multi-resolution efficientnets with metadata. MethodsX, 100864 (2020). <https://doi.org/10.1016/j.mex.2020.100864>
19. Krizhevsky, A., Sutskever, I., Hinton, G.E.: ImageNet classification with deep convolutional neural networks. Commun. ACM **60**(6), 84–90 (2017). <https://doi.org/10.1145/3065386>
20. Russakovsky, O., Deng, J., Su, H., et al.: ImageNet large scale visual recognition challenge. Int. J. Comput. Vis. **115**, 211–252 (2015). <https://doi.org/10.1007/s11263-015-0816-y>
21. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778 (2016). <https://doi.org/10.1109/CVPR.2016.90>

Modular Approach for Neural Networks in Medical Image Classification with Enhanced Fuzzy Integration



Sergio Varela-Santos and Patricia Melin

Abstract In recent years digital image analysis has become a subject of interest for computational scientists partially because of the data-driven society we have today which makes it possible for large datasets of information like images to be widely spread and accessible to more users than ever. This is also because of the resurgence of popularity for neural networks with the deep architectures like Convolutional Neural Networks (CNN) specialized on this type of data. Several of these large datasets are dedicated to medical image diagnosis by classifying medical images like X-Rays or CT scans. CNN's offer a robust learning system that processes and reduces the information contained in an image dataset, other approaches make use of predefined feature extractors like image texture features and use them as inputs for classifiers to learn and categorize, both paradigms have been used with success in image-related problems. In the presented work the contribution is a robust modular approach for combining these two paradigms of image analysis into one cooperative system that is unified by a fuzzy system designed for an output integration resulting in one image classifier that using both perspectives give back one unified response. Experiments were performed on benchmark image datasets and medical image focused datasets for diagnosing Pneumonia, COVID-19 and Tuberculosis with favorable results, giving a more comprehensive image analysis and an alternative to single model architectures that are more commonly used.

Keywords Texture features · Deep learning · Image classification · Local binary patterns · X-ray · Neural networks

S. Varela-Santos · P. Melin (✉)
Tijuana Institute of Technology, Tijuana, BC, Mexico
e-mail: pmelin@tectijuana.mx

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2022
O. Castillo and P. Melin (eds.), *New Perspectives on Hybrid Intelligent System Design based on Fuzzy Logic, Neural Networks and Metaheuristics*, Studies in Computational Intelligence 1050, https://doi.org/10.1007/978-3-031-08266-5_2

1 Introduction

Computer Aided Diagnosis systems have supported physicians in analyzing several different types of diseases in patients over the years [1]. These systems are produced from the research advancements in diagnostic techniques created by the combined effort of medical and computer scientists.

CAD systems are commonly supported by the usage of inference and classification computer algorithms which perform automatic diagnostics of medical data, in the form of medical images [2, 3], electrical signals obtained from the patient's body [4], readings of medical devices [5], and many other forms of data extraction regarding an individual's health [6].

Regarding these supporting classification algorithms, hybrid systems like neuro-fuzzy models [7–9] have become more important in recent years, because with the creation of larger image datasets for testing medical diagnostic models, the models itself may need to include different inference mechanisms not found in a single model, but possible with the combination of models from different areas of that work different in nature but provide an accurate response to the medical image diagnostic problem (image classification problem). In this work, we present an approach to combine classification results from specialized neural network models by using a fuzzy response integrator (Type-1 Fuzzy System) for n -class image classification problems.

1.1 Medical Images

Nowadays, accessibility to digital medical image databases has increased, since the late 90's changed their workflow to a modern operation using digital medical images by implementing the usage of Picture Archiving and Communication Systems (PACS) [10], by making this transition and by the surge of interest in digital image analysis created by the new advancements in modern classification models like deep learning, Hospitals and Research Centers have started to collaborate in order to release Large Medical Image Datasets for scientific usage [11].

In the field of medical image diagnostics radiology is one of the most important areas in which advances in computer-aided decision support have been generated. This is thanks to the now widely available image datasets for many "Chest X-Ray" detectable diseases [12, 13].

Medical Images are commonly defined by the method how images are produced. For ultrasound, these are generated by using sound waves on the body and are able to obtain images of a patient's anatomy, in the case of radiation-based like X-rays, are generated by machines that shot a single x-ray source unto the tissue generating two-dimensional images. Computed tomography (CT) scanners generate higher resolution images with the usage of multiple sources of x-rays [14].

1.2 *Medical Image Classification Using Neural Networks*

Because of these factors the development of new and better solutions using diagnostic technology have been created to help the medical sector. The better a group of data represents a problem, the better the model generated by that group of data would be able to solve real-world problems. Newer models like convolutional neural networks (CNN) require large amounts of images in order to perform an accurate classification and solution to the diagnostic problem [15].

In order to perform a diagnostic assessment on medical images, we need to first pre-label a group of data like images, into pre-defined classes that represent the different states of the problem, the diagnostic problem can be considered a “classification problem” because it involves giving a prediction or result based on the input data and grouping that prediction to one of the pre-defined states of the problem. For example, in a medical problem the patient either will have signs that indicate a verdict of “having a disease”, “not having a disease” and “having more than one disease” each of these verdicts can be grouped in a class, one class commonly is the “healthy patient”, “patient diagnosed with disease 1”, “patient diagnosed with disease 2” and so forth.

In this regard, Neural Networks are a commonly used classification model because it has produced great results in several binary and multiclass problems over the years [16, 17], many of which have created feasible medical diagnosis support systems [18–20]. Particularly in the area of medical image classification Neural Networks work in tandem with computer vision algorithms to detect the most important features of the image dataset and use that information to classify with high accuracy [21].

In Chauhan et al. [22] a DenseNet model was trained for classification using transfer learning on images obtained from Covid-19 patients, the images itself are Chest X-ray images divided into 2 classes, the Covid class and a healthy-normal one. The results seem promising with a 98.45% accuracy for the normal class and a 98.32% for the Covid class.

In Varshmi et al. [23] a Deep Convolutional Networks was used as a feature extraction algorithm, extracting the best features via the convolution operation and instead of using a classification layer like in the common model of CNN’s the authors used another well-known classifier in the Support Vector Machines Classifier (SVM), the experiments were done on a Chest X-Ray image dataset for the diagnosis of Pneumonia. The images were resized to the 224 by 224, standard input size of DenseNet models, the final results were evaluated using AUC metric which reached a 0.8002 in the best result.

In Verma et al. [24] other model focused on the Pneumonia disease is created, this time using convolutional neural networks and data augmentation in order to make up for the lack of images, the augmentation is done by rotating the images and resizing them and using them with the normal images in order to extract meaningful features regardless the angle of the pictures, retaining data invariant characteristics.

1.3 Fuzzy Logic Combined with Modular Neural Networks

Fuzzy Logic was developed by the mathematician Zadeh [25] as an extension to set theory in order to consider the numbers in between 0–1. This principle applied to a classification algorithm considers a prediction as a combination of memberships of more than one class, which in itself may provide a better solution in some cases.

The application of fuzzy logic in hybrid models with other machine learning techniques like neural networks, and in particular modular or ensemble-based architectures have been used since the last decade [26], and still is an important design when we need an extra accuracy on the model's results. When working with ensemble or modular models, the problem is divided into sections, most of which are treated separately, performing an analysis based on each section's capabilities.

Commonly the models selected have some sort of relationship to the problem to solve or at least provide a partial solution to the problem, so when we combine different models that have already provided a solution to a classification problem, we are combining good classifiers into one that provides many solutions to the problem, one for each model, so what would be the correct way to select the classification criteria based on many solutions?, this would be done using an integration mechanism [27], many algorithms have been proposed for this solution, from simple average and weighted average of the solutions [28], the highest of the solutions, and by performing a tie-breaking or combination of the outputs [29]. Another way of combining them is by the usage of a Fuzzy logic-based system, like a type-1 Mamdani fuzzy system. These systems work with a similar input–output system than the one used in neural networks but base its function on rules guided in fuzzy logic. Fuzzy Logic represents an important alternative because aside from the outputs of well-known classifiers, it takes consideration of uncertainty into the problem, which is inherent in medical problems and in the way, humans think and see the world.

Classification models that include fuzzy logic have been able to provide good solutions into real-world problems. In Ontiveros-Robles et al. [30] the authors present a study using type-2 fuzzy classifiers for medical diagnosis problems, performing classification on datasets for diverse medical problems like breast cancer (Breast Cancer Wisconsin, Breast Cancer Coimbra), liver patient identification (Indians Liver Dataset), diabetes patient identification (Pima Indians Diabetes Dataset), symptoms of heart diseases (SPECT Heart), primarily using intelligent models based on type-2 fuzzy logic and supervised learning with a focus on modelling the data uncertainty of each medical dataset.

In Mousavi et al. [31] a robust hybrid intelligent system is proposed, which is based on the combination of the Harmony Search (HS) algorithm and two type-1 fuzzy systems that perform the classification, the idea is to analyze medical datasets and by optimizing a fitness function obtain fuzzy-rules using a modified version of the HS algorithm, the fuzzy rules are personalized for each dataset and then used to perform an accurate classification solution.

In Melin et al. [32] another hybrid model for classification is proposed, this time for the risk diagnosis in developing a heart disease like high blood pressure patients,

the hybrid model was comprised of modular neural networks for pattern recognition and fuzzy logic classifiers, the risk assessment was done by using multiple attributes like age, risk factors and records of ambulatory blood pressure monitoring. The fuzzy logic systems are able to give a classification in the form of a risk diagnosis based on the input of risk factors and the output obtained from the modular neural network trained on the records of blood pressure, thus creating a support system capable of helping medical professionals in detecting heart diagnosis diseases.

This work is organized as follows: in Sect. 2 the proposed modular model is presented and explained, in Sect. 3 the methodology used for experimentation, the datasets used and the detailed parameters used in each of the proposed model's modules is explained, in Sect. 4 the results and comparisons are presented and in Sect. 5 conclusions and future work on the subject are described.

2 Proposed Method

The advancement in computer vision techniques and the usage of newly adopted neural network models has been the main catalyst in the now mainstream usage of deep neural models, and convolutional neural networks that are great on image classification tasks. Commonly before this adoption, the way to solve image-related problems in computational intelligence was to first analyze the images for distinctive features that were present only on images from a specific class, this was known to be the representative features of x-class, so in order to perform classification such features have to be emphasized for them to be used for classification for example lines, edges, objects. Another way of obtaining useful information on class features is to perform feature extraction, this is to use specially designed formulas and algorithms that perform calculations based on information found in an image sample, this can be the quantization of pixel distributions, the extraction of texture-related information and also color distribution information in the cases of color images. With the creation of Convolutional models these steps have been grouped together into a single closed model capable of extracting features from the source image dataset and perform classification.

Our hybrid model seeks to present a simple yet powerful way of grouping together both paradigms of image-related classification analysis into a double agent model that combines the first paradigm of using a feature extraction algorithm and a neural network classifier separately, and the second one that uses a convolutional neural network that performs both functions in a single closed model. The combination of outputs from both paradigms is done by using a Mamdani type-1 fuzzy system in the form of an integrator, that uses the inputs of both neural networks and gives an aggregated final response for the classification.

Experiments were done using 4 databases, and statistical testing is done by comparing the best result from one of the two neural networks and the final aggregated response that takes into consideration the other method as well. In Fig. 1 we can observe the proposed model architecture.

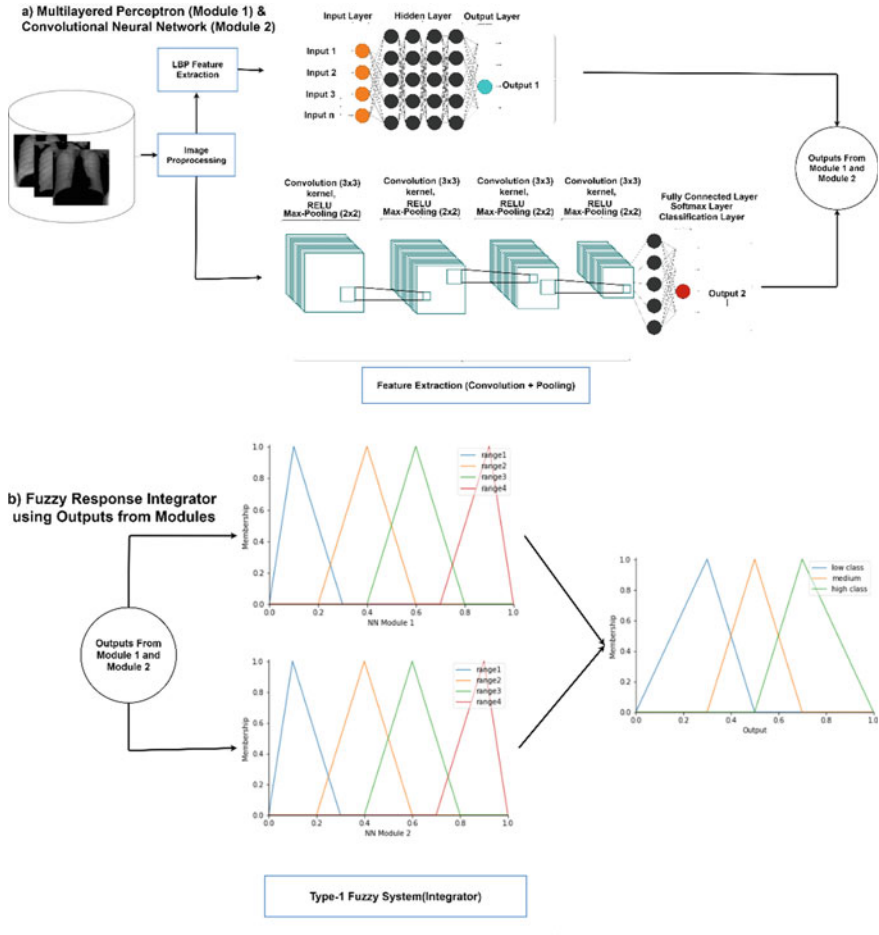


Fig. 1 Proposed method for classification using **a** Multilayered Perceptron and Convolutional Network, **b** Fuzzy Response Integrator that combines the outputs into one integrated classification prediction

3 Methodology

The experimentation was done using medical image datasets for image classification problems, we used as our first dataset, the Montgomery-Shenzen Tuberculosis image dataset [33], a binary class image (400 by 400 pixels per image) dataset for the diagnosis of pneumonia. A Covid-19 binary dataset divided in Covid and nonCovid classes, images are a combination of x-ray and ct scans, only the x-ray part was used for this study (500 by 500 pixels per image). The third dataset was a 3-class Covid dataset created to differentiate Covid, pneumonia and a healthy x-ray this was created using the second dataset and the dataset of pneumonia from Zhang Lab (images were

Table 1 Parameters for module 1 (Multilayered Perceptron)

Network parameters	
Inputs	59
Hidden layers	4
Neurons	59 input layer/30,30,30, 30 hidden layers/1 output layer
Outputs	1
Training algorithm	Scaled conjugate gradient
Performance evaluation measure	Cross entropy

standardized to 500 by 500 pixels per image) [34]. Finally, the fourth and final dataset is the 4-class (covid-19, viral pneumonia, lung opacities and healthy) released by a team of researchers of the university of Qatar and Bangladesh (300 by 300 pixels per image) [35]. In all experiments done a data distribution of train-testing was divided by using 70% of available images for training and the remaining 30% for testing.

The classifiers used for experimentation are designated as “Module 1” and “Module 2”, Module 1 is a Multilayered Perceptron also called in some contexts as fully connected neural network, it uses a learning algorithm to learn patterns from samples and uses a loss or error function that gets minimized as the training advances and the results predicted by the model are closer to those expected. The patterns used for the first module are texture features [36] extracted from the images of the experimental datasets, these features calculate the spatial-distributions of gray level pixels from grayscale images like those found in medical datasets. Module 2 is a standard Convolutional Neural Network that uses 4 convolutional layers to reduce the image used as input into features that are emphasize over the pooling layers and finally classified using the classification layer. Tables 1 and 2 contain more specific data on the parameters of both models.

Table 2 Parameters of for module 2 (Convolutional Neural Network)

Network parameters	
Inputs	Size of image ($400 \times 400/500 \times 500$)
Hidden layers	4 Convolutional (8,16,32,64 kernel size) and 1 fully connected
Outputs	1 classification layer
Learning rate	–
Minibatch size	64
Training algorithm	ADAM optimizer
Performance evaluation measure	Cross entropy

The model itself works like a modular neural network that uses the same input either divided into modules or separated into an independent analysis and combined at the end by an integrator.

For this model, the usage of fuzzy logic in the integrator contributes by modelling uncertainty of the problem and taking it into consideration in the final prediction. The problem’s uncertainty can come from several factors, not knowing for certain the most optimal classifiers for the problem, the noise generated by the medical image-capture device, and several others commonly found when approximating a function instead of having a specific solution. This in combination with the trained neural networks focused on the problem with each different perspectives performs a better classification overall.

The fuzzy integrator contains 2 inputs, one for each of the modules existing in the modular model, input 1 corresponds to the classification value given by multilayered perceptron, and input 2 to the classification given by the convolutional neural network, depending of the classes to the problem we assign one membership function for each class, called “range”, “range 1” for class 1, “range 2” for class 2 of the problem until class n . Only one output of the model is given at the end, and is considered the final prediction for a single image sample. The outputs are “Low Class”, “Medium Class” and “High Class”, if the output of the system is a low number [0–0.5] it would be mostly classified in the “Low Class” or “Medium Class” if it’s a medium number [0.2–0.7], it would fall into the “medium class”, and if it’s a high number [0.5–1] it would fall into the “High Class” membership function, after the fuzzy classification is done, the final crisp number obtained after the defuzzification is then compared with the actual label of the data in order to test the accuracy of the prediction (Table 3).

In Fig. 2 and Table 4 we can observe the membership functions and the fuzzy rules used for the system. The following model was used for Datasets 1 & 2 which are binary class datasets.

The same principle used in the first two experiments is used for the next experiment but adding a third membership function per input, in order to accommodate the third class that the problem has. In Fig. 3 and Table 4 we present the fuzzy integrator and fuzzy inference rules used for the 3-class dataset.

Finally, following the same idea for the last experiment a fourth membership function was added in order to make it compatible for the 4-class scenario. In Fig. 4 and Table 5. we can observe the specifics of the model. For all cases the defuzzification

Table 3 Fuzzy rule set for 2-class problem (Dataset 1&2)

Fuzzy rules	Input 1 MLP-LBP prediction	Input 2 CNN prediction	Output
1	Class 1	Class 1	Low class
2	Class 2	Class 2	High class
3	Class 1	Class 2	Medium class
4	Class 2	Class 1	Medium class

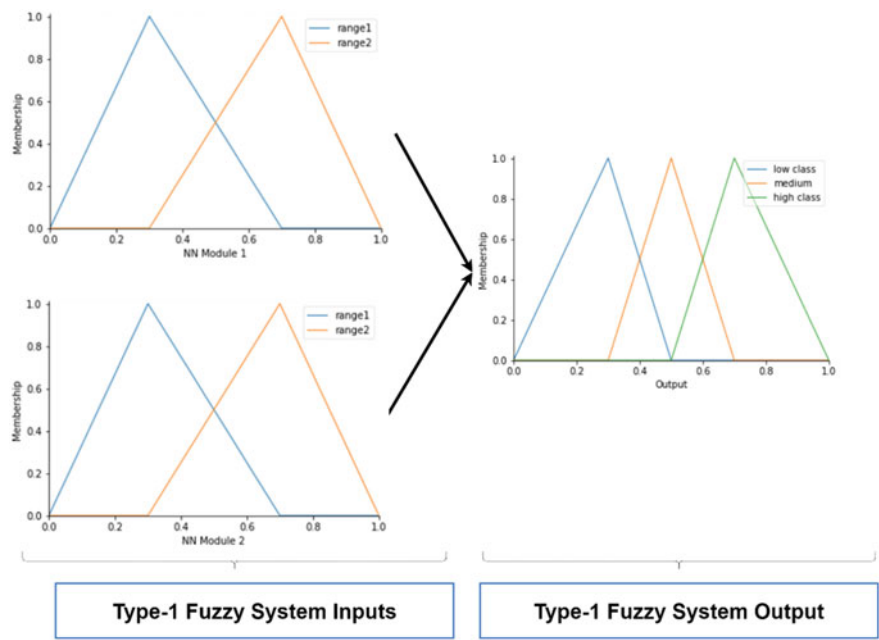


Fig. 2 Type-1 fuzzy system applied to 2-class problems

Table 4 Fuzzy rule set for 3-class problem (Dataset 3)

Fuzzy Rules	Input 1 MLP-LBP prediction	Input 2 CNN prediction	Output
1	Class 1	Class 1	Low class
2	Class 2	Class 2	Medium class
3	Class 3	Class 3	High class
4	Class 1	Class 2	Medium class
5	Class 2	Class 1	Medium class

method used was the centroid and the inferencing method was Max–Min.

4 Results and Discussion

As we explained in the introduction the selected techniques for the training of the modules and the extraction of patterns and feature data are good models precisely created to work in image related problems. Because of that, results obtained were generally good for all the datasets, In Table 6 the results of 30 experiments done for each dataset are shown.

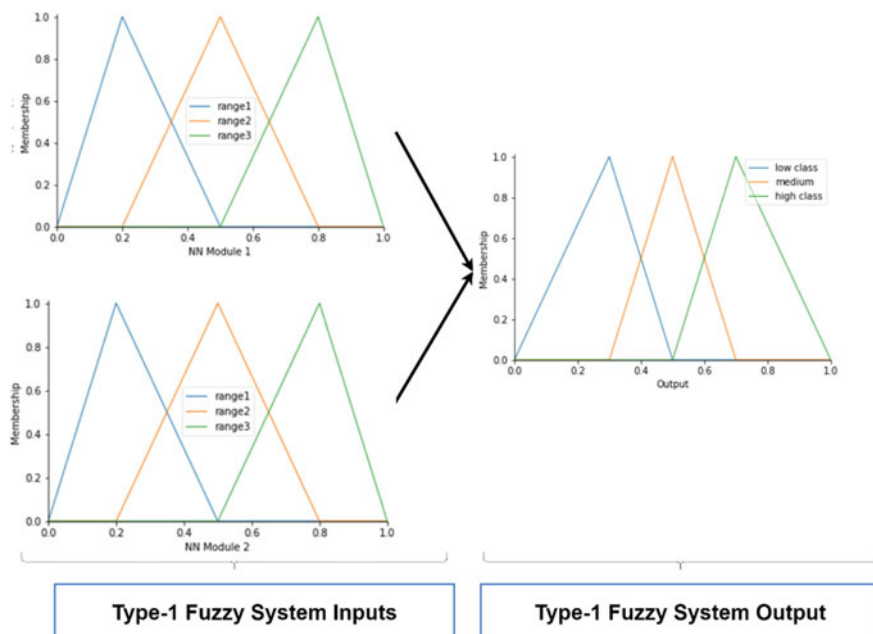


Fig. 3 Type-1 fuzzy system applied to 3-class problems

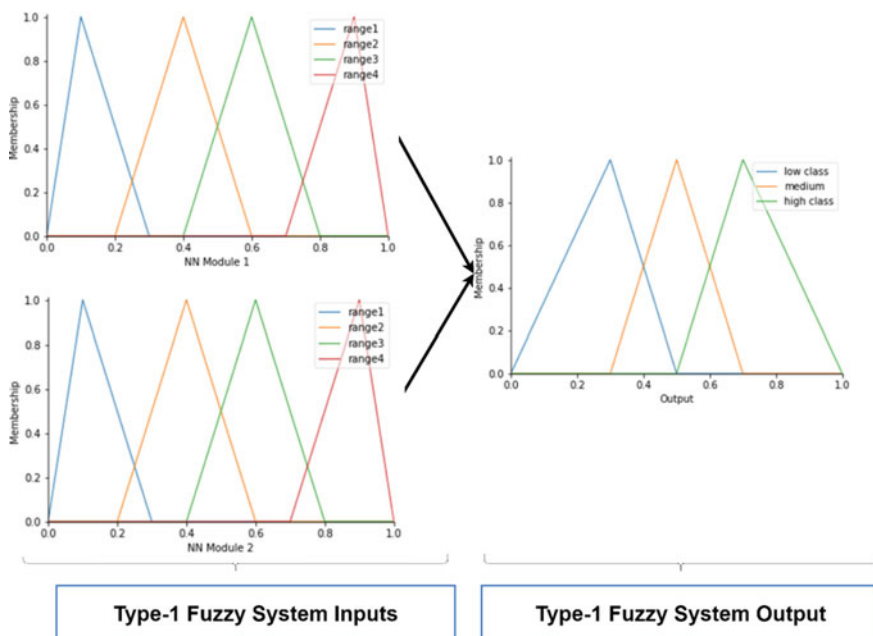


Fig. 4 Type-1 fuzzy system applied to 4-class problems

Table 5 Fuzzy rule set for 4-class problem (Dataset 4)

Fuzzy Rules	Input 1 MLP-LBP prediction	Input 2 CNN prediction	Output
1	Class 1	Class 1	Low class
2	Class 2	Class 2	Medium class
3	Class 3	Class 3	Medium class
4	Class 4	Class 4	High class
5	Class 1	Class 2	Medium class
6	Class 2	Class 1	Medium class

Table 6 Experimental results

	Dataset	Multilayered Perceptron Neural Network with LBP Features (MLP-LBP)	Convolutional Neural Network (CNN)	Proposed Modular Neural Network with Fuzzy Integration (MNN-T1FI)
Montgomery-Shenzen	Average Classification Accuracy on Test Set	0.8914	0.9055	0.9210
	Standard Deviation (σ)	0.009343251	0.008944153	0.005664148
COVID-19 (2-class)	Average Classification Accuracy on Test Set	0.9598	0.9448	0.9649
	Standard Deviation (σ)	0.004334991	0.007591418	0.00262294
COVID-19 and Zhang Lab (3-class)	Average Classification Accuracy on Test Set	0.9524	0.9372	0.9643
	Standard Deviation (σ)	0.006739088	0.003434832	0.004206383
COVID-19, Viral Pneumonia, Normal and Lung Opacities (4-class)	Average Classification Accuracy on Test Set	0.9440	0.9538	0.9744
	Standard Deviation (σ)	0.002440302	0.014117729	0.003420383

In the case of the first dataset that uses images for the tuberculosis disease and has only 2 classes, discrimination between classes becomes an easy task that can be solved with both models, also the number of images of the dataset are 662 which are a few samples in the context of image classification. In this case, the best model was the CNN with 0.9055 testing accuracy and the proposed model boosted the accuracy a little bit more into a 0.9210 testing accuracy. The second dataset that also deals with a binary classification situation, it scores a 0.9598 testing accuracy on the MLP-LBP module, and a 0.9649 on the proposed model. For the third dataset the difficulty is increased by adding a third class into the problem. The best result obtained was with the MLP-LBP model with 0.9524 testing accuracy, and the proposed model increased this accuracy into 0.9643. The fourth dataset increased the problem’s complexity by adding another class and the best module was the CNN, this can also be because of the capabilities of the CNN to work better when there is more images or data available, in this dataset there were 21,165 images in contrast with the first 3 datasets that only used around 700 images each. This advantage gives CNNs the upper hand in dealing with large datasets thus given better results, in the same manner we are using a CNN inside our proposed model, and the combination of fuzzy logic and the predictions of the other model gives back an increase in accuracy, by 2% obtaining a 0.9744 testing accuracy. These are preliminary results, but in order to provide statistical proof we performed the following right tail two sample statistical z-test in Table 7.

In Table 7, we observe a small comparative analysis that performs the statistical Two-Sample Z-Test on the best model from both of the modules and our proposed model with fuzzy logic integration. In the 4 cases is proved that the proposed model reinforces the accuracy of the individual models on the testing set, by including the uncertainty factor and also by providing a second opinion from the results obtained by the second module that scored less, which in itself can be considered proof that the usage of multiple models to solve a problem provides a better understanding and a more precise solution.

Table 7 Comparative statistical analysis

Dataset	HIPOTESIS: Proposed Hybrid Model is better than Best Individual Model	Z-value	Significative evidence
Montgomery-Shenzen (2-class)	MNN-T1FI vs CNN	8.019133	Yes
COVID-19 (2-class)	MNN-T1FI vs MLP-LBP	5.513169	Yes
COVID-19 and Zhang Lab (3-class)	MNN-T1FI vs MLP-LBP	8.204688	Yes
COVID-19, Viral Pneumonia, Normal and Lung Opacities (4-class)	MNN-T1FI vs CNN	7.767425	Yes

5 Conclusions

Following the classification results obtained using the combination of the two already well studied approaches into a singular cooperative system, we can confirm that the usage of more than one image analysis technique provides a better understanding of the problem, extraction of useful information from each image, and as a result a more accurate classification system.

The usage of fuzzy logic into the problem helps in controlling the uncertainty factor always present in medical diagnosis problems and making the classifier obtain a more accurate final result.

More experiments need to be conducted in order to perform a better assessment of the advantages of the usage of a modular image classification system.

So far, favorable results have been obtained with the model and provide an alternative way for image classification effective in analyzing grayscale distribution patterns, similar to those found in Medical Images.

Experimentation on larger multiclass medical datasets needs to be performed, experiments using different membership functions for the fuzzy integrator can be performed and also comparison with other methods.

Acknowledgements we would like to express our gratitude to CONACYT, Tijuana Institute of Technology for the facilities and resources granted for the development of this research.

References

1. Brunetti, A., Carnimeo, L., Trotta, G.P., Bevilacqua, V.: Computer-assisted frameworks for classification of liver, breast and blood neoplasias via neural networks: a survey based on medical images. *Neurocomputing* **335**, 274–298 (2018)
2. Abraham, B., Nair, M.S.: Computer-aided detection of COVID-19 from X-ray images using multi-CNN and Bayesnet classifier. *Biocybernetics Biomed. Eng.* **40**(4), 1436–1445 (2020)
3. Moon, W.K., Lee, Y., Ke, H., Lee, S.H., Huang, C., Chang, R.: Computer-aided diagnosis of breast ultrasound images using ensemble learning from convolutional neural networks. *Comput. Methods Programs Biomed.* **190**, 105361 (2020)
4. Ahmadi, A., Kashefi, M., Shahrokhi, H., Nazaari, M.A.: Computer aided diagnosis system using deep convolutional neural networks for ADHD subtypes. *Biomed. Signal Process. Control* **63**, 102227 (2021)
5. Saygili, A.: A new approach for computer-aided detection of coronavirus (COVID-19) from CT and X-ray images using machine learning methods. *Appl. Soft Comput.* **105**, 107323 (2021)
6. Zheng, S., Shen, Z., Pei, C., Ding, W., Lin, H., Zheng, J., Pan, L., Zheng, B., Huang, L.: Interpretative computer-aided lung cancer diagnosis: From radiology analysis to malignancy evaluation. *Comput. Methods Programs Biomed.* **210**, 106363 (2021)
7. Varela-Santos, S., Melin, P.: A new modular neural network approach with fuzzy response integration for lung disease classification based on multiple objective feature optimization in chest X-ray images. *Expert Syst. Appl.* **168**, 114361 (2021)
8. Ramirez, E., Melin, P., Prado-Arechiga, G.: Hybrid model based on neural networks, type-1 and type-2 fuzzy systems for 2-lead cardiac arrhythmia classification. *Expert Syst. Appl.* **126**, 295–307 (2019)

9. Shihabudheen, K.V., Pillai, G.N.: Recent advances in neuro-fuzzy system: a survey. *Knowl.-Based Syst.* **152**, 136–162 (2018)
10. van Ooijen, P.M.A.: From Physical Film to Picture Archiving and Communication Systems. Springer International Publishing, Basic Knowledge of medical Imaging Informatics (2021)
11. Ziyad, S.R., Radha, V., Vayyapuri, T.: Overview of computer aided detection and computer aided diagnosis systems for lung nodule detection in computed tomography. *Curr. Med. Imaging* **16**(6), 16–26 (2020)
12. Ranschaert, E.R., Morozov, S., Algra, P.R.: Artificial Intelligence in Medical Imaging. Springer International Publishing (2019)
13. Lanca, L., Silva, A.: Digital Imaging Systems for Plain Radiography. Springer, New York (2013)
14. Jacques, S., Christe, B.: Chapter 2—Healthcare technology basics, pp. 21–50. Introduction to Clinical Engineering, Academic Press (2020)
15. Chandola, Y., Virmani, J., Bhadauria, H.S., Kumar, P.: Deep Learning for Chest Radiographs. Computer Aided Classification, Academic Press (2021)
16. Abdelrahman, L., Ghamdi, M.A., Collado-Mesa, F., Abdel-Mottaleb, M.: Convolutional neural networks for breast cancer detection in mammography: a survey. *Comput. Biol. Med.* **131**, 104248 (2021)
17. Calli, E., Sogancioglu, E., van Ginneken, B., van Leeuwen, K.G., Murphy, K.: Deep Learning for chest X-ray analysis: a survey. *Med. Image Anal.* **72**, 102125 (2021)
18. Varela-Santos, S., Melin, P.: A new approach for classifying coronavirus COVID-19 based on its manifestation on chest X-rays using texture features and neural networks. *Inf. Sci.* **545**, 403–414 (2021)
19. Sudharson, S., Kokil, P.: An ensemble of deep neural networks for kidney ultrasound image classification. *Comput. Methods Programs Biomed.* **197**, 105709 (2020)
20. Waheed, A., Goyal, M., Gupta, D., Khanna, A., Al-Turjman, F., Pinheiro, P.R.: CovidGAN: data augmentation using auxiliary classifier GAN for improved Covid-19 detection. *IEEE Access* **8**, 91916–91923 (2020)
21. Varela-Santos, S., Melin, P.: Classification of X-ray images for pneumonia detection using texture features and neural networks. In: Castillo, O., Melin, P., Kacprzyk, J. (eds.) *Intuitionistic and Type-2 Fuzzy Logic Enhancements in Neural and Optimization Algorithms: Theory and Applications. Studies in Computational Intelligence*, Vol. 862. Springer, Cham (2020)
22. Chauhan, T., Palivela, H., Tiwari, S.: Optimization and fine-tuning of DenseNet model for classification of COVID-19 cases in medical imaging. *Int. J. Inf. Manag. Data Insights* **1**(2), 100020 (2021)
23. Varshni, D., Thakral, K., Agarwal, L., Nijhawan, R., Mittal, A.: Pneumonia detection using CNN based feature extraction. 2019 IEEE International Conference on Electrical Computer and Communication Technologies (ICECCT), pp. 1–7 (2019)
24. Verma, G., Prakash, S.: Pneumonia classification using deep learning in healthcare. *Int. J. Innov. Technol. Exploring Eng. (IJITEE)* **9**(4) (2020)
25. Mittal, K., Jain, A., Vaisla, K.S., Castillo, O., Kacprzyk, J.: A comprehensive review on type 2 fuzzy logic applications: past, present and future. *Eng. Appl. Artif. Intell.* **95**, 103916 (2020)
26. Lopez, M., Melin, P.: Response integration in ensemble neural networks using interval type-2 fuzzy logic. 2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence), pp. 1503–1508 (2008)
27. Pulido, M., Mancilla, A., Melin, P.: An ensemble neural network architecture with fuzzy response integration for complex time series prediction. In: Castillo, O., Pedrycz, W., Kacprzyk, J. (eds.) *Evolutionary Design of Intelligent Systems in Modeling, Simulation and Control. Studies in Computational Intelligence*, Vol. 257. Springer, Berlin, Heidelberg
28. Melin, P., Soto, J., Castillo, O., Soria, J.: A new approach for time series prediction using ensembles of ANFIS models. *Expert Syst. Appl.* **39**(3), 3494–3506 (2012)
29. Csiszar, O., Csiszar, G., Dombi, J.: Interpretable neural networks based on continuous-valued logic and multicriteria decision operators. *Knowl.-Based Syst.* **199**, 105972 (2020)

30. Ontiveros-Robles, E., Melin, P., Castillo, O.: Towards asymmetric uncertainty modeling in designing General Type-2 Fuzzy classifiers for medical diagnosis. *Expert Syst. Appl.* **183**, 115370 (2021)
31. Mousavi, S.M., Abdullah, S., Niaki, S.T.A., Banihashemi, S.: An intelligent hybrid classification algorithm integrating fuzzy rule-based extraction and harmony search optimization: medical diagnosis applications. *Knowl. Based Syst.* **220**, 103943 (2021)
32. Melin, P., Miramontes, I., Prado-Arechiga, G.: A hybrid model based on modular neural networks and fuzzy systems for classification of blood pressure and hypertension risk diagnosis. *Expert Syst. Appl.* **107**, 146–164 (2018)
33. Jaeger, S., Candemir, S., Antani, S., Wang, Y.J., Lu, P., Thoma, G.: Two public chest X-ray datasets for computer-aided screening of pulmonary diseases. *Quant. Imaging Med. Surg.* **4**(6), 475–477 (2014)
34. Kermany, D., Goldbaum, M., Cai, W.: Identifying medical diagnoses and treatable diseases by image-based deep learning. *Cell* **172**(5), 1122–1131 (2014)
35. Vaishnavi Jamdade: COVID-19 dataset 3 classes. IEEE Dataport (2020). <https://doi.org/10.21227/q4ds-7j67>
36. Zhao, Y., Zhang, D., Lin, L., Yang, Y.: A method for eliminating the disturbance of pseudo-textural-direction in ultrasound image feature extraction. *Biomed. Signal Process. Control* **71**, 103176 (2021)

Clustering and Prediction of Time Series for Traffic Accidents Using a Nested Layered Artificial Neural Network Model



Martha Ramirez and Patricia Melin

Abstract Our proposal consists of using a nested layer model in which an unsupervised artificial neural network method is used as the first layer to perform tasks of clustering time series corresponding to the statistics of traffic accidents in Mexico for a particular period. As a second layer, a supervised neural network method is used to carry out tasks for the prediction of the number of accidents registered by place of occurrence. The results show that by combining both methods it is possible to use the unsupervised model (first layer) to find similarities in the information and to highlight attributes such as age range, sex, or geographic location, and later by using the supervised method (second layer) to focus on the prediction of the time series considering these attributes.

Keywords Neural networks · Clustering · Time series · Prediction

1 Introduction

For a couple of decades, the amount of data that is generated within organizations and the speed at which this happens has increased significantly. The analysis of historical information frequently is used as a support tool for the decision-making process [1–3]. As it is known, a time series is a sequence or set of data recorded chronologically over a period. Also, predicting the future values of a time series has been a constant challenge for many researchers [4–6].

There are different methods of computational intelligence to perform clustering [7, 8]. These techniques have the objective to find hidden patterns or similar data groups in a dataset [9–11]. As well, there are multiple computational models that have been designed to perform the prediction of time series [12–14]. It is possible that the integration of several of these techniques will be required to solve complex problems in the real world [15–17].

M. Ramirez · P. Melin (✉)
Tijuana Institute of Technology, Tijuana, Mexico
e-mail: pmelin@tectijuana.mx

Artificial Neural Networks (ANNs) are computational models [18–20], that simulate the brain’s behavior, so they can learn directly from the data and be trained to pattern recognition [21, 22]. In the case of the supervised neural networks [23–25] the learning algorithm uses input–output training data to model the dynamic system, on contrary, in the case of the unsupervised neural networks [26, 27] only the input data is given, so they are used to make representative clusters of all the data [28].

At present, there exist several applications based on ANNs that have been demonstrated their efficiency and accuracy to solve diverse problems [29–31]. The main contribution of this paper is to propose a method to perform time series prediction using nested layered neural networks. First at all, unsupervised neural networks were used to generate data clusters, and secondly, a supervised neural network is used to predict future values for each cluster.

This paper is organized as follows. In Sect. 2, the case study is described. The methodology used is explained in Sect. 3. The experimental results are shown in Sect. 4. Finally, in Sect. 5, the general conclusions are presented.

2 Case Study

The problem of road traffic crashes and injuries is a serious public health and development issue, taxing health care systems and undermining their ability to devote limited resources to other areas of need [32].

The study of the Ground Traffic Accidents in Urban and Suburban Areas in Mexico (ATUS statistics) is realized by National Institute of Statistics and Geography (INEGI) and it has the objective to produce annual information on the accident rate of ground transport at the national, federal entity and municipality level, thereby contributing to the planning and organization of both transport and roads [33].

The selected dataset for the period 2012–2018 consists of daily accident records (see Table 1) for the 32 states in Mexico. Although each record consists of 45 attributes, only six attributes were considered to perform the clustering and prediction tasks: state, year, month, day, sex, and age range of the driver.

Each state’s identification number was assigned by state name in alphabetical order, which means geographic location is unrelated (see Table 2).

The analysis of different traffic accident variables like date and time of occurrence, location by urban or suburban area, class and type of accident, type of vehicle involved, rolling surface and driver data, among others, would allow us to know their behavior.

Table 1 Total annual records of traffic accidents in Mexico 2012–2018

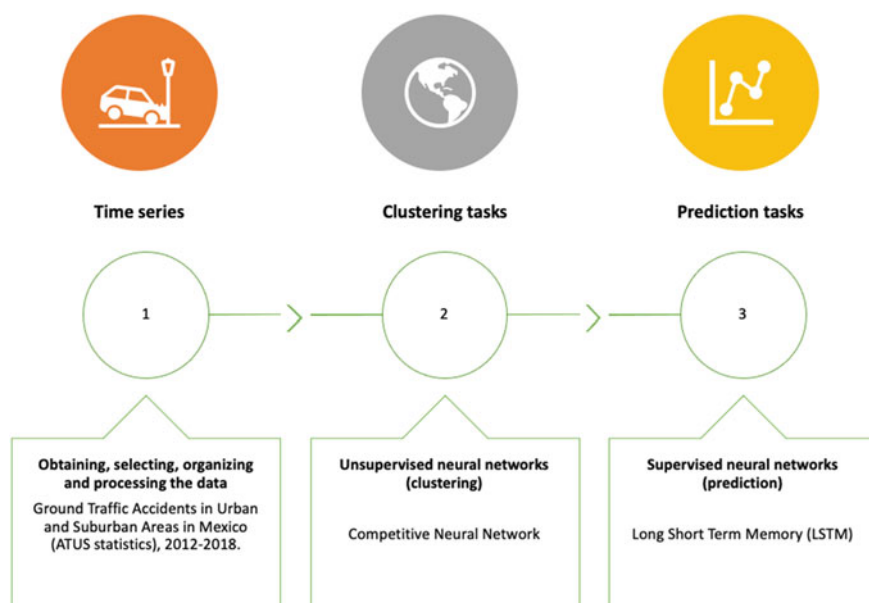
Year	2012	2013	2014	2015	2016	2017	2018
Records	390,411	385,722	380,573	379,948	360,051	367,789	366,498

Table 2 List of 32 states of Mexico by number ID

ID	State	ID	State	ID	State	ID	State
1	Aguascalientes	9	Ciudad de México	17	Morelos	25	Sinaloa
2	Baja California	10	Durango	18	Nayarit	26	Sonora
3	Baja California Sur	11	Guanajuato	19	Nuevo Leon	27	Tabasco
4	Campeche	12	Guerrero	20	Oaxaca	28	Tamaulipas
5	Coahuila	13	Hidalgo	21	Puebla	29	Tlaxcala
6	Colima	14	Jalisco	22	Queretaro	30	Veracruz
7	Chiapas	15	Mexico	23	Quintana Roo	31	Yucatan
8	Chihuahua	16	Michoacan	24	San Luis Potosi	32	Zacatecas

3 Methodology

Our proposal consists of using a nested layer model in which: an unsupervised artificial neural network method is used to perform tasks of clustering time series corresponding to the statistics of traffic accidents in Mexico for a particular period, as first layer. A supervised neural network method is used to carry out tasks for the prediction of the number of accidents registered by place of occurrence, as second layer (see Fig. 1).

**Fig. 1** Conceptual design of the proposed method

The first phase is obtaining, selecting, organizing, and processing the dataset of ground traffic accidents in Mexico. The second phase consists of evaluating the unsupervised neural network for the clustering tasks. In this case we use competitive neural networks [34]. The third phase refers to the evaluation of the supervised neural networks to perform the prediction tasks, such as Long Short-Term Memory (LSTM) neural network which is an upgraded version of Recurrent Neural Network (RNN) [35, 36].

4 Experiments and Results

Based on the monthly number of accidents recorded in the period 2013–2018, the states that recorded similar traffic accident statistics were grouped into four classes (very few, few, many or too many accidents) by using competitive neural networks. As well, based on the sex and age range of the driver similar clusters were formed respectively (see Table 3). For each experiment, 30 executions were performed.

A visual representation of the clusters formed is shown in Fig. 2.

We note that in all simulations, 18 states remained within the cluster “Very Few” accidents, and in the case of the cluster “Too many” accidents, only the state with ID number 19 remained in this category, which represents that this only one state recorded the highest number of accidents in the entire country (see Table 4).

Also, there are 13 states that presented cluster changes in the simulations, basically when the factor age range or sex is considered. The results are presented in Table 5.

In order to perform the prediction tasks, we used a Long Short-Term Memory (LSTM) neural network with 180 neurons in the hidden layer. We considered 70% data for training and 30% for test. For each experiment, 30 executions were performed. The % Root Mean Square Error (RMSE) was used to measure the performance of each neural network.

First, we made the prediction of the number of accidents by state, age range and sex of the driver, based on the data set of the 32 states respectively (see Table 6).

Secondly, we take the parameters obtained by the competitive neural networks, to identify the data of each of the states that belong to each cluster. Then, we use the data corresponding to each cluster to train the LSTM network respectively (see Tables 7, 8, 9 and 10).

Table 3 Clusters of monthly number of accidents recorded by state in the period 2013–2018

Cluster class	Monthly records	Age range	Age range women	Age range men
Very few	28	19	29	18
Few	2	11	1	12
Many	1	1	1	1
Too many	1	1	1	1

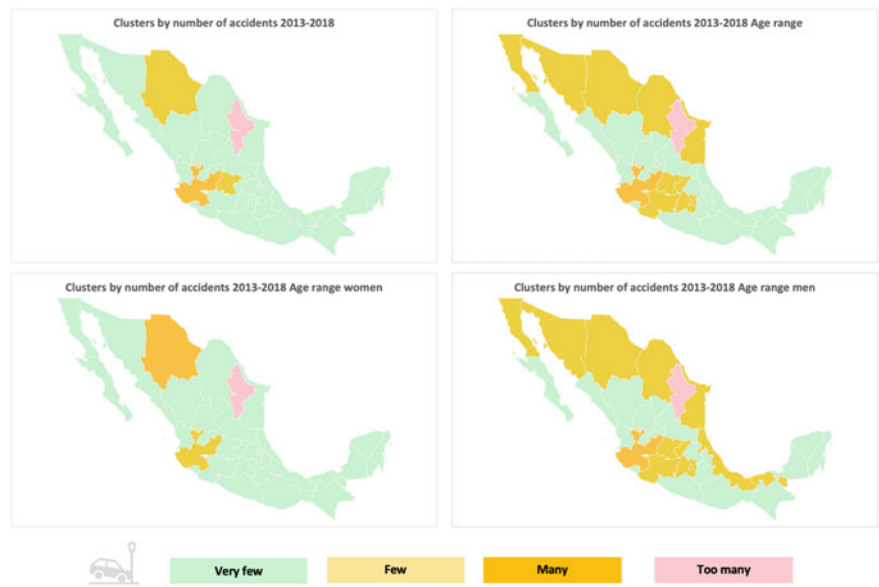


Fig. 2 Clusters by monthly number of accidentes by state 2013–2018

Table 4 Clusters by monthly number of accidentes by state 2013–2018

ID	State	Monthly records	Age range	Age range women	Age range men
1	Aguascalientes	Very few	Very few	Very few	Very few
3	Baja California Sur	Very few	Very few	Very few	Very few
4	Campeche	Very few	Very few	Very few	Very few
6	Colima	Very few	Very few	Very few	Very few
7	Chiapas	Very few	Very few	Very few	Very few
10	Durango	Very few	Very few	Very few	Very few
12	Guerrero	Very few	Very few	Very few	Very few
13	Hidalgo	Very few	Very few	Very few	Very few
18	Nayarit	Very few	Very few	Very few	Very few
19	Nuevo Leon	Too many	Too many	Too many	Too many
20	Oaxaca	Very few	Very few	Very few	Very few
21	Puebla	Very few	Very few	Very few	Very few
23	Quintana Roo	Very few	Very few	Very few	Very few
24	San Luis Potosi	Very few	Very few	Very few	Very few
25	Sinaloa	Very few	Very few	Very few	Very few
27	Tabasco	Very few	Very few	Very few	Very few
29	Tlaxcala	Very few	Very few	Very few	Very few
31	Yucatan	Very few	Very few	Very few	Very few
32	Zacatecas	Very few	Very few	Very few	Very few

Table 5 Multiple clusters by monthly number of accidentes by state 2013–2018

ID	State	Monthly records	Age range	Age range women	Age range men
2	Baja California	Very few	Few	Very few	Few
5	Coahuila	Very few	Few	Very few	Few
8	Chihuahua	Few	Few	Many	Few
9	Ciudad de México	Very few	Few	Very few	Few
11	Guanajuato	Few	Few	Very few	Few
14	Jalisco	Many	Many	Few	Many
15	Mexico	Very few	Few	Very few	Few
16	Michoacan	Very few	Few	Very few	Few
17	Morelos	Very few	Few	Very few	Few
22	Queretaro	Very few	Few	Very few	Few
26	Sonora	Very few	Few	Very few	Few
28	Tamaulipas	Very few	Few	Very few	Few
30	Veracruz	Very few	Very few	Very few	Few

Table 6 Prediction of the number accidents by attributes 2013–2018

Attribute	Average %RMSE	Best %RMSE	Worst %RMSE
State	0.001099408	0.000892990	0.001426313
Age range	0.002031722	0.001478331	0.002679007
Age range women	0.002571199	0.001437401	0.003503766
Age range men	0.002084991	0.001511632	0.002845864

Table 7 Prediction of number accidents by cluster records 2013–2018

Cluster class	Number of elements	Average %RMSE	Best %RMSE	Worst %RMSE
Very few	28	0.001089330	0.000966223	0.001405463
Few	2	0.004298581	0.001711493	0.010981270
Many	1	0.015911502	0.013330322	0.019791860
Too many	1	0.004718022	0.002490341	0.007786377

Table 8 Prediction of number of accidents by cluster age range 2013–2018

Cluster class	Number of elements	Average %RMSE	Best %RMSE	Worst %RMSE
Very few	19	0.001705369	0.001559329	0.002143049
Few	11	0.001379145	0.000869062	0.002850381
Many	1	0.086625433	0.055248528	0.093774513
Too many	1	0.006650431	0.002160609	0.008214891

Table 9 Prediction of number of accidents by cluster age range–women 2013–2018

Cluster class	Number of elements	Average %RMSE	Best %RMSE	Worst %RMSE
Very few	29	0.001621082	0.001378867	0.002058618
Few	1	0.067443688	0.046078795	0.081193765
Many	1	0.008311026	0.007225442	0.010490512
Too many	1	0.005329147	0.001745912	0.009691400

Table 10 Prediction of number of accidents by cluster age range–men 2013–2018

Cluster class	Number of elements	Average %RMSE	Best %RMSE	Worst %RMSE
Very few	18	0.001679730	0.001476107	0.002014695
Few	12	0.001598917	0.001189301	0.002646550
Many	1	0.087618763	0.056619465	0.102783290
Too many	1	0.006796464	0.001795046	0.008346521

The results show that by combining both methods it is possible to use the unsupervised model (first layer) to find similarities in the information and to highlight attributes such as age range, sex, or geographic location, and later by using the supervised method (second layer) to focus on the prediction of the time series considering these attributes.

5 Conclusions

We have presented in this work a nested model for the clustering and prediction of time series of traffic accidents in Mexico using supervised and unsupervised neural networks.

As an unsupervised method, we use a competitive neural network to do the clustering tasks. The simulation results have shown the geographical areas with similar data (clusters), such as the number of accidents, age range, and gender of the driver.

An LSTM neural network, as a supervised method, was used to do the prediction of the number of accidents by state, age range and sex of the driver. As future work, we consider conducting tests with new case studies where the geographic location of the event is significantly relevant data within the time series, to obtain clusters with a greater number of elements, then by dividing the data corresponding to these groups, it is intended to improve the prediction for each region.

References

1. Tsai, Y., Zeng, Y., Chang, Y.: Air pollution forecasting using RNN with LSTM. 2018 IEEE 16th International Conference on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech), 2018, pp. 1074–1079. <https://doi.org/10.1109/DASC/PiCom/DataCom/CyberSciTech.2018.00178>
2. Melin, P., Mancilla, A., Lopez, M., Mendoza, O.: A hybrid modular neural network architecture with fuzzy Sugeno integration for time series forecasting. *Appl. Soft Comput. J.* **7**(4), 1217–1226 (2007), ISSN 1568–4946. <https://doi.org/10.1016/j.asoc.2006.01.009>
3. Melin, P., Castillo, O.: An intelligent hybrid approach for industrial quality control combining neural networks, fuzzy logic and fractal theory. *Inf. Sci.* **177**(7), 1543–1557 (2007). <https://doi.org/10.1016/j.ins.2006.07.022>
4. Sfetos, A., Siriopoulos, C.: Combinatorial time series forecasting based on clustering algorithms and neural networks. *Neural Comput. Appl.* **13**, 56–64 (2004). <https://doi.org/10.1007/s00521-003-0391-y>
5. Li, Y., Bao, T., Gong, J., Shu, X., Zhang, K.: The prediction of dam displacement time series using STL, extra-trees, and stacked LSTM neural network IEEE. Access **8**, 94440–94452 (2020). <https://doi.org/10.1109/ACCESS.2020.2995592>
6. Castillo, O., Melin, P.: Forecasting of COVID-19 time series for countries in the world based on a hybrid approach combining the fractal dimension and fuzzy logic. *Chaos Solitons Fract.* **140** (2020). <https://doi.org/10.1016/j.chaos.2020.110242>
7. Ding, X., Hao, K., Cai, X., Tang, X.-S., Chen, L., Zhang, H.: A novel similarity measurement and clustering framework for time series based on convolution neural networks. *IEEE Access* **8**, 173158–173168 (2020). <https://doi.org/10.1109/ACCESS.2020.3025048>
8. Melin, P., Amezcua, J., Valdez, F., Castillo, O.: A new neural network model based on the LVQ algorithm for multi-class classification of arrhythmias. *Inf. Sci.* **279**, 483–497 (2014). <https://doi.org/10.1016/j.ins.2014.04.003>
9. Austin, E., Coull, B., Zanobetti, A., Koutrakis, P.: A framework to spatially cluster air pollution monitoring sites in US based on the PM2.5 composition. *Environ. Int.* **59**, 244–254 (2013). <https://doi.org/10.1016/j.envint.2013.06.003>
10. Melin, P., Monica, J.C., Sanchez, D., Castillo, O.: analysis of spatial spread relationships of coronavirus (COVID-19) pandemic in the world using self organizing. *Maps Chaos Solitons Fract* **138** (2020). <https://doi.org/10.1016/j.chaos.2020.109917>
11. Melin, P., Castillo, O.: Spatial and temporal spread of the COVID-19 Pandemic using self organizing neural networks and a fuzzy fractal approach. *Sustainability* **13**(8295), 1–17 (2021). <https://doi.org/10.3390/su13158295>
12. Melin, P., Monica, J.C., Sanchez, D., Castillo, O.: A new prediction approach of the COVID-19 virus pandemic behavior with a hybrid ensemble modular nonlinear autoregressive neural network. *Soft Comput.* (2020). <https://doi.org/10.1007/s00500-020-05452-z>
13. Sánchez, D., Melin, P.: Modular neural networks for time series prediction using Type-1 fuzzy logic integration. In: Melin, P., Castillo, O., Kacprzyk, J. (eds.) *Design of Intelligent Systems Based on Fuzzy Logic, Neural Networks and Nature-Inspired Optimization. Studies in Computational Intelligence*, vol. 601, pp. 141–154. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-17747-2_11
14. Castillo, O., Melin, P.: Hybrid intelligent systems for time series prediction using neural networks, fuzzy logic, and fractal theory. *IEEE Trans. Neural Netw.* **13**(6), 1395–1408 (2002). <https://doi.org/10.1109/TNN.2002.804316>
15. Chacón, H.D., Kesici, E., Najafirad, P.: Improving financial time series prediction accuracy using ensemble empirical mode decomposition and recurrent neural networks. *IEEE Access* **8**, 117133–117145 (2020). <https://doi.org/10.1109/ACCESS.2020.2996981>
16. Melin, P., Soto, J., Castillo, O., Soria, J.: A new approach for time series prediction using ensembles of ANFIS models. *Expert Syst. Appl.* **39**(3), 3494–3506 (2012). ISSN 0957–4174, <https://doi.org/10.1016/j.eswa.2011.09.040>

17. Soto, J., Melin, P., Castillo, O.: Time series prediction using ensembles of ANFIS models with genetic optimization of interval Type-2 and Type-1 fuzzy integrators. *Hybrid Intell. Syst.* **11**(3), 211–226 (2014). <https://doi.org/10.3233/HIS-140196>.
18. Valdez, F., Melin, P., Castillo, O.: Modular neural networks architecture optimization with a new nature inspired method using a fuzzy combination of particle swarm optimization and genetic algorithms. *Inf. Sci.* **270**, 143–153 (2014). <https://doi.org/10.1016/j.ins.2014.02.091>
19. Pulido, M., Melin, P.: Comparison of genetic algorithm and particle swarm optimization of ensemble neural networks for complex time series prediction. In: Melin, P., Castillo, O., Kacprzyk, J. (eds.) *Recent Advances of Hybrid Intelligent Systems Based on Soft Computing. Studies in Computational Intelligence*, vol. 915, pp. 51–77. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-58728-4_3
20. Melin, P., Sánchez, D., Castillo, O.: Genetic optimization of modular neural networks with fuzzy response integration for human recognition. *Inf. Sci.* **197**, 1–19 (2012). <https://doi.org/10.1016/j.ins.2012.02.027>
21. Sotirov, S., Sotirova, E., Melin, P., Castillo, O., Atanassov, K.: Modular neural network preprocessing procedure with intuitionistic fuzzy intercriteria analysis method. In: Andreasen, T. et al. (eds) *Flexible Query Answering Systems 2015. Advances in Intelligent Systems and Computing*, vol. 400, pp. 175–186. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-26154-6_14
22. Ramirez, E., Melin, P., Prado-Arechiga, G.: Hybrid model based on neural networks, type-1 and type-2 fuzzy systems for 2-lead cardiac arrhythmia classification. *Expert Syst. Appl.* **126**, 295–307 (2019). <https://doi.org/10.1016/j.eswa.2019.02.035>
23. Moghar, A., Hamiche, M.: Stock market prediction using LSTM recurrent neural network. *Procedia Comput. Sci.* **170**, 1168–1173 (2020). ISSN 1877–0509. <https://doi.org/10.1016/j.procs.2020.03.049>
24. Barbounis, T.G., Theocharis, J.B.: Locally recurrent neural networks for wind speed prediction using spatial correlation. *Inf. Sci.* **177**(24), 5775–5797 (2007). ISSN 0020–0255. <https://doi.org/10.1016/j.ins.2007.05.024>
25. Wei, D.: Prediction of stock price based on LSTM neural network. 2019 International Conference on Artificial Intelligence and Advanced Manufacturing (AIAM), pp. 544–547 (2019). <https://doi.org/10.1109/AIAM48774.2019.00113>
26. Cherif, A., Cardot, H., Boné, R.: SOM time series clustering and prediction with recurrent neural networks. *Neurocomputing* **74**(11), 1936–1944 (2011). <https://doi.org/10.1016/j.neucom.2010.11.026>
27. Melin, P., Castillo, O.: Spatial and temporal spread of the COVID-19 Pandemic using self organizing neural networks and a fuzzy fractal approach. *Sustainability* **13**, 8295 (2021). <https://doi.org/10.3390/su13158295>
28. Melin, P.: Introduction to Type-2 fuzzy logic in neural pattern recognition systems. In: *Modular Neural Networks and Type-2 Fuzzy Systems for Pattern Recognition, Studies in Computational Intelligence*, vol. 389, pp. 3–6. Springer, Berlin, Heidelberg (2012). https://doi.org/10.1007/978-3-642-24139-0_1
29. Zhang, J., Chen, F., Shen, Q.: Cluster-based LSTM network for short-term passenger flow forecasting in Urban Rail Transit. *IEEE Access* **7**, 147653–147671 (2019). <https://doi.org/10.1109/ACCESS.2019.2941987>
30. Li, T., Hua, M., Wu, X.: A hybrid CNN-LSTM model for forecasting particulate matter (PM_{2.5}). *IEEE Access* **8**, 26933–26940 (2020). <https://doi.org/10.1109/ACCESS.2020.2971348>
31. Qian, F., Chen, X.: Stock prediction based on LSTM under different stability. 2019 IEEE 4th International Conference on Cloud Computing and Big Data Analysis (ICCCBDA), pp. 483–486 2019. <https://doi.org/10.1109/ICCCBDA.2019.8725709>
32. Peden, M. et al.: *World report on road traffic injury prevention*, World Health Organization, p. 60 (2004). ISBN 92 4 156260 9. <https://www.who.int/publications/i/item/world-report-on-road-traffic-injury-prevention>
33. “Ground traffic accidents in urban and suburban areas”, INEGI.org.mx. <http://en.www.inegi.org.mx/programas/accidentes/#Documentation>. Accessed 22 Sep 2020

34. Méndez, E., Lugo, O., Melin, P.: A competitive modular neural network for long-term time series forecasting. In: Melin, P., Castillo, O., Kacprzyk, J. (eds.) *Nature-Inspired Design of Hybrid Intelligent Systems, Studies in Computational Intelligence*, vol. 667, pp.243–254. Springer (2012). https://doi.org/10.1007/978-3-319-47054-2_16
35. Hu, Y., Sun, X., Nie, X., Li, Y., Liu, L.: An Enhanced LSTM for trend following of time series. *IEEE Access* **7**, 34020–34030 (2019). <https://doi.org/10.1109/ACCESS.2019.2896621>
36. Zhelev, S., Avresky, D.R.: Using LSTM neural network for time series predictions in financial markets. 2019 IEEE 18th International Symposium on Network Computing and Applications (NCA), pp. 1–5 (2019). <https://doi.org/10.1109/NCA.2019.8935009>

Ensemble Recurrent Neural Networks and Their Optimization by Particle Swarm for Complex Time Series Prediction



Martha Pulido and Patricia Melin

Abstract In this article, we combine convolutional neural networks and particle swarm optimization techniques to design Recurrent Neural Network (RNN) architectures. The proposed particle swarm optimization seeks to find the number of layers, number of modules and number of neurons for layers. This method is applied to time series, the databases used are that of the Petroleum, The US Dollar/MX Exchange Rate and Taiwan Exchange Rate, where the main objective of this work is to find by means optimization algorithm is to minimize the perdition error for each of time series. The simulation results show that the recurrent neural network approach produces a good prediction error.

Keywords Recurrent neural networks · Particle swarm optimization · Time series prediction

1 Introduction

Time series is an ordered sequence of data or observations measured over time. One of the most important utilities of time series is its analysis for the prediction of the measured variable [1–3].

In organizations it is very useful to consider short- and medium-term predictions, for example to find what would happen with the demand for a certain product, future sales, decisions on inventory, supplies, etc.

Recurrent Neural Networks (RNNs) are capable of performing a wide variety of computational tasks including processing sequences, continuation of a trajectory, non-linear prediction, and modeling of dynamical systems.

M. Pulido (✉) · P. Melin
Tijuana Institute of Technology, Tijuana, Mexico
e-mail: martha.pulido@tectijuana.edu.mx

P. Melin
e-mail: pmelin@tectijuana.mx

These networks are also known as space–time or dynamic networks, they are an attempt to establish a correspondence between input and output sequences that they are just temporary patterns.

An RNN can be classified as partially and/or totally recurrent. The totally recurrent are those that each neuron can be connected to any other and their Recurring connections are variable. Partially recurring networks are those that your recurring connections are fixed. The latter are the usual way to recognize or play sequences. They generally have the most forward connections but include a set of feedback connections.

RNN are a field within machine learning in constant development, which in recent times has gained enormous popularity thanks to the promising results obtained in the most diverse fields of application. One of particular interest is the prediction of time series. In a time series, each piece of data has a time dependence on the previous and subsequent samples. This means that a particular analysis of the case is required to be able to make a prediction from the past [4].

Examples of predictive models based on recurrent neural networks can be found in finance [5], electronic commerce [6, 7], capital markets [8, 9], macroeconomic variables [10, 11], health [12, 13], signal processing, meteorology [14, 15], voice recognition [16] and traffic control [17].

In this work the optimization algorithm by means of particles was used since they are tools that help use predict a time series and find good solutions, we have previously done work with this metaheuristic and they have given us good results, we apply it to network optimization neural ensemble for time series.

This work describes the creation of the ensemble recurrent neural network (ERNN), this model is applied to the prediction of the time series [18, 19], the architecture is optimized with particle swarm, (PSO) [20–23]. The responses of the ERNN are integrated with type-1 and type-2 fuzzy systems (IT2FS), [24–27]. The optimization of the recurrent neural network consists in the number of hidden layer (NL), their number of neurons (NN) and the number of modules (NM), in the ERNN, the we integrate responses ERNN, with type-1 and IT2FS and in this way we achieve prediction and Mamdani fuzzy inference system (FIS) has five inputs which are Pred1, Pred2, Pred3, Pred4, and Pred5 and one output is called prediction, the number of inputs of the fuzzy system (FS) is according to the outputs of ERNN and Mamdani fuzzy inference system (FIS) is created, This FIS five inputs, which are Pred1, Pred2, Pred3, Pred4, and Pred5, the range 0 to 1.4, the outputs is called prediction, the range goes from 0 to 1.4 and is granulated into two MFs “Low”, “High”, as linguistic variables [28–30].

This document is conformed as follows: Sect. 2 shows a description of the problem and proposed method, in Sect. 3 Simulation and results, and Sect. 4 Conclusions.

2 Problem Statement and Proposed Method

The proposed method combines recurrent neural networks and particle swarm optimization to design the ERNN architectures. The proposed particle swarm optimization seeks to find the number of layers, number of modules, number of neurons by layers. The proposed method is shown in Fig. 1.

2.1 Description of the Particle Swarm Optimization Applied to Recurrent Neural Network

The parameters of the PSO, are established from previous works [31], as shown in Table 1.

The following equation represent the objective function that we used with genetic algorithm to minimize to prediction error of the time series:

$$ERM = (\sum_{i=1}^d |p_i - x_i|) / d$$

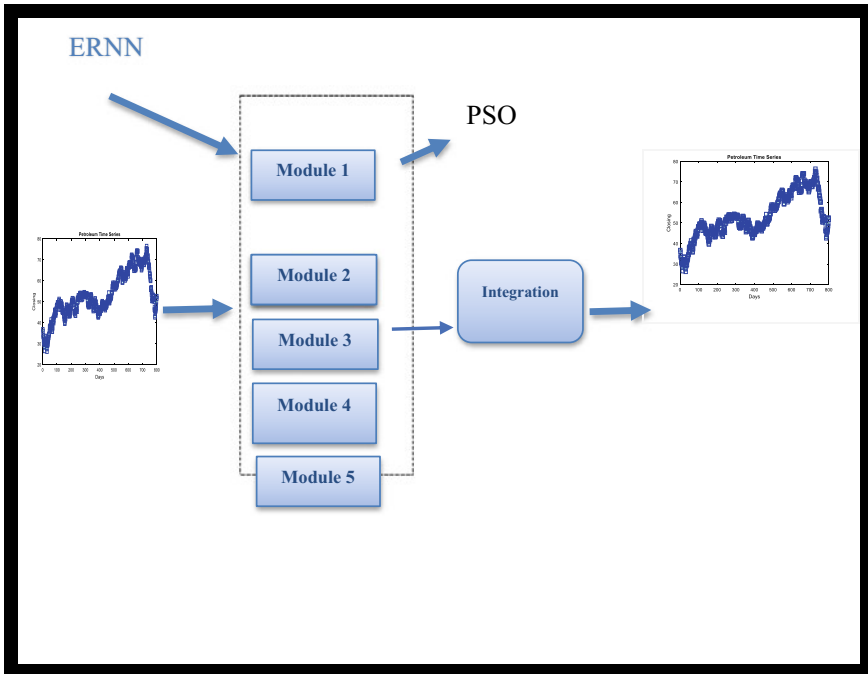


Fig. 1 General Architecture of the proposed method

Table 1 Parameters for the PSO

Parameter	Value
Particles	100
Maximum Iterations	100
C_1	2
C_2	2
w	0.8

Table 2 Parameters for the space search

Parameter	Value	
	minimum	Maximum
NN	1	5
NL	1	3
NN	1	30

$$Prediction\ Error = (ERM_1 + ERM_2 + \dots + ERM_N)/N \quad (1)$$

where p represents the predicted of the data for each of the modules of ensemble recurrent network, X corresponds real data of time series, d the number of data used by time series, ERM is the Prediction error by module of ERNN, N corresponds the number of modules determined by the GA and the *PrediccionError* corresponds to average prediction error achieved by ERNN. The search space parameters as shown in Table 2.

2.2 Data Base

Figure 2 represents the graph of the data Petroleum Time Series [32], where we using 800 data that correspond to period from 07/04/08 to 09/05/11. We used 70% of the for the RNN trainings and 30% to test the RNN.

Figure 3 represents the graph of the data US Dollar/MX Peso time series [33], where we using 800 data that correspond to period from 07/04/08 to 09/05/11. We used 70% of the for the RNN trainings and 30% to test the RNN.

Figure 4 represents the graph of the data Taiwan Stock Exchange time series [34], where we using 800 data that correspond to period from 07/04/08 to 09/05/11. We used 70% of the for the RNN trainings and 30% to test the RNN.

Fig. 2 Series data of Petroleum time series

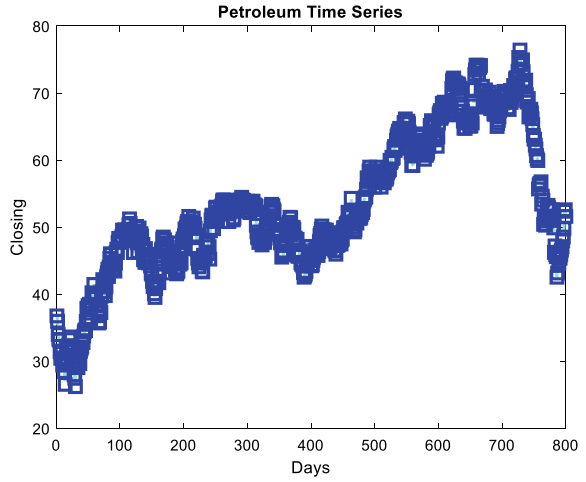
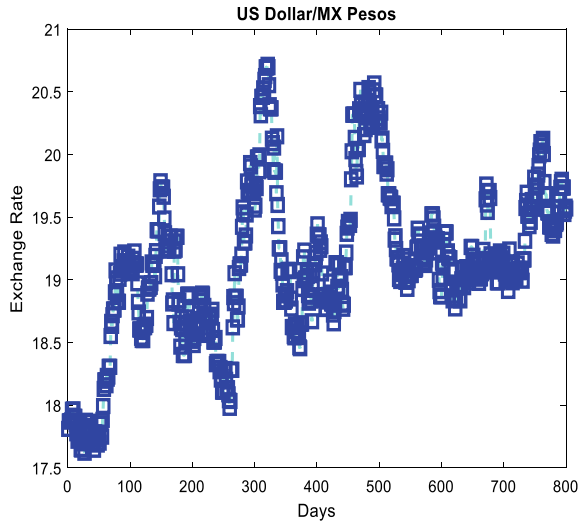


Fig. 3 Series data of US Dollar/MX pesos time series



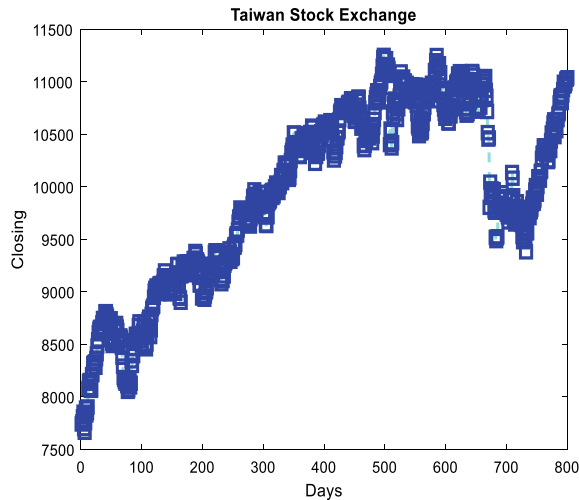
2.3 Description of Type-1 and IT2FS

The next step is the description of the type-1 fuzzy system and IT2FS.

The following equation shows how the total result of the FS is calculated:

$$y = \frac{\sum_{i=1}^n u(x_i) u(x_i)_i}{\sum_{i=1}^n u(x_i)_i} \quad (2)$$

Fig. 4 Series data of Taiwan stock exchange



where u represents MFs and x corresponds the input data.

Figure 5 Show a Mamdani fuzzy inference system (FIS) is created, This FIS five inputs which are Pred1, Pred2, Pred3, Pred4, and Pred5, the range 0 to 1.4., the output is called prediction and the range goes from 0 to 1.4 and is granulated into two MFs “Low”, “High”, as linguistic variables.

Fuzzy system rules are as follows (as shown in Fig. 6), since the fuzzy system has 5 input variables with two MFs and one output with two MFs, therefore the possible number of rules are 32.

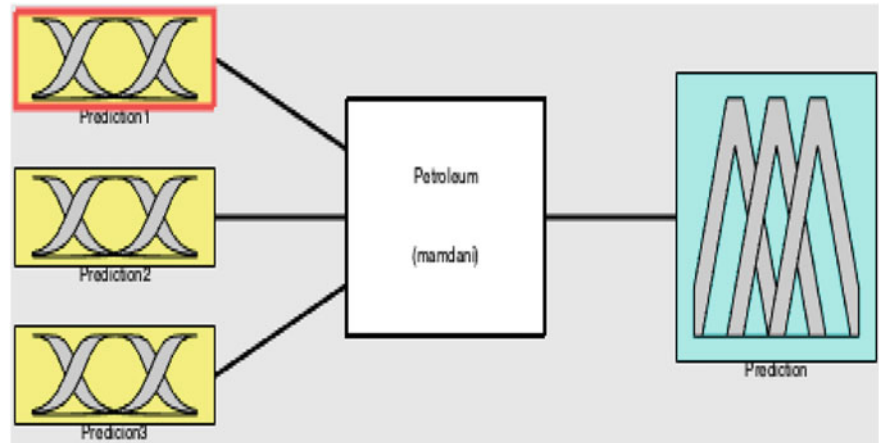


Fig. 5 IT2FS

3 Simulation Results

The proposed method is applied to time series prediction and the results achieved are shown in this Sect. 4. The results of the optimization by means of particles are shown for the time series of Petroleum, Taiwan Stock Exchange and US Dollar/MX pesos.

Table 3 shows the results of 10 experiments of the optimization of ERNN with particle swarm optimization for the Petroleum time series.

Table 4 shows the results of 10 experiments of the type-1 fuzzy for Petroleum time series.

Table 5 represents the prediction of the time series using the IT2FS for the Petroleum time series.

Table 6 shows the results of 10 experiments of the optimization of ERNN with particle swarm optimization for the US Dollar/MX Peso time series.

Table 7 shows the results of 10 experiments of the type-1 fuzzy for the US Dollar/MX Peso time series.

Table 8 represents the prediction of the time series using the IT2FS for the US Dollar/MX Peso time series.

Table 9 shows the results of 10 experiments of the optimization of ERNN with particle swarm optimization for the Taiwan Stock Exchange.

Table 10 shows the results of 10 experiments of the type-1 fuzzy for the Taiwan Stock Exchange time.

Table 11 represents the prediction of the time series using the IT2FS for the Taiwan

Table 3 Results of the optimization de ERNN with PSO, for Petroleum time series

No	Iterations	Particles	Number of modules	Number of layers	Duration	Prediction error
1	100	100	2	16,6	01:38:47	0.019806
2	100	100	1	10	01:40:41	0.01848
3	100	100	2	24 16	01:35:11	0.019806
4	100	100	2	25 15	01:40:41	0.018997
5	100	100	2	25 15	01:15:42	0.018907
6	100	100	2	6,9 28,4	01:45:35	0.019806
7	100	100	1	3	01:35:35	0.01842
8	100	100			01:42:47	0.019888
9	100	100	2	25 15	01:15:42	0.0122111
10	100	100	2	6,9 26,2	01:26:15	0.022751

Table 4 Results for type-1 fuzzy integrator in Petroleum time series

No	Type-1 fuzzy integrator
1	0.3075
2	0.1343
3	0.1383
4	0.1322
5	0.3075
6	0.2012
7	0.1125
8	0.1822
9	0.1383
10	0.3022

Table 5 Results for type-2 fuzzy integrator in Petroleum time series

Experiments	Prediction Error 0.3 uncertainty	Prediction Error 0.4 uncertainty	Prediction Error 0.5 uncertainty
1	0.2258	0.2261	0.2282
2	0.2261	0.2265	0.2268
3	0.2212	0.2233	0.2333
4	0.2247	0.2251	0.2258
5	0.229	0.2335	0.2338
6	0.2219	0.2222	0.2229
7	0.2218	0.2224	0.2230
8	0.2251	0.2258	0.2310
9	0.2255	0.2255	0.2387
10	0.2248	0.2254	0.2394

Stock Exchange.

4 Conclusions

In this paper, the architecture of the recurrent neural network was optimized for the Petroleum, Taiwan Stock Exchange and US Dollar/MX pesos time series and good results were obtained, so we can say that recurrent neural networks do not allow analyzing sequences, and for this it has two inputs, the current data and the previous prediction and by combining these elements it is possible to generate the output of

Table 6 Results of the optimization de ERNN with PSO, for US Dollar/MX Peso time series

No	Iterations	Particles	Number of modules	Number of layers	Duration	Prediction error
1	100	100	2	18 15	01:05:22	0.00062062
2	100	100	3	5,18 15,13 17,18	01:19:33	0.00060268
3	100	100	3	17,5 12,25 20,27	01:28:04	0.00062102
4	100	100	4	10 11 15 18	01:45:20	0.00062632
5	100	100	2	18,19 13,13	01:10:15	0.00060568
6	100	100	3	21,19,14 26,17,12 16,17,18	01:11:03	0.00061166
7	100	100	3	8,22 11,23 16,6	01:33:18	0.00062424
8	100	100	3	18,18,25 15,20,19 16,11,23	01:22:20	0.0062805
9	100	100	3	7,6,11 26,2,18 26,14,16	01:51:21	0.0060541
10	100	100	4	5,27,28 20,20,6 13,12,12 12,26,6	01:44:09	0.00061135

the network as well as preserve the information obtained in previous instants of time, which is precisely equivalent to the memory of the network.

It can be affirmed that the prediction of time series through the use of recurrent neural networks returns very positive results and even in certain circumstances better than those of a statistical method known for its good performance. This means that these models are really useful for predictive applications. However, they need to be improved by special approaches or new experimentation to compensate for the high computational load required for their training.

Table 7 Results for type-1 fuzzy integrator for the US Dollar/MX Peso time series

No	Type-1 fuzzy integrator
1	0.4112
2	0.1120
3	0.1422
4	0.4521
5	0.4425
6	0.4852
7	0.1586
8	0.4220
9	0.4217
10	0.4825

Table 8 Results for type-2 fuzzy integrator for the US Dollar/MX Peso time series

Experiments	Prediction Error 0.3 uncertainty	Prediction Error 0.4 uncertainty	Prediction Error 0.5 uncertainty
1	0.4785	0.4261	0.42282
2	0.4261	0.4265	0.42268
3	0.4268	0.4233	0.42333
4	0.4247	0.4251	0.4258
5	0.4499	0.4335	0.433
6	0.4887	0.4222	0.4229
7	0.4558	0.4544	0.4277
8	0.4490	0.4877	0.4325
9	0.4441	0.4443	0.4337
10	0.4896	0.4987	0.42394

As future work we will consider the optimization of the recurrent neural network with another optimization method, and make comparisons and this by the type-1 and IT2FS [35–38]. We will also test the ability of our method to predict other complex time series.

Table 9 Results of the optimization de ERNN with PSO, for Taiwan Stock Exchange

No	Iterations	Particles	Number of modules	Number of layers	Duration	Prediction error
1	100	100	2	15,16,19 3,25,28	01:42:14	0.00081941
2	100	100	3	4,3 2,11 14,22	01:39:22	0.00082986
3	100	100	3	16 17 15	01:40:13	0.00081966
4	100	100	3	21,17,16 17,19,11 20,15,17	01:42:22	0.00082388
5	100	100	2	15,16,19 3,25,28	01:29:41	0.00089
6	100	100	2	15,6,19 3,25,28	01:23:05	0.00081941
7	100	100	3	4,3 11,26 14,22	01:42:53	0.00082986
8	100	100	3	16 17 15	01:38:17	0.00081966
9	100	100	3	21,17,16 17,19,11 20,15,17	01:41:55	0.00082388
10	100	100	2	17,25,15 36,22,21	01:42:00	0.00073167

Table 10 Results for type-1 fuzzy integrator for the Taiwan Stock Exchange time

No	Type-1 fuzzy integrator
1	0.0399
2	0.0233
3	0.0256
4	0.3861
5	0.3876
6	0.3784
7	0.0246
8	0.2347
9	0.1383
10	0.2148

Table 11 Results for type-1 fuzzy integrator in Taiwan Stock Exchange

Experiments	Prediction Error 0.3 uncertainty	Prediction Error 0.4 uncertainty	Prediction Error 0.5 uncertainty
1	0.0228	0.0030	0.0375
2	0.0218	0.0224	0.02196
3	0.0254	0.0287	0.3015
4	0.00207	0.0245	0.0264
5	0.0235	0.0238	0.0244
6	0.0252	0.0278	0.0294
7	0.0228	0.2557	0.0275
8	0.02299	0.0278	0.0317
9	0.0222	0.0244	0.0277
10	0.02014	0.01999	0.0197

Acknowledgements We would like to express our gratitude to the CONACYT, Tijuana Institute of Technology for the facilities and resources granted for the development of this research.

References

1. Brockwell, P.D., Davis, R.A.: Introduction to Time Series and Forecasting, pp. 1–219. Springer, New York (2002)
2. Davey, N., Hunt, S., Frank, R.: Time Series Prediction and Neural Networks. University of Hertfordshire, Hatfield (1999)
3. Cowpertwait, P., Metcalfe, A.: “Time series”, Introductory Time Series with R., pp. 2–5. Springer Dordrecht, Heidelberg, London, New York (2009)
4. Mikolov, T., Kombrink, S., Burget, L., Černocký, J., Khudanpur, S.: Extensions of recurrent neural network language model. (2011) IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 5528–5531 (2011)
5. Castillo, O., Melin, P.: Comparison of hybrid intelligent systems, neural networks and interval Type-2 fuzzy logic for time series prediction. Proceedings IJCNN (2007), pp. 3086–3091
6. Castillo, O., Melin, P.: Hybrid intelligent systems for time series prediction using neural networks, fuzzy logic, and fractal theory. Neural Netw. IEEE Trans. **13**(6), 1395–1408 (2002)
7. Castro, J., Castillo, O., Melin, P., Mendoza, O., Rodríguez, A.: An interval Type-2 fuzzy neural network for chaotic time series prediction with cross-validation and Akaike test. Soft Computing for Intelligent Control and Robotics, pp.269–285 (2011)
8. Pulido, M., Melin, P., Mendoza, O.: Optimization of ensemble neural networks with Type-1 and interval Type-2 fuzzy integration for forecasting the taiwan stock exchange. Advances in Data Analysis with Computational, pp. 169–181 (2018)
9. Correa, P., Cipriano, A., Nuñez, F., Salas, C., Label, H.: Forecasting copper Electrorefining cathode rejection by means of recurrent neural networks with attention mechanism. IEEE Access **9**, 79080–79088 (2021)
10. Güzelis, C., Yildiz, O.: Recurrent Trend Predictive Neural Network for Multi-Sensor Fire Detection, pp. 84204–84216 (2021)

11. Hoffmann, L.F., Parquet Bizarria, F.C., Parquet Bizarria, J.W.: Detection of liner surface defects in solid rocket motors using multilayer perceptron neural networks. *Polym. Test.* (2020)
12. Sudipto, S., Raghava, G.: Prediction of Continuous B-Cell Epitopes in an Antigen Using Recurrent Neural Network. *National Library of Medicine*, pp. 40–48 (2006)
13. Walid, Alamsyah, A.: Recurrent Neural Network For Forecasting Time Series With Long Memory Pattern, pp. 1–8 (2016)
14. Yao, Q., Dongjin, S., Haifeng, C., Wei, C., Guofei, J., Garrison, C.: A Dual-Stage Attention-Based Recurrent Neural Network for Time Series Prediction. *Computer Science*, Cornell University, pp. 1–7 (2017)
15. Zhang, J., Man, K.F.: Time series prediction using recurrent neural network in multi-dimension embedding phase space. *IEEE Int. Conf. Syst. Man Cybern.* **2**, 11–14 (1998)
16. Zhang, J., Xio, X.: Predicting chaotic time series using recurrent neural network. *IOP Science*, pp. 88–90 (2000)
17. Min, H., Jianhui, X., Shiguo, X., Fu-Liang, Y.: Prediction of chaotic time series based on the recurrent predictor neural network, pp. 3409–3416. *IEEE* (2004)
18. Petnehazi, G.: Recurrent Neural Networks for Time Series Forecasting. *University of Debrecen*, p.p. 1–22 (2019)
19. Rohitash, C., Mengjie, Z.: Cooperative coevolution of Elman recurrent neural networks for chaotic time series prediction. *Neurocomputing* 116–123 (2012)
20. Jerome, T., Connor, R., Douglas, M.: Recurrent neural networks and robust time series prediction. *IEEE Trans. Neural Netw.* **5**(2), 240–254 (1994)
21. Pulido, M., Melin, P.: Ensemble recurrent neural networks for complex time series prediction with integration methods. *Fuzzy Logic Hybrid Extensions of Neural and Optimization Algorithms*, pp. 71–83 (2021)
22. Sherstinsky, A.: Fundamentals of Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) Network. *Physica D: Nonlinear Phenomena*, pp. 1–28 (2020)
23. Mendel, J.: “Uncertain Rule-Based Fuzzy Logic Systems” Introduction an new directions”, pp. 213–231. Prentice-Hall, Inc. (2001)
24. Castillo, O., Melin, P.: Simulation and forecasting complex economic time series using neural networks and fuzzy logic. *Proc. Int. Neural Netw. Conf.* **3**, 1805–1810 (2001)
25. Castillo, O., Melin, P.: “Type-2 Fuzzy Systems”, *Type-2 Fuzzy logic Theory and Application*, pp. 30–43., Ed. Springer (2008)
26. Castillo, O.: *Type-2 Fuzzy Logic in Intelligent Control Applications*. Springer, Berlin, Germany (2012)
27. Karnik, N., Mendel, J.M.: Introduction to type-2 fuzzy logic systems. *IEEE Trans. Signal Process.* **2**, 915–920 (1998)
28. Eberhart, R.C., Kennedy, J.: A new optimizer particle swarm theory. In: *Proceedings of the Sixth Symposium on Micromachine an Human Science*, p. 39–43 (1995)
29. Eberhart, R.C.: *Fundamentals of Computational Swarm Intelligence*, pp. 93–129. Wiley, New York (2005)
30. Melin, P., Pulido, M., Castillo, O.: Ensemble Neural Network with Type-1 and Type-2 Fuzzy Integration for Time Series Prediction and Its Optimization with PSO. *Imprecision and Uncertainty in Information Representation and Processing*, pp. 375–388 (2016)
31. Petroleum Database: <https://mx.investing.com/commodities/crude-oil-historical-data> (June 05,2020)
32. Dollar/MX pesos Database: <http://www.banxico.org.mx> (June 05, 2021)
33. Taiwan Bank Database: www.twse.com/en (June 06, 2021)
34. Melin, P., Sanchez, D.: Multi-objective optimization for modular granular neural networks applied to pattern recognition. *Inf. Sci.* **460–461**, 594–610 (2018)
35. Kennedy, J., Eberhart, R.C.: Particle swarm optimization. In: *Proceedings Intelligent Symposium*, pp. 80–87 (2003)
36. Olivas, F., Valdez, F., Castillo, O., Melin, P.: Dynamic parameter adaptation in particle swarm optimization using interval type-2 fuzzy logic. *Soft Comput.* **20**(3), 1057–1070 (2016)

37. Castillo, O., Castro, J.R., Melin, P., Rodriguez Dias, A.: Application of interval type-2 fuzzy neural networks in non-linear identification and time series prediction. *Soft Comput.* **18**(6), 1213–1224 (2014)
38. Ontiveros, E., Melin, P., Castillo, O.: High order α -planes integration: a new approach to computational cost reduction of general Type-2 fuzzy systems. *Eng. Appl. Artif. Intell.* **74**, 186–197 (2018)

Filter Estimation in a Convolutional Neural Network with Type-2 Fuzzy Systems and a Fuzzy Gravitational Search Algorithm



Yutzil Poma and Patricia Melin

Abstract We propose the adaptation of parameters (number of filters (NF)) of each convolution layer of a convolutional neural network (CNN). Type 2 fuzzy logic is used, which works together with the Fuzzy Gravitational Search Algorithm (FGSA) method, which It is based on the Gravitational Search Algorithm method also known as GSA, which is inspired in Newton's second law and the gravity. The Fuzzy Gravitational Search Algorithm method will help the construction the membership functions of a Type-2 fuzzy system, which we propose for the search and adaptation the filter numbers of each the convolutional layers the convolutional neuron network. The uncertainty between the upper and lower membership functions the Type-2 fuzzy system is manually modified with values of 0.02, 0.05 and 0.08. This work is to demonstrate that Type-2 fuzzy logic used in conjunction with a bioinspired algorithm and working together with the convolutional neural network, obtain better results than applying Type-1 fuzzy logic in image recognition.

Keywords Optimization · Parameter adaptation · Convolutional neural networks · Fuzzy system · Type-2 fuzzy logic · Fuzzy gravitational search algorithm

1 Introduction

In these times the Convolutional neural networks (CNN), have a great boom thanks to their characteristics and their formidable capacity for image recognition and classification. Today there are several types of CNN available, which we can use for the classification and recognition of images such as ResNet [1] AlexNet [2] or LeNet [3].

The CNNs have been used in conjunction with different methods mainly for their classification or recognition of images, for example in the recognition of Chinese spelling sign language [4], or in [5] where these networks are used to speech recognition or face recognition as in [6–10]. Some the models obtained in the different experiments in conjunction with other methods are used to solve problems and expand the

Y. Poma · P. Melin (✉)

Tijuana Institute of Technology, Tijuana, Mexico

e-mail: pmelin@tectijuana.mx

solutions these increasingly complicated using methods such as transfer learning [11, 12], the optimization of neural networks (NN) have helped to obtain best outcome as in [13, 14] where COVID-19 conditions are differentiated from people have good health in chest radiographs using the CNN which is optimized or using an optimized CNN for detection Cancer in the skin [15], or [16] where the convolutional neural network is optimized for intra video prediction.

The case study selected for this research is the ORL database [17] which has been used in many methods and techniques for recognition as in [18, 19], since this database is widely used for tests in the image recognition.

Thanks to the publication of "fuzzy sets" in 1965 by Zadeh [20], fuzzy logic (FL) has been introduced in various hybridized methods with neural networks as well as bio-inspired algorithms among others, as in [21] where fuzzy logic is applied to a monitor system or where prospects are identified based for learning foreign languages through games [22]. Other works where Type-1 FL has been used in [23] where a neural network is optimized using a fuzzy classifier or in [24] which where Type-1 is compared with Type-2 FL by intervals for control problems, as well as in [25] where Type-1 vs Type-2 is statistically analyzed by intervals in a dynamic adaptation of BCO parameters.

Type-2 fuzzy systems (FS) have given remarkable results in the experimentation carried out collate to Type-1 FL, although it's clear that it depends a lot on the problem to be solved for the use the solution and to define if it is required to use Type-1 FL or Type-FL. Some the works in which Type-2 fuzzy logic has been practical are [26, 27] where it is used for control or in which the parameters are dynamically adapted used harmony algorithm with Type-2 FL by intervals [28], some other works where it has been used [29–31].

There are many methods for the optimization of neural networks or other algorithms, the Fuzzy Gravitational Search Algorithm [32] has outstanding in experimentation thanks to its search capacity and the application of fuzzy logic within it. Some the most relevant works where they have been used is in [33, 34].

This work is segmented into: in the Section number 2 there are concepts reference to CNN also Type-2 fuzzy logic. In Sect. 3, we'll see the proposal of this work in which we use Type-2 FL for the adaptation the network parameters. In Sect. 4 we will see the results obtained in the implementation the proposal of this work and the results will be collated with Type-1 FS and Type-2 FS as well as the optimization the network for the adaptation of its parameters. Finalizing, in Sect. 5 are the conclusions the experimentation carried out.

2 Literature Review

In this segmented, we showed the concepts to understand the proposed method in this work:

2.1 Convolutional Neural Networks

One the networks with the most growth in our times is CNN, which is an artificial neural network, and is causative for imitating the natural learning the human being, identifies through characteristics of objects and Thus it manages to classify or recognize the images. This network is categorized by having layers, each one them is in charge of extracting characteristics the images “filtering” the information and in the end as determined it will classify it in a totally connected classification layer. Start the first layer is the convolution layer [35] which extracts the characteristics by means of a filter which multiplies the values of this by the values the image and later adds them in a space called “characteristic map”, in this layer is applied an activation function [36], later this map of characteristics goes to the next layer the network also named “Pooling” or sometimes also called “Aggregation layer”, [37] in this layer is in charge of continuing to filter the data or characteristics taken with an empty mask which varies in size (generally it is $2 * 2$) and it goes through the characteristics map choosing the value that is within this mask already be it the maximum these or the average the total the values that are currently within the mask. ending with the last layer, which is the classification layer, which is fully-connected, in this layer are the number of output classes that are required for the classification or recognition the images [38]. The number of each the layers varies according to the depth the network, the deeper it will take more time and will take more computational resources.

2.2 Type-2 Fuzzy Logic System

In 1999, Mendel and other authors started Type-2 fuzzy logic, which handles rule inserts, these consist of a defuzzification that is in charge of converting real value to fuzzy value, a fuzzy inference machine that applies a fuzzy reasoning to get fuzzy output; an output processor consisting of a reducer that transforms a Type-2 fuzzy set to a Type-1 fuzzy set, and a diffuser that translates an output to a precise value [39].

The membership functions (MF), also called FOU which is the variation between the higher membership function and the lower membership function, depends on the interval between these varies influences the optimal result the problem in question. It could be said that both the upper and lower MF are Type-1 MF. In Fig. 1 we can see a Type-2 membership function.

3 Proposed Method

Based on the experiments executed previously [40] where the parameters the number of filters (NF) of each convolution layer the CNN are adapted, with the difference that in the past a Type-1 FS was used to adapt these parameters.

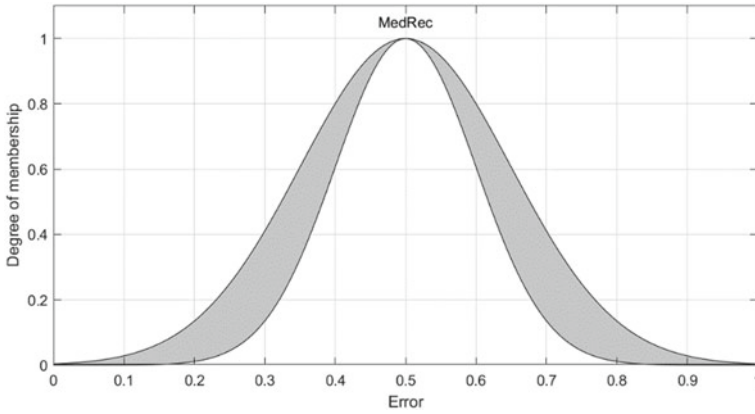


Fig. 1 Example of Type-2 MF

Our proposal consists of adapting the parameters the convolutional neural network, which together with the Fuzzy Gravitational Search Algorithm method adapts the number of filters of each convolutional layer the network. A Type-2 FS was designed, see Fig. 2, it is detailed as follows: it has 1 input which is the Error (E1) that the CNN throws, in addition, RC corresponds to recognition, it varies according to the membership function (MF), also, it has 2 outputs which correspond to the NF1 and the NF2 the convolution layer, each input and output the FS has 3 MF respectively, the fuzzy rules can be found in Table 1, this FS works together with the FGSA method which is in charge of building the functions of Gaussian type membership.

For each membership function corresponding to the input and output they were the Gaussian type with uncertainty in standard deviation, which is defined in Eq. 1 and in Fig. 3.

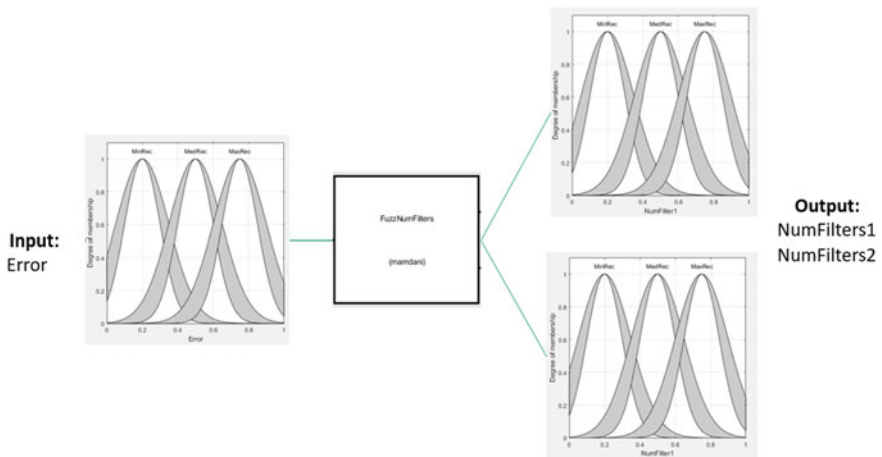
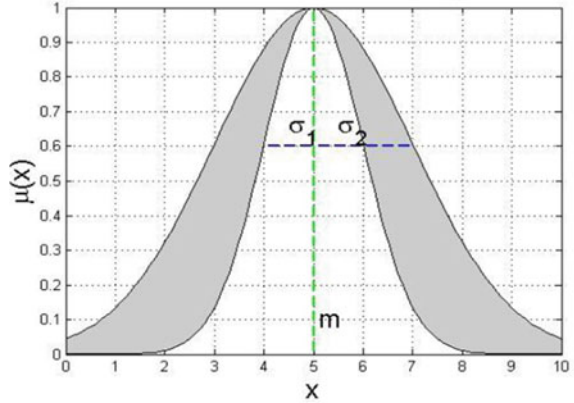


Fig. 2 Type-2 FS

Table 1 Rules of Type-2 FS

Rules						
1	If	E1 is -RC	then	NF1 is -RC	and	NF2 is -RC
2	If	E1 is 1/2RC	then	NF1 is 1/2RC	and	NF2 is 1/2RC
3	If	E1 is + RC	then	NF1 is + RC	and	NF2 is + RC

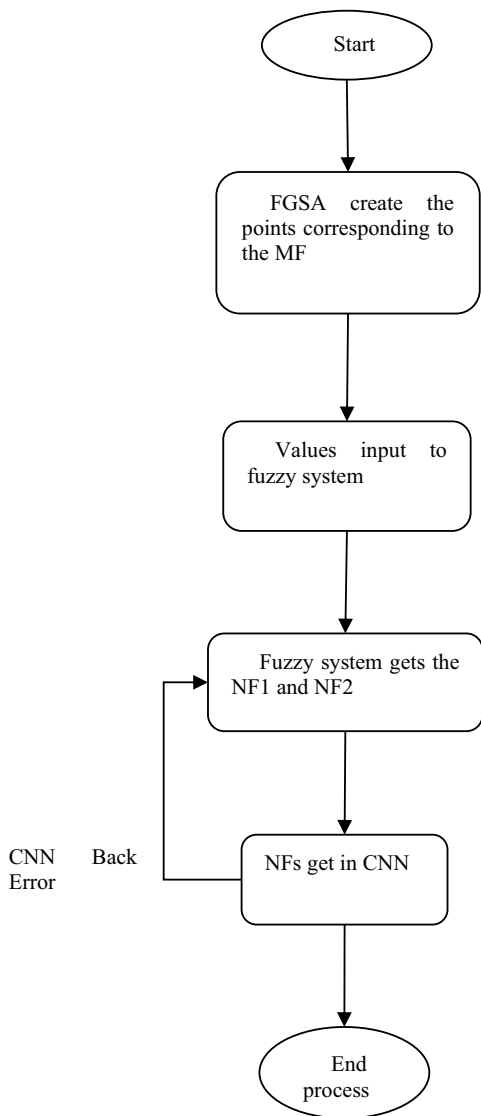
Fig. 3 Gaussian MF with uncertainty SD

$$\mu_{\bar{F}}(x) = \exp \left[-\frac{1}{2} \left(\frac{x-m}{\sigma} \right)^2 \right] \quad \sigma \in [\sigma_1, \sigma_2] \quad (1)$$

$$\bar{\mu}_{\bar{F}}(x) = \exp \left[-\frac{1}{2} \left(\frac{x-m}{\sigma_2} \right)^2 \right]$$

$$\underline{\mu}_{\bar{F}}(x) = \exp \left[-\frac{1}{2} \left(\frac{x-m}{\sigma_1} \right)^2 \right]$$

In Fig. 4 we can find the diagram of our proposed approach which consists of entering the data, later with the support the FGSA method which creates the agents that build the MF the Type-2 FS, so the last one is in charge to adapt the parameters that correspond to the NFs in convolution layer one and convolution layer two. These parameters enter the convolutional neural network thus giving the optimal value found by the FS to NFs the convolution layers respectively. The data (images) pass through CNN thus obtaining their recognition.

Fig. 4 Proposed approach

4 Results and Discussion

The selected case study was the ORL database [17], it consists of 400 images of human faces taken at different angles, there are 40 different people and each one them has 10 images their faces. Each image is black and white and is 112 * 92 pixels size. In Fig. 5 we will see some the images of this database.

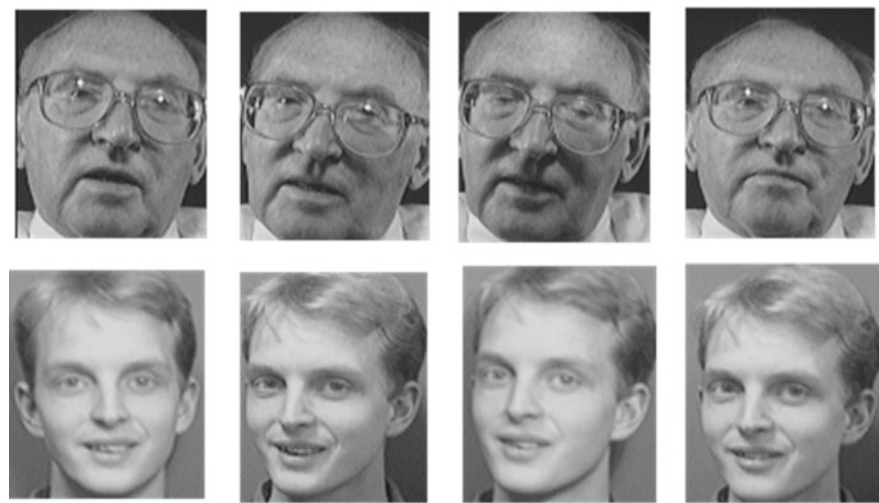


Fig. 5 Examples ORL database

The architecture used for the CNN was having (2) convolution layers and (2) pooling layers respectively and (1) classification layer.

Three main experiments were carried out in which the footprint of insecurity also known as FOU (Footprint of Uncertainty) varies from 0.02, 0.05 and 0.08. Each experiment was performed 30 times for each training epoch, these were 50, 70, 100, 200, 500, 1000 and 2000 training epochs the convolutional neural network. The 60% the data from the ORL database we was used for network training and 40% the data was for tested CNN.

In Table 2, we can spot the results obtained when the insertion footprint is 0.02, which the maximum recognition value found is 96.25% when the neural network is trained 2000 times, the number of filters 1 and 2 is 24 filters for each convolution layer. The average 94.18% for the recognition the images and with a standard deviation (SD) of 1.28.

In the Table 3 we can see the results when the insertion footprint is 0.05, the best result was 96.25% of image recognition with 2000 times of network training, the NFs

Table 2 Results with Type-2 FS with FOU = 0.02

Epochs	Experiment number	Recognition rate	NF1	NF2	Average	SD
50	8	95.62	25	27	92.77	1.22
70	19	95.62	43	39	93.10	1.28
100	20	95	29	39	93.16	1.12
200	28	95.62	23	25	93.37	0.93
500	12	95	36	34	93.33	0.87
2000	2	96.25	24	24	94.18	1.28

Table 3 Results with Type-2 FS with FOU = 0.05

Epochs	Experiment number	Recognition rate	NF1	NF2	Average	SD
50	13	96.87	18	19	92.33	1.45
70	10	96.81	20	21	92.36	1.51
100	2	95	23	23	92.45	1.26
200	12	96.25	28	29	93.18	1.40
500	25	96.87	43	44	93.25	1.27
1000	17	96.01	26	23	93.97	1.21
2000	2	96.25	10	9	94.31	1.15

in convolution layer 1 is 10 while in convolution layer 2 it is 9, while the average was 94.31 with a SD of 1.15.

When the insertion footprint is 0.08 in the Type-2 FS, we can see in the Table 4, that the best value obtained was when the network was trained at 2000 epochs for image recognition with 95.62% in turn, with 40 NFs. in each the convolution layers that make up the convolutional neural network, having a best average of 93.75% and a SD of 1.21.

In Table 5 we'll view the abstract the best values obtained when the insertion footprint is manually modified with 0.02, 0.05 and 0.08.

We can see that the highest image recognition value was 95.62% with 0.08 in the FOU, but if we observe the averages we can define that the best value was obtained was 94.18% in recognition when the FOU is 0.02. In all cases the best recognition value was when the convolutional neural network is trained with 2000 epochs. In

Table 4 Results with Type-2 FS with FOU = 0.08

Epochs	Experiment number	Recognition rate	NF1	NF2	Average	SD
50	1	93.12	13	14	92	0.92
70	6	93.75	31	33	92.25	1.07
100	30	95.62	45	49	92.54	1.12
200	6	95	41	34	93.04	1.20
500	6	96.25	22	22	93.18	1.23
1000	13	95	20	20	93.31	0.99
2000	2	95.62	40	40	93.75	1.21

Table 5 Comparison of best results

FOU	Epochs	Recognition rate	NF1	NF2	Average	SD
0.02	2000	96.25	24	24	94.18	1.28
0.05	2000	96.25	10	9	94.31	1.15
0.08	2000	95.62	40	40	93.75	1.21

Table 6 Comparison with other optimization methods for CNN

CNN detail	Optimized	Recognition rate (%) Max	Recognition rate (%) Max average
Convolutional Neural Network (70 Epochs) Optimized with FGSA [41]	FGSA	97.5	94.43
Convolutional Neural Network (70 Epochs) adaptation of parameters With FL [40]	FL T1	95.62	93.77
Convolutional Neural Network (2000 Epochs) adaptation of parameters FL FOU = 0.02	FL T2	96.25	94.18
Convolutional Neural Network (2000 Epochs) adaptation of parameters with FL FOU = 0.05	FL T2	96.25	94.31
Convolutional Neural Network (2000 Epochs) adaptation of parameters with FL FOU = 0.08	FL T2	95.62	93.75

Table 6 we will view the comparison the results the CNN with different methods in its optimization, from the optimization the CNN with the FGSA method without any help from a FS of any kind, in addition the network was optimized with a Type-1 FS [40] we present the adaptation of parameters the NF of each convolution layer with a Type-2 FS in which the insert fingerprint is modified manually.

Where the best result in image recognition is 97.5% when CNN is optimized with the FGSA, we can also observe that CNN optimized applying Type-1 fuzzy system results in lower results to be used in the search for parameters a type-2 fuzzy when the FOU is 0.05 with an average 94.31% recognition.

5 Conclusions

As final conclusions we can define based on the experiments carried out, we can state that ultimately better results are obtained when the CNN is optimized with the FGSA method both in the maximum recognition and in the average, above the values to the parameter adaptation, sing Type-1 or Type- 2 FS. The comparison the values obtained both in the average and in the maximum of image recognition using

a Type-1 FS is significantly less than using a Type-2 FS in adapting parameters the NFs of each layer of convolution the CNN. It was found that a maximum image recognition is obtained, an equal value using the Type-2 FS, varying FOU of 0.02 and 0.05, but if the averages are bought we can define that the best solution we can find when the inset fingerprint is of 0.05. As future work, it is intended to carry out more experimentation in addition to testing this method with more complex databases.

Acknowledgements We are honored and grateful to be sponsored by CONACYT & the Tijuana Institute of Technology who support us financially with the scholarship number 816488.

References

1. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of IEEE Conference Computer Vision Pattern Recognition (CVPR), pp. 770–778 (Jun 2016)
2. Krizhevsky, A., Sutskever, I., Hinton, G.E.: Imagenet classification with deep convolutional neural networks. In: Proceedings of Advances in Neural Information Processing Systems, pp. 1097–1105 (2012)
3. Lecun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. *Proc. IEEE* **86**(11), 2278–2324 (1998)
4. Gao, Y., Jia, C., Chen, H., Jiang, X.: Chinese fingerspelling sign language recognition using a nine-layer convolutional neural network. *EAI Endorsed Trans. e Learn.* **7**(20), e2 (2021)
5. Hourri, S., Nikolov, N.S., Kharroubi, J.: Convolutional neural network vectors for speaker recognition. *Int. J. Speech Technol.* **24**(2), 389–400 (2021)
6. Liu, W., Zhou, L., Chen, J.: Face Recognition Based on Lightweight Convolutional Neural Networks. *Information* **12**(5), 191 (2021)
7. Li, K., Jin, Y., Waqar Akram, M., Han, R., Chen, J.: Facial expression recognition with convolutional neural networks via a new face cropping and rotation strategy. *Vis. Comput.* **36**(2), 391–404 (2020)
8. Felea, I.I., Dogaru, R.: Improving light-weight convolutional neural networks for face recognition targeting resource constrained platforms. *ESANN 2020*, pp. 199–204 (2020)
9. Deng, Z., Peng, X., Li, Z., Qiao, Y.: Mutual component convolutional neural networks for heterogeneous face recognition. *IEEE Trans. Image Process.* **28**(6), 3102–3114 (2019)
10. Yang, Z., Xiong, H., Chen, X., Liu, H., Kuang, Y., Gao, Y.: Dairy cow tiny face recognition based on convolutional neural networks. *CCBR 2019*, pp. 216–222 (2019)
11. Jialin Pan, S., Yang, Q.: A survey on transfer learning. *IEEE Trans. Knowl. Data Eng.* **22**(10), 1345–1359 (Oct 2010)
12. Tan, C., Sun, F., Kong, T., Zhang, W., Yang, C., Liu, C.: A survey on deep transfer learning. In: Proceedings of International Conference on Artificial Neural Networks, pp. 270–279 (2018)
13. Govindarajan, S., Swaminathan, R.: Differentiation of COVID-19 conditions in planar chest radiographs using optimized convolutional neural networks. *Appl. Intell.* **51**(5), 2764–2775 (2021)
14. Govindarajan, S., Swaminathan, R.: Correction to: differentiation of COVID-19 conditions in planar chest radiographs using optimized convolutional neural networks. *Appl. Intell.* **51**(5), 2776 (2021)
15. Zhang, N., Cai, Y.X., Wang, Y.Y., Tian, Y.T., Wang, X.L., Badami, B.: Skin cancer diagnosis based on optimized convolutional neural network. *Artif. Intell. Med.* **102**, 101756 (2020)
16. Meyer, M., Wiesner, J., Rohlfing, C.: Optimized convolutional neural networks for video intra prediction. *ICIP 2020*, pp. 3334–3338 (2020)

17. Face recognition using ortho-diffusion bases-Scientific Figure on ResearchGate. https://www.researchgate.net/figure/The-face-image-data-from-the-ORL-database_fig2_233545388. Accessed 02 Feb 2021
18. Eleyan, A., Demirel, H.: PCA and LDA Based Neural Networks for Human Face Recognition (2007). <https://doi.org/10.5772/4833>.
19. Wang, Q., Cheng, J., Gao, Q., Zhao G., Jiao, L.: Deep multi-view subspace clustering with unified and discriminative learning. In: IEEE Trans. Multimed. <https://doi.org/10.1109/TMM.2020.3025666>
20. Zadeh, L.A.: Fuzzy sets. *Inf. Control* **8**(3), 338–353 (1965). ISSN 0019-9958
21. Khan, N., Elizondo, D.A., Dekka, L., Molina-Cabello, M.A.: Fuzzy logic applied to system monitors. *IEEE Access* **9**, 56523–56538 (2021). <https://doi.org/10.1109/ACCESS.2021.3072239>
22. Yanes, N., Bououd, I., Alanazi, S.A., Ahmad, F.: Fuzzy logic based prospects identification system for foreign language learning through serious games. *IEEE Access* **9**, 63173–63187 (2021). <https://doi.org/10.1109/ACCESS.2021.3074374>
23. Guzmán, J.C., Melin, P., Prado-Arechiga, G.: Optimization for type-1 and interval type-2 fuzzy systems for the classification of blood pressure load using genetic algorithms. In: *Intuitionistic and Type-2 Fuzzy Logic Enhancements in Neural and Optimization Algorithms*, pp. 63–71 (2020)
24. Castillo, O., Angulo, L.A., Castro, J.R., Valdez, M.G.: A comparative study of type-1 fuzzy logic systems, interval type-2 fuzzy logic systems and generalized type-2 fuzzy logic systems in control problems. *Inf. Sci.* **354**, 257–274 (2016)
25. Angulo, L.A., Mendoza, O., Castro, J.R., Díaz, A.R., Melin, P., Castillo, O.: Fuzzy sets in dynamic adaptation of parameters of a bee colony optimization for controlling the trajectory of an autonomous mobile robot. *Sensors* **16**(9), 1458 (2016)
26. Bernal, E., Lagunes, M.L., Castillo, O., Soria, J., Valdez, F.: Optimization of type-2 fuzzy logic controller design using the GSO and FA algorithms. *Int. J. Fuzzy Syst.* **23**(1), 42–57 (2021)
27. Cuevas, F., Castillo, O., Cortés-Antonio, P.: Design of a control strategy based on type-2 fuzzy logic for omnidirectional mobile robots. *J. Multiple Valued Log. Soft Comput.* **37**(1–2), 107–136 (2021)
28. Valdez, F., Peraza, C.: Dynamic parameter adaptation in the harmony search algorithm for the optimization of interval type-2 fuzzy logic controllers. *Soft Comput.* **24**(1), 179–192 (2020)
29. Karmaka, S., Seikh, M.R., Castillo, O.: Type-2 intuitionistic fuzzy matrix games based on a new distance measure: application to biogas-plant implementation problem. *Appl. Soft Comput.* **106**, 107357 (2021)
30. Valdez, F.: A review of optimization swarm intelligence-inspired algorithms with Type-2 fuzzy logic parameter adaptation. *Soft Comput.* **24**(1), 215–226 (2020)
31. Ontiveros Robles, E., Melin, P., Castillo, O. (2018). Comparative analysis of noise robustness of type 2 fuzzy logic controllers. *Kybernetika* **54**(1), 175–201
32. Sombra, A., Valdez, F., Melin, P., Castillo, O.: A new gravitational search algorithm using fuzzy logic to parameter adaptation. In: *2013 IEEE Congress on Evolutionary Computation*, no. 3, pp. 1068–1074 (2013)
33. Mirjalili, S., Hashim, S.Z.M., Sardroudi, H.M.: Training feedforward neural networks using hybrid particle swarm optimization and gravitational search algorithm. *Appl. Math. Comput.* **218**(22), 11125–11137 (2012)
34. Hatamlou, A., Abdullah, S., Othman, Z.: Gravitational search algorithm with heuristic search for clustering problems. In: *Conference Data Mining Optimization*, pp. 190–193 (June 2011)
35. LeCun, Y., Bengio, Y.: Convolution networks for images, speech, and time-series. *Igarss* **2014**(1), 1–5 (1998)
36. Nair, V., Hinton, G.E.: Rectified linear units improve restricted Boltzmann machines. In: *Proceedings 27th International Conference Machine Learning*, no. 3, pp. 807–814 (2010)
37. Yang, J., Yu, K., Gong, Y., Beckman, T.H.: Linear spatial pyramid matching using sparse coding for image classification. *IEEE Computer Society Conference Computer Vision Pattern Recognition*, pp. 1794–1801 (2009)

38. Venkatesan, R., Li, B.: *Convolutional Neural Networks in Visual Computing: A Concise Guide*. CRC Press (2017)
39. Karnik, N.N., Mendel, J.M., Liang, Q.: Type-2 fuzzy logic systems. *IEEE Trans. FUZZY Syst.* **7**, 16 (1999)
40. Poma, Y., Melin, P.: Estimation the number of filters in the convolution layers of a convolutional neural network using a fuzzy logic system. In: *Fuzzy Logic Hybrid Extensions of Neural and Optimization Algorithms*, pp. 1–14 (2021)
41. Poma, Y., Melin, P., González, C.I., Martínez, G.E.: Optimization of convolutional neural networks using the fuzzy gravitational search algorithm. *J. Autom. Mob. Robot. Intell. Syst.* **14**(1), 109–120 (2020)

Optimization

Artificial Fish Swarm Algorithm for the Optimization of a Benchmark Set of Functions



Cinthia Peraza, Patricia Ochoa, Leticia Amador, and Oscar Castillo

Abstract An efficient artificial fish swarm algorithm (AFSA) for the optimization of a benchmark set of functions is developed in this paper. An analysis of the principal parameters to effect on exploration or exploitation is presented. In this case, two important parameters are changed manually with the main idea to analyze the impact that S and V presents on the development of the AFSA algorithm. The performance and efficiency of the AFSA demonstrated that it is a good algorithm to solve benchmark sets of functions. The AFSA has proven to be successful in the implementation of various benchmarking optimization problems. A comparative with other metaheuristics is presented.

Keywords Artificial fish swarm · Benchmark set of functions · Exploitation · Exploration

1 Introduction

Currently meta-heuristic algorithms are an important field to solve various optimization problems. An interesting way to check the efficiency in the performance of some meta-heuristics is using mathematical functions, and some bio-inspired algorithm that has been implemented this benchmark functions are; in [1] a galactic swarm optimization (GSO) is used for the optimization of mathematical functions presented by Bernal E. et al. in [2] an algorithm based on the behavior of bees (BCO) is used in the benchmark functions presented by Castillo O. et al., in [16] a inspired grey wolf optimizer (GWO) is implemented to solve large-scale function optimization problems presented by Long W. et al., in [18] the algorithm called differential evolution (DE) is used in benchmark functions presented by Ochoa P. et al., in [19] an interesting algorithm based on the music called harmony search (HS) is used for this case study by Peraza, C. et al., and in [21] an improving particle swarm optimization (PSO) is implemented for the mathematical functions presented by Valdez F. et al.

C. Peraza (✉) · P. Ochoa · L. Amador · O. Castillo
Division of Graduate Studies, Tijuana Institute of Technology, Tijuana, México
e-mail: cinthia.peraza@tectijuana.edu.mx

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2022
O. Castillo and P. Melin (eds.), *New Perspectives on Hybrid Intelligent System Design based on Fuzzy Logic, Neural Networks and Metaheuristics*, Studies in Computational Intelligence 1050, https://doi.org/10.1007/978-3-031-08266-5_6

The meta-heuristics algorithms have parameters that defined the part of the heuristic, these parameters allow to measure the efficiency of the exploration and the exploitation in the algorithms, for example; in [2] BCO has alpha and beta parameters, in [18] DE has F and Cr parameters, and in [19] HS has PARate and HMR parameters. In the case of the AFS has S and V parameters. Several authors are interesting in to find the optimal values in this parameters based on different strategies, for mention some; in [4] Chen, G.Z. et al. presents an improved AFSA, in [5] Gao S., et al. analyze some aspects of the behavior fish, and in [25] Zhang, T. et al. presents a proposed to find the optimal values for the S and V applied to planning path of robots.

In this paper, some important contribution is; the first consists in demonstrate that AFSA is an efficiency tool in the optimization of benchmark set of functions, and the second, consists in the exploration of the AFSA, especially in to observe the S and V parameters, the idea is to analyze the way in which these two parameters influence to find the best solutions for case study. An important analysis is presented in the performance of the AFSA based on experimentations.

The following sections describes the organization in this paper: Sect. 2 shows some relevant related works about this research. Section 3 describes the artificial fish swarm optimization. Section 4 describes the benchmark set of functions used. Section 5 presents in detail of the analysis of the S and V parameters on the behavior in this algorithm. Various simulation results are presented in Sect. 6. Section 7 shows a comparative result with other meta-heuristics. Finally, some conclusions and future works are mentioned in the Sect. 8.

2 Related Works

Actually, the bio-inspired algorithm based on behavior of some animals in nature is an important tool to solve problems in the field of computing. In this paper, the interest is focused on the behavior of the fish to find food. Several researches has analyzed and implemented the AFSA in different field of the intelligent computing such as; in fuzzy control [11, 17, 28], in clustering [7, 12], in medicine [15], in smart city [8], and in others important areas [3, 4, 6, 7, 13–15, 20, 26, 27].

In the state of art, several researchers are interested in metaheuristics algorithms to solve problems in the field of computing. A large list of authors is interested in this algorithm to solve various problems, for mentioning some; in [9] a misalignment fault prediction of wind turbines based on improved AFSA is presented, in [10] a layout optimization of fiber bragg grating strain sensor network based on modified AFSA is presented, in [20] a review of the family of AFSA: recent advances and applications is presented, in [22] a Parameters analysis of AFSA is presented, in [23] a modified AFSA is presented, in [24] a new AFSA for dynamic optimization problems is presented, in [26] an improved AFSA is presented, and in [28] an improved AFSA for neighborhood rough set reduction and its application is presented.

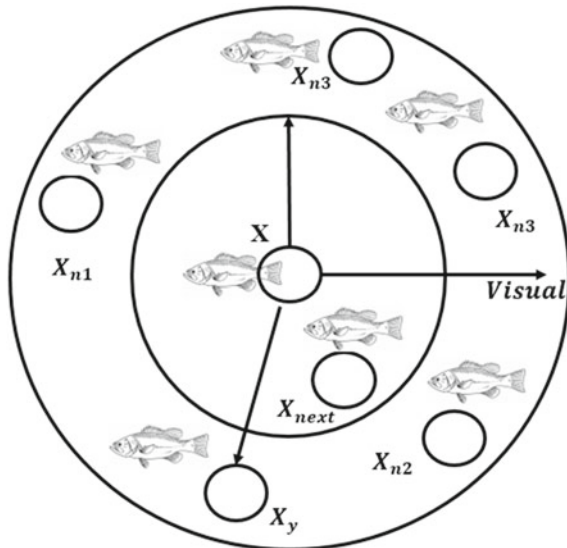
3 Artificial Fish Swarm Algorithm

The main idea that Li proposed in 2002 with the inspiration in the analysis was to simulate a number of ecological behaviors of schooling in the water. This idea allows the creation of the AFSA which was proposed by Yazdani, D. et al. in 2012 [24]. In AFSA the behavior is praying or foraging, where every in nature probes for its prey individually within its visual distance. Figure 1 illustrates the idea in each movement that a fish performs in the algorithm. X represents the actual position of an artificial fish (AF). Step (S) indicates the maximum step-length that a fish can take at each movement, Visual (V) is the visual distance. Each AF inspects the search space around its vision distance. In this case, the best solution in each iteration is changed according to the following condition, the criterion in the realization of a move in a fish is if a new position is better than its current location, the movement is given.

A random distribution called n stars in AFSA for each generation in the search space. For behaviors that can to present a fish in this algorithm, the position on the search space that has major concentration of these behaviors is considered an excellent result in this algorithm; the four behaviors are called; *preying*, *swarming*, *following*, and *random* behavior. In AFSA the dynamism is based with the following equations; Eq. (1) represents the preying behavior, in this algorithm, the fish try to move to locations with highest food in the search space. Let $X_i^{(t)}$ be the current position of the i th AF, and X_j be a state of an AF randomly selected within the visual distance of the i th AF as follows:

$$X_j = X_i^{(t)} + V \times rand() \quad (1)$$

Fig. 1 Graphical representation of the movements for an artificial fish



where V is the visual distance of the AF, $d_{i,j} < V$ and $rand()$ is a random vector with each element between 0 and 1. If $If Y_i < Y_j$, $X_i^{(t)}$ moves a step toward X_j , the Eq. (2) shows this behavior.

$$X_i^{(t+1)} = X_i^{(t)} + S \times rand() \cdot \frac{X_j - X_i^{(t)}}{\|X_j - X_i^{(t)}\|} \quad (2)$$

where $\cdot$ represents multiplication of the corresponding elements in two vectors. In AFSA if the forward condition is not satisfied, a new state X_j is randomly selected using Eq. (1). If the criterion is not satisfied after a number determined of time. $X_i^{(t)}$ moves a step randomly, the Eq. (3) indicates this behavior.

$$X_i^{(t+1)} = X_i^{(t)} + V \cdot rand() \quad (3)$$

The second behavior is called swarming, which represents in nature a defense mechanism form its predators. Let n be a total number of AF while $d_{i,j} < V$, and $X_c = (x_{c1}, x_{c2}, \dots, x_{cd})$ be the central position, where x_{cd} is the d th dimension of X_c , the Eq. (4) described this behavior.

$$x_{cd} = \frac{\left(\sum_{j=1}^{n_f} x_{jd} \right)}{n_f} \quad (4)$$

If the conditions in Eq. (5) are satisfied, i.e.;

$$\left\{ Y_c > Y_i \frac{n_f}{n} > \delta \right. \quad (5)$$

where δ is the crowding factor, this means there is more food in the center, and the area is not overcrowded. As such, $X_i^{(t)}$ moves a step toward the companion center using Eq. (6)

$$X_i^{(t+1)} = X_i^{(t)} + S \times rand() \cdot \frac{X_c - X_i^{(t)}}{\|X_c - X_i^{(t)}\|} \quad (6)$$

However, if the conditions in Eq. (5) are not satisfied, the preying behavior is executed. If $n_f = 0$ this means there is no companion within the visual distance of the i th AF, and the preying behavior is executed.

The third behavior called following is executed when a fish in a location with a better concentration of food, is an observation by other fishes follow. Let $X_i^{(t)}$ be the current position in the i th AF contains more food, i.e., $Y_j - Y_i$, and it is not overcrowded, i.e., $n_f/n < \delta$, $X_i^{(t)}$ moves a step toward X_j , and the Eq. (2) is executed.

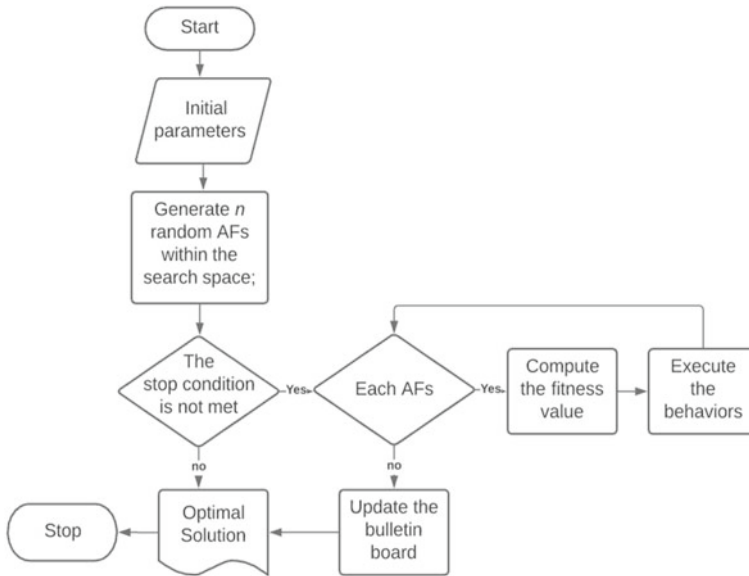


Fig. 2 Flowchart of the procedure in the AFSA

In others words, the preying behavior is executed. Similar to the swarming behavior, the preying is executed if $n_f = 0$.

The final behavior called random, in nature represents a swarm of fishes moving freely to find food. This behavior enables an AF to search for food, or to follow a swarm in a larger space. The AF selects a random behavior when none of the criteria pertaining to the preying, swarming, and following behaviors are satisfied. Equation (7) represents this final behavior:

$$X_i^{(t+1)} = X_i^{(t)} + V \times rand() \quad (7)$$

Figure 2 illustrates the general description of the step in AFSA.

4 Benchmark Sets of Functions

The study case that in this paper is implemented is benchmark sets of functions, this section describes a total of six classical benchmark functions with the main objective to validate to AFSA, such as; Ackley, Giewank, Quarticnoise, Rastrigin, Rosenbrock, Spherical; all functions were evaluated with 10, 30 and 50 dimensions (artificial fish). Figure 3 illustrates the plot, and Table 1 shows the equation for each study case.

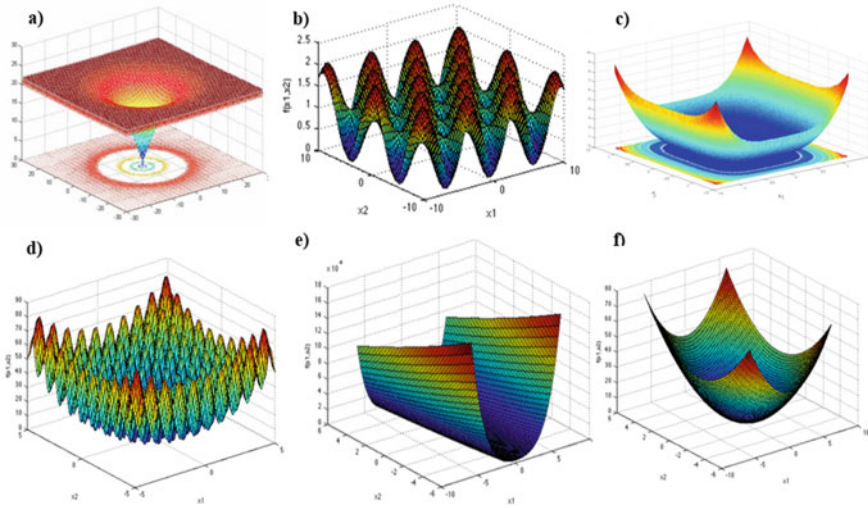


Fig. 3 Plot of the six benchmark mathematical functions; **a** Ackley, **b** Griewank, **c** Quarticnoise, **d** Rastrigin, **e** Rosenbrock, and **f** Sphere

Table 1 Benchmark sets of functions

Number	Name function	Mathematical representation
F1	Ackley	$f(x) = -20e^{-0.2\sqrt{\frac{1}{n_x} \sum_{j=1}^{n_x} x_j^2} - \frac{1}{e^{n_x}} \sum_{j=1}^{n_x} \cos(2\pi x_j)} + 20 + e$
F2	Griewank	$f(x) = \sum_{i=1}^d \frac{x_i^2}{4000} - \prod_{i=1}^d \cos\left(\frac{x_i}{\sqrt{i}}\right) + 1$
F3	Quarticnoise	$f(x) = \sum_{i=1}^d i x_i^4 + \text{rand}[0, 1]$
F4	Rastrigin	$f(x) = 10d + \sum_{i=1}^d [x_i^2 - 10\cos(2\pi x_i)]$
F5	Rosenbrock	$f(x) = \sum_{i=1}^{d-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2]$
F6	Sphere	$f(x) = \sum_{i=1}^d x_i^2$

5 Experimental Results

Several experiments were executed, for each benchmark mathematical function, a total of 5000 generations, the size of the vector solution (artificial fish) is 30. The dimensions that were executed are; 10, 30 and 50. When V is fixed by 0.5 S was varying from 0.1, 0.3, 0.5, 0.7 and 0.9. Four metrics are used for each experimentation; Table 2 represents the results for 10 dimensions.

Table 2 shows that better results are found with S is in the range of [0.7–0.9]. The italic term represents the better average and black color indicates the best results for each mathematical function.

Table 3 shows to change in the V value with 0.1, 0.3, 0.5, 0.7 and 0.9 values and

Table 2 Benchmark sets of functions varying the “S” value for 10 dimensions

Function	Performance index	Varying S				
		0.1	0.3	0.5	0.7	0.9
F1	Average	2.04E+01	2.00E+01	2.01E+01	1.97E+01	<i>1.97E+01</i>
	Standard deviation (σ)	1.24E-01	1.84E-01	4.02E-01	7.86E-02	3.56E-01
	Best	2.01E+01	1.98E+01	1.91E+01	1.96E+01	1.80E+01
	Worst	2.06E+01	2.03E+01	2.10E+01	1.99E+01	1.99E+01
F2	Average	3.57E+02	3.56E+02	2.03E+02	9.96E-01	5.12E-01
	Standard deviation (σ)	1.81E+01	1.06E+01	2.02E+01	4.83E-01	<i>2.32E-01</i>
	Best	3.12E+02	3.38E+02	1.52E+02	1.75E-01	7.64E-02
	Worst	3.93E+02	3.72E+02	2.26E+02	1.85E+00	9.06E-01
F3	Average	2.69E+01	2.59E+01	1.84E+01	3.75E+00	3.24E+00
	Standard deviation (σ)	2.48E+00	1.55E+00	4.12E+00	<i>6.96E-01</i>	9.98E-01
	Best	2.09E+01	2.24E+01	7.83E+00	2.83E+00	1.87E+00
	Worst	3.20E+01	2.85E+01	2.27E+01	4.91E+00	5.76E+00
F4	Average	1.47E+02	1.45E+02	1.23E+02	1.18E+02	<i>7.16E+01</i>
	Standard deviation (σ)	1.27E+01	6.49E+00	1.27E+01	1.99E+01	2.88E+01
	Best	1.08E+02	1.38E+02	9.09E+01	7.86E+01	3.58E+01
	Worst	1.70E+02	1.61E+02	1.35E+02	1.49E+02	1.26E+02
F5	Average	7.29E+06	7.21E+06	6.27E+06	2.72E+02	<i>6.29E+01</i>
	Standard deviation (σ)	2.52E+03	1.49E+04	9.77E+04	3.17E+02	8.86E+01
	Best	7.28E+06	7.18E+06	6.14E+06	1.58E+01	7.28E+00
	Worst	7.29E+06	7.22E+06	6.39E+06	1.12E+03	3.89E+02
F6	Average	2.62E+02	2.57E+02	2.25E+02	3.03E-03	<i>9.98E-04</i>
	Standard deviation (σ)	2.69E-01	1.92E+00	2.66E+00	5.54E-03	9.28E-04
	Best	2.61E+02	2.53E+02	2.23E+02	9.76E-04	1.58E-05
	Worst	2.62E+02	2.58E+02	2.30E+02	2.32E-02	5.50E-03

S is fixed in 0.5. The metrics are; the best, worst, average and standard deviation.

Table 3 shows that better results are found with V is with the value of 0.9. The italic term represents the better average and black color indicates the best results for each mathematical function.

The simulations result with 30 dimensions, with S and varying V is presented in Tables 4 and 5.

Table 5 indicates the values to find by AFSA with 30 dimensions varying the V parameter.

Tables 4 and 5 show that better results are found with the value in the parameter of the S of 0.9.

Tables 6 and 7 show the simulation results with 50 dimensions, with S and V varying, respectively.

Table 3 Benchmark sets of functions varying the “ V ” value for 10 dimensions

Function	Performance index	Varying V				
		0.1	0.3	0.5	0.7	0.9
F1	Average	2.03E+01	2.01E+01	2.01E+01	1.97E+01	<i>1.86E+01</i>
	Standard deviation (σ)	1.07E-01	1.77E-01	1.87E-01	8.87E-02	3.68E+00
	Best	2.01E+01	1.98E+01	1.98E+01	1.95E+01	3.23E+00
	Worst	2.05E+01	2.04E+01	2.05E+01	1.99E+01	1.99E+01
F2	Average	3.62E+02	3.51E+02	2.57E+02	1.43E+00	<i>7.83E-01</i>
	Standard deviation (σ)	1.79E+01	1.61E+01	3.01E+01	7.99E-01	5.38E-01
	Best	3.24E+02	3.07E+02	2.01E+02	4.11E-01	1.62E-01
	Worst	3.93E+02	3.81E+02	3.21E+02	3.43E+00	2.13E+00
F3	Average	2.65E+01	2.65E+01	1.79E+01	3.53E+00	<i>3.18E+00</i>
	Standard deviation (σ)	2.09E+00	2.65E+00	2.25E+00	7.01E-01	1.01E+00
	Best	2.18E+01	1.92E+01	1.29E+01	2.00E+00	1.97E+00
	Worst	3.04E+01	3.21E01	2.31E+01	4.76E+00	6.90E+00
F4	Average	1.45E+02	1.48E+02	1.38E+02	1.01E+02	<i>8.21E+01</i>
	Standard deviation (σ)	1.17E+01	1.01E+01	1.26E+01	3.09E+01	3.12E+01
	Best	1.24E+02	1.29E+02	1.14E02	1.62E+01	3.38E+01
	Worst	1.68E+02	1.72E+02	1.67E+02	1.44E+02	1.47E+02
F5	Average	7.29E+06	7.22E+06	6.39E+06	2.17E+02	<i>4.81E+01</i>
	Standard deviation (σ)	2.07E+03	6.19E+03	7.34E+04	1.96E+02	5.94E+01
	Best	7.28E+06	7.20E+06	6.22E06	5.85E+00	2.12E-01
	Worst	7.29E+06	7.23E+06	6.52E+06	8.85E+02	2.50E+02
F6	Average	2.62E+02	2.58E+02	2.24E+02	3.15E-03	<i>8.75E-04</i>
	Standard deviation (σ)	1.78E-01	1.12E+00	3.70E+00	5.30E-03	4.83E-04
	Best	2.61E+02	2.54E+02	2.14E+02	8.57E-05	2.32E-06
	Worst	2.62E+02	2.58E+02	2.31E02	2.70E-02	2.70E-03

When the dimensions are increased to 50, the best results are with values of 0.7 and 0.9 of the S parameter.

Table 7 shows that when V has a value of 0.9 best results are found by AFSA.

6 Analysis of the Parameters in AFSA

This paper presents an analysis of the performance of the AFSA varying two important parameters which are S and V values. Based on simulation, both parameters were changed with the 0.1, 0.3, 0.5, 0.7 and 0.9 values individually, for example, be fixed S to 0.5 and changed V and the experiments, and be fixed V to 0.5 and changed

Table 4 Benchmark sets of functions varying the “ S ” value for 30 dimensions

Function	Performance index	Varying S				
		0.1	0.3	0.5	0.7	0.9
F1	Average	2.09E+01	2.09E+01	2.07E+01	1.99E+01	1.97E+01
	Standard deviation (σ)	4.02E-02	1.01E-01	2.15E-01	1.28E-01	9.73E-02
	Best	2.08E+01	2.07E+01	2.01E+01	1.97E+01	1.96E+01
	Worst	2.10E+01	2.10E+01	2.10E+01	2.03E+01	1.99E+01
F2	Average	1.26E+03	1.27E+03	1.09E+03	9.98E+01	4.92E+01
	Standard deviation (σ)	3.75E+01	2.91E+01	7.48E+01	4.37E+01	2.59E+01
	Best	1.21E+03	1.19E+03	8.98E+02	5.18E+01	6.17E+00
	Worst	1.34E+03	1.30E+03	1.22E+03	2.40E+02	9.39E+01
F3	Average	2.92E+02	2.78E+02	2.35E+02	3.51E+01	4.92E+01
	Standard deviation (σ)	1.51E+01	1.40E+01	2.76E+01	1.30E+01	2.59E+01
	Best	2.35E+02	2.55E+02	1.80E+02	2.30E+01	1.11E+01
	Worst	3.17E+02	3.10E+02	3.05E+02	8.04E+01	3.62E+01
F4	Average	5.67E+02	5.61E+02	5.42E+02	4.06E+02	2.74E+02
	Standard deviation (σ)	5.22E+01	1.61E+01	2.94E+01	4.03E+01	4.63E+01
	Best	2.99E+02	5.32E+02	4.59E+02	3.29E+02	2.02E+02
	Worst	6.04E+02	5.96E+02	5.86E+02	4.66E+02	3.77E+02
F5	Average	2.35E+07	2.32E+07	2.06E+07	7.44E+03	3.61E+02
	Standard deviation (σ)	9.27E+03	4.28E+04	3.59E+03	5.48E+03	2.97E+02
	Best	2.35E+07	2.31E+07	1.99E+07	1.04E+03	5.94E+01
	Worst	2.35E+07	2.33E+07	2.15E+07	2.56E+04	1.19E+03
F6	Average	7.84E+02	7.69E+02	6.69E+02	1.25E-02	1.44E-03
	Standard deviation (σ)	9.81E-01	4.83E+00	7.95E+00	2.19E-02	7.88E-04
	Best	7.83E+02	7.60E+02	6.48E+02	3.35E-04	8.38E-06
	Worst	7.86E+02	7.75E+02	6.86E+02	8.13E-02	3.90E-03

S varying the values before mentioned. Figure 4 illustrates the idea general in this paper.

A general analysis in both parameters is presented based on experimentation. In this paper an important contribution consists and identifies what parameters affect the exploitation and exploration in the behavior of the AFSA.

Tables 8 and 9 show an analysis for the Sphere function with the values obtained by AFSA for S and V parameters.

Based on the experiments, V parameter in AFSA represents the exploitation when V is greater than 0.9 better results are found in the benchmark mathematical functions, for example; the function of **Sphere** (F6), with V value of 0.5 and S value of 0.9 of best found by AFSA is of **1.58E-05**, and with S value of 0.5 and V value of **2.32E-06**. Similarly, for S parameters, better results are found when this value is in the range

Table 5 Benchmark sets of functions varying the “V” value for 30 dimensions

Function	Performance Index	Varying V				
		0.1	0.3	0.5	0.7	0.9
F1	Average	2.09E+01	2.07E+01	2.07E+01	1.99E+01	<i>1.98E+01</i>
	Standard deviation (σ)	5.71E-02	1.30E-01	1.52E+01	9.79E-02	8.44E-02
	Best	2.08E+01	2.04E+01	2.04E+01	1.98E+01	1.95E+01
	Worst	2.10E+01	2.10E+01	2.10E+01	2.02E+01	1.99E+01
F2	Average	1.27E+03	1.25E+03	1.03E+03	1.10E+02	<i>5.92E+01</i>
	Standard deviation (σ)	3.63E+01	3.02E+01	1.98E+02	4.29E+01	2.42E+01
	Best	1.20E+03	1.16E+03	2.58E+01	4.90E+01	2.71E+01
	Worst	1.33E+03	1.31E+03	1.17E+03	2.42E+02	1.36E+02
F3	Average	2.86E+02	2.82E+02	2.26E+02	3.70E+02	<i>5.92E+01</i>
	Standard deviation (σ)	1.68E+01	1.11E+01	2.50E+01	5.10E+01	2.42E+01
	Best	2.57E+02	2.58E+02	1.78E+02	2.53E+02	1.72E+01
	Worst	3.20E+02	3.08E+02	2.77E+02	4.64E+02	1.36E+02
F4	Average	5.61E+02	5.63E+02	5.50E+02	3.70E+02	<i>3.05E+02</i>
	Standard deviation (σ)	2.59E+01	2.19E+01	2.61E+01	5.01E+01	5.96E+01
	Best	5.10E+02	4.92E+02	4.90E+02	2.53E+02	2.00E+02
	Worst	6.04E+02	5.98E+02	6.03E+02	4.64E+02	4.26E+02
F5	Average	2.35E+07	2.32E+07	2.06E+07	8.32E+03	<i>4.45E+02</i>
	Standard deviation (σ)	8.97E+03	4.79E+04	2.72E+05	6.54E+03	2.48E+02
	Best	2.35E+07	2.31E+07	2.02E+07	1.45E+03	7.42E+01
	Worst	2.35E+07	2.33E+07	2.12E+07	2.76E+04	1.14E+03
F6	Average	7.84E+02	7.68E+02	6.68E+02	4.47E-03	<i>1.19E-03</i>
	Standard deviation (σ)	1.09E+00	7.15E+00	1.03E+01	5.44E-03	1.20E-03
	Best	7.82E+02	7.51E+02	6.49E+02	4.20E-04	1.48E-07
	Worst	7.86E+02	7.75E+02	6.93E+02	2.14E-02	6.30E-03

of [0.7 to 0.9], therefore this parameter is considered to aid in the exploration, in this case, the main analysis consists in identifying the optimal values for each parameter analyzed of S and V .

As part of the analysis in this article, the following can be provided; with respect to the variation of S ; when 10 dimensions are used, all the functions present better results (see Table 2). For example, for the sphere function, good results are obtained in 10, 30 and 50 dimensions, when increasing the size of the dimensions the results are higher in most of the functions, except that of sphere.

Regarding the analysis of the behavior of the values varying V is the following; for few dimensions, better results are obtained when V has a high value, example 0.7 and 0.9, however for dimensions of 30 and 50 the results are not very competitive compared to 10 dimensions, then the variation of V in 30 dimensions is better using

Table 6 Benchmark sets of functions varying the “ S ” value for 50 dimensions

Function	Performance index	Varying S				
		0.1	0.3	0.5	0.7	0.9
F1	Average	2.10E+01	9.27E+00	1.50E+01	7.87E+00	1.53E+01
	Standard deviation (σ)	5.03E-02	4.54E+00	4.65E-01	3.85E+00	1.50E+00
	Best	2.09E+01	1.31E+00	1.40E+01	1.53E+00	1.15E+01
	Worst	2.11E+01	2.09E+01	1.57E+01	1.46E+01	1.76E+01
F2	Average	9.27E+00	2.19E+03	1.70E+03	1.00E+03	4.12E+02
	Standard deviation (σ)	4.54E+00	2.87E+01	1.53E+02	4.97E+00	1.50E+02
	Best	1.31E+01	2.13E+03	1.44E+03	2.93E+02	1.33E+02
	Worst	2.09E+01	2.24E+03	1.98E+03	1.71E+03	6.56E+02
F3	Average	8.17E+02	8.17E+02	6.85E+02	1.71E+02	1.37E+02
	Standard deviation (σ)	2.18E+01	2.18E+01	6.36E+01	4.97E+00	2.91E+01
	Best	7.77E+02	7.77E+02	5.78E+02	1.64E+02	6.32E+01
	Worst	8.59E+02	5.89E+02	8.21E+02	1.94E+02	1.82E+02
F4	Average	9.89E+02	9.89E+02	1.07E+03	7.36E+02	6.01E+02
	Standard deviation (σ)	1.28E+01	1.28E+01	6.24E+01	8.39E+01	2.30E+01
	Best	9.47E+02	9.47E+02	9.44E+02	6.00E+02	5.47E+02
	Worst	1.01E+03	1.03E+03	1.17E+03	8.97E+02	7.05E+02
F5	Average	3.94E+07	3.94E+07	3.40E+07	1.59E+04	5.57E+03
	Standard deviation (σ)	3.92E+04	3.92E+04	6.00E+05	1.05E+04	2.62E+03
	Best	3.93E+07	3.93E+07	3.30E+07	1.01E+03	1.06E+03
	Worst	3.94E+07	3.94E+07	3.51E+07	4.74E+04	9.83E+03
F6	Average	1.27E+03	1.27E+03	1.10E+03	1.06E-01	5.16E-02
	Standard deviation (σ)	4.29E+00	4.29E+00	1.69E+01	6.03E-02	3.64E-02
	Best	1.27E+03	1.27E+03	1.07E+03	8.55E-04	1.92E-04
	Worst	1.29E+03	1.29E+03	1.13E+03	2.03E-01	1.14E-01

a V of 0.9 than a V of 0.1, the same happens with 50 dimensions, the best results obtained are with a V of 0.9, which means that the higher the V value, the better results obtained.

7 Comparison with Others Metaheuristics

Based on experimentation and analysis of the behavior of the AFSA, the best values are the values in S and V parameters are 0.5 and 0.9, respectively. A total of three metaheuristics are analyzed for three mathematical functions. Table 10 shows a comparison with other important metaheuristics, specifically, for the Rastrigin (F4) function

Table 7 Benchmark sets of functions varying the “V” value for 50 dimensions

Function	Performance index	Varying V				
		0.1	0.3	0.5	0.7	0.9
F1	Average	2.10E+01	2.09E+01	2.09E+01	2.02E+01	1.98E+01
	Standard deviation (σ)	2.22E-01	8.54E-02	1.27E-01	1.30E-01	9.03E-02
	Best	2.01E+01	2.08E+01	2.05E+01	1.99E+01	1.96E+01
	Worst	2.11E+01	2.11E+01	2.11E+01	2.04E+01	2.02E+01
F2	Average	2.04E+03	2.20E+03	1.90E+03	3.90E+02	2.35E+02
	Standard deviation (σ)	5.43E+02	3.62E+01	3.18E+02	6.39E+01	5.01E+01
	Best	3.96E+02	2.12E+03	2.67E+02	2.75E+02	1.50E+02
	Worst	2.28E+03	2.31E+03	2.13E+03	4.97E+02	3.37E+02
F3	Average	8.59E+02	8.39E+02	7.36E+02	1.50E+02	9.24E+01
	Standard deviation (σ)	3.43E+02	2.81E+01	5.73E+01	2.83E+01	2.62E+01
	Best	7.71E+02	7.81E+02	5.78E+02	1.00E+02	4.97E+01
	Worst	9.06E+02	8.97E+02	8.32E+02	2.24E+02	1.55E+02
F4	Average	1.01E+03	1.00E+03	9.83E+02	7.19E+02	5.74E+02
	Standard deviation (σ)	2.79E+01	3.35E+01	3.77E+01	6.04E+01	6.47E+01
	Best	9.55E+02	9.25E+02	8.87E+02	6.00E+02	4.72E+02
	Worst	1.06E+03	1.06E+03	1.04E+03	8.28E+02	7.77E+02
F5	Average	3.97E+07	3.92E+07	3.48E+07	1.77E+04	2.06E+03
	Standard deviation (σ)	2.10E+04	7.89E+04	4.97E+05	1.09E+04	1.32E+03
	Best	3.96E+07	3.91E+07	3.41E+07	5.06E+03	3.78E+02
	Worst	3.97E+07	3.93E+07	3.58E+07	5.87E+04	5.14E+03
F6	Average	1.31E+03	1.28E+03	1.09E+03	1.83E-02	3.90E-03
	Standard deviation (σ)	2.18E+00	1.32E+01	1.82E+02	2.49E-02	6.70E-03
	Best	1.30E+03	1.25E+03	1.32E+02	4.07E-04	9.00E-05
	Worst	1.31E+03	1.29E+03	1.15E+03	1.26E-01	3.46E-02

with the variation of the parameter to V, this comparison is with 30 dimensions for two metaheuristics, such as; Bee Colony Optimization (BCO) [2] and Harmony search algorithm (HS) [19].

Table 10 shows better results with HS compare to AFSA respect to average, this comparative demonstrates that AFSA has an error close to HS.

Table 11 shows a comparison with the metaheuristic called Differential Evolution (DE) [18] specifically for the Ackley functions with 30 dimensions compare to the S value of 0.5 and V of 0.9.

Analyzing the results in Table 11, the best result (minimal value) is better with AFSA with a value of **1.95E+01** compare to DE that the value is of **3.21E+02**.

Table 12 shows a comparison with the metaheuristic of DE [18] specifically for

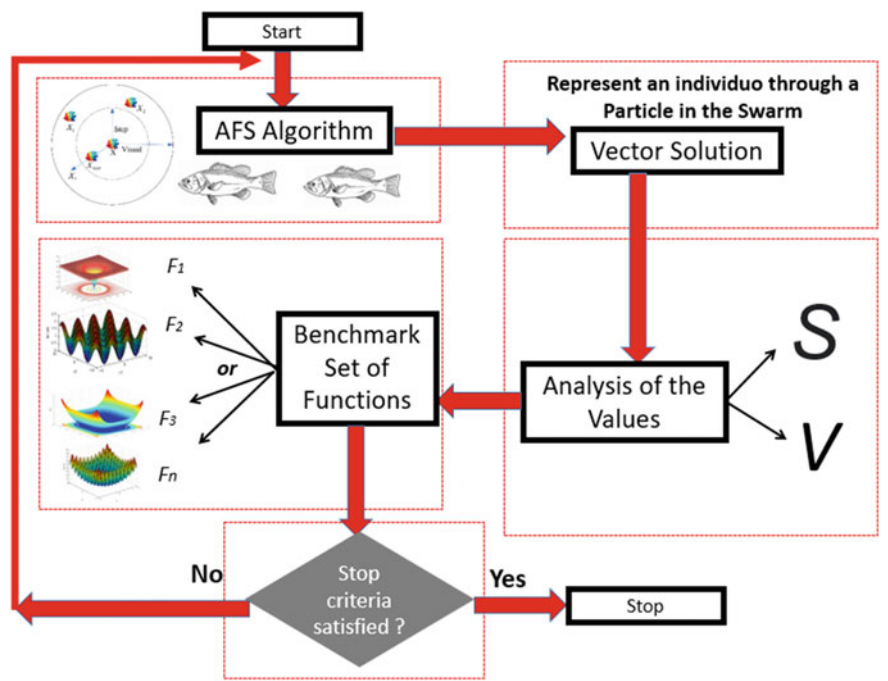


Fig. 4 Flowchart of the idea general in the proposal of this paper

Table 8 Results for the shpere function with varyng in “S” value

Function	Dimensions					
	10		30		50	
	0.7	0.9	0.7	0.9	0.7	0.9
Sphere (F6)	3.03E-03	9.98E-04	1.25E-02	1.44E-03	1.06E-01	5.16E-02

Table 9 Results for the shpere function with varyng in “V” value

Function	Dimensions					
	10		30		50	
	0.7	0.9	0.7	0.9	0.7	0.9
Sphere (F6)	3.15E-03	8.75E-04	4.47E-03	1.19E-03	1.83E-02	9.00E-05

Table 10 Comparative results with meta-heuristics algorithms for Griewank function

Meta-Heuristics algorithms	Performance index			
	Average	Standard deviation (σ)	Best	Worst
Proposed method (AFSA)	3.05E+02	5.96E+01	2.00E+02	4.26E+02
BCO [2]	5.23E-03	N/A	1.14E-04	4.96E-02
HS [19]	5.56E+01	N/A	N/A	N/A

Table 11 Comparative results with meta-heuristics algorithms for Ackley function

Meta-Heuristics algorithms	Performance index			
	Average	Standard deviation (σ)	Best	Worst
Proposed method (AFSA)	1.98E+01	8.44E-02	1.95E+01	1.99E+01
DE [18]	3.21E+02	3.21E+02	3.21E+02	3.21E+02

Table 12 Comparative results with meta-heuristics algorithms for Rosenbrock function

Meta-Heuristics algorithms	Performance index			
	Average	Standard deviation (σ)	Best	Worst
Proposed method (AFSA)	4.45E+02	2.48E-02	7.42E+01	1.14E+03
DE [18]	1.06E+03	4.21E+01	9.31E+02	1.13E+03

the Rosenbrock functions with 30 dimensions compare to the S value of 0.5 and V of 0.9.

Finally, Table 12 respect to Rosenbrock the best algorithm is AFSA with an average of **4.45E+02** compare to DE that has a value of **4.06E+03**, other analysis is the standard deviation, with AFSA the value is of **2.48E+02** and for DE the value is **4.21E+01**. This metrics and analysis allow to demonstrated the efficiency that this proposed method of the variation of the values in S and V parameters.

8 Conclusions

An analyzed of the behavior in S and V parameters is presented in this paper, the contributions most important with which it can be concluded are the following; when lower dimensions for example 10, the AFSA with the variation of S value presents best results. All mathematical functions have the best results when the number of dimensions increasing to 50 and when the V value is of 0.9 (see Tables 6 and 7), thus based on experimentation, an important conclusion is that V parameter represents the exploitation in the algorithm and S indicates the exploration, because the larger V (0.9) the search area is limited to finding local minima in the algorithm, as well as the larger the S means that the algorithm is performing the exploitation in the search space.

As a future works, the proposed analysis in this paper is a principal idea to realize a dynamic adjustment with the values of S and V parameters using a fuzzy logic system, and explore other study cases such as: fuzzy controllers and fuzzy recognition systems.

References

1. Bernal, E., Castillo, O., Soria, J., Valdez, F.: Galactic swarm optimization with adaptation of parameters using fuzzy logic for the optimization of mathematical functions. In: *Fuzzy Logic Augmentation of Neural and Optimization Algorithms: Theoretical Aspects and Real Applications*, pp. 131–140. Springer, Cham (2018)
2. Castillo, O., Amador-Angulo, L.: A generalized type-2 fuzzy logic approach for dynamic parameter adaptation in bee colony optimization applied to fuzzy controller design. *Inf. Sci.* **460**, 476–496 (2018)
3. Chen, Y., Zhu, Q., Xu, H.: Finding rough set reducts with fish swarm algorithm. *Knowl. Based Syst.* **81**, 22–29 (2015)
4. Chen, G.Z., Wang, J.Q., Li, C.J., Lu, X.Y.: An improved artificial fish swarm algorithm and its applications. *Syst. Eng.* **27**(12), 105–110 (2009)
5. Gao, S., Wen, Y.: An improved artificial fish swarm algorithm and its application. In: *2018 IEEE/ACIS 17th International Conference on Computer and Information Science (ICIS)*, pp. 649–652. IEEE (2018)
6. He, Q., Hu, X., Ren, H., Zhang, H.: A novel artificial fish swarm algorithm for solving large-scale reliability–redundancy application problem. *ISA Trans.* **59**, 105–113 (2015)
7. He, S., Belacel, N., Hamam, H., Bouslimani, Y.: Fuzzy clustering with improved artificial fish swarm algorithm. In: *2009 International Joint Conference on Computational Sciences and Optimization*, vol. 2, pp. 317–321. IEEE (2009)
8. Huang, X., Xu, G., Xiao, F.: Optimization of a novel Urban growth simulation model integrating an artificial fish swarm algorithm and cellular automata for a smart city. *Sustainability* **13**(4), 2338 (2021)
9. Hua, Z., Xiao, Y., Cao, J.: Misalignment fault prediction of wind turbines based on improved artificial fish swarm algorithm. *Entropy* **23**(6), 692 (2021)
10. Huang, J., Zeng, J., Bai, Y., Cheng, Z., Feng, Z., Qi, L., Liang, D.: Layout optimization of fiber Bragg grating strain sensor network based on modified artificial fish swarm algorithm. *Opt. Fiber Technol.* **65**, 102583 (2021)
11. Jibril, Y., Salawudeen, A.T., Salawu, A., Zainab, M.: An optimized PID controller for deep space antenna DC motor position control using modified artificial fish swarm algorithm. *Yanbu Journal of Engineering and Science* **13**(1), 45–54 (2021)
12. Krishnaraj, N., Jayasankar, T., Kousik, N.V., Daniel, A.: 2 Artificial Fish Swarm Optimization Algorithm with Hill Climbing Based Clustering Technique for Throughput Maximization in Wireless Multimedia Sensor Network (2021)
13. Li, T., Yang, F., Zhang, D., Zhai, L.: Computation scheduling of multi-access edge networks based on the artificial fish swarm algorithm. *IEEE Access* **9**, 74674–74683 (2021)
14. Lin, M., Hong, H., Yuan, X., Fan, J., Ji, Z.: Inverse kinematic analysis of bionic hands based on fish swarm algorithm. *J. Phys. Conf. Ser.* **1965**(1), 012006 (2021). (IOP Publishing)
15. Liu, Q., Odaka, T., Kuroiwa, J., Ogura, H.: Application of an artificial fish swarm algorithm in symbolic regression. *IEICE Trans. Inf. Syst.* **96**(4), 872–885 (2013)
16. Long, W., Jiao, J., Liang, X., Tang, M.: Inspired grey wolf optimizer for solving large-scale function optimization problems. *Appl. Math. Model.* **60**, 112–126 (2018)
17. Luo, Y., Zhang, J., Li, X.: The optimization of PID controller parameters based on artificial fish swarm algorithm. In: *2007 IEEE International Conference on Automation and Logistics*, 2007, pp. 1058–1062. IEEE

18. Ochoa, P., Castillo, O., Soria, J.: The differential evolution algorithm with a fuzzy logic approach for dynamic parameter adjustment using benchmark functions. In: *Hybrid Intelligent Systems in Control, Pattern Recognition and Medicine*, pp. 169–179. Springer, Cham (2020)
19. Peraza, C., Valdez, F., Castillo, O.: Harmony search with dynamic adaptation of parameters for the optimization of a benchmark set of functions. In: *Hybrid Intelligent Systems in Control, Pattern Recognition and Medicine*, pp. 97–108. Springer, Cham (2020)
20. Pourpanah, F., Wang, R., Lim, C.P., Yazdani, D.: A review of the family of artificial fish swarm algorithms: recent advances and applications (2020). [arXiv:2011.05700](https://arxiv.org/abs/2011.05700)
21. Valdez, F., Vazquez, J.C., Melin, P., Castillo, O.: Comparative study of the use of fuzzy logic in improving particle swarm optimization variants for mathematical functions using co-evolution. *Appl. Soft Comput.* **52**, 1070–1083 (2017)
22. Wang, L.G., Shi, Q.H.: Parameters analysis of artificial fish swarm algorithm. *Comput. Eng.* **36**(24), 169–171 (2010)
23. Xiao, J., Zheng, X., Wang, X., Huang, Y.: A modified artificial fish-swarm algorithm. In: *2006 6th World Congress on Intelligent Control and Automation*, vol. 1, pp. 3456–3460. IEEE (2006)
24. Yazdani, D., Akbarzadeh-Totonchi, M.R., Nasiri, B., Meybodi, M.R.: A new artificial fish swarm algorithm for dynamic optimization problems. In: *2012 IEEE Congress on Evolutionary Computation*, pp. 1–8. IEEE (2012)
25. Zhang, Y., Guan, G., Pu, X.: The robot path planning based on improved artificial fish swarm algorithm. In: *Mathematical Problems in Engineering* (2016)
26. Zhang, C., Zhang, F.M., Li, F., Wu, H.S.: Improved artificial fish swarm algorithm. In: *2014 9th IEEE Conference on Industrial Electronics and Applications*, pp. 748–753. IEEE (2014)
27. Zhou, J., Qi, G., Liu, C.: A Chaotic parallel artificial fish swarm algorithm for water quality monitoring sensor networks 3D coverage optimization. *J. Sens.* (2021)
28. Zou, L., Li, H., Jiang, W., Yang, X.: An improved fish swarm algorithm for neighborhood rough set reduction and its application. *IEEE Access* **7**, 90277–90288 (2019)

Hierarchical Logistics Methodology for the Routing Planning of the Package Delivery Problem



Norberto Castillo-García, Paula Hernández-Hernández,
and Edilberto Rodríguez Larkins

Abstract This chapter proposes a hierarchical logistics methodology based on artificial intelligence and optimization for the routing problem of package delivery companies. These companies face several operational problems in a daily basis. The routing problem in the context of the package delivery companies consists in determine the best route to locally deliver the packages to the customers. The hierarchical logistics methodology consists of two phases. In the first phase the packages whose geographic distances are relatively close are grouped. In the second phase, once the clusters are obtained, the route in each one is optimized. In the literature, this methodology implements the classical fuzzy c-means algorithm and exact optimization. Unlike the literature, in the first phase we propose a variant of fuzzy c-means which measures the proximity between each pair of locations by means of the geographic distance instead of the Euclidean distance. In the second phase we propose the use of approximate optimization through an ant colony optimization algorithm.

Keywords Fuzzy C-means · Ant colony optimization · Traveling salesman problem · Package delivery problem · Routing problem

1 Introduction

The purpose of package delivery companies is to send products from one place to another. The products are typically packed in rectangular-shaped boxes, and hence, they are called packages. Due to the nature of these companies, they face several operational problems every day. In the literature there are documented two major problems that delivery companies have to overcome: how to stowage the packages in the delivery truck and how to determine the best route for locally delivering the packages [1].

N. Castillo-García · P. Hernández-Hernández (✉) · E. Rodríguez Larkins
Department of Engineering, Tecnológico Nacional de México/I.T. Altamira, Mexico
e-mail: paulahdz314@hotmail.com

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2022
O. Castillo and P. Melin (eds.), *New Perspectives on Hybrid Intelligent System Design based on Fuzzy Logic, Neural Networks and Metaheuristics*, Studies in Computational Intelligence 1050, https://doi.org/10.1007/978-3-031-08266-5_7

In this chapter we focus on the second major problem, that is, the problem of determining the best route to efficiently deliver the packages. Specifically, the optimization of a route consists in finding the sequence of destinations (delivery locations) to be visited such that the total cost of the route is minimized. Clearly, there are some benefits in optimizing the routes. One of the benefits is the reduction of fuel consumption. This is so because the cost of the route is typically expressed as the distance that the delivery truck must travel to deliver the packages to the customers (destinations). Since the total cost of the route is minimum, the distance traveled by a delivery truck is also minimum as well as the fuel consumed. Another benefit is the minimization of pollutant emissions since the delivery truck spends less time on the road.

In the literature there is one work that deals with the optimization of routes for package delivery companies [2]. In that work the authors propose a two-phase methodology. In the first phase, the methodology groups together the destinations that are relatively close to each other by means of the well-known Fuzzy C-Means algorithm (FCM). Then, in the second phase, each cluster is optimized by solving the mathematical formulation for the asymmetric variant of the Traveling Salesman Problem (TSP) with the subtour elimination constraints proposed by Miller, Tucker and Zemlin [3]. It is important to point out that the optimization process is performed by the optimization software CPLEX.

In this study we propose a hierarchical logistics methodology which is based on the methodology proposed in [2]. Unlike that work, in the first phase we cluster the destinations by means of a variant of FCM. The variant consists in the use of the geographic distance instead of the classical Euclidean distance. We consider important to mention that FCM has been successfully used in several domains such that [4–6].

Additionally, in the second phase we propose a heuristic approach rather than the exact approach used in [2]. Specifically, we use the well-known Ant Colony Optimization algorithm (ACO) in this phase. The heuristic approach used in the second phase allows us to solve larger instances without depending on any commercial optimization software.

The remainder of this chapter is organized as follows. In Sect. 2 we formally describe the problem faced. Section 3 describes the hierarchical logistics methodology previously mentioned. In Sect. 4 we report the computational experiment conducted to measure the effect of the parameter β (used by ACO in the second phase) on the behavior of our methodology. Finally, in Sect. 5 we discuss the major findings of this study.

2 Problem Definition

Let $D = \{d_1, d_2, \dots, d_n\}$ represent the discrete and finite set of $|D| = n$ destinations. Each destination d_i is described by an ordered pair that represents its global position on the Earth, i.e., $d_i = (\varphi_i, \lambda_i)$. In the ordered pair, φ_i corresponds to the latitude and λ_i corresponds to the longitude.

As mentioned previously, in this study we use the haversine formula [1] to measure the geographic distance between two destinations d_i and d_j . It is important to point out that we use this formula since it is capable to compute the distance between two points placed on a spherical surface such as the Earth. The haversine formula is shown in Eq. (1).

$$\delta(d_i, d_j) = 2r \tan^{-1} \left(\frac{\sqrt{a}}{\sqrt{1-a}} \right) \quad \forall i, j = 1, \dots, n, i \neq j, \quad (1)$$

where:

- $\delta(d_i, d_j)$ is the geographic distance (in kilometers) between destinations $d_i = (\varphi_i, \lambda_i)$ and $d_j = (\varphi_j, \lambda_j)$.
- $a = \sin^2\left(\frac{\varphi_j - \varphi_i}{2}\right) + \cos(\varphi_i) \cos(\varphi_j) \sin^2\left(\frac{\lambda_j - \lambda_i}{2}\right)$.
- r represents the radius of the Earth. In this study we consider $r = 6,371$ km.
- $\tan^{-1}\left(\frac{\sqrt{a}}{\sqrt{1-a}}\right)$ is the angle (in radians) resulting from converting the rectangular coordinates $(\sqrt{a}, \sqrt{1-a})$ to polar coordinates.

The goal of the route planning of package delivery companies is to find a tour of destinations with the minimum cost [2]. In this chapter we model this problem as an application of the well-known Traveling Salesman Problem (TSP). Therefore, the problem consists in finding a permutation of destinations π^* such that the total cost of the tour (permutation) is the minimum. Formally:

$$\pi^* = \operatorname{argmin}_{\pi \in \Pi} \left\{ \sum_{i=0}^{n-1} \delta(d_{\pi(i)}, d_{\pi(i+1)}) + \delta(d_{\pi(n)}, d_{\pi(1)}) \right\},$$

where $d_{\pi(i)}$ is the i -th destination in the permutation π , and Π is the set of all permutations of destinations, that is, the solution space. Notice that the tour cost is computed from the geographic distances among the set of destinations D .

3 Hierarchical Logistics Methodology

This section describes the hierarchical methodology proposed to solve the routing process of package delivery companies described in the previous section. Our methodology consists of two phases: clustering and optimization. The first phase is described in Sect. 3.1 and the second phase in Sect. 3.2.

3.1 Phase 1: Clustering by FCM

The idea of the first phase of the methodology is to cluster the delivery locations according to their global position. Thus, the destinations that are relatively close to each other (in geographic distance) will be in the same cluster. The goal is to assign one delivery truck to one particular cluster. In order to perform the clustering process, we use the well-known Fuzzy C-Means (FCM) algorithm. Nevertheless, unlike in the literature, we do not use the Euclidean distance. Instead, we use the geographic distance among the destinations according to Eq. (1). Algorithm 1 shows the pseudocode of FCM.

Algorithm 1: Fuzzy c-means algorithm

Data: k, m', ϵ and D .
Result: A k -partition of the set D .

Begin **U**
 $stop \leftarrow \text{false}$
while not $stop$ **do**
 computeCentroids()
 computeDistancesToCentroids()
 updateU()
 $stop \leftarrow stopExecution()$
end
getClusters()

As can be observed in Algorithm 1, FCM requires the number of clusters (k), the level of fuzzification (m'), the level of tolerance (ϵ), and the set entire of destinations (D). In the methodology, the number of clusters is a given value since it depends on the number of delivery trucks of the company, and hence, FCM does not determine it. The level of fuzzification m' can theoretically take any value in the domain $(1, \infty)$. Nevertheless, in this study we set $m' = 2$ since it is the value recommended in the literature [2, 7]. The level of tolerance ϵ is a small value used to determine if FCM should continue its execution or not. Here we also use the value recommended in the literature, i.e., $\epsilon = 0.001$ [2, 7]. In addition to the previous parameters, the algorithm also requires the set of delivery locations. As mentioned previously, the set of delivery locations consists of a collection of ordered pairs (φ_i, λ_i) with $1 \leq i \leq n$, in which φ_i and λ_i stands for the latitude and longitude of the i -th destination, respectively.

FCM operates in the following way. The algorithm firstly initializes the matrix $\mathbf{U}_{k \times n}$, which is used to record the membership levels of the n delivery locations to the k clusters. Since this matrix stores membership levels, all the entries are real values between zero and one. As recommended in the literature [2], we initialize this matrix in the following way. Each column of $\mathbf{U}$ has exactly one entry whose value is 1 and the remaining entries are zero. In addition, the rows of $\mathbf{U}$ does not contain only zero values. Figure 1 depicts an example of a matrix $\mathbf{U}$ for $k = 3$ clusters and

Fig. 1 Initialization of a matrix $\mathbf{U}$ with $k = 3$ rows (clusters) and $n = 6$ columns (destinations)

$$\mathbf{U} = \left[\begin{array}{cccccc} \mathbf{1} & 0 & 0 & \mathbf{1} & 0 & 0 \\ 0 & \mathbf{1} & 0 & 0 & \mathbf{1} & 0 \\ 0 & 0 & \mathbf{1} & 0 & 0 & \mathbf{1} \end{array} \right] \left. \vphantom{\begin{array}{cccccc} \mathbf{1} & 0 & 0 & \mathbf{1} & 0 & 0 \\ 0 & \mathbf{1} & 0 & 0 & \mathbf{1} & 0 \\ 0 & 0 & \mathbf{1} & 0 & 0 & \mathbf{1} \end{array}} \right\} k = 3$$

$$n = |D| = 6$$

$n = 6$ destinations.

The second step of FCM is to assign the value of false to the flag *stop*. Steps 3–8 constitutes the main loop of the algorithm. Step 4 (first step in the main loop) consists in computing the centroids $v_1, v_2, \dots, v_k$ considering both the current values of $\mathbf{U}$ and the set of destinations $D = \{(\varphi_1, \lambda_1), (\varphi_2, \lambda_2), \dots, (\varphi_n, \lambda_n)\}$. The computation of the centroids is given by the following equations:

$$v_{i,\varphi} = \frac{\sum_{j=1}^n (U_{ij})^{m'} \times \varphi_j}{\sum_{j=1}^n (U_{ij})^{m'}} \quad \forall i = 1, \dots, k,$$

$$\text{and } v_{i,\lambda} = \frac{\sum_{j=1}^n (U_{ij})^{m'} \times \lambda_j}{\sum_{j=1}^n (U_{ij})^{m'}} \quad \forall i = 1, \dots, k.$$

Once the k centroids have been computed, FCM executes step 5. In this step, the algorithm computes the distances from the centroids to the destinations. Unlike the literature, in this study we propose to compute the geographic distances [see Eq. (1)] rather than the Euclidean distance. Thus, the geographic distances from the centroids to the destinations are given by $\delta(v_i, d_j)$.

The following step of FCM consists in updating the matrix $\mathbf{U}$ according to the recently computed geographic distances from the centroids to the destinations. The purpose of this process is to recompute the membership levels of the destinations to the new centroids. This process is performed by the following equation:

$$\mathbf{U}_{ij}^{\text{upt}} = \left[\sum_{l=1}^k \left(\frac{\delta(v_l, d_j)}{\delta(v_i, d_j)} \right)^{2/(m'-1)} \right]^{-1} \quad \forall i = 1, \dots, k, j = 1, \dots, n.$$

Finally, step 7 of FCM consists in determining if the algorithm should continue its execution. This is achieved as follows. If $|\mathbf{U}_{ij}^{\text{upt}} - \mathbf{U}_{ij}^{\text{prev}}| \leq \epsilon, \forall i, j$ then the procedure `stopExecution()` returns the value of true and the execution of FCM must be stopped. Otherwise, the procedure returns false and the execution of FCM must continue. Here, matrix $\mathbf{U}^{\text{prev}}$ represents the membership values of the previous iteration. Finally, in Step 9 FCM outputs the k clusters of delivery locations.

3.2 Phase2: Optimization by ACO

In the second phase of the methodology, we perform an optimization process in order to determine the best route for each cluster. Unlike the literature [2], in this chapter we approach this phase heuristically. More precisely, we propose an Ant Colony Optimization (ACO) algorithm to optimize each cluster found in the previous phase. ACO is an algorithm based on the collaborative behavior of real ants [8, 9]. Algorithm 2 shows the general pseudocode of ACO.

Algorithm 2: Ant Colony Optimization algorithm

Data: $m, \alpha, \beta, \varphi, \rho, \tau_0, GMAX$ and $\mathbf{D}_{n \times n}$.
Result: A permutation of destinations π .

Begin $\tau_{n \times n}, \eta_{n \times n}$
 $g \leftarrow 1$
while $g \leq GMAX$ **do**
 for $a = 1$ **to** m **do**
 for $i = 1$ **to** n **do**
 selectNextNode()
 pheromoneTrailLocalUpdate()
 end for
 pheromoneTrailGlobalUpdate()
 end for
 $g \leftarrow g + 1$
end
getPermutation()

As can be observed from Algorithm 2, ACO requires the number of ants (m), the parameter used to intensify the contribution of the pheromone trail (α), the parameter used to intensify the contribution of the heuristic information (β), the evaporation rate in the local update (φ), the evaporation rate in the global update (ρ), the initial value of the pheromone trail (τ_0), the subset of n geographic distances of destinations in the current cluster ($\mathbf{D}_{n \times n}$), and the maximum number of ant generations ($GMAX$).

ACO starts by setting the initial values of both the pheromone table τ and heuristic information table η . On the one hand, the pheromone table is initialized with a small value τ_0 , i.e., $\tau_{ij} \leftarrow \tau_0$ for all $i, j = 1, \dots, n$. On the other hand, the heuristic information is initialized as follows:

$$\eta_{ij} \leftarrow \frac{1}{\mathbf{D}_{ij}} \forall i, j = 1, \dots, n,$$

where $\mathbf{D}_{ij}$ is the geographic distance between destinations i and j .

Once the pheromone trails and heuristic information have been initialized, each ant constructs a tour (solution) by iteratively selecting the next destination to be visited.

This selection is performed probabilistically by means of the roulette wheel selection and taking into account both the pheromone trail and the heuristic information according to the following equation:

$$P_{ij} = \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}{\sum ([\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta)}$$

where i is the current node, j is an unselected node and P_{ij} is the probability of selecting arc (i, j) to be included in the current tour. The algorithm computes the probabilities of the nodes which are not in the current tour and selects one of them according to the roulette wheel selection. In this chapter we follow the roulette wheel implementation documented in [10].

When an ant has selected the following node, the algorithm locally updates the pheromone trail of the selected arc. This process is performed as follows:

$$\tau_{ij} = (1 - \varphi)\tau_{ij} + \varphi\tau_0$$

where i is the current node and j is the selected node.

Once an ant has completed a tour, the algorithm globally updates the pheromone trails as follows:

$$\tau_{ij} = (1 - \rho)\tau_{ij} + \rho\Delta\tau_{ij}$$

with:

$$\Delta\tau_{ij} = \frac{1}{OV(\pi)}$$

where $i = \pi(1), \pi(2), \dots, \pi(n)$, $j = \pi(i + 1)$, $OV(\pi)$ is the objective value of permutation π , and $\pi(i)$ is the i -th destination in the tour. This process is repeated until the maximum number of ant generations ($GMAX$) is reached.

4 Computational Experiment

In this section we report the computational experiment conducted to assess the behavior of the hierarchical logistics methodology proposed. More precisely, the goal of the experiment is to measure the effect of the parameter β on the methodology performance. Recall that β is used to intensify the contribution of the heuristic information of the problem in the second phase of the methodology.

Table 1 Parameter values used to conduct the experiment

Methodology phase	Parameter value	Meaning
1	$k = 5$	Number of clusters
	$m' = 2.0$	Level of fuzzification
	$\epsilon = 0.001$	Level of tolerance
2	$m = 10$	Number of ants
	$\alpha = 1.0$	Level of contribution of the pheromone trail
	$\varphi = 0.35$	Evaporation rate in the local update
	$\rho = 0.50$	Evaporation rate in the global update
	$\tau_0 = 0.001$	Initial value of the pheromone trails
	$GMAX = 100$	Maximum number of ant generations

The experiment was conducted on a workstation with an AMD Ryzen Threadripper 3960X 24-core processor at 3.8 GHz and 128 GB of RAM. The methodology was implemented in Java and executed in the Java Runtime Environment 1.8.0_201-b09. In this study we solved 30 artificial instances generated by the automatic tool proposed in [1]. All of these instances follow a uniform distribution and are categorized as small, medium and large according to the number of destinations. Specifically, there are 10 small instances whose number of destinations ranges from 28 to 100 (i.e., $28 \leq n \leq 100$); 10 medium instances with $107 \leq n \leq 199$; and 10 large instances with $209 \leq n \leq 287$. In the experiment, we evaluate 11 different values for the parameter β , namely:

$$\beta = \{0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0\}.$$

We also kept fixed all the remaining parameters. The punctual values for each parameter are reported in Table 1.

The hierarchical logistics methodology was executed 30 times for each instance and for each β -value in order to mitigate the randomness effect. Since there are 30 instances and 11 β -values, the total number of executions is 9900. An execution refers to the solution of a given instance with a specific value of β . At each execution, we record the objective value for each cluster. The global objective value of an execution is obtained by summing the objective values and the computing times of each cluster. Table 2 reports the results of the experiment, that is, the average objective values of the 30 repetitions.

As can be observed from Table 2, the value of β influences the performance of the methodology. Specifically, small β -values lead to poor objective values whereas large values of β yield better objective values. The behavior of the methodology can be clearly observed in Fig. 2. In this figure we depict 11 box plots that represents the objective values (y-axis) with respect to the β -values (x-axis).

Figure 2 provides a clear perspective of the methodology behavior. In this figure we can observe that small values of β not only produce poor objective values on

Table 2 Experimental results for the different values for the parameter β

Dataset	Instance	β -values										
		0.0	0.5	1.0	1.5	2.0	2.5	3.0	3.5	4.0	4.5	5.0
Small	1	84.19	76.17	70.69	67.43	65.83	64.80	64.35	64.17	64.13	64.11	64.06
	2	137.35	117.11	98.80	88.09	81.79	78.22	76.66	75.59	75.26	74.84	74.56
	3	92.55	83.76	76.77	71.62	69.15	68.00	67.13	66.50	66.25	66.11	66.03
	4	76.10	68.54	63.05	59.37	57.59	56.81	56.23	56.09	55.98	55.91	55.87
	5	101.05	88.57	79.14	72.48	68.19	66.41	65.46	64.99	64.77	64.53	64.51
	6	46.18	46.17	46.17	46.17	46.17	46.17	46.17	46.27	46.33	46.43	46.57
	7	142.53	123.38	107.70	95.57	87.67	82.97	80.78	79.11	78.34	77.70	77.32
	8	109.60	96.65	85.26	78.88	74.27	71.88	70.82	69.77	69.40	69.08	69.06
	9	120.67	103.97	91.40	82.29	77.93	75.41	73.73	72.71	72.15	72.03	71.83
	10	119.48	106.52	95.16	86.58	80.44	77.19	75.49	74.50	74.17	73.93	73.77
Medium	1	213.76	178.48	147.42	126.27	113.22	104.71	100.16	96.77	94.49	93.25	92.21
	2	297.73	244.44	192.04	157.91	137.69	126.38	120.15	115.10	112.69	110.17	109.19
	3	151.54	129.95	108.98	95.45	87.42	81.59	78.44	76.31	75.03	74.46	74.06
	4	315.35	254.29	196.41	159.11	139.91	128.53	120.49	116.11	113.19	111.41	109.53
	5	177.04	148.20	123.46	106.88	97.76	92.04	88.68	86.91	85.62	85.02	84.59
	6	232.20	193.18	156.70	131.83	119.12	111.87	105.84	102.12	99.40	98.29	97.44
	7	159.06	134.42	115.47	101.13	93.48	88.00	85.69	83.43	82.62	81.91	81.49
	8	227.20	186.31	148.45	126.06	113.11	105.15	100.26	97.68	95.84	94.53	93.56
	9	164.96	141.53	118.52	104.33	95.48	90.28	86.83	84.82	82.79	82.08	81.35
	10	157.66	136.79	119.21	105.01	96.56	91.16	87.99	85.91	84.49	83.46	83.28

(continued)

Table 2 (continued)

Dataset	Instance	β -values										
		0.0	0.5	1.0	1.5	2.0	2.5	3.0	3.5	4.0	4.5	5.0
Large	1	415.98	337.53	253.47	201.58	171.23	153.51	143.10	136.62	132.57	129.40	127.54
	2	384.23	309.26	232.06	181.40	155.57	139.60	130.45	124.35	120.65	118.08	116.57
	3	434.81	350.27	261.14	202.10	173.70	156.90	147.79	141.38	137.03	134.26	132.16
	4	430.97	347.51	256.86	199.95	168.34	150.91	140.23	133.86	129.12	126.35	124.28
	5	338.21	274.21	212.30	171.55	148.02	134.96	126.97	122.06	118.78	116.26	114.71
	6	374.48	301.80	229.24	181.22	156.52	142.01	132.65	126.61	122.94	120.71	118.54
	7	478.77	377.89	271.46	209.15	175.90	159.32	148.72	141.92	138.14	134.84	132.57
	8	425.38	338.68	251.61	197.54	169.16	152.60	141.89	135.59	131.75	128.86	126.66
	9	467.83	373.64	276.83	214.70	181.59	162.69	151.23	143.50	139.35	135.94	133.87
	10	485.19	380.91	278.64	214.31	179.86	163.06	151.42	145.29	139.80	137.20	134.54

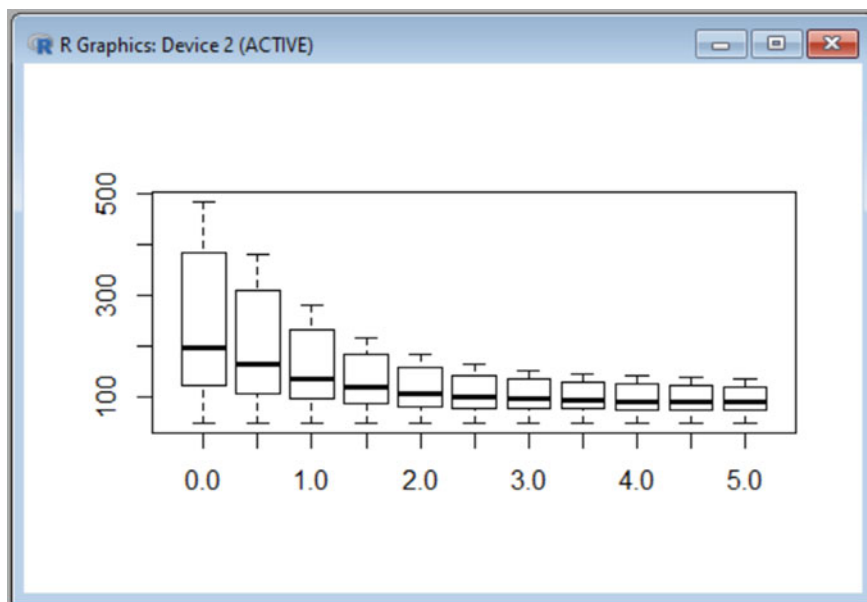


Fig. 2 Box plots representing the behavior of the hierarchical logistics methodology with different values of the parameter β

average, but also a large variability. On the contrary, large values of β produces good objective values and a small variability. Furthermore, from the figure we can observe that the methodology behavior is more stable and produces the best results for β -values ranging from 3.0 to 5.0.

In order to statistically validate the experimental results and observations, we conducted the Friedman test on the values reported in Table 2. The Friedman test is used to determine if at least one treatment is significantly different from the others. In the context of this study, there are 11 treatments (the β -values) and 30 blocks (the instances). We use the statistical software R to automatically conduct the test. The Friedman test found a p -value less than 2.2×10^{-16} , which implies a confidence level of 99.99%. The result of the Friedman test provides strong evidence that the methodology behavior is significantly affected by the value of the parameter β .

5 Conclusions

In this study we have faced the routing planning process of package delivery companies. In particular, we propose a hierarchical methodology consisting in two phases. The first phase consists in clustering the delivery locations by means of the well-known Fuzzy C-Means (FCM) algorithm. In the second phase we perform a heuristic optimization process on each cluster through the Ant Colony Optimization (ACO)

algorithm. In both phases, we measure the geographic distances among the delivery locations by means of the haversine formula since it considers the curvature of the Earth. Unlike the methodology documented in [2], the hierarchical logistics methodology proposed here uses a variant of FCM in which the proximity between each pair of destinations is measured by the geographic distance rather than the Euclidean distance. In addition, in the second phase of the methodology proposed we use a metaheuristic algorithm (ACO) instead of an integer linear programming formulation. Clearly, the metaheuristic algorithm allows us to solve larger instances in a relatively small period of time.

We conducted a computational experiment in order to assess the performance of the methodology proposed. Specifically, the experiment consisted in measuring the effect of the parameter β (used by ACO in the second phase) on the behavior of the methodology. The experimental results clearly indicated that the value of this parameter has a significant effect on the quality of the solutions found with 99.99% of confidence. Therefore, taking into account the evidence of this study, we conclude that our methodology is a good alternative to heuristically solve the routing planning of package delivery companies.

Acknowledgements The authors would like to thank *Tecnológico Nacional de México*, and especially the authorities of *Instituto Tecnológico de Altamira* for their support in this research. The first author also thanks the Mexican Council for Science and Technology (CONACYT) for its support through the Mexican National System of Researchers (SNI) (Grant No. 70157).

References

1. Hernández, P.H., Castillo-García, N., Larkins, E.R., Ruiz, J.G.G., Díaz, S.V.M., Resendiz, E.S.: A fuzzy logic classifier for the three dimensional bin packing problem deriving from package delivery companies application. In: *Handbook of Research on Metaheuristics for Order Picking Optimization in Warehouses to Smart Cities*, pp. 433–442. IGI Global (2019)
2. Hernández-Hernández, P., Castillo-García, N.: Optimization of route planning for the package delivery problem using fuzzy clustering. In: *Technological and Industrial Applications Associated with Intelligent Logistics*, pp. 239–252. Springer, Cham (2021)
3. Chen, D.S., Batson, R.G., Dang, Y.: *Applied integer programming: modeling and solution*. Wiley (2011)
4. Asha, G.R.: Energy efficient clustering and routing in a wireless sensor networks. *Proc. Comput. Sci.* **134**, 178–185 (2018)
5. Miyamoto, S., Ichihashi, H., Honda, K., Ichihashi, H.: Algorithms for fuzzy clustering, pp. 9–12. Springer, Heidelberg (2008)
6. Su, S., Zhao, S.: An optimal clustering mechanism based on Fuzzy-C means for wireless sensor networks. *Sustain. Comput. Inf. Syst.* **18**, 127–134 (2018)
7. Cruz, P.P.: *Inteligencia artificial con aplicaciones a la ingeniería*. Alfaomega (2011)
8. Hernández, P., Gómez, C., Cruz, L., Ochoa, A., Castillo, N., Rivera, G.: Hyperheuristic for the parameter tuning of a bio-inspired algorithm of query routing in P2P networks. In: *Mexican International Conference on Artificial Intelligence*, pp. 119–130. Springer, Berlin, Heidelberg (Nov 2011)

9. Hernández, P.H., Cruz-Reyes, L., Melin, P., Mar-Ortiz, J., Huacuja, H.J.F., Soberanes, H.J.P., Barbosa, J.J.G.: An ant colony algorithm for improving ship stability in the containership stowage problem. In: Mexican International Conference on Artificial Intelligence, pp. 93–104. Springer, Berlin, Heidelberg (Nov 2013)
10. Huacuja, H.J.F., Castillo-García, N.: Optimization of the vertex separation problem with genetic algorithms. In: Handbook of Research on Military, Aeronautical, and Maritime Logistics and Operations, pp. 13–31. IGI Global (2016)

A Novel Distributed Nature-Inspired Algorithm for Solving Optimization Problems



J. C. Felix-Saul, Mario García Valdez, and Juan J. Merelo Guervós

Abstract Several bio-inspired algorithms use population evolution as analogies of nature. In this paper, we present an algorithm inspired by the biological Life-Cycle of animal species, which consists of several stages: birth, growth, reproduction, and death. As in nature, we intend to execute all these stages in parallel and asynchronously on a population that evolves constantly. From the ground up, we designed the algorithm as a cloud-native solution using the cloud available resources to divide the processing workload among several computers or running the algorithm as a cloud service. The algorithm works concurrently and asynchronously on a constantly evolving population, using different computers (or containers) independently, eliminating waiting times between processes. This algorithm seeks to imitate the natural life cycle, where new individuals are born at any moment and mature over time, where they age and suffer mutations throughout their lives. In reproduction, couples match by mutual attraction, where they may have offspring. Death can happen to everyone: from a newborn to an aged adult, where the individual's fitness will impact their longevity. As a proof-of-concept, we implemented the algorithm with Docker containers by solving the OneMax problem comparing it with a basic (sequential) GA algorithm, where it shows favorable and promising results.

Keywords Distributed bioinspired algorithms · Genetic algorithms · Cloud computing

J. C. Felix-Saul (✉) · M. G. Valdez
Department of Graduate Studies, Tijuana Institute of Technology, Tijuana, Mexico
e-mail: jose.felix201@tectijuana.edu.mx

J. J. M. Guervós
Department of Computer Architecture and Technology, Universidad de Granada, Granada, Spain

1 Introduction

Bio-inspired algorithms have been very successful when used to solve complex optimization problems [1, 4, 21], but as their complexity increases so does the computing power required [14]. One strategy to address this processing need is to use distributed computing [20], or the resources available in the cloud [5, 7] to help find the solution. This strategy gives us elasticity to adjust (increase or reduce) the computing power to achieve a balance according to the nature of the problem.

One of the problems that we have identified in this research is that most bio-inspired algorithms are designed with a traditionally sequential perspective [3, 15], where each process must wait for the previous task to finish before continuing. Some architectures address this issue [9, 12, 22], what distinguishes us, is that our proposal presents an algorithm designed in its entirety as a native solution in the cloud, fully distributed, where its processes are executed in parallel and asynchronously. This technique makes it easy to scale the computing power according to the complexity required by the problem [2].

We designed our algorithm using observation, analysis, and abstraction in nature to identify what most species in the animal kingdom have in common, where individuals of a population that best adapt to the environment have greater chances of survival, reproduction, and improvement of the species. Imagining as if the task of writing these rules of evolution was in our hands, questioning how we could solve it?

Like nature, we identified what most species have in common in the life cycle. Our algorithm works on a constantly evolving population, experiencing the stages that all living beings undergo [17]: being born, growing up, reproducing, and dying, where each of these stages will be processes (of our algorithm) that randomly affect the population individuals, emulating the organic way.

The main contribution of the paper is to design and create a new algorithm with the purpose to demonstrate that it is possible to evolve a population of individuals, similar to a Genetic Algorithm (GA), using a distributed, parallel, and asynchronous methodology by implementation and testing of the algorithm.

We present the description of the following sections in our paper. First, we introduce our algorithm inspired by the Life-Cycle of animal species in Sect. 2, followed by our experimental configuration and results in Sect. 3, we analyze and describe some of our research findings in Sect. 4, and we finalize by presenting some inferences based on our experiments' results in Sect. 5.

2 Proposal

Many of the publications with the most influence on our research [8, 11, 12, 22] propose a cloud-native optimization architecture that features population-based algorithms. They present ideas such as those mentioned in Fig. 1, which planted the seed of thought of our current paper.

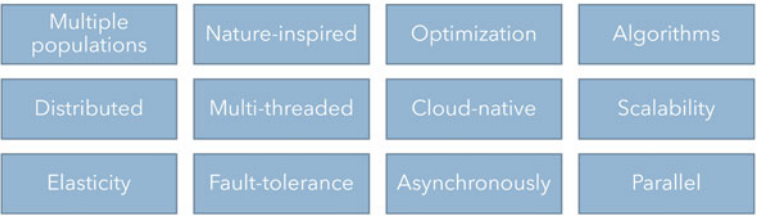


Fig. 1 These ideas planted the seed of thought in our algorithm proposal

Our main inspiration for this algorithm was born from the study and observation of nature, where many successful optimization algorithms have previously found their own. Our focus was not on a single animal species but all of them, using general abstraction to identify what those species have in common.

Some questions in our train of thought were: How does nature work? Is evolution real? How do we enforce natural selection in evolution? What is the role of the attraction between couples on reproduction? How much of a factor is the attraction’s role in the evolution of the species? What is the role of death in all of this? How much of an impact does death have on the evolution of a species? How do we include those ideas into our algorithm?

In our analysis, we consider a broad animal spectrum, from bacteria to humans and lions, finding our inspiration on the biological Life Cycle of the animal species, which consists of several stages [17]: birth, growth, reproduction, and death. As in nature, we intend to execute all these stages in parallel and asynchronously on a population that evolves constantly. The combination of those processes is what we call evolution.

One of the main challenges when designing the algorithm was how to capture the essence of life by reflecting a population evolution?. In our minds, it was clear this task requires a combination of simultaneous working forces that influences the population improvement over time. Our strategy was simple: divide and conquer, looking into the clouds. Enter cloud-computing: previous cloud-computing algorithms have proven successful results working in optimization problems [8, 22].

We turned back to our genetic algorithm knowledge, mixed with our inspiration and cloud computing, and designed the algorithm with the quest to divide the processing workload among multiple computers. This strategy would also make it possible to run our algorithm as a cloud service. Theoretically, one consequence of the workload distribution is to obtain lower execution times.

This algorithm seeks to imitate the natural life cycle, where new individuals are born at any moment and mature over time, where they age and suffer mutations throughout their lives. In reproduction, couples match by mutual attraction, where they may have offspring. Death can happen to everyone: from a newborn to an aged adult, where the individual’s fitness will impact their longevity. The general model concept is shown in Fig. 2. The general flowchart for the Life-Cycle algorithm is described in Fig. 3.

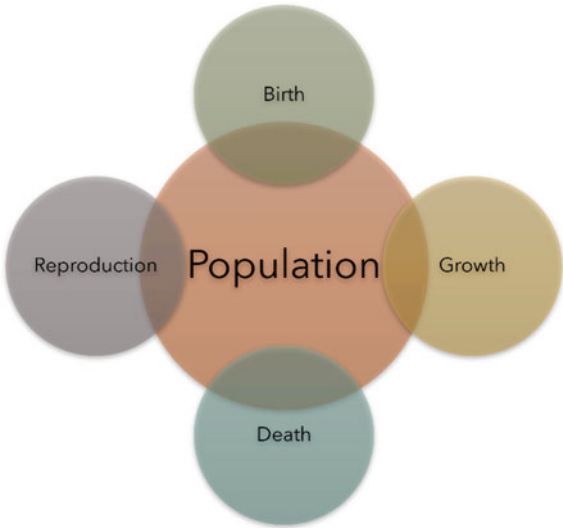


Fig. 2 Algorithm model inspired by the animal species’ biological Life-Cycle

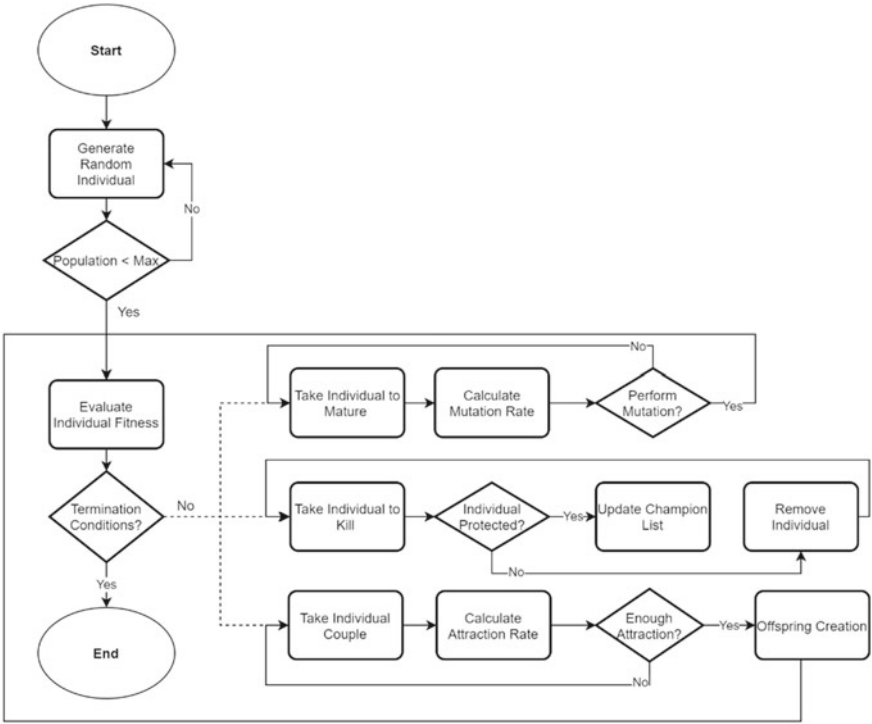


Fig. 3 Life-Cycle algorithm general flowchart

2.1 Birth

This algorithm starts with a randomly generated population, where the processes will interact with the population independently. This means that at any given time, any individual can experience any of these processes. Birth is the first process of our algorithm, and it is responsible for the initial generation of individuals.

2.2 Growth

As in nature, all individuals constantly grow, mature, or age. With increasing age, individuals may lose strength but also gain more knowledge to solve problems. We represent this with a possible mutation in each increment of age. The growth process will take an individual to assess whether it's ready to mature and undergo changes.

2.3 Reproduction

The attraction of a couple will depend on fitness: the better individual's fitness, the more attractive it will be, making it easier to find mating matches. This process will be taking random pairs of individuals to evaluate their attraction as a couple, to try to breed; when the gestation is successful, a new pair of individuals will be born (as the offspring). Not all couples will be compatible, so reproduction will not always be possible, but the problem arises: how to quantify the attraction between two individuals?

We could have used several strategies to evaluate this attraction. Considering this algorithm takes a similar focus as the study of bacteria growth in microbiology, where we can observe and analyze the evolution of a population over time. As in nature' most species, the bigger a specimen is, the more attractive it is to its mating candidates. To be consistent with both ideas in our algorithm, we use the equation of Newton's Law of Universal Gravitation (1) to calculate the attraction between two individuals, where we visualize the individual fitness as its mass when using the equation. In previous work, Newton's Law of Universal Gravitation has shown success in helping to solve optimization problems [16, 18].

$$F = G \frac{m_1 m_2}{r^2}, \quad (1)$$

where:

- **F** is the gravitational force acting between two objects.

- **m1 and m2** are the masses of the objects.
- **r** is the distance between the centers of their masses.
- **G** is the gravitational constant: $6.67430(15) \times 10^{-11} \text{ m}^3 \cdot \text{kg}^{-1} \cdot \text{s}^{-2}$.

2.4 Death

The death stage represents the challenges and adversities that life presents to overcome. This process evaluates the individual resistance to survive in the environment. The better fitness the individual has will increase its chances of survival. As time progresses, the demands of nature will also increase, pushing for only the best individuals to survive.

3 Experiments

As a proof-of-concept, we implemented the algorithm with Docker containers by solving the OneMax problem to compare it with a traditional GA algorithm using the DEAP (Distributed Evolutionary Algorithms in Python) library [6]. The OneMax problem [10, 23] uses a genetic algorithm to evolve a population of randomly generated individuals with zeros and ones, and it stops until a solution of only ones is found.

3.1 Experimental Setup

One strategic difference in our algorithm implementation is the reproduction process, which is flexible to work in parallel with multiple couple selection methods, for example, tournament selection, random selection (Closed Reproduction process), and couple match by creating a new mating individual (Open Reproduction process). We use many combinations of the containerized processes, up to the minimum number required to obtain good results. Figure 4 shows a comparison of only two alternatives for the implementation of the algorithm. Our experimentation started with a configuration of ten processes and ended in five, shown on the left and right sides of Fig. 4, respectively.

3.2 Experiment Configuration

In our four experiments, we needed to match or balance the experimentation parameters according to the specifications of the algorithm used by the DEAP library. This

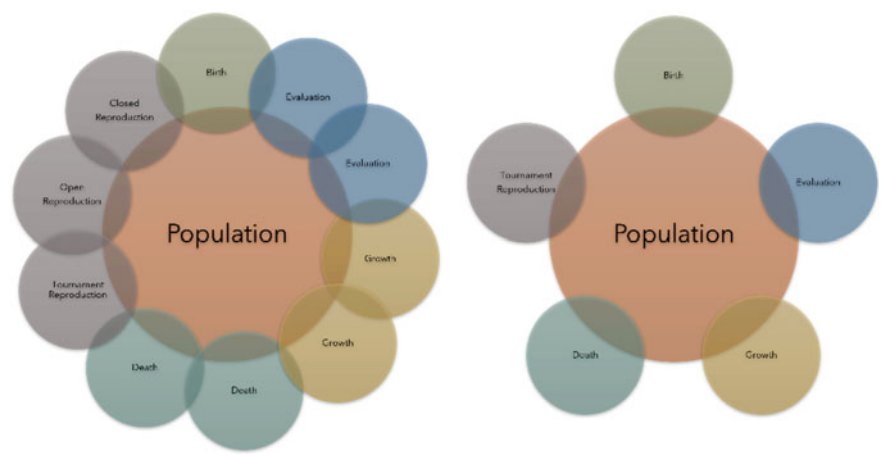


Fig. 4 Life-Cycle algorithm processes in containers comparison

is to be able to verify if our algorithm would also converge on the solutions in a similar number of evaluations and execution time. Table 1 shows the OneMax initial configuration for DEAP and Life-Cycle experiment.

Table 1 OneMax initial configuration for DEAP and Life-Cycle experiment

Configuration	DEAP	Life-Cycle
Population	60	60
Max generation	20	20
Stagnation	Off	10
Chromosome length	20	20
Target fitness	20	20
Crossover rate	100	100
Mutation rate	7	7
Tournament rate	100	50
Tournament sample	3	4
Open reproduction rate	NA	5
Closed reproduction rate	NA	45
Max age	NA	80
Base approval	NA	80
Goal approval	NA	100

3.3 Experiment Results

For each experiment, we ran 60 independent executions per algorithm and recorded the following results: Last Generation, Total Evaluations, Time (seconds), Evaluations/second. The labels used on the results of our summarized experiments tables are the following:

- Last Gen. is the generation that found the solution.
- Eval. is the total number of evaluations the algorithm executed.
- Time (sec) is the total elapsed or wall clock time, in seconds.
- Eval/sec is the calculated rate of evaluations per second.

3.3.1 Experiment 1

The goal of our first OneMax experiment was to make sure our Life-Cycle algorithm worked as expected, converging on the solution. For this experiment, we intentionally used delays (on the tenth of a second magnitude) to follow the Life-Cycle algorithm behavior on the console output. For this experiment, we compared the DEAP algorithm versus the Life-Cycle using ten processes (on containers) where we show the summarized results in Table 2, and Fig. 5 shows their Box and Whisker chart.

Table 2 Experiment 1 results: OneMax DEAP versus Life-Cycle (10 processes)

Run	OneMax DEAP				Life-Cycle (10P)			
1–60	Last Gen	Eval	Time (s)	Eval/s	Last Gen	Eval	Time (s)	Eval/s
Avg	6.5	391	0.036	10,836	9.6	578	6.830	85



Fig. 5 Box and Whisker chart for the Experiment 1: OneMax DEAP versus Life-Cycle

Table 3 Experiment 2 results, Life-Cycle Tournament selection: 10 versus 5 processes

Run	Life-Cycle (Tourn. 10P)				Life-Cycle (Tourn. 5P)			
1–60	Last Gen	Eval	Time (s)	Eval/s	Last Gen	Eval	Time (s)	Eval/s
Avg	7.2	434	0.740	587	8.2	495	0.826	599

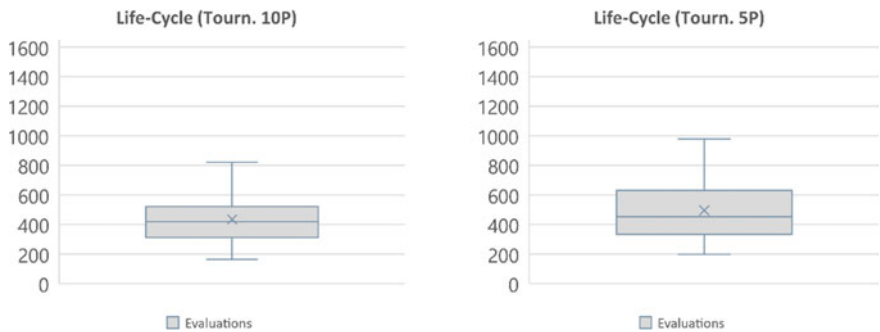


Fig. 6 Box and Whisker chart for the Experiment 2: Life-Cycle Tournament selection (10 vs. 5 processes)

3.3.2 Experiment 2

The goal of our second experiment was to confirm the Life-Cycle algorithm continued working as expected, converging on the solution. For this experiment, we reduced the time used on delays (now on the thousands of a second magnitude) to follow the Life-Cycle algorithm behavior. For this experiment, we only used tournament selection for the reproduction, on the first configuration running the Life-Cycle on ten processes, versus the second configuration where we reduced the processes to the minimum basic 5. We show the summarized results in Table 3, and Fig. 6 shows their Box and Whisker chart.

3.3.3 Experiment 3

The goal of our third experiment was to follow and study the behavior of the Life-Cycle algorithm that continued converging on the solution. For this experiment, we remained using minimal delays (thousands of a second magnitude). For this experiment, we only used tournament selection for the reproduction, on the first configuration running the Life-Cycle on six processes, two of whom was the Death process, versus the second configuration where we reduced the processes to the minimum (five) but increasing the (Death) goal approval range to 115. We show the summarized results in Table 4, and Fig. 7 shows their Box and Whisker chart.

Table 4 Experiment 3 results, Life-Cycle Tournament: 6 processes (Death × 2) versus 5 processes (80–115 goal)

Run	Life-Cycle (Tourn. 6P, Death × 2)				Life-Cycle (Tourn. 5P, 80-115 g)			
1–60	Last Gen	Eval	Time (s)	Eval/s	Last Gen	Eval	Time (s)	Eval/s
Avg	8.0	483	0.846	571	6.4	384	0.695	553

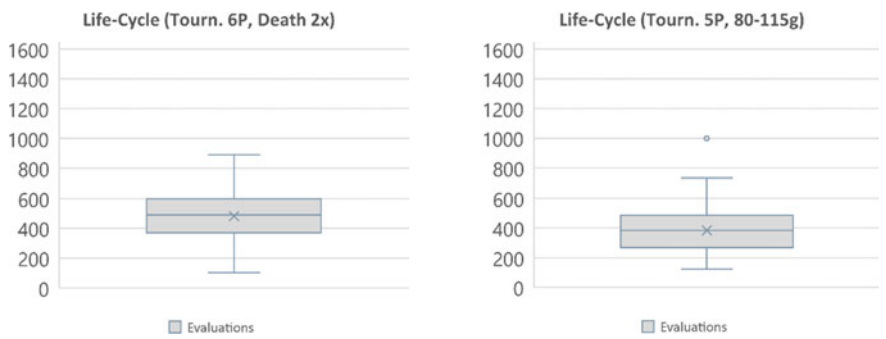


Fig. 7 Box and Whisker chart for the Experiment 3: Life-Cycle Tournament (6 processes double Death, vs. 5 processes with an 80–115 goal approval)

3.3.4 Experiment 4

The goal of our fourth and last experiment was to follow and study the behavior of the Life-Cycle algorithm that continued converging on the solution. For this experiment, we remained using minimal delays (thousands of a second magnitude). For this experiment, we compared the OneMax DEAP implementation versus the Life-Cycle algorithm, only using tournament selection for the reproduction, with the Life-Cycle configuration running on the minimum (five) processes but increasing the (Death) goal approval range to 125. We show the summarized results in Table 5, and Fig. 8 shows their Box and Whisker chart.

Table 5 Experiment 4 results, OneMax DEAP versus Life-Cycle Tournament: 5 processes (80 to 125 goal)

Run	OneMax DEAP				Life-Cycle (Tourn. 5P, 80-125 g)			
1–60	Last Gen	Eval	Time (s)	Eval/s	Last Gen	Eval	Time (s)	Eval/s
Avg	6.5	391	0.036	10,836	6.3	377	0.678	556

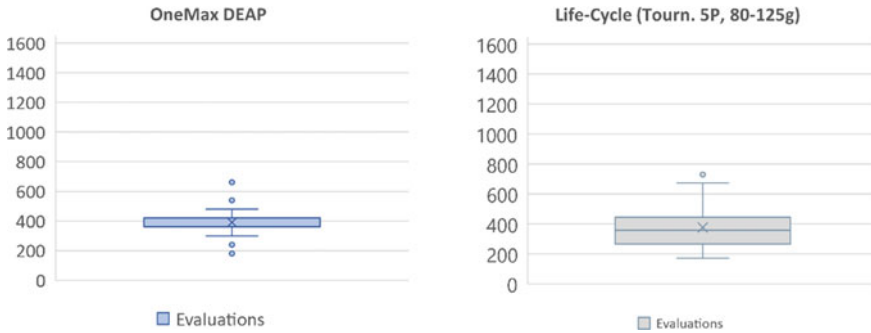


Fig. 8 Box and Whisker chart for the Experiment 4: OneMax DEAP versus Life-Cycle Tournament (5 processes with an 80 to 125 goal approval)

4 Discussion

At some crucial moments during our research, we got the feeling we were doing nature reverse engineering. One strategy was to let our imagination run wild and go for the simple solution. One of our earlier experiment's findings was to find balance, like the yin and yang, between death and reproduction.

In our experiments, we found that when solving simple problems, the cost of communication between containers can become a factor to increase the total performance time, even though, we believe that the opposite must also be true. When solving complex problems, distributing the work on multiple resources should reduce throughput time, making the communication cost negligible.

We must acknowledge the performance time of the DEAP library [6] is highly efficient and short, due in part because the processing work execution is on a single processor with multiple cores, where the communication time is nearly non-existent. In contrast, our algorithm implementation requires a minimum of five containers and a message queue server. Even though the container work has proven to be very effective and efficient [11, 22], we must consider some time was added to our experiments by the minimal delays (in the thousands of a second magnitude) we used to keep the process execution relatively random, to mimic how the life-cycle stages work in nature [17]. As future work, we could test the Life-Cycle algorithm behavior by eliminating all remaining delays.

This scheme allows for multiple parameter's fine tuning, granting us the freedom to experiment with different selection and reproduction strategies simultaneously, which will impact how fast it finds the solution and the obtained quality.

5 Conclusions

We implemented the algorithm with Docker containers by solving the OneMax problem comparing it with a traditional (sequential) GA algorithm, where it showed favorable and promising results. To further validate this work, we could use control or some more complex and demanding problem that requires computing real numbers [13, 19].

As the complexity of problems increases, it is essential to have a scalable, replicable, and fault-tolerant model that uses collaborative techniques to work in the cloud, where multiple resources will be communicating asynchronously. This research has shown that it is possible to evolve a population of individuals, similar to a Genetic Algorithm (GA), using a distributed, parallel, and asynchronous methodology.

Acknowledgements This paper has been supported in part by projects DeepBio (TIN2017–85727–C4–2–P) and TecNM Project 11356.21.

References

1. Acherjee, B., Maity, D., Kuar, A.S.: Ultrasonic machining process optimization by cuckoo search and chicken swarm optimization algorithms. *Int. J. Appl. Metaheuristic Comput. (IJAMC)* **11**(2), 1–26 (2020)
2. Armbrust, M., Fox, A., Griffith, R., Joseph, A.D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I., et al.: A view of cloud computing. *Commun. ACM* **53**(4), 50–58 (2010)
3. Back, T.: *Evolutionary algorithms in theory and practice: evolution strategies, evolutionary programming, genetic algorithms*. Oxford University Press (1996)
4. Castillo, O., Valdez, F., Soria, J., Amador-Angulo, L., Ochoa, P., Peraza, C.: Comparative study in fuzzy controller optimization using bee colony, differential evolution, and harmony search algorithms. *Algorithms* **12**(1), 9 (2019)
5. Eshratifar, A.E., Esmaili, A., Pedram, M.: Bottlenet: a deep learning architecture for intelligent mobile cloud computing services. In: *2019 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. pp. 1–6. IEEE (2019)
6. Fortin, F.A., De Rainville, F.M., Gardner, M.A.G., Parizeau, M., Gagné, C.: Deap: Evolutionary algorithms made easy. *J. Mach. Learn. Res.* **13**(1), 2171–2175 (2012)
7. García-Valdez, M., Mancilla, A., Trujillo, L., Merelo, J.J., Fernández-de Vega, F.: Is there a free lunch for cloud-based evolutionary algorithms? In: *2013 IEEE Congress on Evolutionary Computation*. pp. 1255–1262. IEEE (2013)
8. García-Valdez, M., Merelo, J.J.: Event-driven multi-algorithm optimization: mixing swarm and evolutionary strategies. In: *International Conference on the Applications of Evolutionary Computation (Part of EvoStar)*. pp. 747–762. Springer (2021)
9. García-Valdez, M., Trujillo, L., Merelo, J.J., de Vega, F.F., Olague, G.: The evospace model for pool-based evolutionary algorithms. *J. Grid Comput.* **13**(3), 329–349 (2015)
10. Krejca, M.S., Witt, C.: Lower bounds on the run time of the univariate marginal distribution algorithm on onemax. *Theoret. Comput. Sci.* **832**, 143–165 (2020)
11. Merelo, J.J., Castillo, P.A., García-Sánchez, P., de las Cuevas, P., Rico, N., García Valdez, M.: Performance for the masses: experiments with a web based architecture to harness volunteer resources for low cost distributed evolutionary computation. In: *Proceedings of the Genetic and Evolutionary Computation Conference 2016*, pp. 837–844 (2016)

12. Merelo, J.J., García-Valdez, M., Castillo, P.A., García-Sánchez, P., Cuevas, P., Rico, N.: Nodio, a javascript framework for volunteer-based evolutionary algorithms: first results (2016). [arXiv: 1601.01607](#)
13. Miikkulainen, R., Liang, J., Meyerson, E., Rawal, A., Fink, D., Francon, O., Raju, B., Shahrzad, H., Navruzyan, A., Duffy, N., et al.: Evolving deep neural networks. In: Artificial Intelligence in the Age of Neural Networks and Brain Computing, pp. 293–312. Elsevier (2019)
14. Ontiveros, E., Melin, P., Castillo, O.: High order α -planes integration: a new approach to computational cost reduction of general type-2 fuzzy systems. *Eng. Appl. Artif. Intell.* **74**, 186–197 (2018)
15. Porto, V.W.: Evolutionary programming. In: *Evolutionary Computation* 1, pp. 127–140. CRC Press (2018)
16. Rashedi, E., Nezamabadi-Pour, H., Saryazdi, S.: Gsa: a gravitational search algorithm. *Inf. Sci.* **179**(13), 2232–2248 (2009)
17. Read, K., Ashford, J.: A system of models for the life cycle of a biological organism. *Biometrika* **55**(1), 211–221 (1968)
18. Sanchez, M.A., Castillo, O., Castro, J.R., Melin, P.: Fuzzy granular gravitational clustering algorithm for multivariate data. *Inf. Sci.* **279**, 498–511 (2014)
19. Stanley, K.O., Miikkulainen, R.: Evolving neural networks through augmenting topologies. *Evol. Comput.* **10**(2), 99–127 (2002)
20. Thain, D., Tannenbaum, T., Livny, M.: Distributed computing in practice: the condor experience. *Concurr. Comput. Pract. Exp.* **17**(2–4), 323–356 (2005)
21. Valdez, F.: Swarm intelligence: a review of optimization algorithms based on animal behavior. In: *Recent Advances of Hybrid Intelligent Systems Based on Soft Computing* pp. 273–298 (2021)
22. Valdez, M.G., Guervós, J.J.M.: A container-based cloud-native architecture for the reproducible execution of multi-population optimization algorithms. *Futur. Gener. Comput. Syst.* **116**, 234–252 (2021)
23. Witt, C.: Upper bounds on the running time of the univariate marginal distribution algorithm on onemax. *Algorithmica* **81**(2), 632–667 (2019)

Evaluation and Comparison of Brute-Force Search and Constrained Optimization Algorithms to Solve the N-Queens Problem



Alfredo Arteaga, Ulises Orozco-Rosas, Oscar Montiel, and Oscar Castillo

Abstract The N-Queens problem is relevant in Artificial Intelligence (AI); the solution methodology has been used in different computational intelligent approaches. Max Bezzel proposed the problem in 1848 for eight queens in 8×8 chessboard. After that, the formulation was modified to an N-Queens problem in a chessboard. There are several ways of posing the problem and algorithms to solve it. We describe two commonly used mathematical models that handle the position of queens and restrictions. The first and easiest way is to find one combination that satisfies the solution. The second model uses a more compact notation to represent the queen's positions. This generic problem has been solved with many different algorithms. However, there is no comparison of the performance among the methods. In this work, a comparison of performance for different problem sizes is presented. We tested the Backtracking, Branch and Bound, and Linear Programming algorithms for a different number of queens, reaching 17. In addition, we present statistical comparative experimental results of the different methods.

A. Arteaga · O. Montiel (✉)

Instituto Politécnico Nacional, CITEDI-IPN, Nueva Tijuana, 1310, 22435 Tijuana, Baja California, México
e-mail: oross@ipn.mx

U. Orozco-Rosas

CETYS Universidad, Av. CETYS Universidad No. 4, Fracc. El Lago, 22210 Tijuana, Baja California, México
e-mail: ulises.orozco@cetys.mx

O. Castillo

Tijuana Institute of Technology, Calzada del Tecnológico S/N, Tomás Aquino 22414 Tijuana, Baja California, México
e-mail: ocastillo@tectijuana.mx

1 Introduction

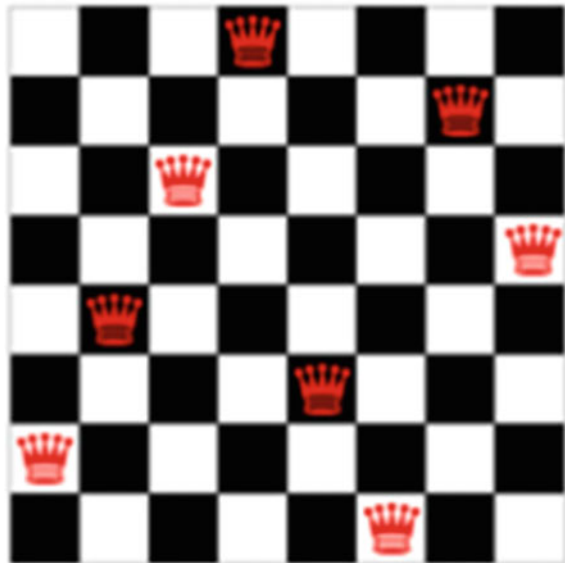
This chapter aims to present the evaluation and comparison of the brute-force search and constrained optimization algorithms to solve the N-Queens problem; specifically, we tested the Backtracking, Branch and Bound, and Linear Programming algorithms, as well as a brief review of each of the mentioned algorithms.

The general idea of the N-queen problem is to place the queens on a chessboard without them getting threatened by each other, as we can see in Fig. 1, where no queens are threatened between them.

The N-Queens problem has its origin in the year 1848, when Max Bezzel, a German chess player, proposed to place a total of 8 queens on a chessboard in such a way that no queen threatens each other [1], that is, that only one queen would meet by row, by column and by diagonal. The problem has two ways to be solved; the first and most straightforward is to find a single combination where the queens are positioned and without threat between each of them the second and most complex way is to find each of the combinations where the queens do not threaten each other. In 1850 Carl Gauss [2] claimed that there were 72 combinations when positioning eight queens on the board. However, this was wrong it was not until 1874 when James Glaisher proved 92 combinations.

The N-Queens problem has multiple methods to be solved, for example, Backtracking [3], heuristic approximations [4], genetic algorithms [5], local search [6], among others.

Fig. 1 An 8×8 chessboard with eight queens



The contribution of this work is to show the efficiency of specific algorithms that are usually used, which is important since there are no reported comparisons of computational behavior when solving the N-Queens problem for different instances.

This chapter is divided as follows. Section 2 presents the related work, which consists of a selection of representative algorithms of different techniques. Section 3, provides two mathematical models for solving the N-Queens problem. Section 4 describes the algorithms used for solving the N-Queens problem, specifically the Backtracking, Branch and Bound, and Linear Programming algorithms. Section 5 shows the experimental results. Finally, in Sect. 6, the conclusions and future work are presented.

2 Related Work

The N-Queens problem has been solved using mathematical and statistical techniques, heuristic and metaheuristic algorithms, neural networks, fuzzy logic, and quantum computing. The different applications, such as object detection [7], pixel space sampling [8], traffic control [9], deadlock prevention [10], among others, where is possible to identify the exponential processing time growth while the input (number of queens) increase, defines the N-Queens problem as a non-deterministic polynomial-time problem with a classification of NP-Complete problem. The N-Queens problem has a computational complexity defined as $O(n^n)$ when it is solved with the Backtracking algorithm, which is a brute force approach. In that sense, several approaches have been proposed to solve the N-Queens problem efficiently and effectively.

In the heuristic and metaheuristic algorithms, we can find the work presented by Al-Gburi et al. [11], where the hybridization of the bat algorithm and the genetic algorithm to solve the N-Queens problem is presented. In that work, the results show that the proposal can solve the N-Queens problem with an input of 8, 20, 50, 100, and 500 queens. In [12], Jain and Prasad present a genetic algorithm to solve the N-Queens problem by the use of an advanced mutation operator, where two instances were tested, with 8 and 50 queens against the classical genetic algorithm, permutation PSO (Particle Swarm Optimization), and swarm refinement PSO, where the proposed algorithm obtained the best results. The PSO algorithm is one of the most efficient and effective swarm intelligence-based algorithms; in that sense, in [13], a research to inspect the effectiveness of the PSO to produce an optimal solution for the N-Queens problem is presented.

Concerning the employment of heuristic and metaheuristic algorithms with parallel computing, Cao et al. [14] presented a parallel implementation on CPU and GPU (Graphics Processing Unit) of the candidate solution evaluation algorithm for the N-Queens problem; in that work, three schemes are applied to the simulated annealing algorithm for problems with a smaller size than 3,000 queens. Another example is presented in [15], where, Jianli et al. employed a parallel genetic algorithm for the N-Queens problem based on a message passing interface using a GPU.

An alternative GPU implementation was presented by Janssen and Liew [16]. In their work, the authors present acceleration of the genetic algorithm using a GPU, where some modifications of the evolutionary operations are implemented to fit the GPU architecture, in that way it is possible to speed up the computation of the solutions.

In the neural networks and fuzzy logic approaches, it can be found the developed neuro-fuzzy architecture for solving the N-Queens problem presented by Nunes et al. [17]. In that work, a modified Hopfield neural network and a valid-subspace technique were employed to create a subspace that contains only the solutions that represent feasible solutions to the problem analyzed. Another contribution employing the Hopfield neural network to solve the N-Queens problem was presented by Waqas and Bhatti [18]. Their proposal presents a solution for the variation of the $N + 1$ -Queens problem, that variation possibilities the solution for large values of N . In [19] Funabiki et al. present a maximum neural network approach for the N-Queens problem. Lakshmi and Muthuswamy [20] presented a proposal to solve the N-Queens problem using a predictive neural network and incorporating the AHP-Fuzzy algorithm in order to reduce the neural network prediction time; with this incorporation, the speedup is increased.

In the quantum computing field, where the quantum computers are based on quantum physics principles to solve the N-Queens problem is necessary to define a quantum state for all the possible solutions found, followed by the application of the Hadamard gate, to produce a quantum register. Once the quantum register is completed, two unitary operators are combined to employ Grover's algorithm through an oracle, which evaluates a function $f_w(x)$ that verifies the proposed solution. For quantum computing implementations, we found the work presented by Souza and Mello [21]. Here, the Grover's algorithm is employed to identify all the safe places where it is possible to place a queen and to get a combination that satisfies the restrictions. Another approach is presented in [22], where a quantum-inspired differential evolution algorithm is proposed. In that work, an hybridization of the differential evolution algorithm that employs the differential evolution operator as an alternative to the classical genetic algorithm's mutation operator and the quantum genetic algorithm has been used to solve the N-Queens problem obtaining better results either in terms of computation time or in term of fitness evolution.

3 Mathematical Models

The N-Queens problem is constrained because the queens need to be safe, and this condition is only satisfied if and only if we have one queen on a row, one queen on an arrow, and one queen in any diagonal. In this way, the queens are not threatened between them. There are several algorithmic approaches to solve this problem. In [23] the row-based, column-based, and several issues regarding problem representation are provided in detail [24]. Provides a survey of known results and research areas for the N-Queen problem; here, the authors describe methods and explain some theorems that help to understand the problem mathematically better. This section explains two

mathematical models chosen according to the software implementations that we achieved.

3.1 Mathematical Model No. 1

This model is the most general and easier to implement. It uses a binary variable x_{ij} ; where i is a column, and j is a row, to represent the existence of a queen in the square (i, j) . In this way, if $x_{ij} = 1$ there is a queen in that square, and if $x_{ij} = 0$ there is no queen. This model imposes restrictions for the rows, columns, and positive and negative diagonals. We say that a diagonal is positive if we can trace a positive angle through the squares in the cartesian plane contrary, the opposing diagonals form a negative angle.

The row restriction prevents the existence of two queens in the same row which can be verified by adding the values of the squares x_{ij} of a row. It is expected to obtain “1” if there is only one queen in the row. This restriction can be formulated as follows,

$$\sum_{j=1}^n x_{ij} = 1 \forall i = \{1, \dots, n\} \quad (1)$$

Similarly, the column restriction prevents that two or more queens from being placed on a column. The restriction is as follows,

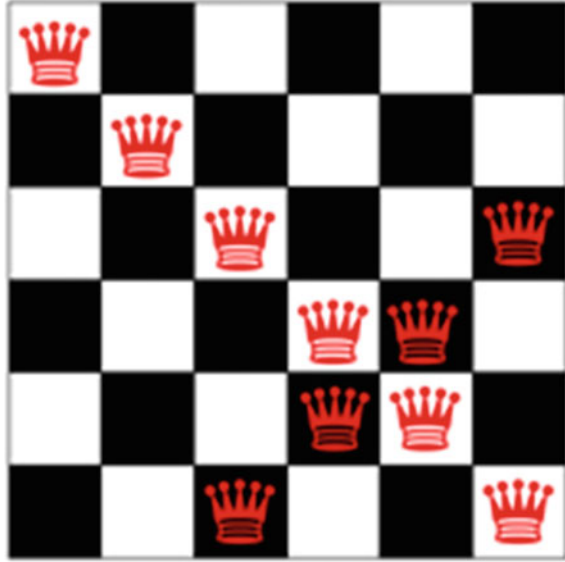
$$\sum_{i=1}^n x_{ij} = 1 \forall j = \{1, \dots, n\} \quad (2)$$

It is more complicated for the diagonals since it is necessary to identify if two or more queens are on the same positive or negative diagonal. Figure 2 shows a 6×6 chessboard describing the positive and negative diagonals. Using this figure, we can verify that it is possible to determine which queens are in the same positive diagonal using,

$$k = i + j \quad (3)$$

By using (3), we can say that all the pairs (i, j) with the same value of k are in the same positive diagonal, for example, the pairs (4,5) and (3,6) are in the same diagonal, and their k value is 9. Just like that, we can see that for this chessboard, the range of k varies from $\{2, \dots, 12\}$ considering all the diagonals. In general, we can say that for a chessboard of size $n \times n$ the range varies from $\{2, \dots, 2n\}$. Depending on the implementation algorithm, it is possible not to consider the squares of the corners; hence, the range of k will vary from $\{3, \dots, 2n - 1\}$.

Fig. 2 Chessboard of size 6×6 . There are four queens in the positive column and six queens in the negative column. The index i identifies the row number and j the column number. A position is a pair (i, j)



For the case where all the squares of the chessboard are considered, we have the expression (4). For the case where the squares are not considered expression (4) is modified only in the valid range of case; i.e., we will write $\forall k = \{3, \dots, 2n - 1\}$.

$$\sum_{i=1}^n \sum_{j=1}^n x_{ij} \leq 1 \forall k = \{2, \dots, 2n\} \quad (4)$$

Similarly, for the negative diagonals, the k value is calculated by

$$k = i - j \quad (5)$$

And all the pairs (i, j) with the same k value will be in the same negative diagonal. The k range values for the Fig. 2 will vary from $\{-5, \dots, 5\}$ considering all the squares of the chessboard. If we did not consider the corner squares, the k range would vary from $\{-4, \dots, 4\}$. For the general case, the negative diagonal restriction produces values of k in the range of $\{-n + 1, \dots, n + 1\}$, which can be expressed by,

$$\sum_{i=1}^n \sum_{j=1}^n x_{ij} \leq 1 \forall k = \{-n + 1, \dots, n + 1\} \quad (6)$$

The searching space of this model is $2^{n \times n}$. For Fig. 2, where $n = 6$, the searching space has 68,719,476,736 possible combinations. For $n = 17$, which is the biggest case that we tested.

3.2 Mathematical Model No. 2

This model only uses the variable x_i to represent all the rows where the queens can be placed on the i position in the domain determined as $\{1, \dots, n\}$; the restrictions only consider the columns and diagonals (positive and negative). All the rows are evaluated by default because a value is assigned automatically since two or more queens cannot be on the same row.

The column restriction does not allow two queens in the same column. This can be verified by checking the variable values if they are different, the restriction is fulfilled. Otherwise, it is not satisfied. This restriction can be written as follow,

$$x_i \neq x_j \forall i \neq j \text{ and } j \in N \quad (7)$$

Analogously

$$x_i - x_j \neq 0 \forall i \neq j \text{ and } j \in N \quad (8)$$

Similar to the first model, the diagonals, positive and negative, can have only one queen. It is known that all the diagonals have angles of $\pm 45^\circ$, depending on it is positive or negative, and using the model definition where the variable value indicates the column and the index indicates the row, we can determine if two queens are in the same diagonal. Viewing the position of the queens as points in the Cartesian plane; say $Q_i(x_i, y_i)$ for the first queen, and $Q_j(x_j, y_j)$ for the second queen, and writing this using the variable nomenclature of the model, we have $Q_1 = (x_1, i)$ and $Q_2 = (x_j, j)$; then $\Delta x = x_j - x_1$ and $\Delta y = j - i$. The tangent of an angle can be calculated using $\frac{\Delta y}{\Delta x}$, and we know that $\tan^{-1} 45 = 1$, therefore, $\Delta x = \Delta y$. Therefore, the diagonal restriction can be written as,

$$|x_i - x_j| \neq |i - j| \forall i \neq j \text{ and } j \in N \quad (9)$$

4 N-Queens Problem Solution Algorithms

This section explains the algorithms that we used to solve the N-Queens problem: they are, Backtracking, Branch and Bound, and Linear Programming algorithms.

4.1 Backtracking Algorithm

This algorithm performs a recursive search on trees. It performs a depth-search, branch by branch exploring all the node subbranches, selecting valid values for one

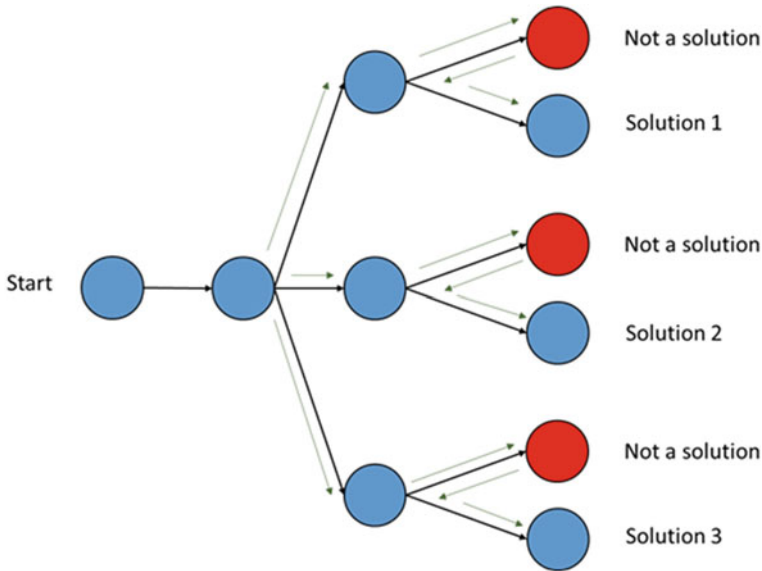


Fig. 3 Backtracking algorithm representation

variable at a time otherwise, it backtracks. Thus, the procedure can find several valid solutions at the branches, finding success nodes. If we find an invalid solution in any part of the branch, it is tagged as a dead-end, and the searching process is finished. Hence, the general idea of this algorithm is to build solutions incrementally, removing those solutions that fail to satisfy the imposed constraints. Figure 3 describes the searching process of the backtracking algorithm on a binary tree.

To solve the N-Queens problem, we used the mathematical model no. 1 explained in Sect. 3.1. Here, it is necessary to consider partial solutions for determining which combinations are valid and which ones are invalid, so if a partial solution is detected in a branch, the algorithm will perform searching in the same branch until a valid combination is found or we reach a dead-end. In this way, the algorithm will search in all the branches moving forward or backward to place the queens without attacking each other [25].

Güldal, Baugh, and Allehaibi [26] in 2006 solved the problem using the Backtracking algorithm. They realized that it was very efficient; however, a bigger chessboard size implies having more queens increasing the processing time exponentially. Table 1 shows some processing times using this algorithm for chessboard sizes up to nine queens.

As mentioned, the Backtracking algorithm needs to check all the possible candidate solutions in each branch. Hence it is a brute force search method where it is necessary to enumerate all the possible candidates that can be the solution to the problem. For example, applying it to the N-Queens problem is necessary to find all the possible combinations in a chessboard of size $N \times N$.

Table 1 Backtracking processing time for obtaining different results

Number of queens	Time
1	≈0 s
2	≈0 s
3	≈0 s
4	≈0 s
5	≈0 s
6	0.03 s
7	0.17 s
8	0.74 s
9	3.57 s
10	17.81 s
11	96.86 s
12	558.07 s
13	3,599.7 s

A block diagram of the algorithm is shown in Fig. 4. It is divided into three parts, the first part is dedicated to variables initialization like the number of queens and commands to measure the execution time. The objective of the second part is to register all the obtained solutions and control the queen's placement on the board. Finally, the third is devoted to determining if the placed queen's position is safe or not, with this it is possible to discriminate the threatened positions using the Backtracking algorithm.

4.2 Branch and Bound Algorithm

This algorithm is used to solve combinatorial optimization problems, works searching solutions in a finite combination tree [27], and with an alternated sequence that generates another partial sequence, evaluating partial classifications determining optimum solutions [28]. The algorithm takes care of finding the optimum solution through combinations, and it can be represented as a combinational tree, as shown in Fig. 5. Where each branch has a partial result, this branch will be marked as a solution. Otherwise, it will be omitted and the algorithm will change from branch until it gets a solution, optimizing execution time for getting all the combinations for the problem.

The Branch and Bound algorithm works by applying a Backtrack only when the row does not have options to evaluate. Thus, that branch is eliminated, and this generates an optimization [29].

Harold and Janice Stone did an exercise where it was registered the size of the combinational tree generated to give a solution to the N-Queens solution [29]. They

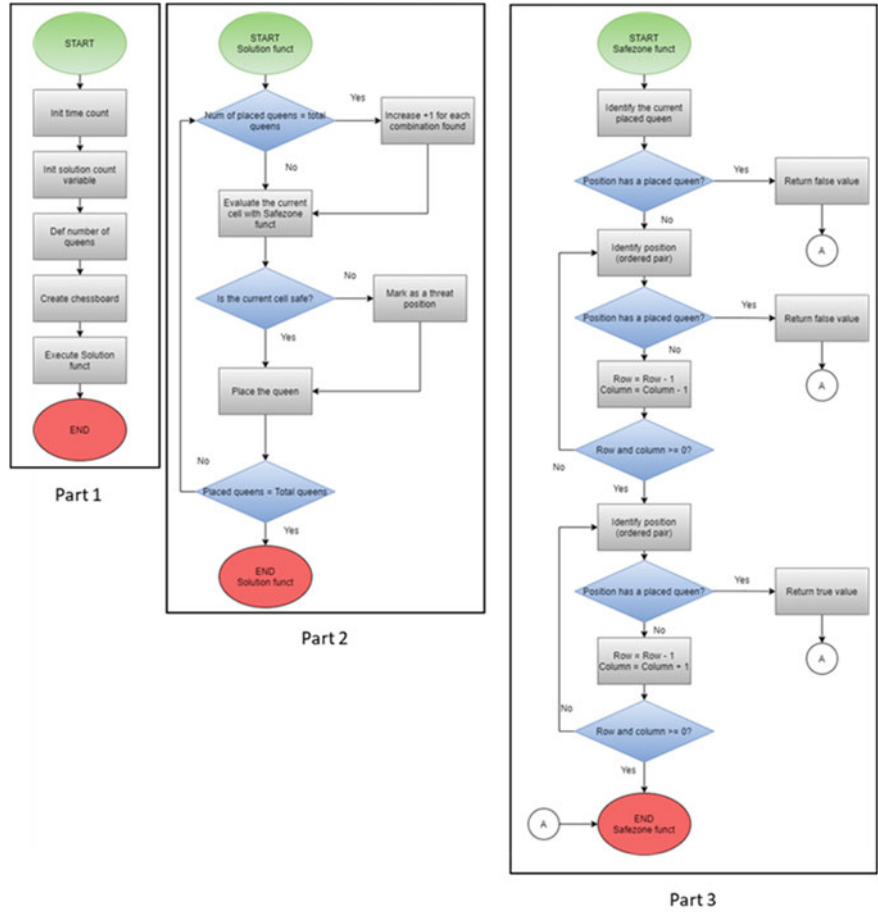


Fig. 4 Block diagram representation of the Backtracking algorithm

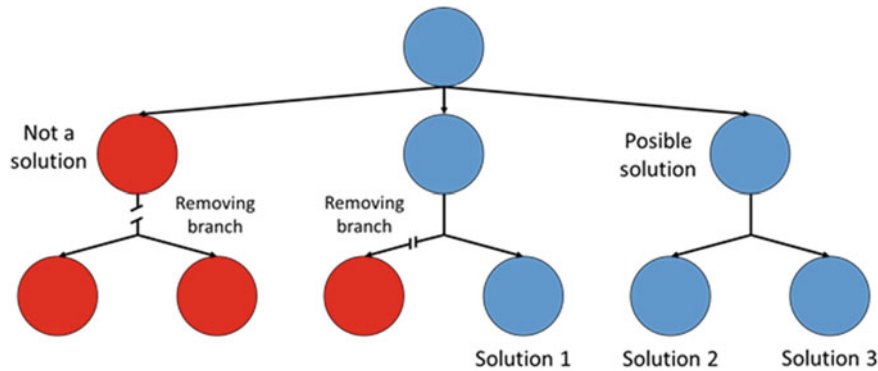


Fig. 5 Branch and Bound algorithm representation using a tree

Table 2 Branch number for N-Queens problem using lexicographic and restrictions variants

Number of queens	Lexicographic size	Restriction size
7	552	416
8	2057	1,415
9	8,394	5,610
10	35,539	20,863
11	166,926	89,670
12	853,189	432,103
13	4,674,890	2,230,980

used two variants, a lexicographic and restrictions. The results obtained are shown in Table 2.

We divided the Branch and Bound algorithm into a diagram consisting of five parts, as shown in Fig. 6. The first one is for setting the initial values for the solution counter and the number of queens to be placed. The second part will place the queens on the board and count the number of solutions found. In the third part, the algorithm alternates between the rows where the queens are going to be placed in a safe position. In the fourth part, the positions that are a threat to the queens are identified. Finally, the last part dismisses the threatening positions for the queens.

In the Branch and bound algorithm, we used the mathematical model no. 1, explained in Sect. 3.1, to solve the N-Queen problem.

4.3 Linear Programming Algorithm

Linear Programming is a methodology initiated by George B. Dantzig, John von Neumann, Leonid Kantorovich, and others in the 1940s. This mathematical methodology includes modeling techniques to formulate real-world problems into linear programs and algorithms for numerically solving linear programs. Linear Programming is employed in several areas such as administration, economy, science, business, engineering, among others.

Linear Programming is capable to solve optimization problems to minimize or maximize a linear objective function subject to linear equality or inequality constraints. It is composed of two elements, the objective function, and the restrictions, the restrictions limit the results using decision variables [30].

To use Linear Programming for the N-Queens problem we employ the formulation described in Sect. 3.2. Given an n number of queens and an $n \times n$ chessboard, we define the total of n queens as follows,

$$\sum_{i=1}^n \sum_{j=1}^n X[i, j] = n \quad (10)$$

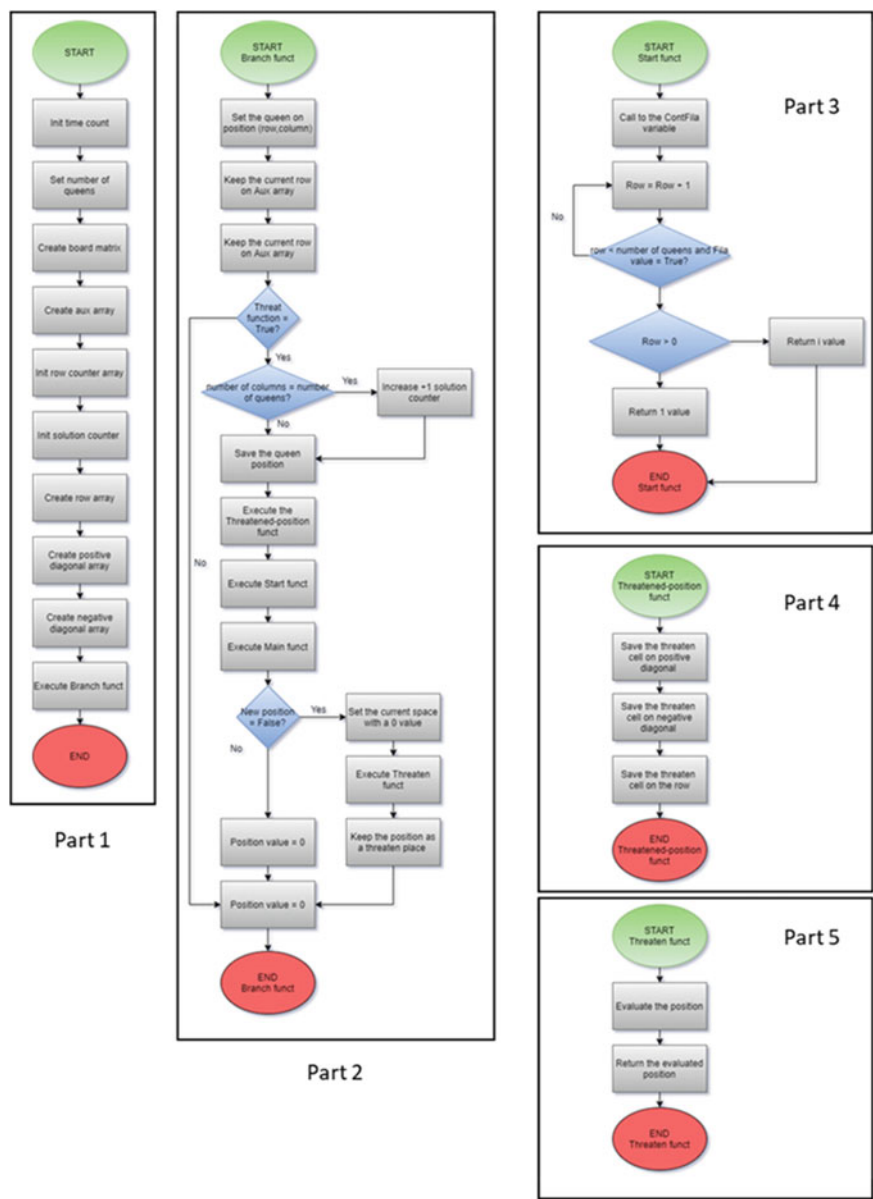


Fig. 6 Block diagram representation of the branch and bound algorithm

where the following rules are considered. Restriction 1, at least a queen needs to be placed on a column.

$$\forall i = 1, \dots, n \sum_{j=1}^n X[i, j] \leq 1 \quad (11)$$

Restriction 2, at least a queen needs to be placed on a row.

$$\forall j = 1, \dots, n \sum_{i=1}^n X[i, j] \leq 1 \quad (12)$$

Restriction 3, at least a queen needs to be placed on a positive diagonal.

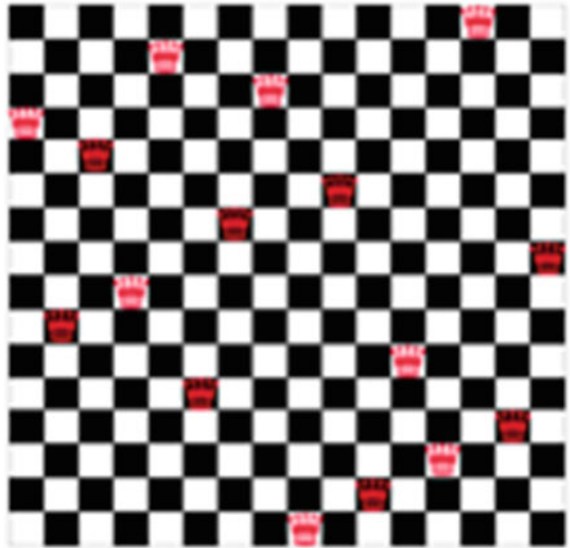
$$\forall k = 2 - n, \dots, n - 2 \sum_{i,j \in 1, \dots, n, i-j=k} X[i, j] \leq 1 \quad (13)$$

Restriction 4, at least a queen needs to be placed on a negative diagonal.

$$\forall k = 3, \dots, 2n - 1 \sum_{i,j \in 1, \dots, n, i+j=k} X[i, j] \leq 1 \quad (14)$$

In [31], Al-Rudaini applied the restrictions previously stated to solve the N-Queens problem with an $N = 16$ using Linear Programming. Figure 7 shows a result employing this conditions.

Fig. 7 The N-Queens problem for sixteen queens was solved with Linear Programming



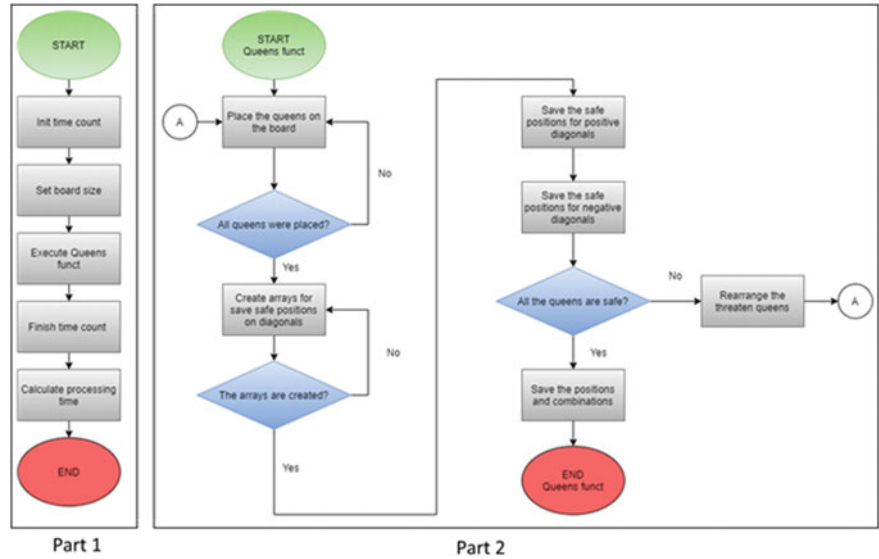


Fig. 8 Block diagram representation of the Linear Programming algorithm

To implement the Linear Programming algorithm to solve the N-Queens problem, we propose to divide the algorithm into two parts, as it is shown in Fig. 8. The first part establishes the number of queens and the chessboard size. The second part named “Queens funct” determines the best position of the queens that comply with the established restrictions.

5 Experiments and Results

This section presents the experimental results of the three tested algorithms for solving the N-Queen problem: Backtracking, Branch and Bound, and Linear Programming algorithms. These algorithms were tested on a 16 core Intel Xeon Gold 5218 CPU @ 2.3 GHz with 512 GB of RAM running Ubuntu 20.04.2 LTS, and they were executed 20 times to obtain the mean and the standard deviation of the execution time for a different number of queens.

5.1 Experiment 1: Backtracking

The results are registered in Table 3. It can be observed an exponential increase in the execution time after solving the problem for 11 queens. Before this instance,

Table 3 Execution time for the N-Queens problem using the Backtracking algorithm

Number of queens	Execution time (mean)	Standard deviation
4	0.00007 s	1.39046E-20
5	0.0002 s	2.78092E-20
6	0.0008 s	1.11237E-19
7	0.003405 s	2.23607E-05
8	0.015385 s	0.000113671
9	0.074975 s	0.000990415
10	0.22796 s	0.017698718
11	1.199305 s	0.021038798
12	6.877155 s	0.069985235
13	42.3444 s	0.744777032
14	278.702635 s	6.718637124
15	1896.905275 s	35.63734744
16	13,887.82939 s	158.5726299
17	109,411.7678 s	4167.875636

the times required for solving the problem were very close to 0 seconds. For the standard deviation, it can be observed that the data is not far enough from the mean until the algorithm worked with 14 queens where the standard deviation increases exponentially in other words, the precision of the algorithm is good, but while the number of queens increases the data is moving away from the mean.

5.2 Experiment 2: Branch and Bound

It is considered a Backtracking variant however, it has many optimizations that reduce the processing time, allowing it to obtain the results in a shorter time. It works by generating a solution tree, and each branch makes it possible to obtain a possible solution. The algorithm performs an analysis to determine if the actual branch is optimal; otherwise, it gets trim, discarding that path as a possible solution, reducing the number of combinations, and taking advantage of the resources in a better way.

The obtained results were registered in Table 4 it can be observed an exponential increase in the execution time after solving the problem for 12 queens; before this instance, the time required for solving the problem was very close to zero seconds. Regarding the standard deviation, it can be observed that the data is not far enough from the mean until the algorithm work with 15 queens, where the standard deviation increases exponentially, almost the same as the Backtracking algorithm.

The registered results in Tables 3, 4, and 5 were used to analyze which one had the best execution time for all the tested algorithms. Starting with Linear Programming, we can observe that the execution time grows exponentially beginning from $N = 14$,

Table 4 Execution time for the N-Queens problem using Branch and Bound algorithm

Number of queens	Execution time (mean)	Standard deviation
4	0.0004 s	5.56184E-20
5	0.0005 s	2.22474E-19
6	0.0009 s	1.11237E-19
7	0.00243 s	4.70162E-05
8	0.008565 s	8.75094E-05
9	0.03553 s	0.000231926
10	0.104655 s	0.01866814
11	0.471255 s	0.013809245
12	2.548845 s	0.021257432
13	14.530725 s	0.110244355
14	89.19125 s	0.622579839
15	583.959185 s	3.048646538
16	4115.949215 s	40.31555712
17	30,531.33199 s	1152.404159

Table 5 Execution time for the N-Queens problem using the linear programming algorithm

Number of queens	Execution time (mean)	Standard deviation
4	0.006335 s	8.12728E-05
5	0.01257 s	0.000126074
6	0.01768 s	0.000119649
7	0.035475 s	0.007626883
8	0.11619 s	0.009265039
9	0.524145 s	0.012362783
10	2.00735 s	0.011765628
11	11.46305 s	0.05936269
12	63.38689 s	0.549429424
13	534.99516 s	1.906083907
14	—s	—
15	—s	—
16	—s	—
17	—s	—

which means, from 14 onwards is possible to observe the growth; for sixteen and seventeen queens, we got no result because of processing time was enormous.

From the results obtained with the Backtracking algorithm, we can observe the notable exponential growing starting on an N value of 15, as we can observe, it passes from 359 to 2,457 s, which let us identify the behavior of the computational complexity and the number of resources needed to perform this operation.

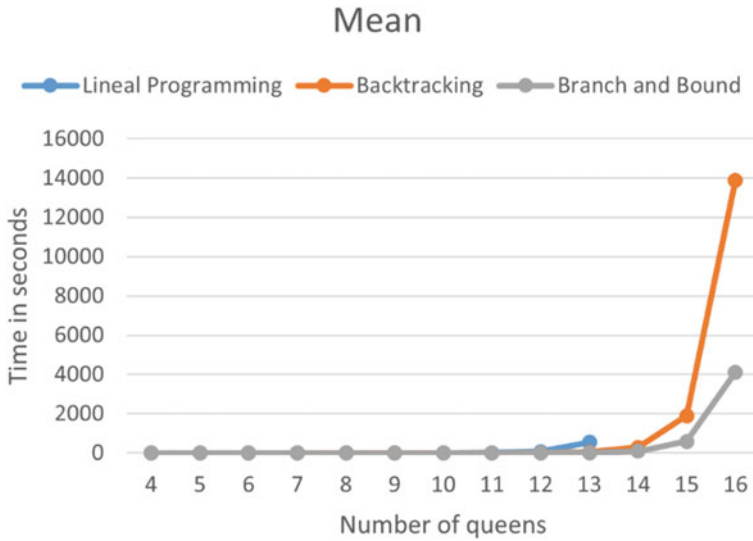


Fig. 9 The results show the mean time obtained from Linear Programming (blue), Backtracking (orange), and Branch and Bound (gray) algorithms

With the Branch and Bound algorithm, we can observe that it is the fastest and best-optimized algorithm because the execution time is shorter than the other two algorithms; as we can see, for $N = 16$ and $N = 17$ it took 5,821 and 42,613 s, respectively which is a shorter time than the one obtained using the Backtracking algorithm, shown in Fig. 9.

5.3 Experiment 3: Linear Programming

Linear Programming is a mathematical method used to optimize an objective function with restrictions that limit the values that the decision variables can take. In that sense, the N-Queens problem can be defined as follows.

$$X_{ij} = f(N) = \begin{cases} 1 & \text{there is no threat in } (i, j) \\ 0 & \text{otherwise} \end{cases} \in \{1, 0\} \quad (15)$$

where:

- X_{ij} is the ordered pair where the queens are placed.
- For the first case: 1 represents the queen on the position (i, j) .
- And, for the second case: 0 represents the position (i, j) where is not possible to place a queen.

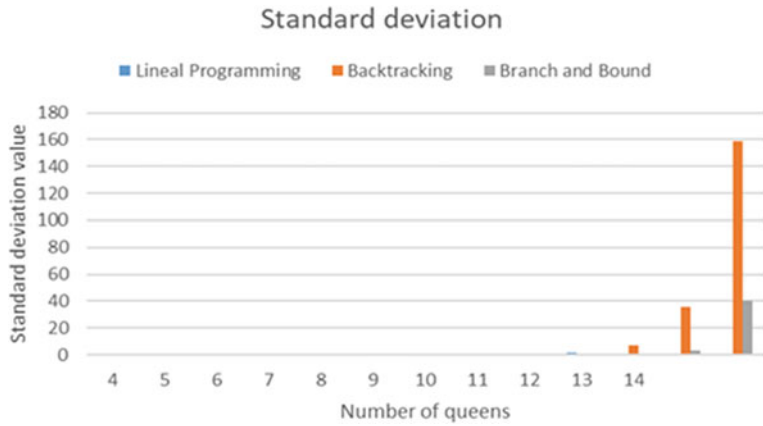


Fig. 10 The standard deviation for the N-Queens problem with Backtracking, Branch and Bound and Linear Programming algorithms

The obtained results in terms of execution time with the Linear Programming algorithm are presented in Table 5. An exponential increase in the execution time can be observed after solving the problem for $N = 10$ queens before this instance, the time required for solving the problem is almost zero (≈ 0 s). For the standard deviation, it can be observed that the results are not far enough from the mean, in other words, the precision of the Linear Programming algorithm is good compared with the Backtracking and Branch and Bound algorithms. However, the linear programming algorithm requires a huge amount of processing time when $N > 13$ queens, for this reason, the solutions for 14, 15, 16, and 17 queens couldn't be obtained in this implementation.

As we can see in Table 5, the standard deviation indicates how scattered the results in terms of execution time are concerning the mean. Figure 10 shows the level of accuracy that each algorithm has when is executed at least 20 times for each instance. The results in Fig. 10 shows that the Backtracking algorithm presents the smallest accuracy and that the Branch and Bound algorithm has the best accuracy.

6 Conclusions and Future Work

The present work evaluated three different algorithms to solve the N-Queens problem brute force and constrained optimization algorithms were tested and evaluated. With the obtained results, we concluded that the Branch and Bound algorithm is the faster one because it can solve the N-Queens problem and get all the combinations that satisfy the restrictions. The Backtracking algorithm also solved the problem; however, the required time to achieve it is longer than the Branch and Bound algorithm. The Linear Programming algorithm is less optimal regarding time for a more

significant number of queens some results could not be obtained in a reasonable, that is the case of fifteen, sixteen, and seventeen queens. The three algorithms show an exponential behavior that is more notable when they work with thirteen queens or more. This allows us to confirm that the input size increases the computational complexity, which will affect the necessary time to get the expected results.

In future work, we will test other methods to solve the N-queen problem, such as heuristic and metaheuristic algorithms, among others, including quantum algorithms, to perform a complexity analysis. In addition, other metaheuristics could be used, like [32–34].

References

1. Rivin, I., Vardi, I., Zimmermann, P.: The n-queens problem. *Am. Math. Mon.* **101**(629–639), 08 (1994)
2. Letavec, C., Ruggiero, J.: The queens problem - delta. *INFORMS Trans. Educ.* **2**(101–103), 05 (2002)
3. Mu, S.C.: Calculating a backtracking algorithm: an exercise in monadic program derivation (2021)
4. Osaghae, E.: Solution to n-queens problem: heuristic approach. *Trans. Mach. Learn. Artif. Intell.* **9**, 26–35 (2021)
5. Sharma, S., Jain, V.: Solving n-queen problem by genetic algorithm using novel mutation operator. *IOP Conference Series: Materials Science and Engineering* **1116**, 012195 (2021)
6. Andov, L.: Local search analysis n-queens problem. In: Griffith University, School of Information and Communication Technology, Intelligent Systems – 2802ICT (2018)
7. Stojkoska, B., Davcev, D., Vladimir, T.: N-queens-based algorithm for moving object detection in distributed wireless sensor networks. In: ITI2008 - 30th International Conference on Information Technology Interfaces, pp. 899–904 (2008)
8. Wang, C.-N., Yang, S.-W., Liu, C.-M., Chiang, T.: A hierarchical decimation lattice based on n-queen with an application for motion estimation. *IEEE Signal Process. Lett.* **10**(8), 228–231 (2003)
9. Sosic, R., Gu, J.: A polynomial time algorithm for the n-queens problem. *ACM SIGART Bull* **1** (1996)
10. Erbas, C., Tanik, M., Aliyazicioglu, Z.: Linear congruence equations for the solutions of the n-queens problem. *Inf. Process. Lett.* **41**, 301–306 (1992)
11. Al-Gburi, A., Naim, S., Boraik, A.: Hybridization of bat and genetic algorithm to solve N-queens problem. *Bull. Electr. Eng. Inform.* **7**, 626–632 (2018)
12. Jain, V., Prasad, J.S.: Solving N-queen problem using genetic algorithm by advance mutation operator. *Int. J. Electr. Comput. Eng.* **8**, 4519–4523 (2018)
13. Ahmed, A., Shah, S., Kamran, A., Sani, A., Bukhari, H.S.: Particle swarm optimization for N-queens problem. *J. Adv. Comput. Sci. Technol.* **1** (2012)
14. Cao, J., Chen, Z., Wang, Y., Guo, H.: Parallel implementations of candidate solution evaluation algorithm for N-queens problem. *Complexity* **2021**, 1–15 (2021)
15. Jianli, C., Zhikui, C., Yuxin, W., He., G.: Parallel genetic algorithm for N-Queens problem based on message passing interface-compute unified device architecture. *Comput. Intell.* (2020)
16. Janssen, D.M., Liew, A.W.: Acceleration of genetic algorithm on GPU CUDA Platform. In: 20th International Conference on Parallel and Distributed Computing, Applications and Technologies (PDCAT), 2019, pp. 208–213 (2019)
17. Nunes, I., Ulson, J., Nunes, A.: Development of neurofuzzy architecture for solving the N-Queens problem. *Int. J. Gen Syst* **34**(6), 717–734 (2005)

18. Waqas, M., Bhatti, A.: Optimization of N+1 queens problem using discrete neural network. *Neural Netw. World.* **27**, 295–308 (2017)
19. Funabiki, N., Takenaka, Y., Nishikawa, S.: A maximum neural network approach for N-queens problems. *Biol. Cybern.* **76**, 251–255 (1997)
20. Lakshmi, A.J., Muthuswamy, V.: A predictive context aware collaborative offloading framework for compute-intensive applications. *J. Intell. Fuzzy Syst.* **40**, 1–12 (2020)
21. de Souza, F., de Mello, F.: N-queens problem resolution using the quantum computing model. *IEEE Lat. Am. Trans.* **15**(3), 534–540 (2017)
22. Draa, A., Meshoul, S., Talbi, H., Batouche, M.: A quantum-inspired differential evolution algorithm for solving the N-queens problem. *Int. Arab. J. Inf. Technol.* **7**, 21–27 (2010)
23. Nadel, B.: Representation selection for constraint satisfaction: a case study using n-queens. *IEEE Expert.* **5**, 16–23 (1990)
24. Bell, J., Stevens, B.: A survey of known results and research areas for N-queens. *Discret. Math.* **309**, 1–31 (2009)
25. Kondrak, G., van Beek, P.: A theoretical evaluation of selected backtracking algorithms. *Artif. Intell.* **89**, 365–387 (1995)
26. Güldal, S., Baugh, V., Allehaibi, S.: N-queens solving algorithm by sets and backtracking (2016)
27. Hazama, K., Ebara, H.: Branch and bound algorithm for parallel many-core architecture. 272–277 (2018)
28. Koontz, W., Narendra, P., Fukunaga, K.: A branch and bound clustering algorithm. *IEEE Trans. Comput.* **C-24**, 908–915 (1975)
29. Stone, H., Stone, J.: Efficient search techniques—an empirical study of the n-queens problem. *IBM J. Res. Dev.* **31**, 464–474 (1987)
30. Nasira, G., Kumar, S.: A backpropagation neural network implementation for hybrid algorithm in solving integer linear programming problems. In: *Computing Communication and Networking Technologies (ICCCNT)*, pp. 1–6 (2010)
31. Al-Rudaini, M.: N-queens problem solving using linear programming in gnu linear programming kit (GLPK) (2016)
32. Olivas, F., Valdez, F., Castillo, O., Gonzalez, C.I., Martinez, G., Melin, P.: Ant colony optimization with dynamic parameter adaptation based on interval type-2 fuzzy logic systems. *Appl. Soft Comput.* **53**, 74–87 (2017)
33. Olivas, F., Valdez, F., Castillo, O., Melin, P.: Dynamic parameter adaptation in particle swarm optimization using interval type-2 fuzzy logic. *Soft Comput.* **20**(3), 1057–1070 (2016)
34. Olivas, F., Valdez, F., Melin, P., Sombra, A., Castillo, O.: Interval type-2 fuzzy logic for dynamic parameter adaptation in a modified gravitational search algorithm. *Inf. Sci.* **476**, 159–175 (2019)

Performance Comparative Between Single and Multi-objective Algorithms for the Capacitated Vehicle Routing Problem



David Bolaños-Rojas, Luis Chávez, Jorge A. Soria-Alcaraz, Andrés Espinal, and Marco A. Sotelo-Figueroa

Abstract Heuristic optimization algorithms are an important and relevant area in Artificial Intelligence. The objective of these algorithms is to find the best solution to a problem from a set of available and feasible alternatives using a guided search based on an empirical criterion at execution time. There is a wide range of algorithms that can be applied to a specific optimization problem with single or multiple objectives. Usually, the novel researcher must deal with the selection of an algorithm to solve their problem. In this work, we compare the performance of two types of optimization algorithms; namely, single objective and multi-objective algorithms to gather information about which type of strategy achieves the best result over the capacitated vehicle routing problem. The aim of this work can be summarized as follows: to find if there exist qualitative differences in the solutions obtained by the usage of mono and multi-objective algorithms and to identify which type of strategy achieves the best quantitative solution.

1 Introduction

The Vehicle Routing Problem (VRP) is a combinatorial problem concerned with finding solutions to a goods distribution situation [1]. It tries to find the set of optimal routes in which a delivery entity or vehicle can traverse a set of delivery points once and return to the origin point or depot. It is also a generalization of the famous travelling salesman problem. Many ways of solving this problem exist, including dynamic programming and the use of evolutionary algorithms.

An instance of the VRP is the Capacitated Vehicle Routing Problem (CVRP), which consists of also considering the additional constraint of capacity of each delivery entity or vehicle. CVRP is also commonly paired with a distance constraint to also assess the cost of transit of the goods.

D. Bolaños-Rojas · L. Chávez · J. A. Soria-Alcaraz (✉) · A. Espinal · M. A. Sotelo-Figueroa
Departamento de Estudios Organizacionales, Division de Ciencias Económico Administrativas,
Universidad de Guanajuato, Guanajuato, México
e-mail: jorge.soria@ugto.mx

2 Important Concepts

2.1 Continuous Optimization

Optimization is the search of the most optimal or the best answer to a particular problem [2]. Therefore, continuous optimization is search for the optimal solution to a problem which can be modelled through a continuous-valued mathematical function.

2.2 Multi-objective Optimization

Multi-objective optimization is very similar to continuous optimization. The main difference is that the algorithms utilized will optimize more than one function. This yields a frontier of optimal values know as Pareto Front. The Pareto Front is a set of solutions for which the increase of fitness to one function, requires the diminishing of fitness in another one.

2.3 Multi-objective Genetic Algorithms

For this research, two novel multi-objective algorithms with known performance were chosen to experiment with. The chosen algorithms are NSGA-II and WASFGA.

2.4 Nsga-Ii

The Non-dominated Sorting Genetic Algorithm II is a multi-objective genetic algorithm that utilizes a non-dominated sorting approach with a $O(MN^2)$ complexity. It also utilizes a selection operator that combines the parents and offspring population and selects the best N solutions (according to the fitness functions) for recombination. It can find a wide spread of solutions along the real Pareto front of the problems according to previous statistical analysis [3].

2.5 Wasfga

The Global weighting achievement secularizing function genetic algorithm is an aggregation-based genetic algorithm. Its main objective is approximating the whole Pareto front of the functions provided. This algorithm classifies solutions at each

iteration into different fronts and then grades them with an achievement secularizing function. This algorithm also allows the decision-maker to weigh in their own observations on the heuristics of the fitness functions [4].

2.6 Library

jMetalPy is a python port of the jMetal framework [5]. It contains classes, functions and methods that ease the process of defining and solving optimization problems utilizing several available metaheuristics. By working on single solutions, it is also capable of employing its algorithms for mono-objective problem solving.

jMetalPy contains several base classes for use as parents in new problem classes. The relevant base class for this paper is Permutation Problem. This class is utilized as a base for combinatorics problems that can be solved by permuting the elements from a solution. Implementing a class descending from a Permutation Problem, required overwriting methods to query the name, to create an initial solution and to evaluate and create a new solution.

jMetalPy also contains a predefined method to run the experiments on an instance, specifying the probability and techniques parameters of the algorithm for crossover, mutation and selection. For this research, the methods used included the Permutation Swap Mutation operator, PMX Crossover operator, CX Crossover operator, Roulette Wheel Selection operator, and Binary Tournament Selection operator.

3 Mono-objective Approach

3.1 Class's Description

The main classes which were directly used for this comparison where from three types: algorithm utility classes, crossover operator classes and selection classes. The only used algorithm utility class was genetic algorithm class. The classes are described below.

Genetic algorithm is a utility class which gets constructed from the desired parameters. Such parameters are the problem instance, population size, offspring population size, mutation operator, crossover operator, termination criterion, selection operator, and a class defining the evaluator type. The evaluator type was set on the multi-processor evaluator class to make use of every available thread on the evaluating computer.

- **Permutation Swap Mutation** does a random index value exchange on the solution array depending on the result of `random.random()`. If it is greater than the defined mutation probability, it will exchange the randomly selected indexes values. If it is lower than it, then it does not mutate the solution.

- **PMX Crossover** is the implementation of the Partially Mapped Crossover genetic crossover algorithm [6, 7]. This operator randomly selects two cutting points. Then, it inherits the outer alleles of the first parent and the inner alleles of the second parent to the first offspring and then the other way around for the second offspring.
- **CX Crossover** is the implementation of the Cycle Crossover genetic crossover algorithm [6, 7]. This operator inherits alleles to an offspring by selecting from either parents' alleles in a particular position such that an invalid individual is not created.
- **Roulette Wheel Selection** is a simple selection operator which first uniformly assigns probabilities of being picked to the solutions in the solution set and then picks n solutions from them.
- **Binary Tournament Selection** starts by uniformly picking pairs of solutions which are then evaluated for fitness. The fittest solution of the pair gets selected for crossover.

3.2 Restrictions

We chose to do analysis on mono-objective meta heuristics for the CVRP, therefore, we evaluated fitness by adding the individual scores of trip solution distance and total weight. Since we evaluated single vehicle instances, the weight value was a constant. The trip distance score was calculated by adding the distances of travel between the nodes in solution's order.

4 Methodology

In order to evaluate several test cases, the library split [1] was used as a source of CVRP problems. This library contains several CVRP problem's data.

The first step was creating a simple parser for the CVRP files. The files were parsed using Python 3's regular expressions with capture groups. Afterwards the files were parsed into Python objects, which were used to instance objects from a class which inherited from jMetalPy's Permutation problem class.

The problem class is the class which implements the methods required by jMetalPy in order to compute the problem using the Permutation problem class's over writable methods. This class was responsible for the evaluation of each solution and for the generation of solutions for each generation of the evolutionary algorithm.

After the problem class was instantiated, the class genetic algorithm is also instantiated by calling its constructor with the required operators as parameters. To ensure consistency among all the tests, the operator was constructed using the same population size, offspring population size, mutation operator, termination criterion and evaluator type. Such parameters are found in the Table 1.

Table 1 Common parameters for the genetic algorithm instance’s constructor call

Population size	200
Offspring population size	200
Mutation operator	Scramble mutation
Mutation probability	0.7
Crossover probability	0.1
Termination criterion	100,000
Evaluator type	Multi process evaluator

5 Experiments

The evolutionary algorithm was run 30 times on 3 different CVRP instances for each of the following parameter combinations:

- PMX Crossover and Roulette Wheel Selection
- PMX Crossover and Binary Tournament Selection
- CX Crossover and Roulette Wheel Selection
- CX Crossover and Binary Tournament Selection.

The mutation operator was set on permutation swap mutation for every run. Each run was recorded into a Pandas data frame and then exported to a csv file. The exported dataset was then analyzed statistically utilizing R.

6 Results

The first test was a Shapiro–Wilk normality test. This test’s null hypothesis is that the data evaluated comes from a normal distribution with equal parameters. The results of the test showed that the data was indeed normal. Such result is favorable, as it allows us to apply an Analysis of Variance Table test (ANOVA). The results of the Shapiro–Wilk normality test on the A-n33-k5-results.csv instance are found in the Table 2.

Since the data showed to be normal, we proceeded by applying an Anova test. This test’s null hypothesis is that the fitness score’s variation for every combination comes

Table 2 Results of Shapiro–Wilk test on A-n33-k5-results.csv instance

Combination	W	p-value
PMX Crossover and Roulette Wheel Selection	0.98695	0.9447
PMX Crossover and Binary Tournament Selection	0. 97,372	0.5531
CX Crossover and Roulette Wheel Selection	0.97809	0.6961
CX Crossover and Binary Tournament Selection	0.96961	0.4327

Table 3 Anova test's results for A-n33-k5-results.csv instance

Df	Sum squared	Mean squared	F value	$P_r(>F)$
3	1,373,717	457,906	162.47	$<2.2e^{-16}$

Table 4 Pairwise test's result for A-n33-k5-results.csv instance

	CX Binary	CX Roulette	PMX Binary
CX Roulette	1		
PMX Binary	$<2e^{-16}$	$<2e^{-16}$	
PMX Roulette	$<2e^{-16}$	$<2e^{-16}$	0.45

from normal distributions with the same parameters. Since the data obtained (showed in Table 3) shows that the p value of such event is very close to zero ($<2.2e^{-16}$), the fitness score of the combinations does not come from normal distributions with equal parameters. Therefore, it is possible to establish in a quantitative way which combination is the best.

Afterwards, a pairwise test with Bonferroni error correction was applied. This test's null hypothesis is that the data of the evaluated pairs comes from a normal distribution with equal parameters. As the Table 4 illustrates, the test showed the algorithms with PMX Crossover algorithm had a sufficiently close to zero value ($<2e^{-16}$), we concluded that there is no significant difference between them. Similarly, the algorithms with CX Crossover seem to come from an equally parameterized normal distribution. We could also see that between the combinations with PMX Crossover and the ones with CX Crossover there is a basically non-existent probability of the data coming from an equally parameterized normal function because its test value is 1 in any instance of the comparison.

As a last test on a individual instance, the Turkey HSD test was applied. Its results can be found in the Table 5. This test helped us reaffirm that the combinations with the same crossover algorithm present no significant difference among them, but there is a very notorious difference between the combinations with different crossover algorithms.

For a second analysis phase, we ran tests to ensure that the information of a single instance was representative of all of them. For these tests, a file called Omnitest.csv was created. This file contains the data of every single experiment run along with

Table 5 Turkey HSD test's results for A-n33-k5-results.csv instance for A-n33-k5-results.csv instance

	p-value
PMX Binary-PMX Roulette	0.2812177
CX Roulette-PMX Roulette	0.0000000
CX Binary-PMX Roulette	0.0000000
CX Roulette-PMX Binary	0.0000000
CX Binary-PMX Binary	0.0000000
CX Binary-CX Roulette	0.9159992

the metadata to identify the algorithms combination which was applied to it and the instance it came from.

The first test to the omnitest.csv file was an Anova test, the results of which can be found in the Table 6. This test's null hypothesis was that the instances come from an equally parameterized normal distribution. Since the P_r value is sufficiently small, we conclude the data comes from distributions. Table 7 helps us prove statistically that each test instance was indeed different.

Knowing this was the case, we proceeded by executing pairwise tests with Bonferri correction on the algorithms in omnitest.csv. Table 7 shows the results of the test, which does not provide enough information to conclude the algorithms have a considerably different performance along all the test CVRP instances.

A pairwise test was also applied to the VRP instances. The results, found in Table 8, show that there is no significant statistical proof that the instance's results come from an equally parameterized normal distribution in any instance. This shows that we indeed tested different unique cases, which allows us to conclude that a single instances test results is representative of the results of any of them.

Afterwards, a Turkey HSD test was applied to the algorithms in omnitest.csv. The results of this test, found in Table 9, showed that there is indeed no significant difference among combinations which apply the same crossover algorithms, but that there is a very significant difference between the combinations that use different crossover algorithms along all tested CVRP instances.

A Turkey HSD test was also applied to the tested CVRP instances in the omnitest.csv file. The test showed that the instances are, again, not related, which

Table 6 Anova test's results for Omnitest.csv

	Df	Sum Sq	Mean Sq	F value	$P_r(>F)$
Algorithms	3	14,056,136	4,685,379	563.76	$<2.2e^{-16}$
Instance	4	812,748,791	203,187,198	24,448.32	$<2.2e^{-16}$

Table 7 Pairwise test's result for algorithms in omnitest

	CX-Binary	CX-Roulette	PMX-Binary
CX-Roulette	1.000	–	–
PMX-Binary	0.084	0.064	–
PMX-Roulette	0.122	0.095	1.000

Table 8 Pairwise test's result for cvrp instances in omnitest.csv

	A-n33-k5	A-n69-k9	A-n80-k10	B-n67-k10
A-n69-k9	$<2e^{-16}$	–	–	–
A-n80-k10	$<2e^{-16}$	$<2e^{-16}$	–	–
B-n67-k10	$<2e^{-16}$	$<2e^{-16}$	$<2e^{-16}$	–
B-n78-k10	$<2e^{-16}$	$<2e^{-16}$	$<2e^{-16}$	$<2e^{-16}$

Table 9 Turkey HSD test’s result for algorithms in omnitest.csv

	diff	lwr	upr	p adj
CX-Roulette-CX-Binary	−11.02286	−36.12120	14.075467	0.6704494
PMX-Binary-CX-Binary	285.64706	260.54873	310.745390	0.0000000
PMX-Roulette-CX-Binary	269.47214	244.37381	294.570474	0.0000000
PMX-Binary-CX-Roulette	296.66992	271.57159	321.768255	0.0000000
PMX-Roulette-CX-Roulette	280.49501	255.39668	305.593338	0.0000000
PMX-Roulette-PMX-Binary	−16.17492	−41.27325	8.923415	0.3460157

reaffirms our conclusion that the fist tests applied to the A-n33-k5-results.csv instance are representative of the performance of the combinations on every CVRP instance tested. The results of the test are found in the Table 10.

Finally, a boxplot was made, to show which was the best algorithm combination for the problem. Table 11 shows the shapiro-wilk test results of normality for A-n33-k5 instance, other instances show similar behavior.

- Crossover algorithms where the main performance differentiators in the test cvrp instances run.

Table 10 Turkey HSD test’s result for CVRP instances in omnitest.csv

Filename	diff	lwr	upr	p adj
A-n69-k9-A-n33-k5	2474.9364	2445.1355	2504.7373	0
A-n80-k10-A-n33-k5	3000.9337	2971.1328	3030.7346	0
B-n67-k10-A-n33-k5	1875.7941	1845.9932	1905.5950	0
B-n78-k10-A-n33-k5	2751.4076	2721.6067	2781.2085	0
A-n80-k10-A-n69-k9	525.9973	496.1964	555.7982	0
B-n67-k10-A-n69-k9	-599.1422	−628.9431	−569.3413	0
B-n78-k10-A-n69-k9	276.4712	246.6703	306.2721	0
B-n67-k10-A-n80-k10	−1125.1395	−1154.9404	−1095.3386	0
B-n78-k10-A-n80-k10	−249.5260	−279.3269	−219.7251	0
B-n78-k10-B-n67-k10	875.6135	845.8126	905.4144	0

Table 11 Shapiro–Wilk normality test’s results on A-n33-k5 instance

	w	p
X	0.95672	0.182
Execution No	0.95672	0.182
NSGA.obj1	0.97991	0.7561
NSGA.obj2	0.98371	0.8712
WASF.GA.obj1	0.96885	0.4125
WASF.GA.obj2	0.97579	0.6196

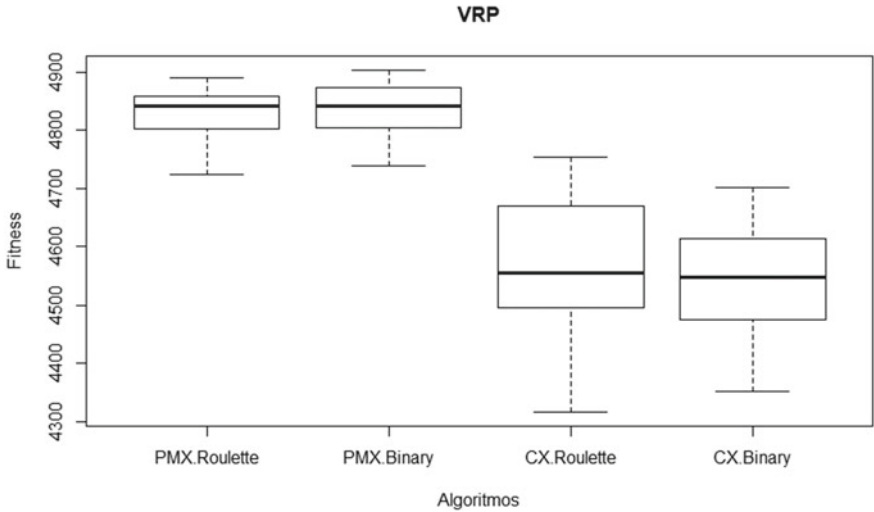


Fig. 1 Boxplot showing algorithm combinations’ fitness in omnitest.csv

- The single instance on which the individual performance tests where run (A-n33-k5-results.csv) was representative of the results to be expected from running the experiments on different instances.
- Since there was different performance among the combinations of algorithms, it is possible to determine quantitatively which combination is the best performing.

The boxplot of the algorithm’s fitness in omnitest.csv shown in Fig. 1 also helps visualize the conclusions we had previously arrived at. It also shows that the best performing algorithm combination is the one with CX Crossover and binary tournament selection algorithms, because it shows the best mean fitness and lower top outliers than the CX Crossover and Roulette wheel selection algorithms combination Fig. 2 shows statistical results on instance A-n33-k5, similar behaviour was found across the dataset.

7 Multi-objective Approach

7.1 Fitness Functions

Since the multi-objective approach does not have the single function limitation that the mono-objective approach did, the total distance and total weight to be delivered functions were chosen to be optimized simultaneously.

```

Tukey multiple comparisons of means
 95% family-wise confidence level

Fit: aov(formula = values ~ ind, data = data.stack)

$ind
              diff          lwr          upr      p adj
Execution.No-X      1.0000    -34.53589    36.53589 0.9999995
NSGA.ob1-X          791.5429    756.00697    827.07875 0.0000000
NSGA.ob2-X          314.8857    279.34983    350.42160 0.0000000
WASF.GA.ob1-X      1491.6286   1456.09268   1527.16446 0.0000000
WASF.GA.ob2-X       492.4000    456.86411    527.93589 0.0000000
NSGA.ob1-Execution.No 790.5429    755.00697    826.07875 0.0000000
NSGA.ob2-Execution.No 313.8857    278.34983    349.42160 0.0000000
WASF.GA.ob1-Execution.No 1490.6286   1455.09268   1526.16446 0.0000000
WASF.GA.ob2-Execution.No 491.4000    455.86411    526.93589 0.0000000
NSGA.ob2-NSGA.ob1   -476.6571   -512.19303   -441.12125 0.0000000
WASF.GA.ob1-NSGA.ob1 700.0857    664.54983    735.62160 0.0000000
WASF.GA.ob2-NSGA.ob1 -299.1429   -334.67875   -263.60697 0.0000000
WASF.GA.ob1-NSGA.ob2 1176.7429   1141.20697   1212.27875 0.0000000
WASF.GA.ob2-NSGA.ob2 177.5143    141.97840    213.05017 0.0000000
WASF.GA.ob2-WASF.GA.ob1 -999.2286  -1034.76446  -963.69268 0.0000000

```

Fig. 2 Turkey HSD test's result on A-n33-k5 instance

7.2 Methodology

The methodology for the multi-objective approach was similar to the previously explained one. The split library was also utilized as the CVRP instances dataset. First a parser for the instances was written. The algorithm classes were then defined with the aforementioned fitness functions and the parsed instances were then run 35 times each with the NSGA-II and WASFGA algorithms.

7.3 Results

After the experiments were run, the statistical analysis was applied to the resulting csv files.

Since every $p > 0.05$, we conclude the test showed there is no significant difference between the performances in the algorithm's performance on this instance.

We then proceeded by applying a Turkey HSD test. This test also showed there is no significant difference the algorithm's performance on a single CVRP instance.

Afterwards, we created an "omnittest.csv" file for the results of the multi-objective experiments in the same way it was created for the mono-objective approach. The same tests that were applied to the mono-objective omnittest file were applied to the multi-objective one.

The first test applied to the multi-objective omnitest file was the Turkey HSD test to the algorithm's performance data. The test showed that both algorithms present no significant performance difference in the dataset.

The second test to this file was a Turkey HSD test applied to the CVRP instance's data. This test showed that every $p < 0.05$, and therefore, there is no significant difference between the tested instances of our dataset. This led us to conclude that the results of the statistical analysis on the on A-n33-k5 instance is valid for all the instances in our dataset.

8 Conclusions

This work has presented a methodology to select the best algorithm configuration based on statistical test for mono an multi objectives using a well know problem.

Anova test were applied to both algorithms since results followed a normal distribution, this was validated by a Shapiro–wilk test.

Anova test identified significance in different performance achieved by several algorithmic configurations.

Pair-wise tests with Bonferroni corrections were applied to identify the specific algorithm configuration with significate results.

Box-plot graphs helped the identification of the best algorithmic configuration.

Results suggest crossover operators have a more impact in the quality of results for both mono objective and multi objective procedures.

Multi objective procedures for VRP showed a longer execution time.

For future work, we suggest applying more crossover operators using this methodology in order to define a set of promising operators, this set can be used in a hyper-heuristic selection algorithm.

References

1. Reinelt, G.: TSPLIB—A traveling salesman problem library. *INFORMS J. Comput.* **3**(4), 376–384 (1991). <https://doi.org/10.1287/ijoc.3.4.376>
2. Andréasson, N., Evgrafov, A., Patriksson, M.: *An Introduction To Continuous Optimization*. Dover Publications Inc., New York (2020)
3. Deb, K., Pratap, A., Agarwal, S., Meyarivan, T.: A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evol. Comput.* **6**(2), 182–197 (2002). <https://doi.org/10.1109/4235.996017>
4. Saborido, R., Ruiz, A.B.: Mariano Luque; Global WASF-GA: an evolutionary algorithm in multi objective optimization to approximate the whole pareto optimal front. *Evol Comput* **25**(2), 309–349 (2017). https://doi.org/10.1162/EVCO_a_00175
5. Benítez-Hidalgo, A., Nebro, A.J., García-Nieto, J., Oregi, I., Ser, J.D.: jmetalpy: A python framework for multi-objective optimization with meta-heuristics. *Swarm Evol. Comput.* **51**, (2019). <https://doi.org/10.1016/j.swevo.2019.100598>

6. Toth, P., Daniele, V.: The vehicle routing problem. Soc. Ind. Appl. Math.: Philadelphia (2001). <https://doi.org/10.1137/1.9780898718515>
7. Umbarkar A.J., Sheth, P.: Crossover operators in genetic algorithms: a review. ICTACT J. Soft Comput. **6**(1) (2015). <https://doi.org/10.21917/ijsc.2015.0150>

Fuzzy Logic

Optimization of a Fuzzy Classifier for Obtaining the Blood Pressure Levels Using the Ant Lion Optimizer



Oscar Carvajal, Patricia Melin, Ivette Miramontes,
and German Prado-Arechiga

Abstract These days, we live in a difficult situation due to the recent pandemic. For this reason, we have the opportunity of utilizing soft computing techniques that help in medical diagnosis. In this paper, the optimization of three proposed classifiers with type-1 fuzzy logic is presented using the Ant Lion Optimizer (ALO). The experimentation was based on the Framingham dataset obtaining good results in the classification of blood pressure levels. The results demonstrate that the optimized fuzzy classifier obtains better results compared with non optimized, and the utilized algorithm helps in optimizing type-1 fuzzy classifiers.

Keywords Blood pressure · Hypertension · Optimization · Fuzzy systems · Ant Lion Optimizer (ALO)

1 Introduction

At present, being aware of our health is of the utmost importance since the fast-paced life we live can wear us down little by little. According to data from the World Health Organization, one of the silent diseases that affects more than 30% of the world population is arterial hypertension, which is the principal cause of the development of a cardiovascular event [1]. Soft computing has been popularly utilized in the medical field to reach the diagnosis of different diseases such as Leukemia [2], Malaria [3], Glaucoma [4], different types of cancer [5–7], among others, in which more than 90% accuracy has been demonstrated in the results obtained. A fuzzy system is provided to obtain the blood pressure level. Another interesting concept that should be mentioned is optimization, which refers to solving a problem in the most efficient way possible. In soft computing, for the improvement of results,

O. Carvajal · P. Melin (✉) · I. Miramontes
Tijuana Institute of Technology, Tijuana, B.C., Mexico
e-mail: pmelin@tectijuana.mx

G. Prado-Arechiga
Cardiodiagnostico Excel Medical Center, Tijuana, B.C., Mexico

bio-inspired algorithms are widely used, which take characteristics of nature for their creation and which have given excellent results in areas such as industrial problems [8], pharmaceutical [9], control [10], scheduling [11], and others.

In this work, the optimization of a fuzzy system to obtain the blood pressure level is presented. It is an important part of a neural-fuzzy model to obtain the risk of developing hypertension [12–14]. The optimization is carried out in the membership functions parameters to perform the corresponding adjustment and accurately diagnose. For that purpose, the Ant Lion Optimizer algorithm (ALO) is used. In order to analyze the performance of the fuzzy system obtained by ALO, a database of four thousand patients is tested.

This paper is structured in this manner: Sect. 2 presents the literature review; in Sect. 3 the proposed method is described. Then in Sect. 4, the results are explained, and finally, in Sect. 5, can be found the conclusions analyzed from this work.

2 Literature Review

This section describes concepts used within the work done.

2.1 *Ant Lion Optimizer*

The Ant Lion Optimizer [15] was proposed by Mirjalili in 2015, inspired by the hunting mechanism of lion ants in nature. This algorithm uses five main steps for prey hunting: random ant walking, trap construction and reconstruction, prey capture and ant entrapment.

2.2 *Type-1 Fuzzy System*

Fuzzy logic is a type of logic that does not imply binary values, involves values between zero and one [16]. It aims to build a model of human thoughts and provide approximate reasoning using imprecise propositions based on fuzzy set theory, similar to classical reasoning utilizing the propositions based on classical set theory [17, 18].

Table 1 Office blood pressure measurement classification

Diagnosis	Systolic (mmHg)		Diastolic (mmHg)
Optimal	< 120	and	< 80
Normal	120–129	and/or	80–84
High-normal	130–139	and/or	85–89
Grade 1 hypertension	140–159	and/or	90–99
Grade2 hypertension	160–179	and/or	100–109
Grade 3 hypertension	≥ 180	and/or	≥ 110
Isolated A.H	≥ 140	and	< 90

2.3 Blood Pressure and Hypertension

The Blood pressure is the force in the walls of the arteries while the heart pumps blood through the body for transporting oxygen and nutrients for its proper function [19, 20]. Blood pressure is classified as Systolic pressure and Diastolic pressure.

Systolic pressure occurs when the heart is in contraction and is identified as the highest number when taking the blood pressure reading.

Diastolic pressure occurs when the heart is in relax mode and the moment in which the heart fills with blood and can be identified as the lowest number when taking the blood pressure reading [21].

In Table 1, it is possible to analyze the classification of blood pressure provided by the European Guidelines for Management the Hypertension. We can see that normal pressure is defined as mediated between 120/80 to 129/84 mm of mercury (mmHg) [22].

With the above, we can define arterial hypertension as a chronic disease in which the arterial pressure with which the blood is pumped to the arteries is increased. Some of the main factors to develop this condition are overweight and obesity [23, 24].

3 Proposed Method

With the aim of a blood pressure classification, a fuzzy classifier is designed due to the need to be aware of our health status, and support cardiologists in obtaining a diagnosis in a precise time. This fuzzy classifier is part of a complete model to obtain the risk of developing hypertension and a cardiovascular event in a period of the next four and ten years respectively [14, 25, 26].

In Figs. 1 and 2, the inputs of the non-optimized fuzzy system are shown, while in Fig. 3, the output variable is illustrated.

The fuzzy classifier consists of two inputs, using the systolic and diastolic pressure as variables; both are divided in seven membership functions. The output using the blood pressure level as variable, which is divided in ten membership functions,

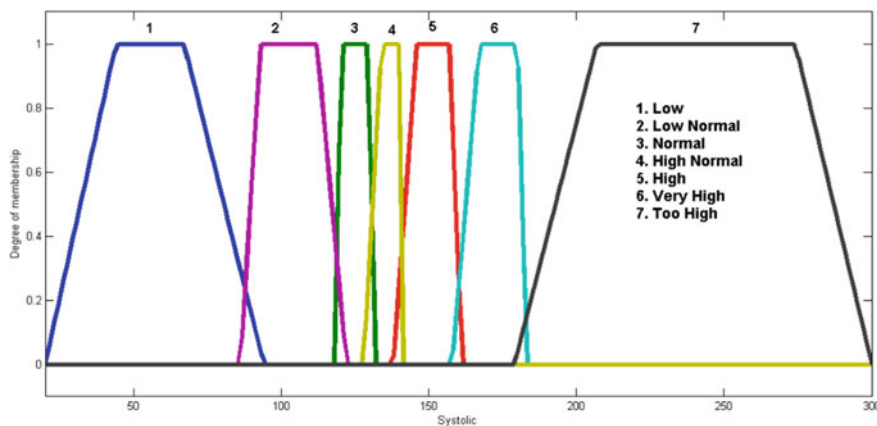


Fig. 1 Input variable Systolic

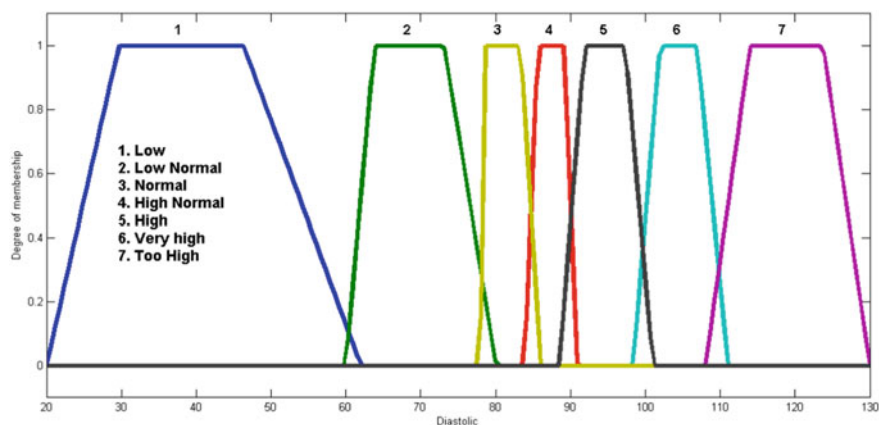


Fig. 2 Input variable Diastolic

being “Hypotension”, “Optimal”, “Normal”, “High-Normal”, “ISH-1”, “Grade-1 Hypertension”, “ISH-2”, “Grade-2 Hypertension”, “ISH-3”, “Grade-3 Hypertension” the linguistic variables, it is worth mentioning that this fuzzy system is based on the European Guide for the management of Hypertension, Table 2 shows the set of optimized rules.

In order to obtain a more precise diagnosis, a fuzzy classifier optimization is performed. For this purpose, the ALO Algorithm is utilized, with the information of 4000 patients, corresponding to a Framingham Heart Study database. Where the reading of each patient’s systolic and diastolic pressure is taken, for the optimization, 720 patients are taken to test the adjustments in the membership functions made by the algorithm, from which we will obtain the fuzzy classifier that generates a better classification. In Fig. 4, the proposed methodology for optimizing the parameters of

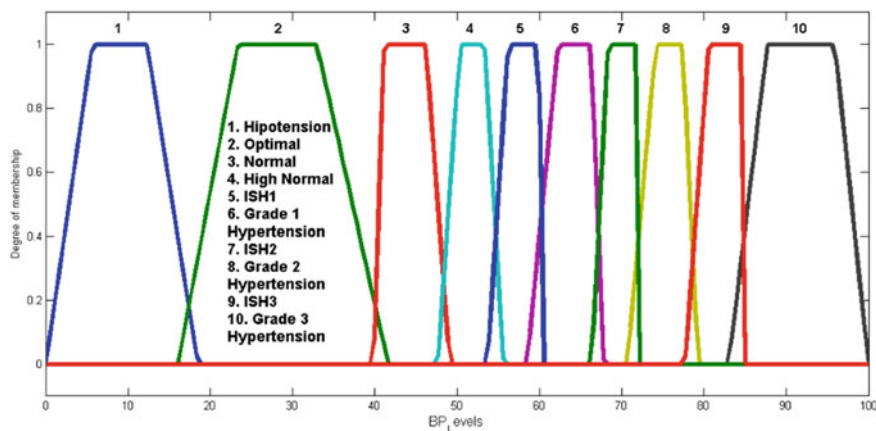


Fig. 3 Output variable BPLevels

the membership functions is illustrated.

The results from each fuzzy classifier created by the ALO algorithm are evaluated with the mean square error (MSE) as the fitness function, mathematically is represented (1).

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{Y}_i - Y_i)^2 \quad (1)$$

where:

n = number of data points,

$\hat{Y}$ = value returned by the model,

Y = the actual value for data point i .

4 Results

In order to analyze the results provided by the ALO algorithm, 30 experiments are done. For all the experiments, the parameters utilized by the algorithm are varied to analyze and find the optimal parameters to solve this optimization problem. Table 3 shows the combination of parameters used, in addition to the percentage of classification achieved.

In Table 1, the highest result of the classification of the patients was 90.78%, which corresponds to experiment 25 highlighted in bold. In comparison, the lowest result was 82.5% in experiment 28, which can be identified with italics. These averages are for 760 patients of the Framingham database selected by using the hold out method.

Table 2 Fuzzy system rules

Systolic/ diastolic	Low	Low-normal	Normal	High-normal	High	Very-high	Too-high
Low	hipotension						
Low normal	Optimal	Optimal					
Normal		Normal	Normal		Grade 1 hypertension		
High normal				High normal			
High		ISH1	ISH1	ISH1	Grade 1 hypertension	Grade2 hypertension	Grade3 hypertension
Very-high		ISH2	ISH2	ISH2	Grade2 hypertension	Grade2 hypertension	Grade3 hypertension
Too-high		ISH3	ISH3	ISH3	Grade3 hypertension	Grade3 hypertension	Grade3 hypertension

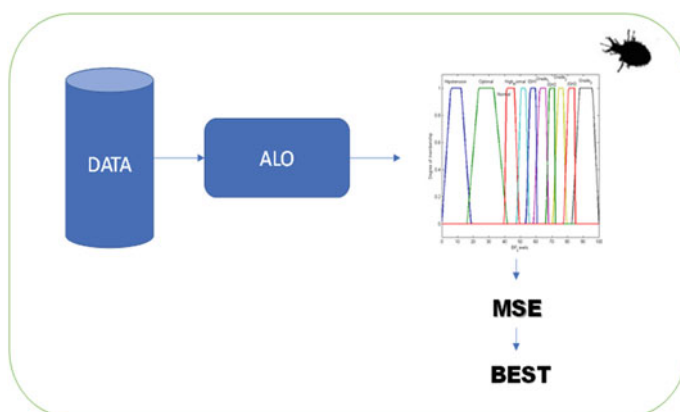


Fig. 4 Fuzzy system optimization method

In Figs. 5 and 6, the input variables with their optimized membership functions are presented, while in Fig. 7, the output variable with its optimized membership functions is presented.

A comparison is made between the diagnoses provided by both optimized and non-optimizing fuzzy systems with a group of 16 patients. In the non-optimized fuzzy classifier, there are three patients incorrectly classified, while the optimized fuzzy system classifies almost all the patients correctly except the first one. Table 4 shows the comparison of fuzzy systems.

In Table 5, a comparison is made between the classification of the non-optimized fuzzy classifier and the best one obtained by the ALO algorithm. In this case, the results are for all the patients of the Framingham database. It can be seen that the results obtained by ALO fuzzy system are better than the results by the fuzzy system without an optimization, taking into account that the results are improved by 8.25%, which means a more accurate diagnosis.

5 Conclusions

A fuzzy classifier is presented to classify blood pressure within a fuzzy-neural model that aims to diagnose people's health status in time. The fuzzy system is designed based on the European Guidelines for managing hypertension, having as inputs the systolic and diastolic blood pressure, respectively and the output the patient's blood pressure level.

In order to obtain a more precise diagnosis, we worked on optimizing the fuzzy classifier system. For this, we use the ALO algorithm, which is in charge of making the necessary adjustments to the membership functions of the classifier. In this case, 30 experiments are performed to analyze the algorithm's behaviour and observe the

Table 3 Parameters used in ALO for each experiment

T1 classifier for blood pressure levels			
Experiments	Agents	Iterations	Classification
1	50	19	85.6579
2	58	17	86.8421
3	28	35	88.0263
4	36	27	85.1316
5	66	15	89.7368
6	26	37	87.8947
7	62	16	89.8684
8	40	24	87.7632
9	52	19	88.1579
10	60	16	87.5
11	46	21	86.4474
12	68	14	89.0789
13	44	22	89.3421
14	30	32	83.2895
15	64	15	90
16	42	23	87.1053
17	48	20	88.2895
18	56	17	88.0263
19	70	14	84.6053
20	22	44	85.3947
21	18	54	88.1579
22	74	13	86.7105
23	76	13	87.6316
24	34	29	86.5789
25	38	26	90.7895
26	54	18	86.8421
27	72	14	85.7895
28	24	41	82.5
29	32	30	88.2895
30	20	49	85.2632
Average			87.22368667
standard deviation			1.949894918

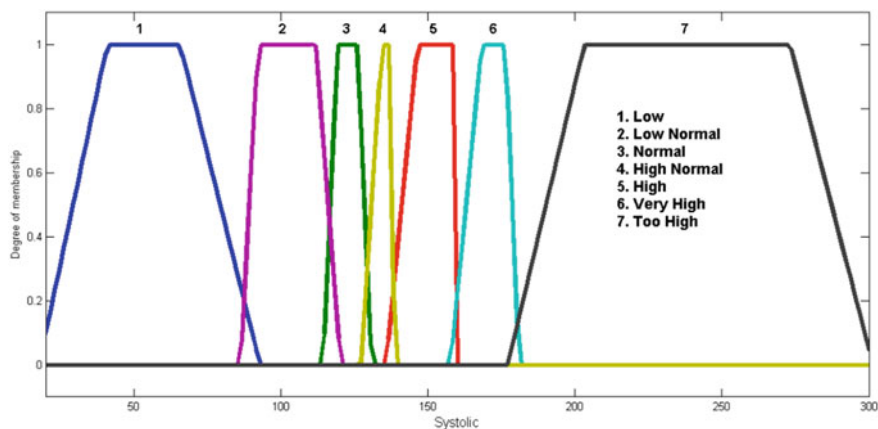


Fig. 5 Optimized systolic variable

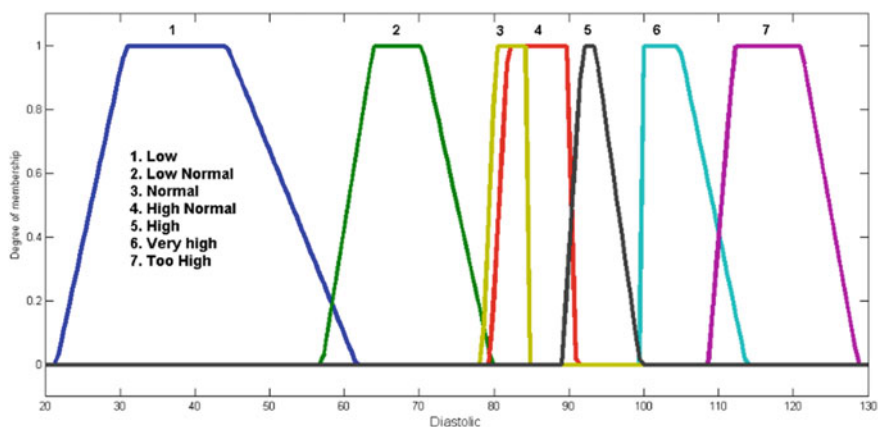


Fig. 6 Optimized diastolic variable

results provided. Compared with the fuzzy results not optimized, an improvement is observed. It can be concluded that the ALO is an efficient algorithm in the optimization solution in fuzzy systems. Future work intends to design the fuzzy proposed with Interval Type-2 and General fuzzy systems to know in what models the best results would be better, and also other metaheuristics could be considered [27–30].

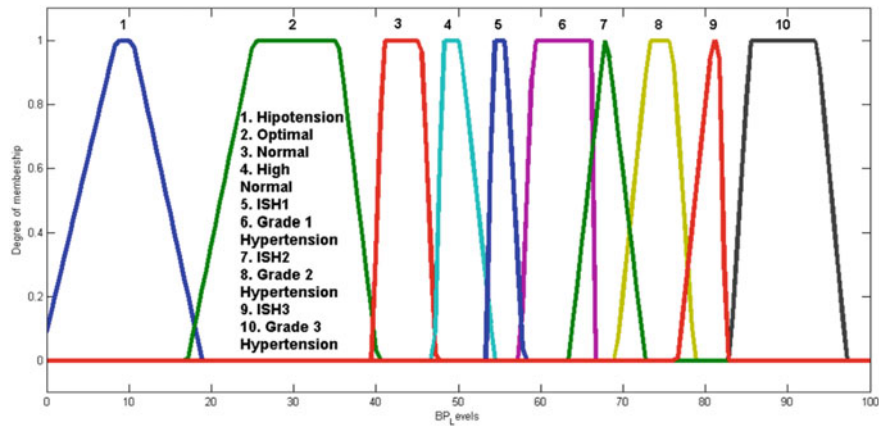


Fig. 7 Optimized BPLevel variable

Table 4 List of patients with the Blood pressure classification

Comparison between classifiers of a sample of the FM database				
Systolic	Diastolic	T1 FL optimized	T1 FL non- optimized	Target
135	97	High normal	High normal	Grade 1 hypertension
158	86	ISH1	ISH1	ISH1
128	77	Normal	Normal	Normal
118	77	Optimal	Optimal	Optimal
115	77.5	Optimal	Optimal	Optimal
112	73	Optimal	Optimal	Optimal
106	67.5	Optimal	Optimal	Optimal
109.5	69	Optimal	Optimal	Optimal
134	89	High normal	High normal	High normal
128	64	Normal	Normal	Normal
131	81	High normal	high Normal	High normal
154	100	Grade 2 hypertension	ISH2	Grade 2 hypertension
100	72.5	Optimal	Optimal	Optimal
117.5	72.5	Optimal	Optimal	Optimal
151	95	Grade 1 hypertension	Grade 1 hypertension	Grade 1 hypertension
149	100	Grade 2 hypertension	ISH2	Grade 2 hypertension

Table 5 Percent of success

Comparison between fuzzy classifiers	
Non-optimized T1 FL	Optimized with ALO
82.52%	90.78%

References

1. World Health Organization.: [https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-\(cvds\)](https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds)) (2017)
2. Sadat Asl, A.A., Zarandi, M.H.F.: A type-2 fuzzy expert system for diagnosis of leukemia. In: Fuzzy Logic in Intelligent System Design, pp. 52–60, (2018)
3. Mohamed, K.A., Hussein, E.M.: Malaria parasite diagnosis using fuzzy logic. *Int. J. Sci. Res.* **5**(6), 2015–2017 (2016)
4. Raghavendra, U., Fujita, H., Bhandary, S.V., Gudigar, A., Tan, J.H., Acharya, U.R.: Deep convolution neural network for accurate diagnosis of glaucoma using digital fundus images. *Inf. Sci. (Ny)* **441**, 41–49 (2018)
5. Esteve, A., et al.: Dermatologist-level classification of skin cancer with deep neural networks. *Nature* **542**, 115 (2017)
6. Shell, J., Gregory, W.D.: Efficient cancer detection using multiple neural networks. *IEEE J. Transl. Eng. Heal. Med.* **5**, 2800607 (2017)
7. Udoh, S.S., Umoh, U.A., Umoh, M.E., Udo, M.E., Fon, M., Udo, E.: Diagnosis of prostate cancer using soft computing paradigms diagnosis of prostate cancer using soft computing paradigms. *Glob. J. Comput. Sci. Technol. D Neural Artif. Intell.* **19**(2), 19–26 (2019)
8. Dhiman, G., Kaur, A.: STOA: A bio-inspired based optimization algorithm for industrial engineering problems. *Eng. Appl. Artif. Intell.* **82**, 148–174 (2019)
9. Zawbaa, H.M., Schiano, S., Perez-Gandarillas, L., Grosan, C., Michrafy, A., Wu, C.-Y.: Computational intelligence modelling of pharmaceutical tableting processes using bio-inspired optimization algorithms. *Adv. Powder Technol.* **29**(12), 2966–2977 (2018)
10. Lagunes, M.L., Castillo, O., Valdez, F., Soria, J.: Comparison of Fuzzy Controller Optimization with Dynamic Parameter Adjustment Based on of Type-1 and Type-2 Fuzzy Logic. In: Castillo, O., Melin, P. (eds.) *Hybrid Intelligent Systems in Control, Pattern Recognition and Medicine*, pp. 47–56. Springer International Publishing, Cham (2020)
11. Domanal, S., Guddeti, R.M., Buyya, R.: A hybrid bio-inspired algorithm for scheduling and resource management in cloud environment. *IEEE Trans. Serv. Comput.* **10**, 1–14 (2017)
12. Guzmán, J.C., Melin, P., Prado-Arechiga, G.: Neuro-fuzzy hybrid model for the diagnosis of blood pressure. In: Melin, P., Castillo, O., Kacprzyk, J. (eds.) *Nature-Inspired Design of Hybrid Intelligent Systems*, pp. 573–582. Springer International Publishing, Cham (2017)
13. Melin, P., Guzmán, J.C., Prado-Arechiga, G.: Introduction to neuro fuzzy hybrid model. In: Melin, P., Guzmán, J.C., Prado-Arechiga, G. (eds.) *Neuro Fuzzy Hybrid Models for Classification in Medical Diagnosis*, p. 103. Springer International Publishing, Cham (2021)
14. Melin, P., Miramontes, I., Prado-Arechiga, G.: A hybrid model based on modular neural networks and fuzzy systems for classification of blood pressure and hypertension risk diagnosis. *Expert Syst. Appl.* **107**, 146–164 (2018)
15. Mirjalili, S.: The ant lion optimizer. *Adv. Eng. Softw.* **83**, 80–98 (2015)
16. Peraza, C., Valdez, F., Castro, J.R., Castillo, O.: Fuzzy dynamic parameter adaptation in the harmony search algorithm for the optimization of the ball and beam controller. *Adv. Oper. Res.* **2018**, 3092872 (2018)
17. John, R., et al.: Type-2 fuzzy logic and systems, *Studies in Fuzziness and Soft Computing*, Springer (2018)
18. Castillo, O., Aguilar, L.T.: Type-2 fuzzy logic in control of nonsmooth systems, *Studies in Fuzziness and Soft Computing*, Vol. **373**, Springer (2019)
19. Kario, K., et al.: Diagnostic value of home blood pressure. In: Stergiou, G.S., Parati, G., Mancia, G. (eds.) *Home Blood Pressure Monitoring*, pp. 45–54. Springer International Publishing, Cham (2020)
20. Dadlani, A., Madan, K., Sawhney, J.P.S.: Ambulatory blood pressure monitoring in clinical practice. *Indian Heart J.* **71**(1), 91–97 (2019)
21. Padmanabhan, S., Aman, A., Dominiczak, A.F.: Genetic basis of blood pressure and hypertension. In: *Manual of Hypertension of the European Society of Hypertension*, pp. 51–65. CRC Press (2019)

22. Zanchetti, A., et al.: 2018 ESC/ESH Guidelines for the management of arterial hypertension. *Eur. Heart J.* **39**(33), 3021–3104 (2019)
23. Bakris, G.L., Sorrentino, M. Braunwald's.: Heart disease family of books, pp. 15–18. Elsevier (2018)
24. Paul, M., et al.: Measurement of blood pressure in humans: a scientific statement from the American heart association. *Hypertension* **73**(5), e35–e66 (2019)
25. Carvajal, O., Melin, P., Miramontes, I., Prado-Arechiga, G.: Optimal design of a general type-2 fuzzy classifier for the pulse level and its hardware implementation. *Eng. Appl. Artif. Intell.* **97**, 104069 (2021)
26. Guzmán, J.C., Miramontes, I., Melin, P., Prado-Arechiga, G.: Optimal genetic design of Type-1 and Interval Type-2 fuzzy systems for blood pressure level classification. *Axioms* **8**(1), 1–35 (2019)
27. Ontiveros, E., Melin, P., Castillo, O.: High order α -planes integration: a new approach to computational cost reduction of General Type-2 fuzzy systems. *Eng. Appl. Artif. Intell.* **74**, 186–197 (2018)
28. Castillo, O., Castro, J.R., Melin, P., Rodriguez Dias, A.: Application of interval type-2 fuzzy neural networks in non-linear identification and time series prediction. *Soft Comput.* **18**(6), 1213–1224 (2014)
29. Olivas, F., Valdez, F., Castillo, O., Melin, P.: Dynamic parameter adaptation in particle swarm optimization using interval type-2 fuzzy logic. *Soft Comput.* **20**(3), 1057–1070 (2016)
30. Olivas, F., Valdez, F., Melin, P., Sombra, A., Castillo, O.: Interval type-2 fuzzy logic for dynamic parameter adaptation in a modified gravitational search algorithm. *Inf. Sci.* **476**, 159–175 (2019)

Optimization of Fuzzy-Control Parameters for Path Tracking of a Mobile Robot Using Distributed Genetic Algorithms



Alejandra Mancilla, Oscar Castillo, and Mario Garcia-Valdez

Abstract Usually, we seek to establish the most optimal route for a given reference, and in this work, we focus on optimizing the parameters of a fuzzy controller that allows us to follow certain path. We know that the optimization of fuzzy controllers is a task with a high demand for computational resources since it is required to simulate the operation of the controller. Recently, distributed and asynchronous genetic algorithms have had good results in this type of optimization using multiple populations and processors to work in parallel with each population. A problem with these algorithms is the need to configure the number of populations and processors. In this work, we focus on testing different configurations for a distributed genetic algorithm applied to optimize membership functions of a fuzzy path tracking controller. We tested configurations with 3, 4, and 7 processors to minimize the evaluation time and minimize the RMSE. We also show that when increasing processors, we observe a reduction in execution time.

1 Introduction

There are many works that tune the parameters of fuzzy controllers, these proposals apply different bio-inspired algorithms [4–6, 8] to search the parameter space to find parameters that reduce the error. Bio-inspired algorithms are used for other aspects not just for controllers, they are used for route planning and obstacle avoidance [1].

A. Mancilla (✉) · O. Castillo · M. Garcia-Valdez
Tijuana Institute of Technology, Tijuana, Mexico
e-mail: alejandra.mancilla@tectijuana.edu.mx

O. Castillo
e-mail: ocastillo@tectijuana.mx

M. Garcia-Valdez
e-mail: mario@tectijuana.edu.mx

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2022
O. Castillo and P. Melin (eds.), *New Perspectives on Hybrid Intelligent System Design based on Fuzzy Logic, Neural Networks and Metaheuristics*, Studies in Computational Intelligence 1050, https://doi.org/10.1007/978-3-031-08266-5_12

In our case we are interested in using a simplified kinematic model of the bicycle type with out using the control law for the controller.

The problem we address in this work is the distributed optimization of a fuzzy rear-wheel controller [9]. In particular, we use a multi-population genetic algorithm (GA) to tune the parameters of the controller's membership functions. To evaluate each individual in the GA's population, we need to run a simulation in which the proposed generated controller is presented with the challenge of following a path for a certain amount of time. In previous work, we established that this type of simulations demand extensive computational resources, needing a few seconds of computation time in most cases [7]. Since the earlier days of GAs, researchers have proposed alternatives to make the algorithm's operation faster by parallelizing the operations. One of the most simple but powerful ways of achieving this goal is to have multiple populations running separately in a single process or thread. The island model is one of the first proposals, but researchers have proposed many other methods since then. Recently, cloud-native architectures are gaining track because they offer certain advantages over centralized systems. In this paper, we apply an event-driven approach to asynchronously execute our proposed GA, intending to reduce the execution time of our previous sequential implementation.

The paper is presented as follows. Next, in Sect. 2 we describe in detail the implementation of our method, how we codify in a chromosome the parameters of the fuzzy controllers' membership functions. In Sect. 3, we test our implementation by optimizing a fuzzy rear-wheel controller using different configurations and describe the benchmark control problem and the results. Finally, our conclusions are presented in Sect. 4.

2 Proposed Method

Running a simulation is very computationally expensive, so we propose a distributed GA, to execute the simulations in parallel. The distributed algorithm we implemented is based on the work of García and Merelo [3] an asynchronous and event-based architecture, for population-based bio-inspired algorithms. Implementing an event-based architecture means that we use message queues to exchange data between processes we call workers, and they take messages asynchronously to process the data (in this case populations). Each worker, runs in a container. Containers are light-weight, isolated, software packages that share operating system resources, using less resources than a traditional virtual machine.

2.1 Distributed GA

Each GA worker inside a container, is prepared to receive a population, and execute in this population several generations of a GA. Figure 1 shows that we need

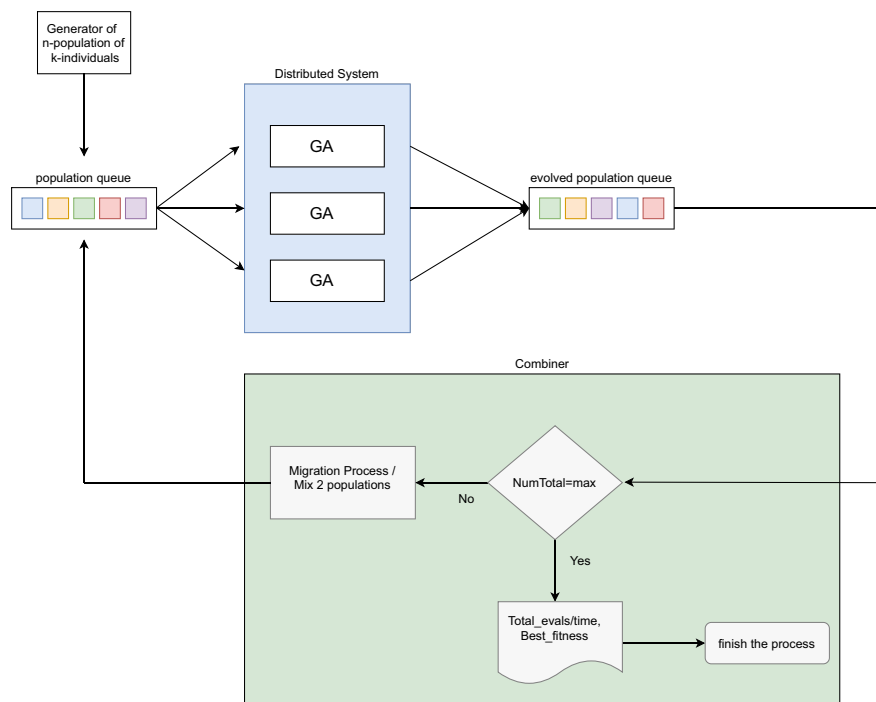


Fig. 1 Distributed algorithm diagram

two queues, one called `population queue` where workers take populations to evolve and another called `evolved population queue` needed to put the resulting evolved population. To initialize the algorithm first a one-time execution process, pushes n populations to the `population queue` each population filled with an initial k random individuals. The main idea is that each worker runs a GA that receives a population as a message and then runs this population for a small number of generations and pushes the resulting population to the output queue `evolved population queue`.

An important component of the architecture is the `Combiner` worker (shown in light blue in the Figure.) The `Combiner` worker is responsible for taking the resulting populations from the queue, and stop the algorithm if the number of function evaluations has been reached or an solution has been found. But, the main responsibility of this component is to mix or combine populations that are arriving from the queues. This operation is similar to a migration between islands in the island GA model.

2.2 Membership Function Optimization

In general, the proposed method to tune the parameters of membership functions (MFs) of a Fuzzy controller is as follows: Each fuzzy variable is defined by a list of MFs that have a certain number of parameters. The chromosome of each individual simply has a gene for each parameter of the MF. To reduce the search space, we can simplify and reduce the number of parameters represented in a chromosome. The main simplifications we can find in this type of controllers are:

- **Symmetrical Functions.** Variables in path tracking usually try to measure the error as a distance from the desired path. This error can be positive or negative depending on the robot or vehicle's side. Because values are symmetrical we can specify symmetrical MFs, and to do this we only need half the number of parameters, because the values are the same on each side only with a different sign.
- **Minimum Error is Zero.** The position of membership functions will some times depend on the minimum error. In this type of controllers, the highest membership for a low error variable will always be at zero, so we can leave these parameters fixed at zero.
- **Extremes Values.** In some of the MFs, extreme values are needed to specify a high error, and we consider a membership of one from some fixed point on. This means that we are only interested in one side of the MF, and we can leave the parameters fixed at some extreme values out of the domain of the fuzzy variable.

For instance, we can define a triangular MF for the fuzzy variable *Low*, representing a low error (distance from the path). If we consider this variable symmetrical and with and always centered at zero, we can represent this function with a single value. We can call this parameter a , and the parameters for the function will be the tuple $(-a, 0, a)$. On the other hand, if we want to make this function asymmetrical, we would need two values a and b , for a tuple $(a, 0, b)$. The problem with this approach is that we can not know if the values will cross each other. Another option is to keep the main values symmetrical and add two variables to change the parameter in a more controlled way. For instance, a , b , c , and d for the tuple $(a + b, 0, a + d)$. Also, as a simplification technique, we can leave some of parameters fixed even for a single fuzzy variable, setting the parameters from the experience of the designer. We compare these approaches in this paper, using chromosomes with four, nine and 18 genes, to represent different levels of simplification for the fuzzy controllers. We describe this configurations in the next sections.

2.3 Four Parameters Configuration

To optimize the membership functions, we can change the values that define them. In the case of triangular functions, we need three parameters, and for the trapezoidal functions, we need four parameters. In this basic case, we only optimize four inputs

(a,b,c,d). For instance, to define the triangular function for the theta error, we only change one variable (b), so the triangular function will always be symmetric and centered at zero. In this case, we do not modify the output omega function, keeping the input values fixed. This is shown in Listing 12.1.

Listing 12.1 Four parameters configuration

```
e_teta_hi_neg = fuzz.trapmf(x_e_teta,[-100,-100,-a,0])
e_teta_lo = fuzz.trimf(x_e_teta, [-b,0,b])
e_teta_hi_pos = fuzz.trapmf(x_e_teta,[0,a,600,600])

error_hi_neg = fuzz.trapmf(x_error, [-600,-600,-c, 0])
error_lo = fuzz.trimf(x_error,[-d,0,d])
error_hi_pos = fuzz.trapmf(x_error,[0,c,600,600])

omega_hi_neg = fuzz.trapmf(x_omega,[-8, -8, -2,-0.5])
omega_lo = fuzz.trimf(x_omega,[-1,0,1])
omega_hi_pos = fuzz.trapmf(x_omega,[0.5,2,8,8])
```

2.4 Nine Parameters Configuration

In this configuration, we search for symmetrical MFs, varying only pertinent parameters of the trapezoidal functions. We established these parameters as symmetrical functions; in the case of the triangular function, the aperture was also symmetrically optimized (c,f,i) with the parameters found in Listing 12.2.

Listing 12.2 Nine parameters configuration

```
e_teta_hi_neg=fuzz.trapmf(x_e_teta,[-100,-100,-a,-a+b])
e_teta_lo = fuzz.trimf(x_e_teta, [-c, 0, c])
e_teta_hi_pos = fuzz.trapmf(x_e_teta,[a+b,a,600,600])

error_hi_neg = fuzz.trapmf(x_error,[-600, -600,-d,-d+e])
error_lo = fuzz.trimf(x_error,[-f,0,f])
error_hi_pos = fuzz.trapmf(x_error,[d+e, d,600,600])

omega_hi_neg = fuzz.trapmf(x_omega,[-8,-8,-g,-g+h])
omega_lo = fuzz.trimf(x_omega, [-i,0,i])
omega_hi_pos = fuzz.trapmf(x_omega,[g+h, g, 8, 8])
```

2.5 18 Parameter Configuration

Here we show an option using eighteen parameters but changing the values so we have asymmetrical functions. We need more parameters to achieve this, as shown in Listing 12.3. Having more parameters give us the possibility of changing the definition of each MF in more ways, but also increases the search space. Also, these parameters supersede all the MF that can be generated with the previous configurations, but with at a higher computational cost.

Listing 12.3 18 parameter configuration

```
e_teta_hi_neg=fuzz.trapmf(x_e_teta,[-100,-100,-a,-a+b])
e_teta_lo = fuzz.trimf(x_e_teta,[-c,0,d])
e_teta_hi_pos = fuzz.trapmf(x_e_teta,[f+e,f,600,600])

error_hi_neg = fuzz.trapmf(x_error,[-600,-600,-g,-g+h])
error_lo = fuzz.trimf(x_error, [-i,0,j])
error_hi_pos = fuzz.trapmf(x_error,[k+l,k, 600,600])

omega_hi_neg = fuzz.trapmf(x_omega,[-8,-8,-m-m+n])
omega_lo = fuzz.trimf(x_omega,[-o,0, p] )
omega_hi_pos = fuzz.trapmf(x_omega,[q+r,q,8,8])
```

Another essential variable to consider is the range of the initial random values since these limit the space search and the minimum and maximum opening of the membership functions.

2.6 Implementation

As we mentioned before, each worker is running inside a Docker container, for running the algorithm, we created a `docker-compose` script, with the configurations needed to create and start each container. As part of the configuration, we can specify the number of containers (workers) that will start in each run. Each container is described and stored as a Docker image, containing all the dependencies, libraries, code, and software needed to automatically create all the infrastructure need to run an experiment. Container images are turned into one or many containers that run in a Docker Engine, this engine runs as a native application in Linux systems and virtually in both macOS and Windows. Containers and virtual machines are both isolated and have the same allocation benefits, but they work differently because containers virtualise the operating system instead of the hardware. Because of this, containers are more portable and efficient.

3 Experiments and Results

As a benchmark for the GA we implemented a fuzzy controller, for the rear-wheel controller described by Paden et al. [9], we implemented a simulation in Python adapting the Python Robotics library by Sakai [10]. For the GA found in each worker we used the DEAP library [2]. For implementing the queue, we used Redis and the py-redis library. The experiments run in a Thinkpad Laptop with 16 GB RAM and an Intel® Core™ i7–8650U CPU @ 1.90 GHz 2.11 GHz processor. We used the Windows 10 Pro, 64 bits operating system version 20H2. With Python 3.7.6 (default, Jan 8 2020, 20:23:39) [MSC v.1916 64 bit (AMD64)]::Anaconda, Inc. on win3, Docker desktop v20.10.6, and Redis 6.2.5-alpine container.

3.1 Fuzzy Control

In the rear-wheel controller described by Paden [9] the error e is measured as the distance between the rear wheel and the nearest point to the trajectory. If the wheel is to the left, the error is considered positive. If the wheel is to the right, then it is negative. The second variable is the angle between the tangent at the nearest point in the trajectory and the bearing vector. The controller's output is the angular velocity (ω) needed to calculate the steering angle of the front wheel.

In Fig. 2 we have the three fuzzy variables described above, and their membership functions. We defined each variable with three membership functions for High-Negative, Low and High-Positive. We have two signs because the error and the heading can be to the left or the right. With these variables, we proposed a controller with nine fuzzy rules. We considered the low membership function as a triangle with the highest membership in zero.

With these variables, we proposed a controller with nine fuzzy rules as seen in Table 1. We considered the low membership function as a triangle with the highest membership in zero.

3.2 GA Setup

To evaluate our proposal we run a preliminary experiment with this setup, see Table 2. For crossover and mutation probability, we tested with two settings, in the first we established the same values for all workers, and in the second, each worker was set at a uniform random value in the range shown in parenthesis in the last two rows. We also incremented the population size as the search space incremented, we also increased the number of individuals in each population. This was necessary, because we need a higher number of evaluations to find an appropriate solution. Finally, we can see

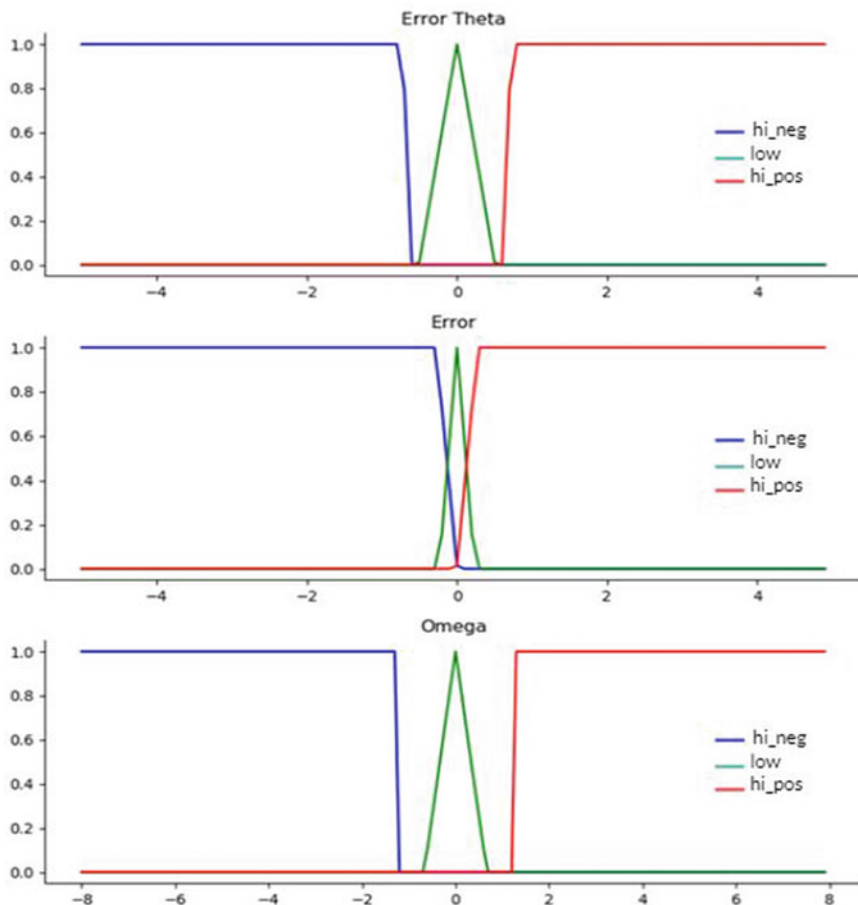


Fig. 2 Membership functions and fuzzy variables for the fuzzy controller

that the number of iterations was doubled in each increment. As the search space increased, the algorithm need more simulations, and more computational resources.

After running several experiments with 3, 4, and 7 workers, here we present the best preliminary configuration. The best results we found are with seven workers. In this case, when initializing the random population, we used two ranges for the initial values: from 0 to 1 and from 0 to 2. As mentioned before, we also compared the fixed and random strategy for setting the GA parameters of each worker, initializing each worker with random parameters for crossover between 0.3 and 0.9 and mutation probability with random values between 0.1 and 0.5.

In Tables 3 and 4 we can see the results of the evaluations per second, the total time, and the root mean square error, for both strategies (fixed and random.) Each time a simulation is executed for evaluating the fitness of an individual in the GA, we

Table 1 Fuzzy rules for the rear-wheel controller

Rule 1:	If θ_e is	hi_neg	and e is	low	then ω is	hi_pos
Rule 2:	If θ_e is	hi_pos	and e is	low	then ω is	hi_neg
Rule 3:	If θ_e is	low	and e is	low	then ω is	low
Rule 4:	If θ_e is	hi_neg	and e is	hi_neg	then ω is	hi_pos
Rule 5:	If θ_e is	hi_pos	and e is	hi_pos	then ω is	hi_neg
Rule 6:	If θ_e is	hi_pos	and e is	hi_neg	then ω is	low
Rule 7:	If θ_e is	hi_neg	and e is	hi_pos	then ω is	low
Rule 8:	If θ_e is	low	and e is	hi_pos	then ω is	hi_neg
Rule 9:	If θ_e is	low	and e is	hi_neg	then ω is	hi_pos

Table 2 Configuration for each of the experiments, number of genes in each column heading

	7 workers					
	4		9		18	
Total population size	100		120		120	
Number of populations	10		10		10	
Population size (each)	10		12		12	
Num. generations	10		10		10	
Iterations	2		4		8	
Crossover probability (random)	0.7	(0.3–0.9)	0.7	(0.3–0.9)	0.7	(0.3–0.9)
Mutation probability (random)	0.3	(0.1–0.5)	0.3	(0.1–0.5)	0.3	(0.1–0.5)

count this as one function evaluation; this is also a measure of the amount of work that is done in parallel and can indicate a speed-up because of the parallel execution. We can see that in both tables this is not the case, the reason for this, is that the Laptop used for running the experiments only has four cores, so it can only run that many processes in parallel. As expected, the total time of execution increased together with the size of the chromosome, but this increment was higher when the range of the search (shown in columns) was broader (0–2). The algorithm gave the best results with nine parameters in the chromosome, and when the search space was limited to the (0,1) range. Also we can observe that choosing a random or fixed parameters does not have a major effect in this type of optimization.

Table 3 Results for fixed crossover and mutation probability

	Range (0–1)			Range (0–2)		
	4	9	18	4	9	18
Evaluations/s	1.53	1.62	1.55	1.60	1.55	1.05
Total time (s)	1182	2646	5504	1106	2754	8034
RMSE	0.8440	0.4171	0.4338	0.8736	0.5687	0.5880

Table 4 Results for random crossover and mutation probability in each worker

	Range (0–1)			Range (0–2)		
	4	9	18	4	9	18
Evaluations/s	1.40	1.51	1.54	1.54	1.37	1.32
Total time (s)	1117	2590	4551	1070	2822	6196
RMSE	0.8850	0.4955	0.7627	0.9341	0.5030	0.5903

4 Conclusions

We presented a design and implementation of a distributed GA for the optimization of the parameters of the membership functions for a fuzzy controller. Preliminary results show that a symmetrical definition of the MF parameters reduces the search space but also gives better results, than having an asymmetrical definition of the parameters. Also, limiting the range in which the parameters are searched, also reduces the space and gives better results and needs fewer evaluations. In our experiments using 9 parameters for the MFs produces better controllers and having the initial random values between 0 to 1 reduces the error further. Finally, symmetry benefits the definition in this type of membership functions.

As future work we need to run the experiments in a computer with more cores or in the cloud, we are now limited to 4 cores. As a next step we will add more membership functions (five) Add other bio inspired population-based algorithms.

References

1. Beleño, R.D.H., Vítor, G.B., Ferreira, J.V., Meirelles, P.S.: Planeación y seguimiento de trayectorias de un vehículo terrestre con base en el control de dirección en un ambiente real. *Scientia et technica* **19**(4), 407–412 (2014)
2. Fortin, F.A., De Rainville, F.M., Gardner, M.A.G., Parizeau, M., Gagné, C.: Deap: Evolutionary algorithms made easy. *The Journal of Machine Learning Research* **13**(1), 2171–2175 (2012)
3. García-Valdez, J.M., Merelo-Guervós, J.J.: A modern, event-based architecture for distributed evolutionary algorithms. In: *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. pp. 233–234 (2018)

4. Hernandez, E., Castillo, O., Soria, J.: Optimization of fuzzy controllers for autonomous mobile robots using the grey wolf optimizer. In: 2019 IEEE international conference on fuzzy systems (FUZZ-IEEE). pp. 1–6. IEEE (2019)
5. Lagunes, M.L., Castillo, O., Soria, J.: Methodology for the optimization of a fuzzy controller using a bio-inspired algorithm. In: North American Fuzzy Information Processing Society Annual Conference. pp. 131–137. Springer (2017)
6. Lagunes, M.L., Castillo, O., Soria, J.: Optimization of membership function parameters for fuzzy controllers of an autonomous mobile robot using the firefly algorithm. In: Fuzzy Logic Augmentation of Neural and Optimization Algorithms: Theoretical Aspects and Real Applications, pp. 199–206. Springer (2018)
7. Mancilla, A., Castillo, O., Valdez, M.G.: Evolutionary approach to the optimal design of fuzzy controllers for trajectory tracking. In: International Conference on Intelligent and Fuzzy Systems. pp. 461–468. Springer (2021)
8. Martinez-Soto, R., Castillo, O., Aguilar, L.T., Baruch, I.S.: Bio-inspired optimization of fuzzy logic controllers for autonomous mobile robots. In: 2012 Annual Meeting of the North American Fuzzy Information Processing Society (NAFIPS). pp. 1–6. IEEE (2012)
9. Paden, B., Čáp, M., Yong, S.Z., Yershov, D., Frazzoli, E.: A survey of motion planning and control techniques for self-driving urban vehicles. *IEEE Transactions on intelligent vehicles* **1**(1), 33–55 (2016)
10. Sakai, A., Ingram, D., Dinius, J., Chawla, K., Raffin, A., Paques, A.: Pythonrobotics: a python code collection of robotics algorithms. *CoRR* **abs/1808.10703** (2018), <http://arxiv.org/abs/1808.10703>

A New Fuzzy Approach to Dynamic Adaptation of the Marine Predator Algorithm Parameters in the Optimization of Fuzzy Controllers for Autonomous Mobile Robots



Felizardo Cuevas, Oscar Castillo, and Prometeo Cortes-Antonio

Abstract This paper presents the fuzzy variant of a recent naturally-based meta-heuristic that follows the rules of the best feeding strategies with the predator and prey rate regime of marine ecosystems, called the fuzzy marine predators algorithm (FMPA). This is presented as an improved variant of the marine predators algorithm and its utility in control applications. Its novelty is presented in the application of Type-1 and Interval Type-2 Fuzzy Logic systems, which balance the degree of exploration and exploitation through its iterations and through the search for the optimum of its parameters FADS and P, the second novelty is given as the fuzzy variation Interval Type-2 of a CF parameter, which varies the advance of the predator, memorized separately by the best fitness achieved by iteration, and in this way to coordinate the actions of exploration and exploitation. the fuzzy marine predator algorithm evaluates its performance with a group of 19 benchmark functions of the CEC-2017 that is composed of 2 unimodal functions, 7 multimodal functions, and 10 hybrid functions, the results show that the fuzzy marine predators algorithm provides better results than MPA. The fuzzy marine predators algorithm is used to search for the best distribution of the values of the parameters of the membership functions of a fuzzy controller for the plant of an autonomous mobile robot, bearing in mind different degrees of noise to analyze the performance when simulating the Interval Type-2 Fuzzy Logic controller. The simulation results provide evidence that the fuzzy marine predator's algorithm exceeds the results of the marine predator's algorithm are compare, in the design and optimization of the Interval Type-2 Fuzzy Logic controller.

Keywords Marine Predators algorithm · Shark smell optimization · Type-2 fuzzy logic system · Interval type-2 fuzzy logic controller · CEC2017 functions · Autonomous mobile robots · Robotics navigation

F. Cuevas · O. Castillo (✉) · P. Cortes-Antonio
Tijuana Institute of Technology, Tijuana, BC, México
e-mail: ocastillo@tectijuana.mx

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2022
O. Castillo and P. Melin (eds.), *New Perspectives on Hybrid Intelligent System Design based on Fuzzy Logic, Neural Networks and Metaheuristics*, Studies in Computational Intelligence 1050, https://doi.org/10.1007/978-3-031-08266-5_13

179

1 Introduction

A trend that has occurred in recent years is that fuzzy controllers are optimized through the use of metaheuristics, the reason is that controllers must be optimized because they often cannot obtain the best performance that is required in the control implementation of the real world [1, 2]. these controllers are regularly identified, commonly as type 1 fuzzy logic controllers (T1FLC and T2FLC) [3, 4].

Where the parts of a set take a numerical value as a measure of the uncertainty in the so-called membership functions. As an extension of the existing fuzzy logic (type 1), we have the type 2 fuzzy logic raised later where the goal is to handle problems with more complexity, that is, with a greater degree of uncertainty, where regularly the type 1 fuzzy logic, does not get a good result. Such techniques can provide very competitive results, although not necessarily the best alternatives with reasonable times and costs [5].

These algorithms have been taken as research motivation by many scientists in the field of intelligent control [6, 7]. Most of these systems are based on nature, generally in a self-organized way [8–10].

We have research applying optional metaheuristic algorithms to improve fuzzy controllers, these other studies use alternative metaheuristic algorithms to optimize fuzzy controllers [11–13].

In the beginning, type 1 fuzzy systems have been improved using metaheuristic algorithms; for example, type 1 fuzzy controller optimization is discussed in [14], the Shark Smell algorithm (SSO) is used to develop the navigation framework following a trajectory of a autonomous mobile robot using an IT2 fuzzy controller too.

Other authors have used fuzzy controllers to control autonomous robots following a trajectory or in search of a position, as described in [15, 16].

So we can find high-performance fuzzy applications, like [17] where also the Type-1 FLC is compared with the Type-2 FLC development.

The main contribution of this research is to provide a dynamic parameter optimization approach that uses a metaheuristic algorithm to achieve optimal performance to generate satisfactory results in the control of a autonomous mobile robot in order to minimize the orientation and position error within the line that serves as the way to the vehicle.

Recent work on this topic has shown that the use of fuzzy dynamic parameter fitting in metaheuristic algorithms for the optimization of mathematical functions and control problems produces competitive results, we propose to use in this research work Interval Type-2 Fuzzy Logic Systems at to perform the dynamic adjustment of parameters of the marine predator algorithm and evaluate the performance of the algorithms applying both (MPA and our proposal) for the optimization of the fuzzy controller of trajectory tracking. In such a way that the efficiency of the marine predator algorithm (MPA) is tested against its fuzzy variant (FMPA).

By applying the approach already in benchmark functions CEC-2017 has shown good performance when applied to the optimization of complex problems, so it is being implemented in the adjustment of the punctual values of membership functions

of a fuzzy controller. A wide variety of works in control today have shown that the adjustment of parameters in metaheuristic algorithms oriented to the optimization of mathematical functions and of course in control applications yield competitive results [18, 19], in this work we apply Type-1 and Type-2 Fuzzy Logic Systems to perform the dynamic adjustment of the algorithm's own parameters and measure its performance when applying it to improve the fuzzy controller. Furthermore, a performance comparison is made between our Fuzzy Marine Predator Algorithm and the MPA metaheuristic algorithm [20]. The approach performs well when applied to the optimization of complex problems, and CEC-2017 functions. The mathematical models and control routines were simulated and validated in Matlab/Simulink and the results were analyzed.

This work is structured as follows: Sect. 2 presents a background of fuzzy logic and introduces the metaheuristics of marine predators and our fuzzy proposal (FMPA). Section 3 presents the design, implementation of IT2FIS in the proposed approach when applied to the optimization of complex problems and CEC-2017 functions. Section 4 presents the case studies where we tested our proposal, CEC-2017 mathematical functions, and the adjustment for a fuzzy controller of an Autonomous Mobile Robot. Section 5 shows the results obtained in our case studies and Statistical comparison. Section 6 closes with the conclusions and some proposals for future work.

2 Fuzzy Logic and Marine Predators Algorithm

The conceptual and algorithmic simplicity of Fuzzy Logic [21, 22] and to the fields in which it has been successfully applied that are diverse, it has currently positioned this methodology in many areas, mainly in control [23, 24].

This section shows the most relevant of the theory and concepts that are most used in this research.

2.1 *Marine Predator Algorithm*

This section analyzes the evolution of the Marine Predator Algorithm (MPA), a recently created metaheuristic proposed by Mirjalili in 2020 as a simple and efficient metaheuristic optimization method [20] based on the “survival of the fittest” theory. MPA follows the rules that naturally govern optimal foraging strategy and finds the policy of rates between predators and prey in marine ecosystems that often change considerably with age and time. Where the eating pattern depends on a random walking strategy; a stochastic process where the next state/position depends on the current state with a probability of change to the next position.

2.2 MPA Formulation

Currently, the MPA algorithm is proving to be one of the most widely used meta-heuristics to solve various types of problems [25, 26]. The main inspiration for MPAs is the generalized foraging strategy, namely the Lévy and Brownian movements in oceanic predators together with the optimal encounter rate policy in the biological interaction between predators and prey. Describe in Fig. 1.

MPA follows the rules that naturally govern optimal foraging strategy and finds the policy of rates between predators and prey in marine ecosystems and like most meta-heuristics, MPA is population-based where the initial solution is uniformly distributed in the search space as an initial step. Its interpretation in mathematical terms originates the following 5 steps and their respective equations:

Step 1. Initialize the problem and parameters.

$$X_0 = X_{min} + Rand(X_{max} - X_{min}) \quad (1)$$

where X_{min} and X_{max} are the lower and upper bound for variables and rand is a uniform random vector in the range of 0–1.

Step 2. Initialize the top predator matrix called Elite.

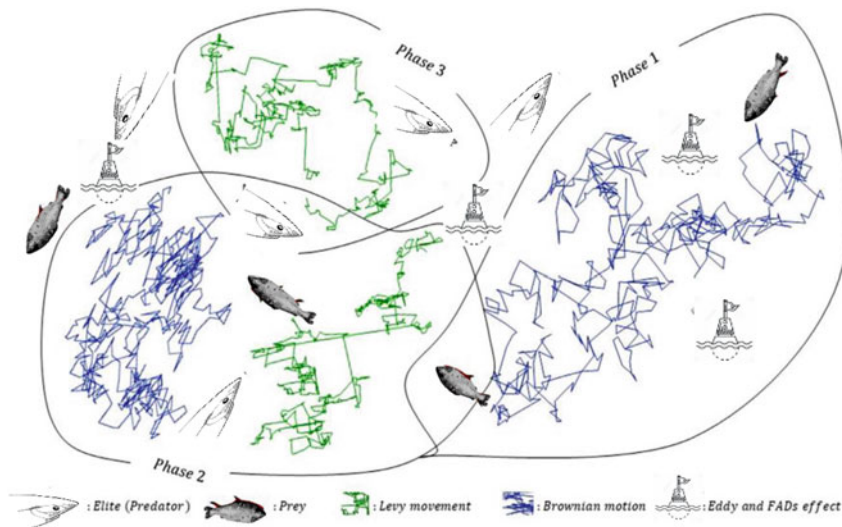


Fig. 1 Description of the three phases of MPA optimization

$$Elite = \begin{bmatrix} X_{1,1}^I & \dots & X_{1,D}^I \\ X_{2,1}^I & \dots & X_{2,D}^I \\ \dots & \dots & \dots \\ X_{n,1}^I & \dots & X_{n,D}^I \end{bmatrix} \quad (2)$$

where X^I is the vector of the top predator, produced n times to form the Elite matrix. n represents the number of search agents, while D is the number of dimensions. At the conclusion of each iteration, the Elite matrix is updated with the top predator replacing it with the best predator.

Step 3. Initialize the second matrix called Prey with the same dimension as Elite.

$$Prey = \begin{bmatrix} X_{1,1} & \dots & X_{1,D} \\ X_{2,1} & \dots & X_{2,D} \\ \dots & \dots & \dots \\ X_{n,1} & \dots & X_{n,D} \end{bmatrix} \quad (3)$$

Equation (3), $X_{i,j}$ has shown the j th dimension of i th prey. Clarifying at this point that the entire optimization process depends mainly on the update of these two matrices.

Step 4. Update the positions memory. To update the MPA with the new solutions matrix, Elite, and Prey, the optimization scenarios will be used to evaluate them. A comparison is made to find out if the new matrix solution is better than the worst historical matrix solution and then the worst historical is removed and substituted with a new one.

Scenario 1 (Exploration): $Iter < \frac{I_{max}}{3}$

$$\begin{aligned} STEPSIZE &= R_B(Elite - R_B * Prey) \\ Prey &= Prey + P.R(STEPSIZE) \end{aligned} \quad (4)$$

By multiplying R_B per dam we generate the movement of the dam. $P = 0.5$ is a constant number and R is a vector of uniform random numbers in $[0, 1]$. This scenario occurs during the first third of the iterations when initially the step that determines the speed of the movement is high for a high exploration capacity.

$$\text{Scenario 2(Exploration-Exploitation): } \frac{I_{max}}{3} < Iter < \frac{2}{3}I_{max} \quad (5)$$

In this phase, both exploration and exploitation are important. Consequently, half of the population is designated for exploration and the other half for exploitations. In this phase, prey is responsible for exploitation and predator for exploration.

Prey (exploitation)

$$\begin{aligned}
 STEPSIZE_{Prey} &= R_L(Elite - R_L * Prey) \\
 Prey &= Prey + P.R(STEPSIZE)
 \end{aligned} \tag{5a}$$

$$\begin{aligned}
 &Predator \text{ (exploitation)} \\
 STEPSIZE_{Prd} &= R_B(R_B * Elite - Prey) \\
 Prey &= Prey + P.CF(STEPSIZE)
 \end{aligned} \tag{5b}$$

Equation (5a) multiplying R_L and $Prey$ simulates the movement of the prey in the Lévy manner, while adding the step size to the position of the prey generates the movement of the prey. Since the size of the Levy distribution step is associated with small steps, this section helps to exploit. Equation (5b) the Multiplication of R_B and $Elite$ the movement of the *predator* is generated in Brownian form, while the *Prey* is updated in its position based on the movement of the predators with Brownian movement.

Scenario 3 (Exploitation): $I > \frac{2}{3} I_{max}$

$$\begin{aligned}
 STEPSIZE &= R_L(R_L * Elite - Prey) \\
 Prey &= Prey + P.CF(STEPSIZE)
 \end{aligned} \tag{6}$$

Equation (6) multiplying from RL and the Elite matrix we generate the movement of the predator in the Lévy strategy, while adding the step size to the position of the Elite matrix we cause the movement of the predator to help update the position of the prey.

Where I is the current iteration while I_{max} is the maximum iteration to reach; $STEPSIZE_{Prey}$ and $STEPSIZE_{Prd}$ is the speed of movement of the predator and the prey; R_B is a vector of random numbers based on the normal distribution that represents the Brownian motion; R_L is a vector of random numbers based on the Lévy distribution that represents the Lévy movement or flights; CF it is an adaptive parameter that determines the size of the advance of the predators' movement; P is a constant number; R is a vector of uniform random numbers between [0 1].

Step 5: Verify if the stopping criteria is met. The process is repeated until the number of iterations (Itermax) is satisfied; otherwise the process is repeated from Steps 2 and 4. Finally, the best solution is achieved and considered as the best result to the problem. The FADs effect along with long steps of Lévy strategy greatly helps MPA to avoid local optima stagnation and better performance of the method. which is calculated as follows in Eq. (7).

$$Prey = Prey + CF * ((X_{min} + rand(SearchAgents_no, dim)) * (X_{max} - X_{min})) * U \tag{7}$$

The adaptive defined convergence factor (CF) parameter represents the intensification or exploitation, the adjustment (P) and randomization parameter (R) represent the exploration of the algorithm. X_{min} and X_{max} represents the vectors of the lower and upper limits of the dimensions.

2.3 Fuzzy Marine Predator Algorithm (FMPA)

The main approach given to the adaptation of dynamic parameters using fuzzy systems is to improve the quality of the results obtained by performing a better local and global search than with the MPA method. The metric used in the input of the fuzzy system is the percentage of elapsed iterations defined by [27], Eq. (8). The output is the fuzzy dynamic adjustment of the FMPA main parameters, the CF parameter that represents the exploitation of the search space defined by Eq. (9), the value is considered fuzzy since it is updated within the FMPA progress and is determined by Eq. (9), where the CF values are in the range [0, 1]. The FADs effect in the range [0.1, 0.3], along with long steps of Lévy strategy greatly to avoid local optima stagnation and better performance of the method, the adjustment (P) with the range [0.35, 0.75] improves algorithm exploration process.

$$Iteration(Iter) = \frac{Current\ Iteration(I)}{Total\ of\ Iterations(I_{max})} \quad (8)$$

Equation (8) generates a percentage of iterations generated to detect the CF values. It is initialized with very low CF values so that the algorithm reaches alternation and then achieves an increase. A process on which the three phases of the FMPA depend on the exploration and exploitation of the search space.

$$CF = \frac{\sum_{i=1}^{r_{cf}} \mu_i^{cf} (cf_{li})}{\sum_{i=1}^{r_{cf}} \mu_i^{cf}} \quad (9)$$

where CF is the speed ratio or when a predator moves towards the prey (fast-slow); r_{cf} is the number of rules of the fuzzy system corresponding to cf ; cf_{li} is the output result for rule i corresponding to cf ; μ_i^{cf} is the membership function of rule i corresponding to cf .

Algorithm 1 shows the pseudo-code of the proposed method.

Algorithm 1 Pseudocode for Fuzzy Predators Marine

Start stage 1: (Initialization)

1. Initialize Search agents (Prey) population $i = 1, 2, \dots, n$
2. Initialize FMPA fuzzy logic system.
3. While termination criterio are not met:

Calculate the fitness, construct the Elite Matrix and accomplish memory.

Marine memory Saving

Fuzzy evaluation of the main parameters P, FAD and CF by FMPA

Start stage 2:(FMPA optimization process is divided into three main phases)

1. **If** $\text{iter} < \text{Max_Iter}/3$

Update prey based on Eq. (4):

2. **Else if** $\text{Max_Iter}/3 < \text{Iter} < 2 * \text{Max_Iter}/3$

For the first half of the populations ($i = 1, \dots, n/2$)

Update prey based on Eq. (5a):

For the second half of the populations ($i = n/2, \dots, n$)

Update prey based on Eq. (5b):

3. **Else if** $\text{Iter} > 2 * \text{Max_Iter}/3$

Update prey based on Eq.(6):

End (if)

Start stage 3: (Detecting top predator)

Recalculate and evaluate fitness

Accomplish memory saving and Elite update

Applying FADs effect and update based on Eq. (7)

End while

3 Fuzzy Logic Systems

The current research proposes to use the architecture of a Type-2 Fuzzy Inference System, described in Fig. 2. Equations (10) and (11), the upper and lower MF are described in a fuzzy set of Interval type 2. where N rules form a Type-2 fuzzy rule base, as follows [28], Eq. (12). The proposed fuzzy system (FMPA) is composed of Interval Type-2 Fuzzy Sets, granulated into two trapezoidal Membership Functions, and one Triangular Membership Functions (IT2) in the input, and the output is composed of three Interval Type-2 Fuzzy Sets, granulated into three Interval Type-2 triangular membership functions. The trapezoidal function in the input Eqs. (10) and (11) is described as:

$$\bar{\mu}_{\bar{A}}(x) = \begin{cases} 0, & \text{if } x \leq a \\ \frac{x-a}{b-a}, & \text{if } a < x \leq b \\ 1, & \text{if } b < x \leq c \\ \frac{(d-x)}{(d-c)}, & \text{if } c < x \leq d \\ 0, & \text{if } x > d \end{cases} \quad (10)$$

$$\mu_{-\bar{A}}(x) = \begin{cases} 0, & \text{if } x \leq e \\ \frac{x-e}{f-e}\mu_f, & \text{if } e \leq x \leq f \\ \frac{(g-x)}{(g-f)}\mu_f, & \text{iff } < x \leq g \\ 0, & \text{if } x > g \end{cases} \quad (11)$$

$$R^i : \text{if } x_1 \text{ is } x_1^l \text{ and } \dots \text{ and } \dots, x_p \text{ is } x_p^l \text{ then } (y \text{ is } y^l) \quad (12)$$

where $X_i^l (i = 1, \dots, p)$ and Y^l are type-2 fuzzy sets, and $x (x^1, \dots, x^l)$ and y are linguistic variables.

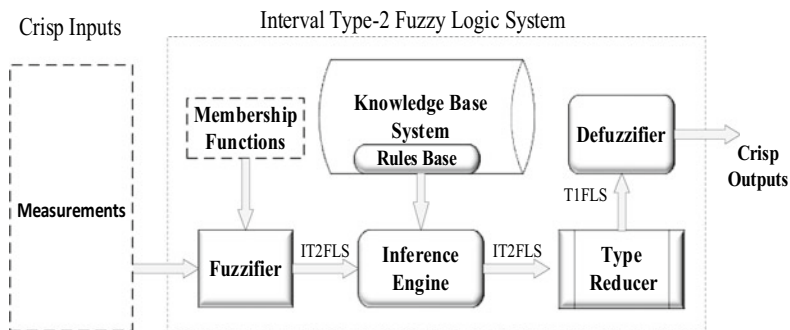


Fig. 2 IT2FLS architecture

The linguistic variable for the input is iteration (Iter), and it is composed of the following MFs: S, M and B. The fuzzy system has three outputs called P, FADs and CF which refers to the main parameters of the FMPA, illustrated in Fig. 3a, b.

The planning of the rules is based on the performance analysis of the algorithm parameters, in such a way that when starting the iterations, CF will explore and then in the final iterations it will exploit the search space, FADs will avoid falling into local optimum and P will contribute to the exploration. Table 1 shows how the result in P, FADs and CF will progressively increase as the iterations progress.

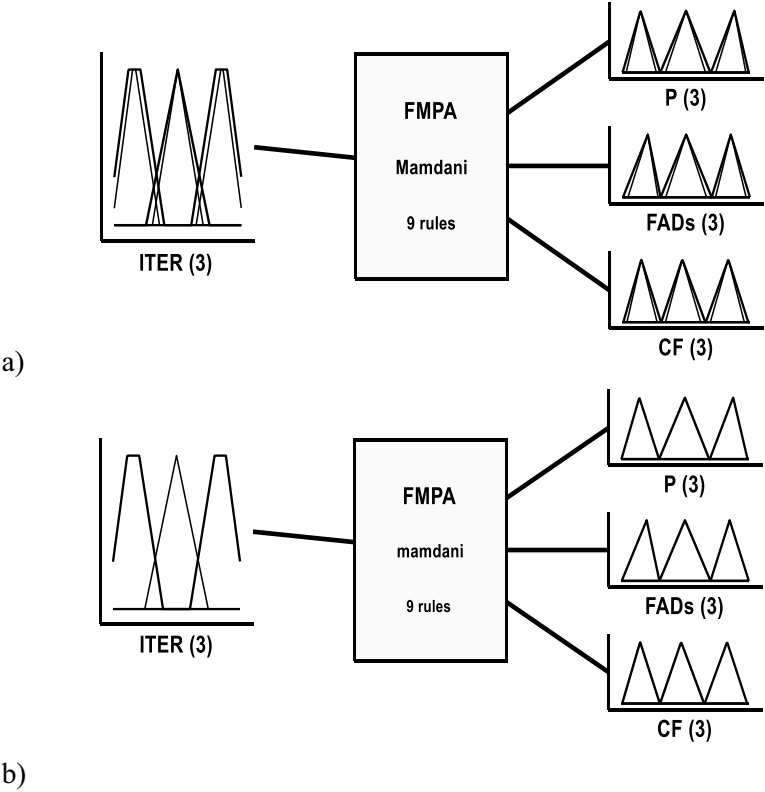


Fig. 3 a Type-1 and b IT2 FIS for the FMPA

Table 1 Rules for type-1 and IT2 fuzzy system for (FMPA)

Iter	P	FADs	CF
S	SN	SN	SN
M	M	M	M
B	BP	BP	BP

The parameter adjustment of the fuzzy FMPA algorithm presents the same rules in both inference systems, T1 and IT2. The parameter setting of the FMPA Type-1 and IT2 algorithm uses the input variable “Iteration (Iter)” and the output variables P, FADs and parameter CF. The input variable is granulated into three membership functions, the first and last trapezoidal and the central triangular named “S”, “M” and “B”. The output variables P, FADs and parameter CF. As shown in Fig. 1, each output variable is made up of three triangle membership functions designated “SN,” “M,” and “BP” As shown in Fig. 3.

As can be seen, the IT2FIS design applied to the adaptation of the FMPA algorithm parameters is similar to type-1, where the input and output membership function presented in Fig. 2b contains iteration as input and P, FADs and CF as outputs, with the same linguistic variables “SN”, “M” and “BP”. The fuzzy rules are the same for both fuzzy systems.

4 Study Cases

One of the few weaknesses that can be found in the design of an FLC (fuzzy logic controller) is the reliance on expert knowledge and a disability in autotuning that contributes to the trial and error search for an optimal controller. Due to this characteristic, it is considered to apply the fuzzy marine predator metaheuristic algorithm (FMPA), which is shown below through two case studies, first in Benchmark CEC-2017 functions to demonstrate its performance and a second case study to demonstrate its robustness to the adjust the membership functions of a path-following Type-2 fuzzy controller.

4.1 Case 1: Benchmark CEC-2017 Functions

In the analysis of the proposed method with respect to the MPA method, CEC-2017 mathematical functions were taken, defined in [29], which are a total of 29 functions and each one seeks to find values with convergence to the global minimum of each function. Functions that are used are shown in Table 2. It contains 29 functions of which the second half of the functions are among the challenging hybrid and composition functions for N problems. for example function 20 is the Hybrid Function of Happycat, Katsuura, Ackley, Rastrigin, Modified Schwefel and Schaffer and Composition Function 26 contain of Expanded Scaffer, Modified Schwefel, Griewank, Rosenbrock and Rastrigin.

Table 2 Benchmark CEC-2017 functions

Type	No	Functions	$F_i^* = F_i(x^*)$
Unimodal functions	1	Shifted and Rotated Bent Cigar function	100
	2	Shifted and Rotated Zakharov function	200
Simple multimodal functions	3	Shifted and Rotated Rosenbrock's function	300
	4	Shifted and Rotated Rastrigin's function	400
	5	Shifted and Rotated Expanded Scaffer's F6 function	500
	6	Shifted and Rotated Lunacek Bi_Rastrigin function	600
	7	Shifted and Rotated Non-Continuous Rastrigin's function	700
	8	Shifted and Rotated Levy function	800
	9	Shifted and Rotated Schwefel's function	900
Hybrid functions	10	Hybrid function 1 (N = 3)	1000
	11	Hybrid function 2 (N = 3)	1100
	12	Hybrid function 3 (N = 3)	1200
	13	Hybrid function 4 (N = 4)	1300
	14	Hybrid function 5 (N = 4)	1400
	15	Hybrid function 6 (N = 4)	1500
	16	Hybrid function 6 (N = 5)	1600
	17	Hybrid function 6 (N = 5)	1700
	18	Hybrid Function 6 (N = 5)	1800
	19	Hybrid function 6 (N = 6)	1900
Composition functions	20	Composition function 1 (N = 3)	2000
	21	Composition function 2 (N = 3)	2100
	22	Composition function 3 (N = 4)	2200
	23	Composition function 4 (N = 4)	2300
	24	Composition function 5 (N = 5)	2400
	25	Composition function 6 (N = 5)	2500
	26	Composition function 7 (N = 6)	2600
	27	Composition function 8 (N = 6)	2700
	28	Composition function 9 (N = 3)	2800
	29	Composition function 10 (N = 3)	2900
Search range: $[-100, 100]^D$			

4.2 Case 2: Optimization of Fuzzy Controllers

To meet the basic conditions of a autonomous mobile robot, established at the beginning of our article, the control proposal of the autonomous mobile robot is as follows: A differential robot model has been used, integrated with two independent driving wheels implemented on the same axis and as support a free front wheel [16, 30].

There are works where fuzzy adjustment has been made using metaheuristics to the controller for the mobile autonomous robot [31–33]. The main importance of the controller is the precision in the search for the desired trajectory, applying 18 rules that define the connections between the linguistic variables of the fuzzy inference system.

We propose the adjustment for a fuzzy controller of an Autonomous Mobile Robot, using the FMPA model proposed for this specific problem; in the following subsections, the fuzzy controller optimization and the fuzzy controller are explained in more detail Figs. 4, 5, 6 and 7.

Where Vr is the reference of the trajectory to follow; Va is the current trajectory of the robot; E and CE are the calculated error and the error change when calculating the difference of Vr minus Va ; U is our already optimized output of the fuzzy system; **disturbance** represents changes in the environment. The path to follow is described in Eq. (13).

$$Path = \begin{cases} v_d = 0.7 * (0.5 - \exp(-u(1))) \\ w_d = 0.5 * \sin(0.4 * u(1)) \end{cases} \quad (13)$$

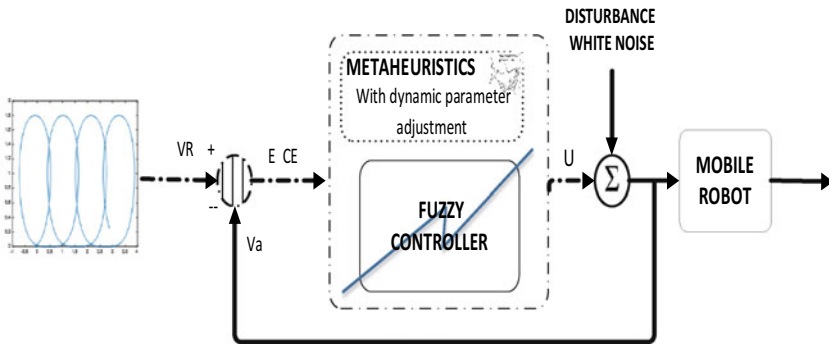


Fig. 4 Fuzzy controller optimization for an autonomous mobile robot

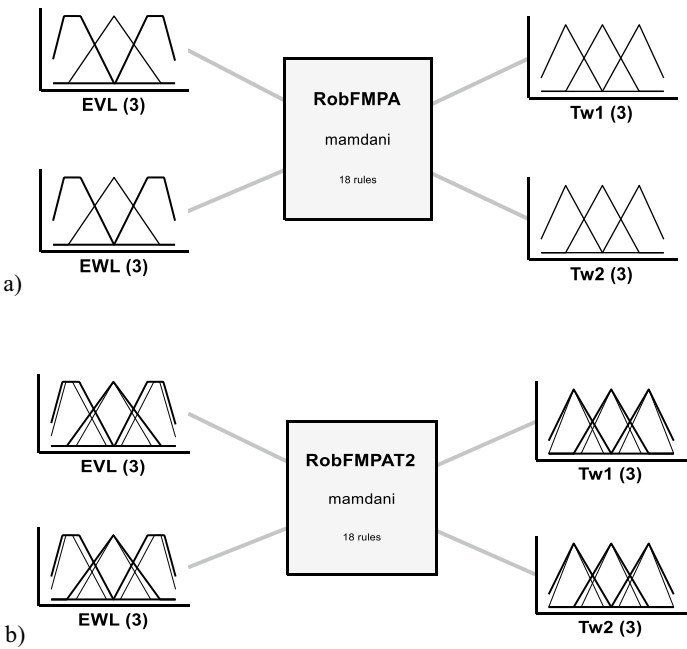


Fig. 7 Fuzzy controller: **a** Type-1 FLC, **b** Interval type-2 FLC

5 Results

The results obtained with optimization of benchmark functions using the fuzzy marine predators method (FMPA), it is being seen in the first part of this section. Advances in tuning fuzzy controller membership functions applied to autonomous mobile robot platforms are presented in the second subsection of this part. A vector of parameters values is obtained that describes the dynamic parameters of the adjusted membership functions.

5.1 Case 1 Results: Benchmark CEC-2017 Functions

Table 3 shows the parameters used in the search for the best performance of the Benchmark functions. Is considered that there are 10 dimension for all functions of this suite. Average and standard deviation are presented in Table 4. Establishing a

Table 3 Parameters used for FMPA and MPA methods

Population	Iterations	Dimensions
30	500	10

Table 4 Results for 10 Dim

Comparative 10D							
		FMFA type-1		FMFA type-2		MPA [20]	
FNC	f_i	Average	Dev. Std	Average	Dev. Std	Averag	Dev. Std
f_1	100	1.00×10^2	4.16×10^{-11}	1.00×10^2	0.00×10^0	1.00×10^2	4.79×10^{-6}
f_3	300	3.00×10^2	0.00×10^0	3.00×10^2	0.00×10^0	3.00×10^2	9.92×10^{-11}
f_4	400	4.00×10^{02}	1.24×10^{-1}	4.00×10^2	0.00×10^0	4.00×10^2	5.70×10^{-2}
f_5	500	5.04×10^2	1.02×10^0	5.00×10^2	0.00×10^0	5.10×10^2	3.95×10^0
f_6	600	6.00×10^2	7.98×10^{-3}	6.00×10^2	0.00×10^0	6.00×10^2	6.16×10^{-4}
f_7	700	7.15×10^2	2.24×10^0	7.00×10^2	0.00×10^0	7.23×10^2	3.91×10^0
f_8	800	8.03×10^2	1.59×10^0	8.00×10^2	0.00×10^0	8.09×10^2	3.12×10^0
f_9	900	9.00×10^0	1.61×10^{-8}	9.01×10^2	1.19×10^{-3}	9.00×10^2	1.63×10^{-2}
f_{10}	1000	1.10×10^3	6.16×10^1	1.00×10^3	0.00×10^0	1.43×10^3	1.41×10^0
f_{11}	1100	1.10×10^3	7.20×10^{-1}	1.10×10^3	5.23×10^{-1}	1.10×10^3	1.27×10^0
f_{12}	1200	1.22×10^3	5.36×10^0	1.20×10^{03}	2.04×10^0	1.24×10^3	5.43×10^1
f_{13}	1300	1.30×10^3	1.30×10^0	1.30×10^3	1.92×10^0	1.30×10^3	2.58×10^0
f_{14}	1400	1.40×10^3	5.20×10^{-1}	1.40×10^{03}	1.39×10^{-1}	1.40×10^3	4.06×10^0
f_{15}	1500	1.50×10^3	2.90×10^{-2}	1.50×10^3	4.81×10^{-2}	1.50×10^3	5.20×10^1
f_{16}	1600	1.60×10^3	2.14×10^{-1}	1.60×10^3	8.07×10^{-2}	1.60×10^3	9.90×10^1
f_{17}	1700	1.70×10^3	7.97×10^{-1}	1.70×10^3	5.28×10^{-1}	1.71×10^3	9.44×10^0
f_{18}	1800	1.80×10^3	3.94×10^{-1}	1.80×10^3	9.26×10^{-2}	1.80×10^3	5.20×10^1
f_{19}	1900	1.90×10^3	3.01×10^{-1}	1.90×10^3	3.20×10^{-1}	1.90×10^3	9.67×10^0

equal footing where each method is tested 30 times with 500 iterations limited to 50,000 function evaluations, with the author of the MPA algorithm. It is the same procedure for T1 as IT2 FIS.

5.2 Case 2: Dynamic Adjustment of Fuzzy Controller Parameters

Table 5 shows the main parameters used in the FMPA, and MPA method, Tables 6 and 7 shows the 30 runs performed to obtain the optimal fuzzy system with FMPA and MPA method, taking the mean square error (MSE) as a metric, described in Eq. (14) shown below.

$$MSE = \frac{1}{n} \sum_{i=1}^n (X_i - Y_i)^2 \quad (14)$$

Table 5 Main parameters used in the experimentation with FMPA type-2 and MPA algorithm

Population	Iterations	Dimensions
50	10	75

Table 6 Results of the experiments with the interval type-2 FLC adjusted with MPA

Experiment	MSE	Experiment	MSE
1	1.47×10^{-3}	16	5.21×10^{-4}
2	1.60×10^{-3}	17	9.98×10^{-4}
3	1.07×10^{-3}	18	5.19×10^{-4}
4	7.54×10^{-4}	19	1.12×10^{-3}
5	1.71×10^{-3}	20	8.20×10^{-4}
6	8.71×10^{-4}	21	3.03×10^{-4}
7	6.05×10^{-3}	22	1.83×10^{-3}
8	3.67×10^{-3}	23	6.88×10^{-4}
9	3.35×10^{-3}	24	3.56×10^{-4}
10	4.85×10^{-3}	25	2.48×10^{-3}
11	2.60×10^{-3}	26	5.36×10^{-4}
12	3.07×10^{-3}	27	1.14×10^{-4}
13	3.04×10^{-3}	28	1.05×10^{-4}
14	8.19×10^{-4}	29	6.74×10^{-3}
15	1.68×10^{-3}	30	2.54×10^{-4}

Table 7 Results of the experiments with the interval type-1 FLC adjusted with FMPA

Experiment	MSE	Experiment	MSE
1	4.10×10^{-4}	16	2.27×10^{-4}
2	7.29×10^{-4}	17	1.32×10^{-4}
3	6.90×10^{-4}	18	2.25×10^{-4}
4	5.77×10^{-4}	19	1.28×10^{-4}
5	6.65×10^{-4}	20	2.06×10^{-4}
6	1.39×10^{-4}	21	1.04×10^{-4}
7	1.84×10^{-4}	22	3.57×10^{-4}
8	1.64×10^{-4}	23	1.61×10^{-4}
9	1.37×10^{-4}	24	1.45×10^{-4}
10	1.79×10^{-4}	25	1.58×10^{-4}
11	1.22×10^{-4}	26	1.67×10^{-4}
12	3.63×10^{-4}	27	3.51×10^{-4}
13	9.14×10^{-6}	28	3.02×10^{-4}
14	2.63×10^{-4}	29	3.11×10^{-4}
15	3.99×10^{-4}	30	3.39×10^{-4}

where X_i is the reference value at the time i ; Y_i is the value produced by the system at the time i , and n is the number of samples taken in the test.

Tables 6, 7 and 8, highlight a change in the results obtained with the MPA method, it has the best MSE of 1.14×10^{-4} , the worst is 6.74×10^{-3} . While our Type-1

Table 8 Results of the experiments with the interval type-2 FLC adjusted with FMPA

Experiment	MSE	Experiment	MSE
1	3.93×10^{-5}	16	3.10×10^{-4}
2	3.19×10^{-5}	17	1.98×10^{-4}
3	6.05×10^{-5}	18	7.04×10^{-4}
4	9.85×10^{-5}	19	3.26×10^{-4}
5	6.16×10^{-5}	20	1.66×10^{-4}
6	1.22×10^{-5}	21	1.30×10^{-4}
7	1.11×10^{-5}	22	9.58×10^{-5}
8	5.82×10^{-5}	23	7.35×10^{-5}
9	3.61×10^{-5}	24	3.54×10^{-4}
10	1.29×10^{-5}	25	3.58×10^{-4}
11	2.22×10^{-5}	26	1.04×10^{-4}
12	8.62×10^{-4}	27	7.83×10^{-5}
13	1.38×10^{-4}	28	9.52×10^{-5}
14	4.33×10^{-4}	29	3.92×10^{-4}
15	4.30×10^{-4}	30	7.52×10^{-5}

Table 9 Summarized the results

	FMPA type-1	FMPA type-2	MPA
Average	2.81×10^{-4}	2.16×10^{-4}	1.6×10^{-3}
Standard deviation	1.80×10^{-4}	2.34×10^{-4}	1.43×10^{-3}

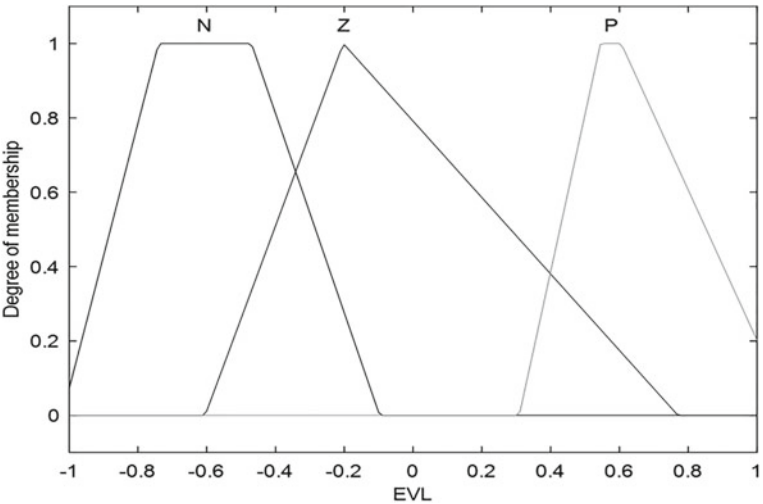


Fig. 8 Input 1: adjusted EVL

proposal gets a better MSE of 9.14×10^{-5} , the worst is 7.29×10^{-4} . However, our proposal with Type-2 FLS achieves a minimal variation in the results; the best MSE with 1.11×10^{-5} , the worst with 8.62×10^{-4} . So it can be concluded that our fuzzy optimized algorithm can perform better in this specific problem.

Table 9 shows a summary of the average results and standard deviation of the optimization methods.

Figures 8, 9, 10, 11, 12, 13, 14 and 15 show the result of the dynamic adjustment of the parameters values of the membership functions of the inputs *evl*, *ewl* and the outputs *Tw1* and *Tw2*, with T1 and IT2FLC respectively.

5.3 Statistical Comparison

In order to validate the performance of the metaheuristic algorithms used to test the proposed optimization methodology of membership functions parameters of the tracking fuzzy controller, a statistical comparison is made to find evidence that the algorithms have performed well in the case study of the differential robot plant. Table 10 shows the representation of the values used in the statistical test, to determine

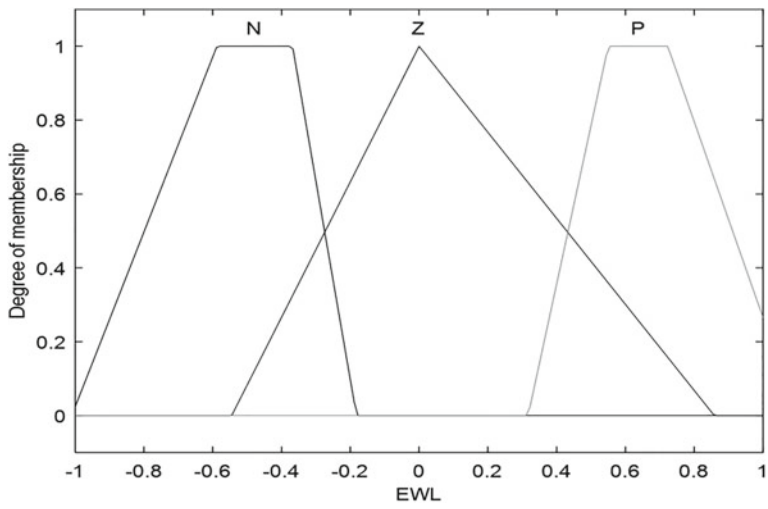


Fig. 9 Input 2: adjusted EWL

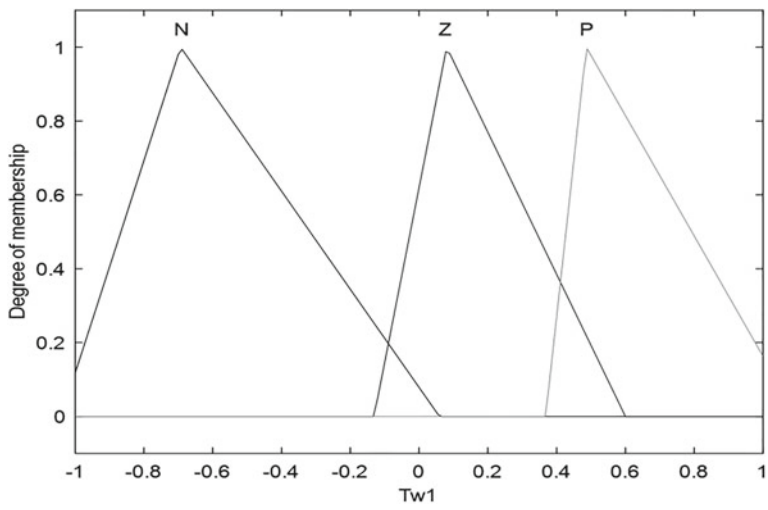


Fig. 10 Output 1: adjusted Tw1

which method produced better results in the optimization of membership function parameters, generating a data vector, which helps to improve the performance of the fuzzy controller for the trajectory tracking process.

Table 11, the null hypothesis (H_0) states that the average MSE error for the FMPS with IT2FLS (μ_1) is greater than or equal to the average MSE error with MPA algorithm (μ_2). The alternative hypothesis (H_a) states that the average MSE

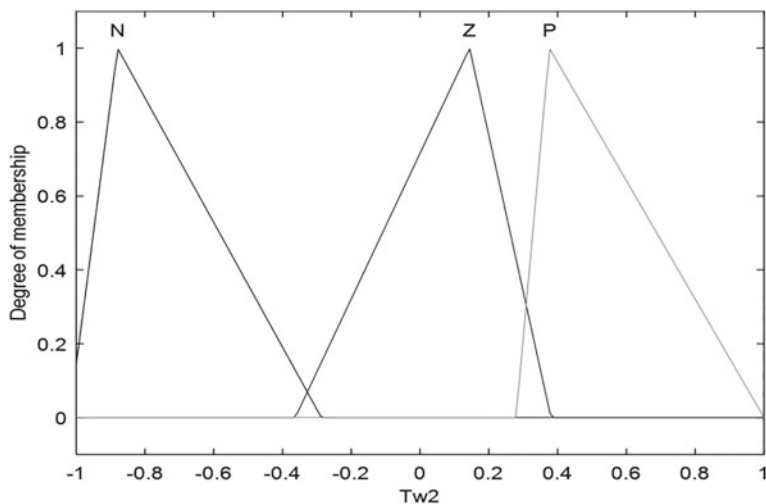


Fig. 11 Output 2: adjusted Tw2

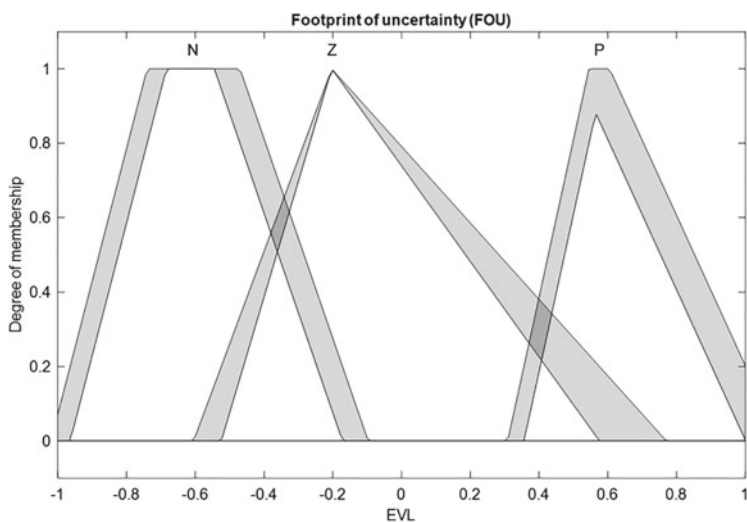


Fig. 12 Input 1: adjusted EVL of type-2

error for the FMFA with IT2FLS (μ_1) is less than the average MSE error for MPA algorithm (μ_2).

The Z statistical test is used, which is based on Eq. (15).

$$Z = \frac{(\bar{X}_1 - \bar{X}_2) - (\mu_1 - \mu_2)}{(\sigma_1 - \sigma_2)} \quad (15)$$

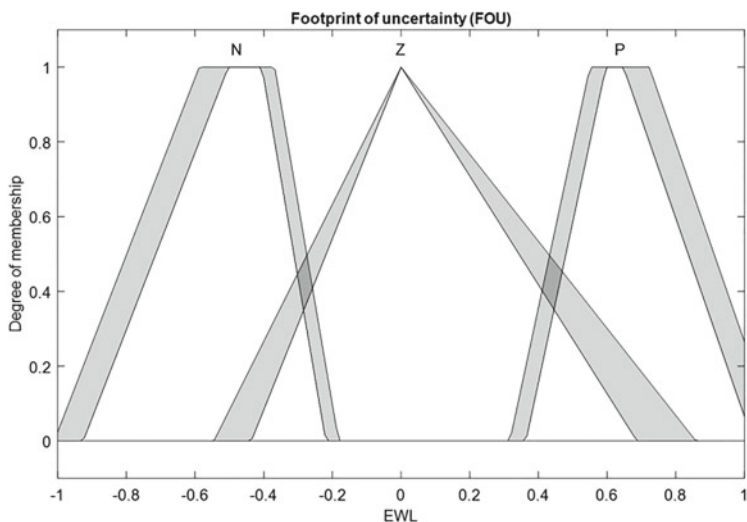


Fig. 13 Input 2: adjusted EWL of type-2

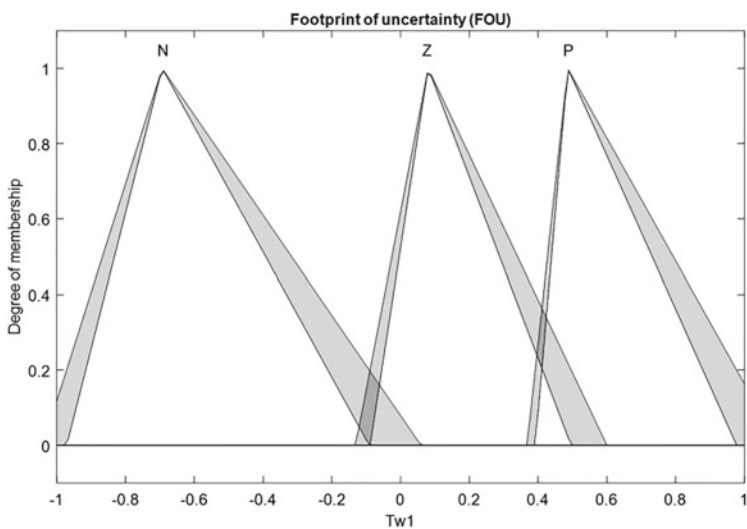


Fig. 14 Output 1: adjusted Tw1 of type-2

6 Conclusions

The proposed methodology was tested with the CEC-2017 reference functions [29] and the MPA method [20]. With a maximum of 30 experiments, with the same parameters for an even comparison, showing results that indicate that FMPA optimizes the

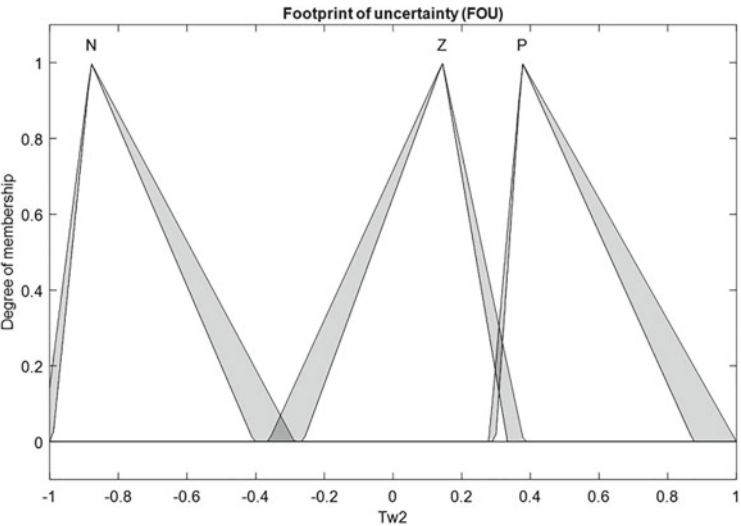


Fig. 15 Output 2: adjusted Tw2 of type-2

Table 10 Statistical data

Variable	Number of samples	Mean	Standard deviation
MPA	30	0.00159	0.0015
FMPA	30	0.00022	0.0002

Table 11 Values used in the statistical z test

Parameters	Value
H_O	$\mu_1 \geq \mu_2$
H_a (claim)	$\mu_1 < \mu_2$
z (observed value)	-4.958
z (critica value)	-1.645
Valor-p (one-tailed)	0.999
Alpha	0.05

reference functions in a better way. On the other hand, the results of the parameter adjustment of the controller membership functions performed with the interval Type-2 Fuzzy Logic Controller (IT2FLC) were very good, as concluded in this investigation. In addition, it was found that the punctual values of the controller membership functions found are close to the set parameters required for the optimal control, so it can be established that the performance of a fuzzy controller can be improved by applying metaheuristic optimization. We did compare two related metaheuristics, our fuzzy proposal and the original MPA in this article, which proved effective in parameter optimization of membership functions. The simulation results show that

the FMPA with IT2 FIS outperforms the MPA for controlling the robot in following trajectory.

In future works, other types of fuzzy systems [34, 35] that handle higher level of uncertainty and that we have not experienced yet for the adjustment of dynamic parameters in metaheuristic algorithms will be considered on and will be used for some trajectory control challenges in future works, and more experiments will be carried out [36].

References

1. Cuevas, F., Castillo, O.: Design and implementation of a fuzzy path optimization system for omnidirectional autonomous mobile robot control in real-time (2018). https://doi.org/10.1007/978-3-319-71008-2_19
2. Ahmed, B.T., Abdulhameed, O.Y.: Fingerprint Authentication using shark smell optimization algorithm. *UHD J. Sci. Technol.* **4**, 28 (2020). <https://doi.org/10.21928/uhdjst.v4n2y2020.pp28-39>
3. Peraza, C., Valdez, F., Castro, J.R., Castillo, O.: Fuzzy dynamic parameter adaptation in the harmony search algorithm for the optimization of the ball and beam controller. *Adv. Oper. Res.* **2018** (2018). <https://doi.org/10.1155/2018/3092872>
4. Bernal, E., Castillo, O., Soria, J., Valdez, F.: Optimization of fuzzy controller using galactic swarm optimization with type-2 fuzzy dynamic parameter adjustment. *Axioms* **8** (2019). <https://doi.org/10.3390/axioms8010026>
5. Castillo, O., Melin, P.: Experimental study of intelligent controllers under uncertainty using type-1 and type-2 fuzzy logic. *Stud. Fuzziness Soft Comput.* **223**, 121–132 (2008). https://doi.org/10.1007/978-3-540-76284-3_10
6. Cuevas, F., Castillo, O., Cortes, P.: Optimal setting of membership functions for interval type-2 fuzzy tracking controllers using a shark smell metaheuristic algorithm (2021). <https://doi.org/10.1007/s40815-021-01136-4>
7. Beirami, H., Zerafat, M.M.: Self-tuning of an interval type-2 fuzzy PID controller for a heat exchanger system. *Iran. J. Sci. Technol. Trans. Mech. Eng.* **39**, 113–129 (2015). <https://doi.org/10.22099/ijstm.2015.2953>
8. Gnanasekaran, N., Chandramohan, S., Kumar, P.S., Mohamed Imran, A.: Optimal placement of capacitors in radial distribution system using shark smell optimization algorithm. *Ain Shams Eng. J.* (2016). <https://doi.org/10.1016/j.asej.2016.01.006>
9. Yang, X.S.: *Nature-Inspired Optimization Algorithms*. Elsevier Inc. (2014). <https://doi.org/10.1016/C2013-0-01368-0>
10. Hsu, C.H., Juang, C.F.: Evolutionary robot wall-following control using type-2 fuzzy controller with species-DE-activated continuous ACO. *IEEE Trans. Fuzzy Syst.* **21**, 100–112 (2013). <https://doi.org/10.1109/TFUZZ.2012.2202665>
11. Mirjalili, S., Hashim, S.Z.M.: A new hybrid PSO-GSA algorithm for function optimization. In: *Proceedings of ICCIA 2010—2010 International Conference on Computer and Information Application*, pp. 374–377 (2010). <https://doi.org/10.1109/ICCIA.2010.6141614>
12. de la O, D., Castillo, O., Astudillo, L., Soria, J.: Fuzzy chemical reaction algorithm with dynamic adaptation of parameters. In: *Advances in Intelligent Systems and Computing*, pp. 122–130 (2018). https://doi.org/10.1007/978-3-319-67137-6_13
13. Amézquita, L., Castillo, O., Soria, J., Cortes-Antonio, P.: Optimal design of fuzzy controllers using the Multiverse optimizer. *Adv. Intell. Syst. Comput.* 1–12 (2021)
14. Cuevas, F., Castillo, O., Cortes-Antonio, P.: Towards an adaptive control strategy based on type-2 fuzzy logic for autonomous mobile robots. In: *Proceedings of 2019 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE)*, pp. 1–6. IEEE Press

15. Masmoudi, M.S., Krichen, N., Masmoudi, M., Derbel, N.: Fuzzy logic controllers design for omnidirectional mobile robot navigation. *Appl. Soft Comput. J.* **49**, 901–919 (2016). <https://doi.org/10.1016/j.asoc.2016.08.057>
16. Melin, P., Astudillo, L., Castillo, O., Valdez, F., Garcia, M.: Optimal design of type-2 and type-1 fuzzy tracking controllers for autonomous mobile robots under perturbed torques using a new chemical optimization paradigm. *Expert Syst. Appl.* **40**, 3185–3195 (2013). <https://doi.org/10.1016/j.eswa.2012.12.032>
17. Castillo, O., Amador-Angulo, L., Castro, J.R., Garcia-Valdez, M.: A comparative study of type-1 fuzzy logic systems, interval type-2 fuzzy logic systems and generalized type-2 fuzzy logic systems in control problems. *Inf. Sci. (NY)* **354** (2016). <https://doi.org/10.1016/j.ins.2016.03.026>
18. Olivas, F., Valdez, F., Castillo, O., Gonzalez, C.I., Martinez, G., Melin, P.: Ant colony optimization with dynamic parameter adaptation based on interval type-2 fuzzy logic systems. *Appl. Soft Comput. J.* **53**, 74–87 (2017). <https://doi.org/10.1016/j.asoc.2016.12.015>
19. Lagunes, M.L., Castillo, O., Soria, J., Garcia, M., Valdez, F.: Optimization of granulation for fuzzy controllers of autonomous mobile robots using the Firefly Algorithm. *Granul. Comput.* **4**, 185–195 (2019). <https://doi.org/10.1007/s41066-018-0121-6>
20. Faramarzi, A., Heidarinejad, M., Mirjalili, S., Gandomi, A.H.: Marine Predators algorithm: a nature-inspired metaheuristic. *Expert Syst. Appl.* **152**, 113377 (2020). <https://doi.org/10.1016/j.eswa.2020.113377>
21. Zadeh, L.A.: Soft computing and fuzzy logic. *IEEE Softw.* **11**, 48–56 (1994). <https://doi.org/10.1109/52.329401>
22. Wu, D.: On the fundamental differences between interval type-2 and type-1 fuzzy logic controllers. *IEEE Trans. Fuzzy Syst.* (2012). <https://doi.org/10.1109/TFUZZ.2012.2186818>
23. Castillo, O., Melin, P.: A review on interval type-2 fuzzy logic applications in intelligent control. *Inf. Sci. (Ny)*. (2014). <https://doi.org/10.1016/j.ins.2014.04.015>
24. Castillo, O., Melin, P., Alanis, A., Montiel, O., Sepulveda, R.: Optimization of interval type-2 fuzzy logic controllers using evolutionary algorithms. *Soft Comput.* (2011). <https://doi.org/10.1007/s00500-010-0588-9>
25. Ho, L.V., Nguyen, D.H., Mousavi, M., De Roeck, G., Bui-Tien, T., Gandomi, A.H., Wahab, M.A.: A hybrid computational intelligence approach for structural damage detection using marine predator algorithm and feedforward neural networks. *Comput. Struct.* **252**, 106568 (2021). <https://doi.org/10.1016/j.compstruc.2021.106568>
26. Riad, N., Anis, W., Elkassas, A., Hassan, A.E.W.: Three-phase multilevel inverter using selective harmonic elimination with marine predator algorithm. *Electron.* **10**, 1–21 (2021). <https://doi.org/10.3390/electronics10040374>
27. Olivas, F., Valdez, F., Castillo, O., Melin, P.: Dynamic parameter adaptation in particle swarm optimization using interval type-2 fuzzy logic. *Soft Comput.* **20** (2016). <https://doi.org/10.1007/s00500-014-1567-3>
28. Liang, Q., Mendel, J.M.: Interval type-2 fuzzy logic systems: theory and design. *IEEE Trans. Fuzzy Syst.* **8**, 535–550 (2000). <https://doi.org/10.1109/91.873577>
29. Awad, N.H., Ali, M.Z., Liang, J.J., Qu, B.Y., Suganthan, P.N.: CEC 2017 special session on single objective numerical optimization single bound constrained real-parameter numerical optimization. *IEEE Congr.* (2017)
30. Castillo, O., Aguilar, L., Cárdenas, S.: Fuzzy Logic tracking control for unicycle mobile robots. *Eng. Lett.* **4** (2006)
31. Amador-Angulo, L., Mendoza, O., Castro, J.R., Rodríguez-Díaz, A., Melin, P., Castillo, O.: Fuzzy sets in dynamic adaptation of parameters of a bee colony optimization for controlling the trajectory of an autonomous mobile robot. *Sensors (Switzerland)*. (2016). <https://doi.org/10.3390/s16091458>
32. Hossain, M.A., Ferdous, I.: Autonomous robot path planning in dynamic environment using a new optimization technique inspired by bacterial foraging technique. *Rob. Auton. Syst.* **64**, 137–141 (2015). <https://doi.org/10.1016/j.robot.2014.07.002>

33. Cuevas, F., Castillo, O., Cortes-Antonio, P.: Dynamic optimal parameter setting with fuzzy argument to metaheuristic algorithm variant for fuzzy tracking controllers. Presented at the INFUS 2021 Conference, Istanbul, Turkey (2021). https://doi.org/10.1007/978-3-030-85626-7_62.
34. Ontiveros-Robles, E., Melin, P., Castillo, O.: Comparative analysis of noise robustness of type 2 fuzzy logic controllers. *Kybernetika* **54**, 175–201 (2018). <https://doi.org/10.14736/kyb-2018-1-0175>
35. Ontiveros, E., Melin, P., Castillo, O.: High order α -planes integration: a new approach to computational cost reduction of general type-2 fuzzy systems. *Eng. Appl. Artif. Intell.* **74**, 186–197 (2018). <https://doi.org/10.1016/j.engappai.2018.06.013>
36. Olivas, F., Valdez, F., Melin, P., Sombra, A., Castillo, O.: Interval type-2 fuzzy logic for dynamic parameter adaptation in a modified gravitational search algorithm. *Inf. Sci.* **476**, 159–175 (2019)

Evaluation of Times and Best Solutions of MFO, LSA and PSO Using Parallel Computing, Fuzzy Logic Systems and Migration Blocks Together to Evaluate Benchmark Functions



Yunkio Kawano, Fevrier Valdez, and Oscar Castillo

Abstract In this chapter, we propose the creation of a development environment in which we will work with each of the optimization algorithms using parallel computing in order to apply the use of migration blocks that allow us to share information between each of the populations of the algorithms, as well as the use the Fuzzy logic systems that allows us to adjust the migration percentage that we will use and the number of times or how often the migration will be applied. The algorithms that we going to use are the following: Moth-Flame Optimization, Lightning Search Algorithm and Particle Swarm Optimization, where on the other hand the use of Fuzzy logic will be applied internally in the process each of the algorithms, which allows us the dynamic adjustment of parameters of each algorithm separately. The algorithms used will be evaluated by the results obtained as well as the individual or total duration of each benchmark function that we are using.

Keywords Parallel · PSO · MFO · LSA · Fuzzy · Migration

1 Introduction

Optimization algorithms can show us great performance for highly complex problems in term of the difficulty that may arise according to the mathematical equation that we are using or the high number of dimensions, which leads us to a very complex problem to be solved, it is said that the classic search methods almost always get stuck in the local minimums and that is why very bad results are obtained when working with high dimensions, such as genetic algorithm (GA), ant colony optimization (ACO) or it can also be the particle swarm optimization (PSO) but the last case we integrate it together with two other algorithms described later [1–3].

The GA, ACO and PSO algorithms are some of the oldest in the literature, which is why they have certain problems when looking for the best solution, although each of them are for different optimization problems. ga is the genetic algorithm that its

Y. Kawano · F. Valdez (✉) · O. Castillo
Tijuana Institute of Technology, Tijuana, BC, Mexico
e-mail: fevrier@tectijuana.mx

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2022
O. Castillo and P. Melin (eds.), *New Perspectives on Hybrid Intelligent System Design based on Fuzzy Logic, Neural Networks and Metaheuristics*, Studies in Computational Intelligence 1050, https://doi.org/10.1007/978-3-031-08266-5_14

205

Table 1 Specs of the used hardware

Specifications	Description	Amount
CPU	Intel i5-9400f	2.9 Ghz, 6 cores
RAM	DDR4 2666 MHz	16 GB
Solid disk	SSD M.2 Crucial P2	500 GB
Software	MATLAB 2017b	R:2300 W: 940 Mb/s

inspiration goes according to its name that would be the genetic reproduction this algorithm I believe that it serves to decrypt since the nature of the algorithm makes all the possible combinations by the chromosome. The ACO algorithm is inspired by ant colonies, it is normally used to find the shortest path, such as the one in the traveling agent problem. PSO is inspired by how birds communicate to search for food. There are new algorithms that are inspired by some behavior of some other animal [4, 5] or living being. As well as there are some that are inspired by phenomena that occur in the natural environment such as lightning, or water currents, etc. [6–8].

In this case, we be working with three optimization algorithms, all three works with populations but one is inspired by the search for food within swarms of birds or fish banks that is a particle swarm optimization (PSO), the next is inspired by how moths move great distances in the darkness of the night towards the moon that is the moth-flame optimization (MFO) and finally we have the one who is inspired by the natural creation of lightning and how each channel or arm of lightning is a possible solution to the problem this is a lightning search algorithm (LSA).

In Table 1 shows the hardware specifications that our computer equipment has to carry out the experiments.

The experiments were run on the MATLAB platform. We use three optimization algorithms which all three work simultaneously in order to share information between them or to obtain better results.

In order to compare the results obtained with other papers, we use the following benchmark functions that will be shown in Table 2.

In the section that follows, a short description about the three optimization algorithms is shown. These are the ones we use to make our algorithm that includes all three and that we apply migration between them.

2 Particle Swarm Optimization (PSO)

The PSO algorithm is search and optimization technique in the field of machine learning. It was developed by Eberhart and Kennedy in 1995 and is inspired by the swarming social behavior of individuals, whether fish or birds in search of food, which are found within a specific search space.

The following equation represents the update of the velocity vector of each particle i in each iteration k (see Table 3).

Table 2 Set of Benchmark Mathematical functions

No.	Function	Range
F1	$\sum_{i=1}^n x_i^2$	$[-5.12, 5.12]$
F2	$\sum_{i=1}^n x_i + \prod_{i=1}^n x_i $	$[-10, 10]$
F3	$\sum_{i=1}^n \left(\sum_{j=1}^i x_j \right)$	$[-100, 100]$
F4	$\max\{ x_i , 1 \leq i \leq n\}$	$[-100, 100]$
F5	$\sum_{i=1}^{n-1} \left[100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2 \right]$	$[-30, 30]$
F6	$\sum_{i=1}^n ([x_i + 0.5])^2$	$[-100, 100]$
F7	$\sum_{i=1}^n i x_i^4 + random[0, 1)$	$[-1.28, 1.28]$
F8	$\sum_{i=1}^n [1 - x_i \sin(\sqrt{ x_i })]$	$[-500, 500]$
F9	$\sum_{i=1}^n [x_i^2 - 10 \cos(2\pi x_i) + 10]$	$[-5.12, 5.12]$
F10	$-20 \exp\left(-0.2 \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}\right) - \exp\left(\frac{1}{n} \sum_{i=1}^n \cos(2\pi x_i)\right) + 20 + e$	$[-32.768, 32.768]$
F11	$\frac{1}{4000} \sum_{i=1}^n x_i^2 - \prod_{i=1}^n \cos\left(\frac{x_i}{\sqrt{i}}\right) + 1$	$[-600, 600]$
F12	$\frac{\pi}{n} \left\{ 10 \sin^2(\pi y_i) + \sum_{i=1}^{n-1} (y_i - 1)^2 [1 + 10 \sin^2(\pi y_{i+1})] + (y_n - 1)^2 \right\} + \sum_{i=1}^n u(x_i, 10, 100, 4)$ $y_i = 1 + \frac{x_i + 1}{4}, u(x_i, a, k, m) = \begin{cases} k(x_i - a)^m, & x_i > a \\ 0 - a < x_i < a \\ k(-x_i - a)^m, & x_i < -a \end{cases}$	$[-50, 50]$
F13	$0.1 \left\{ \sin^2(3\pi y_i) + \sum_{i=1}^{n-1} (x_i - 1)^2 [1 + \sin^2(3\pi y_{i+1})] + (x_n - 1)^2 [1 + \sin^2(2\pi y_n)] \right\} + \sum_{i=1}^n u(x_i, 5, 100, 4)$ $+ , u(x_i, a, k, m) = \begin{cases} k(x_i - a)^m, & x_i > a \\ 0 - a < x_i < a \\ k(-x_i - a)^m, & x_i < -a \end{cases}$	$[-50, 50]$

Table 3 Description of the factors used in the last equation

Factor	Description
v_i^k	Velocity between particle i and iteration k
ω	Inertial weight
φ_1, φ_2	Learning ratios that control of cognitive and social components
$rand_1, rand_2$	Vectors with random numbers between 0 and 1
x_i^k	Current position of particle i and iteration k
$pBest_i$	The best solution found
g_i	The particle position in $pBest_fitness$ of environment of p_i that is a local best

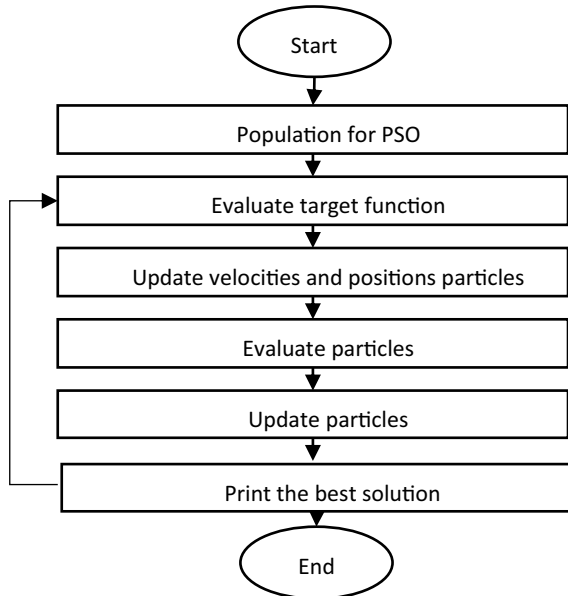
$$v_i^{k+1} = \omega \cdot v_i^k + \varphi_1 \cdot rand_1 \cdot (pBest_i - x_i^k) + \varphi_2 \cdot rand_2 \cdot (g_i - x_i^k) \quad (1)$$

Like all other algorithms, the beginning is from a population with random individuals, where each of them can be said to be a possible solution, depending on the problem, the solution will be easier. For this, the algorithm values that can be modified would be the inertial weight, the weights that control the cognitive and social components.

The following Figure shows the pseudocode of the PSO algorithm.

As shown in Fig. 1, the PSO algorithm creates an initial population, then evaluates with the objective function to obtain the best solution within that population, then the velocities and positions of the particles are update, and so on until the maximum number of iterations is completed [9–11].

Fig. 1 Brief flowchart of PSO algorithm



3 Moth Flame Optimization (MFO)

This optimization algorithm is focused on how moths move through the night, having as a reference point the moon to be able to fly great distances in a straight line, although nowadays, in the nature, there are many obstacles to doing that.

Thanks to the public lightning that confuses them and the moths think they are the moon but as they are closer, they only manage to fly around the lamps.

The mechanism they used is called transverse orientation, which is simply that the moth flies with a certain degree of inclination that causes it to fly in a spiral.

The algorithm assumes that moths and flames are among the possible solutions, where moths are search agents that move around the space and flames are the best position found so far by the moth. These moths fly close to the flame in case they find a better solution. The flame is updated.

The movement of the moth while in search of a better solution is in logarithmic spiral. Where at the beginning of the spiral is a moth and at the end it must be the position of the flame, it should be noted that the range of the spiral must not exceed the search space.

$$S(M_i, F_j) = D_i \cdot e^{bt} \cdot \cos(2\pi t) + F_j \quad (2)$$

where $D_i = |F_j - M_i|$ is the distance between the flame and moth where M_i is the current position of the moth in i and F_j is the current position of the flame in j . b it's a constant that defines the logarithmic spiral. t factor is a random number between negative 1 and positive 1. In the MFO algorithm, there is a constant that is S that is part of the spiral movement, which helps the moth to control to exploration and exploitation around the flames and that they are within the search space.

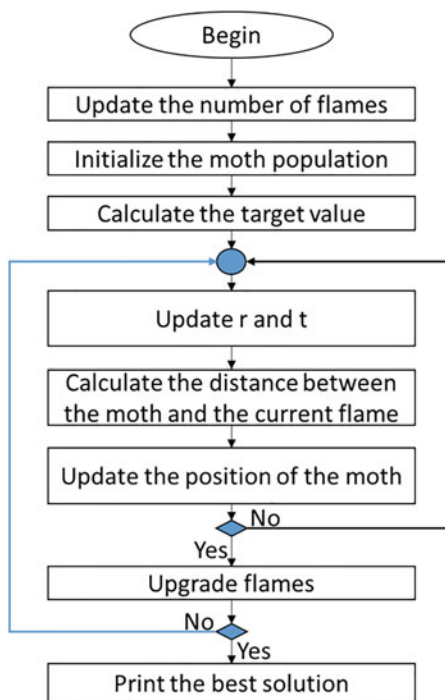
At a certain point of the algorithm, the update of the number of flames is applied since it helps us to improve the exploitation of MFO algorithm. Because the algorithm searches in various positions within the search space, which reduces the number of possibilities we have to exploit the best possible solutions.

According to the following equation it says that the fewer flames we have, they help the algorithm to find better solutions. We suppose it must be because each of the flames are the best solutions found by the moths.

$$total\ number\ of\ flames = \left(N - l * \frac{N - l}{T} \right) \quad (3)$$

where N is the total number of flames, l is the number in which the current iteration is located, and T shows the maximum number of iterations in the algorithm [12–15] (see Fig. 2).

Fig. 2 Flowchart of Moth-flame optimization algorithm



4 Lightning Search Algorithm (LSA)

LSA is an optimization algorithm that is inspired by the natural phenomenon of how lightning is created in the natural environment. A propagation mechanism is used in a staggered manner, which takes the form of an inverted tree, which is why the LSA algorithm has three types of projectiles, where the first projectile is a transition one which generates the first leading population, then continue the space projectiles that try to obtain a better position in the leadership range and finally we have the leading projectiles, which are the projectiles that have the best position (see Fig. 3).

The lightning is composed of different elements of nature, the main ones are a part of hydrogen, nitrogen and oxygen, and many more that help in the process. To generate the lightning with these elements they have to go at great speeds brushing between them until they collide and explode generating the atmosphere. Withing what is lightning, there are three zones or times, those of transition are the first ones that are the strongest or with more energy, then there are those of space that are half lightning where they lose their intensity and finally those of lead that they are found until the end of the lightning, which are the weakest but in the same way they are all a set where all are possible solutions in terms of the algorithm.

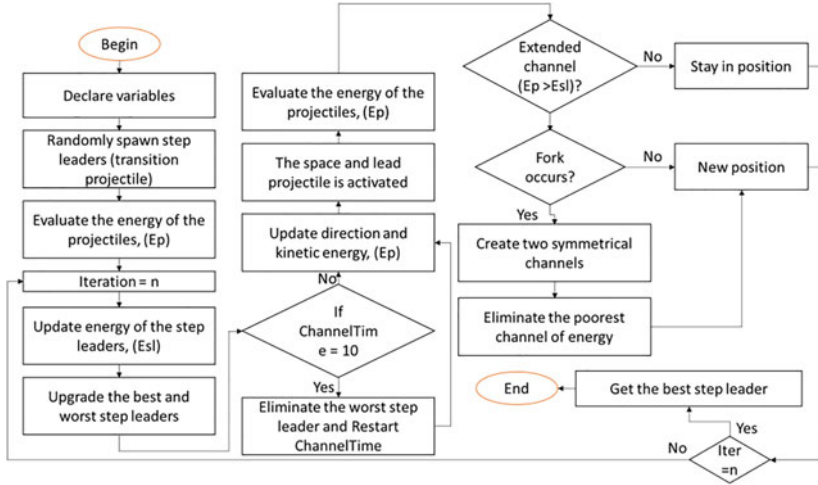


Fig. 3 Flowchart of lightning search algorithm

The velocity of a projectile is obtained with the following equation.

$$v_p = \left[1 - \left(\frac{1}{\sqrt{1 - \left(\frac{v_0}{c} \right)^2}} - \frac{s F_i}{m c^2} \right)^{-2} \right]^{-\frac{1}{2}} \quad (4)$$

where the values of v_0 is the initial velocity of the projectile and v_p is the current velocity of the projectile inside the lightning, c is the speed of a light; F_i is the probability of creating a lightning bolt or rather, it would be the ionization capacity to generate the said lightning. m is the mass that the projectile will have; and s is the distance that a single channel can travel, this last value comes by default as the number 10, which is taken as time.

The equation shows that it is a function that describes how the lightning will be like the position where it is and how big it will be. Lightning being an explosion of energy, when the mass of the projectile is small it tends for the trajectory of the projectile to travel greater distances and according to the algorithm it is understood that it has more possibilities to explore the search space.

Other property of a stepped leader is forking that are two symmetrical branches are created because the nuclei collision of the projectile is realized by using the opposite number as in the next equation.

$$\overline{p_i} = a + b - p_i \quad (5)$$

where $\overline{p_i}$ is the opposite and p_i is the original, in a one-dimensional system; a and b are the limited dimensions that the search space will have. This adjustment helps improve

some the worst solutions within the population. The algorithm uses the bifurcation or forking to generate a symmetrical branches to generate new individuals, which is why it eliminates the worst so as not to modify the size of the initial population [16–18].

5 Parallel MFO, LSA and PSO Algorithms with Fuzzy Systems and Migration

Our proposed algorithm is the combination of three optimization algorithms, where all three works at the same time and have the ability to share information between them to obtain better solutions. As shown in the following flow chart.

In the literature there are different authors who are dedicated to making variants of algorithms using parallelism or migration, as some are the following mentioned citations [10–21] (see Fig. 4).

In our proposal we present three optimization algorithms in parallel so that they work together and that allow us to share individuals from each of the populations to have more variety of individuals, whether bad or good. And that helps us not to get stuck in a single solution, but each time that migration is applied we can have a new best solution. When the migration is applied, a comparison is made between the three populations of each algorithm to identify which is the best of the closest to the best solution and that the winner is the one that shares a portion of the population to the other two algorithms that they are losing in that comparison. At the end of each iteration, the dynamic parameter adjustment is used for each of the algorithms since each algorithm has different variables. Finally, since the iterations are finished, the best solution of each of the algorithms is actually printed, but to avoid confusion we decided that it is better to only show the best solution among the three obtained, since those solutions were obtained with the help of the three algorithms and not separately.

6 Fuzzy Logic System

Fuzzy logic can be said to be an extension to what is traditional logic, where in traditional logic it would be a hot or cold, in fuzzy logic it could be a very cold, cold, hot and very hot.

Fuzzy logic is closer to the thinking used by a common human.

Figure 5, shows an example of how the fuzzy systems would be used, just change the name to Up and Down, and of course the configuration of the rules as shown in Tables 4 and 5.

The fuzzy logic systems have one input and one output, the membership functions are triangular, the value are between 0 and 1 and the rules are those shown in the

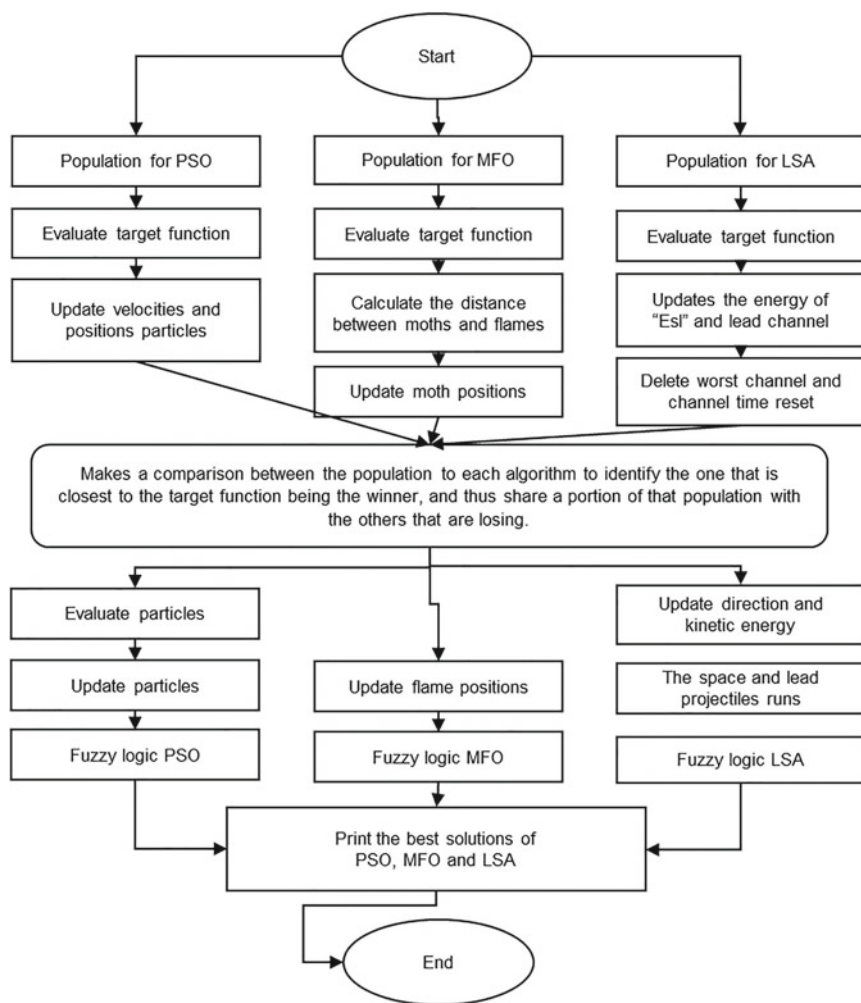


Fig. 4 Flowchart of MFO, LSA and PSO algorithm ready for migration with fuzzy logic

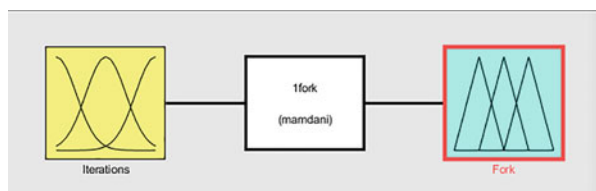


Fig. 5 Example of the fuzzy system of an input and an output used

Table 4 Rules for the fuzzy called Up

	Input		Output
If	Low	Then	Low
If	Low mid	Then	Low mid
If	Mid	Then	Mid
If	Mid high	Then	Mid high
If	High	Then	High

Table 5 Rules for the fuzzy called Down

	Input		Output
If	Low	Then	High
If	Low mid	Then	Mid high
If	Mid	Then	Mid
If	Mid high	Then	Mid low
If	High	Then	Low

Tables 4 and 5, where the first column refers to the inputs and the other two would be the output, column 1 and the output column 2 and for the other case it would be the same input 1 and output is the column 3. That in the same way this could be done for each of the variables of each algorithm but the rules would be the same, or it could have been done together, that is, one input and two outputs [22–24].

7 Experiments

In the development of our algorithm in parallel with the MFO, LSA and PSO algorithms that allows us to use migration between them. This allows us to save handling time, it could be said that it is the time it takes us run each code separately. This as a product gives us the result of each of the optimization algorithms, which the results presented are the average of 30 executions but we only show the best among the three to avoid confusion about the results since the migration is applied to each one of them (see Table 6).

The parameters: population, dimension, iteration and those related to migration for any algorithm used. The Fork and Channel Time parameters are for the LSA, the spiral parameter is for the MFO algorithm, and the cognitive and social parameters are for the PSO algorithm. When we say Up or Down are the dynamic adjustment for the value of this variable.

Table 6 Parameters used in experiments with 3 parallel algorithms

Experiment	1	2	3	4	5	6	7	8	9
Population	100	100	100	100	100	100	100	100	100
Dimension	100	50	20	150	80	80	50	120	80
Iteration	1000	500	200	1500	500	1000	500	1200	500
Fork	0.001	0.001	0.001	0.001	0.05	0.05	Up	Up	Down
Channel time	5	5	5	5	5	5	5	5	5
Spiral (slope)	0.01	0.01	0.01	0.01	0.8	0.5	Up	0.5	Up
Cognitive par	2	2	0.365	2	0.365	0.365	Down	0.365	0.365
Social par	4	4	1.605	4	1.605	1.605	Down	1.605	1.605
% Migration	–	–	–	–	40%	40%	20%	20%	40%
Apply migration	–	–	–	–	50%	50%	40%	–	–

7.1 Experimental Results with PSO

The experiments were carried out with the following computer that contains a Intel i5-9400f processor, has 16 GB of RAM memory, has a high-speed solid hard disk and has a Nvidia GTX 970 with 4 GB of VRAM, MATLAB r2017b and Windows 10 pro are used as software.

The average results of thirty runs of each algorithm are shown below, although of the three algorithms it is only the best value to avoid confusion. The results are separated by the different benchmark functions used.

In Table 7 shows the first three experiments that we did in these cases, neither

Table 7 Average results obtained with the proposed algorithm; parameters shown by experiment in Table 6

Best	Exp. 1	Exp. 2	Exp. 3
F1	1.510E−04	1.745E−09	1.606E−14
F2	9.903E−01	4.142E−02	1.447E−05
F3	8.524E+03	4.614E+02	7.391E+00
F4	9.489E−01	9.002E−01	0.000E+00
F5	2.529E+02	1.099E+02	1.304E+00
F6	6.047E−04	2.424E−09	2.975E−14
F7	2.079E−01	5.252E−02	1.059E−02
F8	−2.568E+04	−1.390E+04	−6.376E+03
F9	7.006E+01	3.197E+01	1.693E+01
F10	2.586E+00	2.355E+00	1.024E+00
F11	8.844E−03	1.018E−02	2.485E−02
F12	2.550E−01	1.974E−01	2.024E−01
F13	2.333E−02	1.000E−02	1.350E−32

Table 8 Average results obtained with the proposed algorithm; parameters shown by experiment in Table 6

Best	Exp. 4	Exp. 5	Exp. 6
F1	8.821E−02	4.111E−05	1.164E−09
F2	4.241E+00	4.097E−01	2.824E−03
F3	3.845E+04	7.174E+03	8.148E+02
F4	9.656E−01	4.587E−01	0.000E+00
F5	3.825E+02	0.000E+00	0.000E+00
F6	2.054E−02	5.831E−05	7.479E−09
F7	8.024E+00	2.235E−01	5.235E−02
F8	−3.638E+04	−2.490E+04	−1.431E+04
F9	1.058E+02	9.063E+01	4.496E+01
F10	2.639E+00	2.244E+00	1.853E+00
F11	4.281E−03	3.038E−03	6.979E−03
F12	1.556E−01	1.596E−01	2.288E−01
F13	2.000E−02	1.350E−32	1.350E−32

migration nor dynamic parameter adjustment was used. Itself, we only change the number of iterations and the dimensions.

It can be seen that the column of experiment 3 is the one with the least dimensions, although in some cases, such as function 11 and 12, it could be said that it loses against experiment 2.

In Table 8, the results for experiments 4 to 6 are shown, in this case, the experiment 4 has 150, experiment 5 has 100 and finally the experiment 6 has 50. This leads to the number of iterations being different since they are the number of iterations $\times$ 10.

And the experiment 6 already integrates what would be a migration of 20% of the population when applied and uses dynamic parameter adjustment for the PSO variables, the fuzzy one is used where it decreases and for the spiral and fork variable the fuzzy one is used where the values rise up to 1. The results are very similar to the previous table since it shows the same pattern in the F11 and F12 (see Table 9).

In this case, 50, 120 and 80 dimensions are used, respectively for each of the experiments. In experiment 7 a 20% migration is applied, in experiment 8 the same migration is applied but it does so between iteration 100 and what would be the total iterations minus 100. Finally, in experiment 9 a 40% migration is used and with the same change in the iterations and separately the parameters are adjusted.

Perhaps the results are very varied, but they help us to know what possibilities we have when migrating with the three algorithms together.

On the other hand, we find the tables that contain the average times that each of the experiments did, separated by functions, since in some cases, depending on the benchmark function we use it takes longer or perhaps less. If you want to know an estimated time per function, it would be to multiply the average value by 30, since the same experiment is executed 30 times to see if the results vary.

As shown in the following three tables, the execution results do not look that great although the fact is that the displayed value is the average of 30 executions, that is,

Table 9 Average results obtained with the proposed algorithm; parameters shown by experiment in Table 6

Best	Exp. 7	Exp. 8	Exp. 9
F1	1.350E−32	6.053E−03	3.387E−03
F2	3.241E−03	9.099E−01	4.966E−01
F3	8.194E+02	1.127E+04	3.813E+03
F4	0.000E+00	5.814E−01	3.584E−01
F5	7.820E−01	3.354E+01	1.517E+01
F6	7.486E−09	2.366E−02	1.014E−03
F7	5.168E−02	3.048E−01	2.268E−01
F8	−1.452E+04	−2.960E+04	−2.017E+04
F9	4.520E+01	1.087E+02	7.256E+01
F10	1.931E+00	2.222E+00	2.221E+00
F11	1.114E−02	3.535E−03	6.627E−03
F12	1.860E−01	1.748E−01	2.651E−01
F13	1.350E−32	1.350E−32	1.350E−32

to get an approximate time of function 1 in the experiment 1 it would be necessary to multiply the value of 0.0004582 by 30 times, which would be 0.013746 s, it may be a short time but there are 13 different functions that in the same way not all functions give the same average execution time so that maybe it would increase a little. This is somewhat confusing.

We can see in Table 10, they are the average of the execution time of each experiment or, as you look at the table, it would be each of the values that they have there. As can be seen in the results, the experiment 3 would be the one with the lowest average time, but according to the parameters used in it, it contains fewer

Table 10 Average time results obtained with the proposed algorithm; parameters shown by experiment in Table 6

Time	Exp. 1	Exp. 2	Exp. 3
F1	4.582E−04	3.297E−04	2.480E−04
F2	5.088E−04	4.074E−04	3.433E−04
F3	3.393E−03	3.454E−03	1.478E−03
F4	5.635E−04	4.377E−04	3.493E−04
F5	7.070E−04	8.022E−04	5.246E−04
F6	6.579E−04	5.484E−04	4.738E−04
F7	8.171E−04	7.438E−04	5.571E−04
F8	9.573E−04	8.933E−04	5.975E−04
F9	8.842E−04	7.336E−04	5.749E−04
F10	9.385E−04	8.373E−04	6.761E−04
F11	1.018E−03	1.123E−03	7.954E−04
F12	3.839E−03	3.201E−03	1.565E−03
F13	3.854E−03	3.238E−03	1.593E−03

iterations and dimensions, here the different parameters do not influence unless we had changed the channel time, which we did not do.

If we compare the time values found between each of the tables that we show, they are all very similar.

In Table 11, should be higher or with more variation because the iterations and the dimensions are higher.

In Table 12 shows how low execution times continue to vary only slightly between the different benchmark functions.

Table 11 Average time results obtained with the proposed algorithm; parameters shown by experiment in Table 6

Time	Exp. 4	Exp. 5	Exp. 6
F1	5.845E−04	4.953E−04	3.210E−04
F2	6.593E−04	5.894E−04	3.832E−04
F3	5.486E−03	8.202E−03	1.663E−03
F4	6.786E−04	6.175E−04	4.396E−04
F5	8.551E−04	7.593E−04	5.634E−04
F6	7.625E−04	6.743E−04	5.243E−04
F7	1.340E−03	8.510E−04	6.568E−04
F8	1.169E−03	9.873E−04	7.456E−04
F9	1.077E−03	9.174E−04	7.145E−04
F10	1.262E−03	9.938E−04	7.583E−04
F11	1.284E−03	1.106E−03	8.606E−04
F12	5.677E−03	3.918E−03	2.409E−03
F13	5.473E−03	3.912E−03	2.449E−03

Table 12 Average time results obtained with the proposed algorithm; parameters shown by experiment in Table 6

Time	Exp. 7	Exp. 8	Exp. 9
F1	3.324E−04	4.963E−04	3.978E−04
F2	3.847E−04	5.905E−04	4.570E−04
F3	1.756E−03	9.141E−03	2.808E−03
F4	4.546E−04	6.184E−04	5.009E−04
F5	5.730E−04	8.049E−04	6.465E−04
F6	5.894E−04	7.296E−04	5.869E−04
F7	9.354E−04	9.112E−04	8.123E−04
F8	8.991E−04	1.070E−03	9.950E−04
F9	7.412E−04	1.008E−03	9.170E−04
F10	8.064E−04	1.049E−03	8.605E−04
F11	8.676E−04	1.183E−03	9.801E−04
F12	2.431E−03	4.497E−03	3.488E−03
F13	2.453E−03	4.512E−03	3.505E−03

8 Conclusions

After analyzing the results obtained, we observed that in some of the benchmark functions they give us very good results regardless of whether we are working with high dimensions, and we saw that it is advisable to use migration and dynamic parameter adjustment to obtain better results compared to the experiments where we did not use them. In terms of times, the difference between one experiment and another is not so much, although we do observe that the LSA algorithm by itself takes longer than the other algorithms. Although the execution times that we show are the average of 30 runs and not the total time it takes per experiment in each benchmark function.

As future work, we will stop doing many more experiments with more variation of parameters but to be able to compare ourselves with the other authors, that although there is no or we did not find another similar algorithm, we only saw them separately. So, we should look for a good rival to be able to make the comparisons in case they are not these algorithms we can look for another that are 3 of the same categories or similar [25].

References

1. Valdez, F., Chaparro, I.: Ant colony optimization for solving the TSP symmetric with parallel processing. In: Joint IFSA World Congress and NAFIPS Annual Meeting, IFSA/NAFIPS 2013, Edmonton, Alberta, Canada, June 24–28, 2013, pp. 1192–1196 (2013). <https://doi.org/10.1109/IFSA-NAFIPS.2013.6608570>
2. Cárdenas, M., Melin, P., Reyes, L.C.: Parallel genetic algorithms for architecture optimization of neural networks for pattern recognition. In: Soft Computing for Recognition Based on Biometrics, pp. 303–315 (2010). https://doi.org/10.1007/978-3-642-15111-8_19
3. Gaxiola, F., Melin, P., Valdez, F., Castro, J.R.: Optimization with genetic algorithm and particle swarm optimization of type-2 fuzzy integrator for ensemble neural network in time series. In: 2016 IEEE International Conference on Fuzzy Systems, FUZZ-IEEE 2016, Vancouver, BC, Canada, July 24–29, 2016, pp. 2067–2074 (2016). <https://doi.org/10.1109/FUZZ-IEEE.2016.7737946>
4. Mirjalili, S., Lewis, A.: The whale optimization algorithm. *Adv. Eng. Softw.* **95**, 51–67 (2016). <https://doi.org/10.1016/j.advengsoft.2016.01.008>
5. Guerrero-Luis, M., Valdez, F., Castillo, O.: A review on the cuckoo search algorithm. In: Fuzzy Logic Hybrid Extensions of Neural and Optimization Algorithms: Theory and Applications, pp. 113–124 (2021). https://doi.org/10.1007/978-3-030-68776-2_7
6. Abualigah, L.M., El-Aziz, M.A., Hussien, A.G., Alsabibi, B., Jalali, S.M.J., Gandomi, A.H.: Lightning search algorithm: a comprehensive survey. *Appl. Intell.* **51**(4), 2353–2376 (2021). <https://doi.org/10.1007/s10489-020-01947-2>

7. Méndez, E., Castillo, O., Soria, J., Melin, P., Sadollah, A.: Water cycle algorithm with fuzzy logic for dynamic adaptation of parameters. In: *Advances in Computational Intelligence-15th Mexican International Conference on Artificial Intelligence, MICAI 2016, Cancún, Mexico, October 23–28, 2016, Proceedings, Part I*, pp. 250–260 (2016). https://doi.org/10.1007/978-3-319-62434-1_21
8. Sirjani, R., Okonkwo, E.C.: A new wind power model using the lightning search algorithm. In: *2016 HONET-ICT, Nicosia, Cyprus, October 2016*, pp. 93–97 (2016). <https://doi.org/10.1109/HONET.2016.7753427>
9. Fierro, R., Castillo, O.: Design of fuzzy control systems with different PSO variants. In: *Recent Advances on Hybrid Intelligent Systems*, pp. 81–88 (2013). https://doi.org/10.1007/978-3-642-33021-6_6
10. Kawano, Y., Valdez, F., Castillo, O.: Evaluation of parallel exploration and exploitation capabilities in two PSO variants with intra communication. In: *Intuitionistic and Type-2 Fuzzy Logic Enhancements in Neural and Optimization Algorithms: Theory and Applications*, pp. 169–184 (2020). https://doi.org/10.1007/978-3-030-35445-9_15
11. Vázquez, J.C., Valdez, F.: Fuzzy logic for dynamic adaptation in PSO with multiple topologies. In: *Joint IFSA World Congress and NAFIPS Annual Meeting, IFSA/NAFIPS 2013, Edmonton, Alberta, Canada, June 24–28, 2013*, pp. 1197–1202 (2013). <https://doi.org/10.1109/IFSA-NAFIPS.2013.6608571>
12. Mirjalili, S.: Moth-flame optimization algorithm: a novel nature-inspired heuristic paradigm. *Knowl.-Based Syst.* **89**, 228–249 (2015). <https://doi.org/10.1016/j.knosys.2015.07.006>
13. Jangir, N., Pandya, M.H., Trivedi, I.N., Bhesdadiya, R.H., Jangir, P., Kumar, A.: Moth-flame optimization algorithm for solving real challenging constrained engineering optimization problems. In: *2016 IEEE Students' Conference on Electrical, Electronics and Computer Science (SCEECS)*, March 2016, pp. 1–5 (2016). <https://doi.org/10.1109/SCEECS.2016.7509293>
14. Li, C., Niu, Z., Song, Z., Li, B., Fan, J., Liu, P.X.: A double evolutionary learning moth-flame optimization for real-parameter global optimization problems. *IEEE Access* **6**, 76700–76727 (2018). <https://doi.org/10.1109/ACCESS.2018.2884130>
15. Tumar, I., Hassouneh, Y., Turabieh, H., Thaher, T.: Enhanced binary moth flame optimization as a feature selection algorithm to predict software fault prediction. *IEEE Access* **8**, 8041–8055 (2020). <https://doi.org/10.1109/ACCESS.2020.2964321>
16. Shareef, H., Islam, M.M., Ibrahim, A.A., Mutlag, A.H.: A nature inspired heuristic optimization algorithm based on lightning. In: *2015 3rd International Conference on Artificial Intelligence, Modelling and Simulation (AIMS)*, December 2015, pp. 9–14 (2015). <https://doi.org/10.1109/AIMS.2015.12>
17. Liu, W., Huang, Y., Zong, X., Shi, H., Ye, Z., Wei, S.: Application of lightning search algorithm in localization of wireless sensor networks. In: *2018 IEEE 4th International Symposium on Wireless Systems within the International Conferences on Intelligent Data Acquisition and Advanced Computing Systems (IDAACS-SWS)*, September 2018, pp. 57–61 (2018). <https://doi.org/10.1109/IDAACS-SWS.2018.8525518>
18. Asvany, T., Amudhavel, J., Sujatha, P.: Lightning Search Algorithm for Solving Coverage Problem in Wireless Sensor Network, p. 15 (2017)
19. Valdez, F., Melin, P., Castillo, O.: Fuzzy control for dynamical parameter adaptation in a parallel evolutionary method combining particle swarm optimization and genetic algorithms. In: *Soft Computing for Intelligent Control and Mobile Robotics*, pp. 161–178 (2011). https://doi.org/10.1007/978-3-642-15534-5_10
20. Valdez, F., Melin, P., Castillo, O.: Optimization of modular neural networks for pattern recognition with parallel genetic algorithms. In: *Advances in Soft Computing-18th Mexican International Conference on Artificial Intelligence, MICAI 2019, Xalapa, Mexico, October 27–November 2, 2019, Proceedings*, pp. 223–235 (2019). https://doi.org/10.1007/978-3-030-33749-0_19
21. Ma, G., Zhou, W., Chang, X.: A novel particle swarm optimization algorithm based on particle migration. *Appl. Math. Comput.* **218**(11), 6620–6626 (2012). <https://doi.org/10.1016/j.amc.2011.12.032>

22. Amador-Angulo, L., Castillo, O.: A new fuzzy bee colony optimization with dynamic adaptation of parameters using interval type-2 fuzzy logic for tuning fuzzy controllers. *Soft Comput.* **22**(2), 571–594 (2018). <https://doi.org/10.1007/s00500-016-2354-0>
23. Alalaween, W.H., Alalawin, A.H., Mahfouf, M., Abdallah, O.H.: A dynamic type-1 fuzzy logic system for the development of a new warehouse assessment scheme. *IEEE Access* **9**, 43611–43619 (2021). <https://doi.org/10.1109/ACCESS.2021.3060293>
24. Bernal, E., Castillo, O., Soria, J., Valdez, F., Melin, P.: A variant to the dynamic adaptation of parameters in galactic swarm optimization using a fuzzy logic augmentation. In: 2018 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE), Rio de Janeiro, July 2018, pp. 1–7 (2018). <https://doi.org/10.1109/FUZZ-IEEE.2018.8491623>
25. Olivas, F., Valdez, F., Melin, P., Sombra, A., Castillo, O.: Interval type-2 fuzzy logic for dynamic parameter adaptation in a modified gravitational search algorithm. *Inf. Sci.* **476**, 159–175 (2019)

Fuzzy Dynamic Parameter Adaptation in the Mayfly Algorithm: Preliminary Tests for a Parameter Variation Study



Enrique Lizarraga, Fevrier Valdez, Oscar Castillo, and Patricia Melin

Abstract Inspired by the flight behavior and the mating process of mayflies, the mayfly algorithm combines the main advantages of swarm intelligence and evolutionary algorithms, resulting in an better performance of the algorithm, so we will analyze the parameters with benchmark functions by to find an adequate range in which some fuzzy rules can be applied, in order to improve the speed of convergence of the algorithm and later be applied to neural network optimization problems due to the algorithms for finding solutions such as particle swarm optimization (PSO), differential evolution (DE), genetic algorithm (GA) and firefly algorithm (FA) have proven to be efficient in terms of convergence for these types of problems, we obtained as a result that the parameter a_2 affects the performance of the mayfly algorithm most, that is, if we increase it, the efficiency of the algorithm decreases, instead we keep it at 1 we will obtain a better result. By varying the parameter a_1 , it was observed that the parameter reaches its best Other values if we leave it between 3 and 5, finally the parameter β did not significant variations in the results.

Keywords Mayfly algorithm · Fuzzy logic · Dynamic parameter adaptation

1 Introduction

Ephemeral is a species of insects whose young would live in water for several years and when they mature they will peel off and fly through the air. However, these mature insects only have the purpose of reproducing since their life span is short.

Inspired by the flight behavior and the mating process of mayflies, the proposed algorithm combines the main advantages of swarm intelligence and evolutionary algorithms.

In fact, it offers a powerful hybrid algorithmic structure, based on ephemeral behavior, for researchers trying to improve the performance of the PSO algorithm

E. Lizarraga · F. Valdez (✉) · O. Castillo · P. Melin
Tijuana Institute of Technology, Tomas Aquino, Tijuana, Mexico
e-mail: fevrier@tectijuana.mx

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2022
O. Castillo and P. Melin (eds.), *New Perspectives on Hybrid Intelligent System Design based on Fuzzy Logic, Neural Networks and Metaheuristics*, Studies in Computational Intelligence 1050, https://doi.org/10.1007/978-3-031-08266-5_15

223

using techniques such as crossover and local search, as PSO has been shown to need some modifications.

Fuzzy logic or multivalued logic is based on the theory of fuzzy sets proposed by Zadeh, which helps us to model knowledge through the use of fuzzy if-then rules. Fuzzy provides a systematic computation to deal with linguistic information, and that improves numerical computation through the use of linguistic labels stipulated by membership functions.

The present one proposes a fuzzy adaptation to the working parameters of the Mayfly algorithm with which we can improve its effectiveness to optimize a neural network.

The structure of this work is as follows in Sect. 2 we have the theoretical framework where the background of the mayfly algorithm will be mentioned, then we have Sect. 3, in this Section the flow diagram of the proposed method will be shown, in Sect. 4 we have parameter impact study where the experiments carried out with benchmark functions will be placed, the Sect. 5 follows, where we have a summary of the experiments and finally the conclusions Section where we can know which parameters impact more performance of the algorithm.

2 Theoretical Framework

Mayfly is a species of insects whose young would live in the water for several years and when they mature, they will peel off and fly through the air. However, these mature insects only have the purpose of reproducing since their life span is short.

By observing the short-lived males and females running in search of mates, the MA algorithm was proposed. Developed from the PSO algorithm, mayflies also update their positions. However, they converge faster to the optimum therefore the Mayfly algorithm is an improvement on the PSO algorithm.

$$X_i^{t+1} = X_i^t + V_i^{t+1} \quad (1)$$

$$v_{ij}^{t+1} = v_{ij}^t + dr \quad (2)$$

$$v_{ij}^{t+1} = v_{ij}^t + a_1 e^{-\beta r_p^2} (pbest_{ij} - x_{ij}^t) + a_2 e^{-\beta r_s^2} (gbest_i - x_{ij}^t) \quad (3)$$

Parameters a_1 and a_2 will be adjusted with fuzzy logic, which represent the constants of positive attraction that are used to scale the contribution of the cognitive and social component, respectively.

Based on the mayfly literature by Konstantinos Zervoudakis, the recommended value for the parameter a_1 is in the range of 1–2 and for the parameter a_2 it is in the range of 1.5–2.

The investigations showed that not only would the best candidates or best historical trajectories work well in guiding individuals in swarms to find the best solution,

the worst and worst historical trajectories would also work well in doing so. Such situations could be treated directly as pairs of oppositions and satisfy the ancient Chinese Yin-Yang philosophy for which the opposition-based learning (OBL) rule was proposed.

The MO algorithm will be divided into two types, the short-lived male for the i th individual in the current iteration t , and the short-lived female. Both types will update their positions with their velocities in the current iteration.

$$p_i(t+1) = p_i(t) + v_i(t) \quad (4)$$

Taking the Yin-Yang philosophy into account, we might find that if the current position was very far from the best, then the opposite position would be close to the best in a given domain of definition. Therefore, we can calculate the opposite of the current position [1]

$$opi(t+1) = a + b - p_i(t) \quad (5)$$

The improved mayfly optimization algorithm

This algorithm includes the combination of features of PSO, GA, and firefly algorithm (FA). Adopting and improving PSO search, MA is a hybrid optimization algorithm with high efficiency, which models the mating pattern of mayflies, this optimization algorithm assumes that a mayfly is an adult after it hatches and surfaces. The fittest survives regardless of life, the position of each ephemeral denotes the possibility of obtaining a desirable solution in that solution space, a group of ephemeral males and another group of ephemeral females are randomly generated [2].

$$v_{id}^{t+1} = v_{id}^t + c_1 e^{-\xi D_p^2} gbest_d - p_{id}^t + c_2 e^{-\xi D_s^2} gbest_d - p_{id}^t \quad (5)$$

Parameters c_1 and c_2 are the constants of positive attraction that represent the participation of the cognitive and social elements, respectively.

Algorithm NMO

It is a negative ephemeral optimization algorithm. In which the males of the mayflies would update their speeds according to the worst candidates along with their worst trajectories. Unlike the normal positive interpretation, in the NMO algorithm, the short-lived males would flee from their worst trajectories and the worst global candidates. The simulation experiments showed that the NMO and MO algorithms would work well optimizing the unimodal or multimodal reference functions, even for the non-symmetric one. However, for unimodal reference functions, MO would perform better than NMO [3].

$$v_i^{t+1} = \begin{cases} gv_i^t + a_1 e^{-\beta r_{mf}^2} x_i^t - y_i^t & f(x_i^t - y_i^t) > f(x_i^t) \\ gv_i^t + flr_1 & f(y_i^t) \leq f(x_i^t) \end{cases} \quad (7)$$

$$\|x_i - y_i\| = \sqrt{\sum_{k=1}^n (x_{ik} - y_{ik})^2} \quad (8)$$

$$v_i^{t+1} = \begin{cases} g v_i^t + a_1 e^{-\beta r_p^2} [x_{hi} - x_i^t] + a_3 e^{-\beta r_g^2} [x_g - x_i^t] & f[x_i^t] > f[x_{hi}^t] \\ g v_i^t + d r_2 & f[x_i^t] \leq f[x_{hi}^t] \end{cases} \quad (9)$$

The parameters a_2 and a_3 are constant, r_p y r_g represent the Cartesian distance between the current individual and the best historical trajectory, d represents the proportion of the wedding dance around the current position, r_2 it is another random number in uniform distribution with interval of -1 and 1 and x_{hi} represents the best historical trajectory.

Balanced mayfly algorithm (BMA)

The algorithm has been designed by the explained conception and the modeling of three algorithms, Particle Swarm Optimization (PSO) algorithm, Firefly Optimization Algorithm (FOA), and Genetic Algorithm (GA) that made it so efficient due to the use of all three popular methods advantages. With hybridizing of the algorithms, the exploitation of the PSO algorithm is combined with exploration features of the other algorithms for achieving higher efficiency in modeling the mayflies, especially the model of breeding. This modeling helps the fittest mayflies to survive after egg hatching without considering their lifespan.

First we look for the ephemeral male with the lowest cost value in the population, then we select the number of seeds, followed by this the ephemeral females attract the elite males to generate a new offspring using this as the growth radius [4].

$$v_{ij}^{t+1} = v_{ij}^t + b_1 e^{-\beta r_p^2 (x_{ij}^{ij} - x_{ij}^t)} + b_2 e^{(-\beta r_g^2) (x_g^j - x_{ij}^t)} \quad (10)$$

The parameters x_{ij}^t and v_{ij}^t represent the location and velocity of the i th mayfly in dimension j , β describes a defined coefficient to make the ephemeral visible to others r_p and r_g represent the restricted Cartesian distance in the range, x_{ij}^t and x_g^j describe the mayfly with the optimal position for the mayfly b_1 and b_2 describe the constants of positive attraction and social factors, respectively.

3 Proposed Method

In this Section, we can see the flow diagram of the proposed method to improve the convergence of the mayfly algorithm (see Fig. 1).

In the original method first we start/initialize the female population/find the best global/evaluate solutions/we check the stop criteria, if its satisfied we find the solution, if not then update velocities and solutions of males and females, evaluate solutions, Rank the mayflies/Mate the mayflies/evaluate offspring/separate offspring to

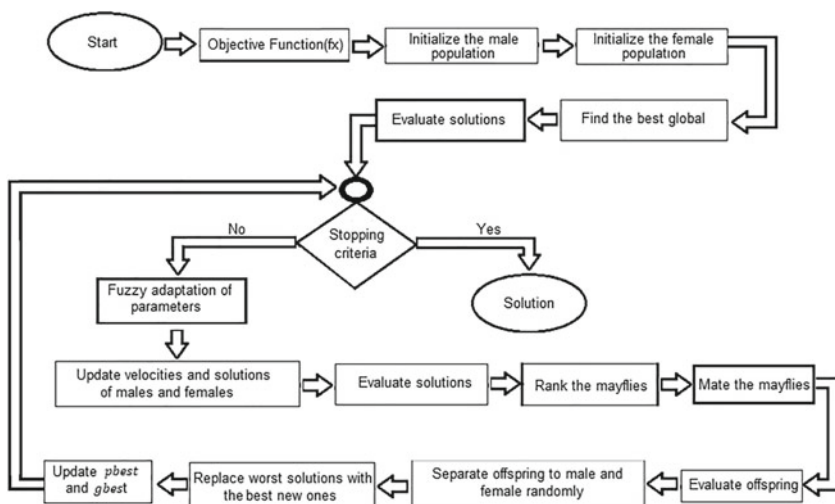


Fig. 1 Represents the diagram of the proposed method

mate and female randomly/replace worst solutions with the best new ones, update $pbest$ and $gbest$ and check the stop criteria again.

In the proposed method after checking the stopping criteria, we will add fuzzy adaptation of the parameters that affect the algorithm performance the most.

4 Parameter Impact Study

In this Section we evaluate the parameters β , $a1$ and $a2$ in order to find which of these most affects the performance of the mayfly algorithm in order to apply fuzzy logic to that parameter.

Convergence graph (Powell)

See Fig. 2.

Convergence graph (Wood)

See Fig. 3 and Table 1.

In the following experiments, 2000 iterations were used, initial population = 20, inertia weight = 0.8, nuptial dance = 2 and random flight = 1 (see Tables 2, 3, 4, 5, 6, 7, 8, 9 and 10).

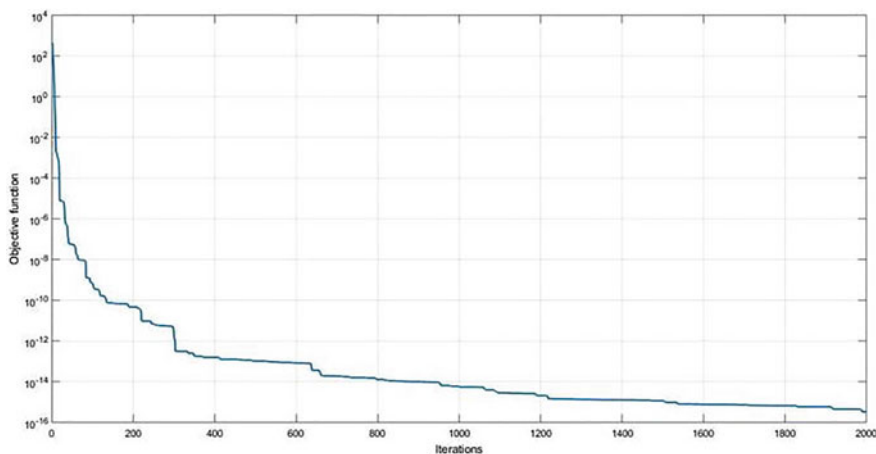


Fig. 2 Represents the convergence of the Powell function in which the parameters used were $a_1 = 10$, $a_2 = 1.5$ and $\beta = 1$

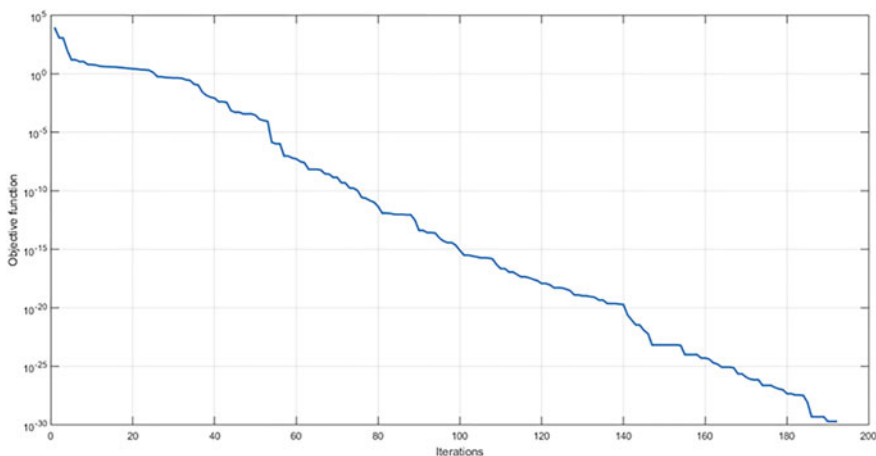


Fig. 3 Represents the convergence of the Wood function in which the parameters used were $a_1 = 5$, $a_2 = 1.5$ and $\beta = 1$

5 Results

In this Section we can determine which parameter will be chosen to apply fuzzy logic and in what range of values (see Fig. 4).

In this figure we can see how the a_1 and a_2 parameters are important when determining the efficiency of the algorithm and we can also see that varying the beta parameter the performance is affected very little (see Fig. 5).

Table 1 Representation of benchmark functions

Abbreviation	Name function
F1	Sphere
F2	Rastrigin
F3	Ackley
F4	Dejong1
F5	Griewank
F6	Schwefel
F7	Styblinski tank
F8	Wood
F9	Hyper-ellipsoid
F10	Powell

Table 2 Performance of benchmark functions for 10 dimensions for beta parameter variation

Function	Beta parameter/10 dimensions			
		1	3	10
F1	$\bar{X}$	1.231×10^{-238}	1.042×10^{-244}	2.146×10^{-249}
	s	0	0	0
F2	$\bar{X}$	3.200×10^{-2}	0	9.950×10^{-2}
	s	1.817×10^{-1}	0	3.036×10^{-1}
F3	$\bar{X}$	2.980×10^0	2.980×10^0	2.980×10^0
	s	1.817×10^{-1}	0	3.036×10^{-1}
F4	$\bar{X}$	2.087×10^{-242}	9.334×10^{-253}	2.127×10^{-245}
	S	1.355×10^{-15}	1.355×10^{-15}	1.355×10^{-15}
F5	$\bar{X}$	4.400×10^{-3}	2.100×10^{-3}	1.700×10^{-3}
	s	7.400×10^{-3}	4.700×10^{-3}	4.200×10^{-3}
F6	$\bar{X}$	4.150×10^3	-3.864×10^2	4.150×10^3
	s	6.826×10^{-1}	6.920×10^0	2.775×10^{-12}
F7	$\bar{X}$	-3.817×10^2	-3.839×10^2	-3.856×10^2
	s	9.206×10^0	8.776×10^0	9.080×10^0
F8	$\bar{X}$	7.530×10^{-2}	1.140×10^{-30}	1.422×10^{-30}
	s	4.125×10^{-1}	2.374×10^{-30}	5.363×10^{-30}
F9	$\bar{X}$	9.632×10^{-258}	4.645×10^{-244}	6.415×10^{-253}
	s	0	0	0
F10	$\bar{X}$	2.0482×10^{-98}	3.1343×10^{-103}	7.2248×10^{-104}
	s	1.1218×10^{-97}	1.7167×10^{-102}	3.9263×10^{-103}

Table 3 Performance of benchmark functions for 10 dimensions for a_1 parameter variation

Function	a_1 parameter/10 dimensions			
		1	3	10
F1	$\bar{X}$	5.5873×10^{-256}	8.2493×10^{-214}	7.9061×10^{-217}
	s	0	0	0
F2	$\bar{X}$	3.3200×10^{-2}	6.6300×10^{-2}	3.3200×10^{-2}
	s	1.8170×10^{-1}	2.5240×10^{-1}	1.8170×10^{-1}
F3	$\bar{X}$	2.9805×10^0	2.9805×10^0	2.9805×10^0
	s	1.3550×10^{-15}	1.3550×10^{-15}	1.3550×10^{-15}
F4	$\bar{X}$	6.7385×10^{-243}	2.3152×10^{-221}	1.5242×10^{-213}
	S	0	0	0
F5	$\bar{X}$	3.3000×10^{-3}	0.0001×10^{-3}	2.0000×10^{-3}
	s	5.2000×10^{-3}	4.7000×10^{-3}	3.7000×10^{-3}
F6	$\bar{X}$	4.1504×10^3	6.8000×10^{-3}	4.1503×10^3
	s	2.7751×10^{-12}	8.7000×10^{-3}	5.4460×10^{-1}
F7	$\bar{X}$	-3.8233×10^2	-3.8459×10^2	-3.860×10^2
	s	1.1342×10^1	8.9023×10^0	8.7454×10^0
F8	$\bar{X}$	3.6016×10^{-31}	3.2762×10^{-30}	5.6805×10^{-30}
	s	1.1646×10^{-30}	6.1005×10^{-30}	1.0670×10^{-29}
F9	$\bar{X}$	9.5773×10^{-241}	4.0063×10^{-223}	5.8795×10^{-212}
	s	0	0	0
F10	$\bar{X}$	2.6545×10^{-17}	1.7270×10^{-16}	3.9018×10^{-16}
	s	9.0270×10^{-17}	2.7811×10^{-16}	6.2289×10^{-15}

In this figure, we can see how the a_1 parameter is important when determining the efficiency of the algorithm because when we put it in one it increases the performance of the algorithm.

6 Conclusions

The parameter a_2 mainly affects the performance of the mayfly algorithm, it could be observed that its performance increases by values close to 1, so it will be one of the parameters to which fuzzy logic will be applied.

The parameter a_1 increases the performance of the algorithm in values between 3 and 5, so it will be the other parameter to take into account for the application of fuzzy logic.

Regarding the parameter Beta, it obtained a better result at a value of 1, however its variation affects very little the performance of the algorithm, so it is not taken into account for the application of fuzzy logic.

Table 4 Performance of benchmark functions for 10 dimensions for a_2 parameter variation

Function	a_2 parameter/10 dimensions			
		1	3	10
F1	$\bar{X}$	5.3682×10^{-139}	2.1765×10^{-13}	1.7390×10^{-1}
	s	2.9375×10^{-138}	1.1867×10^{-12}	1.439×10^{-1}
F2	$\bar{X}$	0	2.3220×10^{-1}	9.9703×10^0
	s	0	5.6550×10^{-1}	4.1971×10^0
F3	$\bar{X}$	2.9805×10^0	2.9805×10^0	3.2146×10^0
	s	1.3550×10^{-15}	1.3550×10^{-15}	1.0280×10^{-1}
F4	$\bar{X}$	1.4992×10^{-140}	3.4933×10^{-12}	1.4790×10^{-1}
	S	7.8799×10^{-140}	1.9039×10^{-11}	1.3640×10^{-1}
F5	$\bar{X}$	5.5000×10^{-3}	8.5000×10^{-2}	1.6300×10^{-2}
	s	7.4000×10^{-3}	1.7500×10^{-1}	1.4600×10^{-2}
F6	$\bar{X}$	4.1504×10^3	4.1504×10^3	4.1504×10^3
	S	2.7751×10^{-12}	2.7751×10^{-12}	2.5400×10^{-2}
F7	$\bar{X}$	-3.8555×10^2	-3.8553×10^2	-3.7200×10^2
	s	8.7980×10^0	8.0342×10^0	1.3761×10^1
F8	$\bar{X}$	2.4568×10^{-30}	4.1500×10^{-2}	1.1230×10^{-1}
	s	8.3848×10^{-30}	2.2740×10^{-1}	1.0210×10^{-1}
F9	$\bar{X}$	2.7140×10^{-136}	4.1774×10^{-14}	4.1250×10^{-1}
	s	1.4145×10^{-135}	1.9507×10^{-13}	4.4690×10^{-1}
F10	$\bar{X}$	1.0525×10^{-158}	1.1838×10^{-8}	2.0000×10^{-2}
	s	5.7455×10^{-158}	3.6954×10^{-8}	3.2000×10^{-3}

Table 5 Performance of benchmark functions for 20 dimensions for β_{eta} parameter variation

Function	Beta parameter/10 dimensions			
		1	3	10
F1	$\bar{X}$	2.2389×10^{-85}	1.4310×10^{-83}	5.7410×10^{-85}
	s	6.2447×10^{-85}	6.4675×10^{-83}	2.5457×10^{-84}
F2	$\bar{X}$	2.1891×10^0	2.5936×10^0	2.1701×10^0
	s	3.6250×10^0	2.3445×10^0	1.2814×10^0
F3	$\bar{X}$	2.9826×10^0	2.9826×10^0	2.9856×10^0
	s	1.1600×10^{-2}	1.1160×10^{-2}	1.6600×10^{-2}
F4	$\bar{X}$	1.4083×10^{-86}	1.9740×10^{-83}	5.3884×10^{-83}
	S	5.32111×10^{-86}	1.0654×10^{-82}	2.9432×10^{-82}
F5	$\bar{X}$	1.5000×10^{-3}	1.1000×10^{-3}	1.3000×10^{-3}
	s	3.0000×10^{-3}	2.8000×10^{-3}	3.0000×10^{-3}
F6	$\bar{X}$	8.3013×10^3	8.3010×10^3	8.3013×10^3
	S	1.2925×10^0	9.4850×10^{-1}	1.2925×10^0
F7	$\bar{X}$	-7.4279×10^2	-7.4047×10^2	7.3714×10^2
	s	2.1879×10^1	2.3545×10^1	2.3164×0^1
F8	$\bar{X}$	3.7865×10^{-31}	1.0871×10^{-30}	1.0398×10^{-30}
	s	7.2171×10^{-31}	1.8127×10^{-30}	1.8174×10^{-30}
F9	$\bar{X}$	1.6499×10^{-84}	1.4959×10^{-82}	5.9297×10^{-80}
	s	7.3705×10^{-84}	7.8305×10^{-82}	3.2474×10^{-79}
F10	$\bar{X}$	7.7396×10^{-108}	9.4023×10^{-81}	1.9654×10^{-104}
	s	4.2390×10^{-107}	5.1499×10^{-80}	1.0765×10^{-103}

Table 6 Performance of benchmark functions for 20 dimensions for a₁ parameter variation

Function	a ₁ parameter/10 dimensions			
		1	3	10
F1	$\bar{X}$	3.7018×10^{-85}	1.1700×10^{-18}	1.3335×10^{-76}
	s	1.7365×10^{-84}	1.1056×10^{-77}	6.1097×10^{-76}
F2	$\bar{X}$	2.1226×10^0	2.5272×10^0	8.6230×10^{-1}
	s	2.1345×10^0	2.8146×10^0	9.3240×10^{-1}
F3	$\bar{X}$	2.9805×10^0	2.9805×10^0	2.9805×10^0
	s	1.3550×10^{-15}	1.3550×10^{-15}	1.3550×10^{-15}
F4	$\bar{X}$	7.3290×10^{-82}	5.7774×10^{-79}	3.6134×10^{-79}
	S	4.0143×10^{-81}	3.1329×10^{-78}	1.3513×10^{-78}
F5	$\bar{X}$	1.1000×10^{-3}	7.0250×10^{-80}	4.93707×10^{-4}
	s	2.8000×10^{-3}	2.0452×10^{-79}	1.9000×10^{-3}
F6	$\bar{X}$	8.3013×10^3	8.3010×10^3	8.3011×10^3
	S	1.2925×10^0	9.4850×10^{-1}	$1.1407 \times 10^{0-7}$
F7	$\bar{X}$	-7.4383×10^2	7.4291×10^2	7.3192×10^2
	s	2.1831×10^1	1.9891×10^1	1.8750×101
F8	$\bar{X}$	5.6132×10^{-31}	1.5920×10^{-1}	2.4200×10^{-2}
	s	1.9415×10^{-30}	7.4650×10^{-1}	1.3250×10^{-1}
F9	$\bar{X}$	5.8224×10^{-83}	9.8517×10^{-72}	1.8958×10^{-74}
	s	3.1608×10^{-82}	5.3960×10^{-71}	1.0383×10^{-73}
F10	$\bar{X}$	3.4879×10^{-19}	3.1166×10^{-16}	1.9360×10^{-16}
	s	1.1700×10^{-18}	7.2152×10^{-16}	2.5051×10^{-16}

Table 7 Performance of benchmark functions for 20 dimensions for a_2 parameter variation

Function	a_2 parameter/10 dimensions			
		1	3	10
F1	$\bar{X}$	1.0091×10^{-30}	4.4000×10^{-3}	1.1916×10^0
	s	5.4293×10^{-30}	9.6000×10^{-3}	6.4280×10^{-1}
F2	$\bar{X}$	2.1557×10^0	1.2795×10^1	5.5739×10^1
	s	1.3598×10^0	1.0197×10^1	2.2720×10^1
F3	$\bar{X}$	2.9847×10^0	3.0682×10^1	3.5516×10^0
	s	1.6100×10^{-2}	9.1300×10^{-2}	1.7040×10^{-1}
F4	$\bar{X}$	2.7401×10^{-30}	8.7000×10^{-3}	1.3084×10^0
	S	1.4763×10^{-29}	1.7800×10^{-2}	8.1950×10^{-1}
F5	$\bar{X}$	1.7000×10^{-3}	1.6000×10^{-3}	9.4200×10^{-2}
	s	3.6000×10^{-3}	4.1000×10^{-3}	4.4400×10^{-2}
F6	$\bar{X}$	8.3011×10^3	8.3016×10^3	8.3009×10^3
	S	1.1407×10^0	1.8843×10^0	1.1520×10^{-1}
F7	$\bar{X}$	7.4192×10^2	7.3318×10^2	6.6449×10^2
	s	2.4846×10^1	2.3973×10^1	3.4779×10^1
F8	$\bar{X}$	2.53200×10^{-1}	3.1840×10^{-1}	1.7640×10^{-1}
	s	9.7680×10^{-1}	3.6610×10^{-1}	2.4900×10^{-1}
F9	$\bar{X}$	5.2077×10^{-32}	4.9800×10^{-2}	1.0782×10^1
	s	2.4898×10^{-31}	8.8820×10^{-2}	5.8278×10^0
F10	$\bar{X}$	3.5991×10^{-156}	4.0187×10^{-9}	1.5000×10^{-3}
	s	1.9711×10^{-155}	3.8543×10^{-9}	2.7000×10^{-3}

Table 8 Performance of benchmark functions for 50 dimensions for beta parameter variation

Function	Beta parameter/50 dimensions			
		1	3	10
F1	$\bar{X}$	2.6622×10^{-18}	8.6067×10^{-18}	3.1524×10^{-18}
	s	4.3049×10^{-18}	2.1176×10^{-17}	7.2702×10^{-18}
F2	$\bar{X}$	2.0367×10^1	2.0235×10^1	2.0017×10^1
	s	7.5310×10^0	5.7713×10^0	3.9721×10^0
F3	$\bar{X}$	3.1119×10^0	3.1232×10^0	3.1250×10^0
	s	4.1982×10^{-2}	3.9477×10^{-2}	3.6415×10^{-2}
F4	$\bar{X}$	1.0703×10^{-17}	2.6249×10^{-9}	5.3448×10^{-18}
	S	2.2941×10^{-17}	1.4377×10^{-8}	1.0206×10^{-17}
F5	$\bar{X}$	9.8667×10^{-4}	5.7667×10^{-4}	1.6000×10^{-3}
	s	3.2129×10^{-3}	2.2190×10^{-3}	4.1000×10^{-3}
F6	$\bar{X}$	2.0755×10^4	2.0754×10^4	2.0754×10^4
	S	2.6869×10^0	3.0072×10^0	2.7302×10^0
F7	$\bar{X}$	1.5629×10^3	1.5929×10^3	1.6262×10^3
	s	6.3748×10^2	6.4187×10^1	5.5695×10^1
F8	$\bar{X}$	1.2424×10^{-31}	1.0058×10^{-31}	7.0257×10^{-32}
	s	2.9164×10^{-31}	1.4505×10^{-31}	1.1683×10^{-31}
F9	$\bar{X}$	8.4588×10^1	9.2035×10^1	5.1836×10^1
	s	3.8959×10^1	4.8202×10^1	1.7352×10^1
F10	$\bar{X}$	4.6825×10^{-78}	3.0523×10^{-97}	1.9030×10^{-86}
	s	8.5490×10^{-79}	1.6583×10^{-96}	1.0423×10^{-85}

Table 9 Performance of benchmark functions for 50 dimensions for a_1 parameter variation

Function	a_1 parameter/50 dimensions			
		1	3	10
F1	$\bar{X}$	2.6622×10^{-18}	8.6067×10^{-18}	3.1524×10^{-18}
	s	4.3049×10^{-18}	2.1176×10^{-17}	7.2702×10^{-18}
F2	$\bar{X}$	2.0367×10^1	2.0235×10^1	2.0017×10^1
	s	7.5310×10^0	5.7713×10^0	3.9721×10^0
F3	$\bar{X}$	3.1119×10^0	3.1232×10^0	3.1250×10^0
	s	4.1982×10^{-2}	3.9477×10^{-2}	3.6415×10^{-2}
F4	$\bar{X}$	1.0703×10^{-17}	2.6249×10^{-9}	5.3448×10^{-18}
	S	2.2941×10^{-17}	1.4377×10^{-8}	1.0206×10^{-17}
F5	$\bar{X}$	9.8667×10^{-4}	5.7667×10^{-4}	1.6000×10^{-3}
	s	3.2129×10^{-3}	5.7667×10^{-4}	4.1000×10^{-3}
F6	$\bar{X}$	2.0755×10^4	2.0754×10^4	2.0754×10^4
	S	2.6869×10^0	3.0072×10^0	2.7302×10^0
F7	$\bar{X}$	1.5629×10^3	1.5929×10^3	1.6262×10^3
	s	6.3748×10^2	6.4187×10^1	5.5695×10^1
F8	$\bar{X}$	1.2424×10^{-31}	1.0058×10^{-31}	7.0257×10^{-32}
	s	2.9164×10^{-31}	1.4505×10^{-31}	1.1683×10^{-31}
F9	$\bar{X}$	8.4588×10^1	9.2035×10^1	5.1836×10^1
	s	3.8959×10^1	4.8202×10^1	1.7352×10^1
F10	$\bar{X}$	4.6825×10^{-78}	3.0523×10^{-97}	1.9030×10^{-86}
	s	8.5490×10^{-79}	1.6583×10^{-96}	1.0423×10^{-85}

Table 10 Performance of benchmark functions for 50 dimensions for a_2 parameter variation

Function	a_2 parameter/50 dimensions			
		1	3	10
F1	$\bar{X}$	7.7702×10^{-18}	6.0070×10^{-1}	1.1431×10^1
	s	1.4913×10^{-17}	4.8350×10^{-1}	3.7181×10^0
F2	$\bar{X}$	2.1481×10^1	1.0418×10^2	3.2912×10^2
	s	1.1336×10^1	6.5609×10^1	5.4157×10^1
F3	$\bar{X}$	2.1481×10^1	3.4216×10^0	4.2677×10^0
	s	1.1356×10^1	3.0420×10^1	3.3240×10^{-1}
F4	$\bar{X}$	4.9914×10^{-18}	3.2050×10^{-1}	1.0555×10^1
	S	1.3840×10^{-17}	3.5010×10^{-1}	3.4654×10^0
F5	$\bar{X}$	9.0310×10^{-4}	1.0000×10^{-2}	3.2400×10^{-1}
	s	3.5000×10^{-3}	1.1500×10^{-2}	8.2600×10^{-2}
F6	$\bar{X}$	2.0754×10^4	2.0754×10^4	2.0756×10^4
	S	3.6931×10^0	3.7302×10^0	3.1330×10^0
F7	$\bar{X}$	1.7510×10^3	1.6892×10^3	1.4881×10^3
	s	5.0532×10^1	6.1035×10^1	7.7586×10^1
F8	$\bar{X}$	4.9107×10^{-30}	4.0600×10^{-2}	1.1170×10^{-1}
	s	1.1347×10^{-29}	2.2380×10^{-2}	1.0650×10^{-1}
F9	$\bar{X}$	3.4501×10^{-15}	7.0498×10^0	2.4866×10^2
	s	3.1062×10^{-15}	8.0035×10^0	8.5808×10^1
F10	$\bar{X}$	1.8524×10^{-155}	1.1834×10^{-8}	2.4866×10^2
	s	1.0416×10^{-154}	2.7001×10^{-8}	8.5808×10^1

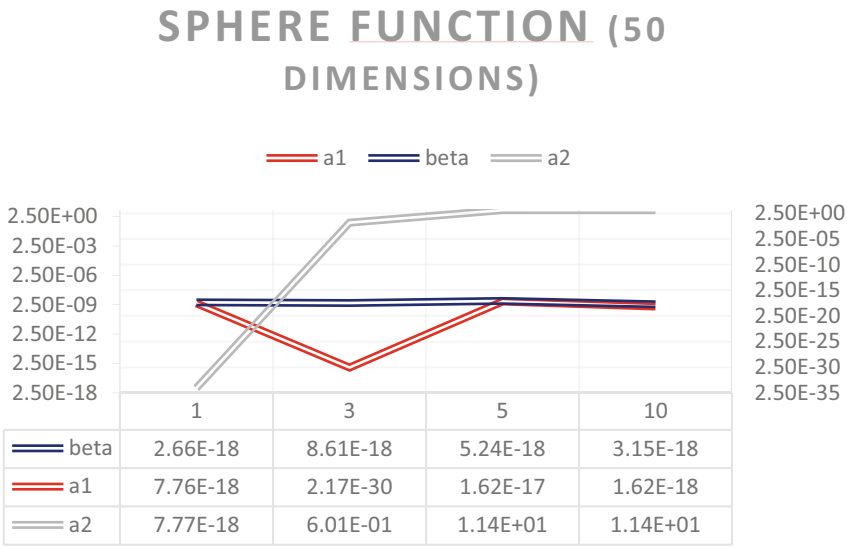


Fig. 4 Summary of performance for benchmark function of 50 dimensions for beta parameter variation (sphere function)

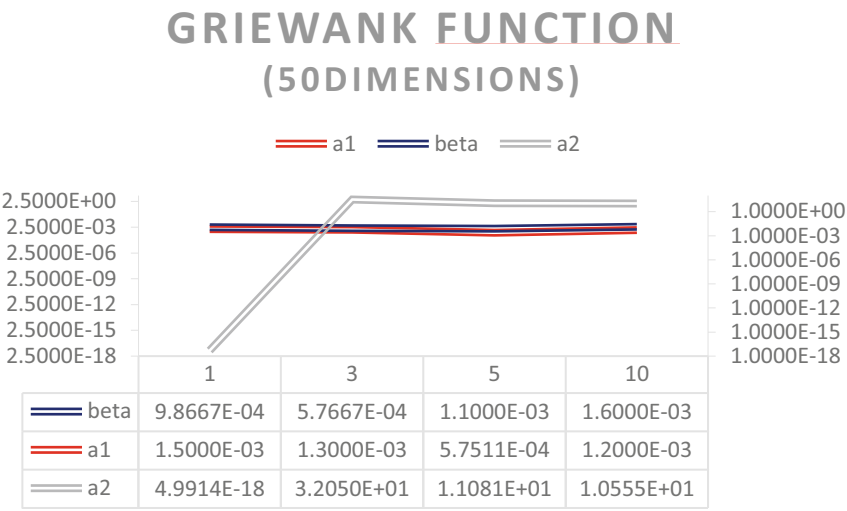


Fig. 5 Summary of performance for benchmark function of 50 dimensions for griewank parameter variation (sphere function)

Future work, we will adapt the parameters a_1 and a_2 using fuzzy rules based on the results obtained from the benchmark tests. Also, consider the parameter adaptation with type-2 fuzzy logic [5, 6].

References

1. Gao, Z.M., Zhao, J., Li, S.R., Hu, Y.R.: The improved mayfly optimization algorithm with opposition based learning rules. *J. Phys. Conf. Ser.* **1693**(1), 0–6 (2020). <https://doi.org/10.1088/1742-6596/1693/1/012117>
2. Guo, X., Yan, X., Jermisittiparsert, K.: Using the modified mayfly algorithm for optimizing the component size and operation strategy of a high temperature PEMFC-powered CCHP. *Energy Rep.* **7**, 1234–1245 (2021). <https://doi.org/10.1016/j.egy.2021.02.042>
3. Chen, L., Xu, C., Song, H., Jermisittiparsert, K.: Optimal sizing and siting of EVCS in the distribution system using metaheuristics: a case study. *Energy Rep.* **7**, 208–217 (2021). <https://doi.org/10.1016/j.egy.2020.12.032>
4. Dey, A., Chattopadhyay, S., Singh, P.K., Ahmadian, A., Ferrara, M., Sarkar, R.: A hybrid meta-heuristic feature selection method using golden ratio and equilibrium optimization algorithms for speech emotion recognition. *IEEE Access* **8**, 200953–200970 (2020). <https://doi.org/10.1109/ACCESS.2020.3035531>
5. Olivas, F., Valdez, F., Melin, P., Sombra, A., Castillo, O.: Interval type-2 fuzzy logic for dynamic parameter adaptation in a modified gravitational search algorithm. *Inf. Sci.* **476**, 159–175 (2019)
6. Ontiveros, E., Melin, P., Castillo, O.: High order α -planes integration: a new approach to computational cost reduction of General Type-2 Fuzzy Systems. *Eng. Appl. Artif. Intell.* **74**, 186–197 (2018)

Optimization: Theory and Applications

Symmetric-Approximation Energy-Based Estimation of Distribution (SEED) Algorithm for Solving Continuous High-Dimensional Global Optimization Problems



Valentín Calzada-Ledesma, Juan de Anda-Suárez, Lucero Ortiz-Aguilar,
Luis Fernando Villanueva-Jiménez, and Rosa Trasviña-Osorio

Abstract The Symmetric-approximation Energy-based Estimation of Distribution (SEED) is a continuous optimization algorithm based on the univariate Estimation of Distribution Algorithms (EDAs). Generally speaking, SEED uses Boltzmann selection and a symmetric divergence measure to guide the evolutionary process toward the optimum by constructing and sampling explicit probabilistic models of promising candidate solutions. SEED has been compared with other univariate EDAs, showing better results for low-dimensional continuous problems, and converging faster than the compared algorithms. In this paper, we propose to use SEED for solving continuous high-dimensional global optimization problems, and the results are compared with those of other representative evolutionary algorithms from the state-of-the-art such as Differential Evolution and Particle Swarm Optimization. To stress the algorithms, the search domain is extended from $x \in [-1000, 1000]^n$ for several benchmark functions whose dimension ranges from 500 to 10,000. Finally, to know the best performing algorithm, the convergence results of all algorithms are statistically compared using the Page's Trend Test.

Keywords Evolutionary algorithms · High-dimensional optimization · Continuous global optimization · Convergence comparison

1 Introduction

Currently, various scientific and industrial applications deal with problems that depend on a large number of variables, for example: nonlinear function optimization problems [11], high-dimensional classification and clustering [9], training a deep learning model [9], high-dimensional non-linear optimal control problems [1],

V. Calzada-Ledesma (✉) · J. de Anda-Suárez · L. Ortiz-Aguilar · L. F. Villanueva-Jiménez · R. Trasviña-Osorio
Tecnológico Nacional de México, Instituto Tecnológico Superior de Purísima del Rincón,
Purísima del Rincón, Guanajuato, México
e-mail: valentin.cl@purisima.tecnm.mx

among others. Furthermore, it is well known that the more variables that need to be analyzed, the computational cost tends to grow exponentially, this problem is called “the curse of dimensionality” [3]. Therefore, the need arises to find new algorithms capable of dealing with these types of problems to improve processing speed in order to implement strategies that can work, as far as possible, in real time [6, 14, 15].

Estimation of Distribution Algorithms (EDAs) are stochastic optimization methods that guide the search for the optimum by building explicit probabilistic models which are iteratively refined to sample better promising solutions. In EDAs, the probabilistic models provide the mutation and selection operators that are commonly used in other evolutionary algorithms and are able to adapt to the structure of the problem, which is an advantage over other metaheuristics [8, 18].

The Symmetric-approximation Energy-based Estimation of Distribution (SEED) is a continuous optimization algorithm designed under a univariate EDAs approach which implements the Boltzmann selection to guide the evolutionary process toward the optimum [4]. In order to implement the Boltzmann selection, an analytical approximation in terms of the Gaussian parameters (μ , $v = \sigma^2$) to the Boltzmann distribution is performed [13, 16]; this is achieved from the minimization of a divergence measure between both distributions. The most used divergence measure is the Kullback–Leibler [13] (which is not symmetric) and has been implemented in several EDAs such as Univariate Marginal Distribution Algorithm (UMDA), Estimation of Multivariate Normal Algorithm with Boltzmann Selection (EMNA-B), and Boltzmann Univariate Marginal Distribution Algorithm (BUMDA) [20]. In contrast, the Jeffreys divergence [12] (which is symmetric) was recently incorporated in SEED, showing statistically better convergence performance than UMDA and BUMDA (which competes with Boltzmann Gaussian-UMDA and EMNA-B [20]). However, all comparisons have been made between algorithms belonging to the EDA family for low-dimensional problems.

In this paper, we propose to use SEED for solving continuous high-dimensional global optimization problems, and the results are compared with those of other representative evolutionary algorithms outside the EDAs family such as Differential Evolution (DE) [7] and Particle Swarm Optimization (PSO) [19]. The objective of this work is to show that SEED is capable of dealing with high-dimensionality. In addition, an experimental design is proposed to show the efficacy of SEED under complicated conditions, for instance, low population, few function calls, and extended search range beyond what is usual in the state-of-the-art. In order to have control over the experimental process, the study is limited to benchmark objective functions from the state-of-the-art, since the optimal value and the search domain are known, which allows an adequate comparison frame between the studied algorithms. Finally, to know the best performing algorithm, the convergence results of all algorithms are statistically compared using the Page’s Trend Test [5].

The rest of this paper is organized as follows. In Sect. 2, the related concepts are described. Section 3 details the experimental design to perform the comparative study. In Sect. 4, the results and a discussion are presented. Finally, our conclusions and directions for future work are offered in Sect. 5.

2 Background

2.1 Continuous Global Optimization Problems.

Global optimization is the process of finding the minimum or maximum value of a function f with n parameters. Generally speaking, for an unconstrained function, the minimization problem may be formulated as:

$$\min_x f(x) \quad (1)$$

where f is the so-called objective function (or fitness function in evolutionary parlance) and x represents a vector of n parameters. If f is a function $\mathbb{R}^n \rightarrow \mathbb{R}$, such that the elements x_i of x and the output value are real numbers, then the global optimization problem is continuous [17]. Finding solutions to continuous global optimization problems can be complicated, for instance, for problems of two or three dimensions the behavior of f can be explored graphically as shown in Figs. 1 and 2, but for higher dimensions it is difficult to know a structure that provides clues about when a candidate solution x' is close to the optimal solution x , so it is hard for an algorithm to find the global optimum consistently and efficiently [17].

For some applications the objective function has certain characteristics such as convexity and differentiability, which allows the application of gradient-based algorithms. However, despite the fact that these properties facilitate the search for optimal values, it can happen that the methods get stuck in a local optimum, therefore, finding the global optimum can still be difficult [17]. Figure 3 shows an example of a local and global optimum values in a continuous function.

For high-dimensional problems (up to 500 or 10,000 parameters), the need for computational resources is also added, since when dimensionality increases the volume of space also increases exponentially and evaluating an objective function can be extremely expensive, not to mention that the search process takes longer. For

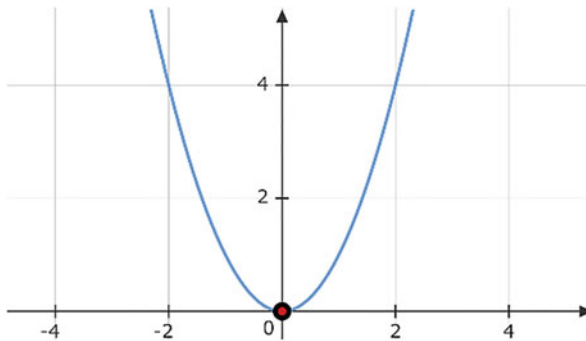


Fig. 1 2D plot of the Sphere function with its global optimum (red circle)

Fig. 2 3D plot of the Sphere function with its global optimum (red circle)

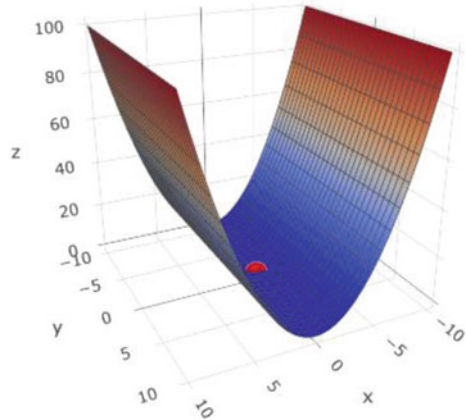
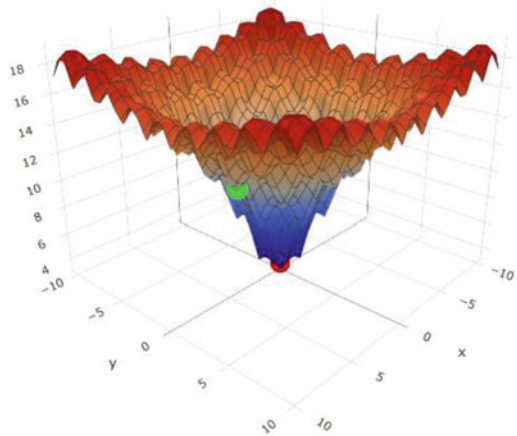


Fig. 3 Plot of the Ackley function with a local optimum (green circle) and its global optimum (red circle)



some algorithms, searching for the solution in a sparse space can make the problem computationally intractable, also this sparsity is a crucial concern for any method that requires results with statistical significance.

2.2 *Symmetric-Approximation Energy-Based Estimation of Distribution (SEED)*

In recent years, several works on EDAs have turned their attention to the Boltzmann distribution (Eq. 2), which belongs to the field of statistical mechanics to model the distribution of particles in certain *energy states*:

$$P_x = P(x; \beta) = \frac{1}{Z} e^{\beta g_x} \quad (2)$$

where Z is a normalizing constant known as the partition function, g_x is the energy of the states x , and $\beta = -\frac{1}{T}$, with T representing the temperature of the system [10]. The Boltzmann distribution has a property: states with lower energy have a higher probability of being occupied than states with higher energy. That is why Boltzmann-based EDAs, use this distribution to improve the search for the global optimum, since a minimization of an *energy* function [10], that is, the negative of the fitness function, can be performed.

Notwithstanding the advantages of employing the Boltzmann distribution, it is well-known that it cannot be used directly to sample new candidate solutions. However, to deal with that problem, a minimization of a divergence measure between the Boltzmann and Gaussian distributions through an approximation in terms of the Gaussian parameters (μ , $\nu = \sigma^2$) can be performed [13, 16]. In this way, it is possible to sample new candidate solutions using the Gaussian distribution “in the eyes” of Boltzmann.

SEED is a univariate EDA which incorporates the Jeffreys divergence to perform such minimization, showing statistically better convergence performance than UMDA, and BUMDA [4]. The discretized parameters μ and ν resulting from this approximation are shown below.

$$\mu \approx \frac{\frac{1}{Z\beta} \sum_i^m \exp(\beta g_{x_i}) x_i + \sum_i^m g_{x_i} x_i}{\frac{m}{\beta(b-a)} + \sum_i^m g_{x_i}} \quad (3)$$

$$\nu \approx \frac{\frac{1}{Z\beta} \sum_i^m \exp(\beta g_{x_i}) (x_i - \mu)^2 + \sum_i^m g_{x_i} (x_i - \mu)^2}{\sum_i^m g_{x_i}} \quad (4)$$

where $Z \approx \frac{m}{(b-a) \sum_i^m \exp(\beta g_{x_i})}$, a and b provide the domain of the fitness function, and m is the number of top-ranked selected individuals (see Algorithm 2). When sampling new candidate solutions using the Gaussian distribution, μ positions the population on a trajectory towards the optimum, and ν provides the diversity, so that SEED can perform a Boltzmann selection-guided exploration in the solution space [4].

As can be seen, the expressions for μ (Eq. (3)) and ν (Eq. (4)) require β . Other Boltzmann-based EDAs have a β in their approximation and use different approaches to set it properly. To deal with this problem, in SEED a minimum-variance estimator is derived, resulting in a self-adaptive mechanism to compute β for each particular fitness function. This is, $\beta = \frac{1}{g_{x^*}}$, where g_{x^*} denotes the fitness value of the best solution x^* , found by the SEED so far. The SEED pseudocode is provided below [4].

Algorithm 1: Pseudocode for SEED**Require:** N : population size, n : dimensions, a : Lower limit, b : Upper limit**Ensure:** $P_{best}^{(t)}$: best solution

1. $t \leftarrow 0$: iteration
2. $P^{(t)} \leftarrow \text{Initialize Population} \sim U(N, a, b)$
3. $g(\vec{x}_i) \leftarrow \text{Evaluate Population}(P^{(t)})$
4. $P_{best}^{(t)} \leftarrow \text{Get Best Solution}(P^{(t)})$
5. **while** Stop Condition is FALSE **do**
6. $S^{(t)} \leftarrow \text{Select Population}(t, g(\vec{x}_i)); \text{ see Algorithm 2.}$
7. Compute $\vec{\mu}^{(t)}$ and $\vec{v}^{(t)}$ from $S^{(t)}$; via Eqs. (3) & (4)
8. $P^{(t+1)} \leftarrow \text{Sample New Population} \sim N(\vec{\mu}^{(t)}, \vec{v}^{(t)})$
9. $g(\vec{x}_i) \leftarrow \text{Evaluate New Population}(P^{(t+1)})$
10. $P_{best}^{(t+1)} \leftarrow \text{Get Best Solution}(P^{(t+1)})$
11. **if** $P_{best}^{(t)} \geq P_{best}^{(t+1)}$ **then**
12. Incorporate $P_{best}^{(t)}$ into $P^{(t+1)}$
13. **end if**
14. $t \leftarrow t + 1$
15. $P_{best}^{(t)} \leftarrow \text{Get Best Solution}(P^{(t)})$
16. **end while**
17. **return** $P_{best}^{(t)}$

It is well-known that an EDA converges to a global optimum if the selection is elitist. To ensure the convergence of the algorithm a selection of top-ranked solutions $S^{(t)}$ of the current population is performed. SEED use the selection method proposed in [20], where $S^{(t)}$ is adaptively formed following a series of criteria as described in Algorithm 2.

Algorithm 2: Pseudocode for the Selection Method of SEED**Require:** t : iteration, $g(\vec{x}_i)$: fitness values of the population**Ensure:** $S^{(t)}$: selected top-ranked solutions

1. **if** $t == 0$ **then**
2. $\theta^{(t)} = \min\{g(\vec{x}_i)\}$
3. **else**
4. $g_{min} = \min\{g(\vec{x}_i) | g(\vec{x}_i) \geq \theta^{(t-1)}\}$
5. $g_{mid} = g(\vec{x}_{N/2})$
6. $\theta^{(t)} = \max\{g_{min}, g_{mid}\}$
7. **end if**
8. $S^{(t)} \leftarrow \text{Select Solutions Where } g(\vec{x}_i) \geq \theta^{(t)}$
9. **return** $S^{(t)}$

2.3 Differential Evolution

Differential Evolution is a stochastic global optimization algorithm, which maintains a population of candidate solutions which, during an iterative process, are evaluated, mutated, and selected [7]. The Mutation operation involves the creation of new candidate solutions from the weighted difference between two members of the current population selected at random and added to a third member of the population, allowing members of the population to be perturbed. Together with the Selection operation, the perturbation effect self-organizes the sampling of the problem space, allowing the search process to be carried out on known areas of interest [7].

Differential Evolution has a $DE/x/y/z$ nomenclature that describes the adopted configuration, where x represents the solution to be perturbed, y is the number of difference vectors used, and z is the recombination operator performed as *bin* for binomial and *exp* for exponential. Although there are several variants of DE, in this paper the $DE/rand/1/bin$ scheme is used. A detailed explanation follows [2, 7]. Initially, a population P of N individuals of size n is uniformly generated over the search space and evaluated using a fitness function. For each individual in P , a ω vector is generated using the Mutation operation defined in Eq. (5):

$$\omega = F \cdot (\varepsilon_2 - \varepsilon_3) + \varepsilon_1 \quad (5)$$

where ε_1 , ε_2 and ε_3 are three different individuals randomly selected at current iteration t , and $F \in (0, 2]$ is a constant factor that controls the amplification of the differential variation. Next, a vector γ is formed using the Crossover operation defined in Eq. (6):

$$\gamma_j = \begin{cases} \omega_j, & rand_j \leq Cr \\ ind_j, & \text{otherwise} \end{cases} \quad (6)$$

where ind_j indicates the j th component of the ω vector, with $j \in \{1, \dots, d\}$; $rand_j \in [0, 1)$ is a uniform random number, and $Cr \in [0, 1]$ is the crossover rate. Finally, the Selection of individuals that will form the new generation ($t + 1$), is defined in Eq. (7), where $f(\cdot)$ is the fitness function.

$$S_{ind} = \begin{cases} \gamma, & f(\gamma) \leq f(ind) \\ ind_j, & \text{otherwise} \end{cases} \quad (7)$$

The process is repeated until the stop criteria are met. A pseudocode of DE is shown in Algorithm 3 [2].

Algorithm 3: Pseudocode for Differential Evolution

Require: N, n, F, Cr : Control parameters

Ensure: $P_{best}^{(t)}$: best solution

1. $P^{(t \leftarrow 0)} \leftarrow \text{Initialize Population} \sim U(N, a, b)$
 2. Evaluate Population($P^{(t)}$)
 3. $P_{best}^{(t)} \leftarrow \text{Get Best Solution}(P^{(t)})$
 4. **while** (stop condition is FALSE) **do**
 5. **for each** $ind \in P^{(t)}$ **do**
 6. $\{\varepsilon_1, \varepsilon_2, \varepsilon_3\} \leftarrow \text{Get Individuals}(P^{(t)})$
 7. $\omega \leftarrow \text{Mutation}(\varepsilon_1, \varepsilon_2, \varepsilon_3, F)$
 8. $\gamma \leftarrow \text{Crossover}(\omega, ind, Cr)$
 9. $S_{ind} \leftarrow \text{Selection}(\gamma, ind)$
 10. Incorporate S_{ind} into $P^{(t+1)}$
 11. **end**
 12. $t \leftarrow t + 1$
 13. Evaluate Population($P^{(t)}$)
 14. $P_{best}^{(t)} \leftarrow \text{Get Best Solution}(P^{(t)})$
 15. **end**
 16. **return** $P_{best}^{(t)}$
-

2.4 Particle Swarm Optimization

Particle Swarm Optimization (PSO) is a bio-inspired algorithm that tries to simulate the collective behavior of animals such as birds and fish in search of food. The particles in the swarm move through an environment guided by the fittest members of the swarm, which bias their movement towards historically good areas of their environment [19].

The PSO algorithm sets initially random positions for all particles over the search space, and random velocities. During the search process, the position of each particle is modified based on its speed, the best known global position in the problem space, and the best known position of a particle. The objective function is sampled after each position update. Over time, by combining the mechanisms of exploration and exploitation of known good positions in search space, the particles converge around one or more optimal values [19].

In the PSO algorithm, each iteration, the velocity of a particle is updated using the next equation:

$$\begin{aligned}
 v_i(t+1) = & v_i(t) + (c_1 \times rand \times (p_i^{best} - p_i(t))) \\
 & + (c_2 \times rand \times (p_{gbest} - p_i(t)))
 \end{aligned} \tag{8}$$

where $v_i(t + 1)$ is the new velocity for the i th particle, c_1 and c_2 are the weighting coefficients for the personal best and global best positions respectively, $p_i(t)$ is the i th position of the particle at iteration t , p_i^{best} is the i th best known position of the particle, $p_{g_{best}}$ is the best position known to the swarm, and $rand \in [0, 1]$ is uniform random number. A position of the particle is updated using the following equation:

$$p_i(t + 1) = p_i(t) + v_i(t) \quad (9)$$

Finally, some considerations for using the PSO algorithm are detailed below. The speed at which a particle can move must be bounded, for example, to a percentage of the size of the search domain. Particles can go outside the boundary of the search domain but can be penalized or biased to return to a position in the search domain. Alternatively, a domain wrapping strategy can be used creating a looping in the chosen dimensionality. The learning factors should be between 0 and 4, usually 2. A coefficient of inertia or moment can be entered to limit the velocity change. A pseudocode of PSO is shown in Algorithm 4.

Algorithm 4: Pseudocode for PSO

Require: ProblemSize, N : Population size

Ensure: $p_{g_{best}}$

1. $Population \leftarrow \emptyset$
 2. $p_{g_{best}} \leftarrow \emptyset$
 3. **for** $i = 1$ **to** N **do**
 4. $P_{velocity} \leftarrow \text{RandomVelocity}()$
 5. $P_{position} \leftarrow \text{RandomPosition}(N)$
 6. $p_{p_{best}} \leftarrow P_{position}$
 7. **if** $f(p_{p_{best}}) \leq f(p_{g_{best}})$ **then**
 8. $p_{g_{best}} \leftarrow p_{p_{best}}$
 9. **end**
 10. **end**
 11. **while** (stop condition is FALSE) **do**
 12. **foreach** $P \in Population$ **do**
 13. $P_{velocity} \leftarrow \text{UpdateVelocity}(P_{velocity}, p_{p_{best}}, p_{g_{best}})$
 14. $P_{position} \leftarrow \text{UpdatePosition}(P_{position}, P_{velocity})$
 15. **if** $f(P_{position}) \leq f(p_{p_{best}})$ **then**
 16. $p_{p_{best}} \leftarrow P_{position}$
 17. **if** $f(p_{p_{best}}) \leq f(p_{g_{best}})$ **then**
 18. $p_{g_{best}} \leftarrow p_{p_{best}}$
 19. **end**
 20. **end**
 21. **end**
 22. **end**
 23. **return** $p_{g_{best}}$
-

Table 1 Test problems used in comparisons

Function	Equation
Ackley	$f(x) = 20 - 20 \exp\left(-0.2 \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}\right) + \exp\left(\sum_{i=1}^n \cos \frac{2\pi x_i}{n}\right) + \exp(1)$
Cigar	$f(x) = x_1^2 + \sum_{i=2}^n 10^6 x_i^2$
Cigar tablet	$f(x) = x_1^2 + \sum_{i=2}^{n-1} 10^4 x_i^2 + 10^8 x_n^2$
Ellipsoid	$f(x) = \sum_{i=1}^n 10^{6 \frac{i-1}{n-1}} x_i^2$
Griewangk	$f(x) = 1 + \sum_{i=1}^n \frac{x_i^2}{4000} - \prod_{i=1}^n \cos \frac{x_i}{\sqrt{i}}$
Rastrigin	$f(x) = 10n + \sum_{i=1}^n [x_i^2 - 10 \cos(2\pi x_i)]$
Rosenbrock	$f(x) = \sum_{i=2}^{n-1} \left(100(x_i - x_{i-1}^2)^2 + (x_i - 1)^2\right)$
Sphere	$f(x) = \sum_{i=1}^n x_i^2$
Two axes	$f(x) = \sum_{i=1}^{n/2} 10^6 x_i^2 + \sum_{i=\frac{n}{2}+1}^n x_i^2$

3 Experimental Design

This section presents the experimental design for the convergence comparison between Symmetric-approximation Energy-based Estimation of Distribution (SEED)¹ [4] and two non-EDAs algorithms: Differential Evolution (DE)² [7] and Particle Swarm Optimization (PSO)³ [19] for high-dimensional problems. The comparison is carried out based on the performance of the algorithms over a benchmark function set (which is reported in Table 1), whose optimum value is 0 for all functions, and under the same experimental scheme as in [4, 20].

In [4], it was experimentally proven that using a population of 50 solutions, and for an extended search domain to $x \in [-1000, 1000]^n$ of the functions of Table 1, SEED converges before 50,000 function calls to the predefined convergence criterion $\epsilon = 1 \times 10^{-6}$ [4] for 50, 100, and 200 dimensions, surpassing other EDAs such as UMDA and BUMDA.

¹ <https://github.com/LIDT-Lab/pySEED>.

² https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.differential_evolution.html.

³ <https://pythonhosted.org/pyswarm/>.

In this work, the Page's trend test is carried out to evaluate the performance of the compared algorithms. This test computes differences between the convergence rate of any two algorithms, based on their performance achieved at different points in time called cut-off points [5]. Since the main objective of this paper is to show that SEED is able to handle high-dimensional problems, three scenarios with 500, 1000 and 10,000 dimensions were considered for each benchmark function of Table 1. Taking in count the results mentioned above, in this experiments the size of the initial population is $N = 50$ for all algorithms. Specifically, for DE: $F = 0.6$ and $Cr = 0.9$ [7]; for PSO: $C_1 = 0.5$, $C_2 = 0.3$, and $w = 0.9$ [19]; and finally, SEED does not require extra parameters. To have statistically meaningful results, 33 independent experiments are performed and Page's trend test is carried out using the median of these experiments on each benchmark problem, and for 10 cut-off points starting at 5000 and ending at 50,000 function calls. The search range is extended to $x \in [-1000, 1000]^n$, and the convergence criterion is set to $\epsilon = 0$.

4 Results

The convergence results of Differential Evolution (DE), Particle Swarm Optimization (PSO), and Symmetric-approximation Energy-based Estimation of Distribution (SEED) for solving high-dimensional problems are shown in Tables 2, 3, and 4 respectively.

In Fig. 4, three convergence plots of some representative functions of a single independent experiment are shown from 500 to 10,000 dimensions, where DE, PSO, and SEED are contrasted each other. It is worth mentioning that the plots are in logarithmic scale to correctly appreciate the algorithms performance.

4.1 Page's Trend Test Statistical Analysis

Over the data presented in Tables 2, 3, and 4, the Page's trend test is carried out, and results in terms of p -values are reported in Table 5. Each p -value represents the probability of rejection of the null hypothesis H_0 : a pair of algorithms converges equally, in favor of the alternative hypothesis H_1 : the algorithm in the row converges faster than some of the algorithms in the columns of Table 5. Rejected hypotheses, at a significance level of $\alpha = 0.1$, are highlighted in a bold typeface.

As can be seen in Table 5, the p -values are equal or close to 0 and 1, this is because the difference in the performance of the algorithms is very wide, this can be clearly observed in the convergence curves of Fig. 4. SEED reaches a value close to the optimum of the function, while DE and PSO obtain values that are too far from it. If the algorithms had been more competitive, the p -values would have had values closer to each other.

Table 2 DE convergence results for 10 cut-off points in 500, 5000 and 10,000 dimensions

500-D	Cut-off points (function calls)									
	5000	10,000	15,000	20,000	25,000	30,000	35,000	40,000	45,000	50,000
Ackley	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01
Cigar	1.95E + 13	1.52E + 13	1.19E + 13	9.90E + 12	8.45E + 12	7.54E + 12	6.72E + 12	6.01E + 12	5.45E + 12	5.03E + 12
Cigar Tablet	3.12E + 11	2.23E + 11	1.85E + 11	1.57E + 11	1.36E + 11	1.19E + 11	1.07E + 11	9.64E + 10	8.75E + 10	7.99E + 10
Ellipsoid	1.01E + 12	5.33E + 11	3.02E + 11	1.93E + 11	1.30E + 11	1.02E + 11	8.69E + 10	7.18E + 10	5.15E + 10	3.83E + 10
Griewangk	4.99E + 03	3.89E + 03	3.29E + 03	2.81E + 03	2.52E + 03	2.25E + 03	2.07E + 03	1.88E + 03	1.76E + 03	1.62E + 03
Rastrigin	2.00E + 07	1.56E + 07	1.28E + 07	1.09E + 07	9.75E + 06	8.66E + 06	7.82E + 06	7.23E + 06	6.62E + 06	6.13E + 06
Rosenbrock	2.54E + 14	1.32E + 14	9.69E + 13	7.77E + 13	6.37E + 13	5.47E + 13	4.84E + 13	4.29E + 13	3.90E + 13	3.62E + 13
Sphere	2.27E + 07	1.73E + 07	1.44E + 07	1.24E + 07	1.11E + 07	9.35E + 06	8.03E + 06	7.28E + 06	6.73E + 06	6.37E + 06
Two Axes	7.58E + 12	4.63E + 12	2.91E + 12	2.12E + 12	1.67E + 12	1.30E + 12	1.06E + 12	8.85E + 11	7.49E + 11	6.10E + 11
5000-D										
Ackley	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01
Cigar	2.22E + 14	2.04E + 14	1.99E + 14	1.94E + 14	1.90E + 14	1.86E + 14	1.83E + 14	1.80E + 14	1.78E + 14	1.76E + 14
Cigar Tablet	3.34E + 12	3.11E + 12	3.02E + 12	2.96E + 12	2.90E + 12	2.84E + 12	2.80E + 12	2.76E + 12	2.73E + 12	2.70E + 12
Ellipsoid	1.56E + 13	1.27E + 13	1.13E + 13	1.01E + 13	9.26E + 12	8.76E + 12	8.36E + 12	7.98E + 12	7.73E + 12	7.50E + 12
Griewangk	4.59E + 04	4.26E + 04	4.13E + 04	4.03E + 04	3.95E + 04	3.89E + 04	3.83E + 04	3.78E + 04	3.73E + 04	3.69E + 04
Rastrigin	2.19E + 08	2.05E + 08	2.00E + 08	1.95E + 08	1.91E + 08	1.88E + 08	1.85E + 08	1.82E + 08	1.80E + 08	1.78E + 08
Rosenbrock	2.67E + 15	1.95E + 15	1.81E + 15	1.72E + 15	1.66E + 15	1.60E + 15	1.55E + 15	1.52E + 15	1.49E + 15	1.46E + 15
Sphere	2.26E + 08	2.08E + 08	2.01E + 08	1.97E + 08	1.93E + 08	1.89E + 08	1.86E + 08	1.84E + 08	1.81E + 08	1.79E + 08
Two Axes	9.38E + 13	8.51E + 13	8.10E + 13	7.83E + 13	7.52E + 13	7.28E + 13	7.10E + 13	6.93E + 13	6.77E + 13	6.62E + 13

(continued)

Table 2 (continued)

500-D	Cut-off points (function calls)									
	5000	10,000	15,000	20,000	25,000	30,000	35,000	40,000	45,000	50,000
10,000-D										
Ackley	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01	2.00E + 01
Cigar	4.48E + 14	3.78E + 14	3.72E + 14	3.68E + 14	3.63E + 14	3.59E + 14	3.56E + 14	3.53E + 14	3.50E + 14	3.47E + 14
Cigar Tablet	5.35E + 12	4.91E + 12	4.83E + 12	4.76E + 12	4.71E + 12	4.66E + 12	4.62E + 12	4.59E + 12	4.55E + 12	4.51E + 12
Ellipsoid	3.21E + 13	2.73E + 13	2.57E + 13	2.41E + 13	2.32E + 13	2.24E + 13	2.18E + 13	2.12E + 13	2.07E + 13	2.02E + 13
Griewangk	1.24E + 05	1.03E + 05	1.02E + 05	1.00E + 05	9.92E + 04	9.81E + 04	9.72E + 04	9.63E + 04	9.56E + 04	9.49E + 04
Rastrigin	4.70E + 08	3.99E + 08	3.92E + 08	3.88E + 08	3.84E + 08	3.80E + 08	3.77E + 08	3.74E + 08	3.71E + 08	3.68E + 08
Rosenbrock	4.57E + 15	3.37E + 15	3.26E + 15	3.16E + 15	3.09E + 15	3.03E + 15	2.98E + 15	2.93E + 15	2.89E + 15	2.86E + 15
Sphere	4.58E + 08	4.27E + 08	4.22E + 08	4.16E + 08	4.12E + 08	4.08E + 08	4.04E + 08	4.02E + 08	3.99E + 08	3.96E + 08
Two Axes	1.78E + 14	1.59E + 14	1.54E + 14	1.51E + 14	1.48E + 14	1.46E + 14	1.43E + 14	1.41E + 14	1.40E + 14	1.38E + 14

Table 3 PSO convergence results for 10 cut-off points in 500, 5000, and 10,000 dimensions

500-D	Cut-off points (function calls)									
	5000	10,000	15,000	20,000	25,000	30,000	35,000	40,000	45,000	50,000
Ackley	2.16E + 01	2.16E + 01	2.16E + 01	2.16E + 01	2.16E + 01	2.16E + 01	2.15E + 01	2.15E + 01	2.15E + 01	2.15E + 01
Cigar	2.06E + 13	1.88E + 13	1.84E + 13	1.84E + 13	1.84E + 13	1.84E + 13	1.83E + 13	1.83E + 13	1.83E + 13	1.83E + 13
Cigar Tablet	2.13E + 11	1.80E + 11	1.76E + 11	1.75E + 11	1.75E + 11	1.75E + 11	1.75E + 11	1.75E + 11	1.75E + 11	1.75E + 11
Ellipsoid	5.50E + 11	4.74E + 11	4.57E + 11	4.49E + 11	4.42E + 11	4.17E + 11	4.17E + 11	4.17E + 11	4.17E + 11	4.17E + 11
Griewangk	4.98E + 03	4.60E + 03	4.55E + 03	4.54E + 03	4.54E + 03	4.53E + 03	4.53E + 03	4.53E + 03	4.53E + 03	4.53E + 03
Rastrigin	1.93E + 07	1.76E + 07	1.75E + 07	1.74E + 07	1.74E + 07	1.74E + 07	1.74E + 07	1.74E + 07	1.74E + 07	1.74E + 07
Rosenbrock	2.54E + 14	2.01E + 14	1.89E + 14	1.88E + 14	1.87E + 14	1.87E + 14	1.87E + 14	1.86E + 14	1.86E + 14	1.86E + 14
Sphere	1.97E + 07	1.83E + 07	1.80E + 07	1.79E + 07	1.79E + 07	1.79E + 07	1.79E + 07	1.79E + 07	1.79E + 07	1.79E + 07
Two Axes	7.48E + 12	6.74E + 12	6.55E + 12	6.50E + 12	6.50E + 12	6.49E + 12	6.49E + 12	6.49E + 12	6.49E + 12	6.49E + 12
5000-D										
Ackley	2.17E + 01	2.17E + 01	2.17E + 01	2.17E + 01	2.17E + 01	2.17E + 01	2.17E + 01	2.17E + 01	2.17E + 01	2.17E + 01
Cigar	3.07E + 14	2.77E + 14	2.75E + 14	2.75E + 14	2.75E + 14	2.75E + 14	2.75E + 14	2.75E + 14	2.75E + 14	2.75E + 14
Cigar Tablet	3.28E + 12	2.90E + 12	2.89E + 12	2.88E + 12	2.88E + 12	2.88E + 12	2.88E + 12	2.88E + 12	2.88E + 12	2.88E + 12
Ellipsoid	1.66E + 13	1.56E + 13	1.55E + 13	1.55E + 13	1.55E + 13	1.55E + 13	1.55E + 13	1.55E + 13	1.55E + 13	1.55E + 13
Griewangk	7.29E + 04	6.70E + 04	6.67E + 04	6.67E + 04	6.67E + 04	6.66E + 04	6.66E + 04	6.66E + 04	6.66E + 04	6.66E + 04
Rastrigin	2.84E + 08	2.65E + 08	2.64E + 08	2.63E + 08	2.63E + 08	2.63E + 08	2.63E + 08	2.63E + 08	2.63E + 08	2.63E + 08
Rosenbrock	7.00E + 15	5.05E + 15	5.01E + 15	5.01E + 15	5.01E + 15	5.01E + 15	5.01E + 15	5.01E + 15	5.01E + 15	5.00E + 15
Sphere	2.85E + 08	2.63E + 08	2.62E + 08	2.62E + 08	2.62E + 08	2.62E + 08	2.62E + 08	2.62E + 08	2.62E + 08	2.62E + 08
Two Axes	1.42E + 14	1.29E + 14	1.28E + 14	1.28E + 14	1.28E + 14	1.28E + 14	1.28E + 14	1.28E + 14	1.28E + 14	1.28E + 14

(continued)

Table 3 (continued)

500-D	Cut-off points (function calls)									
	5,000	10,000	15,000	20,000	25,000	30,000	35,000	40,000	45,000	50,000
10,000-D										
Ackley	2.17E + 01	2.17E + 01	2.17E + 01	2.17E + 01	2.17E + 01	2.17E + 01	2.17E + 01	2.17E + 01	2.17E + 01	2.17E + 01
Cigar	6.18E + 14	5.72E + 14	5.70E + 14	5.70E + 14	5.70E + 14	5.69E + 14	5.69E + 14	5.69E + 14	5.69E + 14	5.69E + 14
Cigar Tablet	6.18E + 14	5.72E + 14	5.70E + 14	5.70E + 14	5.70E + 14	5.69E + 14	5.69E + 14	5.69E + 14	5.69E + 14	5.69E + 14
Ellipsoid	3.87E + 13	3.57E + 13	3.56E + 13	3.56E + 13	3.56E + 13	3.55E + 13	3.55E + 13	3.55E + 13	3.55E + 13	3.55E + 13
Griewangk	1.69E + 05	1.50E + 05	1.49E + 05	1.49E + 05	1.49E + 05	1.49E + 05	1.49E + 05	1.49E + 05	1.49E + 05	1.49E + 05
Rastrigin	6.46E + 08	5.90E + 08	5.87E + 08	5.87E + 08	5.87E + 08	5.87E + 08	5.87E + 08	5.87E + 08	5.87E + 08	5.87E + 08
Rosenbrock	9.97E + 15	8.52E + 15	8.48E + 15	8.47E + 15	8.47E + 15	8.47E + 15	8.47E + 15	8.47E + 15	8.47E + 15	8.47E + 15
Sphere	6.53E + 08	5.97E + 08	5.95E + 08	5.94E + 08	5.94E + 08	5.94E + 08	5.94E + 08	5.94E + 08	5.94E + 08	5.94E + 08
Two Axes	2.97E + 14	2.71E + 14	2.70E + 14	2.70E + 14	2.70E + 14	2.70E + 14	2.70E + 14	2.69E + 14	2.69E + 14	2.69E + 14

Table 4 SEED convergence results for 10 cut-off points in 500, 5000, and 10,000 dimensions

500-D	Cut-off points (function calls)									
	5000	10,000	15,000	20,000	25,000	30,000	35,000	40,000	45,000	50,000
Ackley	2.16E + 01	2.13E + 01	4.17E + 00	2.35E-02	3.65E-04	8.11E-06	9.54E-07	9.54E-07	1.19E-07	1.19E-07
Cigar	1.66E + 11	2.84E + 07	5.83E + 01	1.24E-02	6.91E-06	9.54E-07	1.19E-07	1.19E-07	1.19E-07	1.19E-07
Cigar Tablet	3.86E + 09	4.65E + 03	2.67E-01	1.30E-04	9.54E-07	9.54E-07	0.00E + 00	0.00E + 00	0.00E + 00	0.00E + 00
Ellipsoid	8.04E + 08	8.31E + 02	8.27E-02	4.28E-05	2.38E-07	1.19E-07	1.19E-07	1.19E-07	1.19E-07	1.19E-07
Griewangk	2.79E + 00	1.09E-02	5.48E-06	9.54E-07	1.19E-07	1.19E-07	1.19E-07	1.19E-07	1.19E-07	1.19E-07
Rastrigin	5.39E + 03	6.77E + 01	9.50E-03	4.29E-06	1.19E-07	1.19E-07	1.19E-07	1.19E-07	1.19E-07	1.19E-07
Rosenbrock	8.37E + 15	8.37E + 15	8.37E + 15	8.37E + 15	8.37E + 15	8.37E + 15	8.37E + 15	8.37E + 15	8.37E + 15	8.37E + 15
Sphere	4.69E + 02	6.95E-02	4.10E-05	2.38E-07	9.54E-07	1.19E-07	1.19E-07	1.19E-07	1.19E-07	1.19E-07
Two Axes	1.05E + 10	1.08E + 04	3.76E-01	1.67E-04	9.54E-07	1.19E-07	1.19E-07	1.19E-07	1.19E-07	1.19E-07
5000-D										
Ackley	2.16E + 01	2.15E + 01	2.11E + 01	7.90E + 00	6.71E-01	3.04E-02	2.25E-03	2.07E-04	2.04E-05	2.74E-06
Cigar	4.86E + 13	5.73E + 11	3.80E + 09	2.54E + 05	1.07E + 02	5.13E-01	4.47E-03	4.55E-05	1.07E-06	9.54E-07
Cigar Tablet	8.35E + 11	9.41E + 09	1.02E + 06	5.82E + 02	1.71E + 00	1.86E-02	1.86E-04	2.38E-06	9.54E-07	9.54E-07
Ellipsoid	1.80E + 12	1.33E + 10	1.63E + 06	7.05E + 02	1.83E + 00	1.73E-02	1.69E-04	2.50E-06	9.54E-07	9.54E-07
Griewangk	1.71E + 03	6.86E + 00	2.13E-01	2.47E-03	2.86E-05	9.54E-07	9.54E-07	9.54E-07	9.54E-07	9.54E-07
Rastrigin	5.90E + 05	2.18E + 04	8.77E + 02	2.18E + 00	1.99E-02	1.72E-04	2.27E-06	3.58E-07	1.19E-07	1.19E-07
Rosenbrock	9.65E + 16	9.65E + 16	9.65E + 16	9.65E + 16	9.65E + 16	9.65E + 16	9.65E + 16	9.65E + 16	9.65E + 16	9.65E + 16
Sphere	1.29E + 06	1.62E + 03	5.16E + 00	6.73E-02	7.26E-04	1.11E-05	5.96E-07	2.38E-07	1.19E-07	1.19E-07
Two Axes	1.11E + 13	1.41E + 11	7.21E + 08	2.01E + 04	1.63E + 01	1.16E-01	9.23E-04	9.66E-06	3.58E-07	1.19E-07

(continued)

Table 4 (continued)

500-D	Cut-off points (function calls)									
	5000	10,000	15,000	20,000	25,000	30,000	35,000	40,000	45,000	50,000
10,000-D										
Ackley	2.16E + 01	2.14E + 01	2.10E + 01	1.56E + 01	1.17E + 00	8.60E-02	6.88E-03	6.75E-04	8.00E-05	9.78E-06
Cigar	1.83E + 14	4.20E + 12	7.69E + 10	5.54E + 08	2.84E + 04	3.57E + 01	3.47E-01	4.45E-03	6.32E-05	1.55E-06
Cigar Tablet	1.85E + 12	4.27E + 10	4.95E + 08	2.34E + 04	3.79E + 01	3.76E-01	4.62E-03	6.22E-05	1.43E-06	9.54E-07
Ellipsoid	5.45E + 12	8.33E + 10	9.56E + 08	5.20E + 04	6.20E + 01	5.55E-01	7.97E-03	9.86E-05	1.91E-06	9.54E-07
Griewangk	8.99E + 03	4.65E + 01	1.29E + 00	2.57E-02	4.88E-04	1.01E-05	9.54E-07	9.54E-07	2.38E-07	1.19E-07
Rastrigin	3.52E + 06	5.46E + 04	4.72E + 03	2.68E + 01	2.65E-01	3.68E-03	6.51E-05	1.67E-06	9.54E-07	9.54E-07
Rosenbrock	1.93E + 17	1.93E + 17	1.93E + 17	1.93E + 17	1.93E + 17	1.93E + 17	1.93E + 17	1.93E + 17	1.93E + 17	1.93E + 17
Sphere	5.31E + 06	1.09E + 04	3.27E + 01	4.82E-01	9.71E-03	1.63E-04	3.34E-06	9.54E-07	2.38E-07	1.19E-07
Two Axes	7.51E + 13	9.72E + 11	1.02E + 10	2.01E + 06	1.09E + 03	3.02E + 00	3.38E-02	4.12E-04	5.13E-06	9.54E-07

Fig. 4 Convergence plots of Cigar, Rastrigin, and Griewangk functions of a single independent experiment for 500, 5000, and 10,000 dimensions. The search range was extended to $x \in [-1000, 1000]^n$

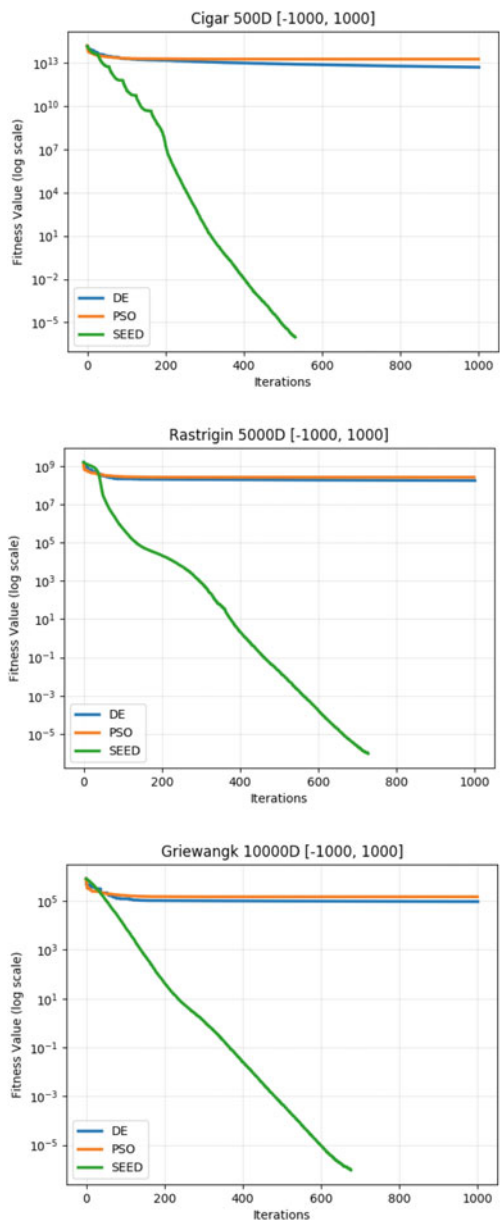


Table 5 Convergence results (p -values) for the Page's trend test on problems with 500, 5000 and 10,000 dimensions

		DE	PSO	SEED
500D	DE	–	1	1
	PSO	0	–	1
	SEED	0	0	–
5000D	DE	–	1	1
	PSO	0	–	1
	SEED	0	0	–
10000D	DE	–	1	1
	PSO	0	–	0.999996
	SEED	0	0.000004	–

After studying the results of the Page's trend test, the following analysis can be drawn. For problems in 500 dimensions, SEED shows the best convergence behavior, followed by PSO, while DE presents the worst convergence. For problems in 5000 dimensions, SEED presents the best convergence and DE the worst. For problems in 10,000 dimensions, SEED shows the best convergence, followed by PSO and DE, respectively.

In summary, we can conclude that, under the experimental design of this work, the statistical tests suggest that SEED performs better than the compared algorithms for high-dimensional problems.

5 Conclusions

The Symmetric-approximation Energy-based Estimation of Distribution (SEED) is a continuous optimization algorithm based on the univariate Estimation of Distribution Algorithms (EDAs).

In this work, we used SEED to solve continuous high-dimensional global optimization problems up to 10,000 dimensions, and the results were compared with those of other evolutionary algorithms representative of the state-of-the-art, such as Differential Evolution (DE) and Particle Swarm Optimization (PSO), by means of the Page's trend test statistical analysis. Through the results presented in Sect. 4, it was concluded that the performance of SEED is superior to the DE and PSO, this conclusion is supported by the results of the statistical analysis. It should be noted that SEED, unlike DE and PSO, can optimize continuous global problems of up to 10,000 dimensions with less than 50,000 function calls, in addition, SEED does not require additional control parameters, this fact facilitates its application at a technical level, avoiding an empirical study of parameters tuning.

As future work, the use of SEED to solve scientific and industrial problems that depend on a large number of variables is being considered.

Acknowledgements The authors wish to thank the Instituto Tecnológico Superior de Purísima del Rincón for the resources and support for the development of this research. We released SEED in a package called pySEED, available at the following link: <https://github.com/LIDT-Lab/pySEED>.

References

1. Boyko, A.I., Oseledets, I.V., Ferrer, G.: TT-QI: faster value iteration in tensor train format for stochastic optimal control. *Comput. Math. Math. Phys.* **61**(5), 836–846 (2021)
2. Calzada-Ledesma, V., Puga-Soberanes, H.J., Rojas-Domínguez, A., Ornelas-Rodríguez, M., Carpio-Valadez, J.M., Gómez-Santillán, C.G.: Comparing grammatical evolution's mapping processes on feature generation for pattern recognition problems. In: *Nature-Inspired Design of Hybrid Intelligent Systems*, pp. 775–785. Springer, Cham (2017)
3. Chen, S., Montgomery, J., Bolufé-Röhler, A.: Measuring the curse of dimensionality and its effects on particle swarm optimization and differential evolution. *Appl. Intell.* **42**(3), 514–526 (2015)
4. de Anda-Suárez, J., Carpio-Valadez, J.M., Puga-Soberanes, H.J., Calzada-Ledesma, V., Rojas-Domínguez, A., Jeyakumar, S., Espinal, A.: Symmetric-approximation energy-based estimation of distribution (SEED): a continuous optimization algorithm. *IEEE Access* **7**, 154859–154871 (2019)
5. Derrac, J., García, S., Hui, S., Suganthan, P.N., Herrera, F.: Analyzing convergence performance of evolutionary algorithms: a statistical approach. *Inf. Sci.* **289**, 41–58 (2014)
6. Fan, Q., Chen, Z., Li, Z., Xia, Z., Yu, J., Wang, D.: A new improved whale optimization algorithm with joint search mechanisms for high-dimensional global optimization problems. *Eng. Comput.* **37**(3), 1851–1878 (2021)
7. Feoktistov, V.: *Differential Evolution*, pp. 1–24. Springer, US (2006)
8. Gallagher, M., Freaan, M.: Population-based continuous optimization, probabilistic modelling and mean shift. *Evol. Comput.* **13**(1), 29–42 (2005)
9. Gambella, C., Ghaddar, B., Naoum-Sawaya, J.: Optimization problems for machine learning: a survey. *Eur. J. Oper. Res.* **290**(3), 807–828 (2021)
10. Greiner, W., Neise, L., Stöcker, H.: *Thermodynamics and Statistical Mechanics*. Springer Science & Business Media (2012)
11. Gu, Q., Wang, Q., Li, X., Li, X.: A surrogate-assisted multi-objective particle swarm optimization of expensive constrained combinatorial optimization problems. *Knowl.-Based Syst.* **223**, 107049 (2021)
12. Jeffreys, H.: An invariant form for the prior probability in estimation problems. *Proc. R. Soc. London. Ser. A. Math. Phys. Sci.* **186**(1007), 453–461 (1946)
13. Kullback, S., Leibler, R.A.: On information and sufficiency. *Ann. Math. Stat.* **22**(1), 79–86 (1951)
14. Long, W., Wu, T., Liang, X., Xu, S.: Solving high-dimensional global optimization problems using an improved sine cosine algorithm. *Expert Syst. Appl.* **123**, 108–126 (2019)
15. MiarNaeimi, F., Azizyan, G., Rashki, M.: Horse herd optimization algorithm: a nature-inspired algorithm for high-dimensional optimization problems. *Knowl.-Based Syst.* **213**, 106711 (2021)
16. Mühlenbein, H.: The equation for response to selection and its use for prediction. *Evol. Comput.* **5**(3), 303–346 (1997)

17. Mullen, K.M.: Continuous global optimization in R. *J. Stat. Softw.* **60**(1), 1–45 (2014)
18. Pelikan, M., Hauschild, M.W., Lobo, F.G.: Estimation of distribution algorithms. In: *Springer Handbook of Computational Intelligence*, pp. 899–928. Springer, Berlin (2015)
19. Poli, R., Kennedy, J., Blackwell, T.: Particle swarm optimization. *Swarm Intell.* **1**(1), 33–57 (2007)
20. Valdez, S.I., Hernández, A., Botello, S.: A Boltzmann based estimation of distribution algorithm. *Inf. Sci.* **236**, 126–137 (2013)

Optimization Models and Methods for Bin Packing Problems: A Case Study on Solving 1D-BPP



Jessica González-San Martín, Laura Cruz-Reyes, Bernabé Dorronsoro, Marcela Quiroz-Castellanos, Héctor Fraire, Claudia Gómez-Santillán, and Nelson Rangel-Valdez

Abstract The Bin Packing Problem (BPP) is a classic optimization problem that is known for its applicability and complexity, which belongs to a particular class of problems called NP-hard, in which, given a set of items of variable size, we search to accommodate them inside fixed size containers, seeking to optimize the number of containers to be used, that is, using the least number of containers to place the most significant number of items possible. For this problem, there are different variants such as one-dimensional (1D-BPP), two-dimensional (2D-BPP), or three-dimensional (3D-BPP) bin packing, multi-objective bin packing (MBP), dynamic bin packing (DBP), among others. The BPP has been preserved as a current study problem due to the various applications that it offers; therefore, in the current state of the art, there are different algorithms, mainly heuristics, for solving the problem, however, currently, there are still open aspects of this problem that have not been studied in-depth, such as changes in the environment (dynamism) and the presence of various objectives (multi-objective). This work presents a brief review of BPP optimization models and methods to solve it and its variants; we identify some methods that better impact performance and highlight future lines of research on this problem. We also present the GGA-CGT-II algorithm as a case study for 1D-BPP extended from the Grouping Genetic Algorithm with Controlled Gene Transmission (GGA-CGT) algorithm. In the proposed version, two different strategies were implemented:

J. G.-S. Martín (✉) · L. Cruz-Reyes · H. Fraire · C. Gómez-Santillán
Tecnológico Nacional de México, Instituto Tecnológico de Ciudad Madero, Cd. Madero,
Tamaulipas, México
e-mail: jessica_san_martin@hotmail.com

B. Dorronsoro
Computer Science Engineering Department, University of Cadiz, Puerto Real, Cadiz, Spain

M. Quiroz-Castellanos
Artificial Intelligence Research Center, Universidad Veracruzana, Xalapa-Enriquez, Veracruz,
México

N. Rangel-Valdez
CONACyT Research Fellow at Graduate Program Division, Tecnológico Nacional de México,
Instituto Tecnológico de Ciudad Madero, Cd. Madero, Tamaulipas, México

(a) Method to calculate various theoretical lower limits of the optimal solution and select the best one for each instance and (b) Instance reduction method to simplify the problem.

Keywords Bin packing · Strategies · Multiobjectives · Dynamism

1 Introduction

The bin packing problem (BPP) is defined as a problem that consists of storing objects of different sizes or weights in the fewest number of fixed-size containers [17, 30]. It is a classic NP-hard combinatorial optimization problem [19]. The most commonly used methods for its solution are through metaheuristics that include intelligent strategies since the complexity of the problem makes its solution computationally difficult through exact algorithms.

BPP has been preserved as a current study problem due to the various applications it offers; for this reason, today in the state-of-the-art, there are various heuristic algorithms for solving the problem, and there are different variants, such as one-dimensional (1D-BPP), 2-dimensional (2D-BPP) or 3-dimensional (3D-BPP) bin packing, multi-objective bin packing (MBP), dynamic bin packing (DBP), among others.

Over the years, different research groups have carried out various studies on this problem, generating metaheuristics that have shown satisfactory performance; however, at present, there are still open aspects of this problem that have not been studied in-depth, such as changes in the environment (dynamism) and the presence of various objectives (multi-objective). To date, there is no heuristic or metaheuristic algorithm capable of finding the optimal solution for all possible instances of a problem of this type despite the scientific community's efforts.

This work aims to study the optimization methods to solve problems of packaging objects in containers, that is, Bin packing and its variants, to identify those methods that provide a better impact on performance and highlight future lines of research on this problem.

The document continues as follows: Sect. 2 presents the Bin Packing Problems and the instances used for the proposed analysis. Then, Sect. 3 presents related works to bin packing and its variants, and the algorithms selected as a case study are described. Next, Sect. 4 presents the methodology followed for the redesign of the GGA-CGT algorithm. Then, in Sect. 5 the experiments and results obtained are described. And finally, Sect. 6 presents conclusions and future research directions for the bin packing problems.

2 The Bin Packing Problems

2.1 The One-Dimensional Bin Packing Problem and Variants

The one-dimensional bin packing problem (1D-BPP) is a well-known grouping combinatorial optimization problem defined as follows: given an unlimited number of bins with a fixed capacity c and a set of n items, each with a specific weight w_i , BPP aims to find the minimum number of bins needed to pack all the items without violating the capacity of any bin.

Formally, given a set $N = 1, \dots, n$ of items, a set of bins with the same capacity $c > 0$, and let w_i be the respective weight of each item $i \in N$ with $0 < w_i \leq c$. BPP consists of grouping all the elements in bins so that the sum of their weights does not exceed c , and the number of bins m used minimal [24]. In other words, we want to create a partition of N into the minimum number of subsets B_j possible:

$$\bigcup_{j=1}^m B_j = N$$

such that:

$$\sum_{i \in B_j} w_i \leq c, \quad 1 \leq j \leq m, \quad i \in N \quad (1)$$

BPP belongs to the NP-hard class [6, 19] because it requires many computational resources to be solved. Furthermore, the problem complexity grows exponentially with the problem size since the number of possible partitions is higher than $(n/2)^{n/2}$. This implies there is no efficient algorithm to find an optimal solution for every instance of BPP.

For this problem, there are some variants such as 2-dimensional (2D-BPP) [23, 25] or 3-dimensional (3D-BPP) bin packing [9]; In these variants [5], only the number of dimensions of both the items and the containers changes. Several practical constraints may also be part of any n -dimensional packing problem ($n = 1, 2, 3$). Recent works approaching practical constraints include the works of [12, 15].

Iori [21] present a survey for two-dimensional packing problems and resume the surveys published in the last two decades for all variants. However, these surveys do not include two relevant and realistic environmental conditions: the presence of multiple objectives and variations on time. We briefly describe them in the following sections.

2.2 Multiobjective Bin Packing Problem

More attention has been paid to the multi-objective needs of the systems that represent these problems in the last decade. The first multi-objective bin packing problem (MBP) was extended to minimize the number of containers and the average deviation of the center of gravity in the containers [26]. Since then, many other studies have included this objective of “load balancing” (Lara and Labrador 2010), as well as minimizing the maximum length of a container [14] and minimizing costs [32]. The community shows intensive activity modeling and solving for the bi-objective 2D-BPP (e.g., [2, 20, 28]).

Many recent works focus on modeling cloud computing problems as bin packing problems [3, 38].

The multi-objective bin packing problem is structured according to the standard multi-objective problem format, given in Eq. 2.

$$\text{minimize } F(x) = (f_1(x), \dots, f_m(x))^T \quad (2)$$

$$\text{s.t. } x \in \Omega$$

In Eq. 2, the decision vector x belongs to the feasible region Ω . The objective vector F translates x in the objective space using m objective functions. The decision vector x for a container packaging problem with N elements is typically represented by two binary matrices such as those given in Eqs. 3 and 4.

$$x_{ij} \in \{0, 1\}, \forall i \in \{1, \dots, \overline{M}\}, j \in \{1, \dots, N\} \quad (3)$$

$$y_i \in \{0, 1\}, \forall i \in \{1, \dots, \overline{M}\} \quad (4)$$

x represents the load matrix, and y is the vector of open containers. The index i is the counter of the containers, and j is the counter of the objects. The variable $\overline{M}$ represents the theoretical maximum number of containers, so x is usually an $N \times N$ matrix, and y is a matrix of length N . The best solutions in Ω involve trade-offs between competing targets. Together, these solutions are called the Pareto set, and their image in the target space is the Pareto front.

Multi-objective evolutionary algorithms (MOEA) have been the most popular method for solving multi-objective optimization problems given their ability to find Pareto-front approximations [41].

Consequently, for multi-objective bin packing problems, approximate algorithm heuristics have been combined with MOEA, such as the multi-objective evolutionary optimization of swarms of particles (MOEPSO) [26] or the non-dominated genetic classification algorithm II (NSGA-II) [14].

2.3 *Dynamic Bin Packing*

The dynamic bin packing problem (DBP) was defined and analyzed by Coffman et al. [13]. This problem is a generalization of the classic problem of packing objects [4]. DBP has been used widely to model problems of consolidation of resources in cloud computing and is defined by [11] in the following way:

Objects with sizes that are real numbers in $(0, 1)$ arrive in time, reside for a period, and leave at an arbitrary time. Thus, an item's size and arrival time are only known when the item arrives, and the departure time is only known when the item leaves.

An infinite supply of unit-size containers is also provided. Each item must be assigned to a container from the moment it arrives until it leaves so that the total size of the articles in a container does not exceed the container's capacity at any time. Migration of items between containers is not allowed; however, reorganization of items within a container is allowed. The goal is to minimize the maximum number of containers always used.

Recent models formulation of dynamic packing includes dynamic demand with capacities varied over time [36], dynamic capacity allocation [23], and dynamic storing and cooling of objects [37].

3 Related Works

This section presents some related works to bin packing and its variants to observe the existing works in the literatura (Table 1).

We hope that the papers presented above can serve as a reference guide for researchers to explore and develop more robust state-of-the-art metaheuristic algorithms to solve emerging variants of bin packing problems.

Next, we present a study focused on the 1D-BPP and an improvement of the GGA-CGT algorithm. There are different algorithms in the current state-of-the-art, mainly heuristic for solving the problem. For this work, those that reported the best results were selected as case studies.

Table 1 Some works of BPP in the literature

Work	Type	Objectives	Restrictions	Strategies
A greedy memetic algorithm for a multi-objective Dynamic bin Packing problem for storing cooling objects [37]	Multi-objective	3	4	NSGA-II and GRASP
On solving multi-objective bin packing problems using evolutionary particle swarm optimization [24]	Multi-objective	3	3	MOEAs and PSO
Dynamic multidimensional bin packing (Epstein and Levy 2010)	Dynamic	1	2	A modified version of NFDH (Next-Fit Decreasing Heuristic) for 2D-BPP
Online Dynamic Bin Packing [11]	Dynamic	1	3	Study the environment in which the elements are oriented. Study the design and analysis of algorithms for particular input types
A Dynamic Multiarmed Bandit-Gene Expression Programming Hyper-Heuristic for Combinatorial Optimization Problems [31]	Dynamic	1	2	Uses a reward based on multiple dynamic bandits extreme values as an online heuristic selection mechanism to select the appropriate heuristic to be applied at each iteration
More than bin packing: Dynamic resource allocation strategies in cloud data centers [39]	Dynamic	1	2	Virtual Machines, Location Controllers, Reassignment Controllers

4 Case Study: State-of-the-Art Solution Methods for 1D-BPP

4.1 Grouping Genetic Algorithm with Controlled Gene Transmission

The Grouping Genetic Algorithm with Controlled Gene Transmission (GGA-CGT) is proposed by [30] to solve BPP. It is one of the best algorithms found in the state-of-the-art to solve this problem; it focuses on the transmission of the best genes on the chromosomes, keeping a balance between selective pressure and diversity in the population, to favor the generation and evolution of high-quality solutions. This algorithm was executed once with the parameters that the author has established. See the procedure in Algorithm 1.

Algorithm 1: GGA-CGT

```

1  Generate an initial population P with FF-ñ
2  while generation < max_gen and Size(best_solution > L2) do
3    Select n individuals to cross using Controlled_Selection
4    Apply Gene_Level_Crossover + FFD to the nc selected individuals
5    Apply Controlled_Replacement to introduce progeny
6    Select nm individuals and elite clone solutions using Controlled_Selection
7    Apply Adaptive_Mutation + RP to the best nm individuals
8    Apply Controlled_Replacement to introduce clones
9    Update the global_best_solution

```

4.2 Consistent Neighborhood Search for One-Dimensional Bin Packing

The Consistent Neighborhood Search for one-dimensional Bin Packing (CNS-BP) algorithm is a metaheuristic proposed by Buljubašić and Vasquez [10] that consists of performing a local search to derive a feasible solution with a given number of containers, m , starting from $m = UB - 1$, where UB is an upper bound obtained by using a variant of the classical First Fit heuristic. It uses a shortening to simplify the problem and then applies two local searches: Tabu Search and Hill Climbing Descent. This algorithm shows successful results in its experimentations. It obtains solutions equal to or better than the other algorithms in the state-of-the-art, even the Hard28 instances that are considered one of the most difficult within BPP.

Algorithm 2: CNS_BP

```

1 Remove the pair of objects (i,j) such that  $w_i + w_j = C$ 
2 Calculate the lower bound  $LB$ 
3 Randomly shuffle the set of items
4  $S \leftarrow$  Complete solution obtained by First Fit
5  $m \leftarrow$  Number of bins in  $S$ 
6 while  $m > LB$  and the time limit does not exceed, do
7    $m \leftarrow m - 1$ 
8   Construct the partial solution  $P$  with  $m - 2$  containers
9    $S' \leftarrow CNS(P)$ 
10   If solution  $S'$  is not complete, then it ends
11    $S \leftarrow S'$ 
12   returns  $S$ 
13 end while

```

5 Design of GGA-CGT-II as an Extension of GGA-CGT

This section provides a general description of the methods developed to redesign the GGA-CGT algorithm. Later, each of the methods implemented in the new version of the algorithm is presented in detail.

5.1 Design of Strategies and Analysis of Their Impact on Performance

Some of the most representative strategies were identified in the algorithms selected as a case study with the literature review. Then, these strategies were taken as an example to implement them in one of the algorithms (GGA-CGT) and see their impact on performance in isolation and later together. Table 3 lists these strategies and their detailed description. The first column is the selected strategy, and in the second column is its description and reference to the algorithm that was initially used (Table 2).

As mentioned previously, each of the selected strategies was developed in the GGA-CGT algorithm. Furthermore, each method was implemented in isolation; an independent version of the algorithm was created for each method, generating two different versions (Limits and reduction) to measure each developed method's impact on the performance. Finally, the robust algorithm was created after conducting the corresponding experiments, including the two selected methods. Experiments and results are detailed in Sect. 5.

Table 2 Representative strategies selected for their implementation in the GGA-CGT algorithm

Strategies	Description
Method to calculate different lower bounds	From the CNS_BP algorithm [10], the calculation of some lower limits was implemented, and the best one was selected for each instance. $L_1 = \lceil \frac{\sum_{i=1}^n w_i}{C} \rceil$ [33], $L_*^{(p)}$ [18] con $p = 10$, L_2 , L_3 [27] y $L_\emptyset$ [1]
Instance reduction method to simplify the problem	From each instance, those pairs of objects (i, j) that fill the container's capacity were removed. $(w_i + w_j = C)$ [10]

Table 3 Results obtained by the original GGA-CGT algorithm the limit version and the best two state-of-the-art for the BPPLIB* instances

Class	Inst	CNS_BP		GGA-CGT (original)		GGA-CGT (limits version)		
		Opt	T(s)	Opt	T(s)	Opt	T(s)	% Imp
Uniform	80	80	0.06	80	1.87	80	1.21	0
Triplets	80	80	0.09	80	2.87	80	1.65	0
Data set 1	720	720	0.08	720	18.61	720	1.84	0
Data set 2	480	480	0.04	480	6.96	480	0.72	0
Data set 3	10	10	0.01	10	45.18	10	7.64	0
Was 1	100	100	0.00	100	0.07	100	0.04	0
Was 2	100	100	0.02	100	14.46	100	8.27	0
Gau 1	17	17	2.62	16	4.19	17	0.99	5.88
Hard28	28	20	6.34	16	23.20	17	12.23	3.57
Total	1615	1607	9.26	1602	117.40	1604	34.60	0.12
Grand total			1753		1900		1323	

5.2 Method to Calculate Different Lower Bounds

As is known, the lower limit of a BPP instance indicates the number of containers that should at least be used to place the n objects of the instance [29]. Of course, it would be beneficial if the lower bound matches the optimal number of containers to use, so getting a lower bound as close to optimal is essential. Thus, a solution obtained from BPP when applying a heuristic can be optimal if it coincides with the value of any of the lower limits.

Researchers have proposed various methods for calculating the lower bound of an instance, some of them closer to the number of containers of the optimal solution than others.

This work implemented a method used in the CNS_BP algorithm [10], which calculates different lower limits. The lower bound used for each of the instances was the best of the following five:

$L_1 = \lceil \frac{\sum_{i=1}^n w_i}{C} \rceil$ proposed by [33], $L_*^{(p)}$ [18] with $p = 10$, L_2 , L_3 [24], and L_ϑ [1]. It is worth mentioning that four of the limits, except L_2 , were initially implemented in CNS_BP. Algorithm 3 presents the method used in this proposal.

Algorithm 3: Method to calculate different lower bounds

Procedure LB

```

1  calculate  $L_1, L_*^{(p)}, L_2, L_3, L_\vartheta$ 
2  if  $L_1 > L_2$  then  $lb \leftarrow L_1$  else if  $lb \leftarrow L_2$  end if;
3  if  $L_*^{(p)} > lb$  then  $lb \leftarrow L_*^{(p)}$  else if  $lb$  end if;
4  if  $L_3 > lb$  then  $lb \leftarrow L_3$  else if  $lb$  end if;
5  if  $L_\vartheta > lb$  then  $lb \leftarrow L_\vartheta$  else if  $lb$  end if;
6  return  $lb$ 
end LB
```

This method consists of two stages: first, each lower limit is calculated independently; that is, the corresponding method is carried out to calculate each one of them; second, the LB procedure oversees selecting the best of the limits. First, the call is made to each of the methods that calculate their respective limits (line 1), then L_1 and L_2 are compared, the best one is assigned to the variable lb to continue with the evaluation (line 2). Then, the same process is carried out with the limits pending evaluation; the best one will gradually replace the previous one saved in lb (line 3–5). Upon completion, the process will return lb , which will contain the best-calculated limit (line 6–7).

5.3 Instance Reduction Method to Simplify the Problem

Over the years, optimization problem researchers have developed various reduction methods, some more complex and sophisticated than others. These methods have been of great help since, by simplifying a problem, you can work by focusing on one part of it.

The version of the algorithm proposed here uses a simple reduction procedure used in CNS_BP [10]. This procedure consists of setting the assignments of all pairs of objects that can fill a container. More precisely, once a set of pairs of elements (i, j) such that $(w_i + w_j = C)$ is identified, the problem can be reduced by eliminating those elements (or establishing their assignments). This same reduction has been used in various articles on BPP. Figure 1 presents an example of how the reduction method works.

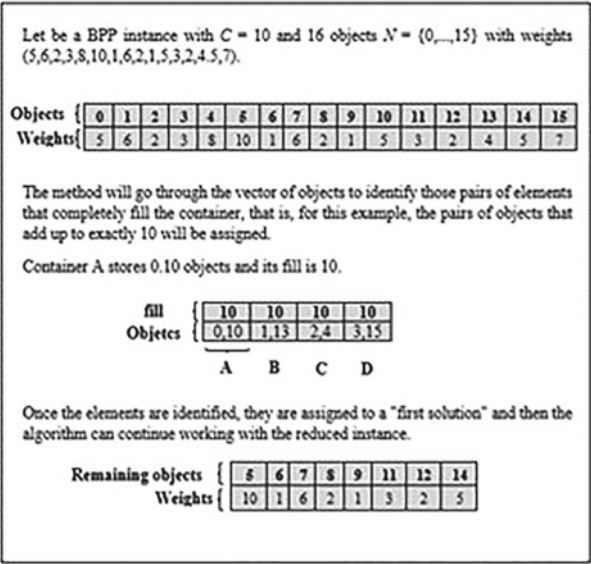


Fig. 1 Example of how the instance reduction method works to simplify the problem

6 Experimentations and Results with GGA-CGT-II

This section presents the experiments carried out to observe the impact on the performance of the methods implemented in the GGA-CGT algorithm first independently and later together to compare their performance against the best approaches in the literature. An analysis with statistical support is also presented to evaluate the proposed version.

The experiments presented here were carried out in the institution's Cluster, a CentOS release 6.7 operating system, GCC version 5.1.0, and an Intel Xeon E5-2650 2.30 GHz processor. The language used for the compilation of the program is C++, and the instances used for its evaluation were the set BPPLIB* (Uniform, Triplet, Data Set 1, 2, and 3, Was 1 and 2, Gau 1, Hard28). It is also pertinent to mention that, for each experiment, a single run per configuration was carried out.

6.1 Instances for 1D-Bin Packing

In the literature, there are various instances for the Bin Packing problem that different researchers use to compare the qualities of their strategies against those of other state-of-the-art algorithms. These instances make up a set of 1615 standard cases recognized by the scientific community; however, over the years and the work carried

out, three other groups of instances have been proposed, adding up to a little more than 6000 test cases. Such instances can be found on recognized Internet sites (Unibo).

For this work, a subset of 1615 instances was selected as test cases, which we call BPPLIB* because it is a portion of the BPPLIB set [16]. The instances considered have been divided into four groups, taking into account their origin and the place from which they were obtained: Uniform [7], Triplets [7], Data Set 1, 2 and 3 [22], Was 1 [34], Was 2 [35], Gau 1 [40] and Hard28 [8].

6.2 Experiment 1: Calculation of Different Lower Bounds

The results obtained for the “limits” version of the algorithm with the set of BPPLIB* instances are presented below. Table 3 shows for each class of instances, the number of instances and, for each solution approach, the number of instances in which the algorithm obtains an optimal solution (Column Opt.) and the average execution time, measured in seconds. (Column T). The last column (% imp) shows how much the proposed version improved the original algorithm in the number of optimal found.

The previous table shows that the “limits” version of the algorithm manages to solve all the Gau_1 and 17 instances of the Hard28 family, improving by 0.12% concerning the original algorithm; however, the method implemented independently follows falling below of CNS_BP algorithm.

6.3 Experiment 2: The Proposed Methods Together (GGA-CGT-II)

To investigate the efficacy of the proposed version GGA-CGT-II, the results obtained were compared with those obtained by the best approaches in the literature: CNS_BP [10] and the original algorithm of GGA-CGT [30] for the BPPLIB* instances. Furthermore, a single run was carried out for each of the experiments presented here; that is, the instances were evaluated only once per algorithm.

Table 4 shows, for each class of instances, the number of instances and, for each solution approach, the number of instances in which the algorithm obtains an optimal solution (Column Opt.) and the average execution time, measured in seconds (Column T). Finally, the last column (% imp) shows how much the proposed version improved the original algorithm in the number of optimal found.

From Table 4, it can be observed the superiority of the proposed version GGA-CGT-II over the class of Hard28 instances, which contains the most difficult instances for the known heuristic algorithms, positioning itself as the best metaheuristic algorithm, since it manages to surpass the results obtained by CNS_BP. It is also observed

Table 4 Results were obtained using the original GGA-CGT algorithm, the reduction version, and the two best of the state-of-the-art BPPLIB* instances

Class	Inst	CNS_BP		GGA-CGT (original)		GGA-CGT (limits version)		
		Opt	T(s)	Opt	T(s)	Opt	T(s)	% Imp
Uniform	80	80	0.06	80	1.87	80	2.35	0
Triplets	80	80	0.09	80	2.87	80	3.44	0
Data set 1	720	720	0.08	720	18.61	720	18.85	0
Data set 2	480	480	0.04	480	6.96	480	7.13	0
Data set 3	10	10	0.01	10	45.18	10	47.25	0
Was 1	100	100	0.00	100	0.07	100	0.19	0
Was 2	100	100	0.02	100	14.46	100	16.14	0
Gau 1	17	17	2.62	16	4.19	17	4.73	5.88
Hard28	28	20	6.34	16	23.20	28	22.1	42.86
Total	1615	1607	9.26	1602	117.40	1615	123.27	0.81
Grand total			1753.00		1900.00		2085.00	

that the proposed work improves by 5.89% the performance of the original GGA-CGT algorithm for the Gau1 instances and 42.86% for the Hard28 instances. Furthermore, improving by a total of 0.81% the algorithm's performance for the subset of BPPLIB* instances.

7 Conclusions and Future Work

According to the experimental results, the following was obtained:

- (1) GGA-CGT-II considerably improves the number of instances resolved optimally; it manages to fully resolve the subset of BPP test instances (1615 instances).
- (2) It improves the performance of the original algorithm (GGA-CGT) and the CNS_BP heuristic by 0.81% and 0.5%, respectively.

With this work, it was possible to identify some promising lines of research and works that could be addressed in the future:

One-Dimensional Bin Packing Problem:

- (1) Integration of construction and destruction processes.
- (2) Development of a hyper-heuristic to select between different metaheuristics.
- (3) Integration of different fitness functions.
- (4) Process of characterization of instances in the search process of metaheuristic algorithms.

Multiobjective Bin Packing Problem:

- (1) Review hybrid versions of nature-inspired metaheuristic algorithms that have been used to solve BPP and other variants that involve additional constraints.
- (2) A comparative performance analysis study of the state-of-the-art implementation of different nature-inspired metaheuristic algorithms to identify the best possible methods for handling multi-objective BPP.
- (3) Implement a more natural way to code solutions, using a representation scheme based on groups and variation operators appropriate to the multi-objective BPP.
- (4) Selection between different decoding heuristics to fill in the gaps in a partial design of the problem.
- (5) Development of an update mechanism to know when an operator improves the fitness and diversity values as the calculation evolves. This mechanism could be integrated with the selection of the crossover operator to provide better control of exploration and exploitation in the algorithm.

Dynamic Bin Packing Problem:

- (1) A comparative performance analysis study of the state-of-the-art implementation of different nature-inspired metaheuristic algorithms to identify the best possible methods for handling DBP.
- (2) An in-depth investigation of the limitations of the DBP problem in which each item can be assigned to only a subset of containers to satisfy the given constraints.
- (3) Classify/characterize objects and analyze each class to explore and exploit their properties [11].
- (4) Build new DBP models and generate test instances for this problem. Unfortunately, there are few mathematical formulations of DBPP, and there are no test instances available in the literature.

References

1. Adriana, A.C.F., Ribeiro, C.C., Fred, G., Aloise, D.J.: A hybrid improvement heuristic for the one-dimensional bin packing problem. *J. Heuristics* **10**(March), 205–229 (2004)
2. Arbib, C., Marinelli, F., Pizzuti, A.: A number of bins and maximum lateness minimization in two-dimensional bin packing. *Eur. J. Oper. Res.* **291**(1), 101–113 (2021)
3. Aydın, N., Muter, İ., Birbil, Şİ: Multi-objective temporal bin packing problem: an application in cloud computing. *Comput. Oper. Res.* **121**, 104959 (2020)
4. Balogh, J., Békési, J.: Semi-on-line bin packing: a short overview and a new lower bound. *CEJOR* **21**(4), 685–698 (2013)
5. Bansal, N., Correa, J.R., Kenyon, C., Sviridenko, M.: Bin packing in multiple dimensions: inapproximability results and approximation schemes. *Math. Oper. Res.* **31**(1), 31–49 (2006)
6. Basse, S.: Introduction to Design and Analysis. Editorial Addison-Wesley Publishing Company, Computer Algorithms (1998)
7. Beasley, J.E.: OR-library: distributing test problems by electronic mail. *J. Oper. Res. Soc.* **41**, 1069–1072 (1990). Disponible en internet vía <http://people.brunel.ac.uk/~mastjjb/jeb/orlib/binpackinfo.html>

8. Belov, G.: Problems, “Models and Algorithms in One and Two Dimensional Cutting”. Ph.D. thesis. Fakultät Mathematik und Naturwissenschaften der Technischen Universität Dresden (2004)
9. Bortfeldt, A.: A genetic algorithm for the two-dimensional strip packing problem with rectangular pieces. *Eur. J. Oper. Res.* **172**, 814–837 (2006)
10. Buljubašić, M., Vasquez, M.: Consistent neighborhood search for one-dimensional bin packing and two-dimensional vector packing. *Comput. Oper. Res.* **76**, 12–21 (2016)
11. Burcea, M.: Online dynamic bin packing. Doctoral dissertation, University of Liverpool (2014)
12. Cid-Garcia, N.M., Rios-Solis, Y.A.: Positions and covering: a two-stage methodology to obtain optimal solutions for the 2d-bin packing problem. *PLoS One* **15**(4), e0229358 (2020)
13. Coffmann, E.G., Garey, M.R., Johnson, D.S.: Dynamic bin packing. *SIAM J. Comput.* **12**:227–260 (1983). (Coppersmith D)
14. Dahmani, N., Clautiaux, F., Krichen, S., Talbi, E. G.: Self-adaptive metaheuristics for solving a multi-objective 2-dimensional vector packing problem. *Applied Soft Computing*, **16**, 124–136 (2014)
15. Dell’Amico, M., Furini, F., Iori, M.: A branch-and-price algorithm for the temporal bin packing problem. *Comput. Oper. Res.* **114**, 104825 (2020)
16. Delorme, M., Iori, M., Martello, S.: BPPLIB: a library for bin packing and cutting stock problems. *Optim. Lett.* **12**, 235–250 (2018)
17. Falkenauer, E.: A Hybrid Grouping Genetic Algorithm for Bin Packing. *J. Heuristics* 5–30 (1996)
18. Fekete, S. P., Schepers, J.: New classes of fast lower bounds for bin packing problems. *Mathematical programming*, **91**(1), 11–31 (2001)
19. Garey, M.R., Johnson, D.S.: Computers and Intractability, vol. 174. Freeman, San Francisco (1979)
20. Griffiths, V., Scanlan, J.P., Eres, M.H., Martinez-Sykora, A., Chinchapatnam, P.: Cost-driven build orientation and bin packing of parts in selective laser melting (SLM). *Eur. J. Oper. Res.* **273**(1), 334–352 (2019)
21. Iori, M., de Lima, V.L., Martello, S., Miyazawa, F.K., Monaci, M.: Exact solution techniques for two-dimensional cutting and packing. *Eur. J. Oper. Res.* **289**(2), 399–415 (2021)
22. Klein, R., Scholl, A.: Bin packing. Disponible en internet via <http://www.wiwi.uni-jena.de/Entscheidung/binpp/>
23. Levin, M.S.: Towards bin packing (preliminary problem survey, models with multiset estimates). [arXiv:1605.07574](https://arxiv.org/abs/1605.07574) (2016)
24. Liu, D.S., Tan, K.C., Huang, S.Y., Goh, C.K., Ho, W.K.: On solving multi-objective bin packing problems using evolutionary particle swarm optimization. *Eur. J. Oper. Res.* **190**(2), 357–382 (2008)
25. Lodi, A., Martello, S., Vigo, D.: Heuristic algorithms for the three-dimensional bin packing problem. *Eur. J. Oper. Res.* **141**, 410–420 (2002)
26. Maclean, K.D., Ødegaard, F.: Dynamic capacity allocation for group bookings in live entertainment. *Eur. J. Oper. Res.* **287**(3), 975–988 (2020)
27. Martello, S., Toth, P.: Lower bounds and reduction procedures for the bin packing problem. *Discret. Appl. Math.* **28**(1), 59–70 (1990)
28. Polyakovskiy, S., M’Hallah, R.: A hybrid feasibility constraints-guided search to the two-dimensional bin packing problem with due dates. *Eur. J. Oper. Res.* **266**(3), 819–839 (2018)
29. Pérez-Ortega, J., Castillo-Zacatelco, H., Vilarino-Ayala, D., Mexicano-Santoyo, A., Zavala-Díaz, J.C., Martínez-Rebollar, A., Estrada-Esquivel, H.: A new heuristic strategy for the bin packing problem. *Ing. Invest. y Tecnol.* **17**(2), (2016)
30. Quiroz-Castellanos, M., Cruz-Reyes, L., Torres-Jimenez, J., Gómez, C., Huacuja, H.J.F., Alvim, A.C.: A grouping genetic algorithm with controlled gene transmission for the bin packing problem. *Comput. Oper. Res.* **55**, 52–64 (2015)
31. Sabar, N.R., Ayob, M., Kendall, G., Qu, R.: A dynamic multiarmed bandit-gene expression programming hyper-heuristic for combinatorial optimization problems. *IEEE Trans. Cybern.* **45**(2), 217–228 (2014)

32. Sathe, M., Schenk, O., Burkhart, H.: Solving Bi-objective Many-Constraint Bin Packing Problems in Automobile Sheet Metal Forming Processes, pp. 246–260. Springer, Berlin (2009)
33. Scholl, A., Klein, R., Jurgens, C.: Bison: A fast hybrid procedure for exactly solving the one-dimensional bin packing problem. *Comput. Oper. Res.* **24**(7), 627–645 (1997)
34. Schwerin, P., Wäscher, G.: The bin-packing problem: a problem generator and some numerical experiments with FFD packing and MTP. *Int. Trans. Oper. Res.* **4**, 337–389 (1997)
35. Schwerin, P., Wäscher, G.: A new lower bound for the bin-packing problem and its integration to MTP. *Pesqui. Oper.* **19**, 111–129 (1999)
36. Silva, A., Aloise, D., Coelho, L.C., Rocha, C.: Heuristics for the dynamic facility location problem with modular capacities. *Eur. J. Oper. Res.* **290**(2), 435–452 (2021)
37. Spencer, K.Y., Tsvetkov, P.V., Jarrell, J.J.: A greedy memetic algorithm for a multi-objective dynamic bin packing problem for storing cooling objects. *J. Heuristics* **25**(1), 1–45 (2019)
38. Srichandan, S., Kumar, T.A., Bibhudatta, S.: Task scheduling for cloud computing using multi-objective hybrid bacteria foraging algorithm. *Future Comput. Inform. J.* **3**(2), 210–230 (2018)
39. Wolke, A., Tsend-Ayush, B., Pfeiffer, C., Bichler, M.: More than bin packing: dynamic resource allocation strategies in cloud data centers. *Inf. Syst.* **52**, 83–89 (2015)
40. Wäscher G., Gau T.: Heuristics for the one-dimensional cutting stock problem: a computational study *OR Spektrum*, **18**, 131–144, (1996)
41. Zhou, A., Qu, B.Y., Li, H., Zhao, S.Z., Suganthan, P.N., Zhang, Q.: Multiobjective evolutionary algorithms: a survey of the state of the art. *Swarm Evolut. Comput.* **1**(1), 32–49 (2011)

CMA Evolution Strategy Applied to Optimize Chemical Molecular Clusters $MxNz$ ($x + y \leq 5$; $M = N$ or $M \leq N$)



J. M. Pérez-Rocha, Andrés Espinal, Erik Díaz-Cervantes,
J. A. Soria-Alcaraz, M. A. García-Revilla, and M. A. Sotelo-Figueroa

Abstract The chemical molecular clusters generation is widely used to find new chemical components. The goal is to find a stable geometry that has the lowest energy on the Potential Energy Surface (PES). Has already been shown that PES is an NP-hard problem and have been used some metaheuristics to try to obtain good results. The PES of a molecular cluster is calculated using the Density Functional Theory (DFT), there exist many ways to calculate it, but the Chemical researchers prefer to use programs how have been widely tested. Covariance Matrix Adaptation Evolutionary Strategy (CMA-ES) is an evolutionary algorithm with real representation that uses few parameters. The present paper proposed a CMA-ES running over High-Performance Computing to search geometries for molecular clusters using the Gaussian 09 package who uses the DFT to calculate the PES of the cluster, the results were contrasted with the obtained by a Micro Evolutionary Algorithm running over one single node.

J. M. Pérez-Rocha · A. Espinal · J. A. Soria-Alcaraz · M. A. Sotelo-Figueroa (✉)
Departamento de Estudios Organizacionales, División de Ciencias Económico Administrativas,
Universidad de Guanajuato, Fraccionamiento 1, Col. El Establo S/N, C.P. 36250 Guanajuato, Gto,
México
e-mail: masotelo@ugto.mx

J. M. Pérez-Rocha
e-mail: jm.perez.rocha@ugto.mx

A. Espinal
e-mail: aespinal@ugto.mx

J. A. Soria-Alcaraz
e-mail: jorge.soria@ugto.mx

E. Díaz-Cervantes
Departamento de Alimentos, Centro Interdisciplinario del Noreste (CINUG), Universidad de
Guanajuato, 37975 Tierra Blanca, Guanajuato, México
e-mail: e.diaz@ugto.mx

M. A. García-Revilla
Departamento de Química, División de Ciencias Naturales Y Exactas, Universidad de
Guanajuato, Noria Alta S/N, C.P. 36050 Guanajuato, Gto, México
e-mail: magarcia@ugto.mx

Keywords PES · CMA-ES · Chemical molecular cluster · DFT

1 Introduction

The molecular cluster generation has been widely studied due to the applications that can be given in a number of fields [1–4], the objective is to find the lowest energy on the Potential Energy Surface (PES) [5].

It has been shown that the PES is an NP-Problem Complexity [5, 6] and for this reason is necessary to use techniques to approximate a solution in a reasonable time.

There exists at least two ways to calculate the PES, with Molecular Dynamics (MD) [7] or Density Functional Theory (DFT) [8].

In [7] algorithms review using MD is made, is shown the Simulated Annealing (SA) [9], Genetic Algorithm (GA) [10] and Basin-hopping Algorithm (BHP) [11] are been used to the molecular cluster generation.

In [8] algorithms review using DFT is made, most of them are based on population candidate solutions [12, 13]. Those algorithms are executed using High-Performance Computing (HPC) [14–16] using any standard quantum-chemistry software to compute the DFT on the candidate solution such as Gaussian [17], Quantum Espresso [18], or Siesta [19]. The algorithms used are GA [10], Cellular Automata (CA) [20], Adaptive-Learning Global Optimization Algorithm (ALGOA) [21], Grammatical Evolution (GE) [22].

Both MD and DFT was used as fitness function into the algorithms to make the optimization process.

To obtain better results in [4] was proposed an evolutionary framework based on Micro-differential Evolution [23] to avoid the parameter tuning [24] and the HPC. It also was proposed to reduce the search space assigning the first atom to the point (0, 0, 0) in the Cartesian Space.

The Covariance Matrix Adaptation Evolutionary Strategy (CMA-ES) [25] is an evolutive algorithm based on population used to optimize values in the continuous space. One advantage of the CMA-ES is that the parameter tuning is obviated due the parameters are calculated based on the dimensions of the problem and includes a way to determinate the algorithm convergence.

In the present work, an evolutionary-based framework based on Evolutionary Strategy CMA is proposed to find the equilibrium structure of some molecular clusters. The proposed framework is based on the 3-dimensional space to locate each atom and uses an HPC based on queues to calculate quickly the energy of each isomers using the DFT through Gaussian Software. The results are compared graphically with the obtained by MiDECO [4] implementation because the execution of each instance takes several days to be obtained.

2 Potential Energy Surface

The way to determine the equilibrium in an N-atom cluster is using the Potential Energy Surface (PES) [5], looking for an N-atom cluster with the lowest Energy.

This problem has been proved that is an NP problem, there not exists an algorithm that in polynomial time find the solution [6]. There was proposed a way to estimate the number of local minima in the PES of an N-atom cluster using $n \propto e\alpha N$ where α is a constant based on the system [26, 27].

Using the potential between atoms pair, the PES from an N-atom cluster can be defined as follows:

$$V(r^N) = \frac{1}{2} \sum_{i \neq j}^N (r_i - r_j)$$

This function calculates the PES of all possible vectors $r^N = (r_1, r_2, \dots, r_N)$ in a 3 N-dimensional space.

Is possible to calculate the PES of an N-atom cluster using chemical software applying DFT like Gaussian.

3 Covariance Matrix Adaptation—Evolution Strategy

The Covariance Matrix Adaptation - Evolution Strategy (CMA-ES) [25, 28, 29] is an evolutive algorithm working over the continues spaces and is based-on population.

The Covariance Matrix Adaptation - Evolution Strategy (CMA-ES) [25, 28, 29] is an evolutive algorithm working over the continues spaces and is based-on population. The CMA-ES is based on the Evolutive Strategy (μ, λ) [30] where μ parents produces λ sons, in this case both parameters are calculate by the Algorithm and is not necessary to made an parameter tuning to find them.

The evolutive process is made by a Covariance Matrix Adaptation that represents a multivariate normal distribution. The algorithm is shown in Algorithm 1. The CMA is updated using the Rank- μ -update based-on the μ best sons taken from the λ sons produced.

Algorithm 1 Covariance Matrix Adaptation Algorithm

```

Initialize  $m \in \mathbb{R}^n$ ,  $\sigma \in \mathbb{R}^+$ ,  $\lambda$ ,  $\mu$ 
Initialize  $C = I$ ,  $p_c = 0$ ,  $p_\sigma = 0$ 
while not terminate do
  Sampling  $\theta_i = m + \sigma y_i$ ,  $y_i \sim \mathcal{N}(0, C)$ ,  $i = 1, \dots, \lambda$ 
  Evaluating  $f(\theta_i)$ ,  $i = 1, \dots, \lambda$ 
   $m \leftarrow m + \sigma \bar{y}$ , where  $\bar{y} = \sum_1^\mu w_i y_{i:\lambda}$ 
   $p_\sigma \leftarrow (1 - c_\sigma) p_\sigma + \sqrt{c_\sigma (2 - c_\sigma)} \mu_w C^{\frac{1}{2}} \bar{y}$ 
   $\sigma \leftarrow \sigma \exp\left(\frac{c_\sigma}{d_\sigma} \left(\frac{\|p_\sigma\|}{\varepsilon \|\mathcal{N}(0, I)\|} - 1\right)\right)$ 
   $p_c \leftarrow (1 - c_c) p_c + \sqrt{c_c (2 - c_c)} \mu_w \bar{y}$ 
   $C \leftarrow (1 - c_1 - c_\mu) C + c_1 p_c p_c^T + c_\mu \sum_1^\mu w_i y_{i:\lambda} y_{i:\lambda}^T$ 
end while

```

4 Experiments

The follow test instances was taken from [4], there includes homo and hetero-atomic clusters with some cationic, anionic, and neutral species:

- *B4*
- *B4—anion*
- *B5*
- *B5—anion*
- *B5—cation*
- *GamAsn*($m + n = 3$)
- *GamAsn*($m + n = 4$)
- *GamAsn*($m + n = 5$)

To evaluate the proposed solution is necessary to the follow:

1. Is necessary to convert the proposed solution as an Gaussian Input File, telling what is the optimization level to be used and the position of each atom in the Cartesian Space, every 3 values are the position of one atom and the first atom always be in the point (0, 0, 0).
2. The program Gaussian is executed until the program finish successfully or with an error. This process is executed in parallel using the process queue and is distributed by the HPC among the nodes.
3. The log file is analyzed to obtain the energy, if Gaussian finish with an error is considered that the energy es infinity.
4. The energy is returned to the CMA-ES proposed solution to continue the evolutive process.

The proposed framework was implemented in Python 3 and was ran under a HPC Cluster with nodes Intel Xeon E5, 3.5 GHz with 32 GB RAM and Debian/Linux operating system and was used 256 cores.

5 Results

The results obtained by the CMA-ES are equivalent with the obtained in [4], most of them are the state-of-art molecular clusters.

Figure 1 shows some homo-atomic molecules with 4 atoms in their anionic and neutral form.

Figure 2 shows pure cluster molecules with 5 atoms, showing that is possible to obtain with the CMA-ES the minimal structures in their anionic, cationic and neutral form.

In the case of the hybrid clusters tested, the *GamAsn* with $m + n = 3$, $m + n = 4$, and $m + n = 5$ are basically identical to the reported in the state-of-art. Figures 3, 4, and 5 shows the Geometries obtained.

Fig. 1 Geometries for B_4 clusters obtained using CMA-ES

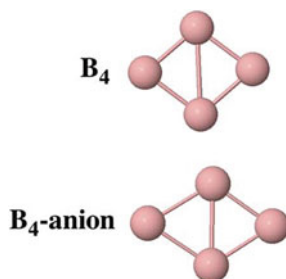


Fig. 2 Geometries for B_5 clusters obtained using CMA-ES

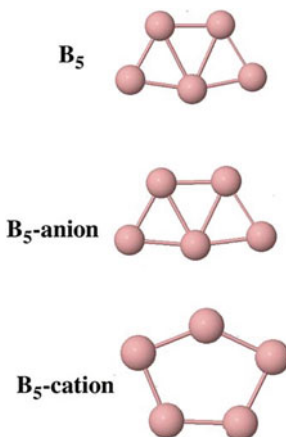


Fig. 3 Geometries for *Gam*
Asn ($m + n = 3$) clusters
obtained using CMA-ES

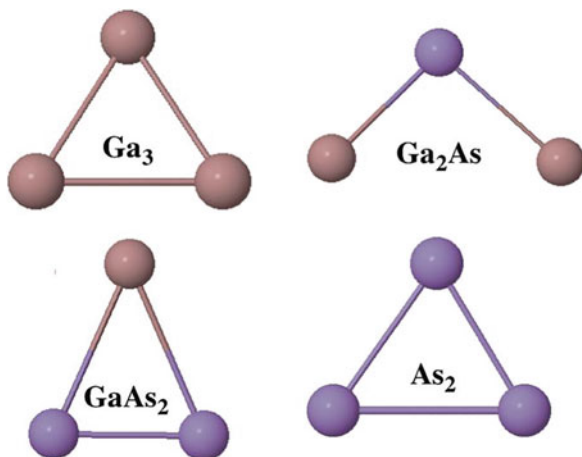


Fig. 4 Geometries for *Gam*
Asn ($m + n = 4$) clusters
obtained using CMA-ES

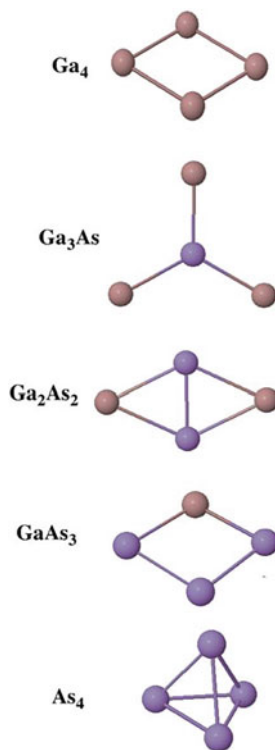
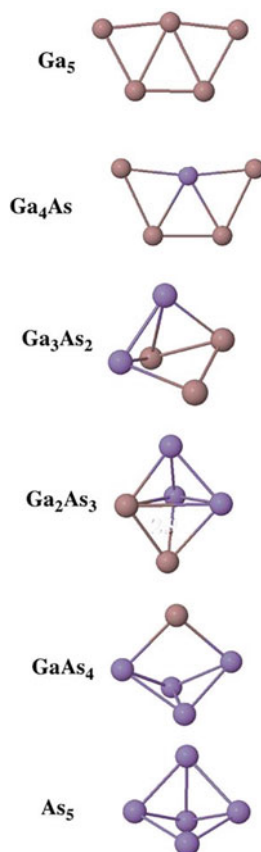


Fig. 5 Geometries for *Gam*
Asn ($m + n = 5$) clusters
obtained using CMA-ES



6 Conclusions and Future Work

A novel evolutionary framework based on Covariance Matrix Adaptation has been proposed to compute stable cluster conformations using HPC with queue. The proposed framework was tested with sets from three to five atoms in their different forms, the results obtained was the same of the state-of-art without parameter tuning process.

With this framework is possible to test clusters with more 10 atoms to impact the cluster conformation in another fields, this because in some papers are refereed to the 10 atoms is to complex to be conformed the cluster.

Acknowledgements We acknowledge to the Laboratorio Nacional de Caracterización de Propiedades Físicoquímicas y Estructura Molecular (UG-UAA-CONACYT, Project: 123732).

References

1. Darby, S., Mortimer-Jones, T.V., Johnston, R.L., Roberts, C.: Theoretical study of Cu–Au nanoalloy clusters using a genetic algorithm. *J. Chem. Phys.* **116**(4), 1536–1550 (2002)
2. Guzmán-Ramírez, G., Aguilera-Granja, F., Robles, J.: DFT and GEGA genetic algorithm optimized structures of Cu_nV ($v = \pm 1, 0, 2$; $n = 3\text{--}13$) clusters. *Eur. Phys. J. D* **57**(1), 49–60 (2010)
3. Rodríguez-Jiménez, J.A., Díaz-Cervantes, E., Aguilera-Granja, F., Robles, J.: Computational study of GaN_mAs_n ($m + n = 2\text{--}9$) clusters using DFT calculations. *J. Nanoparticle Res.* **21**(11), 219 (2019)
4. Sotelo-Figueroa, M.A., García-Revilla, M.A., Espinal, A., Díaz-Cervantes, E.: Micro-differential evolution cluster-optimizer (MiDECO): an open-access software for the optimization of molecular clusters M_xN_y ($x + y \leq 5$; $M = \text{N}$ or $M \leq \text{N}$). *J. Nanoparticle Res.* **23**(4), 93 (2021)
5. Wille, L.T., Vennik, J.: Computational complexity of the ground-state determination of atomic clusters. *J. Phys. A: Math. Gen.* **18**(8), L419–L422 (1985)
6. Garey, M.R., Johnson, D.S.: *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman & Co. (1990)
7. Rossi, G., Ferrando, R.: Searching for low-energy structures of nanoparticles: a comparison of different methods and algorithms. *J. Phys. Condens. Matter* **21**(8), 084208 (2009)
8. Ferrando, R.: Determining the equilibrium structures of nanoalloys by computational methods. *J. Nanopart. Res.* **20**(7), 179–200 (2018)
9. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. *Science (N. Y.)* **220**(4598), 671–680 (1983)
10. John Henry Holland: *Adaptation in natural and artificial systems*. MIT Press, An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence (1992)
11. Leary, R.H.: Global optimization on funneling landscapes. *J. Global Optim.* **18**(4), 367–383 (2000)
12. Blum, C., Roli, A.: Metaheuristics in combinatorial optimization. *ACM Comput. Surv.* **35**(3), 268–308 (2003)
13. Katoch, S., Chauhan, S.S., Kumar, V.: A review on genetic algorithm: past, present, and future. *Multimed. Tools Appl.* **80**(5), 8091–8126 (2021)
14. Alexandrova, A.N., Boldyrev, A.I.: Search for the $L^{0/1/-1}(n = 57)$ n lowest-energy structures using the Ab initio gradient embedded genetic algorithm (GEGA). Elucidation of the chemical bonding in the lithium clusters. *J. Chem. Theory Comput.* **1**(4), 566–580 (2005)
15. Alexandrova, A.N., Boldyrev, A.I., Zhai, H.-J., Wang, L.-S.: All-boron aromatic clusters as potential new inorganic ligands and building blocks in chemistry. *Coord. Chem. Rev.* **250**(21–22), 2811–2866 (2006)
16. Díaz-Cervantes, E., Poater, J., Robles, J.: Marcel swart unraveling the origin of the relative stabilities of group 14 M_2N_2 $2+$ ($M, N = \text{C}, \text{Si}, \text{Ge}, \text{Sn}, \text{and Pb}$) isomer clusters. *J. Phys. Chem. A* **117**(40), 10462–10469 (2013)
17. Frisch, M.J., Trucks, G.W., Schlegel, H.B., Scuseria, G.E., Robb, M.A., Cheeseman, J.R., Scalmani, G., Barone, V., Mennucci, B., Petersson, G.A., Nakatsuji, H., Caricato, M., Li, X., Hratchian, H.P., Izmaylov, A.F., Bloino, J., Zheng, G., Sonnenberg, J.L., Hada, M., Ehara, M., Toyota, K., Fukuda, R., Hasegawa, J., Ishida, M., Nakajima, T., Honda, Y., Kitao, O., Nakai, H., Vreven, T., Montgomery, Jr., J.A., Peralta, J.E., Ogliaro, F., Bearpark, M., Heyd, J.J., Brothers, E., Kudin, K.N., Staroverov, V.N., Kobayashi, R., Normand, J., Raghavachari, K., Rendell, A., Burant, J.C., Iyengar, S.S., Tomasi, J., Cossi, M., Rega, N., Millam, J.M., Klene, M., Knox, J.E., Cross, J.B., Bakken, V., Adamo, C., Jaramillo, J., Gomperts, R., Stratmann, R.E., Yazyev, O., Austin, A.J., Cammi, R., Pomelli, C., Ochterski, J.W., Martin, R.L., Morokuma, K., Zakrzewski, V.G., Voth, G.A., Salvador, P., Dannenberg, J.J., Dapprich, S., Daniels, A.D., Farkas, Ö., Foresman, J.B., Ortiz, J.V., Cioslowski, J., Fox, D.J.: *Gaussian09 Revision E.01*. Gaussian Inc., Wallingford CT (2009)

18. Giannozzi, P., Andreussi, O., Brumme, T., Bunau, O., Buongiorno Nardelli, M., Calandra, M., Car, R., Cavazzoni, C., Ceresoli, D., Cococcioni, M., Colonna, N., Carnimeo, I., Dal Corso, A., de Gironcoli, S., Delugas, P., DiStasio Jr, R.A., Ferretti, A., Floris, A., Fratesi, G., Fugallo, G., Gebauer, R., Gerstmann, U., Giustino, F., Gorni, T., Jia, J., Kawamura, M., Ko, H.-Y., Kokalj, A., Küçükbenli, E., Lazzeri, M., Marsili, M., Marzari, N., Mauri, F., Nguyen, N.L., Nguyen, H.-V., Otero de-la Roza, A., Paulatto, L., Poncé, S., Rocca, D., Sabatini, R., Santra, B., Schlipf, M., Seitsonen, A.P., Smogunov, A., Timrov, I., Thonhauser, T., Umari, P., Vast, N., Wu, X., Baroni, S.: Advanced capabilities for materials modelling with quantum ESPRESSO. *J. Phys. Condens. Matter* **29**(46), 465901 (2017)
19. García, A., Papior, N., Akhtar, A., Artacho, Blum, V., Bosoni, E., Brandimarte, P., Brandbyge, M., Cerdá, J.I., Corsetti, F., et al.: Siesta: recent developments and applications. *J. Chem. Phys.* **152**(20), 204108 (2020)
20. Yañez, O., Báez-Grez, R., Inostroza, D., Rabanal-León, W.A., Pino-Rios, R., Garza, J., Tiznado, W.: AUTOMATON: a program that combines a probabilistic cellular automata and a genetic algorithm for global minimum search of clusters and molecules. *J. Chem. Theory Comput.* **15**(2), 1463–1475 (2019)
21. Zhang, J., Glezakou, V.-A., Rousseau, R., Nguyen, M.-T.: NW-PEsSe: an adaptive-learning global optimization algorithm for nanosized cluster systems. *J. Chem. Theory Comput.* **16**(6), 3947–3958 (2020)
22. Yoshikawa, N., Terayama, K., Sumita, M., Homma, T., Oono, K., Tsuda, K.: Population-based de novo molecule generation, using grammatical evolution. *Chem. Lett.* **47**(11), 1431–1434 (2018)
23. Salehinejad, H., Rahnamayan, S., Tizhoosh, H.R.: Micro-differential evolution: diversity enhancement and a comparative study. *Appl. Soft Comput.* **52**, 812–833 (2017)
24. Gunawan, A., Lau, H.C., Lindawati.: Fine-tuning algorithm parameters using the design of experiments approach. In: *Grammar Design for Derivation Tree Based Genetic Programming Systems*, pp. 278–292. Springer, Berlin (2011)
25. Hansen, N., Kern, S.: Evaluating the CMA evolution strategy on multimodal test functions. In: Yao, X., et al. (eds.) *Parallel Problem Solving from Nature PPSN VIII*, vol. 3242 of LNCS, pp. 282–291. Springer (2004)
26. Stillinger, F.H.: Exponential multiplicity of inherent structures. *Phys. Rev. E* **59**(1), 48–51 (1999)
27. Stillinger, F.H., Weber, T.A.: Hidden structure in liquids. *Phys. Rev. A* **25**(2), 978–989 (1982)
28. Beyer, H.-G., Sendhoff, B.: Covariance Matrix Adaptation Revisited—The CMA Evolution Strategy—, pp. 123–132 (2008)
29. Hansen, N., Ostermeier, A., Gawelczyk, A.: On the adaptation of arbitrary normal mutation distributions in evolution strategies: the generating set adaptation. In: *6th International Conference on Genetic Algorithms*, Morgan Kaufmann Publishers (editor), pp. 57–64. San Francisco, CA (1995)
30. Beyer, H.-G., Schwefel, H.-P.: Evolution strategies—a comprehensive introduction. *Nat. Comput.* **1**(1), 3–52 (2002)

Specialized Crossover Operator for the Differential Evolution Algorithm Applied to a Car Sequencing Problem with Constraint Smoothing



Javier Manzanares, Elvi Sánchez, Héctor Puga, Manuel Ornelas,
and Martin Carpio

Abstract The Car Sequencing Problem (CSP) proposed by the car manufacturer RENAULT in the ROADEF'2005 challenge, is a variant of Permutation Flow Shop Scheduling (PFFS), in which there is a production line with serial workstations where accessories (options) are installed, and a paint color is applied to a sequence of incoming vehicles. For this problem, the configuration of the accessories to be installed is provided in advance, as well as the paint color to be applied for each vehicle, so you can create groupings (classes) of vehicles that will have the same color and to which the same accessories are installed. The objective is to find the input sequence that minimizes the number of violations of the work constraints imposed on each station and the number of paint tool purges due to a color change. This paper describes a Lexicographic Multi-Objective Differential Evolution algorithm with chromosome repairer applied to CSP that introduces the development of a new crossover operator. Such an operator estimates the classes of vehicles that are more difficult to sequence by considering the demand (utilization rate) that each attachment and color has by the vehicles in the input sequence. The demand is calculated from the work constraints imposed on each station. The proposed algorithm was tested on a set of instances provided by RENAULT. The proposal showed an improvement in the results reported by the company in 61% of the instances. In addition, it was observed that starting a sequence with a vehicle of difficult class and then interleaving one or several of lower difficulty, provides a greater distance between vehicles; decreasing waiting times and smoothing the violations in the work restrictions assigned to each station, therefore, this new operator allows generating individuals with a better balance between their elements in the sequence according to the lexical order established in the objectives.

Keywords Permutation flow shop scheduling · Car sequencing problem · Differential evolution · Crossover operator

J. Manzanares · E. Sánchez · H. Puga · M. Ornelas · M. Carpio (✉)

Division of Postgraduate Studies and Research, National Technology of México, León Institute of Technology, León, Guanajuato, Mexico

e-mail: juanmartin.carpio@leon.tecnm.mx

1 Introduction

The multi-objective car sequencing problem (MCS) was first presented in 2005 during the event organized by the French Society of Operational Research and Decision Support (ROADEF) [1] as a challenge to the scientific community. The MCS introduces the concept of constraint smoothing, in relation to the objectives of minimization of work overloads (violations of ratio constraints) in the assembly line; which are classified as difficult (H) and easy (L), in addition, to introduce a third objective that considers the size of the paint batch, seeking to reduce the consumption of solvents used in the cleaning of the equipment with which the vehicles are painted in the paint shop, the above is achieved by minimizing the color changes (P). To solve the MCS problem, exact methods such as integer programming and ad-hoc methods have been applied. Heuristic greedy methods [2], local search [1], genetic algorithms [3] and iterative tabu search [4] have also been used, to mention a few. The challenge offered a comprehensive view of the proposed methods, and the winning algorithm was implemented in European pilot factories of the RENAULT company.

The problem addressed in this paper poses for its solution, a lexicographic multi-objective approach that assumes a priority order in the set F of objective functions. In our case, the order is given in the MCS instances used. In this sense, RENAULT provided 3 sets of instances identified as X, B and A. The set of instances X and B were used for the preliminary phases of the challenge and to choose the winning team the set A was used. In this work we used the set A of instances and as a reference the results reported by the participants of that contest, which are still valid as a comparison tool in recent works such as [2, 5].

For each workstation in the assembly line, constraints are established. The ratio constraints of the i -th workstation are expressed as N_i/P_i and represent the capacities of each station, where N_i denotes the maximum number of vehicles that can be served by station i , in a block of P_i vehicles. When this ratio is exceeded, overloads occur; the number of overloads corresponds to the number of vehicles that exceed N_i .

In this paper we apply the Differential Evolution (DE) algorithm proposed in [6] called DECR to provide a solution to the MCS problem. The algorithm proposed in this work, which we call DECR-s, incorporates a chromosomal repairer that allows the elimination of non-feasible individuals generated in the random process of the classical mutation and crossover operators, allowing to generate an optimal sequence by selecting the most suitable vehicle based on its position and the number of overloads generated by permuting the positions, a crossover operator is proposed that considers the characteristic information of the MCS problem such as option utilization rate and class difficulty that were applied in a genetic algorithm in [7]. Additionally, the concept of paint color demand was considered, which, has not been contemplated so far in the state of the art, therefore, the solution representation is structured with two chromosomes in which the proposed crossover operator was applied.

The rest of the article is organized as follows: Sect. 2 briefly describes the Differential Evolution with Specialized Chromosome Repairer (DECR-s) algorithm on which the proposed crossover operator works; Sect. 3 explains the specialized crossover operator in more detail; Sect. 4 shows the experimentation performed and the results obtained. It ends with conclusions and future work.

2 Differential Evolution with Specialized Chromosome Repairer Algorithm (DECR-s)

In the article [6] the version of the Differential Evolution with Chromosome Repairer (DECR) algorithm was presented, as an adaptation of the classical DE to the car sequencing problem, where 4 aspects were considered: generation of the initial population, discretization, structure of the individual, execution approaches of the algorithm and the mutation and crossover operators. The difference with this work is the implementation of a specialized crossover operator and the application of the DE/Best/2 mutation operator [8].

Based on the 4 aspects of improvement for the DE method, the main features of the DECR-s algorithm are described. First, the Hamming distance [9] was used to generate classes of cars requiring the same options. Subsequently, the mutation process was applied using the change-based operator [10] to generate the initial population and finally the chromosome repairer [6] was applied for discretization of the individual.

The solution representation contemplates chromosomes A and B. Chromosome A is a vector whose components are genes containing identifiers belonging to the classes. Chromosome B is a vector whose components are genes containing paint color identifiers corresponding to the classes. In addition, on a chromosome (either A or B) the same identifier can appear several times. The individual by means of these two chromosomes integrates the information corresponding to the objectives to be minimized which are: constraint smoothing (associated to the classes) in the high priority workstations, low priority workstations and the color changes.

In the evolutionary process, the DECR-s algorithm is executed respecting the priority of the objectives to be minimized, considering the population obtained from one objective in the next objective, and choosing the one with the best performance by stage to update the population.

For the mutation process, the classical mutation operator identified as DE/best/2 was implemented. The operator takes four randomly selected individuals X_{r1} , X_{r2} , X_{r3} , X_{r4} different from each other, and different from the individual to replace X_{ij} . The mutated individual X_m is obtained according to Eq. 1, where X_{best} is the best individual in the current population and φ is the parameter that controls the mutation rate, which in this case takes the values 2 or 1.

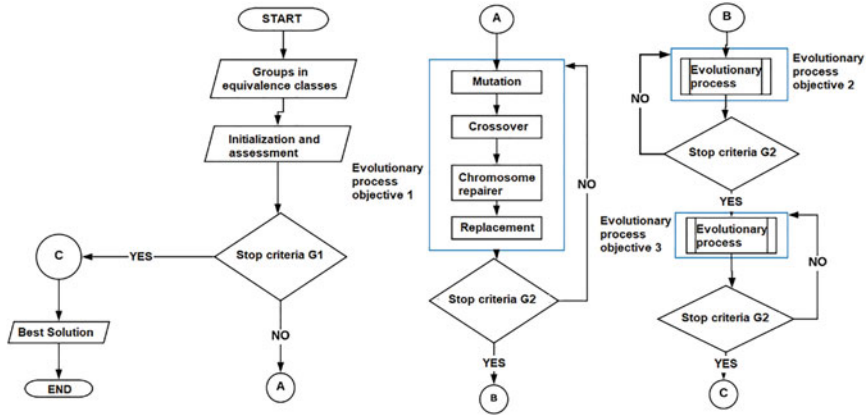


Fig. 1 Diagram flow DECR-s algorithm

$$X_m = X_{best} + j(X_{r1} + X_{r2} - X_{r3} - X_{r4}) \quad (1)$$

$$r1 \neq r2 \neq r3 \neq r4 \neq rj$$

Figure 1 shows the flow diagram of the DECR-s algorithm, where the processes described above can be observed.

In this algorithm the process of replacement of individuals within the evolutionary process, is performed in a stepwise manner, only if it is satisfied that $f_s(X_i) \leq f_s(X_{rj})$, where $f_s(X_i)$ and $f_s(X_{rj})$ are the fitness of the modified individual X_i and the individual X_{rj} to be replaced, see pseudocode 1.

Pseudocode 1 Replacement of the population according to the lexicographical order of the objectives

Replacement process(s)

```

if ( $f_3(X_i) \leq f_3(X_{rj})$ ) then
  if ( $f_2(X_i) \leq f_2(X_{rj})$ ) then
    if ( $f_1(X_i) \leq f_1(X_{rj})$ ) then
       $X_{rj} = X_i$ 
    End if
  End if
End if
End if

```

3 Specialized Crossover Operator

The classical binomial crossover operator (Eq. 2) generates a random value that is contrasted with a reference parameter CR (crossover factor) that controls the recombination rate.

$$X_t = \begin{cases} X_m & \text{if } rand(0, 1) < CR \\ X_{rj} & \text{otherwise} \end{cases} \quad (2)$$

The intent of the specialized crossover operator is to replace the random value with a value calculated from the MCS problem characteristics. The following information is used for this purpose:

- Ratio constraints: attention capacity for each workstation.
- Difficulty of a class: a class is difficult when the vehicles belonging to it require one or more options (attachments) with high demand, and/or the number of options to be installed is close to the total number of options available on the assembly line.
- Option demand: indicates the degree of utilization of each of the attachments by the vehicles to be sequenced. If demand is high the station capacity may be exceeded.
- Color demand: Ratio of how many vehicles require a color compared to the total number of vehicles in the input sequence and the maximum number of vehicles that can be painted with the same color.

In addition to reducing uncertainty in the evolutionary process, this operator seeks to contribute to better results mainly in the constraint smoothing objectives.

3.1 Specialized Crossover Operator for Chromosome a (Classes)

Chromosome A is the main structure used by the evolutionary process when constraint smoothing objectives need to be processed. In this case, the specialized crossover operator employs the concept of option demand to establish the difficulty of a given class to be sequenced. The latter value replaces the random value used in the classical operator.

To do this, the demand for option i , denoted d_i , is calculated by dividing the number of vehicles of all classes requiring option i , denoted by nb_i by the ceiling function (greater or equal integer) of multiplying the ratio constraint N_i/P_i by the size of the input sequence n (Eq. 3) [7].

$$d_i = nb_i \div [n * N_i / P_i] \quad (3)$$

If $d_i \geq 1$ the station capacity will be exceeded, if $0 < d_i < 1$ the station will not be exceeded and if $d_i = 0$ then the option was never required.

Subsequently, we proceed to calculate the difficulty of each of the k classes, called D_k by applying Eq. 4, where op_{ik} takes the value of 1 if class k requires option i and 0 otherwise. The number of high priority (h) and low priority (l) options are considered for the calculation.

$$D_k = \sum_{i=1}^{|h+l|} op_{ik} * d_i \quad (4)$$

This specialized crossover operator seeks to keep as many genes of the mutated individual X_m as possible. After calculating D_k , the genes of greater or lesser class difficulty can be selected; this gives rise to two application schemes of the specialized operator, which are explained as follows.

3.1.1 Scheme 1 (Difficult Classes)

This scheme generates a crossover individual X_t by placing at the beginning of the sequence the classes with greater difficulty and then the classes with lesser difficulty. During the evolutionary process, the chromosomal repairer takes care of interleaving the classes of lower difficulty among those of higher difficulty.

Which, in a real environment, means that the vehicles that require more attachments would start in the sequence, occupying all or most of the workstations, but to avoid overloads the vehicles that need less attachments will be interleaved.

This is achieved by considering 4 cases, where it is observed that the individual X_t is built gene by gene according to the conditions defined in Eq. 5, where D_k^u means the difficulty of class k in the individual u with $u = X_m$ in the first condition, $u = X_{rj}$ in the second and in the third $u = X_m$ for the left term and $u = X_{rj}$ for the right term.

$$X_t = \begin{cases} X_{rj} & \text{if } D_k^u > CR \\ X_m & \text{if } D_k^u > CR \\ X_m & \text{if } D_k^u > D_k^u \\ X_{rj} & \text{otherwise} \end{cases} \quad (5)$$

3.1.2 Scheme 2 (Easy Classes)

This scheme is the inverse of scheme 1, creating the X_t crossover individual by placing at the beginning of the sequence those classes with lower difficulty and then those with higher difficulty. Letting the chromosomal repairer take care of interleaving the classes of greater difficulty among those of lesser difficulty.

In contrast to the previous scheme, a lighter load is set at the beginning to give the last vehicles a higher workload. This is done following the same comparisons previously mentioned with the only modification of the sign (Eq. 6) where D_k^u means the difficulty of class k in individual u with $u = X_m$ in the first condition, $u = X_{rj}$ in the second and in the third $u = X_m$ for the left term and $u = X_{rj}$ for the right term.

$$X_t = \begin{cases} X_{rj} & \text{if } D_k^u < CR \\ X_m & \text{if } D_k^u < CR \\ X_m & \text{if } D_k^u < D_k^u \\ X_{rj} & \text{otherwise} \end{cases} \quad (6)$$

3.2 Specialized Crossover Operator for Chromosome B (Color)

This operator focuses on the grouping of vehicles of the same color based on the demand for a color, denoted d_c and respecting the hard constraint of the problem Q (lot size; maximum number of consecutive cars of the same color that can be grouped). The demand for color c is calculated by dividing the number of vehicles requiring that color, denoted by v_c by the ceiling function (integer greater than or equal to) of multiplying the input sequence size n by the hard constraint Q .

$$d_c = v_c \div [n * Q] \quad (7)$$

This operator uses two schemes to create the crossover individual by grouping vehicles in small groups at the beginning or at the end of the sequence; in the case of scheme 1 it starts with those vehicles with lower demand of the color and at the end those with higher demand, in scheme 2 it starts with those with lower demand and at the end those with lower demand. This is done by Eqs. 8 and 9, where d_c^u means the difficulty of class k in individual u with $u = X_m$ in the first condition, $u = X_{rj}$ in the second and in the third $u = X_m$ for the left term and $u = X_{rj}$ for the right term.

$$X_t = \begin{cases} X_{rj} & \text{if } D_k^u < CR \\ X_m & \text{if } D_k^u < CR \\ X_m & \text{if } D_k^u < D_k^u \\ X_{rj} & \text{otherwise} \end{cases} \quad (8)$$

$$X_t = \begin{cases} X_{rj} & \text{if } D_k^u > CR \\ X_m & \text{if } D_k^u > CR \\ X_m & \text{if } D_k^u > D_k^u \\ X_{rj} & \text{otherwise} \end{cases} \quad (9)$$

Table 1 Cases of application of the crossover operator within the DECR-s

Chromosome	Case 1	Case 2	Case 3	Case 4
A (classes)	Scheme 1	Scheme 2	Scheme 1	Scheme 2
B (color)	Scheme 1	Scheme 2	Scheme 2	Scheme 2

Table 2 Instances of set A ranked by level of difficulty and priority of objectives

Groups	# Instances	Identificator
Color	4	PHEL1, PHEL2, PHEL3, PHE
Easy	5	HEPL1, HEPL2, HEPL3, HEPL4, HELP
Difficult	7	HDPL1, HDPL2, HDPL3, HDLP1, HDLP2, HDLP3, HDP

4 Experiment Design

For the crossover operator and its schemes, 4 cases of combination of the A and B chromosomes were designed for the assembly line as shown in Table 1.

In the experiments we used set A of the instances provided by RENAULT taken from the Constraint Satisfaction Problem Lib repository (see www.csplib.org). This set consists of 16 instances (Table 2) which are divided, in terms of difficulty level and goal priority, into three groups: color, easy and difficult. The names assigned to each instance correspond to their characteristics, for example, PHEL1 means that target P (color changes) is more important than target H (high priority overloads) and this in turn more important than L (low priority overloads). The letter E means that the objective H is considered as easy to optimize for RENAULT and the letter D as difficult to optimize; if there is more than one instance of this type it is assigned a consecutive integer. In some instances, the objective L is not considered.

The DECR-s algorithm was implemented in JAVA and compiled with IntelliJ IDEA Community Edition 2018. The experiments were run on a Toshiba computer with an Intel i3-3217u processor and 6 GB of RAM with Windows 10.

The parameters used for the DECR-s algorithm, were mutation factor $\varphi = 1$, crossover factor $CR = 0.9$ and population size $N = 10$, both parameters used in the state of the art. For the execution approach, two values of generations $G_1 = 25$ in general and $G_{1,1} = 100$ for each objective were used.

5 Results

The performance of the DECR-s algorithm was compared against the results of the simulated annealing algorithm reported by RENAULT and the DECR algorithm.

Relative percentages of the results and a statistical analysis are presented. In the case of the statistical analysis of the smoothing objective a Friedman test was applied

Table 3 Results for color instances

	P1	P2	P3	P4	H1	H2	H3	H4	L1	L2	L3	L4
Case 1	30	10	65.2	69	197	43	464.6	398	61	5	842.8	NA
Case 2	31.2	10	71	69.5	179	50.2	488.8	394.4	63.8	8.1	850	NA
Case 3	30	11	64	69,6	197	48	463.8	398.6	61	5	842.6	NA
Case 4	30	11	64	69	196	53.6	461.8	392	59.6	5.9	843	NA
DECR-s	30	10.5	64	69	196.5	45.5	464.2	395	61	5	846.5	NA
DECR	31.2	11	73.88	74.37	200	49,8	502.55	435.3	63.5	4.7	854.4	NA
RENAULT	30	11	64	69	197	48	462	392	61	5	883	NA

since the results are not normally distributed, and for the color change's objective the Student's t-test was applied since the results are normally distributed; in both cases it was used with a significance level $\alpha = 0.05$.

The number of overloads and color changes obtained for the group of color instances are shown in Table 3, which records the results obtained by the DECR-s algorithm for all experimental cases and the average value (fifth row) of the cases, together with the results reported by the DECR algorithm (average value) and RENAULT.

Figure 2a plots the average of the results of the DECR-s, DECR and RENAULT algorithms for the group of color instances. For the other groups, the overall performance of the DECR-s algorithm with respect to the DECR and RENAULT algorithms is shown in Fig. 2b for the easy ones and in Fig. 2c for the difficult ones.

Figures 2a and b show in them, the DECR-s (solid line) algorithm competed in most instances with Renault and improved the DECR algorithm.

Figure 2c in these instances, the results are more unstable, however, the DECR-s algorithm, in general, improves the results obtained by the DECR algorithm and achieves a better result than the RENAULT algorithm.

5.1 Results Per Case of the Crossover Operator

From the information obtained from all the experimentation, the relative percentages of the cross operator with respect to the results reported by RENAULT were calculated, where better means the cases where the fitness is lower than that of RENAULT and worse are higher.

In Table 4, in a general way the total relative percentage of the 46 comparisons performed for the 4 types of cases are shown. The data for the best and worst categories show that the DECR-s algorithm improved the performance of the DECR algorithm in any of its cases.

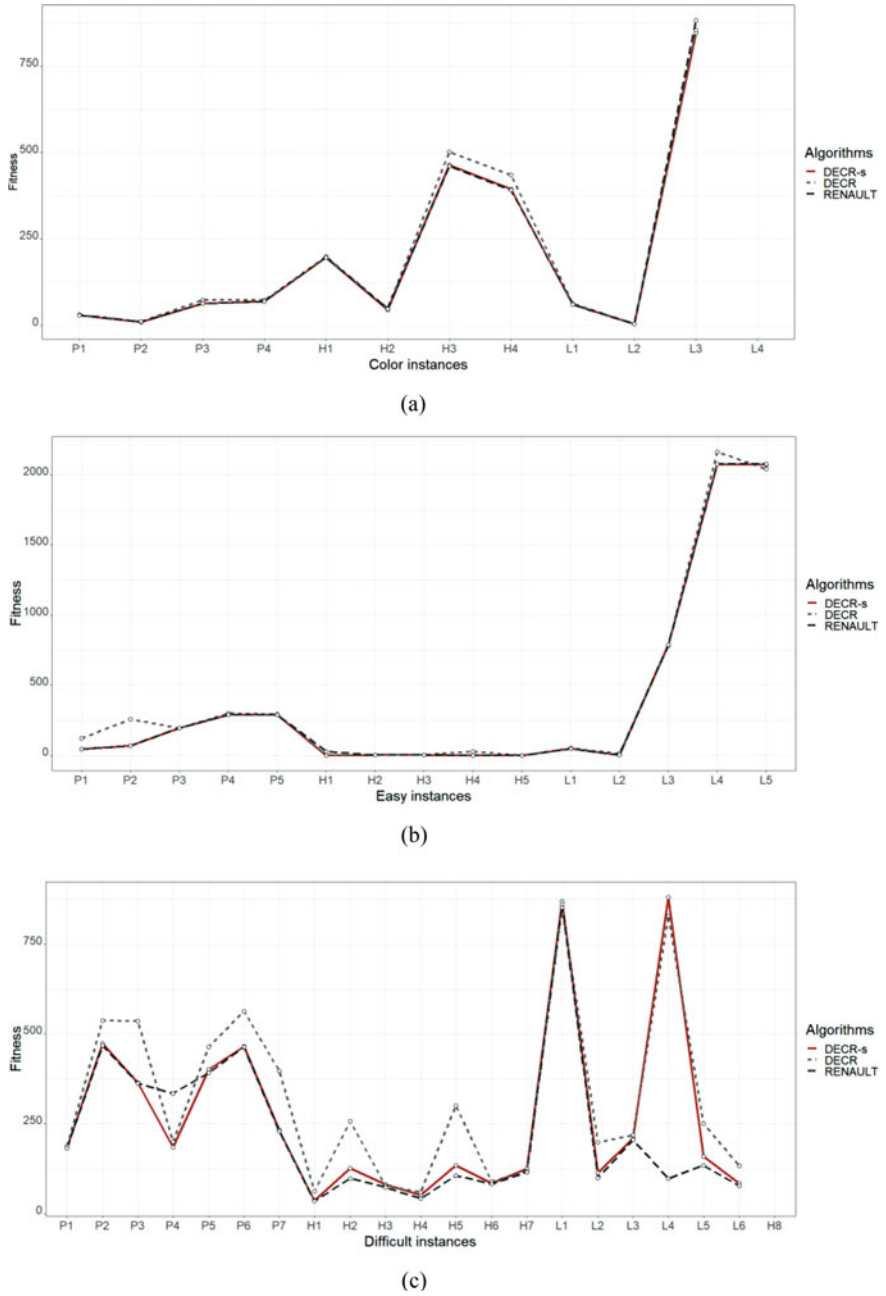


Fig. 2 Fitness of the P, H and L objectives obtained by the DECR-s, DECR and RENAULT algorithms for the **a** color group **b** easy group and **c** difficult group instances

Table 4 Total relative percentages of the 4 cases of the experimental design

	Case 1 (%)	Case 2 (%)	Case 3 (%)	Case 4 (%)	DECR (%)
Better	52	29	50	39	19
Worse	22	27	28	35	44

5.2 Statistical Analysis of Cases

This section first presents an analysis to determine if among the 4 cases there is one that is better. Subsequently, an analysis is conducted to determine if the performance of the 4 cases is the same or different than that of the RENAULT for each of the objectives (P, H, L).

The p -values obtained for objectives H and L are shown in Table 5 where it is observed that the p -values obtained exceed the level of significance; therefore, the null hypothesis cannot be rejected, i.e., it is not possible to determine if any of the 4 cases is better.

To determine the performance of the 4 cases, a Friedman test with $\alpha = 0.05$ was applied. Table 6 shows that the values obtained for objectives H and L exceed the level of significance. Therefore, the null hypothesis cannot be rejected and it is not possible to determine if any of the 4 cases is better than the RENAULT.

To contrast the performance of the cases against that reported by RENAULT, a Friedman test was applied again, finding that only for objective H the p -value shows that there is a significant difference in performance. The Ranks test is applied to identify which of the algorithms has better performance (Table 7) where it can be observed that RENAULT has the best performance.

Table 5 Friedman test results of the crossover operator cases for the H and L objectives

Objective	Significance
H	0.325
L	0.417

Table 6 Friedman test results for cases and RENAULT

Objective	Significance
H	0.011
L	0.180

Table 7 Average ranges for cases and RENAULT

Objective	Average ranges				
	RENAULT	Case 1	Case 2	Case 3	Case 4
H	1,91	2,75	3,31	3,53	3,50

For the objective P a Student's t-test with $\alpha = 0.05$ was applied pairwise between the cases and the RENAULT. In this case all p -values were in the range $[0.203, 0.990]$ so no difference was found in the performance of the algorithms.

6 Conclusion

In this work, a specialized crossover operator was proposed to improve the performance of the DECR algorithm. This operator is included in the DECR-s algorithm, which does not depend on randomness to generate the crossover individual, but on specific information of the MCS problem such as: proportion constraints, classes, and their difficulty, demands of the options and the paint color. The crossover process allows selecting a gene (vehicle) to place it in a certain position (best position) achieving balance and separability among the vehicles that make up the individual (sequence) which allows decreasing the number of overloads and color changes.

It was observed that in the case of chromosome A, depending on the case used, it is possible to decide the order of the sequence, either entering first the classes with greater difficulty or with less difficulty. Whereas, in the case of chromosome B the sequence order is decided by the paint color, which forms large and small groupings of vehicles that share the same color; allowing the chromosome repairer to make intercalations.

Through the analysis of the total relative percentages by scheme, it can be observed that case 1 is the one that provides the best results, having 52% in results where it beats the state of the art, winning with 2% to case 3 and with 23% to case 2 and 13% to case 4. This indicates that by starting a sequence with a vehicle belonging to a class of greater difficulty, interspersing one or more vehicles belonging to a class of lesser difficulty, a greater distance is obtained between equal vehicles, ensuring that the one that requires a greater workload is attended, without leaving others waiting or overloading the workstation.

The overall results show that DECR-s improves the performance of its predecessor DECR. With reference to RENAULT the DECR-s algorithm achieves competitive results in 10 out of 16 instances which represents 61%. However, aspects were observed that could improve the performance of DECR-s in future research, including that the crossover operator considers the number of options to be installed per class, and the specialization of the classical mutation operator supports the crossover operator specialized in gene recombination.

Acknowledgements The authors wish to thank the Consejo Nacional de Ciencia y Tecnología (CONACYT) of México, through grants for postgraduate studies: 482923 (J. Manzanares), 634738 (E. Sánchez) and the Tecnológico Nacional de México/Instituto Tecnológico de León, for the support provided for this research.

References

1. Solnon, C.: The car sequencing problem: overview of state-of-the-art methods and industrial case-study of the ROADEF'2005 challenge problem. *Eur. J. Oper. Res.* **191**, 912–927 (2008)
2. Zufferey, N.: Tabu Search Approaches for Two Car Sequencing Problems with Smoothing Constraints, de Metaheuristics for Production Systems, vol. 60, pp. 167–190. Springer, Francia (2016)
3. Zinflou, A., Gagn, C., Gravel, M.: Design of an efficient genetic algorithm to solve the industrial car sequencing problem. In: *Advances in Evolutionary Algorithms*. InTech (2008)
4. Cordeau, G.J.-F.: Iterated tabu search for the car sequencing problem. *Eur. J. Oper. Res.* **191**, 945–956 (2008)
5. Zhang, X.-Y., Gao, L., Wen, L., Huang, Z.-D.: A hybrid algorithm based on tabu search and large neighbourhood search for car sequencing problem. *J. Cent. South Univ.* **25**, 315–330 (2018). <https://doi.org/10.1007/s11771-018-3739-2>
6. Sánchez Márquez, E.M., Puga Soberanes, H.J., Mancilla Espinoza, L.E., Carpio Valadez, J.M., Ornelas Rodríguez, M., Manzanares Cuadro, J.I.: Algoritmo de evolución diferencial con reparador cromosómico aplicado a un problema de secuenciación de vehículos. *RCS* **148**, 279–292 (2019). <https://doi.org/10.13053/rcs-148-8-21>
7. Zinflou, A., Gagné, C., Gravel, M.: Crossover Operators for the Car Sequencing Problem, pp. 229–239. Springer, Berlin, Heidelberg (2007)
8. Li, G.-Y., Liu, M.-G.: The summary of differential evolution algorithm and its improvements. In: *2010 3rd International Conference on Advanced Computer Theory and Engineering (ICACTE)*. IEEE (2010)
9. Labib, K., Uznanski, P., Wolleb-Graf, D.: Hamming distance completeness. In: *30th Annual Symposium on Combinatorial Pattern Matching (CPM 2019)*, Schloss Dagstuhl-Leibniz-Zentrum für Informatik, p. 14 (2019)
10. Syswerda, G.: *Schedule Optimization Using Genetic Algorithms*, pp. 332–349 New York (1991)

A Brave New Algorithm to Maintain the Exploration/Exploitation Balance



Cecilia Merelo , Juan J. Merelo , and Mario García-Valdez 

Abstract At the beginning of this year one of the authors read “A brave new world”, a novel by Aldous Huxley. This book describes a dystopia, which anticipates the development of world-scale breeding technology, and how this technology creates the optimal human race. Taking into account that when talking about genetic algorithms our goal is to achieve the optimum solution of a problem, and this book kind of describes the process for making the “perfect human”, or rather the “perfect human population”, we will try to work on this parallelism in this paper, trying to find what is the key to the evolution processes described in the book. The goal is to develop a genetic algorithm based on the fecundation process of the book and compare it to other algorithms to see how it behaves, by investigating how the division in castes affects the diversity in the population. In this paper we describe the implementation of such algorithm in the programming language Julia, and how design and implementation decisions impact algorithmic and runtime performance.

Keywords Evolutionary algorithm · Metaheuristics · Literary-inspired algorithms · Exploration/exploitation balance

1 Introduction

Population-based algorithms [21] are global optimization, stochastic methods that use different techniques for exploring the search space in such a way that it makes finding the solution feasible in a reasonable amount of time. As such, they must

C. Merelo (✉) · J. J. Merelo
University of Granada, Granada, Spain
e-mail: ceci.merelo@gmail.com

J. J. Merelo
e-mail: jmerelo@ugr.es

M. García-Valdez
Tijuana Technological Institute, Tijuana, Mexico
e-mail: mario@tectijuana.edu.mx

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2022
O. Castillo and P. Melin (eds.), *New Perspectives on Hybrid Intelligent System Design based on Fuzzy Logic, Neural Networks and Metaheuristics*, Studies in Computational Intelligence 1050, https://doi.org/10.1007/978-3-031-08266-5_20

strike a balance between feasibility (exploitation of the features found in the current population of solutions to optimize fitness) and off-line performance (exploration of parts of the search space that might hold the key to those solutions) [26]. And in this balance, diversity is one of the keys [1]. This drives the search for new algorithms that explicitly try to keep this balance in a *comfort zone*.

In this search, inspiration may arrive from unexpected places. At the beginning of the year one of the authors read the famous book by Aldous Huxley: *A brave new world* [11]. The novel is a dystopy that describes the development in reproductive technology, psychological manipulation (including virtual-reality plays called *feelies* [13]) and classical conditioning [2]. It introduced interesting ideas in several areas, but in this paper we are going to focus in how they optimize the population.

In order to maximize efficiency and decrease unhappiness, the book describes how the population is divided in *castes*, assigned since birth, where everyone knows and accepts their place. That way, they achieve an “optimum world“, whose optimization is based on these reproduction restrictions and in the overall balance the division in castes creates, not in an individual. This tension between the individual happiness or realization and the overall harmony or optimal state is, precisely, the main plot driver, becoming a literary harbinger of the comparison between individual and population-based optimization algorithms, although of course the intention of the author was to compare collectivist and individualist culture [14].

When we talk about evolutionary algorithms the target is reaching the optimum solution for a problem, and this book perfectly describes the process through which they have reached the perfect human race. Therefore we want to develop an algorithm based on the book’s fecundation process and compare its behaviour with a basic genetic algorithm. Our main assumption during the design of this process is that the division in castes affects the poblation’s diversity; we can draw a parallel between the collectivism/individualism tensions inherent in the novel and the exploration/exploitation balance in population based global algorithms, and thus try and apply their solutions to our problems. The main conclusion we draw from the book is that what makes collective society happy might make some individuals extremely isolated and sad; but if we apply this to our optimization realm we can see that what Aldous Huxley might be unwittingly proposing is the rough layout of an optimization algorithm, where individuals with different fitnesses undergo different differentiation/evolution/reproduction processes. We will take it from here to design an optimization algorithm, which of course we call **Brave New Algorithm**. Essentially, **BNA** is an evolutionary algorithm with mating restrictions and, additionally, differentiated application of operators depending on the fitness value.

When designing an algorithm from scratch, we must also make technical choices on how this implementation is going to take place, as well as how the whole process of going from design (our *user stories*) to final implementation (product) can be performed according to best practices in software engineering. Implementation matters [16], and an agile development process can help us get the final result in the most efficient way, guaranteeing the quality of any software product [17]. This manifesto also encourages open science, which is the model that we are following from the onset of this research line.

The rest of the paper is organized as follows: next, the state of the art in this area is examined. The algorithm (and its implementation) are described in Sects. 3 and 4. The first experiments performed with this implementation of the algorithm are shown in Sect. 5. Finally, we will discuss the results and present our conclusions in Sect. 6.

2 State of the Art

One of the *koans* written by Goldberg in “Zen and the art of the evolutionary algorithm” states that we should let Nature be our guide. This has profitably led to many population-based metaheuristics that are inspired by the behavior of different species [19], going as far as the behavior of pigeons in a park [4]. The exhaustion of the pool of species with collective behavior to mimic has led further away, for instance to zombies [20], which eventually has led to backlash [24] claiming how metaphors obscure understanding of new algorithms, and do not advance the field of optimization. In fact, evolutionary algorithms are not the best way to reflect social processes [6], but in this case the intent of Aldous Huxley was exactly the opposite: how an industrial, at scale, version of biological evolution applied to the whole human race (except for what they called “savages”) could determine social processes.

This does not imply, however, that metaphors are necessarily unhelpful. The potential of a book such as the one we deal with here to inspire optimization has, however, not been realized, although it has been mentioned at least one in relation with an evolutionary algorithm: [25] mentioned one of the “methods” of the book, “screening out savages”, as a way of, apparently, giving 0 fitness to missiles that didn’t meet the constraints of a “commanded flight profile”.

Curiously enough, another oblique reference to Brave New World via the sentence

Consider the horse. They considered it.

in [23] brings us to the main theme in this paper. By “considering the horse”, the author of the novel refers to how exploration, the enhancement of diversity, is able to find new solutions to problems. Keeping diversity high becomes specially necessary in dynamic environments [7] since it allows the population to maintain a certain *memory* of what happened in the past.

Many different techniques have been used to enhance diversity in evolutionary algorithms, from migration policies [10] through simple mutation [12] to *reactive* methods that are “aware” of diversity and kick-in when diversity is low [27]. However, this adds an additional layer of complexity to the algorithm, needing the establishment of thresholds and/or dynamic measures.

In this paper we will present a proof of concept and initial experiments for BNA, which preserves diversity through *natural* mechanisms, and uses it to achieve higher performance in, initially, static problems.

3 Algorithm's Nature

As it was mentioned before, the algorithm is based in the optimization's process of the human race described on the book, thus we are talking about an algorithm based in the evolution of a population, it will follow the structure of evolutionary algorithms specifically a *genetic algorithm*. The book describes how they achieved the perfect human race working with an assembly line with different phases. This will be reflected with a *generational evolutionary algorithm* with selection, crossover, mutation and replacement operators.

The process begins in the ***Fecundation Room***, here the eggs are created and fertilized. Once the fertilization is finished all the eggs got to the ***Hatchery*** where the caste to which each individual will belong is decided. Huxley describes how the higher castes (*Alpha* and *Beta*) are supplied a higher amount of nutrients and hormones during the incubation. While the lower castes (*Gamma*, *Delta*, and *Epsilon*) are deprived of these elements, needed for the development. To imitate this "lack of nutrients", in the algorithm developed we will deprive the lower castes of the operators, they will only mutate. With all that has been mentioned, the castes will be developed in the following way:

- **Alphas:** in the books they are the most intelligents, the elite belongs to this group. They have responsibilities, they are ones that take decisions. In our implementation they will be reproduced with other individuals of the caste and they will evolve with all the operators.
- **Betas:** in the book they are less intelligents than the before mentioned and their main role is working in administrative tasks. In the implementation, the crossover will only be with individuals from the Alpha caste,
- **Gammas:** in the book they are subordinates, whose tasks require ability. In the implementation they will only mutate, but using local search
- **Deltas and Epsilons:** in the book both these castes are employees of the other castes and do repetitive works. In the implementation they will only have mutation by fixed segment.

With this structure in mind the metaheuristic will be divided in the following phases:

- **Fecundation room:** the individuals are created in a randomized way.
- **Hatchery room:** in this phase we will divide the population in castes. We will do this following the fitness value of the individual as the criteria. Furthermore, each caste will have a different population percentage. Because in the book they mention that lower castes are produced with the *Bokanovsky's process*, where an embryo is divided into 96 identical twins. In the algorithm this will be reflected in the population size, that will descend when the caste is higher.
- **Caste evolution:** each caste will follow a different process, as it was mentioned before

We are not talking about static castes, they are generated at the beginning of each generation. Let's imagine that we have a population size of ten, each individual with a fitness value. In the first iteration the population will be divided following that value. After that, each individual will follow the evolution process corresponding to the caste. At the end of each generation all the chromosomes will be mixed, regardless of the caste. The next generation will start dividing these chromosomes in castes again.

4 Implementation

In first implementations and versions this algorithm was written in Python [18], but a lot of performance issues arose which made the algorithm hard to analyze. This is why for this second proof of concept, one of the requirements was to use a language that provided more performance, and also at the same time could have the potential for parallelism. This is why for this version we've chosen a language that joined the "Petaflop club", which includes the languages that overcome the 1 petaflop/second as peak performance. We are talking about Julia [3] programming language, a multiparadigm dynamically typed language. In this section we will give a bit of insight about the data structures and implementation.

The agile manifesto has seldom been used in the scientific realm. This manifesto has the customer at its center, and exhorts to follow best practices that guarantee quality software through the elaboration of minimally valuable products; applying it to science in general as suggested by the agile science manifesto [17] it includes also results and any other *artifact* into the agile development cycles, with frequent interaction among the different stakeholders.

Besides, agile science is open science, and all experiments, configuration files, and data files, as well as obviously source, are available with an open source license in GitHub <https://github.com/cecimerelo/UnAlgoritmoFeliz>.

Following best practices in this area, Domain Driven Design [8] methodology was applied to the problem domain. This allowed us to go from the existing user stories (which can be read in the above-mentioned GitHub repository) to the data structures used here.

As we mentioned, Julia is a dynamically typed language, but it has some advantages from static typed ones, making it possible to indicate the types for some variables. So, for making a good use of this paradigm the castes have been defined as different types, for example, for alpha caste:

```
@with_kw struct ALPHA <: Caste
    name::String = "ALPHA"
end
```

The `with_kw` decorator is used for defining default values for the entity. Also, the castes have been defined as immutable, because the names will not change during the execution.

This differentiation will allow us to use the multiple dispatch, for example, when defining the evolution operators. As we mentioned in previous sections, depending on the castes the individual will go through different operators. In order to avoid using conditionals to check the caste we just need to do the following:

```

selector_operator(
    caste::ALPHA,
    caste_population
)
selector_operator(
    caste::BETA,
    caste_population,
    alpha_reproduction_pool
)
selector_operator(
    caste,
    caste_population
)

```

With the selector operator defined that way we make sure that each individual go through the corresponding process.

Following again best practices in the area, all functionality was individually tested using automatic testing facilities in GitHub. Eventually, it was uploaded to the Julia module repository, JuliaHub and is currently available to be used from any Julia application from <https://juliahub.com/ui/Packages/BraveNewAlgorithm/hZZw9/0.1.0>.

5 Experimental Results

In this section we will run some experiments in order to see how the algorithm behaves regarding diversity. First we will establish which metric will be used to measure it. Then we will analyse the brave new algorithm, and compare it to a basic genetic algorithm with no castes division.

5.1 Diversity Analysis

Maintaining the *diversity* is crucial for avoiding the early convergence to local optima. Rosca [22] concluded that the poblations seem to get stuck in a local optima when the entropy didn't change or decreased drably in sequential generations. In genetic programming when we talk about diversity we refer to the structural differences such as the amount of different genotypes in the population or the singularity of the fitness values [15]. In this section we will analyze the diversity of our algorithm to study how division in castes affects.

Table 1 Configuration parameters for the diversity exploration

Configuration parameters	Value
Chromosome size	20
Population size	100
Maximum generations	15
Alpha population percentage	10
Beta population percentage	20
Gamma population percentage	20
Delta population percentage	20
Epsilon population percentage	30
Mutation rate for all castes	10

There are different ways of calculating the diversity: genotypic diversity, fenotypic diversity, entropy, pseudo-isomorphism, edit distance, etc. Among which we chose *entropy*, that describes the distribution of the poblation around the different fitness values, and the *edit distance*, in which each individual it's evaluated against the best individual found so far. These metrics has been chosen since according to Burke [5] entropy and edit distance show a great correlation with the increment and decrement of the fitness value.

5.2 Diversity in Brave New Algorithm

For these experiments we will use the configuration shown in Table 1. The fitness functions evaluated will be the Rastrigin function from the *Black-box optimization benchmarking* [9]. This is a multimodal function, that allows to check the behavior of the algorithm in this kind of problems and is especially appropriate to check if diversity is maintained; a loss of diversity will make the algorithm perform poorly on this function.

In Fig. 1 the data has been taken from the Rastrigin execution that returned the best fitness value. In the graphic on top we have the fitness value that resulted from each generation. As we can see the edit distance metric is closely related to the top graphic. The smallest edit distance is in the last generation, so we can conclude that the lower the edit distance the higher possibility of the algorithm to get stuck in a local optima.

Now let's compare the entropy for the best and worst execution of the Rastrigin function to see if the entropy has anything to do with the algorithm outcome. As we know, algorithms seem to get stuck in local optima when entropy doesn't change or decrease drably in sequential generations. In Fig. 2 we can see this case in the execution with the worst fitness value. The entropy decreases more than 2 points from generation 0 to generation 30. Whilst in the case of the best execution the

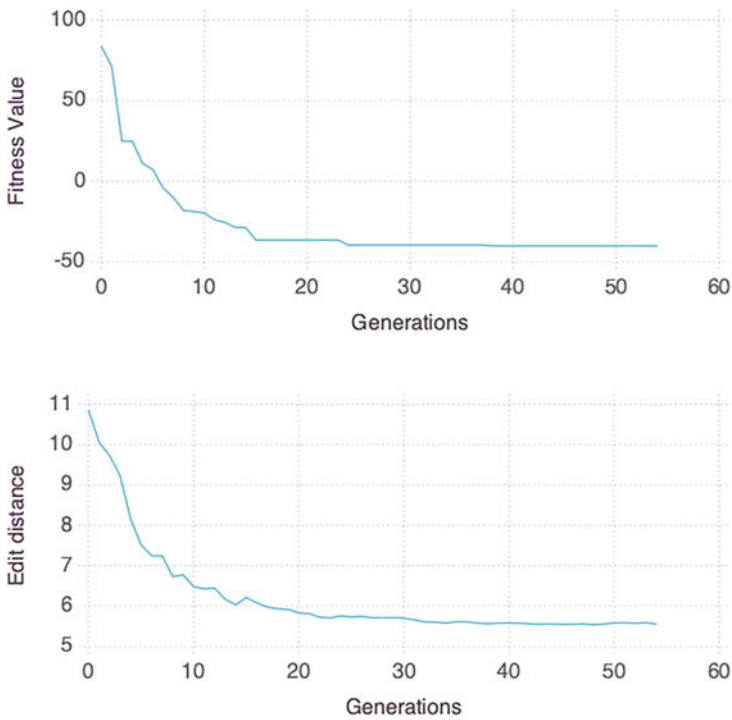


Fig. 1 Diversity in the execution with the best fitness value for the Rastrigin function

entropy slope is less strong. Also in the execution with the best fitness we can see how when the entropy stays with similar values for multiple generations is when it gets stuck in local optima.

With this information we can now compare the diversity of our algorithm with the behaviour of a genetic algorithm without castes division.

5.3 Diversity on A Basic Genetic Algorithm

We need to prove that, effectively, the results of the algorithm are obtained thanks to its preservation of diversity. We will now compare the diversity of the Brave new algorithm with a “basic“ genetic algorithm, meaning an algorithm without castes division, in order to check how the division affects the diversity.

In Table 2 we can see the results comparison for both algorithms. Having a higher value on the entropy standard deviation means that the values are spread out over a wider range, this is the case for the Brave new algorithm. Also, it has a better median of the fitness value. In Fig. 2 we can see how the entropy in the basic algorithm looks like more variant, but the values are only between 5 and 5.5 (Fig. 3).

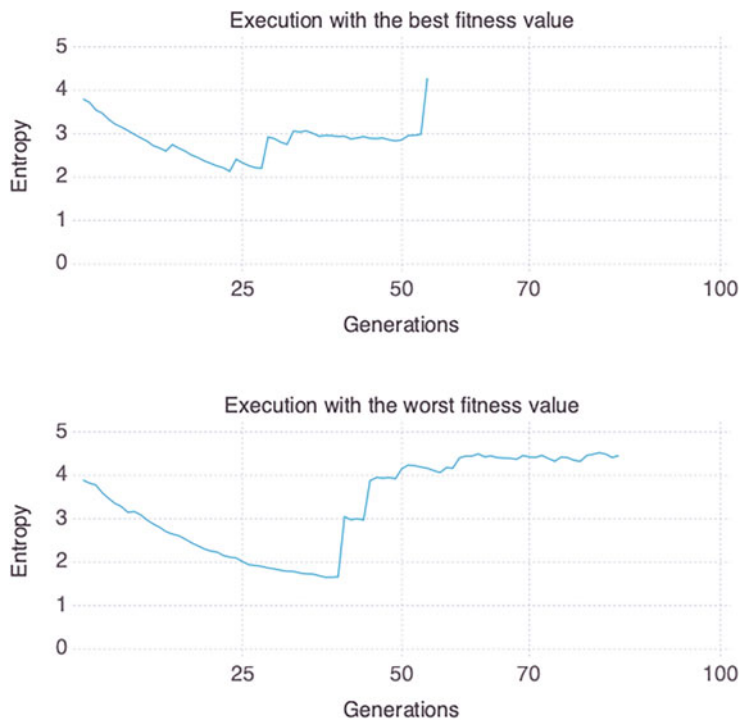


Fig. 2 Comparison of the diversity between the execution with the best and the worst fitness value for the Rastrigin function

Table 2 Entropy results for Brave New Algorithm (BNA) and a genetic algorithm without castes division (GA)

Algorithm	Entropy median	f. value median	Entropy σ	f. value σ
BNA	2.9	-39.56	0.41	26.12
GA	5.27	528.21	0.01	224.09

For this section we can conclude that because the Brave new algorithm has a higher standard deviation on the entropy the diversity is maintained throught more generations that for the basic genetic algorithm, resulting in better median for the fitness value.

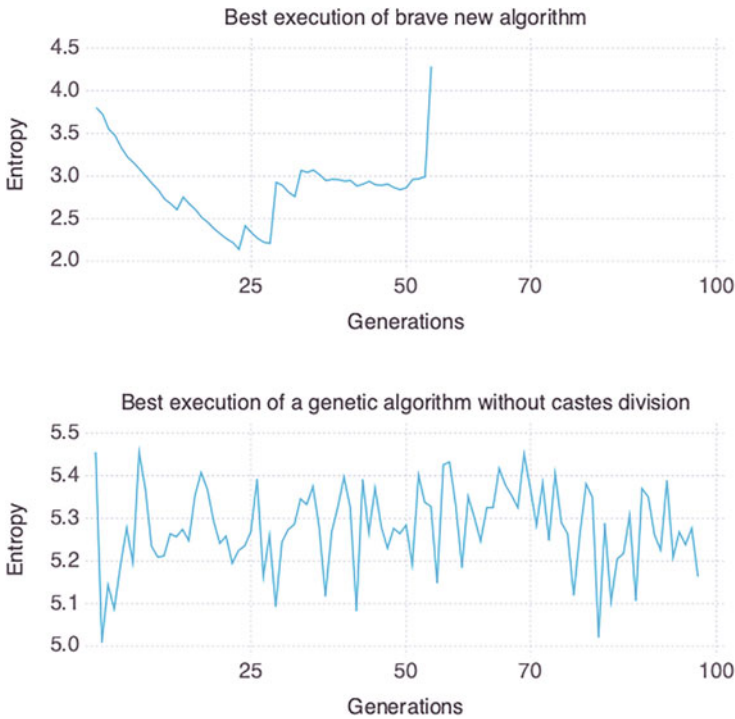


Fig. 3 Comparison of the diversity between the best executions of the Brave New Algorithm and a genetic algorithm without castes division

6 Discussion and Conclusions

We began the paper asking ourselves how the division in castes affected the diversity of the algorithm, with the goal in mind of making the algorithm avoid getting stuck in local optima as late as possible. We took the entropy as the metric for measuring this, and through the different experiments we’ve reached the conclusion that the division make the algorithm keep the diversity more generations than the basic genetic algorithm.

For future works on this algorithm it would be interesting to label all the individuals in the population and investigate how they move between the different castes. Also, explore how the diversity behaves when talking about Multiobjective Optimization Problems.

References

1. Alba, E., Dorronsoro, B.: The exploration/exploitation tradeoff in dynamic cellular genetic algorithms. *IEEE Trans. Evol. Comput.* **9**(2), 126–142 (2005)
2. Bernheim, D., Rangel, A.: Addiction, cognition, and the visceral brain. Retrieved January **23**, 2003 (2002)
3. Bezanson, J., Karpinski, S., Shah, V.B.: Julia programming language. <https://julialang.org/>
4. Blanco, A.L., Chaparro, N., Rojas-Galeano, S.: An urban pigeon-inspired optimiser for unconstrained continuous domains. In: 2019 8th Brazilian Conference on Intelligent Systems (BRACIS), pp. 521–526. IEEE (2019)
5. Burke, E., Gustafson, S., Kendall, G.: Diversity in genetic programming: An analysis of measures and correlation with fitness. *IEEE Trans. Evolut. Comput.* **8**, 47–62 (2004). <https://doi.org/10.1109/TEVC.2003.819263>
6. Chattoe, E.: Just how (un) realistic are evolutionary algorithms as representations of social processes. *J. Artif. Soc. Soc. Simul.* **1**(3), 2 (1998)
7. Cruz, C., González, J.R., Pelta, D.A.: Optimization in dynamic environments: a survey on problems, methods and measures. *Soft. Comput.* **15**(7), 1427–1448 (2011)
8. Evans, E., Evans, E.J.: Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional (2004)
9. Finck, S., Hansen, N., Ros, R., Auger, A.: Real-parameter black-box optimization benchmarking 2010: Presentation of the noiseless functions (2014)
10. García-Sánchez, P., Eiben, A.E., Haasdijk, E., Weel, B., Merelo-Guervós, J.J.: Testing diversity-enhancing migration policies for hybrid on-line evolution of robot controllers. In: Di Chio, C., Agapitos, A., Cagnoni, S., Cotta, C., de Vega, F.F., Di Caro, G.A., Drechsler, R., Ekárt, A., Esparcia-Alcázar, A.I., Farooq, M., Langdon, W.B., Merelo-Guervós, J.J., Preuss, M., Richter, H., Silva, S., Simões, A., Squillero, G., Tarantino, E., Tettamanzi, A.G.B., Togelius, J., Urquhart, N., Uyar, A.Ş., Yannakakis, G.N. (eds.) *Applications of Evolutionary Computation*, pp. 52–62. Springer, Berlin (2012)
11. Huxley, A.: Brave new world. Ernst Klett Sprachen (2007)
12. Jackson, D.: Mutation as a diversity enhancing mechanism in genetic programming. In: Proceedings of the 13th Annual Conference on Genetic and Evolutionary Computation, pp. 1371–1378 (2011)
13. Lecakes Jr, G.D.: The matrix revisited: A critical assessment of virtual reality technologies for modeling, simulation, and training. Ph.D. thesis, Rowan University (2021)
14. Mathews, G.: Happiness, culture, and context. *Int. J. Wellbeing* **2**(4) (2012)
15. McDonnell, J.R.: Genetic programming exploratory power and the discovery of functions (1995)
16. Merelo, J., Romero, G., Arenas, M.G., Castillo, P.A., Mora, A.M., Laredo, J.L.J.: Implementation matters: programming best practices for evolutionary algorithms. In: International Workshop on Artificial Neural Networks, pp. 333–340. Springer (2011)
17. Merelo-Guervós, J.J.: Agile (data) science: a (draft) manifesto. *CoRR* (2021). <https://arxiv.org/abs/2104.12545>
18. Merelo Molina, C.: Un mundo feliz. https://figshare.com/articles/preprint/Un_Mundo_feliz_pdf/16414035/1 (2021). <https://doi.org/10.6084/m9.figshare.16414035.v1>
19. Nedjah, N., Mourelle, L.D.M., Morais, R.G.: Inspiration-wise swarm intelligence meta-heuristics for continuous optimisation: a survey-part i. *Int. J. Bio-Inspired Comput.* **15**(4), 207–223 (2020)
20. Nguyen, H.T., Bhanu, B.: Zombie survival optimization: a swarm intelligence algorithm inspired by zombie foraging. In: Proceedings of the 21st International Conference on Pattern Recognition (ICPR2012), pp. 987–990. IEEE (2012)
21. Prügel-Bennett, A.: Benefits of a population: five mechanisms that advantage population-based algorithms. *IEEE Trans. Evol. Comput.* **14**(4), 500–517 (2010)

22. Rosca, J.P.: Entropy-driven adaptive representation. In: *Proceedings of the Work-shop Genetic Programming: From Theory to Real-World Applications*, pp. 23–32 (1995)
23. Slegers, J., van den Berg, D.: Backtracking (the) algorithms on the Hamiltonian cycle problem. *CoRR* (2021). <https://arxiv.org/abs/2107.00314>
24. Sörensen, K.: Metaheuristics-the metaphor exposed. *Int. Trans. Operat. Res.* **22**(1), 3–18 (2015). <https://doi.org/10.1111/itor.12001>, <https://onlinelibrary.wiley.com/doi/abs/10.1111/itor.12001>
25. Wollam, J., Kramer, S., Campbell, S.: Reverse engineering of foreign missiles via genetic algorithm. In: *38th Aerospace Sciences Meeting and Exhibit*, p. 685 (1999)
26. Xu, J., Zhang, J.: Exploration-exploitation tradeoffs in metaheuristics: survey and analysis. In: *Proceedings of the 33rd Chinese Control Conference*, pp. 8633–8638. IEEE (2014)
27. Zaharie, D., Zamfirache, F.: Diversity enhancing mechanisms for evolutionary optimization in static and dynamic environments. In: *3rd Romanian-Hungarian Joint Symposium on Applied Computational Intelligence*, pp. 460–471. Citeseer (2006)

A New Optimization Method Based on the Lotka-Volterra System Equations



Hector Carreon and Fevrier Valdez

Abstract This new method proposes the interaction that exists in nature between plants through a network of fungi, in nature there is a universe that we cannot see, where there is a frenetic exchange of information and resources between these two living beings such as plants and fungi, between 450 and 480 million years ago began the colonization of plants in the terrestrial environment and this would not have been carried out but through the symbiosis between plants and fungi over the centuries. In this research, we propose to model an optimization method for this type of interaction between plants and fungi by means of the Lotka-Volterra system of equations, 36 mathematical functions will be used to evaluate its performance and, in another study, we will evaluate it against other methods to check its effectiveness. The Lotka-Volterra system of equations offers us to model the different types of relationships that we find between these two living entities that we propose in this study.

Keywords Optimization · Lotka-Volterra · Symbiosis · Metaheuristic

1 Introduction

Within the field of artificial intelligence, it is well known that many bio-inspired algorithms have been developed in nature and basically, they have been carried out with the purpose of finding an optimal solution in a minimum time. We are proposing a new optimization method inspired by the symbiotic relationship that exists between plant roots and a network of fungi. Trappe defines the fungal network as “double absorption organs that are formed when symbiotic fungi live within the healthy absorption organs (roots, rhizomes or thalli) of terrestrial, aquatic or epiphytic plants”, to model this phenomenon. Natural we are proposing to use the Lotka-Volterra system of equations.

H. Carreon · F. Valdez (✉)
Tijuana Institute of Technology, Tijuana, BC, México
e-mail: fevrier@tectijuana.mx

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2022
O. Castillo and P. Melin (eds.), *New Perspectives on Hybrid Intelligent System Design based on Fuzzy Logic, Neural Networks and Metaheuristics*, Studies in Computational Intelligence 1050, https://doi.org/10.1007/978-3-031-08266-5_21

317

The basic objective of this research is to develop an optimization method using the Lotka-Volterra system of equations, where the fungal network plays the main role and is the reason for the inspiration of this research.

The Lotka-Volterra equation system has been used in many investigations such as: A Numerical Study on Predator Prey Model (2012), where they present the simulation of stochastic processes of the continuous time Lotka-Volterra model [1], An Approximate Analytical Algorithm for Solving The Multispecies Lotka-Volterra Equations (2012), where a new efficient method called parametric iteration method (PIM) is applied to accurately solve the Multispecies Lotka-Volterra Equations (MLVE) [2], An Implementation of Continuous Genetic Algorithm in Parameter Estimation of Predator-Prey Model (2016), where a continuous genetic algorithm was implemented to estimate parameters in a predator-prey model of Lotka-Volterra type [3], etc., to name a few.

The main contribution of this research is to use the Lotka-Volterra System of Equations: predator-prey, competitive model and cooperative model, to model the behavior that occurs in the subsurface between plant roots and fungi, which form a fungal network, to solve optimization problems. It seems incredible and we do not realize what is happening under our feet, most of the trees are connected through a network of fungi, with the purpose of exchanging resources to survive in such a competitive ecosystem, fungi obtain carbon in the form of sugars from plants, Meanwhile, the fungal network provides benefits to plants through increased acquisition of nutrients especially nitrogen, phosphorus, water and other minerals, as well as increased resistance against biotic and abiotic stresses, it has also been found that through the fungal network, warning signals are sent to plants in response to an attack by pests, predators, fire, etc., [4, 5].

The article is divided into 5 sections. Section 1 provides a brief introduction to the inspiration of the proposed method, Sect. 2 refers to the basis of this algorithm which is optimization, in Sect. 3 we present the Lotka-Volterra system of equations that are the basis of this method that models this natural phenomenon, in Sect. 4 the method is proposed, Sect. 5 shows the results and in Sect. 6 we present the conclusions we reached with the results presented above.

2 Optimization

From observing the social and collective behavior of some animals and insects, optimization algorithms have been developed to solve complex problems of daily life, for example the Particle Swarm Optimization (PSO) algorithm [6], Artificial Bee Colony (ABC) [7], Ant Colony Optimization (ACO) [8], Cuckoo Algorithm [9], Flower Pollination Algorithm (FPA) [10], etc. Most random algorithms are inspired by nature to mathematically model the solutions that have taken them for thousands of years, especially to follow search patterns, algorithms inspired by nature such as

Evolutionary Programming (EP) [11], Optimization by Swarm of Particles (PSO) [12], Genetic Programming (GP) [13], etc., are very effective in solving certain optimization problems, there is no super algorithm that solves all optimization problems, a new algorithm is always necessary to solve any optimization problem in less time, here the No Free Lunch theorem (NFL) [14] applies, that is, there is no algorithm that can solve all optimization problems.

In this article we propose a new algorithm inspired by a fungal network to solve optimization problems.

3 Lotka-Volterra System Equations

Table 1 shows the Lotka-Volterra System Equations (LVSE), this are Nonlinear Ordinary Differential Equations (ODE), the LVSE consists of the following models:

Predator–Prey (1) and (2) [15–23],

Competitive (3) and (4) [23–26] and.

Cooperative (5) and (6) [26, 27].

Parameters $\alpha, \beta, \gamma, \delta, \epsilon, \eta \in \mathbb{R} +$, and initial conditions x, y are positive real numbers.

Where x and y represent population densities of a prey and a predator, respectively, and $\alpha, \beta, \gamma, \delta$ are positive parameters. Here α represents the natural growth rate of the prey in the absence of predators, β represents the effect of predation on the prey, γ represents the natural death rate of the predator in the absence of prey, δ represents the efficiency and propagation rate of the predator in the presence of prey, ϵ represents the influence of population x on population y and η represents the influence of the population and on itself.

Table 1 Lotka-Volterra system equations

Model	Equations
Predator–Prey (Defense)	$\frac{dx}{dt} = x(\alpha - \beta y) \dots \dots (1)$ $\frac{dy}{dt} = y(-\gamma + \delta x) \dots \dots (2)$
Competitive (Colonization)	$\frac{dx}{dt} = x(\alpha - \beta x - \gamma y) (3)$ $\frac{dy}{dt} = y(\delta - \epsilon x - \eta y) (4)$
Cooperative (Resource exchange)	$\frac{dx}{dt} = x(\alpha - \beta x + \gamma y) (5)$ $\frac{dy}{dt} = y(\delta + \epsilon x - \eta y) (6)$

4 Proposed Method

The method consists of using the LVSE: Predator–Prey (Defense), Cooperative Model (Resource exchange) and Competitive Model (Colonization) with the definition of the parameters of Table 3, such as population, dimensions, epochs, iterations, etc., initially find the best fitness, start the calculation with the LVSE and randomly take the value of one of the 3 models, compare the fitness and update it, if necessary, return to the fitness calculation with the LVSE in a cycle as long as the stop condition is not met, as show in the flowchart of Fig. 1.

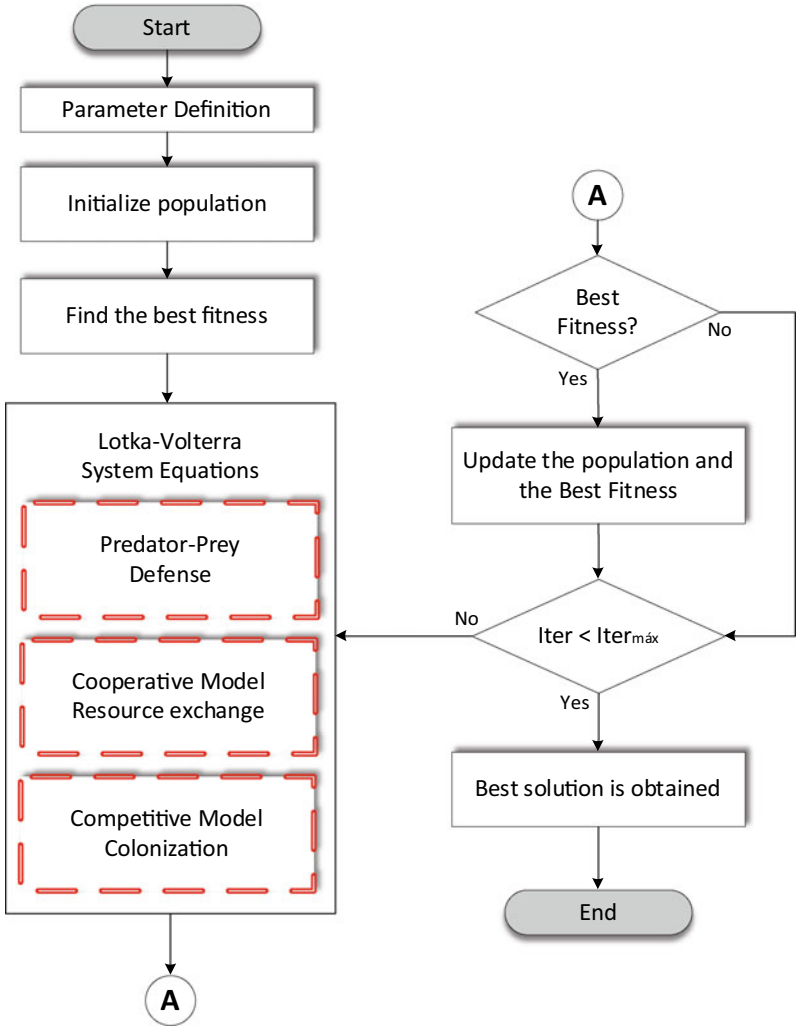


Fig. 1 Flowchart of the proposed algorithm with Lotka-Volterra equations

Table 2 shows the pseudocode of the proposed algorithm described above and Table 3 shows the parameters used.

Figure 2 shows us 5 of the 36 functions with which the experiments were carried out and Fig. 3 shows the 5 functions with which the comparisons with other methods were carried out.

Table 2 Pseudocode of the proposed algorithm with LVSE

Action	Description
Start	Start of the algorithm
Parameter initialization	Initialization of the parameters: dimensions, iterations, epochs, etc
Initialize population	Population initialized
Find the best fitness	A previous evaluation is made to find the best fitness of according to the reproduction method used
Operators	Biological operator: Defense—Resources Exchange—Colonization
Evaluate the fitness	Once the Lotka-Volterra system equations have been applied, fitness is evaluated
Update population and fitness	If the current fitness is better than the previous one, it is updated, but it returns again, applying the LVSE
Iter<Itermax	Algorithm stop condition
Best fitness	Result with the best fitness
End	End of the algorithm

Table 3 Parameters of the proposed algorithm with LVSE

Parameters	
Population	20
Populations	2
Dimensions	30, 50 and 100
Epochs	30
Iterations	30, 50, 100 and 500
Bunds	2, −2
Initial time	t0
Final time	tfinal
α—Grow rate—population x	Positive constant ∈ R+
β—Influence pop x on itself	Positive constant ∈ R+
γ—Influence pop y on pop x	Positive constant ∈ R+
δ—Grow rate—population y	Positive constant ∈ R+

(continued)

Table 3 (continued)

Parameters	
ε –Influence population x on population y	Positive constant $\in \mathbb{R}^+$
η –Influence population y on itself	Positive constant $\in \mathbb{R}^+$
x_n –Initial population x	Positive constant $\in \mathbb{R}^+$
y_n –Initial population y	Positive constant $\in \mathbb{R}^+$
Functions	36

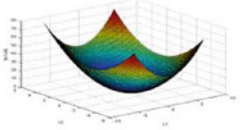
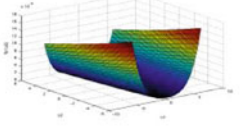
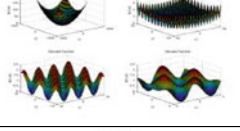
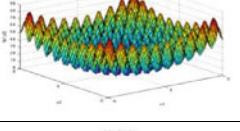
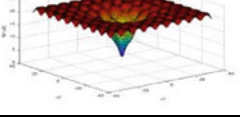
Fun	Graph	Equation
1		$f(\mathbf{x}) = \sum_{i=1}^d x_i^2$
2		$f(\mathbf{x}) = \sum_{i=1}^{d-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2]$
3		$f(\mathbf{x}) = \sum_{i=1}^d \frac{x_i^2}{4000} - \prod_{i=1}^d \cos\left(\frac{x_i}{\sqrt{i}}\right) + 1$
4		$f(\mathbf{x}) = 10d + \sum_{i=1}^d [x_i^2 - 10 \cos(2\pi x_i)]$
5		$f(\mathbf{x}) = -a \exp\left(-b \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2}\right) - \exp\left(\frac{1}{d} \sum_{i=1}^d \cos(cx_i)\right) + a + \exp(1)$

Fig. 2 Benchmark functions with which the experiments were performed

For this research, experiments were carried out with 36 CEC2013 benchmark functions as can be seen in Fig. 3, with 5 functions such as: Sphere, Ackley, Rosenbrock, Rastrigin and Griewank [28], comparisons were made with other methods.

Table 4 shows the 36 benchmark functions with which the experiments that were the reason for this research were carried out.

N	Graph	Range	Type	Formulation
F1		[-5.12, 5.12]	U	$f(x) = \sum_{i=1}^d x_i^2$
F2		[-32, 32]	M	$f(x) = -20 \exp \left[-0.2 \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} \right] - \exp \left[\frac{1}{d} \sum_{i=1}^d \cos(2\pi x_i) \right] + (20 + e)$
F3		[-5, 10]	M	$f(x) = \sum_{i=1}^{d-1} [(1 - x_i)^2 + 100(x_{i+1} - x_i^2)^2]$
F4		[-5.12, 5.12]	M	$f(x) = 10 * (d) + \sum_{i=1}^d [x_i^2 - 10 \cos(2\pi x_i)]$
F5		[-600, 600]	M	$f(x) = \frac{1}{4000} \sum_{i=1}^d x_i^2 - \prod_{i=1}^d \cos\left(\frac{x_i}{\sqrt{i}}\right) + 1$

Fig. 3 Benchmark functions with which comparisons were made with other methods

Table 4 36 benchmark functions

Function	Name	Function	Name
1	Sphere	19	Eggholder
2	Rosenbrock	20	Beale
3	Griewank	21	Holder-table
4	Rastrigin	22	Branin
5	Ackley	23	Levy
6	Dixon	24	Levy 13
7	Michalewicz	25	Shaffer 2
8	Powell	26	Shaffer 4
9	Rotate hyper ellipsoid	27	Shubert
10	Schwefel	28	Bohachevsky 1
11	Styblinski-Tang	29	Bohachevsky 2
12	Sum different powers	30	Bohachevsky 3
13	Sum squares	31	Booth
14	Trid	32	Matyas
15	Zakharov	33	Mccormick
16	Bukin no. 6	34	Easom
17	Cross-in tray	35	Goldstein-price
18	Drop-wave	36	Three-hump camel

5 Results

Table 5 shows the results of the experiments performed with 36 mathematical functions for 30 dimensions and 30 iterations, the results are in descending order of the standard deviation (SD) values, experiments were also performed for 30 × 50, 30 × 100, 30 × 500, 50 × 30, 50 × 50, 50 × 100, 50 × 500, 100 × 30, 100 × 50, 100 × 100 and 100 × 500.

Table 5 Results for 30×30 (Dimensions x Iterations)

30 Dimensions $\times$ 30 Iterations				
No	Function	Sum	M30 $\times$ 30	SD30 $\times$ 30
36	Ackley	1.69E-06	5.62E-08	2.45E-08
35	Styblinski	1.17E-06	3.91E-08	2.31E-08
34	Levy13	1.42E-06	4.72E-08	2.30E-08
33	Beale	1.46E-06	4.87E-08	2.29E-08
32	Bohache1	1.34E-06	4.46E-08	2.27E-08
31	Trid	1.44E-06	4.80E-08	2.22E-08
30	Shubert	1.54E-06	5.13E-08	2.22E-08
29	SDP	1.23E-06	4.09E-08	2.16E-08
28	Zakharov	1.32E-06	4.41E-08	2.11E-08
27	Mccormick	1.17E-06	3.89E-08	2.05E-08
26	Eggholder	1.04E-06	3.46E-08	2.04E-08
25	Powell	1.35E-06	4.49E-08	2.02E-08
24	Bohache3	1.10E-06	3.67E-08	1.99E-08
23	Sphere	1.42E-06	4.72E-08	1.98E-08
22	Bukin6	1.01E-06	3.35E-08	1.96E-08
21	Branin	1.34E-06	4.45E-08	1.95E-08
20	Michalewicz	1.05E-06	3.51E-08	1.94E-08
19	RHE	1.16E-06	3.88E-08	1.93E-08
18	Levy	1.08E-06	3.60E-08	1.91E-08
17	Hump	1.22E-06	4.06E-08	1.90E-08
16	Dixon	9.35E-07	3.12E-08	1.88E-08
15	Sum squares	1.21E-06	4.03E-08	1.87E-08
14	Schwefel	1.49E-06	4.95E-08	1.86E-08
13	Gold	1.17E-06	3.91E-08	1.85E-08
12	Booth	1.42E-06	4.73E-08	1.82E-08
11	Bohache2	1.26E-06	4.19E-08	1.81E-08
10	Rastrigin	1.15E-06	3.85E-08	1.69E-08
9	Schaffer4	1.06E-05	3.52E-07	1.50E-08
8	Crossist	5.89E-07	1.96E-08	1.13E-08
7	Rosenbrock	5.02E-07	1.67E-08	1.06E-08
6	Holder Table	5.26E-07	1.75E-08	1.02E-08
5	Matyas	4.70E-07	1.57E-08	7.77E-09
4	Griewank	3.88E-07	1.29E-08	7.35E-09
3	Shaffer2	1.23E-07	4.09E-09	2.64E-09
2	Drop	8.59E-08	2.86E-09	2.62E-09
1	Easom	1.12E-22	3.72E-24	4.26E-24

Figures 4 and 5 show the behavior of the experiments made with the 36 mathematical functions of the mean and standard deviation for 30 dimensions and 30, 50, 100 and 500 iterations.

Table 6 shows the mean and SD for 30, 50 and 100 dimensions of the Sphere, Ackley, Rosenbrock, Rastrigin and Ackley functions of the LVSE method with which the hypothesis tests were performed against 17 other methods (Table 7).

Table 7 shows the 17 methods with which hypothesis tests were performed against the LVSE method.

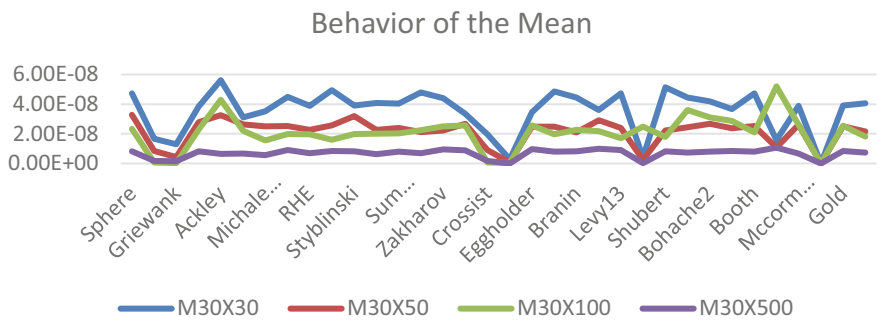


Fig. 4 Behavior of the mean for 30 dimensions

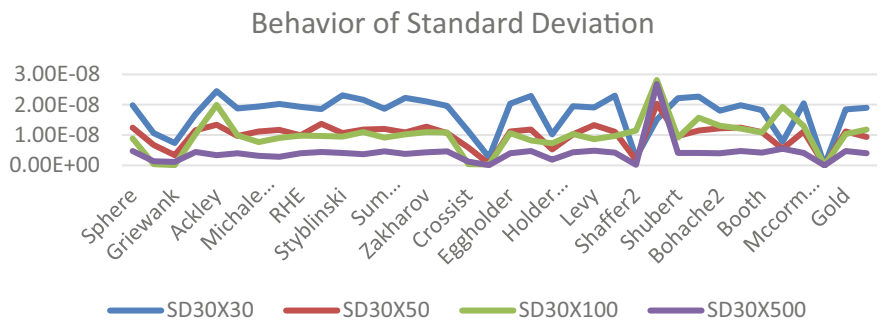


Fig. 5 Behavior of the SD for 30 dimensions

Table 6 Mean and SD for 30, 50 and 100 dimensions of LVSE

Lotka-Volterra system equations							
Fun	Name	30 Dimensions		50 Dimensions		100 Dimensions	
		Mean	SD	Mean	SD	Mean	SD
f1	Sphere	8.11E−09	4.79E−09	8.56E−09	4.23E−09	7.00E−09	3.83E−09
f2	Ackley	6.58E−09	3.32E−09	1.49E−08	5.59E−09	1.16E−07	2.13E−08
f3	Rosenbrock	1.64E−09	1.39E−09	8.26E−10	6.32E−10	1.03E−09	5.81E−10
f4	Rastrigin	8.13E−09	4.42E−09	7.36E−09	3.90E−09	8.40E−09	3.98E−09
f5	Griewank	1.44E−09	1.21E−09	1.25E−10	9.09E−11	2.99E−09	4.99E−10

Table 8 shows the results for 30 dimensions of 70 hypothesis tests that were performed with other methods in which the LVSE method obtained better results in 63, as can be seen in the final part of this table, finally LVSE obtained 90% with best results.

Table 7 Methods by which the hypothesis tests were performed

N	Algorithm name
1	ABC—Artificial bee colony [29, 30]
2	BA—Bat algorithm [31, 32]
3	CS—Cuckoo search [9, 29, 33]
4	DE—Differential evolution [34–36]
5	FA—Firefly algorithm [29, 37]
6	FPA—Flower pollination algorithm [10, 38, 31]
7	GA—Genetic algorithm [29, 34, 39]
8	GSA—Gravitational search algorithm [29, 40]
9	GWO—Grey Wolf optimizer [29, 41]
10	HS—Harmony search [29, 42]
11	PSO—Particle swarm optimization [6, 29, 34]
12	SPSO—Speciation-based PSO [34]
13	WSA—Whale swarm algorithm [34, 43]
14	SD Clone—Self defense mechanism of plants—clone [44]
15	SD Pollinat—Self defense mechanism of plants—pollinat [44]
16	FWA—FireWorks Algorithm [45, 46]
17	SHADE—Success History Based Adaptive DE [47]

Table 8 Hypothesis tests for 30 dimensions

Fun	SD—Clone [44]				SD—Pollinat [44]			
	Mean	Std Dev	Z	Eviden	Mean	Std Dev	Z	Eviden
f1	1.53E−08	6.42E−08	−0.61	Yes	2.94E−16	1.23E−15	9.27	No
f2	1.94E−04	6.99E−04	−1.52	Yes	5.28E−14	1.68E−14	10.84	No
f3	4.98E+00	7.54E+00	−3.62	Yes	1.96E+01	8.62E+00	−12.45	Yes
f4	4.22E−04	1.61E−03	−1.44	Yes	1.63E−05	8.92E−05	−1.00	Yes
f5	4.92E−09	4.02E−09	−4.54	Yes	3.39E−04	1.86E−03	−1.00	Yes
Fun	BA [31]				CS [29]			
	Mean	Std Dev	Z	Eviden	Mean	Std Dev	Z	Eviden
f1	2.38E+01	6.85E+00	−19.03	Yes	2.13E−04	2.18E−04	−5.35	Yes
f2	1.52E+01	1.13E+00	−73.68	Yes	2.15E+00	1.09E+00	−10.80	Yes
f3	4.29E+05	3.14E+05	−7.48	Yes	2.88E+01	3.29E+01	−4.79	Yes
f4	4.64E+02	7.87E+01	−32.29	Yes	5.34E+01	1.22E+01	−23.97	Yes
f5	9.09E+01	2.46E+01	−20.24	Yes	2.72E−02	5.86E−02	−2.54	Yes

(continued)

Table 8 (continued)

Fun								
Fun	DE [35]				HS [29]			
f1	1.59E−03	4.28E−04	−20.35	Yes	3.66E−02	5.95E−03	−33.69	Yes
f2	9.78E−03	1.31E−03	−40.89	Yes	2.32E+00	4.68E−01	−27.15	Yes
f3	3.38E+01	2.25E+01	−8.23	Yes	1.02E+02	6.06E+01	−9.22	Yes
f4	3.72E+01	1.17E+01	−17.41	Yes	2.89E+01	4.68E+00	−33.82	Yes
f5	3.18E−03	1.57E−03	−11.09	Yes	2.71E−02	2.34E−02	−6.34	Yes
Fun	FWA [45]				PSO [29]			
f1	3.08E−02	3.38E−02	−4.99	Yes	9.28E−05	5.09E−05	−9.99	Yes
f2	5.00E−02	1.19E−02	−23.01	Yes	2.62E−03	5.91E−04	−24.28	Yes
f3	3.09E+01	7.28E+00	−23.28	Yes	5.06E+01	3.83E+01	−7.24	Yes
f4	2.64E+00	1.13E+00	−12.82	Yes	8.76E+01	3.13E+01	−15.33	Yes
f5	8.00E−04	7.26E−04	−6.04	Yes	1.93E−03	3.33E−03	−3.17	Yes
Fun	GA [29]				ABC [29]			
f1	4.20E−03	3.88E−03	−5.93	Yes	1.00E−15	3.26E−16	9.27	No
f2	4.71E−01	5.22E−01	−4.94	Yes	1.75E−12	1.18E−12	10.84	No
f3	1.59E+01	2.54E+01	−3.43	Yes	3.46E−01	6.01E−01	−3.15	Yes
f4	1.31E+01	4.13E+00	−17.37	Yes	1.14E−02	4.33E−02	−1.44	Yes
f5	4.43E−04	5.20E−04	−4.67	Yes	4.61E−03	1.26E−02	−2.00	Yes
Fun	FA [29]				GSA [29]			
	Mean	Std Dev	Z	Eviden	Mean	Std Dev	Z	Eviden
f1	1.99E−03	5.13E−04	−21.25	Yes	9.45E−17	3.81E−17	9.27	No
f2	1.06E−02	1.42E−03	−40.89	Yes	7.16E−09	1.69E−09	−0.85	Yes
f3	3.41E+01	1.84E+01	−10.15	Yes	2.70E+01	7.45E+00	−19.85	Yes
f4	3.71E+01	9.92E+00	−20.48	Yes	2.63E+01	5.69E+00	−25.32	Yes
f5	3.38E−03	1.59E−03	−11.64	Yes	3.19E−02	3.06E−02	−5.71	Yes
Fun	GWO [29]				FPA [31]			
f1	3.09E−103	1.03E−102	9.27	No	2.12E+00	1.14E+00	−10.19	Yes
f2	1.14E−14	3.49E−15	10.84	No	5.06E+00	1.57E+00	−17.65	Yes
f3	2.69E+01	7.92E−01	−186.03	Yes	8.28E+03	8.61E+03	−5.27	Yes
f4	3.82E−01	1.47E+00	−1.42	Yes	1.47E+02	2.61E+01	−30.85	Yes
f5	1.30E−03	4.09E−03	−1.74	Yes	7.76E+00	3.37E+00	−12.61	Yes
			Yes	No	Total	%		
			63	7	70	90		

Table 9 Hypothesis test for 50 dimensions

Fun	SD-clone [44]				SD-pollinat [44]			
	Mean	Std dev	Z	Eviden	Mean	Std dev	Z	Eviden
f1	1.59E-05	7.25E-05	-1.20	Yes	1.99E-14	1.08E-13	11.09	No
f2	1.13E-02	5.97E-02	-1.04	Yes	9.15E-14	2.03E-14	14.63	No
f3	6.42E+00	8.79E+00	-4.00	Yes	4.47E+01	1.38E+01	-17.73	Yes
f4	3.01E-03	9.60E-03	-1.72	Yes	5.69E-03	3.11E-02	-1.00	Yes
f5	1.09E-05	5.90E-05	-1.01	Yes	5.17E-04	2.83E-03	-1.00	Yes
Fun	DE [35]				PSO [29]			
f1	7.19E-03	1.79E-03	-22.00	Yes	1.69E-01	6.80E-02	-13.61	Yes
f2	1.86E-02	2.05E-03	-49.70	Yes	1.32E-01	1.09E-01	-6.63	Yes
f3	4.80E+01	7.41E-01	-354.80	Yes	1.71E+02	4.27E+01	-21.93	Yes
f4	7.81E+01	2.27E+01	-18.84	Yes	2.18E+02	5.16E+01	-23.14	Yes
f5	5.91E-03	1.09E-03	-29.70	Yes	1.71E-01	7.64E-02	-12.26	Yes
Fun	CS [29]				GA [29]			
f1	9.90E-01	7.89E-01	-6.87	Yes	1.77E+00	6.66E-01	-14.56	Yes
f2	4.84E+00	1.32E+00	-20.08	Yes	1.73E+00	3.15E-01	-30.08	Yes
f3	2.28E+02	1.27E+02	-9.83	Yes	2.96E+01	5.07E+01	-3.20	Yes
f4	1.15E+02	2.07E+01	-30.43	Yes	3.70E+01	7.63E+00	-26.56	Yes
f5	6.21E-01	2.13E-01	-15.97	Yes	5.05E-02	1.61E-02	-17.18	Yes
Fun	SHADE [47]				HS [29]			
f1	0.00E+00	0.00E+00	11.09	No	3.54E-01	7.94E-02	-24.42	Yes
f2	2.09E+01	1.68E-01	-680.30	Yes	3.12E+00	3.14E-01	-54.42	Yes
f3	4.28E+01	5.52E+00	-42.47	Yes	2.04E+02	7.36E+01	-15.18	Yes
f4	5.86E+01	1.11E+01	-28.86	Yes	8.26E+01	8.04E+00	-56.27	Yes
f5	7.37E-02	3.67E-02	-10.98	Yes	1.22E+00	7.73E-02	-86.45	Yes
Fun	BA [31]				GSA [29]			
f1	8.90E+00	4.55E+01	-1.07	Yes	4.54E-16	2.19E-16	11.09	No
f2	7.20E-01	1.53E+01	-0.26	Yes	7.27E-02	2.73E-01	-1.46	Yes
f3	3.95E+05	8.78E+05	-2.46	Yes	7.62E+01	3.50E+01	-11.92	Yes
f4	1.06E+02	8.49E+02	-0.68	Yes	5.07E+01	1.26E+01	-22.04	Yes
f5	3.05E+01	1.55E+02	-1.08	Yes	1.24E+00	2.44E-01	-27.84	Yes
Fun	ABC [29]				FA [29]			
	Mean	Std Dev	Z	Eviden	Mean	Std Dev	Z	Eviden
f1	2.06E-12	2.06E-12	11.09	No	1.15E-02	3.20E-03	-19.68	Yes
f2	4.17E-06	2.61E-06	-8.72	Yes	2.49E-02	3.66E-03	-37.26	Yes
f3	1.05E+00	1.03E+00	-5.58	Yes	5.85E+01	3.18E+01	-10.08	Yes
f4	1.33E+00	9.16E-01	-7.95	Yes	7.45E+01	2.47E+01	-16.52	Yes
f5	2.37E-03	5.65E-03	-2.30	Yes	7.01E-03	2.04E-03	-18.82	Yes

(continued)

Table 9 (continued)

Fun								
Fun	GWO [29]				FPA [31]			
f1	1.30E−78	2.57E−78	11.09	No	5.78E+00	1.82E+00	−17.39	Yes
f2	1.77E−14	3.17E−15	14.63	No	5.95E+00	1.17E+00	−27.85	Yes
f3	4.73E+01	9.07E−01	−285.64	Yes	3.00E+04	1.81E+04	−9.08	Yes
f4	5.66E−01	1.61E+00	−1.93	Yes	2.79E+02	3.52E+01	−43.41	Yes
f5	3.09E−03	6.05E−03	−2.80	Yes	2.14E+01	6.32E+00	−18.55	Yes
			Yes	No	Total	%		
			63	7	70	90		

Table 10 Hypothesis tests for 100 dimensions

Fun	FPA [38]				GA [34]			
	Mean	Std Dev	Z	Eviden	Mean	Std Dev	Z	Eviden
f1	4.37E+01	3.21E+ 01	−7.45	Yes	5.62E−06	2.84E−07	−108.24	Yes
f2	1.59E+00	1.81E−01	−48.06	Yes	−	−		
f3	4.44E+02	8.41E+01	−28.92	Yes	1.18E−08	1.00E−09	−50.99	Yes
f4	3.53E+00	7.74E−01	−24.98	Yes	6.04E−06	2.37E−07	−139.37	Yes
f5	5.10E−02	1.35E−02	−20.65	Yes	1.84E−02	7.36E−04	−136.93	Yes
Fun	PSO [34]				WSA [34]			
f1	7.21E−06	8.43E−07	−46.80	Yes	1.00E+00	6.68E−09	−7.11E+08	Yes
f2	−	−			−	−		
f3	5.48E−08	3.26E−08	−9.03	Yes	1.01E−02	8.13E−05	−680.44	Yes
f4	7.17E−06	8.45E−07	−46.42	Yes	1.77E−04	5.07E−05	−19.12	Yes
f5	2.47E−02	3.72E−03	−36.37	Yes	5.00E−01	4.56E−13	−5.48E+09	Yes
Fun	DE [34]				SPSO [34]			
f1	3.99E−05	1.12E−05	−19.51	Yes	4.73E−05	5.86E−06	−44.20	Yes
f2	−	−			−	−		
f3	2.17E−07	7.72E−08	−15.32	Yes	1.54E−05	3.58E−06	−23.56	Yes
f4	4.35E−05	9.03E−06	−26.38	Yes	1.64E−04	1.29E−05	−69.63	Yes
f5	1.12E−01	2.74E−02	−22.39	Yes	2.99E−01	1.32E−02	−124.07	Yes
			Yes	No	Total	%		
			25	0	25	100		

Table 9 shows the results for 50 dimensions of 70 hypothesis tests that were performed with other methods in which the LVSE method obtained better results in 63, as can be seen in the final part of this table, finally LVSE obtained 90% with better results.

Table 10 shows the results for 100 dimensions of 25 hypothesis tests that were performed with other methods in which the LVSE method obtained better results in

25, as can be seen in the final part of this table, finally LVSE obtained 100% with best results.

6 Conclusions

We worked with 36 mathematical functions described in Table 2, tests were carried out for 30, 50 and 100 dimensions and 30, 50, 100 and 500 iterations, hypothesis tests were carried out with 5 mathematical functions: Sphere, Ackley, Rosenbrock, Rastrigin and Griewank (Fig. 3), in order to perform the hypothesis tests, these functions were common in 17 methods in the literature, in the statistical tests we found that for 30 dimensions of 70 tests the LVSE method was better in 63, for 50 dimensions of 70 tests the LVSE method was better in 63 and for 100 dimensions of 25 tests the LVSE method was better in 25, in general in this investigation 165 hypothesis tests were carried out, of which the LVSE method was better in 151, that is to say in 91.5% of all the tests the LVSE method was better, we can also conclude that if we increase the dimensions, the method can obtain better results in exchange for a higher computational cost, in the future we will continue to carry out experiments where we include Neural Networks (NNs) and Type-1 Fuzzy Logic (T1FLS) and Interval Type-2 Fuzzy Logic (IT2FLS), like in [48–50].

References

1. Laham, M., Krishnarajah, I., Jumaat, A.: A numerical study on predator prey model. *Int. J. Modern Phy. Conf. Ser.* **09**, 347–353 (2012)
2. Alavi, A., Ghorbani, A.: An approximate analytical algorithm for solving the multispecies Lotka-Volterra equations. *Appl. Appl. Math. Int. J.* **7**, 636–647 (2012)
3. Windarto (2016) An implementation of continuous genetic algorithm in parameter estimation of predator-prey model. In: *AIP Conference Proceedings*, p 1718
4. Gilbert, L., Johnson, D.: Chapter four—plant-plant communication through common mycorrhizal networks. In: Guillaume, B. (ed.) *Advances in Botanical Research*, vol. 82, pp. 83–97. Academic Press (2017)
5. Bonfante, P., Genre, A.: Mechanisms underlying beneficial plant–fungus interactions in mycorrhizal symbiosis. *Nat Commun* **1**, 48 (2010)
6. Kennedy, J., Eberhart, R.: Particle swarm optimization. In: *Proceedings of the IEEE International Conference on Neural Networks*. IEEE, Piscataway, NJ, USA, pp. 1942–1948 (1995)
7. Karaboga, D.: An idea based on honey bee swarm for numerical optimization, Technical Report-TR06, Eng Fac Comput Eng Dep ERCIYES University (2005)
8. Dorigo, M.: Optimization, learning and natural algorithms. Ph.D. thesis, Politecnico di Milano, Milano, Italy (1992)
9. Alcantara Neto, M.C., Araujo, J.P.L., Barros, F.J.B., Silva, A.N., Cavalcante, G.P.S., D'Assuncao, A.G.: Bioinspired multiobjective synthesis of X-band FSS via general regression neural network and cuckoo search algorithm. *Microw. Opt. Technol. Lett.* **57**, 2400–2405 (2015)

10. Yang, X.-S.: Flower pollination algorithm for global optimization. In: Durand-Lose, J., Jonoska, N. (eds.) *Unconventional Computation and Natural Computation (UCNC 2012)*. Springer, vol. 7445, pp. 240–249. Berlin, Heidelberg, Germany (2012)
11. Yao, X., Liu, Y., Lin, G.: Evolutionary programming made faster. *IEEE Trans. Evol. Comput.* **3**(2), 82–102 (1999)
12. Yang, X.S., Cui, Z., Xiao, R., Gandomi, A.H., Karamanoglu, M.: *Swarm Intelligence and Bio-Inspired Computation Theory and Applications*, 1st Edn, pp. 3–21. Elsevier, London (2013)
13. Bhowan, U., Zhang, M., Johnston, M.: Genetic Programming for classification with unbalanced data. In: Esparcia-Alcázar, A.I., Ekárt, A., Silva, S., Dignum, S., Uyar, A.Ş. (eds.) *Genetic Programming. EuroGP 2010. Lecture Notes in Computer Science*, vol. 6021, pp. 1–13. Springer, Berlin, Heidelberg (2010)
14. Adam, S.P., Alexandropoulos, S.A.N., Pardalos, P.M., Vrahatis, M.N.: No free lunch theorem: a review. In: *Approximation and Optimization Algorithms, Complexity and Applications*, pp. 57–82. Springer Nature, Switzerland (2019)
15. Mickens, R.: A note on exact finite difference schemes for modified Lotka-Volterra differential equations. *J. Differ. Equ. Appl.* **24**, 1–7 (2018)
16. Al-Kahby, H., Dannan, F., Elaydi, S.: Nonstandard discretization methods for some biological models. In: Mickens, R.E., (ed.) *Applications of Nonstandard Finite Difference Schemes*, pp. 181–243. World Scientific Singapore (2000)
17. Murray, J.D.: *Mathematical Biology*, See Sections 3.1 and 3.3. Springer-Verlag, Berlin (1989)
18. Roeger, L.-I.W.: Dynamically consistent Lotka-Volterra competition models from nonstandard finite-difference schemes. *Discrete Contin. Dyn. Syst. Ser. B* **9**, 415–429 (2008)
19. Turchin, P.: *Complex Population Dynamics: A Theoretical/Empirical Synthesis*, See Section 4.2. Princeton University Press, Princeton, NJ (2006)
20. Bacaër, N.: A Short History of Mathematical Population Dynamics, pp. 70–76. (2011)
21. Canale, R.P.: An analysis of models describing predator-prey interaction. *Biotechnol. Bioeng.* **12**, 353–378 (1970)
22. Evans, C., Findley, G.: Analytic solutions to a family of Lotka-Volterra related differential equations. *J. Math. Chem.* **25**, 181–189 (1999)
23. Anisiu, M.-C.: Lotka, Volterra and their model. *Didactica Math.* **32**, 9–17 (2014)
24. Dimas, M.A., Gjini, E.: Modeling competitive mixtures with the Lotka-Volterra framework for more complex fitness assessment between strains. *Front. Microbiol.* **11**, 572487 (2020)
25. Voroshilova, A., Wafubwa, J.: Discrete competitive Lotka-Volterra model with controllable phase volume. *Systems* **8**(17), 1–12 (2020)
26. Wang, C., Li, L., Zhang, Q., Li, R.: Dynamical behaviour of a Lotka-Volterra competitive-competitive-cooperative model with feedback controls and time delays. *J. Biol. Dyn.* **13**, 43–68 (2019)
27. Lu, G., Lu, Z.: Non-permanence for three-species Lotka-Volterra cooperative difference systems. *Adv Differ Equ* **2017**, 152 (2017)
28. Surjanovic, S., Bingham, D.: Virtual library of simulation experiments: test functions and datasets, (2013). Retrieved from <http://www.sfu.ca/~ssurjano>, 17 September 2021
29. Rakhshani, H., Rahati, A.: Snap-drift cuckoo search: a novel cuckoo search optimization algorithm. *Appl. Soft Comput.* **52**(2017), 771–794 (2017)
30. Karaboga, D., Basturk, B.: A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm. *J. Global Optim.* **39**(3), 459–471 (2007)
31. Sakib, N., Kabir, W.U.I., Rahman, M., Alam, M.: A comparative study of flower pollination algorithm and bat algorithm on continuous optimization problems. *Int. J. Appl. Inform. Syst.* **7**, 13–19 (2014)
32. Yang, X.S.: A new metaheuristic bat-inspired algorithm. In: González, J.R., Pelta, D.A., Cruz, C., Terrazas, G., Krasnogor, N. (eds.) *Nature Inspired Cooperative Strategies for Optimization (NICSO 2010)*. Studies in Computational Intelligence, vol. 284, pp. 65–74. Springer, Berlin, Heidelberg (2010)
33. Yang, X.S., Deb, S.: Cuckoo search via Lévy flights. In: *Proceedings of World Congress on Nature & Biologically Inspired Computing (NaBIC 2009)*. IEEE Publications, USA, pp. 210–214 (2009)

34. Zeng, B., Gao, L., Li, X.: Whale swarm for function optimization. In: Part, I. (ed.) *Huazhong University of Science and Technology, Wuhan, China, Intelligent Computing Theories and Application 13th International Conference, ICIC 2017 Liverpool, UK, August 7–10, 2017 Proceedings*, pp. 624–639. Springer International Publishing AG (2017)
35. Tvrdík, J., Poláková, R.: Competitive differential evolution applied to CEC 2013 problems. In: 2013 IEEE Congress on Evolutionary Computation, pp. 1651–1657. (2013)
36. Price, K., Storn, R., Lampinen, J.: *Differential Evolution: A Practical Approach to Global Optimization*, pp. 37–131. Springer, Berlin, Germany (2005)
37. Yang, X.S.: Firefly algorithms for multimodal optimization. In: Watanabe, O., Zeugmann, T. (eds.), *Proceedings of Fifth Symposium on Stochastic Algorithms, Foundations and Applications. Lecture Notes in Computer Science*, vol. 5792, pp. 169–178. Springer (2009)
38. Hector Carreon, Fevrier Valdez and Castillo O.: Fuzzy flower pollination algorithm to solve control problems. In: *Hybrid Intelligent Systems in Control, Pattern Recognition and Medicine, Studies in Computational Intelligence*, pp. 119–154. Springer Nature, Switzerland AG (2020)
39. Holland, J.: *Adaptation in Nature and Artificial Systems*. University of Michigan Press, Ann Arbor, MI, USA (1975)
40. Rashedi, E., Nezamabadi-Pour, H., Saryazdi, S.: GSA: a gravitational search algorithm. *Inf. Sci.* **179**(13), 2232–2248 (2009)
41. Mirjalili, S., Mirjalili, S.M., Lewis, A.: Grey Wolf Optimizer. *Adv. Eng. Softw.* **69**, 46–61 (2014)
42. Geem, Z.W., Kim, J.H., Loganathan, G.V.: A new heuristic optimization algorithm: harmony search. *Simulation* **76**(2), 60–68 (2001)
43. Zeng, B., Gao, L., Li, X.: Whale swarm algorithm for function optimization. In: Huang, D.S., Bevilacqua, V., Premaratne, P., Gupta, P. (eds.) *Intelligent Computing Theories and Application. ICIC 2017. Lecture Notes in Computer Science*, vol. 10361, pp. 624–639. Springer, Cham (2017)
44. Caraveo, C., Valdez, F., Castillo, O.: A New Bio-inspired Optimization Algorithm Based on the Self-defense Mechanism of Plants in Nature. In: *SpringerBriefs in Applied Sciences and Technology Computational Intelligence. Division of Graduate Studies Tijuana Institute of Technology Tijuana, Baja California, Mexico*, pp. 23–52 (2019)
45. Barraza, J., Melin, P., Valdez, F., González, C.: Fireworks algorithm (FWA) with adaptation of parameters using fuzzy logic. In: Melin, P., Castillo, O., Kacprzyk, J. (eds.) *Nature-Inspired Design of Hybrid Intelligent Systems. Studies in Computational Intelligence*, vol. 667, pp. 313–327. Springer, Cham (2017)
46. Tan, Y., Zhu, Y.: Fireworks algorithm for optimization. In: *International Conference In Swarm Intelligence*, pp. 355–364. (2010)
47. Tanabe, R., Fukunaga, A.: Evaluating the performance of SHADE on CEC 2013 benchmark problems. In: 2013 IEEE Congress on Evolutionary Computation, pp. 1952–1959. (2013)
48. Olivas, F., Valdez, F., Melin, P., Sombra, A., Castillo, O.: Interval type-2 fuzzy logic for dynamic parameter adaptation in a modified gravitational search algorithm. *Inf. Sci.* **476**, 159–175 (2019)
49. Ontiveros, E., Melin, P., Castillo, O.: High order α -planes integration: a new approach to computational cost reduction of General Type-2 Fuzzy Systems. *Eng. Appl. Artif. Intell.* **74**, 186–197 (2018)
50. Castillo, O., Castro, J.R., Melin, P., Rodríguez Dias, A.: Application of interval type-2 fuzzy neural networks in non-linear identification and time series prediction. *Soft Comput.* **18**(6), 1213–1224 (2014)

Hybrid Intelligent Systems

A Comparison of Replacement Operators in Heuristics for CSP Problems



Lucero Ortiz-Aguilar, Valentín Calzada-Ledesma, Juan de Anda-Suárez, Rogelio Bautista-Sánchez, and Natanael Zapata-Gonzalez

Abstract The selection and design of heuristics for metaheuristics and hyperheuristics is an actual problem for the solution of different Benchmarking and real problems. Since there is no methodology or algorithm that can solve all problems at their optimum (this according to the No-Free Lunch theorem), it is necessary to adjust artificial intelligence techniques for solving each problem. Different heuristics have replacement criteria embedded in their process, but there is no well-defined in the state of the art which is the impact of this component on the performance of the heuristic. Therefore, this work shows a comparison of different replacement criteria and through non-parametric statistical tests, we show which of the criteria is the best for each element of a given set of heuristics. The heuristics are applied to a set of Constraints Satisfaction (CSP) problems.

1 Introduction

For many years, some researchers aim was to know which the best algorithm, to be used for a specific instance or set of problems was reported by Rico. Rice [1], proposed how to choose a method from a preselected set of algorithms, achieving effective results for a set of problems. However, designing or selecting an appropriate algorithm for a given problem sometimes requires expert knowledge. Likewise, each problem can have different characteristics or restrictions, which in some cases makes it relatively difficult to propose a good solution. Some well-known constraint satisfaction problems in the state of the art are graph coloring and the Vehicle Routing Problem.

L. Ortiz-Aguilar (✉) · V. Calzada-Ledesma · J. de Anda-Suárez · N. Zapata-Gonzalez
Tecnológico Nacional de México, Instituto Tecnológico Superior de Purísima del Rincón,
Purísima del Rincón, Guanajuato, México
e-mail: lucero.oa@purisima.tecnm.mx; ldm_oa@hotmail.com

R. Bautista-Sánchez
Centro de Ingeniería y Desarrollo Industrial, Apodaca, Mexico

The Vehicle Routing Problem (VRP) has different restrictions such as minimizing distance, time, capacity, and deliveries. This problem aims to find the sub-tour of n cities, without repeating two cities on the same tour or different tours. In the Capacitated Vehicle Routing Problem's (CVRP) state of the art research exist different variants that consider this basic definition and extra restrictions. On the other hand, the Graph Coloring problem is a well know problem, which has been solved by exact methods, heuristics, meta-heuristics, and hyper-heuristics.

In the literature, the exact methods have been applied to both problems in which the restrictions are flexible limited, therefore it is possible to propose a solution in a short period of time before the problem expires. By contrast, the heuristic and meta-heuristic performances depend strongly on their design and parameter tuning. This design in some cases must be ad-hoc to each problem or problem domains.

Some Hyper-heuristics variations depend on the type of learning used, e.g., with online learning [2] and offline learning [3]. Another classification is according to the nature of the heuristics, which can be perturbation and generation. The perturbation heuristics have been applied with a generic methodology in [4], however, it only focuses on knowing what the minimum set of heuristics is and leaving the characteristics of the heuristics as an area of opportunity.

In this work, we focus on improving the design of each heuristic individually for both problems, which will allow improving the design of the Hyperheuristic. Our proposal also provides information that allows us to understand the performance of heuristics and hyper-heuristic in the problem of interest.

The remaining content of this article contains a description of Background in Sect. 2, which covers a review on heuristics, hyper-heuristics, and the constraint satisfaction problem used. The methodology used is show in Sect. 3. The found results and finding including performance comparison are described in Sect. 4. Finally, concluding remarks are presented in Sect. 5.

2 Background

In this Section, we focus to concepts about Constraint satisfaction problem, Acceptance criteria method and heuristic information that we will use in this work.

2.1 Constraint Satisfaction Problems

The Constraint Satisfaction Problem (CSP) was defined formally in the computer science area by [99] especially to imagen processing. In general, a CSP can be defined as a set of variables $V = \{v_1, v_2, \dots, v_n\}$, a set of values $W = \{w_1, w_2, \dots, w_m\}$ and a set of restrictions for each variable denoted as $S = \{s_{v1}, s_{v2}, \dots, s_{vn}\}$. The problem is "What is the assignment of values W for each variable V such all the restrictions are satisfied?" [5]. In this work we used two different problems of CSP: Graph coloring

and Capacitated Vehicle Routing Problem. In this work we used two well-known constraint satisfaction problems Graph coloring and Vehicle Routing Problem.

Graph Coloring Problem (GC-P). This problem consists of labeling each vertex of a given graph G with q colors. The problem must consider the restriction that two adjacent vertexes must be labeled with a different color. The Graph Coloring problem demonstration as NP-Hard problem was proposed in [6]. The benchmark instances can be found in mat.gsia.cmu.edu.¹ In this problem, the only hard constraint is not repeat two adjacent nodes with the same color. Given a graph G with V a set of nodes $v_1, \dots, v_n$ and $E = e_1, \dots, e_m$ edges. The cost of a problem solution is the sum of the nodes connected with the same color as:

$$F(G) = \sum_{i=0}^n \sum_{i=0}^n f(m) \quad (1)$$

$$f(m) = \begin{cases} if(v_i, v_j \in E) \cap (q_{vi} == q_{vj}) & 1 \\ othercase & 0 \end{cases} \quad (2)$$

Vehicle Routing Problem. The Capacitated Vehicle Routing Problem (CVRP) is a variant of VRP [7]. In this problem we have an undirected graph G , m vehicles, Q capacity and a set of Cities $C = \{c_0, c_1, \dots, c_n\}$. Formally the City c_0 is the *depot*, each vehicle this must visit these cities starting from the *depot* and coming back to this.

Given distance or travel time matrix $D = (d_{i,j})$ between cities c_i and c_j , for each city c_i has a demand of things q_i [8] A route $R = \{\vec{r}_0, \vec{r}_1, \dots, \vec{r}_m\}$, and $\vec{r}_i$ is a permutation of the cities starting and finishing at the depot c_0 . The cost of a problem solution is the sum of the costs of each route of R as:

$$F(R) = \sum_{i=1}^m Cost(\vec{r}_i) \quad (3)$$

$$Cost(\vec{r}_i) = \sum_{j=0}^k d_{j,j+1} \quad (4)$$

This problem aims to determine for each vehicle the lowest cost (see Eqs. 3 and 4) tour or distance or travel time, considering the max capacity.

¹ <http://mat.gsia.cmu.edu/COLOR/instances.html>mat.gsia.cmu.edu.

2.2 Heuristics

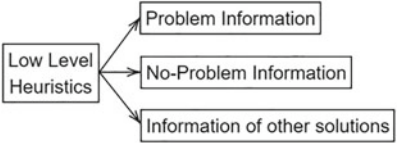
The main component of hyper-heuristics is the pool of low-level heuristics, in this investigation, we used 11 well-known of these studied in the state of art [4].

Perturbative heuristics are called low-level, as they operate directly on the candidate solutions of the underlying optimization problem. According to the information on the problem, these can resolve, we can group the heuristics into three: problem information, no problem information and other solutions information (see Fig. 1).

With the aforementioned low level heuristic aggrupation, we have selected 11 heuristics that belong to the three aforementioned categories. The proposal of this article is to determine the best acceptance criteria for each heuristic. This lets to improve the designing and testing of the heuristic’s algorithms. For all instances, it will be applied k times for each heuristic. The heuristics were applied to the two problems and their respective instances were:

- 1. **k-flip (H_1)**. Its aim is to change the value of one or k variables for another feasible value [9]. For the Graph Coloring, problem k-flip changes the color of a node to another feasible. According to [10] in CVRP their behavior implies change a city to another specific truck.
- 2. **K-Swap (H_2)**. This heuristic selects two or more variables and then interchange their values between them (if this movement is possible), otherwise, the change is not made. For the Graph Coloring Problem, we interchange the colors between two nodes. On CVRP, also is named k-interchange [11].
- 3. **Best Single Perturbation (H_3)**. This heuristic chooses a variable according to the list of restrictions and changes its value for another feasible. This movement produces a solution with better cost or in the worst case, the same cost [40]. The variable selected to be modified is elected according to the next position of the last variable which was chosen (sequentially).
- 4. **Statical Dynamic Perturbation (H_4)**. This heuristic chooses a variable and changes its value randomly [40], which is selecting variables according to a probability distribution of the frequency in the las k iterations. The variables with fewer changes will have a higher probability to be selected [4].
- 5. **Two Points Perturbation (H_5)**. This is a particular case of the K-Swap, with a value of $k = 2$.
- 6. **Double dynamic Perturbation (H_6)**. Double dynamic Perturbation (H_6). This heuristic returns the best solution between an initial solution and a solution modify by Statical Dynamic Perturbation [40].

Fig. 1 Low level heuristic aggrupation



7. **Move to less conflict (H_7)**. This selects a random variable, and it assigns to another feasible value which generates the least cost for the solution [12].
8. **Min-Conflicts (H_8)**. This heuristic chooses a random variable, and it assigns a value that generates the lowest cost [12].
9. **First-Fit (H_9)**. This heuristic changes the value of a variable to other feasible, which is the least repeated in other variables [12].
10. **Worse-fit (H_{10})**. This heuristic assigns the most repeated value if the movement is feasible, to a randomly selected variable [13].
11. **Burke-Abdullah (BA) (H_{11})**. This heuristic selects a variable applying Fail-First or Brelaz Heuristic [14] and its value is changing to another according to the application of the algorithms such as Minimum conflict, Randomly, Sequential selection, and least Constrained and the solution with the best fitness is selected.

2.3 Acceptance Criteria

The study conducted by Warren et al. in [15] is focused on document different acceptance criteria Methods for single-objective local search metaheuristics, in which the move acceptance methods phase is embedded. These methods basically determine if the solution is accepted or rejected and continue with the search process. These methods of move acceptance can be applied to heuristics, that the generic scheme of a heuristic can be seen as the following steps:

- Initial solution or solutions.
- Choice of variable to modify.
- Change value.
- Evaluation the new solution.
- Apply acceptance criteria method.
- Return solution.

These steps are shown in Fig. 2, where we show three colors one for phases that may have more than one operation involved denoted with blue, the replacement criterion phase in red, and the start and finish phases are green.

According to the nature of the acceptance methods in [15] the metaheuristics can be classified into the following groups:

- Nature of accept/reject decision.
 - Stochastic

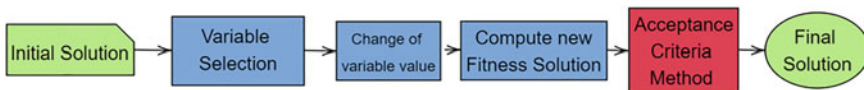


Fig. 2 Heuristic process

- Non-stochastic: Basic and threshold
- Nature of how the algorithmic parameters are set:
 - Memoryless. Static and dynamic
 - Memory. Adaptive.

In this work, in order to improve the stage in red indicated in Fig. 2, we used:

- **Basic (OR)**. If the fitness of the new solution (s_m) is better than the previous one (s_a), then the new solution (S_{new}) is returned. The above is represented as:

$$S_{new} \leftarrow \begin{cases} s_m & f'(s_m) \leq f(s_a) \\ s_a & \text{otherwise} \end{cases} \quad (5)$$

- **Threshold (T)**. Proposed by [16], they define the change of solutions according to:

$$s_{new} \leftarrow \begin{cases} s_m & f'(s_m) \leq f(s_a) + T \\ s_a & \text{otherwise} \end{cases} \quad (6)$$

The parameter T was taken in accordance with those proposed by [15].

- **Great Deluge (GD)**. This criteria was proposed by [17] and belongs to the non-stochastic threshold move acceptance method. Which is defined as:

$$s_{new} \leftarrow \begin{cases} s_m & f'(s_m) \leq \max(f(s_a), \tau_i) \\ s_a & \text{otherwise} \end{cases} \quad (7)$$

$$\tau_i = LowerBound + (f_0 + LowerBound) * \left(1 - \frac{T_{elapsed}}{T_{total}}\right)$$

where f_0 is an initial solution, *LowerBound* is the best know and $T_{elapsed}$ and T_{total} is time in seconds.

3 Methodology

Our methodology for selecting the best acceptance criteria is described as:

- (a) **Step 1**. Select the set of instances to solve. This set of instances must be an optimization problem that can be solved by partitions. This is because a pre-design phase is handled according to the methodologies proposed by [8, 18, 19]. This allows problems such as graph coloring, capacitated routing problems, to be solved using the same heuristics without making ad-hoc modifications.

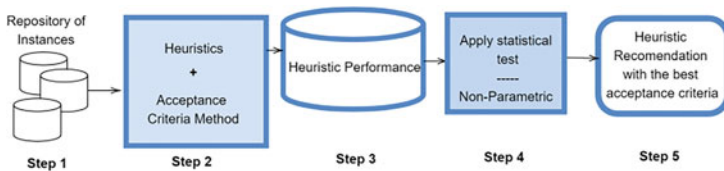


Fig. 3 Methodology of acceptance method for heuristics

- (b) **Step 2.** At least two different types of acceptance criteria are selected, and these are combined with the heuristics. This means that if 3 criteria and 11 heuristics are selected in total, there would be 33 different heuristics.
- (c) **Step 3.** Each heuristic with the different acceptance criteria is applied to set of the instances.
- (d) **Step 4.** With the results of applying the different heuristics with each of the replacement criteria, we apply the non-parametric statistical tests to determine which of them has better performance.
- (e) **Step 5.** The heuristics with the best performance are returned according to the results of the statistical tests.

The previous steps are summarized in the diagram in Fig. 3.

4 Results

This section describes our experiments in detail for Graph Coloring (GC) and Capacitated Vehicle Problem (CVRP) benchmarks used in this paper. We give the configuration for the implementation of the 11 heuristics used. Finally, we described the statistical tests that we used to compare our results of experimental methodology.

Our approach was implemented in JAVA language with JDK 1.8 using the IDE NetBeans IDE 8.2. The experiments were executed on a computer with processor Intel i3-1005G1, 1.19 GHz, 8 GB DDR3 RAM, and operational system Windows 10 Home. The tests presented in this work were executed in a common notebook, with a single processor, it is showing the effectiveness of the exposed methodology.

For each heuristic, a limit of 100,000 function calls was given in each test run, for all instances. We applied the Shapiro Wilks test to check if the data results were normal or not, hence choose a better representative (average or Medium). If the data behavior is according to a normal distribution, the average was taken as representative and otherwise the median.

4.1 Graph Coloring

We used the benchmark proposed for the second DIMACS challenge on Graph Coloring [20] and this is tested with 31 runs. In Tables 1 and 2 we showed our results. We denote the best results with a bold face.

We applied the same procedure with the statistical tests of Friedman (FT), Alienated Friedman (AFT), and Quade (Qt) to distinguish the behavior of heuristics set. We established $\alpha = 0.05$ and $h_0 =$ as there are no differences between the performance of the heuristics and as $h_a =$ as there are differences between the performance of the heuristics. Table 3 shows the ranks obtained in the three statistical tests. We found.

We summarize according to the criteria, which were the heuristics that had the best performance:

- GD: H_1, H_5, H_8 heuristics.
- OR: $H_2, H_3, H_4, H_6, H_9, H_{10}$ and H_{11} heuristics.
- T: H_7 .

Based on the results we can concluded that the best acceptance criteria are Basic (OR) and Great Deluge (GD). There is no best criterion for all heuristics [21], but we can improve the performance of these heuristics by applying a different acceptance criterion. The simplest criterion for the amount of information it needs is Basic, and its performance with respect to GD is very close, so it could be inferred that both

Table 1 Heuristics results for graph coloring instances H_1 to H_5 . Where basic (OR), Threshold (T) and Great Deluge (GD) are the acceptance criteria

	H_1			H_2			H_3			H_4			H_5		
Instance	OR	T	GD	OR	T	GD	OR	T	GD	OR	T	GD	OR	T	GD
2-Insertions_3.col	0	17	0	0	18	0	0	17	0	0	19	0	0	17	0
2-Insertions_4.col	6	132	7	3	135	2	7	136	8	6	137	6	3	136	3
david.col	2	37	2	3	38	3	2	38	2	2	36	2	1	36	1
huck.col	0	26	0	1	28	1	0	28	1	0	27	1	0	27	0
jean.col	1	24	1	1	24	1	1	27	1	1	25	1	1	25	0
mugg100_1.col	0	40	0	0	41	0	0	40	0	0	41	0	0	42	0
mugg100_25.col	0	40	0	0	41	0	0	40	0	0	41	0	0	41	0
mugg88_1.col	0	38	0	0	36	0	0	37	0	0	35	0	0	37	0
mugg88_25.col	0	38	0	0	35	1	0	35	0	0	36	0	0	38	0
myciel3.col	0	6	0	0	4	1	0	5	0	0	5	0	0	5	0
myciel4.col	0	14	0	1	15	1	0	14	0	0	15	0	0	14	0
myciel5.col	1	41	0	1	36	1	0	41	1	0	41	0	0	40	0
myciel6.col	2	106	1	2	106	2	2	103	2	2	112	2	1	108	1
myciel7.col	3	294	3	3	300	5	3	294	3	2	301	3	2	293	3

Table 2 Heuristics results for graph coloring instances H_6 to H_{11} . Where basic (OR), Threshold (T) and Great Deluge (GD) are the acceptance criteria

Instance	H_6			H_7			H_8			H_9			H_{10}			H_{11}		
	OR	T	GD	OR	T	GD	OR	T	GD	OR	T	GD	OR	T	GD	OR	T	GD
2-Insertions_3.col	18	19	19	16	19	18	0	16	0	1	17	2	1	18	2	17	17	18
2-Insertions_4.col	138	133	134	136	135	137	7	137	7	19	135	16	18	136	18	134	134	135
david.col	34	35	38	37	38	38	2	39	2	16	37	17	18	34	17	37	38	37
huck.col	25	26	28	27	27	26	0	28	0	7	27	7	9	28	9	27	27	28
jean.col	26	25	26	24	24	26	1	25	1	6	26	7	7	24	7	25	24	26
mugg100_1.col	43	42	41	42	40	43	0	41	0	2	42	3	2	41	2	40	41	43
mugg100_25.col	41	42	41	41	43	39	0	40	0	2	41	3	3	40	2	42	40	40
mugg88_1.col	35	36	35	34	36	38	0	37	0	1	36	2	2	37	3	34	35	36
mugg88_25.col	38	38	37	35	37	36	0	37	0	2	35	2	2	36	2	34	39	34
myciel3.col	4	5	5	5	5	5	0	5	0	0	4	0	0	4	0	4	5	5
myciel4.col	15	15	14	15	13	15	0	14	0	1	14	1	1	13	1	12	14	14
myciel5.col	39	36	40	41	42	40	1	39	0	5	40	6	6	39	6	37	40	39
myciel6.col	112	109	107	108	106	107	1	108	1	20	108	20	17	107	18	103	110	108
myciel7.col	294	295	296	294	297	297	3	297	2	49	294	52	51	295	51	291	292	296

Table 3 Heuristics ranks per Classes, Friedman (Fr), Aligned Friedman (A-Fr) and Quade (Qu). Where basic (OR), Threshold (T) and Great Deluge (GD) are the acceptance criteria.

	H ₁			H ₂			H ₃			H ₄		
	OR	T	GD	OR	T	GD	OR	T	GD	OR	T	GD
Instance												
Friedman Ranks	6.86	23.71	6.25	8	24.64	9.18	6.68	25.11	7.71	6.04	27.18	6.86
Quade Ranks	6.68	24.75	6.23	8.59	24.61	10.06	6.4	26.12	7.36	6.28	26.54	6.9
Friedman Aligned	1462	4827	1451.5	1506	4755.5	1539.5	1457.5	4816.5	1475	1448.5	4848.5	1461
H ₅												
H ₆												
Instance	OR	T	GD	OR	T	GD	OR	T	GD	OR	T	GD
Friedman Ranks	5.04	25.46	5	25.7	25.61	26.18	24.5	25.71	27	6.68	26.18	5.86
Quade Ranks	5.65	25.29	5.07	24.8	26.01	26.95	24.06	25.43	27.2	6.57	25.65	6.18
Friedman Aligned	1430	4861	1425	4823	4810	4843.5	4800.5	4858.5	4856	1459.5	4823	1446
H ₉												
H ₁₀												
Instance	OR	T	GD	OR	T	GD	OR	T	GD			
Friedman Ranks	13.4	24.39	14.29	14.1	23.36	14.14	20.64	24.11	25.5			
Quade Ranks	12.8	23.95	13.73	13.7	22.8	13.84	20.8	24.35	25.7			
Friedman Aligned	1853	4805	1939	1929	4750	1918	4663.5	4828.5	4784			

can obtain similar fitness. With this context, we could say that, due to the amount of information and its simplicity, we recommend using the Basic criterion.

4.2 Capacitated Vehicle Routing Problem

Three sets of the state of the art were used and tested with 41 runs:

1. Christofides, Mingozzi and Toth (CMT) 14 instances, proposed by [22].
2. Golden, Wasil, Kelly and Chao (GWKC) 20 instances, proposed by [23]

In the Tables 4, 5 and 6 we showed the fitness values for the instances and the lowest city cost tour is indicated in bold.

We applied the same procedure with the statistical tests of Friedman (FT), Alienated Friedman (AFT), and Quade (Qt) to distinguish the behavior of heuristics set. We established $\alpha = 0.05$ and $h_0 =$ as there are no differences between the performance of the heuristics and as $h_a =$ as there are differences between the performance of the heuristics. Table 7 shows the ranks obtained in the three statistical tests for CVRP.

We summarize according to the criteria, which were the heuristics that had the best performance:

- GD: $H_1, H_5, H_6, H_7, H_9, H_{10}, H_{11}$ heuristics.
- OR: $H_4, H_8,$ and H_{11} heuristics.
- T: H_1, H_2, H_6, H_7 and H_{11} heuristics.

Based on the results we can concluded that the best acceptance criteria are Threshold (T) and Great Deluge (GD) for this set instances. In this case, there is some similar performance for all heuristics, for example, H_7 and H_{11} .

5 Discussion and Future Work

In this work, a methodology is proposed to select the acceptance criteria method for low-level heuristics oriented to the solution of instances of different constraint satisfaction problems. The proposal was applied to two different problems well known and studied in the state of the art which was the coloring of graphs and the vehicle routing problem with a specific capacity, GCP and CVRP respectively.

The methodology is focused on select the correct acceptance method to heuristics that can be applied to different constraint satisfaction problems which can be considering as a pool for hyperheuristics. In the application to the GCP and CVRP problems, we can identify which are the better criteria acceptance according to the performance of each heuristic.

The identification and reduction of heuristics to find the solution to complex problems is an optimization strategy that can improve the hyper heuristic design. The

Table 4 Results for CVRP-Capacitated Instances heuristics H_1 to H_4

	H_1				H_2				H_3				H_4			
	OR	GD	T	OR	GD	T	OR	GD	T	OR	GD	T	OR	GD	T	T
CMT1	1,626	1,701	1,701	1,701	1,505	1,611	937	937	940	1,597	1,531	1,602				
CMT2	6,840	6,573	6,791	6,737	6,767	6,513	2,667	2,531	2,629	6,772	6,594	7,071				
CMT3	2,796	2,719	2,691	2,659	2,700	2,747	1,599	1,575	1,712	2,639	2,764	2,703				
CMT4	2,459	2,584	2,529	2,488	2,438	2,350	1,114	1,165	1,101	2,517	2,748	2,631				
CMT5	3,496	3,601	3,628	2,686	2,688	2,732	1,599	1,651	1,556	3,726	3,560	3,733				
CMT6	2,659	2,735	2,775	2,417	2,363	2,379	1,114	1,158	1,090	2,462	2,623	2,609				
CMT7	2,705	2,497	2,441	2,706	2,845	2,722	1,273	1,236	1,288	2,774	2,586	2,831				
CMT8	3,366	3,420	3,406	3,422	3,299	3,397	1,692	1,659	1,659	3,356	3,072	3,307				
CMT9	5,064	5,172	5,093	5,043	4,782	4,998	2,156	2,255	2,217	5,150	5,177	4,922				
CMT10	6,601	6,897	6,990	6,924	6,754	6,647	2,667	2,715	2,685	6,756	6,750	6,731				
CMT11	1,808	1,601	1,808	1,731	1,699	1,678	937	937	886	1,587	1,791	1,784				
CMT12	2,710	2,741	2,641	2,730	2,790	2,601	1,273	1,276	1,276	2,732	2,693	2,674				
CMT13	3,356	3,455	3,358	3,381	3,335	3,275	1,692	1,669	1,594	3,378	3,561	3,348				
CMT14	5,244	5,263	5,224	5,429	5,409	5,254	2,156	2,206	2,202	4,980	5,135	5,203				
GWKC1	26,798	25,771	27,301	24,396	25,467	24,419	5,068	4,659	4,706	27,853	26,303	28,390				
GWKC2	3,250	3,347	3,151	3,192	3,274	3,162	940	949	926	3,343	3,341	3,236				
GWKC3	4,324	4,488	4,261	4,521	4,257	4,546	1,230	1,169	1,187	4,305	4,219	4,622				
GWKC4	5,667	5,927	5,726	5,895	5,749	5,527	1,568	1,494	1,508	5,585	5,433	5,815				
GWKC5	4,077	3,903	4,060	3,937	4,084	3,971	644	686	723	4,037	4,246	4,186				
GWKC6	6,044	5,864	5,865	5,491	5,780	5,579	885	865	842	5,793	5,776	5,717				

(continued)

Table 4 (continued)

	H_1			H_2			H_3			H_4			T
	OR	GD	T	OR	GD	T	OR	GD	T	OR	GD	T	
GWKC7	7,904	7,877	8,217	7,652	7,481	7,936	1,062	1,080	1,118	8,017	8,008	7,814	
GWKC8	10,773	10,719	10,255	9,957	10,183	9,857	1,498	1,437	1,399	10,478	10,049	10,455	
GWKC9	3,317	3,360	3,317	3,216	3,265	3,281	1,209	1,165	1,207	3,411	3,322	3,284	
GWKC10	4,616	4,911	4,717	4,655	4,708	4,774	1,629	1,584	1,552	4,827	4,996	4,937	
GWKC11	6,624	6,650	6,754	6,896	6,530	6,595	2,181	2,189	2,085	6,535	6,663	6,520	
GWKC12	45,638	47,030	45,146	41,249	43,401	42,282	7,654	7,728	7,462	47,474	47,811	45,086	
GWKC13	9,411	9,345	9,074	9,108	8,936	8,715	2,905	2,865	2,831	9,019	9,025	8,985	
GWKC14	69,209	69,511	69,768	61,877	63,002	61,220	11,940	11,695	11,718	68,694	70,141	69,227	
GWKC15	103,497	103,010	103,180	89,594	96,444	94,314	15,844	15,845	16,360	103,571	109,620	107,493	
GWKC16	35,844	36,922	34,901	29,432	31,973	34,465	10,949	10,949	10,949	34,068	36,435	37,105	
GWKC17	49,936	50,598	54,233	44,824	48,638	43,901	12,108	11,808	12,036	52,245	54,472	52,790	
GWKC18	63,565	62,847	65,309	54,984	52,663	53,126	11,090	11,141	11,139	63,931	62,066	65,470	
GWKC19	78,020	76,101	76,753	69,264	68,009	67,427	11,641	11,372	11,757	77,580	74,587	76,188	
GWKC20	2,312	2,378	2,305	2,387	2,444	2,303	750	781	751	2,297	2,463	2,316	

Table 5 Results for CVRP-Capacitated Instances heuristics H_5 to H_8

	H_5				H_6				H_7				H_8			
	OR	GD	T		OR	GD	T		OR	GD	T		OR	GD	T	
CMT1	1,721	1,650	1,698		1,500	1,664	1,664		1,731	1,731	1,731		868	896	893	
CMT2	6,564	6,540	6,650		7,152	6,742	6,742		6,888	6,888	6,888		2,792	2,698	2,563	
CMT3	2,667	2,579	2,728		2,695	2,608	2,608		2,617	2,617	2,617		1,582	1,637	1,804	
CMT4	2,611	2,463	2,612		2,536	2,306	2,306		2,408	2,408	2,408		1,068	1,160	1,159	
CMT5	3,373	3,572	3,552		3,581	2,608	2,608		3,170	3,170	3,170		1,705	1,648	1,722	
CMT6	2,741	2,644	2,700		2,700	2,306	2,306		2,632	2,632	2,632		1,185	1,116	1,138	
CMT7	2,725	2,530	2,542		2,427	2,577	2,577		2,635	2,635	2,635		1,228	1,320	1,302	
CMT8	3,600	3,384	3,547		3,255	3,293	3,293		3,369	3,369	3,369		1,615	1,658	1,654	
CMT9	5,058	4,984	5,186		5,127	5,096	5,096		5,346	5,346	5,346		2,197	2,131	2,257	
CMT10	6,724	7,371	6,486		6,669	6,742	6,742		6,888	6,888	6,888		2,530	2,752	2,796	
CMT11	1,672	1,611	1,626		1,602	1,664	1,664		1,742	1,742	1,742		959	888	910	
CMT12	2,577	2,533	2,867		2,832	2,577	2,577		2,636	2,636	2,636		1,219	1,183	1,230	
CMT13	3,395	3,268	3,697		3,224	3,293	3,293		3,398	3,398	3,398		1,607	1,623	1,687	
CMT14	5,129	5,250	5,119		4,847	5,096	5,096		5,346	5,346	5,346		2,018	2,196	2,163	
GWKC1	27,720	27,653	27,752		26,483	13,312	13,312		16,606	16,606	16,606		5,336	4,885	5,273	
GWKC2	3,057	3,275	3,384		3,362	3,589	3,589		3,589	3,589	3,589		1,042	1,036	985	
GWKC3	4,644	4,407	4,373		4,313	5,040	5,040		5,083	5,083	5,083		1,331	1,363	1,311	
GWKC4	5,654	5,739	5,574		5,592	6,750	6,750		6,809	6,809	6,809		1,776	1,760	1,779	
GWKC5	4,124	4,043	4,224		4,016	2,158	2,158		2,823	2,823	2,823		816	774	798	
GWKC6	5,877	5,625	5,651		5,505	2,748	2,748		3,121	3,121	3,121		1,190	1,243	1,261	

(continued)

Table 5 (continued)

	H ₅			H ₆			H ₇			H ₈		
	OR	GD	T	OR	GD	T	OR	GD	T	OR	GD	T
GWKC7	7,662	7,820	7,853	7,628	3,366	3,366	4,588	4,588	4,588	1,897	2,013	1,790
GWKC8	10,427	10,686	10,232	10,743	4,217	4,217	5,056	5,056	5,056	2,530	2,655	2,662
GWKC9	3,175	3,339	3,479	3,426	2,376	2,376	2,515	2,515	2,515	1,262	1,155	1,112
GWKC10	4,800	4,922	4,735	5,010	3,443	3,443	3,574	3,574	3,574	1,711	1,793	1,737
GWKC11	6,828	6,430	6,450	6,551	4,713	4,713	4,804	4,804	4,804	2,593	2,370	2,520
GWKC12	47,059	45,217	46,103	45,956	21,080	21,080	23,153	23,153	23,153	8,748	9,562	8,834
GWKC13	9,023	9,267	9,121	9,127	6,191	6,191	6,484	6,484	6,484	3,595	3,498	3,627
GWKC14	68,665	71,288	67,033	66,898	26,117	26,117	28,356	28,356	28,356	13,186	13,404	13,441
GWKC15	107,154	105,148	102,132	101,752	38,892	38,892	40,581	40,581	40,581	19,492	19,527	19,368
GWKC16	35,719	34,653	35,126	33,183	21,378	21,378	23,315	23,315	23,315	11,765	11,851	11,561
GWKC17	52,416	52,741	53,869	47,692	26,699	26,699	29,378	29,378	29,378	12,386	12,517	12,364
GWKC18	62,925	63,674	60,662	61,370	25,909	25,909	30,824	30,824	30,824	12,587	12,470	12,801
GWKC19	77,457	77,085	79,181	80,370	31,506	31,506	35,074	35,074	35,074	14,157	13,412	14,289
GWKC20	2,296	2,379	2,342	2,362	2,581	2,581	2,581	2,581	2,581	735	758	758

Table 6 Results for CVRP-Capacitated Instances heuristics H_9 to H_{11}

	H_9			H_{10}			H_{11}		
	OR	GD	T	OR	GD	T	OR	GD	T
CMT1	1,687	1,581	1,743	1,643	1,599	1,670	1,664	1,664	1,664
CMT2	2,625	2,768	2,615	2,647	2,581	2,640	2,577	2,577	2,577
CMT3	3,460	3,339	3,388	3,613	3,399	3,508	3,293	3,293	3,293
CMT4	5,182	5,040	4,553	5,173	5,154	5,203	5,096	5,096	5,096
CMT5	6,603	6,584	5,823	6,410	6,877	6,657	6,742	6,742	6,742
CMT6	1,689	1,593	1,570	1,628	1,631	1,684	1,664	1,664	1,664
CMT7	2,600	2,411	2,749	2,815	2,575	2,612	2,577	2,577	2,577
CMT8	3,007	3,207	2,861	3,202	3,453	3,474	3,293	3,293	3,293
CMT9	5,085	4,746	4,808	5,133	5,106	5,296	5,096	5,096	5,096
CMT10	6,349	6,818	6,453	6,932	6,734	6,719	6,742	6,742	6,742
CMT11	2,687	2,678	2,686	2,755	2,690	2,613	2,608	2,608	2,608
CMT12	2,594	2,606	2,740	2,392	2,555	2,704	2,306	2,306	2,306
CMT13	2,317	2,362	2,137	3,650	3,506	3,555	2,608	2,608	2,608
CMT14	1,990	1,776	1,973	2,854	2,724	2,844	2,306	2,306	2,306
GWKC1	18,242	17,980	18,008	28,317	27,501	27,230	13,312	13,312	13,312
GWKC2	48,960	49,871	44,906	46,710	46,049	44,897	21,080	21,080	21,080
GWKC3	47,633	47,520	46,792	73,505	70,398	66,717	26,117	26,117	26,117
GWKC4	76,427	75,531	72,978	107,273	105,627	106,350	38,892	38,892	38,892
GWKC5	35,500	34,964	36,132	37,669	38,017	38,100	21,378	21,378	21,378
GWKC6	37,501	36,953	36,738	49,293	52,514	52,831	26,699	26,699	26,699
GWKC7	38,698	39,219	38,796	65,811	62,557	62,848	25,909	25,909	25,909
GWKC8	47,746	48,753	48,367	75,543	79,503	78,579	31,506	31,506	31,506
GWKC9	2,346	2,411	2,480	2,238	2,410	2,307	2,581	2,581	2,581
GWKC10	3,261	3,220	3,342	3,326	3,318	3,382	3,589	3,589	3,589
GWKC11	4,359	4,411	4,267	4,294	4,352	4,578	5,040	5,040	5,040
GWKC12	5,718	5,613	5,543	5,815	5,905	5,590	6,750	6,750	6,750
GWKC13	3,492	3,505	3,749	4,210	4,271	4,235	2,158	2,158	2,158
GWKC14	5,898	5,668	6,162	5,605	5,983	6,064	2,748	2,748	2,748
GWKC15	6,800	6,770	6,780	7,992	7,948	8,032	3,366	3,366	3,366
GWKC16	9,231	9,278	9,004	10,195	10,327	10,629	4,217	4,217	4,217
GWKC17	3,409	3,239	3,305	3,436	3,258	3,302	2,376	2,376	2,376
GWKC18	4,464	4,679	4,425	4,807	4,879	4,950	3,443	3,443	3,443
GWKC19	6,744	6,855	6,581	6,736	6,553	6,691	4,713	4,713	4,713
GWKC20	8,419	8,331	8,569	9,291	8,930	9,154	6,191	6,191	6,191

Table 7 Heuristics ranks per Classes, Friedman (Fr), Aligned Friedman (A-Fr) and Quade (Qu). Where basic (OR), Threshold (T) and Great Deluge (GD) are the acceptance criteria

	H_1			H_2			H_3			H_4		
	OR	GD	T	OR	GD	T	OR	GD	T	OR	GD	T
A-FR	2,711.0	1,271.0	1,271.0	1,825.0	2,006.0	1,422.0	2,044.5	1,645.0	1,563.5	1,583.0	1,807.0	1,863.0
Fr	2.6	1.7	1.7	2.1	2.1	1.8	2.1	2.0	1.8	1.9	2.1	2.0
Qu	2.4	1.8	1.8	2.1	2.1	1.8	2.0	2.2	1.8	2.0	2.2	1.8
H_5	H_6			H_7			H_8					
	1,799.0	1,633.0	1,821.0	2,519.0	1,367.0	1,367.0	1,751.0	1,751.0	1,751.0	1,589.0	1,773.0	1,891.0
	2.1	1.8	2.1	2.4	1.8	1.8	2.0	2.0	2.0	1.8	2.0	2.1
	2.1	1.7	2.2	2.0	2.0	2.0	2.0	2.0	2.0	1.8	2.0	2.2
H_9	H_{10}			H_{11}								
	2,001.0	1,841.0	1,411.0	1,782.0	1,605.0	1,866.0	1,751.0	1,751.0	1,751.0			
	2.2	1.9	1.8	2.1	1.7	2.2	2.0	2.0	2.0			
	2.3	1.8	1.9	2.1	1.6	2.3	2.0	2.0	2.0			

methodology presented allows generating a better selection of heuristic components and impact in the level of generality that can be used to solve different problems of satisfaction of constraints simultaneously under the hyperheuristic approach. Finally, the methodology makes it possible to improve the search for solutions to sets of problems by exploring with the diversification of some components described in Sect. 2.3.

References

1. Rice, J.R.: The algorithm selection problem. In: *Advances in Computers*, pp. 65–118. Elsevier (1976)
2. Pillay, N.: A review of hyper-heuristic for educational timetabling. Springer (2014)
3. Yates, W.B., Keedwell, E.C.: An analysis of heuristic subsequences for offline hyper-heuristic learning. *J. Heurist.* **25**(3), 399–430 (2019)
4. Ortiz-Aguilar, L., Carpio, M., Rojas-Domínguez, A., Ornelas-Rodriguez, M., Puga-Soberanes, H.J., Soria-Alcaraz, J.A.: A methodology to determine the subset of heuristics for hyperheuristics through metalearning for solving graph coloring and capacitated vehicle routing problems. *Complexity* (2021)
5. Jeavons, P., Cohen, D., Cooper, M.C.: Constraints, consistency and closure. *Artif. Intell.* **101**(1), 251–265 (1998)
6. Karp, R.: *Complexity of Computer Computations*. Springer Us, Boston, MA (1972)
7. Dantzig, G.B., Ramser, J.H.: The truck dispatching problem. *Manage. Sci.* **6**(1), 80–91 (1959)
8. Alba, E., Dorronsoro, B.: Computing nine new best-so-far solutions for capacitated vrp with a cellular genetic algorithm. *Inf. Process. Lett.* **98**(6), 225–230 (2006)
9. de Werra, D.: Heuristics for graph coloring. In: *Computational Graph Theory*, pp. 191–208. Springer (1990)
10. Helsgaun, K.: General k-opt submoves for the lin–kernighan tsp heuristic. *Math. Program. Comput.* **1**(2–3), 119–163 (2009)
11. Derbel, H., Jarbouli, B., Bhiri, R.: A skewed general variable neighbourhood search algorithm with fixed threshold for the heterogeneous fleet vehicle routing problem. *Ann. Oper. Res.* **272**(1–2), 243–272 (2019)
12. Soria-Alcaraz, J.A., Özcan, E., Swan, J., et al.: Iterated local search using an add and delete hyper-heuristic for university course timetabling. *Appl. Soft Comput.* **40**, 581–593 (2016)
13. Habashi, S.S., Salama, C., Yousef, A.H., et al.: Adaptive diversifying hyper-heuristic based approach for timetabling problems. In: *2018 IEEE 9th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)*, pp. 259–266. IEEE, (2018)
14. Gent, I.P., Ewan, M., Patrick, P., et al.: An empirical study of dynamic variable ordering heuristics for the CSP. In: Freuder, E. (ed.) *Principles and Practice of Constraint Programming CP96*. Lecture Notes in Computer Science, vol. 1118, pp. 179–193. Springer, Berlin, Heidelberg (1996)
15. Jackson, W.G., Özcan, E., John, R.I.: Move acceptance in local search metaheuristics for cross-domain search. *Expert Syst. Appl.* **109**, 131–151 (2018)
16. Dueck, G., Scheuer, T.: Threshold accepting: a general purpose optimization algorithm appearing superior to simulated annealing. *J. Comput. Phys.* **90**(1), 161–175 (1990)
17. Dueck, G.: New optimization heuristics: The great deluge algorithm and the record-to-record travel. *J. Comput. Phys.* **104**(1), 86–92 (1993)
18. Ortiz-Aguilar, L.D.M., Carpio, M., Soria-Alcaraz, J.A., Puga, H., Díaz, C., Lino, C., Tapia, V.: Training OFF-Line Hyperheuristics For Course Timetabling Using K-Folds Cross Validation. In: *la revista programación matemática y software*. (2016)

19. Soria-Alcaraz, J.A., Ochoa, G., Carpio, M., et al.: Effective learning hyper-heuristics for the course timetabling problem. *Eur. J. Operat. Res.* 10 (2014)
20. Leighton, F.T.: A graph coloring algorithm for large scheduling problems. *J. Res. Natl. Bur. Stand.* **84**(6), 489–506 (1979)
21. Wolpert, D.H., Macready, W.G.: No free lunch theorems for optimization. *IEEE Trans. Evol. Comput.* **1**(1), 67–82 (1997)
22. Christofides, N.: The vehicle routing problem. In: christofides, N., Mingozzi, A., Toth, P., Sandi, C. (eds.) *Combined optimization*. (1979)
23. Golden, B.L., Wasil, E.A., Kelly, J.P., et al.: The impact of metaheuristics on solving the vehicle routing problem: algorithms, problem sets, and computational results. In: *Fleet Management and Logistics*, pp. 33–56. Springer (1998).
24. Ortiz-Aguilar, L.D.M., Carpio, M., Puga, H., Soria-Alcaraz, J.A., Ornelas-Rodríguez, M., Lino, C.: Increase methodology of design of course timetabling problem for students, classrooms, and teachers. In: *Nature-Inspired Design of Hybrid Intelligent Systems*, pp. 713–728. Springer, Cham (2017)

Synchronisms Using Reinforcement Learning as an Heuristic



Omar Zamarrón, Mauricio A. Sanchez, and Antonio Rodríguez-Díaz

Abstract The concept of Synchronism describes circumstances that appear meaningfully related yet nevertheless lack a casual connection. Every day we make use of this concept, from forming new relationships to grooming a new set of skills. Therefore one day in the future we will be able to make use of them for a specific goal. Reinforcement Learning (RL) is a type of learning framework where the solution is presented as a Markov Decision Process (MDP) where the goal is to learn by trial and error, emulating how the human begins to learn. With this type of framework, we can solve very complex non-linear and not deterministic problems. In this novel method, we train the model using a non-deterministic RL and try to find the causal link between those circumstances by observation. This is known as the “Keen Eye” where each circumstance finds a consensus link affecting the outcome using the knowledge from each circumstance involved. We trained agents in two experiments with different domains, the first experiment tries to solve a linear non-deterministic problem, while the second is a non-linear and non-deterministic simulation. With this type of framework, the most common metric is to gather the reward value obtained from the outcome of the experiments, therefore we use this value to measure how performance behaves. Finally, with the results of experiments we discovered that compared with traditional RL, this method can train RL models faster, by obtaining better rewards earlier and makes the RL less susceptible to fall into a local minimum.

Keywords Agents · Reinforcement learning · Deep learning · Machine learning

O. Zamarrón (✉) · M. A. Sanchez · A. Rodríguez-Díaz
Universidad Autónoma de Baja California, Tijuana, Baja California, México
e-mail: omar.zamarron@uabc.edu.mx

M. A. Sanchez
e-mail: mauricio.sanchez@uabc.edu.mx

A. Rodríguez-Díaz
e-mail: ardiaz@uabc.edu.mx

Calzada Universidad, Guadalajara 14418, México

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2022
O. Castillo and P. Melin (eds.), *New Perspectives on Hybrid Intelligent System Design based on Fuzzy Logic, Neural Networks and Metaheuristics*, Studies in Computational Intelligence 1050, https://doi.org/10.1007/978-3-031-08266-5_23

1 Introduction

At the beginning of the *XX* century commenced the formal study of human civilization and society with the help of technology, in particular, computers make it possible be able to automate data processing therefore, social-scientist started to observe and propose models trying to emulate how humans behave and how society can react to certain events [1].

At the half of the *XX* century scientist proposed in using virtual worlds to model social behavior and similar kinds of studies, One of the methods whose origins started with the observation of humans and animals called reinforcement learning, it open the door to three lines of investigation.

The first line of investigation is where the philosophy of the method resides it concerns learning by trial and error and started in the psychology of animal learning, this line of investigation started some of the earliest work in artificial intelligent and provoked reinforcement learning to be consider as an alternative method of artificial intelligent in the early 1980 [2].

The other line concerns the problem of optimal control and its solution using dynamic programming, the term “optimal control” it begin to make appearance at the end of 1950 to describe problems regarding designing a controller to minimize a measure of an dynamic environment behavior over time, one of the solutions was developed by Richard Bellman, using concepts of dynamics systems states and of a value functions better know as “optimal return function” defined as a functional equation this is often called the Bellman equation, the methods that can resolve this kind of equations are know as dynamic programming, this kind of problems has a stochastic version of the optimal control problem this is know as the Markovian decision process (MDP) in 1960 Ron Howard come up with a iteration method, on the present time is know as the policy. All of this lines of investigation are essential elements to form theory and algorithms of modern reinforcement learning [3–5].

In the present time, the third line of investigation is combining the other ones and make use of modern techniques such as machine learning and deep learning. Machine learning in part is inspired by how the cells of the brain interact, this model was created by Donald Hebb in 1949 creating the theory around neurons excitement and communication [6], around 1957 Fran Rosenblatt combined the Hebb model and planned to create a machine learning not a program otherwise a custom machine called Mark 1 perception whose had been constructed for image recognition. In the late 1970 and early 1980, the research on neural networks was abandoned by computer science and AI researchers, this caused that during this time machine learning industry focus primarily on neural networks going the different path of Artificial intelligence where the main focus was probability theory and statistics. With the rise of computers been more accessible in the 1990 Machine learning begin to flourish, this successful event is attributed to the result of internet growth, benefiting from the spreading availability of digital data.

Deep learning start to making an appearance around 1999 when computers started becoming faster at processing data and GPU where begin developed, originating

possible to train convolutional neural networks without the layer by layer pre training those it became apparent it has significant advantages in terms of efficiency and speed, in current time the processing of big data and the evolution of artificial intelligence are dependent on deep learning were is still evolving [intro7].

2 Background

2.1 Synchronisms

The term synchronism is attributed to Swiss psychologist and psychoanalyst Carl Jung, in 1960 he defined synchronicity as “a casual connection, meaningful coincidence, a casual parallelism or meaningful coincidence of two or more events where something other than the probability of chance is involved” [7].

He believed that the events marked as coincidence were not due to the chance of happening instead, he gives credence by contrast that these kinds of events are connected directly to a specific “observer’s mind” or keen eye, which will provide powerful insight and guidance.

Jung gives an example of one of his woman patients who were, “psychologically inaccessible” that is, rational and logical to an extreme, after several attempts Jung could not get her to open up about her emotional life, she sealed herself with her intellect and was cold and rigid as a result. The woman told Jung about a dream she had the night before, in which someone had given her a piece of jewelry in the shape of a golden scarab beetle. While she was telling him this dream, Jung heard a tapping sound on the window. He opened the window and caught a “fairly large flying insect”, as it flew in, which turned out to be a scarab beetle with a gold-green color that resembled the one which the woman dreamed of, Jung handed the beetle to the woman and said, “Here is your scarab” [8].

Jung remarks that Synchronicity is not merely the intuitive meaning to questionable events, rather it implies that such events have meaning by themselves.

2.2 Agents

In the formal literature, an intelligent agent is defined as “an entity capable of processing, perceiving and be able to rationally respond to the environment with the help of sensors it can perceive, also with the help of actuators the agent can interact with the environment” [9].

In the field of agents, it exists a large number of variations on how to define an agent model. in the 20th century, around 1987, a philosopher named Bratman created a model called the BDI (Believes, Desires, Intentions) see the model on Fig. 1, this model is one of the most used in the present time [10].

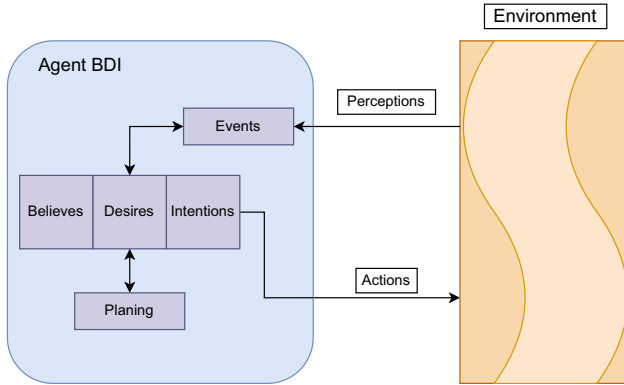


Fig. 1 BDI agent model

Using set theory, we can define the properties and attributes of an agent, for example, an agent that has rules, vision, and actuators.

$$Agent = \{rules, vision, actuators\} \quad (1)$$

On Eq. 1 it can be said an agent has a systems of rules, a vision system and the actuators of an agent can be view as a function that can map every possible percept sequence to a possible action the agent can perform or to a coefficient, feedback element, a function, or a constant that affects the eventual actions [11], the actuators can be mapped by a function show in Eq. 2.

$$f : P^* \rightarrow A \quad (2)$$

The design of an agent heavily depends on the dynamics that emerge with the interactions with the environment, the result of a good design gives a complex and explainable agent, instead of a chaotic over qualified agent.

2.3 Machine Learning

Machine learning is often referred to as a machine learning algorithm, which is an algorithm that is capable to learn from data. In 1997 a computer scientist Tom Mitchell provides a concise definition “A computer program is said to learn from experience E with respect to some class of task T and performance measure P, if its performance at a task in T, as measured by P, improves with experience E” [12].

Machine learning came to revolutionized on how the data take a major role in solving complex problems it gives many advances not only in the area of computer science yet it goes beyond to other areas [13].

2.4 Deep Learning

In the literature, deep learning is defined as a specific kind of machine learning [14]. The techniques of machine learning depend heavily on human involvement in the effort to carefully design features for the raw data, giving more digestible data for the algorithm to learn. Deep learning methods are capable of dealing with unchanged, un-manipulated raw data through multiple levels of representation or abstraction, each representation doesn't have human intervention, the model itself automatically discovers [15].

2.5 Reinforcement Learning

Reinforcement learning has its origins around the 20th century between the years 1950–1960 was commonly used for describing problems trying to find the optimal criterion and what kind of control rules lead to it [16]. Optimal control is typically defined as a set of differential equations whose point towards the minimum values of a function error.

R.L. historically Emerge from the combination of the studies on how humans and animals learn using a stimulus in other words by try and error approach, and the development on control problems optimization [17].

This kind of learning is under the category of unsupervised machine learning and is considered a framework as we can see in Fig. 2.

The reward is an analogy on how the stimulus affects the agent, it acts as a motivator while the agent goes from one state to another, the better rated the action that the agent performs is, the greater the reward it receives which leads to an optimal crite-

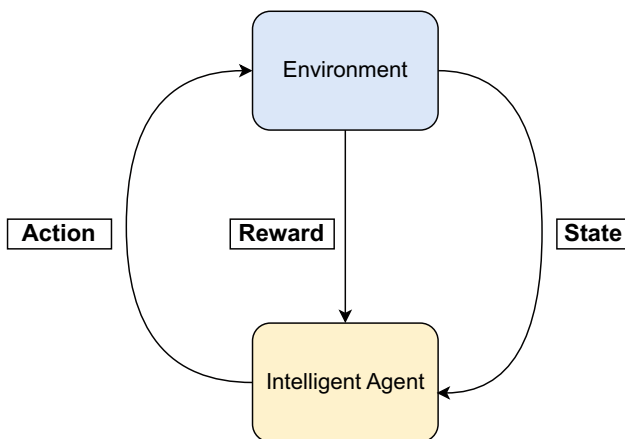


Fig. 2 Reinforcement learning framework

tion action performed. The agents are the principal focus of reinforcement learning because they perceive, associate, and interacts with the environment that is why the environment has to abide by the following characteristics [18].

- It has to be finite and observable.
- It can be deterministic or non-deterministic.
- It can use discrete or continuous data.
- It can have one agent or multiple agents.

2.6 Q-Learning

Q-learning is commonly used as the loss function of a deterministic environment, Reinforcement learning work under the instance of the Markov Decision Process (MDP) where the sets of states are finite, the sets of actions of each state are finite and the environment has to satisfy the following properties.

$$v(s, \pi) = \sum_{t=0}^{\infty} \gamma^t E(r_t | \pi, s_0 = s) \quad (3)$$

In other words Eq. 3 tells the probability of reaching the state s' of the state s by an action a is independent of other states or actions in the past t [19].

Q-learning is a type of temporal differentiable learning by combining elements of dynamic programming with Markov Decision Process. Stone [20], in MDP an agent objective is to find a strategy π and to maximize the sums of the discounted expected reward. To solve this kind of problem there is a iterative search method know as Putterman, this method searches a fixed point on the Bellman equation

$$v(s, \pi^*) = \max_a \left\{ r(s, a) + \gamma \sum_{s'} p(s'|s, a) v(s', \pi^*) \right\} \quad (4)$$

where $r(s, a)$ is the reward for taking action a at state s , s' is the next state, and $p(s'|s, a)$ is the probability of transiting to state s' after taking action a in state s .

The idea of Q-learning is that we can define a function Q where we can maximize the discount factor, $v(s, \pi)$ is and optimal function that has the highest values of all states given an action, there for this method is used as a loss function

$$Q^*(s, a) = r(s, a) + \gamma \sum_{s'} P(s'|s, a) v(s', \pi^*)$$

$$v(s, \pi^*) = \max_a Q^*(s, a) \quad (5)$$

$$Q_{t+1}(s_t, a_t) = (1 - \alpha_t) Q_t(s_t, a_t) + \alpha_t \left[r_t + \gamma \max_a Q_t(s_{t+1}, a) \right] \quad (6)$$

There is a relationship between the to optimal functions $Q^*(s, a)$ and $v(s, \pi)$ given in Eq. 5 meaning the maximum expected total reward when starting at s is the maximum of all possible actions, using $Q^*(s, a)$ we can extract the optimal policy π^* by choosing the action a that gives maximum reward for state s . Q-learning provides a algorithm procedure where the agents can start with random $Q(s, a)$ values for all $s \in S, a \in A$, and update the Q values with the Eq. 6 in other words, Q-learning tries estimate the value of an action that the agent took, and every update will try to make the best possible action with the highest value of Q [21].

3 Related Work

The use of Deep Learning provides a state-of-the-art solution to big and complex problems, integrating D.L. in reinforcement learning has become the norm [22, 23], creating a hybrid system with sufficient tools capable of finding the generalized form of the problems to solve.

Our approach does not rely on pixels inputs, can work with vectorized raw data, and can be suitable in a large wide variety of gamma of problems/applications.

3.1 Most Common Methods of Deep Reinforcement Learning

The most common area in the recent time in the context of applying of Deep Learning is in video games, where tries to play different types such like First Persons Shooters (FPS), arcade games, platforms games, Role Playing Games (RPG) and Real Time Strategy Games (RTS), The latter been the most influential kind of this kind of learning with AlphaGo [24].

Is important to understand what kind of inputs and training, the methods can handle, we can se a brief and concise information in the following Table 1 [22].

3.2 Real Life Applications of Deep Reinforcement Learning

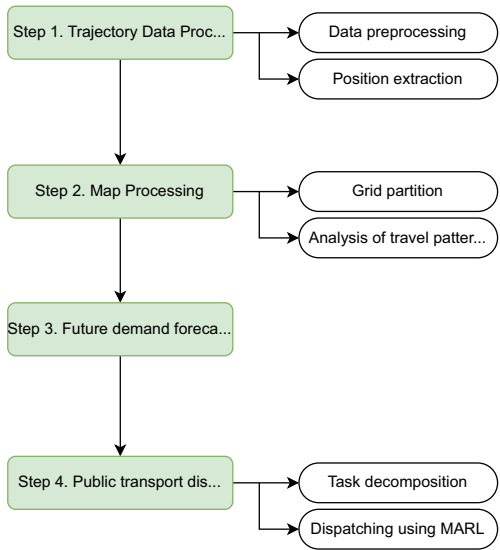
Finding optimal and mining trajectories has been a hot topic in the process of creating a smart city [25].

Interesting research assembles a multiagent reinforcement learning to tackle this and other problems that emerge from begetting a smart city. Technology brew the arise of improving people's living standards, the demand of traveling by public transport increased, usually the public transport system is outdated and drivers usually rely on their operational experience or cruise randomly to find passengers [26], the pipeline they proposed is define in Fig. 3.

Table 1 Overview of Deep Learning methods using R.L

Method	Input			Architecture			Training					
	Feature	Pixels	Text	CNN	RNN	Ext.Mem	MLP	Supervised	Q-learning	Actor-Critic	ES	GA
DQN	o	●	o	●	o	o	o	o	●	o	o	o
DRQN	o	●	o	●	●	o	o	o	●	o	o	o
Gorila	o	●	o	●	o	o	o	o	●	o	o	o
A3C/A2C	o	●	o	●	●	o	o	o	o	●	o	o
Progressive Networks	o	●	o	●	o	o	o	o	o	●	o	o
UNREAL	o	●	o	●	●	o	o	o	o	●	o	o
Distributional DQN	o	●	o	●	o	o	o	o	●	o	o	o
ACKTR	o	●	o	●	o	o	o	o	o	●	o	o
Deep GA	o	●	o	●	o	o	o	o	o	o	o	●
IMPALA	o	●	o	●	o	o	o	o	o	●	o	o
Ape-X DQN	o	●	o	●	o	o	o	o	●	o	o	o
Zero Order	●	o	o	o	o	o	●	o	o	o	o	o
COMA	●	o	o	o	●	o	o	o	o	●	o	o
LSTM-DQN	o	o	●	o	●	o	o	o	●	o	o	o
DRRN	o	o	●	o	o	o	●	o	●	o	o	o
Golovin	o	o	●	o	●	o	o	o	o	o	o	o

Fig. 3 Pipeline dispatching public transport system



4 Using Synchronisms as an Heuristic for RL

The idea of the proposed model is to take advantage of the concept of synchronisms, in daily life humans make use of this concept without been conscious about it from honing new set new skills to forming new relationships therefore one day in the future for a specific situation they can be used [27].

The model is divided in two phases for training, phase one involve on a set of individuals agents learning to solve the environment using Reinforcement learning with Temporal Difference Learning (TDL) in combination with a gradient policy, this kind of hybrid systems is know as Actor-Crit Method [28].

The second phase involves merging each of the agent’s knowledge and updating each agent parameters, this phase is known as the keen eye, or in other words is the training phase of the meta-agent where the objective is to boost or improve agents that are below average in the performance score, herein agent with better performance scores contribute more to the updates of the parameters.

Figure 4 Describe the general view of the RSL model, where a group of agents work/learn independently trying to solve the environment that there are interacting, in a determinate time a meta-agent also know as the keen eye will give guidance to the agents on the feedback with respect of the knowledge of all the agents.

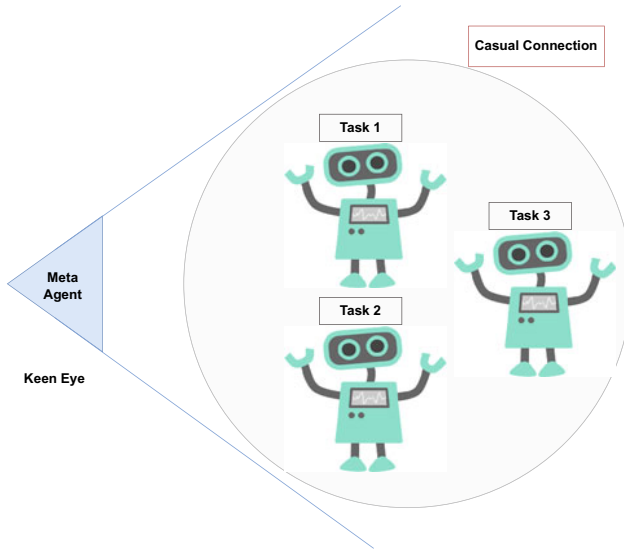


Fig. 4 SRL diagram model

4.1 *RL Model Using Gradient Descent*

An alternative for working with no deterministic environment or high dimension space environments in reinforcement learning is possible to use a Gradient Descent method, this method is called the gradient policy method.

The policy is the probability distribution, for a given state there is a probability the agent selecting a action. Said policy is represented by π and the parameters of π are represented by θ those it can be said that is a function of θ and represented as $J(\theta)$.

$$\pi \rightarrow \theta \therefore J(\theta) \quad (7)$$

The policy π is a function of some weights θ there be for the policy tries to maximize $J(\theta)$ by using the gradient ascent $\nabla J(\theta)$.

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} J(\theta_t) \quad (8)$$

All methods that utilize Eq. 8 are called Policy Gradient Methods, the policy in this kind of method is continuous and finite, this enables it to work with environments of continuous actions spaces.

The explore/exploit dilemma on gradient-based methods are not an issue because it guaranteed that all actions get a sample at least some of the time and the agent can continue to build the model of the environment, this can be accomplish by using a numerical preference of a state and action pair defined as $h(s, a, \theta)$ for each action/state.

For the agent to be able to chose actions, we can use a exponential distribution function such as softmax

$$\pi(a|s, \theta) = \frac{e^{h(s,a,\theta)}}{\sum_b e^{h(s,a,b,\theta)}} \quad (9)$$

The value of $\sum_b e^{h(s,a,b,\theta)}$ is the sum of all possible actions and has to add to 1.

There are some advantageous boons and disadvantages for using this kind of methods, the boons are the following

- The policy is a probability distribution.
- The policy is a continuous and finite function.
- Work with more complex continuous spaces.

The downsides

- Sample inefficiency, the only that matters is one before or after.
- Credit assignment problem, it needs a large number of episodes to converge.
- Unstable solutions, depends strongly on the hyper-parameters.

4.2 Agent Learning Model

In our agent model, we call and use the function $J(\theta)$ to define the performance of our agent

$$J(\theta) \stackrel{\text{def}}{=} v_{\pi_\theta}(s_0) \quad (10)$$

$\nabla J(\theta)$ depends on the distribution of states encountered, because $J(\theta)$ is a complex mathematical function, is difficult to calculate $\nabla J(\theta)$, there before in this case there is an approximation using the Policy Gradient Theorem and is defined by.

$$\nabla J(\theta) \propto \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla_\theta \pi(a|s, \theta) \quad (11)$$

Equation 11 Gives a relationship between the gradient of $\nabla J(\theta)$ and known quantities of experience by being proportional to the distributions of states that turns into an expectation value $\sum_s \mu(s)$ multiply by the sum of all the value actions given a state for the policy π , $\sum_a q_\pi(s, a)$ multiply by the gradient of the policy with respect to θ , $\nabla_\theta \pi(a|s, \theta)$

$$\nabla J(\theta) = E_\pi \left[\sum_a q_\pi(s_t, a) \nabla_\theta \pi(a|s_t, \theta) \right]$$

$$\nabla J(\theta) = E_\pi \left[\sum_a \pi(a|s_t, \theta) q_\pi(s_t, a) \frac{\nabla_\theta \pi(a|s_t, \theta)}{\pi(a|s_t, \theta)} \right] \quad (12)$$

$$\nabla J(\theta) = E_\pi \left[q_\pi(s_t, A_t) \frac{\nabla_\theta \pi(A_t|S_t, \theta)}{\pi(A_t|S_t, \theta)} \right] \quad (13)$$

Because $\sum_s \mu(s_t)$ is the expectation value of the policy π we can change it to E_π on Eq. 12, then we multiply and divide by de policy to get rid of $\sum_a \pi(a|s_t, \theta)$ on Eq. 13 yielding the expectation value of policy π , multiply by the expected value of the action value function with the state S_t and action A_t on time t multiply by the gradient of the policy for action A and state S on time t , divided by the policy on action and state on time $\frac{\nabla_\theta \pi(A_t|S_t, \theta)}{\pi(A_t|S_t, \theta)}$, because $\sum_a \pi(a|s_t, \theta)$ is an expectation value of the action function value by simplifying we can said that is precisely the expectation values of the returns.

$$\nabla J(\theta) = E_\pi \left[G_t \frac{\nabla_\theta \pi(A_t|S_t, \theta)}{\pi(A_t|S_t, \theta)} \right] \quad (14)$$

Where G_t is the weights of the gradient, we try to maximize the probability of the return G by tilting the gradient provoking the agent to do an action A on state S with respect θ also $\pi(A_t|S_t, \theta)$ its there to prevent sampling bias.

Using the following equivalent we can further more simplify the Eq. 14

$$\begin{aligned} \frac{\nabla x}{x} &= \nabla \ln x \\ \nabla_\theta J(\theta) &= E_\pi \left[G_t \nabla_\theta \ln \pi(A_t|S_t, \theta) \right] \end{aligned} \quad (15)$$

This implies that the updates for the value of the weights θ_t are the following

$$\theta_{t+1} = \theta_t + \alpha G_t \nabla_\theta \ln \pi(A_t|S_t, \theta) \quad (16)$$

The Eq. 16 above-mentioned is the same as Eq. 8 of the Gradient Descent Method.

Our agent will learn each time step also refer as Temporal Difference Learning (TDL) and our policy is a gradient base one, the combination of TDL and gradient policy is know as the Actor-Critic learning.

This learning method has two neural networks the actor-network generates actions to get rewards and the critic network will evaluate the actions generated from the actor-network (Fig. 5).

In our case, we will use a variant of the Actor-Critic architecture, where instead of using two separate neural networks we will use one but the hidden layer will be

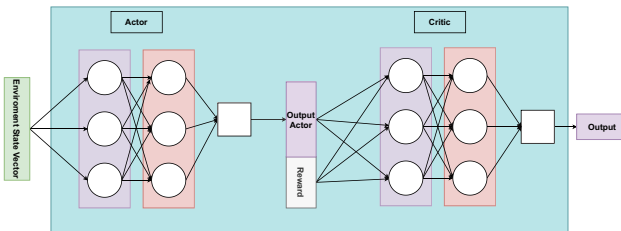


Fig. 5 Actor-critic network architecture

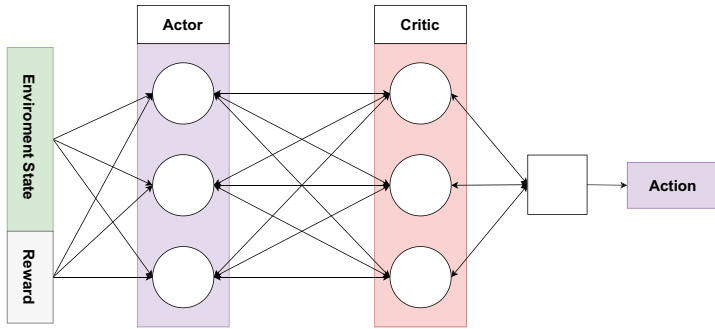


Fig. 6 Actor-critic with a single network

an Actor hidden layer and a Critic hidden layer they will share the same input layers with distinct outputs, this way is simpler than having two distinct networks (Fig. 6).

The network will calculate $V(S_t)$, the critic estimate of the value of the current state S in time t and $V(S_{t+1})$ the critic estimate of the value of the state S in the next step time $t + 1$

$$V(s) = E_{\pi} \left[G_t | S_t = s \right] \quad (17)$$

The State Value is defined as the expected returns given a state following the policy π_{θ} , we used this to compute the loss functions of the actor layer

$$\delta = R_t + \gamma V(S_{t+1}, w) - V(S_t, w) \quad (18)$$

This baseline equation will replace G_t on the Eq. 16, δ will tilt the weights towards a more profitable actions those leaving the gradient policy as

$$\nabla_w J(w) = E_{\pi} \left[(R_t + \gamma V(S_{t+1}, w) - V(S_t, w)) \nabla \ln \pi(A_t | S_t, w) \right] \quad (19)$$

The Critic layer is interested in improving the accuracy of the estimates values towards the true values of the current state by minimizing the total residual of the sum of squares therefor the loss functions to use is δ^2 and the update of weights w .

$$J(w) = \frac{1}{2} \left(R_t + \gamma V(S_{t+1}, w) - V(S_t, w) \right)^2$$

$$\nabla_w J(w) = \left(R_t + \gamma V(S_{t+1}, w) - V(S_t, w) \right) \quad (20)$$

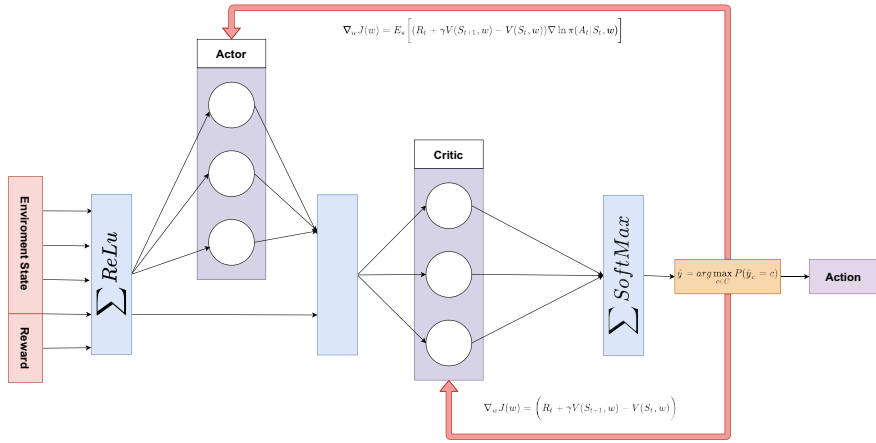


Fig. 7 The learning module of the agent

Figure 7 is the representation of the final diagram consisting of the neural network architecture for the individual learning model the agent possesses and the Algorithm 1 introduces the steps of how the individual agent learns using TDL.

The input data of the agent is a reward and a vector of the changes that happened in the environment commonly referred to as the state, each time step the agent will make an action, with each action taken the environment will respond with all the changes that happened, yielding a reward, the agent will take that data and learn from it. This interaction is described on Fig. 8.

Algorithm 1: Individual Agent learning steps

```

Initialize actor and critic weights  $\theta, w$ ;
for Large number of episodes do
  reset environment;
  for each step of the episode do
    Select action according to the policy  $\pi$  of the actor layer;
    Take action and receive reward, new state from the critic layer;
    Compute  $\delta = R_t + \gamma V(S_{t+1}, w) - V(S_t, w)$ ;
    Compute Actor layer loss  $\delta$ ;
    Compute Critic layer loss  $\delta^2$ ;
    Backpropagate();
  end
end

```

The hyperparameters of the model are the learning rate lr where we recommend a small value between $lr \in [1e-5, 0.1]$, and the count of neurons in each layer where the actor layer has to have more neurons than the critic layer, the recommended values are multiples of 2; $value \in [2^8, 2^{12}]$.

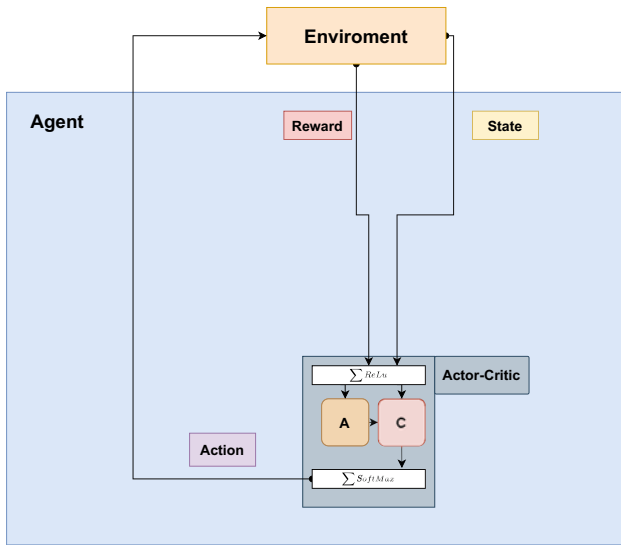


Fig. 8 SRL individual agent model

4.3 Keen Eye

The definition of keen eye talks about an unusual ability to notice or recognize a specific thing or situation [29], Carl Jung portrayed it as an observer's mind providing powerful insight and guidance to explain a relationship between two events that could not be interpreted however there is a probability of happening [30].

Taking into account Carl Jung's definition of the keen eye, our model constructs a Meta-Agent that tries to guide and give insight to all the agents that belong to the same spatial domain but a level lower than the Meta-Agent.

The meta agent tries to find a casual connection of the knowledge of every agent that belongs to the same spatial domain, this connection is a consensus by aggregating all the same attribute appears several times in the knowledge [31] of each agent.

The proposed model measures consensus by using the weighted average of the Knowledge of each agent involve and a bias vector defined in Eq. 21, this measure can be easily replaced with another metric such as:

- Sum of Product
- Weighted Sum
- A Neural Network

The meta agent observes each knowledge of all the agents W and makes a weighted average, considering more the weights of agents that got better performance score X , the result is the keen eye guidance for updating the weights of each agent (Fig. 9).

$$Ke = \frac{\sum_{a=1}^n X_a W_a}{\sum W} \quad (21)$$

Algorithm 2 introduce the details on how the meta-agent updates the weights of all the individual agents.

Algorithm 2: Meta-Agent guidance

```

Get all the agents belonging in the same space as the meta-agent
Initialize  $X$  and normalize with respect of the performance score of all agents
if  $X$  is None then
  | Initialize  $X$  with 1
end
for each agent belonging to meta-agent do
  | get  $W_a$ 
  | compute  $Ke = \frac{X_a W_a}{\sum W}$ 
  | update  $W \leftarrow Ke$ 
end

```

The meta-agent only has one hyper-parameter wish is the performance score of all agents, if the parameter X is null or none the meta-agent will take into account all agents with the same weight otherwise the better the performance the agent has the more the meta-agent take it in consideration for the consensus.

This update occurs every P time steps the agents take, the analogy is that all agents gather to discuss the knowledge they acquire during P interactions with the environment, therefore, the meta-agent imposes guidance intending to improve the performance of the agents with the lower performance score.

5 Study Case: Cartpole

This environment corresponds to the version of the cart-pole problem described by Barto, Sutton, and Anderson, [32], Fig. 10 is the schematic representation of the top of a cart to which a rigid pole is attached.

The cart can freely move within the bounds of the one dimension track (left or right), the pole is free to move only on the vertical plane of the cart and space. The cart will apply an impulsive force f of fixed magnitude by moving left or right in the track at discrete time intervals.

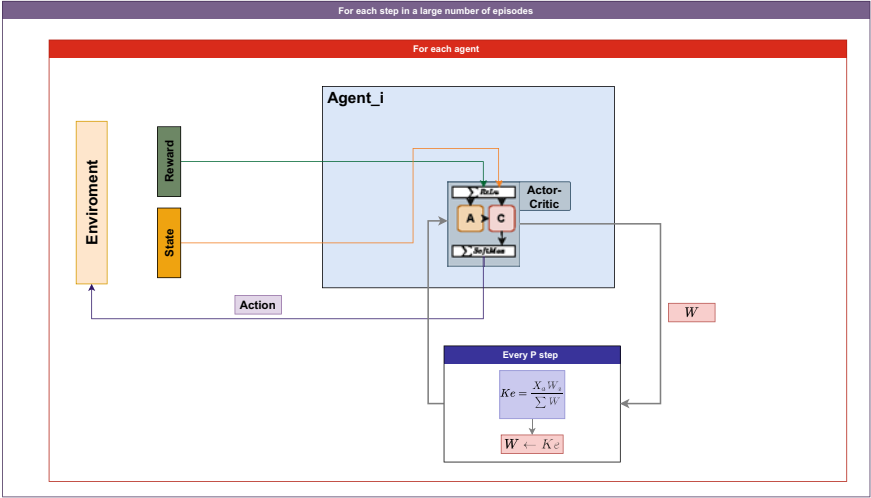


Fig. 9 Diagram of the SRL meta agent model

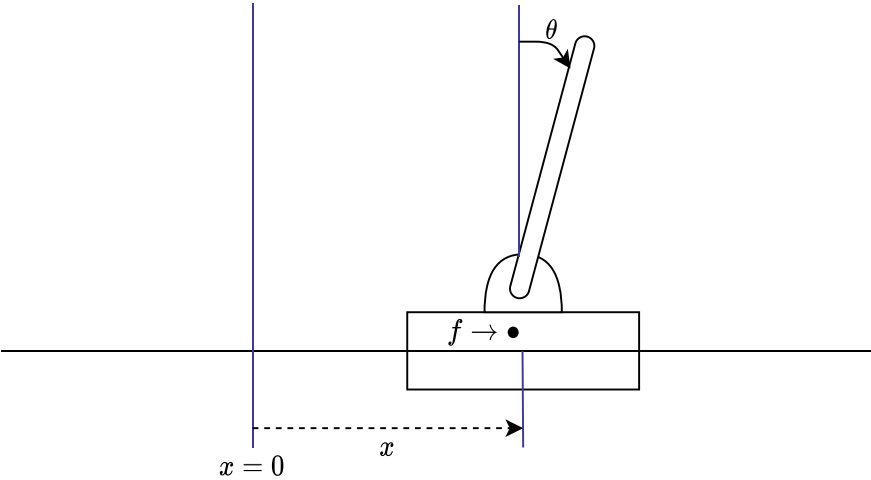


Fig. 10 Cartpole system to be controlled

5.1 Defining the Environment

The environment has four states, the position of the cart in the track x , the angle of the pole with the vertical θ , the cart velocity $\hat{x}$ and the rate of change of the angle $\hat{\theta}$ of the pole.

$$environment = \{x, \theta, \hat{x}, \hat{\theta}\} \tag{22}$$

On this environment is assumed that the equations of motion of the cartpole system are not known, the agent will try to learn the controller of the cart. At each time step the agent receives a vector giving the cartpole system state at that instant, if the pole falls or the cart hits the track boundary the environment will give a reward of -1 otherwise it will give a reward of plus 1 each time step the pole is balance.

The parameters of the state of the environment has the following thresholds.

$$\begin{aligned} x &: \pm 0.8, \pm 2.4m \\ \theta &: 0, \pm 1, \pm 6, \pm 12^\circ \\ \dot{x} &: \pm 5, \pm \infty m/s. \\ \dot{\theta} &: \pm 50, \pm \infty^\circ/s. \\ R &: -1, +1 \end{aligned}$$

These values constitute the vector the agent will receive from the environment where if the reward is above 200 points it is considered the environment solved.

5.2 Defining the Agents

For this study case, two types of agents were taken into account, one type used only reinforcement learning and the other type uses the proposed reinforcement learning model, both types of agents will learn from the same environment in parallel independently.

All types of agents had a learning model and some actions, the agents with the reinforcement learning only model A_{RL} and the agents with the synchronisms reinforcement learning model A_{SRL} can make actions *actions* whose are move to the left or move to the right.

$$A_{RL} = \{RL, actions\} \quad (23)$$

$$A_{SRL} = \{SRL, actions\} \quad (24)$$

$$actions = \{move\ left, move\ right\} \quad (25)$$

The SRL model need to have multiple agents learning different tasks nevertheless in this experiment all the agents will learn the same task independently as shown on Fig. 11 in this case the number of agents to use will be five but it can be any number of agents.

For the RL agent will learn only with vanilla Reinforcement learning as show in Fig. 12.

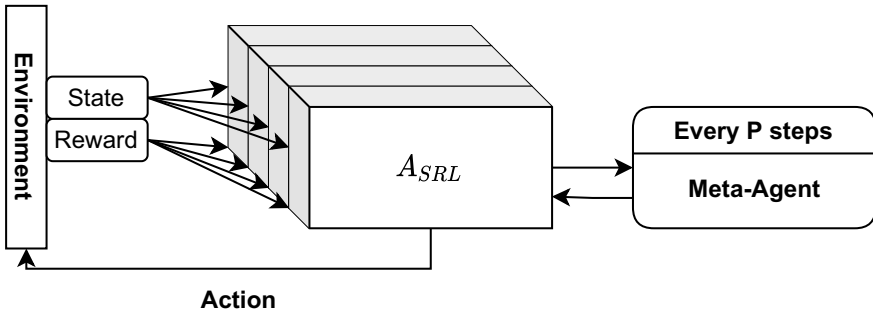
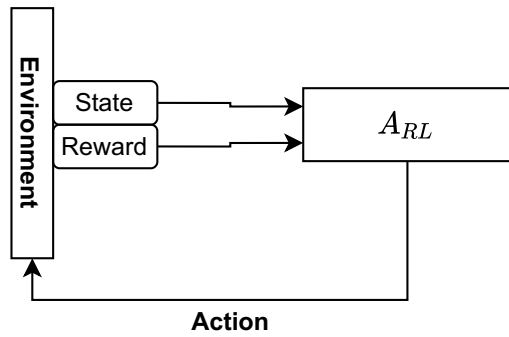


Fig. 11 SRL agents model

Fig. 12 RL agent model



6 Results and Discussions

When working with Reinforcement learning algorithms they tend to be unquestionable variables in regards to the performance and considerably sensible to a diversity of factors such as the implementation of the algorithm, hyper-parameters, the kind of problem it has to resolve or even the random seed, normally the principal metric for this kind of algorithms is the accumulative reward of each episode nevertheless this metric doesn't tell us much rather than it works and with only this metric is hard to identify what weakness or in what aspect the model is failing [33].

For the results of the experiments we will measure the traditional reward and also metrics that measure reliability according to Chan, Fishman, Canny, Korattikara and Guadarrama on the paper "Measuring the reliability of reinforcement learning algorithms" [34]. With this metrics we can get a better view of the strong points of each model and also their weakness this metrics are:

- The reward, for measuring the general result.
- Dispersion Across Runs (DR), it will give us the stability of our model between episodes.
- The variance of the runs, how the model is behaving in a set of episodes.

The first part of the experiment will run up to 5000 episodes, the environment is considered solve if the average reward is above 200 points.

At Fig. 13 it clearly can observe that the learning tendencies are going up, in general with the reward as the metric we can see and say that the agent with only reinforcement learning could solve the environment.

On the other hand in Fig. 14, also shows a very clear tendencies on the rise and in general taking account only the reward is the same as agents with only RL, it solve the environment, doing a more profound analysis on the results, despise both solved the environment with average rewards in some episodes above the 200 points mark, there ir clearly some difference between the two models.

This differences we can start seen it if we take in consideration the average of all episode on Fig. 15 on average the model of only RI will get a reward of more or less of 180 points those it would not solve the environment most of the time.

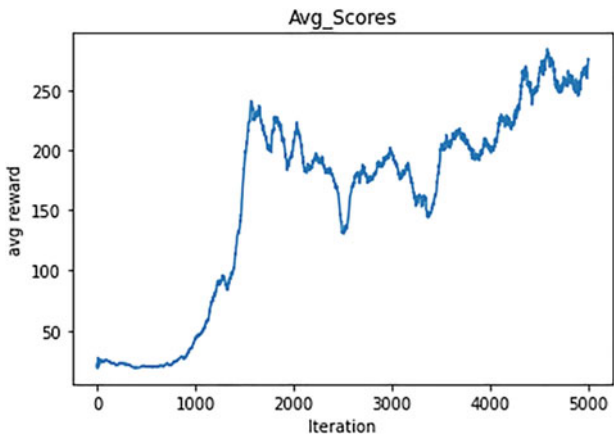


Fig. 13 Using

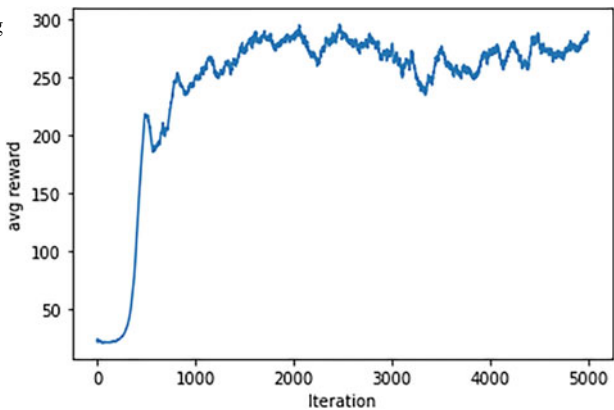


Fig. 14 Using synchronisms reinforcement learning

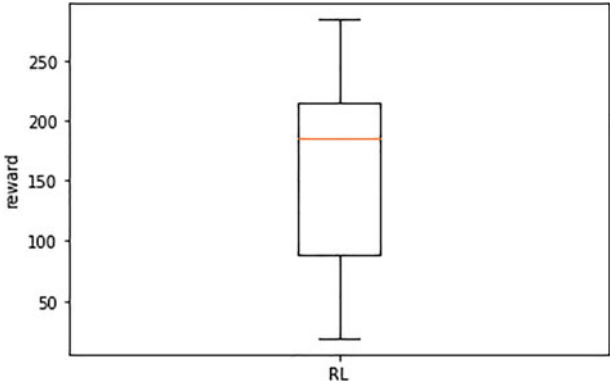


Fig. 15 Whiskers plot of the reward in RL

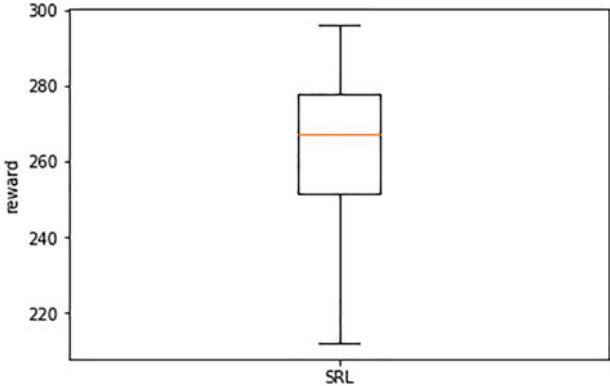


Fig. 16 Whiskers plot of the reward in SRL

In Fig. 16 the difference is more present, with the model proposed on average we obtain more or less a reward around 270 points which if we run a episode it is more likely it will solve the environment.

6.1 Comparing Both Models Using Reliability Metrics

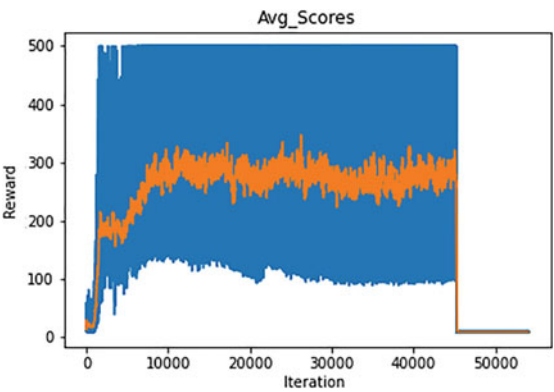
As propose on [34] with only the reward as a metric we can only measure some aspects of the models but it doesn't contribute for determining the problems of the model and in this case measuring the reliability is the prime objective.

Table 2 is the comparison of all the metrics, SRL model is very dispersed at the beginning as we can see with the metric disperse across runs then it stabilize for the rest of the episodes, this corroborates with the variance from the start is high

Table 2 Metrics comparing RL with SRL

	Iteration				
Metric	0–1000	1001–2000	2001–3000	3001–4000	4001–5000
Reward RL	23.390	151.420	181.013	187.752	247.835
Reward SRL	142.416	271.061	278.537	258.359	272.898
Dispersion across Runs RL	4.789	67.852	18.508	20.677	20.591
Dispersion across Runs SRL	92.631	12.361	7.224	9.121	6.595
Variance RL	22.954	4608.495	342.894	427.960	424.427
Variance SRL	8589.166	152.940	52.244	83.268	43.540

Fig. 17 Brittle fracture in RL



meaning the data, in this case, the reward is changing more frequently but as more episodes are taken it stabilize, on the other hand, the RL model it starts very stable but at the end it doesn't get a good enough stabilization, making the SRL a more reliable model on the long run.

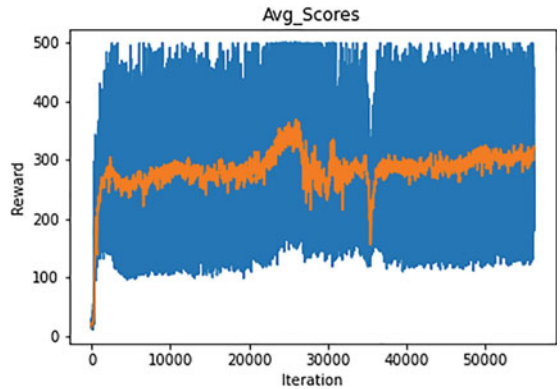
The second part of the experiment was to attack the Brittle fracture [35] where if an agent has a very sparse reward compare with the average the agent will break and will not know what to do bringing the learning curve to a halt and into a regress.

In this part the number of episodes increase until the Brittle fracture happens, we can see it around 45000 episodes.

Figure 17 the Brittle fracture happen and the agent broke therefore the agent couldn't recover instead the tendencies of the learning went down and never recovered.

On Fig. 18 it also happens around the same episode, on this case the agent could recover keeping the tendencies of the learning curve going up, this can be attributed

Fig. 18 Brittle fracture in SRL



to the part of the model of the meta-agent where it tries to fix any extreme dropout by pondering all the knowledge of all the agents acting as a hard reset as we can see on this figure.

7 Conclusions and Future Work

The proposed method obtained noticeable results although at the preliminary conclusions both methodologies deliver similar results, on average it solve the problems more times than not, take apart the meticulous analysis on Table 2 each metric calculated at a fixed point of the length of the experiment giving us a more detail understanding on how the models behaved in each stage of the simulation. Starting with the reward at Table 3.

Our proposed model in the early stage of the simulation (0 to 1000) episodes netted an undoubtedly higher reward with an increase of 6 times more in comparison, at the middle points of the simulation from previous episodes the SRL model had a 90%, 3% self increment and a small decrease of 8% respectably this corresponds with Fig. 14 where the tendencies of the learning expeditiously are going up and slowly getting stabilized, in comparison in this stage of the simulation with the RL model our model had a 55% increase in the reward value. In the final stage, both models generated satisfactory results where our model only had a 10% increment in

Table 3 Reward comparison between the two models

Reward						
Model/Episodes	0–1000	1001–2000	2001–3000	3001–4000	4001–5000	Total avg
RL	23.390	151.420	181.013	187.752	247.835	158.282
SRL	142.416	271.061	278.537	258.359	272.898	244.654

Table 4 Comparison between dispersion across runs of the models

Dispersion across runs						
Model/Episodes	0–1000	1001–2000	2001–3000	3001–4000	4001–5000	Total avg
RL	4.789	67.852	18.508	20.677	20.591	26.483
SRL	92.631	12.361	7.224	9.121	6.595	25.586

comparison with the RL model. Although the final increment is lower than the rest of the stages the main reason could be the reliability of the model as we analyzed on other important metrics such as the Dispersion across runs measuring the precision of the model.

In the early stage At Table 4 our model had very disperse rewards corresponding to the unquestionably higher rewards values seen when measuring only the reward, this can be view as the model netting with every new episode a higher and better reward but making it very unstable in the early stages of the simulation in comparison the RL model is very stable but also very low on precision, through the middle and final stages the proposed model accomplish a good fitting those the dispersion across the fixed points are low this mean the model accomplish a good stabilization or in other words a very precise model. The final metric we measure express the randomness our models have (Table 5).

By measuring the variance we can determine the randomness of the model at the earliest stages it seems the randomness of the model proposed is high this indicates and it concurs with the metrics of Tables 3, 4, where the reward is changing rapidly with an upwards tendencies on every episode it is natural to have high values of variance it is not necessarily a bad situation, on later stages the model tends to decrease at a stable rate the variance those making the model very precise and stable at the final stage of the simulation, in comparison the RL model also has a very similar pattern the difference is the decrease rate making it less stable.

Finally regarding the brittle problem although the proposed model is more stable and has better precision this problem still occurs as seen in Fig. 18 the difference is it can recover from a extreme dropout maintaining the learning tendencies going stably upwards however this do not guarantee the problem solve given that it is possible instead the learning rate tends to go downwards there before the problem is not solved.

Table 5 The comparison of randomness using the variance between models

Variance					
Model/Episodes	0–1000	1001–2000	2001–3000	3001–4000	4001–5000
RL	22.954	4608.495	342.894	427.960	424.427
SRL	8589.166	152.940	52.244	83.268	43.540

7.1 Future Work

In future work we like to test the model proposed on different types of simulations such as video games or robot focus or social economic focus, the results show it has potential to solve non linear and complex problems also had the model take in account different interactions at the meta level in summary:

- Test the model on more complex simulation.
- Test the model on real life simulation.
- Change the meta-agent having different meta-agents interacting.

Acknowledgements This research was supported/partially supported by MyDCI (Maestría y Doctorado en Ciencias e Ingeniería) and CONACYT (Consejo Nacional de Ciencia y Tecnología).

References

1. Schuler, D.: Social computing - introduction to the special section. Commun. ACM **37**(1), 28–29 (1994). <https://doi.org/10.1145/175222.175223>. <https://doi.org/10.1145/175222.175223>
2. Barto, A.G., Sutton, R.S., Brouwer, P.S.: Associative search network: a reinforcement learning associative memory. Biol. Cybern. **40**(3), 201–211 (1981). ISSN: 1432-0770. <https://doi.org/10.1007/BF00453370>. <https://doi.org/10.1007/BF00453370>
3. Bellman, R.: A Markovian decision process. J. Math. Mech. **6**(5), 679–684 (1957). <http://www.jstor.org/stable/24900506>
4. Bellman, R.E., Dreyfus, S.E.: Functional Approximations and Dynamic Programming. RAND Corporation, Santa Monica (1959)
5. Bertsekas, D.: Dynamic programming and optimal control **1** (1995)
6. Shaw, G.L.: Donald Hebb: behavior. In: Palm, G., Aertsen, A., (eds.), Brain Theory, pp. 231–233. Springer, Berlin (1986). isbn: 978-3-642-70911-1
7. Jung, C.G.: Synchronicity An Acausal Connecting Principle. Princeton University Press (2012)
8. Jung, C.G.: Synchronicity An Acausal Connecting Principle. Princeton University Press (1960)
9. Russell, P.N.S.: Artificial intelligence: a modern approach (2021)
10. Developing Intelligent Agent Systems: A Practical Guide (2004)
11. Pomerleau, D.A.: Neural network perception for mobile robot guidance (1993)
12. Mitchell, T.M.: Machine learning (1997)
13. Ji, Y., et al.: A survey on tensor techniques and applications in machine learning. IEEE Access **7**, 162950–162990 (2019). <https://doi.org/10.1109/ACCESS.2019.2949814>
14. Bengio, Y., Goodfellow, I., Courville, A.: Deep learning (2017)
15. Navarro-Almanza, R., Juárez-Ramírez, R., Licea, G., Castro, J.R.: Automated ontology extraction from unstructured texts using deep learning (2020)
16. Beysolow, T.: II. Applied Reinforcement Learning with Python 2019
17. Rozonoer, L., Mirkin, B., Muchnik, I.: Braverman Readings in Machine Learning. Key Ideas from Inception to Current State (2017)
18. Biswas, M., Nandy, A.: Reinforcement learning basics (2018)
19. Bhar, R., Hamori, S.: Hidden Markov models
20. Stone, P.: Q-learning. In: Sammut, C., Webb, G.I., (eds.), Encyclopedia of Machine Learning and Data Mining, pp. 1033–1033. Springer, Boston, (2017). ISBN: 978-1-4899-7687-1. https://doi.org/10.1007/978-1-4899-7687-1_689. https://doi.org/10.1007/978-1-4899-7687-1_689

21. Sewak, M.: Deep Reinforcement Learning. Springer, Singapore (2019). <https://doi.org/10.1007/978-981-13-8285-7>. <https://doi.org/10.1007>
22. Justesen, N., et al.: Deep learning for video game playing. *IEEE Trans. Games* **12**(1), 1–20 (2020). <https://doi.org/10.1109/TG.2019.2896986>
23. Shao, K., et al.: A Survey of Deep Reinforcement Learning in Video Games (2019). [arXiv:1912.10944](https://arxiv.org/abs/1912.10944) [cs.MA]
24. Schrittwieser, D., Simonyan, J., Silver, K.: Mastering the game of Go without human knowledge. *Nature* (2017)
25. Zhang, D., Tian, H., Shan, L., Munir, S., Stankovic, J.A., (eds.), Online Taxicab Demand Model From Big Sensor Data in a Roving Sensor Network (2014)
26. Jin, J., et al.: An information framework for creating a smart city through internet of things. *IEEE Internet Things J.* **1**(2), 112–121 (2014). <https://doi.org/10.1109/JIOT.2013.2296516>
27. Harris, K.: Synchronicity: The Magic, The Mystery, The Meaning (2015)
28. Williams, R.J., Baird, L.C.: A mathematical analysis of actor-critic architectures for learning optimal controls through incremental dynamic programming. In: *Proceedings of the Sixth Yale Workshop on Adaptive and Learning Systems* (1990)
29. Keen/sharp eye. 2021. <https://www.merriam-webster.com/dictionary/keen>
30. Walborn, F.: Chapter 3—Carl Jung. In: Walborn, F., (ed.), *Religion in Personality Theory*, pp. 41–64. Academic, San Diego (2014). ISBN: 978-0-12-407864-2. <https://doi.org/10.1016/B978-0-12-407864-2.00003-5>. <https://www.sciencedirect.com/science/article/pii/B9780124078642000035>
31. Marsala, C., Bouchon-Meunier, B.: Construction of Fuzzy Classes by Fuzzy Partitioning. In: Larsen, H.L., et al., *Flexible Query Answering Systems*, pp. 497–506. Physica-Verlag HD, Heidelberg. ISBN: 978-3-7908-1834-5
32. Barto, A.G., Sutton, R.S., Anderson, C.W.: Neuronlike adaptive elements that can solve difficult learning control problem. *IEEE Trans. Syst, Man, Cybern* (1983)
33. Packer, C., et al.: Assessing generalization in deep reinforcement learning (2019). [arXiv:1810.12282](https://arxiv.org/abs/1810.12282) [cs.LG]
34. Stephanie, C., Chan, Y., et al.: Measuring the reliability of reinforcement learning algorithms (2020). [arXiv: 1912.05663](https://arxiv.org/abs/1912.05663) [stat.ML]
35. Linse, T., et al.: A convergence study of phase-field models for brittle fracture. *Eng. Fract. Mech.* **184**, 307–318 (2017). ISSN: 0013-7944. <https://doi.org/10.1016/j.engfractmech.2017.09.013>. <https://www.sciencedirect.com/science/article/pii/S0013794417307488>

A Mathematical Deduction of Variational Minimum Distance in Gaussian Space and Its Possible Application to Artificial Intelligence



Juan de Anda-Suárez, Lucero Ortiz-Aguilar, Valentín Calzada-Ledesma, Luis Fernando Villanueva-Jiménez, Rosa Trasviña-Osorio, and Germán Pérez-Zúñiga

Abstract Artificial intelligence is an area of knowledge that in many of its applications involves statistical inferences to achieve the desired results: the Gaussian space has allowed modeling and adapting different algorithms in machine learning, digital image processing, evolutionary algorithms, and natural language processing, among others. One form of the procedure followed by such algorithms is to sample in space and infer distances or Gaussian behavior empirically, thus generating uncertainty or convergence deficiencies. We propose a variational derivation of the minimum distance in Gaussian space in terms of the parameters μ and σ , which allows us to infer behavioral changes in this space using the minimum distance. As a case study, we analyze an estimation distribution algorithm with our minimum distance inference, which allows controlling changes in the search space using reduced populations due to its analytical nature. Finally, we demonstrate the mathematical results of comparison between other estimation distribution algorithms using the proposed minimum distance.

Keywords Artificial intelligence · Variational · Gaussian space · UMDAc · Evolutionary algorithms · Fisher metric

1 Introduction

Artificial intelligence (AI) has its origins in the motivation to simulate the cognitive processes of human intelligence such as logic, reasoning, signal processing, and natural language [1, 2]. Such processes occur every day in humans. However, in a computational context, they have different degrees of difficulty; for example, a green

J. de Anda-Suárez (✉) · L. Ortiz-Aguilar · V. Calzada-Ledesma · L. F. Villanueva-Jiménez · R. Trasviña-Osorio · G. Pérez-Zúñiga
Tecnológico Nacional de México-Instituto Tecnológico Superior de Purísima del Rincón,
Purísima del Rincón, Gto, México
e-mail: Juan.ds@purisima.tecnm.mx

meadow and comparing its difference with the same scenario containing a grazing cow. So the way to solve this dilemma is through mathematics and its properties.

The different mathematical theories have nurtured the branches of artificial intelligence: particularly, statistical and probabilistic theory [3, 4], which allow the generation of descriptive or inferential models such as the cases of some machine learning algorithms based on Bayesian theory [5]; Estimation Distribution Algorithms [6, 7]; Gaussian filters for signal and image processing [8–10]; statistical inference of data topology [11], among others [12, 13].

In the context of statistical inference, the different probability distributions have allowed the generation of inferential models adapted to the problem of Machine Learning [14], Estimation Distribution Algorithms [6, 15, 16], as well as auxiliary to improve the performance of different AI methodologies; particularly, inference-based on the Gaussian distribution space [17–19].

The Gaussian probability distribution function occurs in many of nature's phenomena, so AI inherits this property when representing such phenomena. One strategy in research is to experiment with the problem to be treated and fit the information to a Gaussian model [20, 21], while another path follows Gaussian inference considering that the problem has this profile [22, 23], either by prior knowledge or for simplicity.

The investigations in the previous paragraph deal with point elements of the Gaussian space, which can be dynamic or static. However, their objectives are outside of describing the relationships and changes in this space. Then, the need arises to describe information that the Gaussian manifold can provide for the a priori treatment; particularly—the minimum distance between Gaussian distributions—.

We propose a study of the Gaussian manifold using the Fisher metric to obtain its geometry [24, 25]; the resulting arc length. Through the metric, we calculate a minimum distance estimate using the variational calculus and the Euler equation in the μ - σ space, in addition, we propose a study of the dynamics and relationship between the minimum Gaussian distance in the marginal univariate scheme.

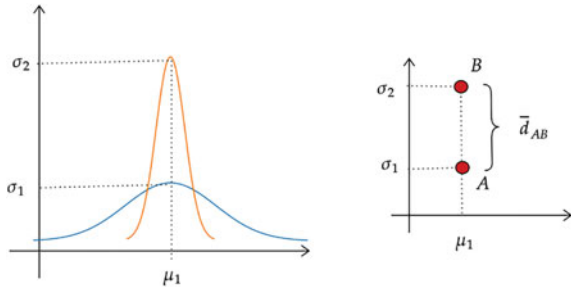
The content of the paper is distributed as follows: Sect. 2 presents the elements of theoretical estimation of the Gaussian variational minimum distance; Sect. 3 describes the application of variational minimum distance in a case study of an AI algorithm, in particular, the Univariate Marginal Distribution Algorithm (UMDAc); Sect. 4 shows the results obtained from the dynamic behavior and its connection to the minimum distance; Sect. 5 states the conclusions and possible future work.

2 Methodology

2.1 Fisher Information Metric

IA algorithms based on the Gaussian probability distribution, denoted by Eq. (1), have to deal with the μ and σ parameters; these parameters generate evolutionary

Fig. 1 Distance between two points in $\mu - \sigma$ space



or positional changes in the Gaussian space. Therefore a question arises, what is the curvature between two points? To illustrate this question, we start from Fig. 1: showing the relationship that exists between two Gaussians with the variation of σ and keeping μ constant. We observe two points A and B in the $\mu - \sigma$ space. Then, how can we measure the distance between two points in the statistical space? One answer is the Fisher metric [26, 27].

$$p(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(x - \mu)^2}{2\sigma^2}\right] \quad (1)$$

In information theory, the Fisher metric denoted by Eq. (2) describes the statistical geometry of a system in a smooth manifold; the metric connects to the relative entropy through the second derivative of the latter.

$$g_{jk}(\theta) = \int_{\mathcal{X}} \frac{\partial \ln[p(x; \theta)]}{\partial \theta_j} \cdot \frac{\partial \ln[p(x; \theta)]}{\partial \theta_k} p(x; \theta) dx \quad (2)$$

where $\theta_i = [\mu, \sigma]$ and $\ln[p(x; \theta)]$ is given by Eq. (3):

$$\ln[p(x, \mu, \sigma)] = -\frac{1}{2} [\ln(2\pi\sigma^2)] - \frac{(x - \mu)^2}{2\sigma^2} \quad (3)$$

In Eqs. (4) and (5), we present the necessary derivations to make the metric calculation:

$$\frac{\partial \ln[p(x; \mu, \sigma)]}{\partial \mu} = \frac{(x - \mu)}{\sigma^2} \quad (4)$$

$$\frac{\partial \ln[p(x; \mu, \sigma)]}{\partial \sigma} = -\frac{1}{\sigma} + \frac{(x - \mu)^2}{\sigma^3} \quad (5)$$

By means of Eq. (4), we calculate the first element g_{11} of the metric and taking into account that $\int (x - \mu)^2 p(x; \mu, \sigma) dx = \sigma^2$, we find the result of Eq. (6):

$$g_{11} = \frac{1}{\sigma^4} \int_{\mathcal{X}} (x - \mu)^2 p(x; \mu, \sigma) dx = \frac{1}{\sigma^2} \quad (6)$$

Following a procedure similar to Eq. (6), Eq. (7) shows the result of the metric component in g_{12} and g_{21} , which are zero due to property $\int (x - \mu)^{2k-1} p(x; \mu, \sigma) dx = 0, k = 1, 2, 3, \dots$:

$$g_{12} = -\frac{1}{\sigma^3} \int_{\mathcal{X}} (x - \mu) p(x; \mu, \sigma) dx + \frac{1}{\sigma^5} \int_{\mathcal{X}} (x - \mu)^3 p(x; \mu, \sigma) dx = 0$$

$$g_{12} = 0 = g_{21} \quad (7)$$

Finally, Eq. (8) estimates the element g_{22} , where $\int (x - \mu)^4 p(x; \mu, \sigma) dx = 3\sigma^4$:

$$g_{22} = \frac{1}{\sigma^2} \int_{\mathcal{X}} p(x; \mu, \sigma) dx - \frac{2}{\sigma^4} \int_{\mathcal{X}} (x - \mu)^2 p(x; \mu, \sigma) dx$$

$$+ \frac{1}{\sigma^6} \int_{\mathcal{X}} (x - \mu)^4 p(x; \mu, \sigma) dx = \frac{1}{\sigma^2} - \frac{2}{\sigma^2} + \frac{3}{\sigma^2} = \frac{2}{\sigma^2} \quad (8)$$

We compile the results of Eqs. (6)–(8), and write the metric in its matrix form, denoted by Eq. (9); g_{jk} has the property of being a diagonal matrix that contributes to the geometry of the Gaussian space as the inverse of the square of the standard deviation and will allow us to calculate the arc length.

$$g_{jk}(\theta) = \begin{pmatrix} \frac{1}{\sigma^2} & 0 \\ 0 & \frac{2}{\sigma^2} \end{pmatrix} \quad (9)$$

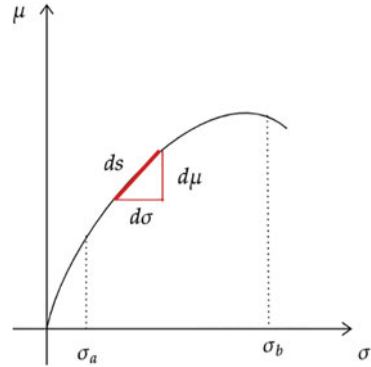
2.2 Gaussian Arc Length

In mathematical theory, arc length is the measure of the distance between two given points on a curve; in Fig. 2, we represent an infinitesimal distance, which is related to the infinitesimal change of σ and μ . Equation (10) mathematically define the arc length and its relation to the Fisher information metric (note, we use the summation suppression notation proposed by Einstein [28]).

$$ds^2 = g_{jk}(\theta) d\theta^j d\theta^k \quad (10)$$

A more explicit form of Eq. (10) is Eq. (11), which is the arc length for an infinitesimal segment in terms of μ and σ .

Fig. 2 Illustration of arc length in the $\mu - \sigma$ space



$$ds^2 = \frac{1}{\sigma^2} d\mu^2 + \frac{2}{\sigma^2} d\sigma^2 \quad (11)$$

If we factorizing $d\sigma$ of Eq. (11), then we can write the result of Eqs. (12) and (13):

$$ds^2 = \frac{1}{\sigma^2} \left[\left(\frac{d\mu}{d\sigma} \right)^2 + 2 \right] d\sigma^2 \quad (12)$$

$$ds = \frac{1}{\sigma} \sqrt{\left(\frac{d\mu}{d\sigma} \right)^2 + 2} d\sigma \quad (13)$$

Finally, the total arc length results from the Riemann sum in the sigma interval of the infinitesimal arc length; Eq. (14) describes this fact mathematically.

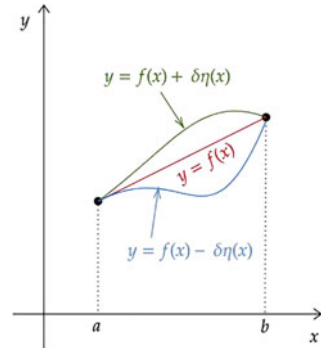
$$I = \int_{\sigma_a}^{\sigma_b} \frac{\sqrt{\left(\frac{d\mu}{d\sigma} \right)^2 + 2}}{\sigma} d\sigma \quad (14)$$

In Eq. (14), we have contained all the possible distances that exist between two points in the μ - σ space. However, we are interested in finding the minimum distance that exists between two Gaussians, so to answer this question in Sect. 2.3, we will approach the problem from a variational calculus point of view.

2.3 Variational Minimum Distance in Gaussian Space

In the present section, we address the problem of estimating the minimum distance between two points: In Fig. 3, the solid red line represents a functional joining two

Fig. 3 Graphical representation of the variations of a functional



points; the green curve illustrates a positive delta variation, and the blue curve denotes a negative delta variation. Based on the representations in Fig. 3, a question arises, which of all possible variations of a functional minimizes the distance between two points in Gaussian space?

Euler and Lagrange posed a solution to the previous question through the mathematical study of the extremals of a functional. A functional is a function where its domain groups functions as arguments; Eq. (15) defines a functional mathematically [29, 30].

$$I = \int_a^b f[y(x), y'(x); x] dx \quad (15)$$

where $y'(x) = \frac{dy}{dx}$, we observe that the result of Eq. (14) has the same form as Eq. (15), then from the definition of the functional of variational calculus; Eq. (16) represents the function containing the extrema in the Gaussian space:

$$\mathcal{L}\left[\sigma, \mu, \frac{d\mu}{d\sigma}\right] = \frac{\sqrt{(\mu')^2 + 2}}{\sigma} \quad (16)$$

Note that $\mu' = \frac{d\mu}{d\sigma}$. A functional as in Eq. (16) contains an extreme value if the condition presented in Eq. (17) has solution.

$$\frac{d}{d\sigma} \left(\frac{\partial \mathcal{L}}{\partial \mu'} \right) - \frac{\partial \mathcal{L}}{\partial \mu} = 0 \quad (17)$$

In the context of $\mu - \sigma$ space, we write the calculations corresponding to the extreme value of Eqs. (14), (18) and (19):

$$\frac{\partial \mathcal{L}}{\partial \mu'} = \frac{\mu'}{\sigma \sqrt{\mu'^2 + 2}} \quad (18)$$

$$\frac{d}{d\sigma} \left(\frac{\mu'}{\sigma \sqrt{\mu'^2 + 2}} \right) = 0 \quad (19)$$

The solution to Eq. (19) is a direct integration, which results in a constant C_1 ; Eq. (20) shows the result in terms of the statistical parameters:

$$\frac{\mu'}{\sigma \sqrt{\mu'^2 + 2}} = C_1 \quad (20)$$

Solving Eq. (20) for μ and simplifying some calculations, we obtain the results of Eq. (21) and (22):

$$\mu'^2 = C_1^2 \sigma^2 (\mu'^2 + 2) \quad (21)$$

$$\mu'^2 (1 - C_1^2 \sigma^2) = 2C_1^2 \sigma^2 \quad (22)$$

Recalling that $\mu' = \frac{d\mu}{d\sigma}$; Eq. (21) denotes a differential equation of separable variables:

$$\pm \left(\frac{d\mu}{d\sigma} \right) = \pm \frac{\sqrt{2} C_1 \sigma}{\sqrt{1 - C_1^2 \sigma^2}} \quad (23)$$

The solution for Eq. (21) is a definite integral on the intervals $[\mu_0, \mu]$ and $[\sigma_0, \sigma]$, Eq. (24), where Eq. (25) results:

$$\pm \int_{\mu_0}^{\mu} d\mu = \pm \int_{\sigma_0}^{\sigma} \frac{\sqrt{2} C_1 \sigma}{\sqrt{1 - C_1^2 \sigma^2}} d\sigma \quad (24)$$

$$\pm \mu = \pm \mu_0 \mp \frac{\sqrt{2 - 2C_1^2 \sigma^2}}{C_1} \Bigg|_{\sigma_0}^{\sigma} \quad (25)$$

The value obtained in Eq. (25) estimates the variational changes of μ with respect to the existing changes in σ ; resulting in the minimum changes between these statistical quantities. In the following sections, we will demonstrate the importance of Eq. (25) in the study of computational optimization algorithms, in the particular case of the Univariate Marginal Distribution Algorithm (UMDAc), as well as a modification to its inferential scheme to deal with problems where the population is restricted.

3 Connection Between UMDAc and the Minimum Variational Distance

In the present section, we address the application of the minimum variational distance to the field of AI, particularly to the Univariate Marginal Distribution Algorithm (UMDAc) [31, 32], which is based on time-varying Gaussian processes; we study the theoretical connection between the dynamic changes of the Gaussian distribution and the minimum distance between them; we also plan an approximate formulation of the general dynamic behavior of UMDAc.

3.1 Univariate Marginal Distribution Algorithm

In computational optimization, UMDAc belongs to evolutionary-based on estimation distribution algorithms (EDA), where UMDAc considers the independence of probability distribution among the variables of the problem; where Eq. (26) represents this fact:

$$p_i(x_k, t) = \sum_x p(x, t) \quad (26)$$

where x_k represents the elements of x dimension.

The n -dimensional probability distribution estimate can be written as the multiplier of each dimension separately; denoted by Eq. (27):

$$P(x, t + 1) = \prod_{i=1}^n p^s(x_i, t) \quad (27)$$

where $p^s(x_i, t)$ represents the probability for the i -th element at time t . If we particularize UMDAc with a Gaussian distribution definition, Eq. (1) can be written as Eq. (2):

$$P(x, t + 1) = \prod_{i=1}^n G^s(x_i, t; \mu_i, \sigma_i) \quad (28)$$

denoted by $G^s(x, t; \mu, \sigma)$ as the Gaussian probability distribution [33].

Based on the result of Eq. (1), we describe the algorithm presented in Algorithm 1: step 1, initialization of the population with random numbers in the solution domain; step 2, evaluation of the population in the cost function; step 3, sampling of the population; step 4, estimation of the μ and σ parameters; step 5, generation of the new population elements based on Eq. (29); Step 6, continue the process until

the stopping criterion is met.

$$x_i = \sigma_i \left(\frac{12}{k} \right)^{1/2} \left(\sum_{j=1}^k r_j - \frac{k}{2} \right) + \mu_i \quad (29)$$

where k is a fitting parameter and r_j is a uniformly distributed number.

Algorithm 1 UMDAc Pseudocode

Require: np : population size , $f(X)$: fitness function

Ensure: $\hat{F}_{best}$: Best solution.

- 1: $J_j^i \leftarrow \text{Initial-population}(S, \mu^0, \nu^0)$
 - 2: $\text{Evaluate}(J_j^i, f(x))$
 - 3: **while** ($f_{x_{best}} \leq \epsilon$) **do**
 - 4: $T^i \leftarrow \text{Top-ranked}(J_j^i)$
 - 5: $\mu^{i+1}, \nu^{i+1} \leftarrow \text{UpdateSelectionOperators}(T^i)$
 - 6: $J_j^{i+1} \leftarrow \text{NewJacobiSubset}(S, \mu^{i+1}, \nu^{i+1})$
 - 7: $\hat{F}_{best} \leftarrow J_0^{i+1}$: Get Best solution
 - 8: **end while**
 - 9: **return** $\hat{F}_{best}$
-

3.2 Minimum Distance in Gaussian UMDAc

In Sect. 2, we obtained the minimum distance result between two normal distributions; we then present a minimum distance connection study in the Gaussian dynamical system for UMDAc.

UMDAc in each iteration dynamically changes its distribution propagation: there exists a minimum distance; to measure this distance, we start from the result of Eq. (20) and find the constant of connection between Gaussians, given by Eq. (30).

$$C_1 \approx \frac{\frac{\mu_{(i+1)} - \mu_i}{\sigma_{(i+1)} - \sigma_i}}{\sigma_i \sqrt{\frac{\mu_{(i+1)} - \mu_i}{\sigma_{(i+1)} - \sigma_i} + 2}} \quad (30)$$

where $\mu_{(i+1)}$, μ_i , $\sigma_{(i+1)}$, and σ_i are the mean and standard deviation; in the $i + 1$ and the i -th iteration, respectively.

Equation (30) allows us to study the minimum distance constant in the evolution of the UMDAc algorithm; this fact is of vital importance since it describes the geometrical behavior before the computational optimization—marking an approximation to a geometrical treatment of the problems in the evolutionary approach—.

In Sect. 4, we address an experimental setup based on the cost functions used in the literature, intending to contrast the behavior of UMDAc in a minimum distance environment, which allows us to generate inferential models and improve the performance of this algorithm.

4 Experimental Setup

This section divides its content into two parts: the first part corresponds to the experimental setup, as well as parameters for the operation of the UMDAc, the second subsection shows the empirical results of the functions, as well as an inferential model obtained from the results.

4.1 Fitness Functions and Parameter Configuration

We have selected 11 comparative fitness functions. These functions have different degrees of difficulty in converging to the optimum, allowing us to study if there is a similar minimum distance behavior in The Search by UMDAc; Table 1 describes their mathematical definitions, as well as the location of their optimum.

Table 1 Fitness function for experimental contrast

Function	Equation	Domain	Optimum
Sum Cancellation	$\frac{1}{10^{-5} + \sum_{i=1}^n y_i }$ where $y_1 = x_1$, and $y_i = y_{i-1} + x_i$ for $i > 1$	$x \in [-0.16, 0.16]^n$	10^5
Ackley	$20 - 20 \exp\left(-0.2\sqrt{\sum_{i=1}^n x_i^2/n}\right) + \exp\left(\sum_{i=1}^n \cos 2\pi x_i/n\right) + \exp(1)$	$x \in [-32.768, 32.768]^n$	0
Griewangk	$1 + \sum_{i=1}^n x_i^2/4000.0 - \prod_i \cos(x_i/\sqrt{i})$	$x \in [-600, 600]^n$	0
Sphere	$\sum_{i=1}^n x_i^2$	$x \in [-10, 5]^n$	0
Ellipsoid	$\sum_{i=1}^n 10^{6\frac{i-1}{n-1}} x_i^2$	$x \in [-10, 5]^n$	0
Cigar	$x_1^2 + \sum_{i=2}^n 10^6 x_i^2$	$x \in [-10, 5]^n$	0
Tablet	$10^6 x_1^2 + \sum_{i=2}^n x_i^2$	$x \in [-10, 5]^n$	0
Cigar Tablet	$x_1^2 + \sum_{i=2}^{n-1} 10^4 x_i^2 + 10^8 x_n^2$	$x \in [-10, 5]^n$	0
Two axes	$\sum_{i=1}^{n/2} 10^6 x_i^2 + \sum_{i=\frac{n}{2}+1}^n x_i^2$	$x \in [-10, 5]^n$	0
Rastrigin	$10n + \sum_{i=1}^n [x_i^2 - 10 \cos(2\pi x_i)]$	$x \in [-5.12, 5.12]^n$	0
Rosenbrock	$\sum_{i=1}^{n-1} \left(100(x_i - x_{i-1}^2)^2 + (1 - x_i)^2\right)$	$x \in [-10, 10]^n$	0

In an effort to establish reliable experiments, we have set the domain of all fitness functions to $[-5, 5]$, as well as a population of 200 individuals and 10,000 function calls.

4.2 Experimental Results

Based on the fitness functions in Table 1, we generated a set of 31 experiments to obtain the statistics on the change of minimum distance between dynamic distributions in UMDA; we report the results in Table 2 conjugated by three cutoff points, where each cutoff estimates the value of the constant C_1 : We show four representative examples, for a 2-dimensional problem, to observe the contrast of minimum distance change; Sphere and Ackley functions have similar minimum distance geometry performance, while Rosenbrock and Rastrigin share behavior, finally, Griewangk differs in its numerical trend to the rest of the functions. Table 2 exemplifies the geometric impact of minimum distance on continuous feature optimization problems.

Figure 4a shows the trend of the sphere function in minimum distance convergence for three particular dimensions; Fig. 4b represents the convergence for the Ackley function problem; both representative functions, as well as the 11 functions that we have proposed for the experimentation—the constant tends to a univariate exponential type minimum distance constant—.

The results presented in Table 2 and Fig. 4 demonstrate the minimum distance behavior between Gaussian distributions, suggesting that the behavior of UMDAc in the presented methodology has an exponential profile. This profile allows us to propose in the following section a minimum distance inference model to estimate the μ and σ^2 parameters, thus providing the algorithm with a reduction of population elements, which is a desirable case in the face of expensive problems in function evaluation.

Table 2 Comparison of change of constant c in different iteration states

Fitness functions	Iteration = 10		Iteration = 20		Iteration = 30	
	1D	2D	1D	2D	1D	2D
Ackley	2.19	4.72	1.2×10^7	1.2×10^7	1.0×10^{14}	1.9×10^{14}
Sphere	0.01	0.02	26.51	26.38	8.1×10^3	4.0×10^3
Rosenbrock	0.18	0.11	60.35	173.54	122.61	115.76
Rastrigin	0.67	0.40	3.54	11.09	2040.5	504.63
Griewangk	1.2×10^{-6}	1×10^{-3}	0.007	0.001	0.002	0.010

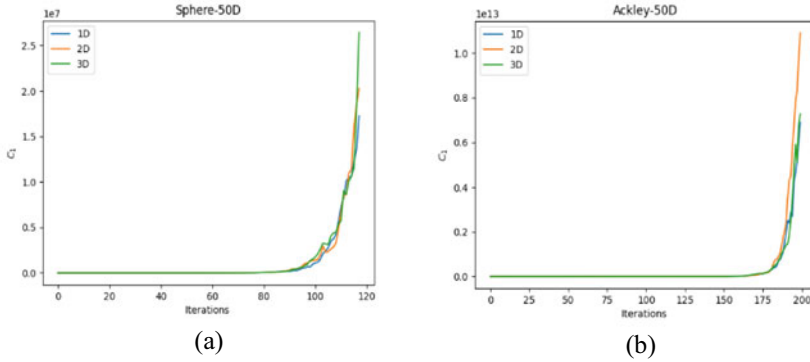


Fig. 4 Comparison of distance change C_1 concerning iterations: (a) sphere function and (b) Ackley function; both functions represented in 1-dimension, 2-dimension, and 3-dimension

4.3 Resulting Mathematical Model

Based on the results presented in the previous section, we find a minimum distance approximation model that characterizes the geometrical behavior of UMDAc when compared to classical optimization functions; Eq. (31) defines the model:

$$C_1 \approx e^{\beta \cdot It} \quad (31)$$

where β is a modulation parameter for each characteristic problem, It is the iteration value of the algorithm. The result of Eq. (30) can be substituted in Eq. (25), thus generating—a new formulation of statistical inference in computational optimization problems based on the minimum distance and the marginal distribution scheme—resulting in Eq. (32):

$$\mu = \mu_0 - \frac{\sqrt{2 - 2e^{2\beta \cdot It}} \sigma^2}{e^{\beta \cdot It}} \Big|_{\sigma_0}^{\sigma} \quad (32)$$

Equation (31) describes the inferential statistical behavior of an optimization problem in classical model functions: Eq. (31) decreases the number of population elements, since, in the face of high dimensionality and computational cost functions, UMDAc needs considerable populations due to its statistical nature, so the knowledge of such behavior allows to improve the performance in the face of these problems.

5 Conclusions

A minimum distance estimate between Gaussian distributions under a variational approach with applications to artificial intelligence, we have presented, which we deduced from the variational calculus obtaining analytical mathematical results, which we applied to UMDAc, subsequently, we calculated an approximation model from the constant minimum distance estimate; we tested 11 fitness functions that generated a new characterization model in the univariate marginal distribution scheme.

The minimum distance estimate showed, in Sect. 4, evidence of being an adequate descriptor of the geometric behavior in continuous optimization problems under a Gaussian distribution scheme, this fact allowed in Sect. 4.2 to generate an exponential profile parameter inference model, which can help in problems where the evaluation of the fitness function involves computational costs.

Equation (23) describes the minimum distance between two points in the normal distribution space, which motivates us as future work to study the possibilities of information that Eq. (23) can provide in different areas of artificial intelligence; particularly in Machine Learning, where the Gaussian space has had a considerable impact.

References

1. Ertel, W.: Introduction to Artificial Intelligence. Springer (2018)
2. Kubat, M.: An Introduction to Machine Learning. Springer (2017)
3. Silhavy, R., Silhavy, P., Prokopova, Z.: Computational and Statistical Methods in Intelligent Systems. Springer (2018)
4. Golden, R.: Statistical Machine Learning: A Unified Framework. CRC Press (2020)
5. Theodoridis, S.: Machine Learning: A Bayesian and Optimization Perspective. Academic Press (2020)
6. Larrañaga, P., Lozano, J.A.: Estimation of Distribution Algorithms: A New Tool for Evolutionary Computation. Springer (2012)
7. Doerr, B., Neumann, F.: Theory of Evolutionary Computation: Recent Developments in Discrete Optimization. Springer (2019)
8. Jiang, X.J., Scott, P.J.: Free-Form Surface Filtering Using Wavelets and Multiscale Decomposition. Academic Press (2020)
9. Nandan, D., Kanungo, J., Mahajan, A.: An error-efficient Gaussian filter for image processing by using the expanded operand decomposition logarithm multiplication. *J. Ambient Intell. Humanized Comput.* (2018)
10. Diniz, P.S.R., Naylor, P.A., Suykens, J.: Academic Press Library in Signal Processing: Signal Processing Theory and Machine Learning. Academic Press (2013)
11. Pezzotti, N., Höllt, T., Lelieveldt, B., Eisemann, E., Vilanova, A.: Hierarchical stochastic neighbor embedding. *Comput. Graph. Forum* **35**(3), 21–30 (2016)
12. Chen, Y., Cheng, N., Cai, M., Cao, C., Yang, J., Zhang, Z.: A spatially constrained asymmetric Gaussian mixture model for image segmentation. *Inf. Sci.* **575**, 41–65 (2021)
13. Cocho, G., Rodríguez, R.F., Sánchez, S., Flores, J., Pineda, C., Gershenson, C.: Rank-frequency distribution of natural languages: a difference of probabilities approach. *Phys. Stat. Mech. Appl.* **532**, 121795 (2019)

14. Rasmussen, C.E.: Gaussian processes in machine learning. In: Bousquet O., von Luxburg U., Rätsch G. (eds) *Advanced Lectures on Machine Learning. Lecture Notes in Computer Science*, vol. 3176 (2004)
15. Armañanzas, R., Inza, I., Santana, R., Saeys, Y., Flores, J.L., Lozano, J.A., Van de Peer, Y., Blanco, R., Robles, V., Bielza, C., Larrañaga, P.: A review of estimation of distribution algorithms in bioinformatics. *BioData Min.* **1**(6) (2008)
16. Krejca, M.S., Witt, C.: Theory of estimation-of-distribution algorithms. In: Doerr, B., Neumann, F. (eds.) *Theory of Evolutionary Computation. Natural Computing Series*. Springer (2020)
17. Strahl, J., Honkela, T., Wagner, P.: A Gaussian process reinforcement learning algorithm with adaptability and minimal tuning requirements. In: *de Artificial Neural Networks and Machine Learning–ICANN*. Springer (2014)
18. Blomqvist, K., Kaski, S., Heinonen, M.: Deep convolutional Gaussian processes. In: *de Machine Learning and Knowledge Discovery in Databases*. Springer (2020)
19. Wang, T., Xu, L., Li, J.: Scalable deep convolutional random kernel learning in Gaussian process for image recognition. *Neurocomputing* **456**, 288–298 (2021)
20. Erickson, C.B., Ankenman, B.E., Sanchez, S.M.: Data from fitting Gaussian process models to various data sets using eight Gaussian process software packages. *Data in Brief* **18**, 684–687 (2018)
21. Jabłońska-Sabuka, M., Teuerle, M., Wylomańska, A.: Bivariate sub-Gaussian model for stock index returns. *Phys. Stat. Mech. Appl.* **486**, 628–637 (2017)
22. Garrido-Merchán, E.C., Hernández-Lobato, D.: Dealing with categorical and integer-valued variables in Bayesian optimization with Gaussian processes. *Neurocomputing* **300**, 20–35 (2020)
23. Gianniotis, N., Schnörr, C., Molkenhain, C.: Approximate variational inference based on a finite sample of Gaussian latent variables. *Pattern Anal. Appl.* **19**, 475–485 (2016)
24. MacKay, D.J.C., MacKay, V.J.C., Kay, D.J.C.M.: *Information Theory, Inference and Learning Algorithms*. Springer (2003)
25. Gamal, A.E., Kim, Y.-H.: *Network Information Theory*. Cambridge University Press (2011)
26. Shun-Ichi, A.: *Differential-Geometrical Methods in Statistics*. Springer (1985)
27. Itoh, M., Shishido, Y.: Fisher information metric and Poisson kernels. *Differ. Geom. Appl.* **26**(4), 347–356 (2008)
28. d’Inverno, R.: *Introducing Einstein’s Relativity*. Clarendon Press (1992)
29. Rindler, F.: *Calculus of Variations*. Springer (2018)
30. Thornton, S., Marion, J.B.: *Classical Dynamics of Particles and Systems*. Cengage Learning (2003)
31. Ghosh, A., Muehlenbein, H.: Univariate marginal distribution algorithms for non-stationary optimization problems. *Int. J. Knowl.-Based Intell. Eng. Syst.* **8**(3), 129–138 (2004)
32. Dang, D., Lehre, P., Nguyen, P.: Level-based analysis of the univariate marginal distribution algorithm. *Algorithmica* **81**, 668–702 (2019)
33. Mühlenbein, H.: The equation for response to selection and its use for prediction. *Evolut. Comput.* **5**(3), 303–346 (1997)

A Model for Learning Cause-Effect Relationships in Bayesian Networks



Jenny Betsabé Vázquez-Aguirre, Nicandro Cruz-Ramírez,
and Marcela Quiroz-Castellanos

Abstract The study of causality has its origins more than 300 years ago; several disciplines have focused efforts on explaining the natural causal process, for example, Cognitive Psychology (CP) and Artificial Intelligence (AI). Performing a causal inference may be a simple task for humans, for that there are multiple efforts to replicate and explain causality based on prior knowledge. From CP, there have been proposals that aim to describe how are learned naturally cause-effect relationships, and from AI, multiple algorithms have focused on estimating or predicting causality, such as the intervention of variables in Bayesian Networks (BN). However, this method requires prior knowledge that indicates the cause-effect direction within the connections in the network. Our principal goal is an integrated algorithm for the construction of graphical models (CBN) such from datasets can learn the causal relationships giving direction to the arcs and identify causes-and-effects within the network. The results obtained with this approach have been encouraging. The CBNs acquired can be used to intervene variables and estimate causal probabilities. We have tested with real datasets from repositories and comparing our results with an algorithm (and two metrics) belonging to the AI.

Keywords Rescorla-wagner model · Causal Bayesian networks · Intervention of variables

J. B. Vázquez-Aguirre (✉) · N. Cruz-Ramírez · M. Quiroz-Castellanos
Instituto de Investigaciones en Inteligencia Artificial, Universidad Veracruzana, Xalapa, México
e-mail: jennybey13@gmail.com

N. Cruz-Ramírez
e-mail: ncruz@uv.mx

M. Quiroz-Castellanos
e-mail: maquiroz@uv.mx

1 Introduction

The causality study has its origins approximately 300 years ago with Kant and Hume, both—although opposing each other—try to explain how causal knowledge is acquired naturally. Several disciplines have had this same object of analysis; one of them is Cognitive Psychology, which oversees studying the brain processes carried out to extract information from the environment and decide what actions to take appropriately; some of these processes are attention, problem-solving, learning, memory, language, reasoning, and thinking [1].

To understand cognition, Cognitive Psychology observes behavior while performing tasks such that it tries to explain how the brain is capable of receiving, selecting, and processing information [2].

Another area interested in the mental processes study is Artificial Intelligence (AI), which tries to create machines -hardware- and programs -software- that can reproduce human behavior [3]. It is relevant to emphasize the fundamental difference between these two disciplines. The difference settles down in the application object. Cognitive Psychology tries to discover the actual processes that the brain follows when it processes data; for its part, AI tries to implement these processes in machines (hardware and software) [1].

Cognitive Psychology's origins date back to the first half of the nineteenth century. In the late 1950s was exposed the cognitive revolution, processes learning generated gave rise to this discipline, adopting a computer-based model of the brain. For this reason, according to cognitive psychologists, all mental processes can be reproduced, and there is an algorithm that does it [2]. Thus, the origins of both disciplines can be linked (Cognitive Psychology and AI) so that some terms that allow us to understand the study of the mechanisms of both areas may be similar. For example, within Cognitive Psychology, stimuli can be understood as basic units of information known as inputs in AI. The brain takes these units and then transforms them on others known as outputs, through processes or rules; this latter could be reproduced by algorithms. The idea of natural information processing as sequences of computational methods operating on mental representations remains the cornerstone of modern Cognitive Psychology [3].

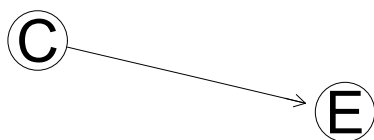
Due causal cognition is a fundamental key to learning, reasoning, and decision-making, it is part of the mental processes that cognitive psychologists have been paying attention to and trying to explain [1]. It is important to note that although causal cognition does not include the totality of cognition, it is a fundamental part of brain life [4].

A great deal of the research on causal cognition has focused on three fundamental inquiries:

1. How should humans represent causal structures?
2. How should they learn those causal representations?
3. How should they reason with those representations?

A valid answer to the first question is the expression $C \rightarrow E$ (C causes E); this is the most used, where C represents the cause that provokes E .

Fig. 1 Graph of a simple causal structure



Given the importance of causal cognition, several proposed models have tried to explain: how natural learning of causal relationships carries out [5–8].

From an AI perspective, there are also two fundamental questions related to causality [9].

1. What is empirical evidence required for legitimate inference of cause-effect relationships?
2. Given that we are willing to accept causal information about a phenomenon, what inferences can we draw from that information and how?

Figure 1 shows a graphic representation of the cause-effect relationship that in both perspectives can be applied (Artificial Intelligent and Cognitive Psychology).

Inside AI, some algorithms try to learn causality from datasets; one of them is through an alteration on a conventional Bayesian Network (BN) [10]. The simple BN structure can be similar to the representation in Fig. 1. This causal estimation model was presented in 1995 by Judea Pearl [10], and its validation was studied in 2019 by Vázquez [11]. In Sect. 3, we introduce a brief description of its operation and complications; the latter gave rise to the present work, which proposes to integrate a Cognitive Psychology algorithm for learning causal relationships from data. This algorithm was proposed in 1972 by Rescorla-Wagner [12], and we use it for building Bayesian Networks that we now call Causal Bayesian Networks (CBN).

It is important to note that despite the Rescorla-Wagner model have its origins several years ago, it is still in force and is highly studied. This model, due to its good results, has been accepted by Cognitive Psychologists. That is since it responds to philosophical currents describing how causal learning takes place in the brain. Therefore, we have selected the Rescorla-Wagner model for building Causal Bayesian Networks; specifically, we used it to provide the direction of the arcs.

In this work, we introduced the Rescorla-Wagner model (Sect. 2) and the Intervention of variables through Causal Bayesian Networks (Sect. 3). Later, we explain our algorithm proposal and the techniques used for its construction; additionally, how we have integrated the whole algorithm (Sect. 4).

We also present the datasets (Sect. 5) and results obtained by comparing the networks built with our model and those done through traditional AI techniques (Sect. 6). Finally, we provide a series of conclusions that have given us the prelude to future work derived from this research (Sect. 7).

2 Rescorla-Wagner Model: Cognitive Psychology

The Rescorla-Wagner model was proposed in 1917 by both experimental psychologists, Robert A. Rescorla and Allan R. Wagner [12]. This method belongs to Classical conditioning or Pavlovian Conditioning and has its origins in Behaviorism; it is necessary to explain what type of conditioning is referring Rescorla-Wagner model to introduce it.

Behaviorism was presented in 1913 by John B. Watson. The aim was to study the learning process done naturally. He proposes the study of psychology not as the study of the mind but as the experimental analysis of behavior [13]. This movement upholds that if we want to learn how are carried out learning processes, it will be necessary to do through observable phenomena. For this reason, he proposed the behavior as the study subject; due to the behavior can be observed and described objectively [14].

Classical conditioning is an example of Behaviorism and belongs to this psychological movement. The most common technique used is the constant applied of an Unconditioned Stimulus or natural stimulus, along with Conditioned Stimulus. An Unconditioned Stimulus is a stimulus that produces a response immediately identified by the organism that receives it. It may be the outcome of a natural-born structure or a previously learned feature; the Unconditioned Stimulus generates an Unconditioned Response. On the other hand, a Conditioned Stimulus is an initially neutral stimulus related to the Unconditioned Stimulus; this produces a Conditioned Response.

The following is an example related to Classical Conditioning: the dog and the bell.

- The experimental subject (the dog) will listen to the ring of a bell that, on its own, this sound will not provoke a single response in the organism (Initially neutral stimulus registered as Conditioned Stimulus).
- The second stimulus given to the dog is a class of reinforcement. For example, food (Unconditioned Stimulus); that will provoke a natural response in the organism.
- The food will produce salivation as the natural response presented by the dog when facing this (Unconditioned Stimulus), which is considered Unconditioned Response.
- During the conditioning process (performed as a series of tests), the bell will always ring a couple of minutes before showing the food to the dog.
- After the conditioning process, the dog will produce salivation by the sound of the bell on its own, this action being a Conditioned Response. The result on the dog's behavior is known as associative learning (Classical Conditioning).

The main characteristics of Classical Conditioning are the following [15]:

- The origin of the response: unchained by the stimulus.
- Procedure: the reinforcement is always given after the Conditioned Stimulus regardless behavior of the organism is.
- Nervous system: autonomous.
- Responses: visceral.

- Conscious or not: Unconscious.
- Association: between stimuli (Unconditioned Stimulus—Conditioned Stimulus).
- Laws: contiguity.
- Conditioned Response and Unconditioned Response: Always the same.
- Partial reinforcement effect: reduces the strength of the response.

Now, according to the Rescorla-Wagner model, conditioning is a variation of associative strength. It relates the Conditioned Stimulus to the Unconditioned Stimulus, given consideration to their intensities as parameters. The previous statement means that a stimulus is gradually conditioning as long as it serves as an unanticipated predictor of its consequences [12]. Let's go back to the example of the dog and the bell: every test where the dog receives a Conditioned Stimulus (the bell ring) develops an associative strength related to the prediction of the food appear (Unconditioned Stimulus). The associative strength will increase on every test performed, so, as the number of tests increases, the dog's associative strength will increase to 100%. The last occurs when the subject is sure about the event; in this case, the dog can be sure that the food will appear after the bell sound. In other words, the objective of the Rescorla-Wagner model is to describe the association between conditioned and unconditioned stimuli based on the degree to which the Conditioned Stimulus takes a predictive value about the occurrence of Unconditioned Stimulus. The model holds that organisms can learn that an event causes a consequence depending on how predictable is that consequence based on another event [7].

Formally, Eq. (1) presents the linkage between stimuli and the change of the associative strength:

$$\Delta v_i = \alpha\beta\left(\lambda - \sum v_i\right) \quad (1)$$

where:

- v_i is the change in associative strength for Conditioned Stimulus on one trial.
- α and β represent the Conditioned Stimulus and Unconditioned Stimulus intensities, respectively. Their values vary from 0 to 1.
- λ is the asymptotic or the maximum value of learning. Characteristically, $\lambda = 1$ when the Unconditioned Stimulus is presented and $\lambda = 0$ when it is not presented.
- $\sum v_i$ is the sum of the associative strengths of all stimuli present.

The Rescorla-Wagner model is considered the most influential associative model of Pavlovian Conditioning in the last quarter of a century. Due to its findings on conditioning appear to bear similarity to human inference is adopted as a model for learning, categorization, and causal inference [16].

3 Causal Bayesian Networks: Artificial Intelligent

The intervention operator (called do or set) is an operator proposed by Judea Pearl in 1995 and works from a BN; in these models, each node in the graph corresponds to a variable in the dataset, and arcs represent the conditional dependency relationships between them. Pearl suggests conventional BN can be used to learn causality. His proposal consists of performing surgery on the BN generated from the dataset so that the new network identifies causal relationships [17].

For carrying out a successful intervention through the do-operator, it is necessary to remove all the arcs received on the cause node and then extract the new probability expression (causal model). For that, it should be considered the d-separation criterion; so that the new BN (now we call it Causal Network or Mutilated Network) be equivalent to the original one in terms of the Markov blanket [9].

The probability model of a BN extracted from a directed acyclic graph is given by Eq. (2) [9]:

$$P(x_1, \dots, x_n) = \prod_i P(x_i | pa_i) \quad (2)$$

where,

- $P(x_1, \dots, x_n)$ is the joint probability of the BN.
- $P(x_i | pa_i)$ is the conditional probability of each variable given its parents.

Once it is done the surgery into the graph, some inference rules proposed by Pearl are applied,¹ and subsequently, the new probability expression is extracted, which is obtained from Eq. (3) [9]:

$$P_x(v) \prod_{i|V_i \in X} P(v_i | pa_i) \quad (3)$$

where,

- $P_x(v)$ denotes the probability resulting from the intervention $do(X = x)$ for the variable X .
- $P(v_i | pa_i)$ denotes the probability of each variable along the causal path (set V) given its parents.

¹ Details for the application of the inference rules in: Pearl, J.: Causality: models, reasoning and inference. Cambridge, UK: Cambridge University Press. 19 (2000). Chapter 3, Sect. 3.4.2 Inference Rules.

To exemplify, we present some results obtained by the operator. First, a brief explanation of its implementation and the weaknesses it presents for causal estimation. Second, our proposal for its possible solution by the integration of Causal Bayesian Networks and the Rescorla-Wagner model.

Example 1 Figure 2 shows the conventional structure of a BN. It was obtained in the R package, using a method based on metric and search (BIC and Hill-Climbing, respectively).

Suppose we want to intervene Landscape Transformation variable (*landtrans* on the network) to see its effect and possible potential cause on the Ecological Integrity variable (*eicclass* on the network). If we can see in the network of Fig. 2, the *landtrans* variable has 3-parents, which allows it to perform the mutilation in the network adequately. Figure 3 presents the new structure (mutilated network).

Fig. 2 Bayesian network obtained from the data. The green nodes represent the cause-and-effect variables

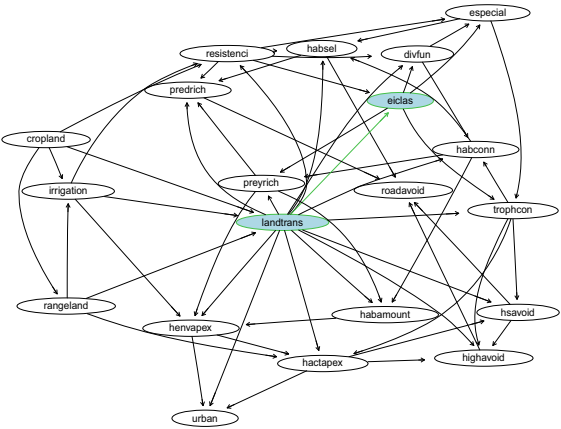


Fig. 3 Intervened Causal Network for the variable cause *landtrans*. The blue nodes represent the causal route

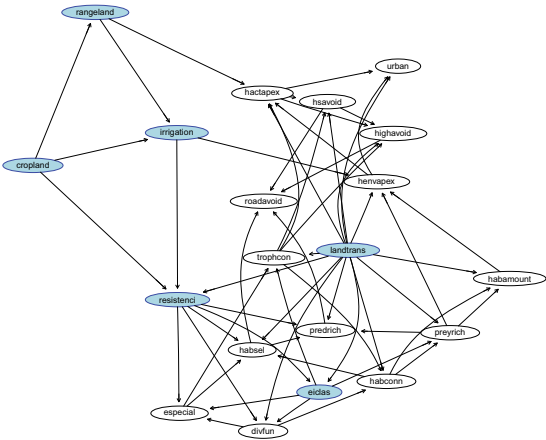
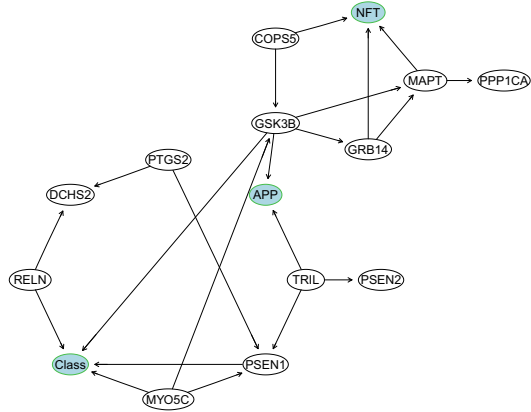


Fig. 4 Bayesian network for expression levels. The green nodes represent the cause-and-effect variables



From a mutilated network, it is possible to extract the new causal probability expression (see Eq. (3)) using the inference rules proposed by Peral [9].

Example 2 The BN shown in Fig. 4 represents gene expression levels related to Alzheimer’s disease.

Suppose that the variables *NFT* or *APP* could be a cause. In this case, the intervention is not possible because when the algorithm deletes parents of the causal node; will be deleted all causal paths too. So, the system will return that the intervention of causal variable (*NFT* or *APP*) about effect variable (*Class*) is equal to *Class*, i.e., the probability value of the *Class* variable is the same before and after the intervention.

Based on the above examples, it is possible to appreciate that the intervention of variables through the do-operator has some limitations. A disadvantage of the do-operator is that it does not have the ability to select or indicate the best route to build the BN—since it starts from a ready-made network -, as seen in Example 2.

Therefore, a central point of this research is to integrate the Rescorla-Wagner model in order to construct the BN and subsequently intervene on variables through the do-operator.

4 The Algorithm CBN-RW

This section describes the algorithm for building Causal Bayesian Networks with the integration of the Rescorla-Wagner model. We will start with the description of the measures used to perform the hypothesis tests, followed by the description of the *RW* library from which we chose the learning rates of the Rescorla-Wagner model. Finally, we present the CBN-RW algorithm, which allows us to build graphic models from a dataset.

4.1 Information Measures from Hypothesis Tests

As we mentioned in Sect. 3 in a typical BN, the connections between each pair of variables represent the dependence between them. One way to find this dependence is through hypothesis tests. These tests allow us to identify which variables are dependent; the tests can be per pair of variables or among two variables given a set of other ones.

For the construction of BNs, the algorithm proposed for this work used the information measures known as Mutual Information and Mutual Conditional Information proposed by Cruz 2001 [18].

Through Eq. (4) is estimated the Mutual Information between a pair of variables.

$$H(X; Y) = I(X, Y) = -\sum_{x,y} P(x, y) \log \frac{P(x, y)}{P(x)P(y)} \quad (4)$$

The Mutual Information also represents the Information Gain that the variables contribute to the model. So that the variables with the highest gain could be considered to form part of the graphic model we are looking for, but not before passing the corresponding hypothesis tests.

In case of requiring the Mutual Conditional Information, that is, two variables given a set of other variables, use is made of the Eq. (5).

$$H(X; Y|Z) = I(X, Y|Z) = -\sum_{x,y,z} P(x, y, z) \log \frac{P(x, y|z)}{P(x|z)P(y|z)} \quad (5)$$

We will try to find from the Eqs. (4), and (5) there is a dependence between X and Y , using that dependence as an indicator to place an arc between both variables. Both equations assume that the probability distributions involved are known. However, this is not always possible with real datasets; since most of them correspond to data samples. So, we will express the calculated terms as estimators, that is, $\hat{H}$ and $\hat{I}$. Kullback proposes the use of these estimators as measures to determine the dependence of two variables, and for this, he makes use of the Eq. (6) [19].

$$T = 2N\hat{I} \quad (6)$$

where:

- N represents the sample size, that is, the number of cases in the database.
- $\hat{I}$ is the Mutual Information or Mutual Conditional Information calculated for the pair of variables X and Y to be tested.

According to Kullback, the T statistic is asymptotically distributed as a χ^2 with $(X - 1)(Y - 1)$ degrees of freedom for the case of Mutual Information; and for the Mutual Conditional Information with $(X - 1)(Y - 1)Z$ degrees of freedom. Where X and Y represent the cardinality of the number of values that the variables X and

Y can take, on the other hand, Z represents the multiplication of the cardinalities of the number of values that each variable can take within the Z set [19].

Once we have estimated the T statistic and the χ^2 (with its respective degrees of freedom), we must perform the hypothesis test. For which we have that, reject H_0 if [19]:

$$T \geq \chi^2_{(X-1)(Y-1)}(\alpha) \quad (7)$$

$$T \geq \chi^2_{(X-1)(Y-1)Z}(\alpha) \quad (8)$$

The last indicates that X and Y are independent, so we cannot place a union between both variables.

4.2 Package ‘Ndl’

In this section, we present details for obtaining the Learning Rate corresponding to the Rescorla-Wagner model. For that, we did employ the *ndl* package implemented in the R program. This package performs learning and classification models based on the Rescorla-Wagner equations and their equilibrium equations [20]. The package provides three types of functionalities: (1) discriminative learning based directly on the Rescorla-Wagner equations, (2) a function that implements the naive discriminative reader, and a model for silent (single word) reading, and (3) a classifier based on the equations of equilibrium.

We use the following functions to find the values corresponding to learning through the Rescorla-Wagner model [21].

- The *RescorlaWagner* function presents an iterative simulation based on the Rescorla-Wagner equations. Through a naive discriminative learning model and classification, it bases the Rescorla-Wagner equations.
- *ndlCuesOutcomes* function is using to indicates the cause-and-effect variables in a specific formula. This formula has the form $Y \sim X$. Where Y represents the effect variable and X is a set of possible cause variables.
- *estimateWeights* function uses the equilibrium equations for the Rescorla-Wagner model of Danks; this function estimates the weights (associative strengths) for cue-outcome pairs.
- *estimateActivations* function uses the equilibrium association strengths (weights) for the Rescorla-Wagner model to estimate. This function allows estimate the activations for outcomes.

4.3 Description of the CBN-RW Algorithm

1. The algorithm has been configured to work only with discrete variables.
2. Initial hypothesis tests will be performed for all pairs of variables in the data set, according to those described in Sect. 1.1 Eq. (4).
3. If the test indicates that the variables are not independent, the algorithm will place a directionless arc between variables and save that connection.
4. Once the hypothesis tests for each pair of variables (X, Y) have been completed, the hypothesis tests are performing for 3-sets of them ($X, Y|Z$). According to described in Sect. 1.1 Eq. (5).
5. If the hypothesis indicates that X and Y are conditionally dependent given Z , the algorithm will calculate the Associative Strength between X and Y according to describing in Sect. 1.2.
6. If the Associative Strength is major o equal to the maximum Learning Rate given by the user, the algorithm searches the connection ($X - Y$) on the initial set (Step 3), and if this exists, will lend a direction from X to Y . Else, if the connection does not exist it will be added, respecting the orientation from X to Y .
7. If the Associative Strength is minor to the Learning Rate, the algorithm will search the connection in the initial set (Step 3), and if it exists must be eliminated.
8. If the hypothesis test indicates that X and Y are conditionally independent given Z , the algorithm will search it in the previous set (Step 3), and in case of exists must be removed.
9. Once the tests for the conditional set are complete, the algorithm obtains the final connections. These connections allow us to recognize causes-and-effects. That is, it lets to identify the direction of the arrow between the nodes to draw the final CBN with them.

5 Selection of Experimental Datasets

To perform the data collection, we rely principally on datasets from repositories. To carry out causal analysis or Bayesian Networks have been proposed some of these datasets (Table 1).

The *Variables* column indicates the total number of variables per dataset; the *Cases* column contains the total number of cases for each one; in the end, the *Source* column tells us where comes from each dataset.

It is important to remember that our algorithm proposal is exhaustive; therefore, depending on the number of variables per set, the processing time to find a network can be very high.

In Sect. 6, we present the Causal Bayesian Networks obtained with our proposal for each dataset. In addition, we performed the intervention of variables on each of

Table 1 Description of the source of each dataset and its general characteristics

Dataset	Variables	Cases	Source
<i>Brca</i>	4	286	Repository [22]
<i>COVID_AC</i>	10	894	Real data_repository [23]
<i>COVID_BC</i>	10	7602	Real data_repository [23]
<i>COVID_CDMX</i>	10	21,038	Real data_repository [23]
<i>LUCAS</i>	12	2000	Repository [24]

the networks; we did that in the different possible potential causes, and we compared with a traditional AI algorithm used for the construction of conventional Bayesian Networks.

The construction of conventional BNs was carried out in R through the *bnlearn* package. We did use the Hill-Climbing algorithm with the Bayesian Information Criterion score (*bic*) and multinomial log-likelihood (*loglik*) score.

- *bic* is equivalent to the MDL score, which is also known as the Schwarz Information Criterion.
- *loglik* is the equivalent of the entropy measure used by the Weka software.

6 Results

In this section, we present the most relevant results for the set of tested databases. Likewise, we will show some of the Causal Bayesian Networks -before and after the intervention of variables-, which we obtained with our algorithm (CBN-RW) and the traditional algorithm (Hill-Climbing); this last ran with two metrics (HC-BIC and HC-LOGLIK).

We have intervened the same cause variables in each network obtained by both algorithms (CBN-RW and Hill-Climbing), with the final purpose to estimate the proportion of Causal Routes found with each of them.

The figures below show the CBN, and the conventional BN obtained with the dataset called *Brca*.

Figure 5 shows us the CBN acquired with the CBN-RW algorithm and Fig. 6 with the Hill-Climbing algorithm ((a) HC-BIC and (b) HC-LOGLIK). The aim is to find the most number of arrows that arrived at the *Menopause* variable. After obtaining both networks, we did intervention of variables in them, selecting the variables: *Recurrence*, *Age*, and *Irradiation*.

Suppose we want to intervene the variable *Age* in the three networks. Once the inference rules proposed by Pearl are applied and, all the nodes that reach the cause variable are disconnect, it is possible to find a causal route in the network built with CBN-RW (see Fig. 7).

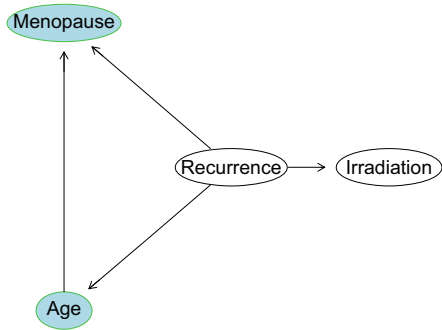


Fig. 5 Causal Bayesian Network created with CBN-RW, dataset *Brca*. The green nodes represent the cause-and-effect variables

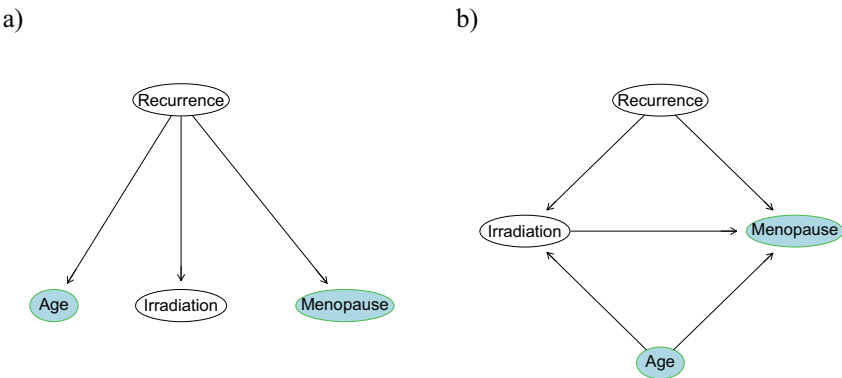


Fig. 6 Causal Bayesian Network created with Hill-Climbing algorithm (**a** HC-BIC and **b** HC-LOGLIK), dataset *Brca*. The green nodes represent the cause-and-effect variables

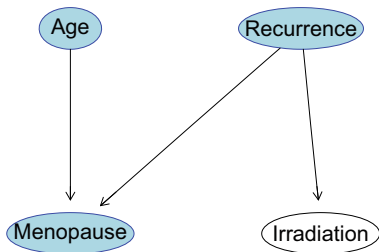


Fig. 7 CBN-RW mutilated after the intervention; data set *Brca*, cause *Age*, and effect *Menopause*. The blue nodes represent the causal route

However, it was not feasible to obtain a causal route for the network built through HC-BIC (see Fig. 8a). On the other hand, the network built from HC-LOGLIK gets 3-causal routes, one per intervened variable. Although, in each intervention, the causal routes involve all the nodes in the model, as we can see in Fig. 8b. For the above, we get expressions of causal probability very long, which will complicate subsequent calculations for estimating causal probability.

Another dataset that we analyze corresponds to cases of COVID in Mexico. We start with the set corresponding to Baja California state (dataset *COVID-BC*). In Fig. 9 we present the network obtained with the CBN-RW algorithm created from these data.

From the same dataset of COVID corresponding to the Baja California state, we have obtained the BN with the traditional algorithm of AI (HC-BIC and HC-LOGLIK); the resulting network is shown in Fig. 10.

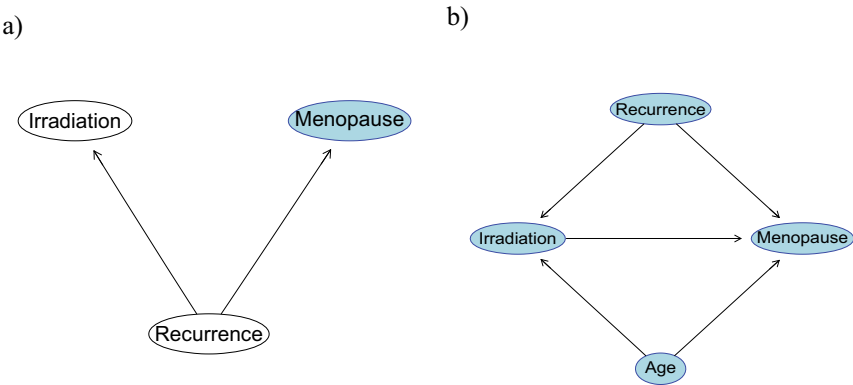
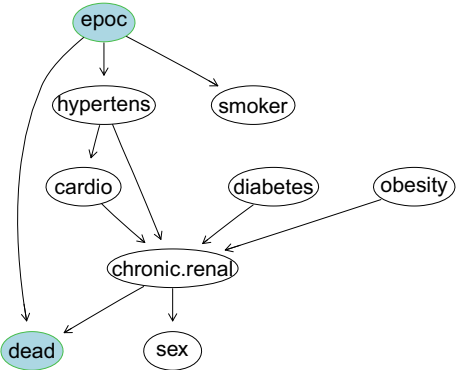


Fig. 8 CR-HC (a HC-BIC and b HC-LOGLIK) mutilated after the intervention; data set *Brca*, cause *Age*, and effect *Menopause*. The blue nodes represent the causal route

Fig. 9 Causal Bayesian Network created with CBN-RW, dataset *COVID-BC*. The green nodes represent the cause-and-effect variables



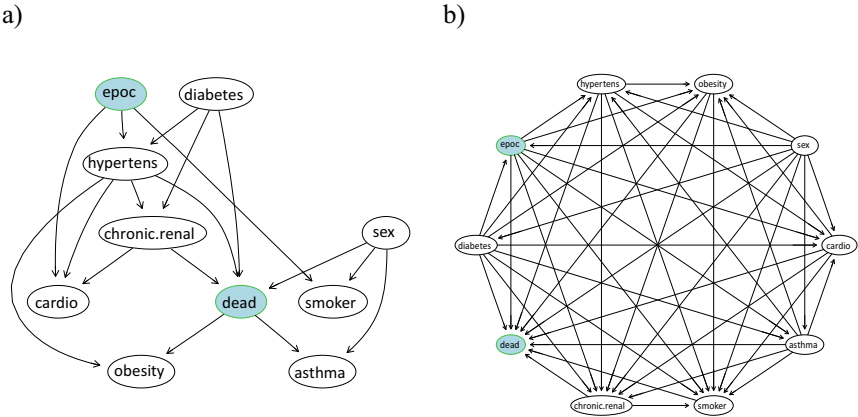


Fig. 10 Causal Bayesian Network created with Hill-Climbing algorithm (**a** HC-BIC and **b** HC-LOGLIK), dataset *COVID-BC*. The green nodes represent the cause-and-effect variables

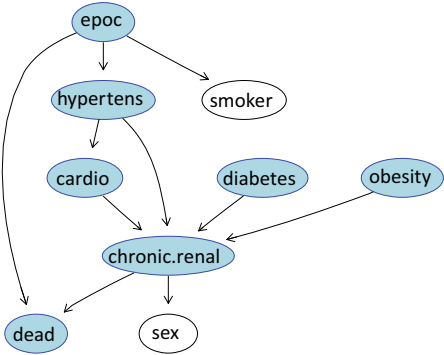
We can see that, from this dataset, the traditional algorithm built a network more connected than our proposed algorithm; having, 10-arcs with the proposed algorithm, 17-arcs with the traditional HC-BIC, and 45-arcs by HC-LOGLIK.

From the network built with our proposal, we have 8-variables to intervene, for which we find 6-causal routes in total. In Fig. 11, we show the intervened network for the *epoc* variable.

From the BN of Fig. 10, we also intervened the *epoc* variable; the resulting mutilated network is shown in Fig. 12. With this network (Fig. 10a), we had a total of 9 possible causes to intervene, of which only 4-causal routes we found. Concerning the intervention about HC-LOGLIK (Fig. 10b), we obtained 9-causal paths of 9-intervened variables anew the routes involve all the nodes in the network.

Another set of data corresponds to COVID cases in the state of Aguascalientes (dataset *COVID-AC*), for which we also obtained three networks, one per each algorithm. Figure 13 shows the network obtained through CBN-RW, and Fig. 14 shows

Fig. 11 CBN-RW mutilated after the intervention; data set *COVID-BC*, cause *epoc*, and effect *dead*. The blue nodes represent the causal route



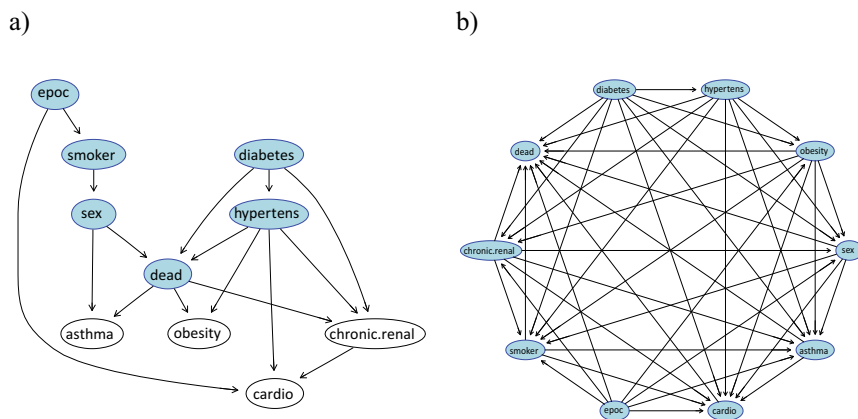


Fig. 12 CR-HC (a HC-BIC and b HC-LOGLIK) mutilated after the intervention; data set *COVID-BC*, cause *epoc*, and effect *dead*. The blue nodes represent the causal route

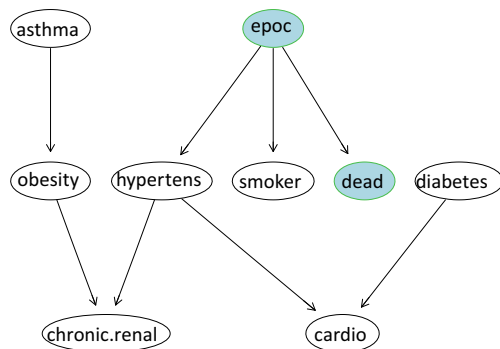


Fig. 13 Causal Bayesian Network created with CBN-RW, dataset *COVID-AC*. The green nodes represent the cause-and-effect variables

the network create with HC for the two metrics.

This example is a case in which the connections towards the effect-variable in the traditional HC-BIC algorithm are more favorable than our proposal. Wherefore that for the total of potential causes intervened (8 in total), we find three causal routes with HC-BIC, and with CBN-RW just one. Notwithstanding the HC-LOGLIK algorithm found 7-causal routes of the 9-intervened variables, this algorithm obtained a total of 45-arcs. This last provoked that each intervention has involved all the nodes in the network, indicating that the new probability equations be longer.

Figure 15 shows the causal route obtained from the *epoc* variable to the *dead* variable with the CBN-RW algorithm; about the *COVID-AC* dataset. Due to applying inference rules, we can observe only two arcs as part of the route between the possible

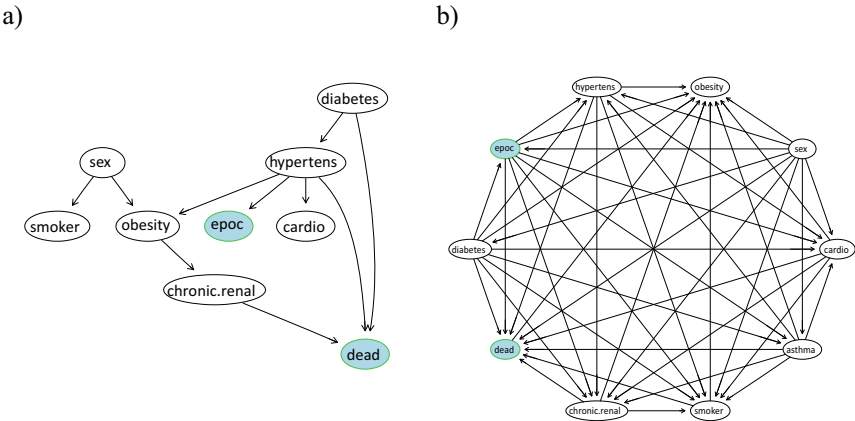
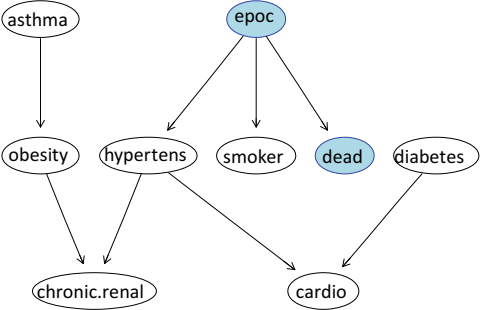


Fig. 14 Causal Bayesian Network created with Hill-Climbing algorithm (**a** HC-BIC and **b** HC-LOGLIK), dataset *COVID-AC*. The green nodes represent the cause-and-effect variables

Fig. 15 CBN-RW mutilated after the intervention; data set *COVID-AC*, cause *epoc*, and effect *dead*. The blue nodes represent the causal route



cause-and-effect variables. This result answers the autonomy assumption proposed by Pearl and that we have mentioned in the Sect. 3.

An additional dataset of COVID also tested corresponds to those collected for Mexico City (dataset *COVID- CdMX*). Figure 16 shows the network generated with our algorithm. With this dataset, we got 25 arcs with our proposed, 22 with the conventional algorithm (HC-BIC), and 45 by HC-LOGLIK (Fig. 17).

We carried out a total of 27 interventions on this dataset; 9 for the network obtained with our algorithm and 9 for each network obtained with the traditional algorithm. In total, we found 7-causal routes with our proposal and 4 with each original network (HC-BIC and HC-LOGLIK). In Fig. 18, we present an example of the causal path obtained when we intervened *epoc* variable in the causal network.

We also tested a dataset called LUCAS. We obtained the dataset from a repository that has been proposed in a challenge to perform causal analysis. The total number of arcs obtained for this dataset with our algorithm was 25, 14 arcs with the traditional AI algorithm (HC-BIC), and 66 arcs by HC-LOGLIK (see Figs. 19 and 20).

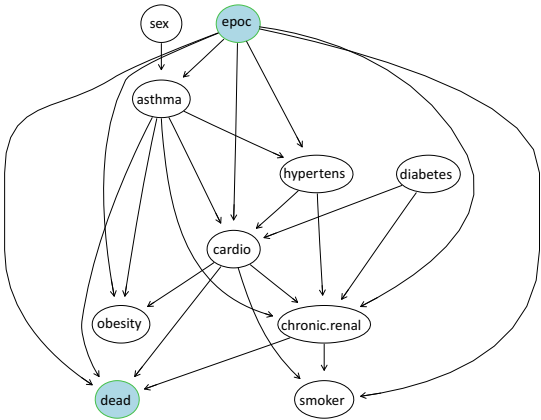


Fig. 16 Causal Bayesian Network created with CBN-RW, dataset *COVID- CdMX*. The green nodes represent the cause-and-effect variables

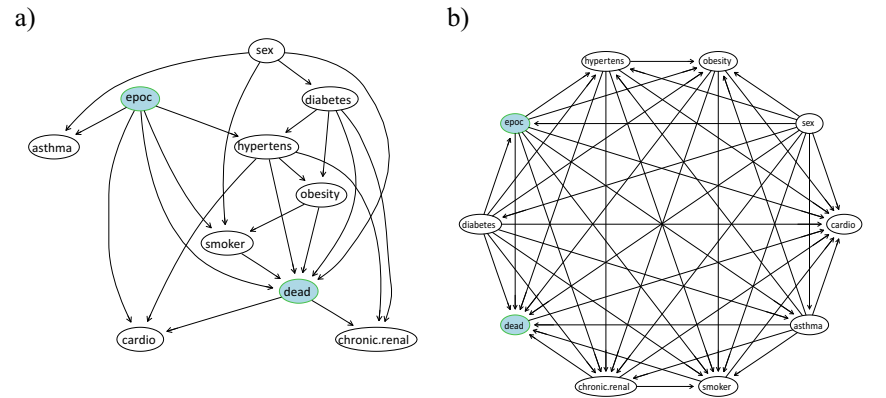
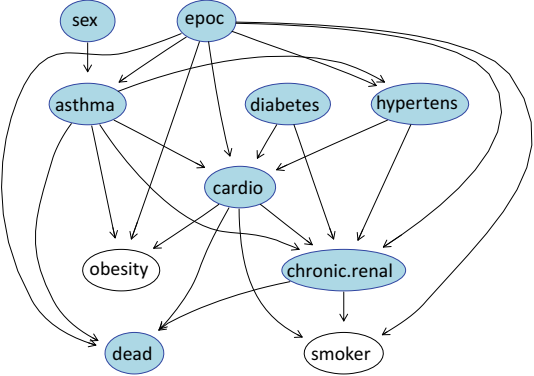


Fig. 17 Causal Bayesian Network created with Hill-Climbing algorithm (**a** HC-BIC and **b** HC-LOGLIK), dataset *COVID-CdMX*. The green nodes represent the cause-and-effect variables

Fig. 18 CBN-RW mutilated after the intervention; data set *COVID-CdMX*, cause *epoc*, and effect *dead*. The blue nodes represent the causal route



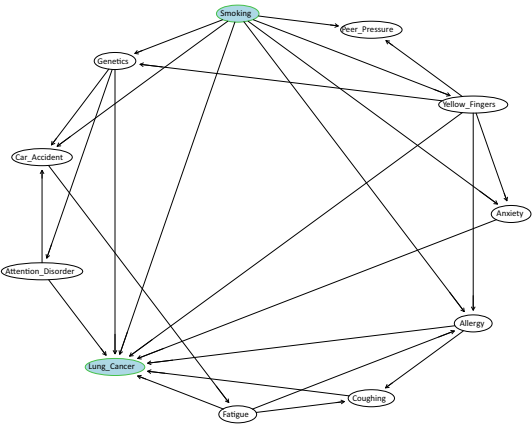


Fig. 19 Causal Bayesian Network created with CBN-RW, dataset *LUCAS*. The green nodes represent the cause-and-effect variables

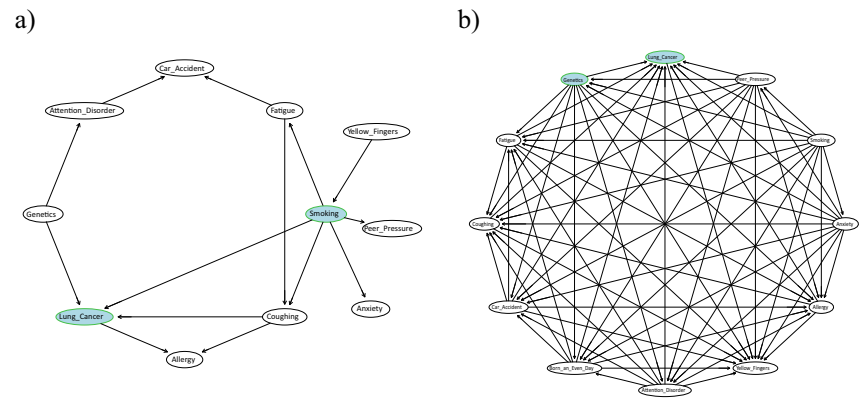


Fig. 20 Causal Bayesian Network created with Hill-Climbing algorithm (**a** HC-BIC and **b** HC-LOGLIK), dataset *LUCAS*. The green nodes represent the cause-and-effect variables

Figure 21 shows us an intervention on the CBN-RW. For the intervention, we have selected the *Smoking* variable as a possible cause and *Lung Cancer* as the effect. The interventions made to the networks with this dataset only gave good results with the proposed algorithm; it found 2-routes in the two intervened variables, while the traditional algorithm did not find routes in either of its two versions.

As a complement, we present Table 2, which shows the total of interventions carried out in each causal and conventional network. It is necessary to highlight that due to the BNs building techniques, some variables may not appear in the final model; for this reason, the number of nodes may be less than the total number of variables in the original datasets.

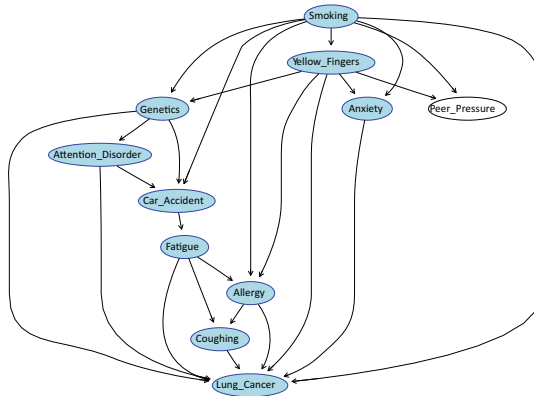


Fig. 21 CBN-RW mutilated after the intervention, data set *LUCAS*, cause *Smoking*, and effect *Lung Cancer*. The blue nodes represent the causal route

From the results shown in Table 2, it is possible to see that the number of arcs obtained by CBN-RW, and HC-BIC algorithms is approaching, in contrast with the traditional HC-LOGLIK algorithm. For example, all networks built with our proposal got 72-arcs, and with the HC-BIC algorithm, 66-arcs in total. However, with the HC-LOGLIK algorithm, the total number of connections was equal to 207. This result is necessary highlighting because having more arcs does not guarantee that the algorithm finds more causal routes.

We have obtained 35% of Causal Routes from the interventions made on the networks built with HC-BIC; compared to our algorithm propose, we have 22% more causal routes found from the interventions; that is, 57% of the interventions find a causal path. Finally, the HC-LOGLIK algorithm found 15% causal routes more than our proposal. However, all these routes imply that most nodes or all nodes are in the new probability expression, which will complicate the calculation to obtain the causal estimate.

We must remember that the intervention proposal made by Judea Pearl assumes that only the minimum nodes necessary should be involved to access a causal probability -on each intervention-, this set being sufficient to predict the effect, having external interventions with a minimum of information.

7 Conclusions

As we commented at the beginning of this chapter, the construction of BN with traditional AI techniques can result in graphic models that do not meet the minimum specifications to perform intervention of variables, or, for the cases in which it is possible to do the interventions, the equations extracted in each of them are very complex. According to Pearl, through each intervention, it is possible to identify

Table 2 Presents the results obtained by each algorithm. The table shows the number of **Nodes** (variables) contained on each dataset, the total number of **Arcs** obtained by the algorithms, the number of **Causes** per intervened, the number of Causal Routes found (**CR**), and only for the case of the CBN-RW algorithm the Learning Rate (**LR**) with which it was executed

Dataset	CBN-RW					BN-HC (BIC)					BN-HC (LOGLIK)				
	Nodes	Arcs	Causes	CR	LR	Nodes	Arcs	Causes	CR	Nodes	Arcs	Causes	CR	Nodes	Arcs
<i>Brca</i>	4	4	3	1	1	4	3	3	0	4	6	3	3		
<i>COVID_AC</i>	9	8	8	1	0.99	9	10	8	3	10	45	9	7		
<i>COVID_BC</i>	9	10	8	6	0.99	10	17	9	4	10	45	9	9		
<i>COVID_CDMX</i>	10	25	9	7	0.99	10	22	9	4	10	45	9	4		
<i>LUCAS</i>	11	25	2	2	0.90	11	14	2	0	12	66	2	0		
Totals	43	72	30	17	4.87	44	66	31	11	46	207	32	23		
Proportion of causal routes found by algorithm	CBN-RW					BN-HC (BIC)					BN-HC (LOGLIK)				
	0.57					0.35					0.72				

the effect that one variable has on another with the least amount of information, so the probability expressions that include all the nodes of the network or most of them, do not comply with this assumption, not to mention that the more complex the probability expression, the more complex the probability calculation.

The objective of obtaining graphic models in that the intervention of variables is possible; in addition to extracting the new probability equations, is to calculate the causal effects, which indicate the strength with which one variable can be the cause of another.

As we could observe in the results, change the algorithm (Hill-Climbing) or the learning rate (RW) in the construction of the networks provides significant changes in the results; notably, the models can be very complex each time we make any of these changes.

BIC metric has been widely used for the construction of graphic models because it balances the adjustment of the model and its complexity. We could observe the above when we changed to the Loglik metric; although this got most causal routes, the resulting graphic models presented much greater complexity, causing most of the paths found to involve the entire network in each of the interventions carried out.

It is necessary to highlight that Judea Pear assures that through the intervention of variables, the paths between causes and effects will only include the nodes involved, which he calls autonomy in the mechanisms; the latter cannot be validated with highly connected models since, in these models, the nodes involved in each intervention include the whole networks. Also, the new probability expression obtained from the causal path will become more complex with highly connected networks. Let us also remember this to extract each causal route. It is necessary to select each node given its parents that will exponentially increase the algorithm complexity as the datasets grow and the connections in the networks also grow.

An important point to highlight is that we found some equivalence between the traditional AI model with the BIC metric and our proposed model since the number of final connections (arcs) obtained with both algorithms is not very variable. However, our algorithm has better results in terms of intervention, finding more causal routes, which indicates that we can obtain more causal probabilities.

As we mentioned at the beginning of this document, the gathering of causal routes will be an ally to continue with the estimation of causality that, although this depends on the data and its distribution, it is possible to access it once it is gotten the equation of probability; just as we did with our proposal.

Finally, once obtaining the causal routes, we are ready to estimate the causal effects; these are the final objective of the causal estimation and our motivation for proposed the algorithm for the construction of CBNs.

Acknowledgements This research was supported Consejo Nacional de Ciencia y Tecnología (CONACYT), Mexico, CVU: 860640.

References

1. Danks, D.: Privileged (Default) causal cognition: a mathematical analysis. *Front. Psychol.*, 9 (2018). <https://doi.org/10.3389/fpsyg.2018.00498>
2. Eysenck, M.W., Keane, M.T., Bucciarelli, M.: *Psicología cognitiva*. Idelson-Gnocchi (2006)
3. Smith, E.E.: Cognitive psychology: history. In: *International Encyclopedia of the Social & Behavioral Sciences*, pp. 103–109. Elsevier (2015)
4. Sloman, S.: *Causal Models: How People think about the World and its Alternatives*. Oxford University Press (2005)
5. Griffiths, T.L., Tenenbaum, J.B.: Structure and strength in causal induction. *Cogn. Psychol.* **51**, 334–384 (2005). <https://doi.org/10.1016/j.cogpsych.2005.05.004>
6. Perales, J.C., Catena, A., Ramos, M.M., Maldonado, A.: Aprendizaje de relaciones de contingencia y causalidad: Una aproximación a las tendencias teóricas actuales. *Psicológica* **20**, 163–193 (1999)
7. H Vogel, E., A Soto, F., E Castro, M., A Solar, P.: Modelos matemáticos del condicionamiento clásico: evolución y desafíos actuales. *Revista Latinoamericana de Psicología* **38**(2), 215–243 (2006)
8. Yarlett, D.G., Ramscar, M.J.: Causal Induction and the Revision of Belief. *Proceedings of the Annual Meeting of the Cognitive Science Society*, p. 25 (2003)
9. Pearl, J.: *Causality: Models, Reasoning and Inference*, p. 19. Cambridge University Press, Cambridge, UK (2000)
10. Pearl, J.: Graphical Models for Probabilistic and Causal Reasoning. In: *Quantified Representation of Uncertainty and Imprecision*. Springer Netherlands, pp. 367–389 (1998)
11. Vazquez, A.J.: Evaluación del operador de intervencion en redes bayesianas causales. *Centro de Investigación en Inteligencia Artificial, Universidad Veracruzana* (2010)
12. Rescorla, R.A.: A theory of Pavlovian conditioning: Variations in the effectiveness of reinforcement and nonreinforcement. *Current research and theory*, 64–99 (1972)
13. Pellón, R.: Watson, Skinner y Algunas Disputas dentro del Conductismo. *Revista colombiana de psicología*. **22**(2), 389–399 (2013)
14. Ormrod, J.E., Sanz, A.J.E., Soria, M.O., Carnicero, J.A.C.: *Aprendizaje humano*. Madrid, Spain: Pearson Educación., 4 (2005)
15. Pérez, A.M., Cruz, J.E.: Conceptos de condicionamiento clásico en los campos básicos y aplicados. *Interdisciplinaria*. **20**(2), 205–227 (2003)
16. Cheng, P.W.: From covariation to causation: a causal power theory. *Psychol. Rev.* **104**, 367–405 (1997). <https://doi.org/10.1037/0033-295x.104.2.367>
17. Pearl, J.: Causal diagrams for empirical research. *Biometrika* **82**, 669–688 (1995). <https://doi.org/10.1093/biomet/82.4.669>
18. Cruz, R.N.: *Building Bayesian Networks from data: a constraint- based approach*. Ph.D. thesis, Disertación doctoral, Universidad de Sheffield, Sheffield, Gran Bretaña (2001)
19. Kullback, S.: *Information Theory and Statistics*. Dover, New York (1959)
20. Arppe, A., Hendrix, P., Milin, P., Baayen, R. H., Sering, T., & Shaoul, C.: Package ‘ndI’ (2018)
21. Danks, D.: Equilibria of the Rescorla-Wagner model. *J. Math. Psychol.* **47**(2), 109–121 (2003)
22. Current protocols: Basic Protocol 2: testing DAGs against categorical DAT. <https://doi.org/10.1002/cpz1.45>
23. Covid-19 México: Official web (2020). <https://datos.covid-19.conacyt.mx/#DOView>
24. K4A: Causality Challenge #1: Causation and Prediction Challenge (2008): <https://www.k4all.org/project/causality-challenge-1-causation-and-prediction-challenge/>

Eureka-Universe: A Business Analytics and Business Intelligence System



José Fernando Padrón-Tristán, Laura Cruz-Reyes,
Rafael Alejandro Espín-Andrade, Carlos Eric Llorente-Peralta,
Claudia Guadalupe Gomez-Santillan, Alejandro Castellanos-Alvarez,
and Jordan Michelt Aran-Pérez

Abstract this paper describes the universal analyst: Eureka-Universe, a framework for Business Analytics and Business Intelligence based on compensatory fuzzy logic (CFL), which solves problems that handle vague, incomplete or, inaccurate information. Eureka-Universe facilitates knowledge discovery (KD), knowledge engineering and, decision analysis and modeling. The architecture of Eureka-Universe mainly includes a project manager, a scientific core, and algebraic and graphical editors for developing KD tasks. The project manager allows to create or modify models of a problem. The scientific core performs the KD tasks through an evolutionary algorithm. The text editor executes KD tasks, and the graphical editor displays and modifies FPs seen as fuzzy trees. Eureka-Universe can perform inference tasks by combining the discovery and evaluation tasks; furthermore, it offers interpretability possibilities by analyzing the FPs seen as fuzzy trees in the graphical editor. The Eureka-Universe architecture is described and validated by realistic study cases. Finally, the paper envisions new features to develop as future work for this framework.

Keywords Eureka-Universe · Compensatory fuzzy logic · Fuzzy predicates · Knowledge discovery

J. F. Padrón-Tristán (✉) · C. E. Llorente-Peralta
Tecnológico Nacional de México, Instituto Tecnológico de Tijuana, Tijuana, México
e-mail: d96071173@cdmadero.tecnm.mx

L. Cruz-Reyes · C. G. Gomez-Santillan · A. Castellanos-Alvarez · J. M. Aran-Pérez
Tecnológico Nacional de México, Instituto Tecnológico de Ciudad Madero, Ciudad Madero, México
e-mail: lauracruzreyes@itcm.edu.mx

C. G. Gomez-Santillan
e-mail: claudia.gomez@itcm.edu.mx

R. A. Espín-Andrade
Autonomous University of Coahuila, Saltillo, Mexico

1 Introduction

Business Intelligence (BI) is an interactive process to explore and analyze structured information about an area (typically stored in a data warehouse) to discover trends or patterns to derive ideas and draw conclusions. It includes reporting findings and making changes [1].

The fundamental objective of BI is to support organizations sustainably and continuously to improve their competitiveness, providing the information necessary for decision-making.

Business analytics (BA) is the general term for any data analytics in business problems, while data analytics refers to any analysis of any data type. Hence, BA can be widely applied to almost any area [2].

In a more than 10,000 technology executives survey, 23.5% responded that a BI or BA platform is their next planned software purchase [3].

According to a study by Gartner, the technology category of “analytics and business intelligence” is the top priority of chief information officers. Further, Gartner finds that the top technology priority of chief financial officers is analytics [4].

BA Systems are toolboxes that need users to know algorithms and mathematical and computer sciences theories. On the other hand, Business Intelligence (BI) systems have been evolving towards frameworks which show by diverse sophisticated visualization ways and dashboards, information about the decision making problem, using specific business analytics techniques.

BA and BI systems are not used by decision makers, such as CEOs and entrepreneurs, the real owners of business ideas and knowledge about them, limiting the effective direct use of those systems in business decision-making real problems.

Wide knowledge discovery (WKD) is a transdisciplinary scientific strategy developed by the Eureka Community to contribute to the use and development of BI and BA. WKD integrates knowledge engineering, decision analysis, and knowledge discovery into a single process to reach adequate decision-making support [Axioms].

Eureka Universe contributes to Analytics integrating soft computing strategies like general hybridization, theoretical hybridization, and WKD by compensatory fuzzy predicates. The latter works towards a new transdisciplinary concept of interpretability. Eureka Universe uses predicates as the form of representation, CFL logical system is the way of evaluation, and genetic algorithms as the way of searching; this is called “WKD by fuzzy logic predicates” (WKDFLP).

Eureka-Universe [5] is a unique Business Analytics (BA) and Business Intelligence (BI) system which allows to solve independent tasks and to mix them towards Decision Making Problems solutions, without needing any users knowledge about mathematics and computers sciences theories and algorithms; just their knowledge about the business and common sense for the interaction with a graphical and based on natural language editor.

The buildup of semantic integrators like the Eureka Universe platform is a sound and promising contribution to realizing the augmented analytics paradigm using WKDFLP.

Eureka-Universe has been possible because of the scientific results of the cooperation into the framework of Eureka's Community and its networks Eureka Iberoamerica and Eureka International. They have elaborated transdisciplinary theories based on Fuzzy Logic, which allows modeling and interpretation by natural and professional languages to analyze metacognitive systems and processes like evaluation, decision making, reasoning, and logical and statistical inferences. As a result, transdisciplinary theories based on Fuzzy Logic have been elaborated, which allow decision-making problems modeling and their interpretation by natural and professional languages. Moreover, they are helpful for the analysis of metacognitive systems and processes like evaluation, decision making, reasoning, and inferences.

WKDFLP is possible through Eureka-Universe from the discovery, evaluation, and inference of fuzzy predicates.

2 Background

2.1 Knowledge Discovery

Knowledge is defined as the result of processed information understandable by a human and that will modify the behavior of a human or mechanical agent [6].

Knowledge discovery from data (KDD) is the nontrivial extraction of implicit, previously unknown, and potentially useful information from data [7]. KDD extracts potentially helpful and previously unknown knowledge from vast amounts of data.

KDD creates techniques adapted to such a problem, integrates several knowledge acquisition approaches that generate new solutions and new scientific problems that will create new solutions and new scientific problems [6].

KDD field draws on statistics, databases, and artificial intelligence to construct tools that let users gain insight from massive data sets. Thus, an important KDD goal is to "turn data into knowledge" [8].

Given a set of facts (data) F , a language L , and some measure of certainty C , we define a pattern as a statement S in L that describes relationships among a subset F_S of F with a certainty c , such that S is simpler (in some sense) than the enumeration of all facts in F_S [7].

A pattern that is interesting and certain enough is called knowledge. Thus, the output of a program that monitors the set of facts in a database and produces patterns in this sense is KDD.

Found patterns must fulfill a determined grade of certainty and must be novel, valuable, and nontrivial. On the other hand, the pattern search algorithms must be efficient, meaning that the execution time and space used are expressed by a polynomial function of low degree [7].

Due to the ease of transforming logical propositions into a language close to natural or professional, fuzzy logic is an approach that has an advantage over others that have recently emerged, in addition to its applications in KDD, which are helpful due to its interpretability [9].

2.2 Fuzzy Logic

Fuzzy logic (FL) is a discipline proposed in the sixties by [10] that describes how human thought is constructed not by numbers but by linguistic labels. Although linguistic terms are less precise than numerical data, they express knowledge in terms more accessible to human understanding [11].

Fuzzy logic is a convenient approach for decision-making. Its capacity to incorporate verbal statements into formal modeling is advantageous for a systematic treatment of actual situations. This advantage comes from a proper interaction between the decision-maker's information and expert reasoning [12].

FL defines the principle of incompatibility in which describes that precision and its meaning are exclusive characteristics: "as the complexity of a system increases, our ability to be precise and construct instructions about its behavior decreases" [11].

While classic logic works with absolute values like truth or false and zero or one, FL works with grades of certainty, taking values in the interval $[0, 1]$.

The grades of certainty, the closer the truth value of a proposition is to zero, the closer it is to the value of false; on the other hand, the closer it is to the value of one, the closer it is to the value of true and, the truth value 0.5 means that the logical statement is as true as it is false [13].

Fuzzy predicates

Predicates have been historically used to represent knowledge in formal sciences like mathematics or physics [14].

A predicate is usually the condition of a rule in a certain logic, and the action is an associated class, meaning that we predict an action for an input instance that makes true the condition. It is often used for the market basket or transactional data analysis [15]. Also, Fuzzy predicates can deal with binary relations as well. They are adequate tools for preferences between alternatives and attributes [14].

According to [14], a fuzzy predicate is a function $G(z_1, z_2, \dots, z_n)$, defined in fuzzy variables $z_1, z_2, \dots, z_n$, whose range of values is a statement whose truth value is estimated by values of the interval $(0,1)$. This truth value is obtained from the composition of fuzzy variables (known as individual fuzzy predicates) and fuzzy logic operators.

For fuzzy variables, FL introduces the concept of a membership function:

Let M be a set, x be an element of M . A fuzzy subset A of M is defined as a set of ordered pairs $\{(x, \mu_A(x))\}$, $\forall x \in M$, $\mu_A(x)$ which is a characteristic MF that takes its values in a well-ordered set E , which indicates the degree or level of membership of an element x to a subset A [14].

2.3 Compensatory Fuzzy Logic

Compensatory fuzzy logic (CFL) is a multivalent logic approach that must satisfy the characteristics of a descriptive and normative approach to decision making [16].

CFL is formed by a quartet of continuous operators conjunction (c), disjunction (d), fuzzy strict order (o), and negation (n), which satisfy a group of axioms, among which are those of compensation and veto [17]:

1. The compensation axiom indicates that if a predicate is significantly impacted due to the evaluation of some linguistic variable, it can be compensated by evaluating some other variable included in that predicate:

$$\min(x_1, x_2, \dots, x_n) \leq c(x_1, x_2, \dots, x_n) \leq \max(x_1, x_2, \dots, x_n)$$

2. The veto axiom refers to the fact that the compensation axiom cannot be applied under certain circumstances.

$$\text{if any } x_i = 0, \text{ then } c(x) = 0$$

2.4 Archimedean Compensatory Fuzzy Logic

Compensatory Archimedean fuzzy logic (ACFL) is built from the union of two different fuzzy logic systems: a continuous Archimedean fuzzy logic and a CFL [18].

Archimedean continuous logic is a t-norm and t-conorm logic (triangular conjunction and disjunction, respectively) system, while CFL can be obtained from quasiarithmetic mean operators.

ACFL defines two operators called non-rebuttal and assertion (Table 1). The name

Table 1 Quantifiers for ACFL [18]

Quantifier	Description	Defined by
Universal of No refutation	For the finite set X and the continuous Archimedean t-norm	$\forall_T X_i \in x = c_t(x_1, x_2, \dots, x_n)$
Universal of Affirmation	For the finite set X and the conjunction of a CFL based on quasi arithmetic equations	$\forall_c X_i \in x = c_c(x_1, x_2, \dots, x_n)$
Existential of No refutation	For the finite set X and the continuous Archimedean t-norm	$\exists_T X_i \in x = d_t(x_1, x_2, \dots, x_n)$
Existential of Affirmation	For the finite set X and CFL	$\exists_c X_i \in x = d_c(x_1, x_2, \dots, x_n)$

of the non-refutation operator is because it tends to reject the acceptance that propositions are true, just as the affirmation operator tends to confirm the truth value of each proposition.

ACFL has a property of affirmation and refutation at the same time, which are complementary properties. The operators of these properties are other approaches to the well-known concepts of Necessity and Possibility that appeared in Modal Logic [19] and included in the FL theory of possibility.

For some logical proposition, a quartet is formed by the value of non-refutation of the necessity of the proposition, the value of affirmation of the necessity of the proposition, the value of non-refutation of the possibility of the proposition, and the value of affirmation of the possibility of the proposition [18].

2.5 Genetic Algorithms

Genetic Algorithms (GA) are search algorithms based on concepts of natural selection and natural genetics [20]. GA is highly parallel, mathematical, adaptive search procedures (i.e., problem-solving methods) based loosely on the processes of natural genetics and Darwinian survival of the fittest. These algorithms apply genetically-inspired operators to populations of potential solutions in an iterative fashion, creating new populations while searching for an optimal (or near-optimal) solution to the problem at hand [21].

Next is described the basic iterative model of the genetic algorithms: A new population is created from an existing population employing evaluation, selection, mutation, and reproduction. This process repeats itself until the population converges on an optimal solution or some other stopping condition is reached [21].

Example of a GA algorithm [20].

-
1. formulate initial population
 2. randomly initialize population
 3. repeat
 4. evaluate objective function
 5. find fitness function
 6. apply genetic operators
 7. reproduction
 8. crossover
 9. mutation
 10. until stopping criteria
-

Next are listed main Genetic operators for a GA [22]:

1. **Selection.** The GA algorithm employs a roulette wheel to assign probabilities to individuals and select them for creating the next generation proportional to their

fitness (objective) values. Other selection operators are Boltzmann, Tournament, Rank, Steady-state, Truncation and, Local selection.

2. **Crossover.** Selected individuals are employed to create the new generation. This is simulated by combining two solutions to produce two new solutions. Single-point and double-point are, among others, techniques for the crossover operator in the literature.
3. **Mutation.** The mutation operator maintains the diversity of the population by introducing another level of randomness. In addition, this operator prevents similar solutions and increases the probability of avoiding local solutions in the GA algorithm.

2.6 Fuzzy Inference

In real-life problems, immediate information (data, experience, and reasoning) is used to make not directly observed inferences. Instead, observations of the world reveal other facts or consequences that can be known from the observed data through inference.

In a broad sense, inference ranges from implication to the operative mental process through which a conclusion is reached based on specific information [23].

From formal logic, inference has been understood [24] as “the passage from one set of propositions to another; the first set is called the class of premises and the second the class of conclusions”. This definition points to inference as a syllogistic structure: a three-level logical entity: two premises and a conclusion.

Fuzzy inference is the method to get fuzzy sets from the combinations of other fuzzy sets with rules of the form:

$$\text{IF } X \text{ is } A \text{ THEN } Y \text{ is } B$$

where A is the antecedent and B is the consequent. Also, A and B are linguistic states modeled by fuzzy sets and describes the relation between two variables:

$$X \text{ and } Y : A \rightarrow B$$

This implication is done through an operation between fuzzy sets. Thus then, the membership function of the result can be obtained as the minimum or the product of the membership functions $\mu_A(x)$ and $\mu_B(x)$. Thus, the membership function of the implication IF $A \dots$ THEN $\dots B$ is:

$$\mu_{A \rightarrow B}(x) = \min(\mu_A(x), \mu_B(x))$$

The most common way of dealing with the precision in the inference of fuzzy systems is based on error [25]. The operator For All is used to calculate the accuracy of the fuzzy predicates:

$$\forall p(x) = e^{-\sum \ln[p(x)]}$$

2.7 Fuzzy Interpretability

In [17], a formal definition of interpretability is presented, which is the property that a logical theory fulfills so that there is a bilateral relationship between the results. But, on the other hand, their meanings are represented in a language close to natural or professional.

Interpretability is one of the advantages of KD through fuzzy predicates since it allows the possibility of being translated into the natural language or expressed graphically through graphs, trees, or maps, among others [17].

Interpretability can be subjective; it depends on the prior knowledge and experience of who is in charge of conducting the assessment. Therefore, it is necessary to consider both objective and subjective indices.

Objective indices allow fair comparisons between different fuzzy models designed to solve the same problem.

Subjective indices guide the design of customized fuzzy models, making it easy to consider user preferences and expectations during the design process.

Since interpretability is subjective, this trade-off between interpretability and complexity must also be considered from the human perspective [26].

Integration, interaction, validation, and trust are, among others, reasons to justify the interpretability in the KD employing FL. Trust is the most critical reason since FL allows users to be sure of how the results obtained are produced [27].

3 Eureka-Universe

3.1 Architecture

The architecture of Eureka-Universe includes a (Fig. 1):

Project manager. Through this module, problems are modeled starting from a dataset.

Scientific core. This module performs KD tasks.

Algebraic editor. These modules allow executing KD tasks starting on created projects.

Graphic editor. This module allows the creation or edition of fuzzy predicates viewed as fuzzy trees.

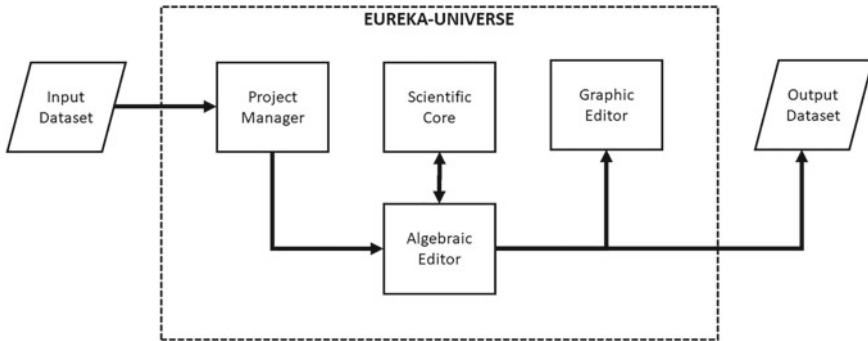


Fig. 1 Eureka-Universe's architecture

3.2 Scientific Core

The scientific core is a search GA that performs evaluation and discovery tasks. It is based on the work of Llorente-Peralta et al. in [13]. In addition, they developed the GA-GPv2, a GA that performs not only mutation but, among other tasks, the optimization of the generic continuous logic variable (GCLV).

GCLV is a special member function (MF) that finds the best parameters that involve the shape of the graphic of the MF through the m parameter in the discovery task. The GCLV can take figure ranges from sigmoidal, gaussian, and negative sigmoidal, among others convex. The formula gives GCLV:

$$GCLV(x, \alpha, \gamma, m) = \frac{\text{sigm}(x, \alpha, \gamma)^m (1 - \text{sigm}(x, \alpha, \gamma)^{1-m})}{M}$$

With $M = m^m (1 - m)^{1-m}$

Evaluation task

The evaluation task computes the truth value of a fuzzy predicate on an instance or an entire dataset; the truth value means the grade of certainty or the degree of fulfillment of the predicate on an instance.

All linguistic variables must be fuzzified; all instances' values are transformed to a range in $[0, 1]$ through MFs. Then, logical operations are performed according to the selected type of logic (Fig. 2).

In Eureka-Universe, predicates are expressed in prefix notation and associated under the logical operator, so the predicate

$$\text{State 1 AND State 2 AND State 3 AND } \dots \text{ AND State } N$$

is showed as

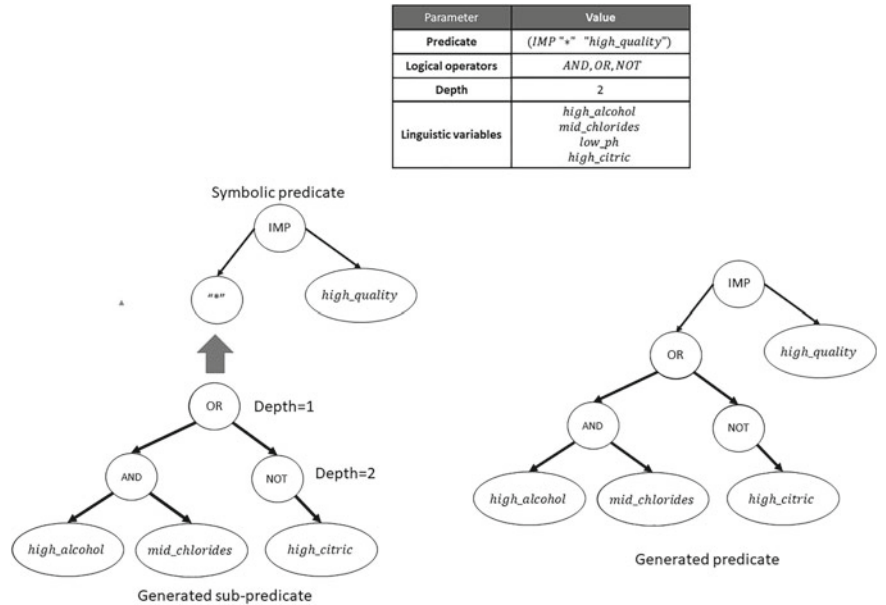


Fig. 2 Generation of fuzzy predicates in discovery task

Table 2 Two instances of wine dataset

Field	Instance 1	Instance 2
Fixed_acidity	7.40	7.90
Volatile_acidity	0.59	0.35
Citric_acid	0.08	0.46
Residual_sugar	4.40	3.60
Chlorides	0.09	0.08
Free_sulfur_Dioxide	6.00	15.00
Total_sulfur_dioxide	29.00	37.00
Density	1.00	1.00
pH	3.38	3.35
sulphates	0.50	0.86
alcohol	9.00	12.80
quality	4.00	8.00

(AND State1 State2 State3 . . . StateN)

For example, the evaluation task was performed using the red-wine dataset. It is wanted to know if the selected attributes contribute to the quality of a set of wines (Table 2).

First, it is built the fuzzy predicate:

Table 3 Linguistic states

Label	Column	MF	Par 1	Value	Par 2	Value
<i>high_alcohol</i>	Alcohol	sigmoid	Γ	11.65	β	9.00
<i>low_PH</i>	PH	Negative sigmoid	Γ	3.38	β	2.93
<i>high_quality</i>	quality	sigmoid	Γ	5.5	β	4.0

Table 4 Fuzzyfied values according to MFs

Field	Instance 1	Fuzzyfied value 1	Instance 2	Fuzzyfied value 2
<i>low_pH</i>	3.38	0.5129	3.35	0.43581
<i>high_alcohol</i>	9.00	0.0100	12.80	0.88017
<i>high_quality</i>	4.00	0.0100	8.00	0.99952

(IMP (AND “*high_alcohol*” “*low_PH*”) “*high_quality*”).

Then the linguistic states are defined in Table 3.

To fuzzyfy values in the instances (Table 4) is used the formula of MF selected for sigmoid, and

$$f(x, \gamma, \beta) = \frac{1}{1 + e^{-\alpha(x-\gamma)}}$$

negative sigmoid

$$f(x, \gamma, \beta) = 1 - \frac{1}{1 + e^{-\alpha(x-\gamma)}}$$

with $\alpha = \frac{\ln 0.99 - \ln 0.01}{\gamma - \beta}$

These fuzzified values in Table 4 replace the linguistic states for every instance to perform the logical operations according to the selected type of logic. The evaluation of the fuzzy predicate using the CFL based on the geometric mean (CFLGMB) for the instances is shown in Table 5.

Table 5 Evaluation of predicate for instances one and two

Predicate	Instance 1	Instance 2
<i>low_pH</i>	0.5129	0.43581
<i>high_alcohol</i>	0.0100	0.88017
<i>high_quality</i>	0.0100	0.99952
(AND “ <i>high_alcohol</i> ” “ <i>low_PH</i> ”)	0.0716	0.09380
(IMP (AND “ <i>high_alcohol</i> ” “ <i>low_PH</i> ”) “ <i>high_quality</i> ”)	0.7337	0.99329

Discovery task

Finds relationships between the linguistic states of a data set that meet user specifications.

The discovery of fuzzy predicates is a task that allows generating knowledge in the form of a list of predicates that fulfills specified characteristics given to the search algorithm.

These characteristics that are the configuration of the search algorithm are, among others:

Symbolic predicate: it specifies the wanted structure of the generated predicates, i.e.:

(AND “*”).

(IMP “*” “*high_quality*”)

Type of logic: ACFL, CFL based on Geometric mean, arithmetic mean, among others [A31].

Depth: maximum number of logical operators to be nested.

Linguistic variables: a list of linguistic variables that may participate in the construction of the algorithm.

The minimum truth value: the minimum truth value of the generated predicates to include in the results.

The maximum amount of results: the limit amount of predicates to be listed in the results.

The “*” state in the symbolic predicate is known as “*star predicate*”, “*generator*” or “*wildcard*”, the generated sub-predicates must fulfill the rest of the listed configuration. Then, these predicates will replace the wildcard (Fig. 2).

3.3 Project Manager

Main window

The main window is a project manager (Fig. 3); through it, it can be possible to:

1. Create a new project
2. Explore created projects

By selecting a project in the project’s explorer:

3. Add a new KDD task
4. Select a KDD task
5. Open a selected task
6. Manage configuration of the project
7. See results of KDD tasks previously performed

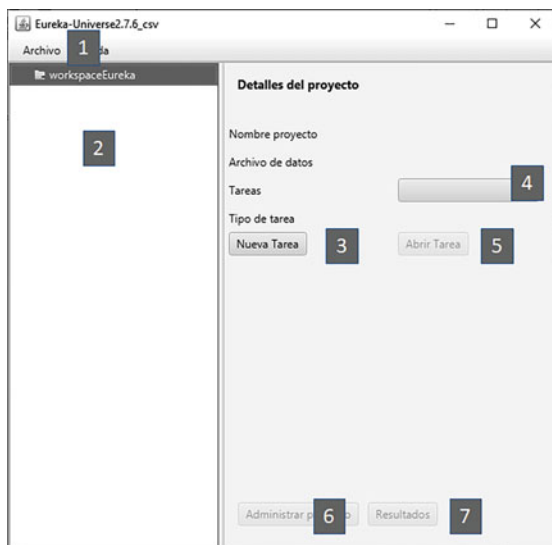


Fig. 3 The main window of Eureka-Universe

New project window

This module associates linguistic variables starting on a dataset as following (Fig. 4):

1. The dataset is selected in the first area
2. The task Combobox allows choosing a default KDD for the project
3. Through this area are added linguistic variables.
4. In the linguistic variables list, these can be edited, modifying their linguistic labels and their membership functions
5. Finally, the project and default KDD are named and saved.

The configuration tab allows to see and edit the settings for the search algorithm in the discovery of predicates task.

3.4 Task Manager

Algebraic editor

The algebraic editor allows adding and modifying linguistic variables and fuzzy predicates from a created project. Also, it allows the current KDD task to be performed and calls to the graphic editor to see constructed predicates.

In Fig. 5 are shown all the sections of the algebraic editor for the discovery task, which are:

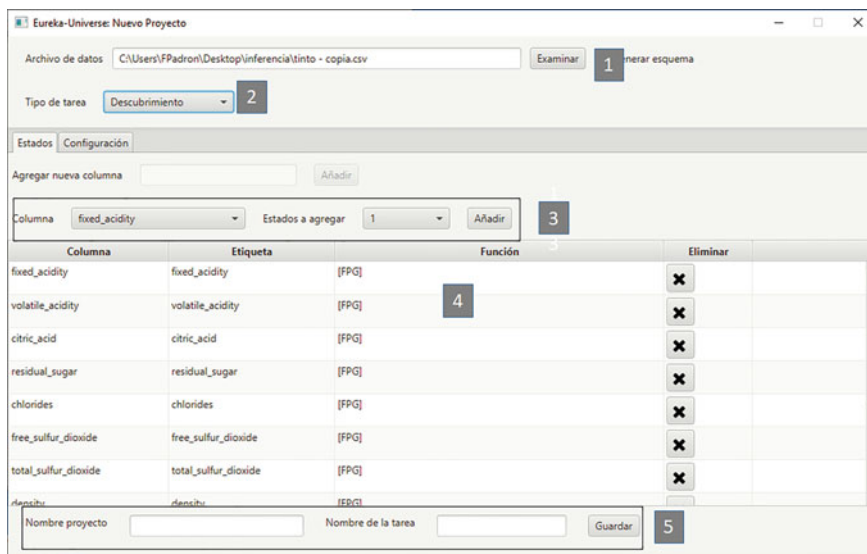


Fig. 4 Module New project

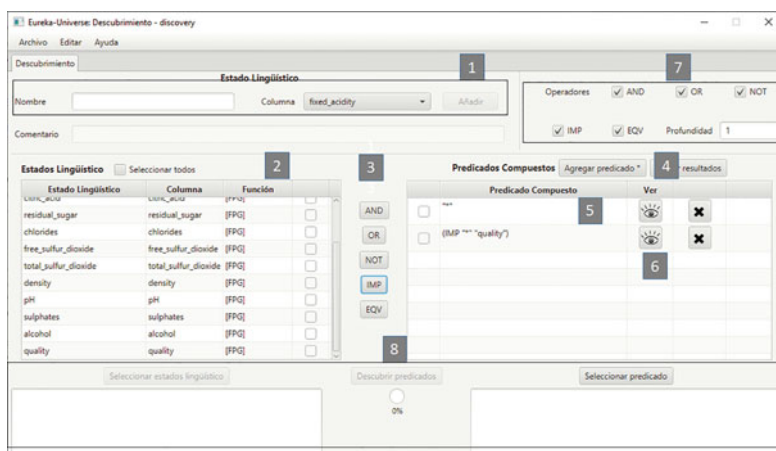


Fig. 5 Algebraic editor for discovery task

1. An area to add linguistic variables to the variables list (2).
2. The *linguistic variables list* allows these variables to be seen, edited, or added to symbolic predicates.
3. The *logic operators buttons* to construct the symbolic predicates
4. The “generator” or “wildcard” button, when clicked, will add a *star predicate* to the symbolic predicates list.

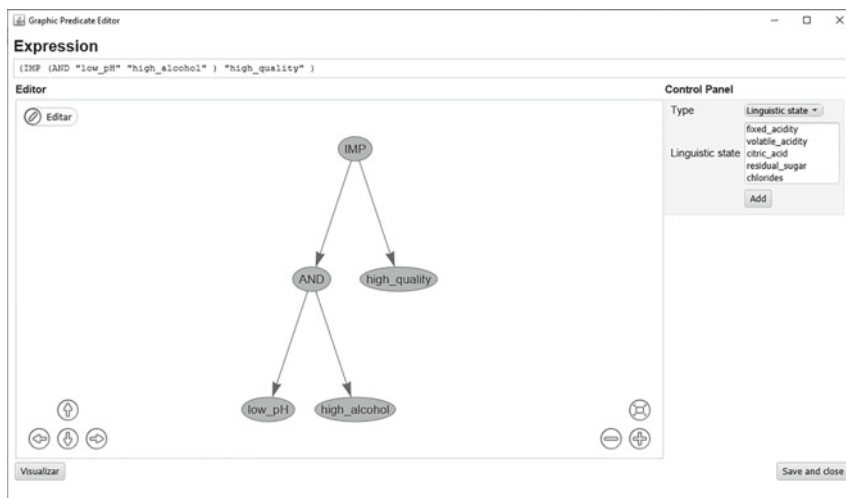


Fig. 6 The graphic editor

5. The *symbolic predicates* list.
6. The *Eye* button to call the graphic editor.
7. The *operators checkboxes* to be included in the *predicates generator*.
8. The execution section, to perform the discovery task that will take into account the selected: symbolic predicate (5), linguistic variables (2), and the logical operators (7)

Graphic editor

The graphic editor allows to see and edit a predicate as a fuzzy tree (Fig. 6), it includes:

1. The graphic area
2. A control panel to add logical operators and linguistic variables
3. A section to see the predicate as a text
4. Controls to zoom and move the graphic
5. A button to see the MF of the selected linguistic variable, if defined

4 Case of Study

The iris dataset is used as the case study, which contains the instances for 150 flowers divided into three types: Versicolor, Virginica, and Setosa.

The columns of the dataset are the attributes of the flowers as the width and length of the petal and sepal; the decision variable is the type of flower according to its attributes.

The types of flowers are replaced for a numerical identifier of a class to address this problem as one of classification.

The dataset is divided into two parts, one of which will be taken to discover predicates and the second for the prediction of the classes of each instance through discovered predicates.

The discovery task

The discovery task on the first dataset allows generating fuzzy predicates. Therefore, it is proposed to carry out three discovery tasks in which the following parameters are configured for each search algorithm.

Each discovery task includes the symbolic predicate that will differ from the others only for the class involved:

(IMP " * ""class1")

(IMP " * ""class2")

(IMP " * ""class3")

It is assigned one linguistic variable for every column in the dataset. Condition variables are assigned GMFs with no parameters defined to let the search algorithm find them:

1. *sepallenght*: GMF[]
2. *sepalwidth*: GMF[]
3. *petallenght*: GMF[]
4. *petalwidth*: GMF[]

On the other hand, the decision variables are assigned singleton MF. The singleton MF has only one parameter and works as follows:

$$f(x, a) = \begin{cases} 1 & \text{IF } x = a \\ 0 & \text{otherwise} \end{cases}$$

So the classes are assigned as follows:

1. *class1*: singleton[a = 1]
2. *class2*: singleton[a = 2]
3. *class3*: singleton[a = 3]

Depth of the generated predicates is assigned to 1.

It only generates predicates with the logical operator AND.

The type of logic selected is the CFLGMB.

Following are the results of the antecedents of generated predicates and their corresponding linguistic variables (Tables 6, 7, and 8).

Table 6 Predicate 1 and linguistic variables

Predicate	(AND “ <i>sepalwidth</i> ” “ <i>petallength</i> ”)		
Linguistic variable	γ	B	m
Sepalwidth	3.99959	2.33069	0.90953
Petallength	3.38223	1.21970	0.02927

Table 7 Predicate 2 and linguistic variables

Predicate	(AND “ <i>petallength</i> ” “ <i>petalwidth</i> ”)		
Linguistic variable	γ	B	m
Petallength	5.37249	1.47754	0.95164
Petalwidth	1.00708	0.20448	0.01224

Table 8 Predicate 3 and linguistic variables

Predicate	(AND “ <i>petallength</i> ” “ <i>sepallength</i> ”)		
Linguistic variable	γ	B	M
Petallength	5.67597	1.29068	0.93168
Sepalwidth	7.87363	4.68049	0.96811

The evaluation task

The antecedents of generated predicates were performed on the second dataset, and Table 9 shows a fragment of the computed truth values that were used to classify the instances.

In the classification, the truth values of the antecedents for every instance were compared, and the antecedent with the highest truth value assigns the class to the instance.

The accuracy of the classification on the iris dataset was 91%.

Interpretability of predicates

It is possible to interpret the predicates according to generated predicates and their linguistic states, which form the shapes shown in Figs. 7, 8, and 9.

This way, predicates can be read as follows:

1. (IMP (AND “*sepalwidth*” “*petallength*”) “*class1*”)

Short sepal width AND Short petal length IMPLIES Setosa

2. (IMP (AND “*petallength*” “*petalwidth*”) “*class2*”)

Short petal length AND Long petal width IMPLIES Versicolor

3. (IMP (AND “*petallength*” “*sepallength*”) “*class3*”)

Long petal length AND Long spal length IMPLIES Virginica

Table 9 Results on the testing dataset (fragment)

Sepal lenght	Sepal width	Petal lenght	Petal widt	Real class	Truth value 1	Truth value 2	Truth value 3	Obtained class
4.9	3	1.4	0.2	1	0.32186	0.1179	0.01876	1
4.7	3.2	1.3	0.2	1	0.40395	0.11152	0.01558	1
5	3.6	1.4	0.2	1	0.60952	0.1179	0.02009	1
5.1	3.5	1.4	0.3	1	0.55471	0.11787	0.02152	1
5.1	3.8	1.5	0.3	1	0.72076	0.12461	0.02258	1
4.4	3.2	1.3	0.2	1	0.40395	0.11152	0.01266	1
5.1	3.8	1.9	0.4	1	0.72114	0.15502	0.02736	1
4.6	3.2	1.4	0.2	1	0.40464	0.1179	0.01526	1
6.9	3.1	4.9	1.5	2	0.07937	0.16789	0.31494	3
5	2	3.5	1	2	0.06739	0.26978	0.05365	2
6.2	2.2	4.5	1.5	2	0.03931	0.14438	0.17985	3
6.3	2.5	4.9	1.5	2	0.03867	0.16789	0.22032	3
7.7	2.8	6.7	2	3	0.00888	0.06008	0.67204	3
6.3	2.7	4.9	1.8	3	0.04939	0.0736	0.22032	3
6.7	3.3	5.7	2.1	3	0.04415	0.03956	0.35056	3
7.2	3.2	6	1.8	3	0.02909	0.09726	0.48907	3
6.2	2.8	4.8	1.8	3	0.06152	0.07103	0.20003	3
6.1	2.6	5.6	1.4	3	0.02156	0.26667	0.23647	2

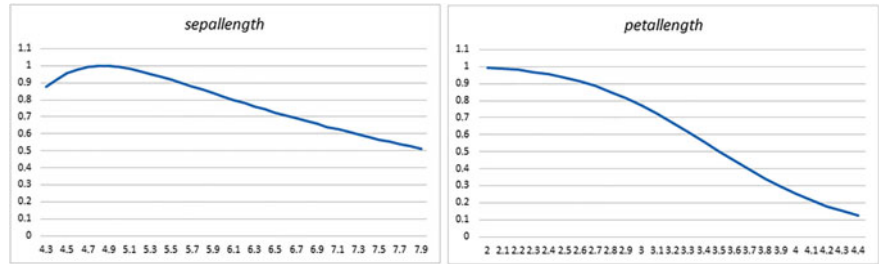


Fig. 7 Linguistic states for predicate 1

5 Conclusions

Eureka-Universe is a platform helpful in decision-making, allowing users only to be experts in their respective areas without knowledge of mathematics, algorithms, nor computer sciences.

Eureka-Universe is a Business Analytics and Business Intelligence (BI) system that allows solving independent tasks and mixing them towards decision-making solutions.

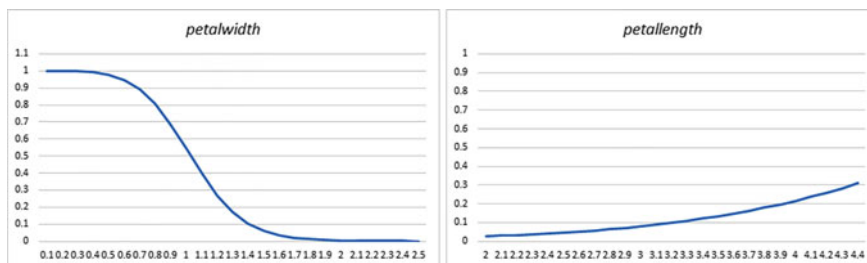


Fig. 8 Linguistic states for predicate 2

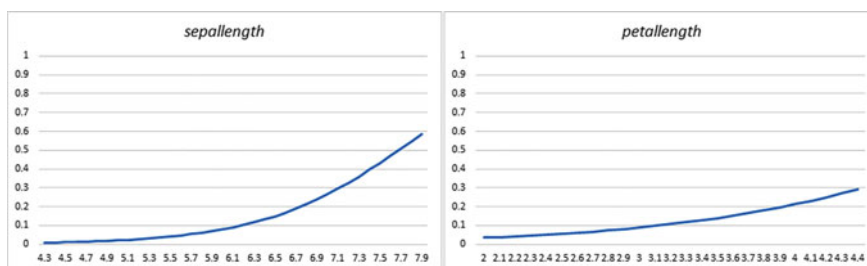


Fig. 9 Linguistic states for predicate 3

In this work, there were addressed concepts about Compensatory Fuzzy Logic, Archimedean fuzzy logic, inference, and interpretability as the background of Eureka-Universe.

It addressed a classification problem over the iris dataset; it was preprocessed to split into two parts to discover predicates and classify unseen data. The accuracy on classification was 91%.

Also, in this study case, through the Graphic editor was possible to interpret the linguistic states to translate predicates into a language close to natural.

Future work

Future works for Eureka-Universe include the automation of KDD tasks, such as the inference, the interpretability of predicates, and chain sequential tasks through workflows.

References

1. Cano, J.L.: Business Intelligence: Competir Con información, p. 319. Banesto, Fundación Cultur [ie Cultural] (2007)
2. Duan, L., Xiong, Y.: Big data analytics and business analytics. J. Manag. Analyt. **2**(1), 1–21 (2015)

3. Mamani, Y.: Business Intelligence: herramientas para la toma de decisiones en procesos de negocio. Universidad Nacional Micaela Bastidas de Apurímac (2018)
4. Holsapple, C., Lee-Post, A., Pakath, R.: A unified foundation for business analytics. *Decis. Support Syst.* **64**, 130–141 (2014)
5. Padrón-Tristán, J.F., Cruz-Reyes, L., Espin-Andrade, R.A., H.J., Castellanos-Alvarez, A., Llorente-Peralta, C.E., Arán-Pérez, J.M.: Eureka-Universe (2.8.4_1) (2020)
6. Kodratoff, Y.: Knowledge discovery in texts: a definition, and applications. In: International Symposium on Methodologies for Intelligent Systems, pp. 16–29. Springer, Berlin, Heidelberg (1999)
7. Frawley, W.J., Piatetsky-Shapiro, G., Matheus, C.J.: Knowledge discovery in databases: an overview. *AI Mag.* **13**(3), 57–57 (1992)
8. Pazzani, M.J.: Knowledge discovery from data? *IEEE Intell. Syst. Appl.* **15**(2), 10–12 (2000)
9. Padrón-Tristán, J.F., Cruz-Reyes, L., Espín-Andrade, R.A., Llorente-Peralta, C.E.: A Brief review of performance and interpretability in fuzzy inference systems. *New Perspectives on Enterprise Decision-Making Applying Artificial Intelligence Techniques*, pp. 237–266 (2021)
10. Zadeh, L.A.: Fuzzy sets. *Inf. Control* **8**(3), 338–353 (1965)
11. Pérez-Pueyo, R.: Procesado y optimización de espectros raman mediante técnicas de lógica difusa: aplicación a la identificación de materiales pictóricos. Universitat Politècnica de Catalunya, Departament de Teoria del Senyali (2005)
12. Espin-Andrade, R.A., Téllez, G.M., González, E.F., Marx-Gómez, J., Lecich, M.I.: Compensatory Logic: A fuzzy normative model for decision making. *Investigación Oper.* **27**(2), 184–193 (2013)
13. Llorente-Peralta, C.E., Cruz-Reyes, L., Espín-Andrade, R.A.: Knowledge discovery using an evolutionary algorithm and compensatory fuzzy logic. In: *Fuzzy Logic Hybrid Extensions of Neural and Optimization Algorithms: Theory and Applications*, pp. 363–383. Springer, Cham (2021)
14. Rey, M.I., Galende, M., Fuente, M.J., Sainz-Palmero, G.I.: Multi-objective based fuzzy rule based systems (FRBSs) for trade-off improvement in accuracy and interpretability: a rule relevance point of view. *Knowl. Based Syst.* **127**, 67–84 (2017)
15. Cordovés, T.C., Suárez, A.R., Andrade, R.A.E.: Knowledge discovery by fuzzy predicates. In: *Soft Computing for Business Intelligence*, pp. 187–196. Springer, Berlin, Heidelberg (2014)
16. Espin-Andrade, R.A., González-Caballero, E., Pedrycz, W., Fernández-González, E.: Archimedean-compensatory fuzzy logic systems. *Int. J. Comput. Intell. Syst.* **8**(sup2), 54–62 (2015). <https://doi.org/10.1080/18756891.2015.1129591>
17. Espin-Andrade, R.A., Gonzalez, E., Pedrycz, W., Fernandez, E.: An interpretable logical theory: the case of compensatory fuzzy logic. *Int. J. Computat. Intell. Syst.* **9**(4), 612–626 (2016)
18. Espin-Andrade, R.A., González-Caballero, E., Pedrycz, W., Fernández, G.E.: Archimedean-compensatory fuzzy logic systems. *Int. J. Comput. Intell. Syst.* **8**(sup2), 54–62 (2015). <https://doi.org/10.1080/18756891.2015.1129591>
19. [A20] Garson, J.: Modal Logic, The Stanford Encyclopedia of Philosophy (Fall 2018 Edition), Edward N. Zalta (ed.) (2009). <https://plato.stanford.edu/archives/fall2018/entries/logic-modal/>.
20. Mathew, T.V.: Genetic algorithm. Report submitted at IIT Bombay (2012)
21. Wang, S.C.: Genetic algorithm. In: *Interdisciplinary Computing in Java Programming*, pp. 101–116. Springer, Boston, MA (2003)
22. Mirjalili, S.: Genetic algorithm. In: *Evolutionary Algorithms and Neural Networks*, pp. 43–55. Springer, Cham (2019)
23. González-Ramírez, C.M.: Aproximación al concepto de inferencia desde dos modelos de comprensión: modelo estratégico y modelo de construcción e integración. *Literatura y lingüística* **35**, 295–312 (2017)
24. Bunge, M.: La investigación científica: su estrategia y su filosofía. Siglo XXI (2002)
25. Galende, M., Sainz, G.I., Fuente, M.J.: Accuracy-interpretability trade-off for precise fuzzy modeling using simple indices. Application to Industrial plants. *IFAC Proc.* **44**(1), 12656–12661 (2011)

26. Razak, T.R., Garibaldi, J.M., Wagner, C., Pourabdollah, A., Soria, D.: Interpretability and complexity of design in the creation of fuzzy logic systems—a user study. In: 2018 IEEE Symposium Series on Computational Intelligence (SSCI), pp. 420–426. IEEE (2018)
27. Alonso, J.M., Castiello, C., Mencar, C.: Interpretability of fuzzy systems: current research trends and prospects. In Springer handbook of computational intelligence, pp. 219–237. Springer, Berlin, Heidelberg (2015)

Neural Networks and Learning

Extension of Windowing as a Learning Technique in Artificial Noisy Domains



David Martínez-Galicia, Alejandro Guerra-Hernández, Xavier Limón, Nicandro Cruz-Ramírez, and Francisco Grimaldo

Abstract Windowing is a guided sub-sampling method conceived to reduce the number of training examples when inducing decision trees, while preserving accuracy. However, its use has been discouraged when dealing with noisy datasets because of a lack of observed benefits. This work proposes modeling such domains probabilistically to study the performance of Windowing under controlled conditions, i.e., on artificial discrete datasets with binary class affected by different levels of noise. An extension of Windowing that removes all inconsistency in the datasets as a preprocessing step, Windowing Inconsistency Filtering (WIF), is proposed to cope with noise in class values. J48, as implemented in Weka, is adopted as a tree-inductive algorithm. WIF, Windowing, and J48 are then compared on a repeated 10-folds cross-validation process over noisy and clean test sets. Results show that the three algorithms obtain similar predictive performance, as expected, but WIF keeps reducing the number of used training examples despite the presence of noise. Future work includes considering different inductive algorithms, e.g., Naïve Bayes, to establish if the adopted extension generalizes beyond decision trees.

Keywords Windowing · Noise filtering · Sub-sampling · Inconsistency

D. Martínez-Galicia (✉) · A. Guerra-Hernández · N. Cruz-Ramírez
Instituto de Investigaciones en Inteligencia Artificial, Universidad Veracruzana, Xalapa, México
e-mail: davidgalicia@outlook.es

A. Guerra-Hernández
e-mail: aguerra@uv.mx

N. Cruz-Ramírez
e-mail: ncruz@uv.mx

X. Limón
Facultad de Estadística e Informática, Universidad Veracruzana, Xalapa, México
e-mail: hlimon@uv.mx

F. Grimaldo
Departament d'Informàtica, Universitat de València, València, España
e-mail: francisco.grimaldo@uv.es

1 Introduction

Supervised learning algorithms are used in the Knowledge Discovery in Databases (KDD) process for constructing models that predict the value of a class variable, exploiting possible relationships among the features describing data [3]. The performance of these algorithms depends on the size and the quality of the training data. Several problems can arise during this process, e.g., models with a low specificity because of an imbalanced class distribution; excessive computational costs due to a huge number of examples, and unreliable training data caused by value corruptions and missing values.

Windowing is a guided sub-sampling method that aims to induce suitable decision trees selecting a small portion of the available training data [14, 15]. Algorithm 1 describes it: The process starts with a small random sample of the training data. This sample is known as the Window, and its size is a parameter that is usually set between 5 and 10%. Then, a model is induced with the Window, and it is used to classify the remaining examples, i.e., those that were not selected as part of the Window. The misclassified examples are moved to the Window to start the induction again. This iterative step tries to gather all the informative examples for learning while discarding the redundant covered ones. When a stop condition is reached the loop is broken, and the method returns the Window and the final model. In the original version of Windowing, the stop condition is to find no more misclassified examples, but this can be relaxed, e.g., establishing an acceptable percentage or number of misclassified examples.

Algorithm 1 Windowing

```

Input: examples, size
1 window  $\leftarrow$  sample(examples, size)
2 examples  $\leftarrow$  examples - window
3 repeat
4   stopCondition  $\leftarrow$  True
5   model  $\leftarrow$  induce(window)
6   for(example in examples) do
7     if((classify(model, example)  $\neq$  |class(example))
8       window  $\leftarrow$  window  $\cup$  {example}
9       examples  $\leftarrow$  examples - {example}
10    stopCondition  $\leftarrow$  False
11  end if
12  end for
13 until stopCondition
14 return window, model

```

This work is encouraged by the advantages of Windowing reported in previous research. The most evident is that it preserves accuracy using up to 3% of the available training examples [8, 9, 11]. In its beginning, Windowing was used to induce decision trees and production rules [5, 14, 15], but it has been shown that it exhibits the same behavior when adopting different inductive algorithms, e.g., Bayesian Classifiers, Artificial Neural Networks, and Support Vector Machines [11]. Furthermore, the percentage of used training examples has a strong correlation with the model accuracy. The more accurate is the obtained model, the less data is used to compute it. This suggests that Windowing demands more data when trying to induce accurate models on difficult domains [10].

This observation is particularly true when facing noisy domains, where the model accuracy decreases, and the Window size grows. Wirth et al. [19] identified this behavior, motivating the detailed analysis of noisy domains. Cattlet [2] studied the Windowing performance adopting as noise definition random class corruptions. His results suggested that Windowing has robust performance in terms of accuracy, but its learning time increases. Some strategies to deal with this problem include a Windowing variant that ensembles production rules from a noise-robust inductor as FOIL [5] and the proposal of using Windowing in a distributed settings where the benefits of its heuristic selecting examples outperform the costs of learning [8, 9].

The lack of a clear and precise definition of noise obscures the study of Windowing in such domains. Previous studies generate noisy datasets adopting random class corruptions, assuming that datasets from well-known repositories do not contain noise [2, 21]. Historically, noise modeling has roots in the study of missing values [16] and its taxonomy [17]. However, Frénay et al. propose a modeling focused on class noise [4]. We adopt an extended version of the modeling of Frénay that allows the generation of artificial datasets under noise-controlled conditions. The modeling consists of a Bayesian Networks (BN) encoding a discrete domain and two variables defining the noise distribution. Although the generated datasets adopt a definition of noise similar to the work of Catlett [2], these variables control which examples are noisy and the observed value of the class. This information enables the measurement of noise added to the Window.

Windowing Inconsistency Filtering (WIF) is a Windowing extension to cope with class noise, implementing a preprocessing stage based on the concept of inconsistency. Inconsistency is a set of examples that have the same attribute values but different class values. WIF solves them by deleting the examples belonging to the less observed class. This method is compared with the original version of Windowing and the traditional approach for induction using the full training set with J48. Comparisons are performed in terms of accuracy and Area Under the ROC Curve (AUC); and in terms of the size and quality of the Window, i.e., the percentages of used training examples, the selected noisy examples, and discarded clean examples.

The paper is organized as follows: Sect. 2 describes the proposed noise modeling, Sect. 3 studies the behavior of Windowing in a case study, Sect. 4 presents the obtained results; and Sect. 5 discusses conclusions and future work.

2 Noise Modeling

Hickey [6] describes noise as errors or corruptions which obscure the relationship between the attributes and the class, identifying three main types of noise:

1. Insufficiency of the description domain, where there is a need to consider more attributes to fully describe the class distribution.
2. Corruption of attribute values, where there are errors or missing values in one or more attributes of an example.
3. Erroneous classification of examples, where corruptions cause class values to change and do not represent the original distribution.

The first type of noise is hard to solve, requiring a domain expert to diagnose it and to guide the feature extraction process. Instead, the latter types are relatively more manageable because errors can be described using the available variables. This does not imply that background knowledge is not required, however, it is easier to determine how noise distributes using available data than to find the possible external factors that cause it.

Nettleton et al. [12] propose a different taxonomy of noise covering three factors: the component it affects, its distribution, and its magnitude, being the distribution the most helpful factor for describing noise. This approach provides not only a basic description of corruption locations but also explains how noise performs. The distribution specifies which examples are noisy and the probability of changing to another class value given a variable set.

The tradition of modeling noise using distributions can be traced to the work of Rubin [16] describing the distribution of missing values as the conditional probabilities that corresponds to the process causing them. Schafer et al. [17] present their own modeling of missing values based on two concepts: pattern of missingness and missing mechanism. While the pattern of missingness represents the configuration of the variables with missing values, the missing mechanism specifies the behavior of the data using a joint probability distribution. In this modeling, the definition of pattern of missingness takes a main roll since it determines the structure of the missing mechanism.

For instance, if the pattern of missingness is univariate, i.e., only a variable is affected by missing values, the joint distribution of the mechanism can be extended by a binary variable *Error*. This variable determines in which conditions the missing values arise. Frénay et al. [4] also propose a modeling based on the work of Schafer et al., although their approach describes class noise using a univariate pattern. Their mechanism adds the variable *ObservedClass* to explain how class values are affected by noise, i.e., the probabilities of changing from one value to other.

We extend the noise modeling of Frénay et al. in two directions. First, we consider all the possible interactions between the domain variables and the noise variables, defining four types of models. Second, we propose using BNs to represent discrete domains. The adoption of BNs enables the description of well-known phenomena obtained from two sources:

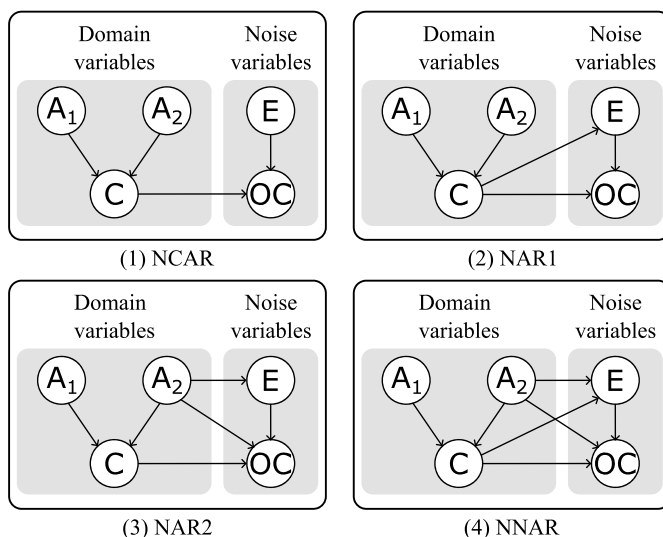


Fig. 1 Proposed class noise models: Noisy Completely at Random (NCAR), Noisy at Random 1 (NAR1), Noisy at Random 2 (NAR2), and Noisy Not at Random (NNAR) models. Notation: A_1 for Attribute 1, A_2 for Attribute 2, C for True Class, E for Error, and OC for Observed Class

1. Search algorithms: The structure and parameters of BNs can be inferred from a set of observations using algorithms, e.g., hill-climbing, genetic search and PSO.
2. Human expertise: If there are not enough observations, domain expert can suggest the structure and parameters from his experience on the domain.

Figure 1 shows the models instantiated with a simple domain of three variables:

1. Noisy Completely at Random (NCAR) Model: In this model, there are no statistical relationships that affects the variable *Error*. The noise produced by this model could be considered the worst since it does not follow any pattern in the way it affects examples.
2. Noisy at Random 1 (NAR1) Model: It represents a specialization of the NCAR model where the variable *TrueClass*(C) has a statistical relationship with the variable *Error*. This relationship allows that each class value could have a different probability of noise.
3. Noisy at Random 2 (NAR2) Model: It describes a distribution based on a non-empty set of attributes with statistical relationships with the variable *Error*. This model enables the definition of specific probabilities for each possible instance of the attribute set.
4. Noisy Not at Random (NNAR) Model: The NNAR model is the most complex. It assumes that the probability of noise changes depending on specific scenarios when a set of attributes and *TrueClass*(C) take fixed values. All the variables in the attribute set and the *TrueClass*(C) have statistical relationships with *Error*.

Table 1 Parameters setting of the genetic search

Parameter	Value	Parameter	Value
Descendant population	200	Cross over	True
Population size	20	Mutation	True
Runs	10	Tournament selection	False
Seed	1	Markov blanket classifier	False

3 Case Study

The probabilistic model of noise NCAR detailed in Sect. 2 is adopted here to synthesize a well-known domain and to generate artificial datasets under noise-controlled conditions. The adoption of the NCAR model enables the comparison with Catlett’s work and the analysis of sampling performance of Windowing. This attempt represents an early stage of describing how Windowing performs on noisy domains.

The Qualitative Bankruptcy¹ dataset is the adopted case study. This dataset illustrates with 250 examples the risk of bankruptcy in companies based on six attributes established by one of the largest Korean commercial banks [7]. All the attributes can take three different values: Negative (N), Average (A), and Positive (P), while the class variable can take only two values: Bankrupt (B) and Non-Bankrupt (NB). The adoption of QB is motivated by its low complexity in terms of the number of variables and results that suggest different classifiers get high levels of accuracy,² i.e., the kind of domain where Windowing performs well.

3.1 Data Generation

Since the noise model needs a probabilistic definition, the domain is encoded as a BN induced from the dataset, using BayesNet from Weka [20]. The parameters setting for this algorithm is a simple estimator with $\alpha = 0$ and a genetic search with MDL as aptitude. Table 1 describes the configuration of the genetic search.

After obtaining the structure and the parameters of the BN, the NCAR model is completed by adding the noise variables, specifying no statistical relationships affecting the variable *Error* and determining the probability of noise.

Figure 2 shows an instantiated NCAR model with the probability of 10% for the variable $Error = T$, which suggests the datasets generated from this model will contain a similar percentage of noisy examples. Five models with different distributions in the variable *Error* between 0 and 40% with increments of 10% are

¹ <https://www.openml.org/d/1495>.

² <https://www.openml.org/t/9958>.

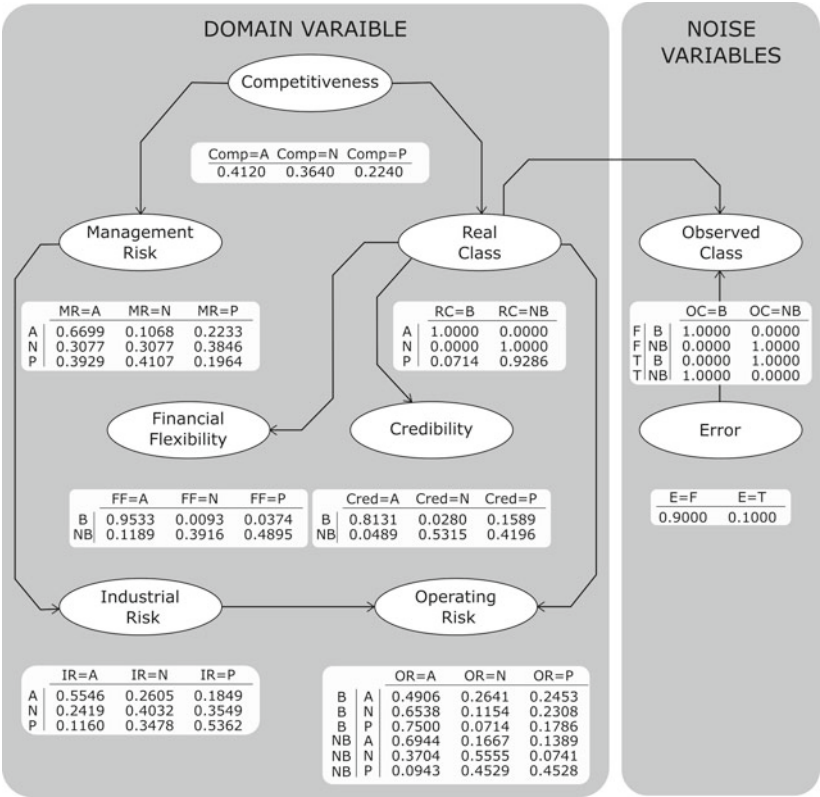


Fig. 2 NCAR model to generate datasets with 10% of class noise. The conditional probability table of *Error* defines the noise level in the generated datasets

considered. For each model, six datasets are generated with 250, 500, 1000, 2000, 5000, and 10,000 examples. Tetrad [18], a software that creates causal and statistical models, is adopted as a tool for generating these datasets.

3.2 Inductive Algorithms

J48, the decision trees inductor of Weka [20], is the base learning algorithm adopted for this case study. Three inductive methods are considered: The traditional induction using J48 on the whole training set; Windowing as defined in Sect. 1, and Windowing Inconsistency Filtering (WIF, Algorithm 2).

The preprocessing stage of WIF groups the examples with the same attribute values to find inconsistency sets, i.e., sets of examples with more than 2 class values. When such sets are detected, WIF keeps the examples of the class value

with more observations. This heuristic is founded on the hypothesis that the low probability of class noise produces corrupted instances belong to the less observed class. However, its use is limited on discrete domains since the notion of inconsistency is not compatible with continuous variables due to their large number of values.

Algorithm 2 Windowing Inconsistency Filtering

Input: examples, size

```

1  groups ← group_by(examples, attributes(examples))
2  for (group in groups) do
3    classValues ← set(group[class])
4    commonClass ← mode(group[class])
5    if (|classValues| > 1)
6      group ← group[class == commonClass]
7    end if
8  end for
9  examples ← ungroup(groups)
10 return windowing(examples, size) // call to Algorithm 1

```

3.3 Methodology

The experiments run the selected methods on the thirty artificial datasets generated as explained in Sect. 3.1. Respecting the configuration of J48, the default parameters setting in Weka is adopted, excepting that we enable the Laplace Smoothing correction. Provost [13] suggests this correction to improve the probability estimation on tree leaves when the sampling method modifies the class distribution, as Windowing does [10]. Even though most of the parameters remain fixed during the experimentation. The parameter for using the J48's post-pruning method is modified to assess its impact.

Concerning the setting of the Windowing methods, a stratified random sampling performs to obtain the initial windows with 20% of the training set. A 10-folds cross-validation process repeated ten times evaluates all the approaches. Three factors are considered in this work to assess the performance: the models' predictive performance, the data usage, and the noise filtered.

The predictive performance of the obtained models is measured using two metrics: accuracy and Area Under the ROC Curve (AUC). The decision trees perform over clean and noisy test sets. In the first scenario, an instantiated NCAR model generates

the clean test sets. However, this situation represents an ideal case. In the second scenario, a cross-validation process generates the noisy test sets. For datasets with noise levels higher than 10%, test sets contain similar noise distribution since the validation split is a random stratified sample.

Considering the data usage, the percentage of examples used for the induction reveals the reductions performed by the methods. Values closer to 90% imply that all the training set is used, as the traditional approach does in a tenfold cross-validation process.

Three metrics are adopted to measure the inconsistency filtering efficiency in the final Window: The percentage of inconsistency, the proportion of clean examples removed erroneously (ER1) [1], and the proportion of noise not removed (ER2) [1]. ER2 is adopted to measure the percentage of noise that remains in the samples to compare the performance of Windowing and WIF.

4 Results

Although not shown because of the available space, accuracy and AUC results show that the decisions trees of all the methods perform similarly over noisy test sets: Noise causes a linear performance degradation. The performance ranges 90% in test sets with 10% of noise, decreasing down to 60% in sets with 40% of noise.

Regarding the predictive performance over clean test sets, Fig. 3 shows the results in terms of AUC. While the columns represent the size of the training dataset, the rows represent the noise level. Higher numbers of available training examples tend to improve the performance in noisy domains, probably because decision trees are robust enough to deal with noise if there is enough data for the induction. Indeed, the worst results are with noise levels higher than 30% and less than 2,000 examples. However, the post-pruning method can improve the performance in these conditions.

Although the results in terms of accuracy seem similar to those of AUC, models get slightly lower levels of accuracy. This observation could probably be due to the predictive accuracy achieved in each class and an imbalanced class distribution. However, it must be validated with experimentation.

It is well-known that Windowing achieves drastic reductions of the training data [15], but this behavior has not been corroborated in noisy domains up to now. Figure 4 shows that the percentage of used training examples of the original version of Windowing is near 20% with no noise. However, this behavior tends to decrease with higher noise levels. When there is 40% of noise, the reductions are minimal. The reductions performed by the proposed variant are considerably better. When Windowing uses 88% of the training data, WIF uses 43% for the dataset of 10,000 examples and 40% of noise.

Table 2 shows the number of inconsistent examples on each dataset. The proportion of inconsistency varies depending on two factors: the size of the dataset and the noise levels. Despite results in noisy datasets, data with 0% noise have a low portion of inconsistencies, this is probably explained by the probability distribution

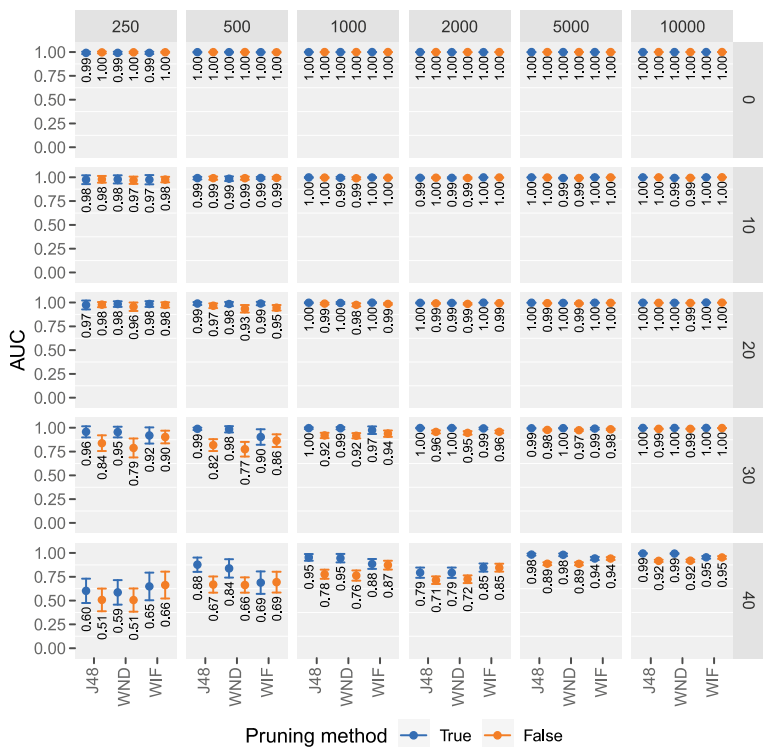


Fig. 3 AUC results for J48, Windowing (WND), and Windowing Inconsistency Filtering (WIF). Columns vary the number of training examples and rows the percentage of noise

of *TrueClass* when the attribute *Competitiveness* takes the value of N . However, the occurrence of inconsistencies is no likely to happen, hence it can be obviated.

Table 3 presents the number of examples after the preprocessing. Results suggest that the higher level of noise, the more data is deleted. Indeed, the proportion of deleted examples is near to the noise percentage.

Percentages of ER1 (clean examples removed) and ER2 (noise not removed) closer to zero suggest that most of the noise is filtered without deleting non-corrupted data. Table 4 describes the results of these metrics. Even though this preprocessing removes just inconsistencies, it deletes up to 95% of noise when there is a low percentage of noise and a high amount of data. Furthermore, the reductions achieved in the dataset with 40% of noise and 10,000 examples are not negligible.

The preprocessing stage gets results near desirables levels of ER1 and ER2. However, some specific scenarios affect its performance, e.g., an inconsistency set with a perfectly balanced class distribution. This problem could be solved deleting the whole inconsistency.

Figure 5 presents the comparison of the methods using the ER2 metric. This figure corroborates that Windowing aggregates all the noise in the Window since it does not

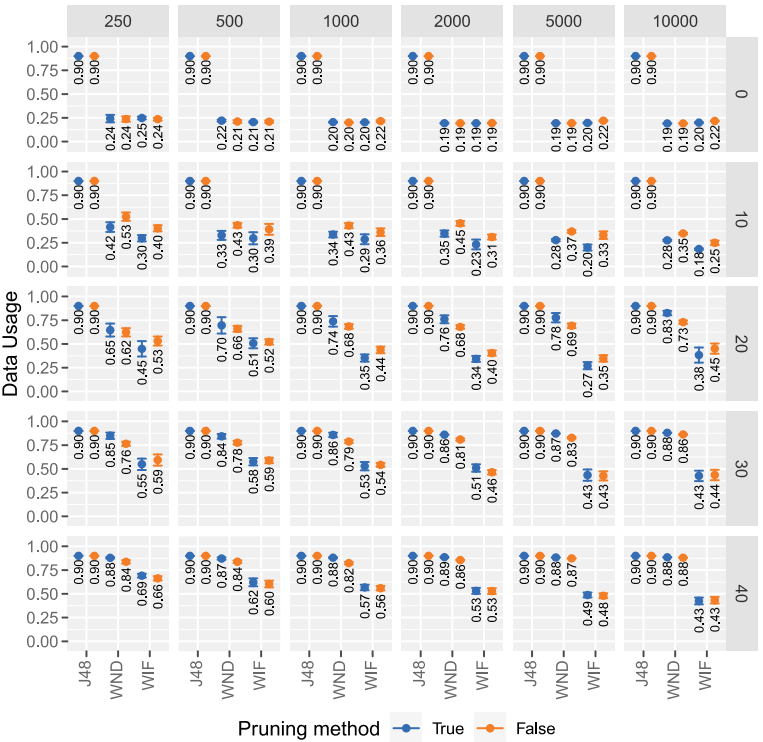


Fig. 4 Percentage of used examples for J48, Windowing (WND), and Windowing Inconsistency Filtering (WIF). Columns vary the number of training examples and rows the percentage of noise

Table 2 Inconsistent examples in the datasets

	Noise level	0%	10%	20%	30%	40%
Examples	250	2	69	89	111	130
	500	0	158	265	289	304
	1,000	0	478	641	729	738
	2,000	7	1,225	1,581	1,673	1,752
	5,000	76	3,951	4,475	4,660	4,738
	10,000	182	8,775	9,402	9,704	9,762

adopt a strategy to filter it and then noisy and counter examples are indistinguishable. Though, WIF achieves to reduce the noise from 40% up to 95%. As mentioned earlier, the data size is a determining factor for the success of WIF.

Table 3 Examples after the inconsistency filtering algorithm

	Noise level	0%	10%	20%	30%	40%
Examples	250	249	231	221	211	202
	500	500	469	418	398	388
	1,000	1000	921	835	780	725
	2,000	1997	1816	1621	1474	1355
	5,000	4983	4518	4084	3543	3226
	10,000	9962	9005	8079	6958	6221

Table 4 Results of ER1 and ER2 after the inconsistency deletion algorithm

Noise level		10%		20%		30%		40%	
Examples	Metric	ER1	ER2	ER1	ER2	ER1	ER2	ER1	ER2
	250	1.85	55.88	5.03	62.75	8.98	71.08	11.68	68.75
	500	2.44	60.78	7.97	54.05	9.28	57.23	14.56	69.69
	1,000	1.75	28.40	3.75	32.84	8.29	46.00	14.23	52.95
	2,000	0.99	15.30	2.97	20.86	5.24	23.68	15.13	42.90
	5,000	0.46	11.58	0.91	9.89	4.00	17.03	10.95	29.08
	10,000	0.44	4.51	0.93	6.58	1.91	9.96	6.71	16.93

5 Conclusions

Previous works have studied the behavior of Windowing in noisy domains focusing on its predictive performance, the complexity of the learned models, and its learning time. However, their conclusions discouraged the use of Windowing since it is not able to obtain more accurate models than the traditional induction does using all the training data. The lack of a clear definition of noise makes it difficult to understand how Windowing is affected by noisy examples. This paper proposes using a probabilistic model that describes the performing of class noise in well-known domains and generates artificial datasets with noise-controlled conditions. The generated datasets provide information about which examples are noisy and enable the description of the Window composition. Regarding the sampling, results suggest that the big data reductions performed by Windowing tends to decrease in noisy domains. Indeed, there are no significant reductions in datasets with 30% of noise or more. This behavior implies that all the noise is incorporated into the Window.

This paper also proposes a Windowing extension with a preprocessing stage to delete inconsistency. The preprocessing algorithm, inconsistency filtering, is founded on a simple heuristic: If there is class noise in low proportions, the corrupted examples should belong to the minority class in an inconsistency. Results support that the proposed extension can reduce up to 55% of the data usage and 75% noise compared to its original formulation in datasets with 40% noise.

This work suggests future lines of research on Windowing, including:

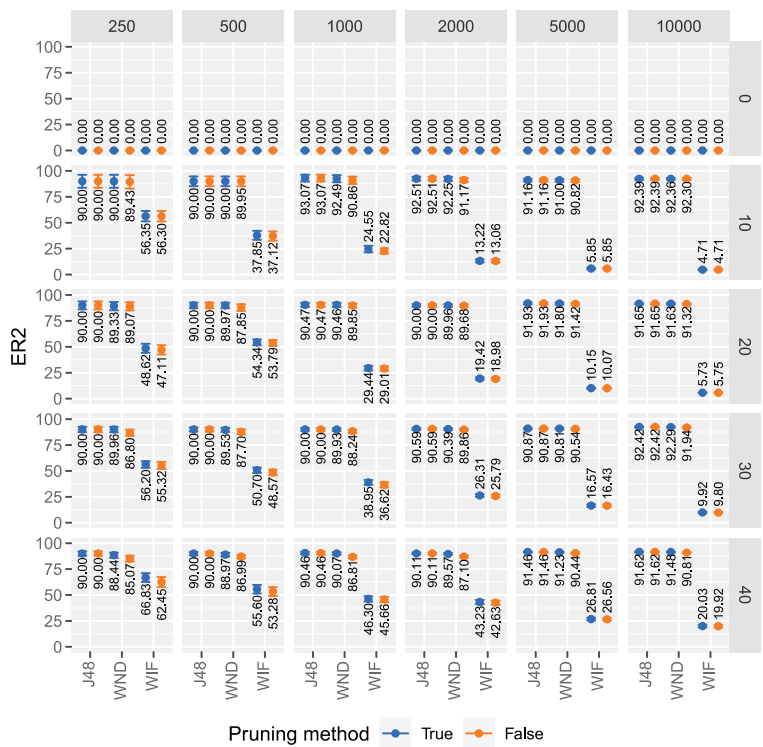


Fig. 5 ER2 results for J48, Windowing (WND), and Windowing Inconsistency Filtering (WIF). Columns vary the number of training examples and rows the percentage of noise

1. Studying the generalization of the inconsistency deletion algorithm. Results show that the J48 algorithm is robust to noise when the post-pruning method is enabled. However, when this method is disabled, WIF gets better results in domains with 30% of noise or more. It is necessary to determine if the preprocessing stage is helpful when other classifiers are adopted, particularly those that do not employ a method to deal with noise, e.g., naïve Bayes.
2. Analyzing the Windowing behavior in other class noise models. This paper proposes three types of class noise that are not usually studied, i.e., NAR1, NAR2 and NNAR. These models represent more realistic scenarios of noise that can lead to a better understanding of how Windowing methods deal with noise.
3. Generating artificial continuous domains for the evaluation of learning techniques. The suggested model works with discrete BN, restricting the data generation to nominal datasets. Though, real-world data does not only contain discrete variables. The creation of mixed datasets opens a new research line focused on proposing new filtering noise techniques.

Acknowledgements The first author was funded by a scholarship from Consejo Nacional de Ciencia y Tecnología (CONACYT), Mexico, CVU: 895160.

References

1. Brodley, C.E., Friedl, M.A.: Identifying mislabeled training data. *JAIR* **11**, 131–167 (1999). <https://doi.org/10.1613/jair.606>
2. Catlett, J.: Mega induction: a test flight. In: *Machine Learning Proceedings 1991*. Elsevier, pp. 596–599 (1991). <https://doi.org/10.1016/b978-1-55860-200-7.50121-5>
3. Fayyad, U., Piatetsky-Shapiro, G., Smyth, P.: From data mining to knowledge discovery in databases. *AI Mag.* **17**, 3 (1996). <https://doi.org/10.1609/aimag.v17i3.1230>
4. Frenay, B., Verleysen, M.: Classification in the presence of label noise: a survey. *IEEE Trans. Neural Netw. Learn. Syst.* **25**, 845–869 (2014). <https://doi.org/10.1109/tnnls.2013.2292894>
5. Fürnkranz, J.: Noise-tolerant windowing. *IJCAI* (1998)
6. Hickey, R.J.: Noise modelling and evaluating learning from examples. *Artif. Intell.* **82**, 157–179 (1996). [https://doi.org/10.1016/0004-3702\(94\)00094-8](https://doi.org/10.1016/0004-3702(94)00094-8)
7. Kim, M.-J., Han, I.: The discovery of experts' decision rules from qualitative bankruptcy data using genetic algorithms. *Expert Syst. Appl.* **25**, 637–646 (2003). [https://doi.org/10.1016/s0957-4174\(03\)00102-7](https://doi.org/10.1016/s0957-4174(03)00102-7)
8. Limón, X., Guerra-Hernández, A., Cruz-Ramírez, N., Acosta-Mesa, H.-G., Grimaldo, F.: A windowing strategy for distributed data mining optimized through GPUs. *Pattern Recogn. Lett.* **93**, 23–30 (2017). <https://doi.org/10.1016/j.patrec.2016.11.00>
9. Limón, X., Guerra-Hernández, A., Cruz-Ramírez, N., Grimaldo, F.: Modeling and implementing distributed data mining strategies in JaCa-DDM. *Knowl. Inf. Syst.* **60**, 99–143 (2018). <https://doi.org/10.1007/s10115-018-1222-x>
10. Martínez-Galicia, D., Guerra-Hernández, A., Cruz-Ramírez, N., Limón, X., Grimaldo, F.: Windowing as a sub-sampling method for distributed data mining. *Math. Comput. Appl.* **25**, 39 (2020). <https://doi.org/10.3390/mca25030039>
11. Martínez-Galicia, D., Guerra-Hernández, A., Cruz-Ramírez, N., Limón, X., Grimaldo, F.: Towards windowing as a sub-sampling method for distributed data mining. *Res. Comput. Sci.* **149**, 3 (2020)
12. Nettleton, D.F., Orriols-Puig, A., Fornells, A.: A study of the effect of different types of noise on the precision of supervised learning techniques. *Artif. Intell. Rev.* **33**, 275–306 (2010). <https://doi.org/10.1007/s10462-010-9156-z>
13. Provost, F.: Tree induction for probability-based ranking. *Mach. Learn.* **52**, 199–215 (2003). <https://doi.org/10.1023/a:1024099825458>
14. Quinlan, J.: *Induction over large data bases*. Stanford University (1979)
15. Quinlan, J.: Learning efficient classification procedures and their application to chess end games. *Mach. Learn.* **1**, 463–482 (1983). https://doi.org/10.1007/978-3-662-12405-5_15
16. Rubin, D.B.: Inference and missing data. *Biometrika* **63**, 581–592 (1976). <https://doi.org/10.1093/biomet/63.3.581>
17. Schafer, J.L., Graham, J.W.: Missing data: our view of the state of the art. *Psychol. Methods* **7**, 147–177 (2002). <https://doi.org/10.1037/1082-989x.7.2.147>
18. Scheines, R., Spirtes, P., Glymour, C., Meek, C., Richardson, T.: The TETRAD project: constraint based aids to causal model specification. *Multivar. Behav. Res.* **33**, 65–117 (1998). https://doi.org/10.1207/s15327906mbr3301_3
19. Wirth, J., Catlett, J.: Experiments on the costs and benefits of windowing in ID3. In: *Machine Learning Proceedings 1988*. Elsevier, pp. 87–99 (1988). <https://doi.org/10.1016/B978-0-934613-64-4.50015-3>

20. Witten, I.H., Frank, E., Hall, M.A., Pal, C.J.: Data mining. Morgan Kaufmann Publisher (2016)<https://doi.org/10.1016/C2015-0-02071-8>
21. Zhu, X., Wu, X.: Class noise vs. attribute noise: a quantitative study. *Artif. Intell. Rev.* **22**, 177–210 (2004). <https://doi.org/10.1007/s10462-004-0751-8>

Why Rectified Linear Activation Functions? Why Max-Pooling? A Possible Explanation



Julio C. Urenda and Vladik Kreinovich

Abstract At present, the most successful machine learning technique is deep learning, that uses rectified linear activation function (ReLU) $s(x) = \max(x, 0)$ as a non-linear data processing unit. While this selection was guided by general ideas (which were often imprecise), the selection itself was still largely empirical. This leads to a natural question: are these selections indeed the best or are there even better selections? A possible way to answer this question would be to provide a theoretical explanation of why these selections are—in some reasonable sense—the best. This paper provides a possible theoretical explanation for this empirical fact.

1 Formulation of the Problem: An Explanation is Needed

Deep learning is the most successful machine learning tool. At present, the most successful machine learning technique is deep learning; see, e.g., [2]. It is a version of neural networks where:

- in contrast to the previously used 3-layer schemes,
- many consecutive layers of neurons are used.

What makes deep learning successful? Simply increasing number of layers is not sufficient to make deep learning successful. The current success is also largely due to the appropriate selection of other features.

J. C. Urenda

Departments of Mathematical Sciences and Computer Science, University of Texas at El Paso,
500 W. University, El Paso, TX 79968, USA
e-mail: jcurenda@utep.edu

V. Kreinovich (✉)

Department of Computer Science, University of Texas at El Paso, 500 W. University,
El Paso, TX 79968, USA
e-mail: vladik@utep.edu

One of such features is the selection of the rectified linear activation function (ReLU) $s(x) = \min(x, 0)$ as a non-linear data processing unit. This selection was guided by general ideas (which were often imprecise).

At present, the selection of ReLU is mostly empirical. At present, the selection of ReLU is still largely empirical—researchers:

- tried different selections corresponding to the original imprecise ideas, and
- chose the selections that worked the best.

A natural question. The empirical nature of these selections leads to the following natural question:

- Are these selections indeed the best?
- Or are there even better selections—and we do not know about them, since they have never been tried?

What we do in this paper. In this paper, we provide a theoretical explanation of why this (and other) selections are—in some reasonable sense—the best.

2 Why Rectified Linear Neurons: Our Explanation

The main purpose of machine learning: reminder. The main purpose of machine learning is:

- based on examples $(x^{(k)}, y^{(k)})$, $k = 1, \dots, K$,
- to come up with an algorithm $f(x)$ that fits all these examples, i.e., for which $f(x^{(k)}) \approx y^{(k)}$ for all k .

Once the machine learning tool is trained, we can use it to compute $y = f(x)$ for any given x .

In many applications, decreasing computation time is the main objective. In many practical problems, we need the result y as soon as possible. An example for this need is when computing the control values y based on the current state x of a self-driving car. In such situations, it is necessary to minimize the time needed for computing $f(x)$.

How neural networks operate: a brief reminder. In a neural network, computations mean interchangingly applying:

- linear transformations, and
- nonlinear transformations $z = s(x)$.

So, it is important to select the corresponding function $s(x)$ which is the fastest to compute.

In general, which functions are the fastest to compute? In a computer, every computation is performed as a sequence of hardware supported operations. These are min, max, sum, and product. Among these functions:

- min and max are the fastest,
- sum is somewhat slower, and
- the product is slower than the sum.

Thus, the fastest possible function $s(x)$ is the one that can be computed:

- by a single hardware supported operation,
- and ideally, by the fastest of them.

So, it is reasonable to use min and max.

Which activation functions are the fastest to compute? When we compute $s(x)$:

- we can use the input x , and
- we can use constants $c, c', \dots$

For activation functions:

- Computing $\min(x, x)$ or $\max(x, x)$ does not make sense, since this is simply x .
- Similarly, computing $\min(c, c')$ or $\max(c, c')$ does not make sense—we simply get one of these constants.

So, the only operations that make sense are $\min(x, c)$ and $\max(x, c)$, for some constant c .

Which constant should we use? Out of all possible constants c , the constant $c = 0$ is the fastest to generate, since 0 is the default value of each computer-stored variable.

Conclusion of this section. So, we end up with:

- function $\max(x, 0)$ —which is exactly the rectified linear function, and
- function $\min(x, 0)$ —which is, in some reasonable sense, equivalent to ReLU.

So, we have indeed provide a simple theoretical explanation for the empirical success of rectified linear activation functions.

3 Why Max-pooling

Need for pooling. One of the main applications of neural networks is to process pictures. In a computer, a picture is represented by storing intensity values for each pixel. For color pictures, we need intensity values corresponding to three basic colors

There are millions of pixels. Processing all these millions of values would take a lot of time. To save this time, we can use the fact that for most images:

- once we know what is in a given pixel,
- we can expect approximately the same information in the neighboring pixels.

Thus, to save time:

- instead of processing each pixel one by one,
- we can combine (“pool”) values from several neighboring pixels into a single value.

Which pooling operation should we select? The whole objective of pooling is to speed up data processing. From this viewpoint, we need to select a pooling operation which is the fastest to perform. This means that we need to select a pooling operation which is performed:

- by using the smallest possible number of hardware supported computer operations, and
- these operations should be the fastest.

If we use only one hardware supported operation, we get $\min(a, b)$, $\max(a, b)$ (and $a + b$). This is exactly what works well in deep learning; see, e.g., [2].

4 Which Fuzzy Operations?

We can apply the same ideas to selecting “and”- and “or”-operations (t-norms and t-conorms) in fuzzy logic; see, e.g., [1, 3–7]. We can then conclude that:

- Among all possible “and”-operations, the fastest is $\min(a, b)$.
- Among all possible “or”-operations, the fastest is $\max(a, b)$.

So, we get yet another explanation of why $\min(a, b)$ and $\max(a, b)$ are empirically successful.

Acknowledgements This work was supported in part by the National Science Foundation grants 1623190 (A Model of Change for Preparing a New Generation for Professional Practice in Computer Science), HRD-1834620 and HRD-2034030 (CAHSI Includes).

It was also supported by the program of the development of the Scientific-Educational Mathematical Center of Volga Federal District No. 075-02-2020-1478.

The authors are thankful to all the participants of the International Seminar on Computational Intelligence ISCI’2021 (Tijuana, Mexico, August 17–19, 2021), especially to Oscar Castillo and Patricia Melin, for valuable discussions.

References

1. Belohlavek, R., Dauben, J.W., Klir, G.J.: Fuzzy Logic and Mathematics: A Historical Perspective. Oxford University Press, New York (2017)
2. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press, Cambridge, Massachusetts (2016)
3. Klir, G., Yuan, B.: Fuzzy Sets and Fuzzy Logic. Prentice Hall, Upper Saddle River, New Jersey (1995)
4. Mendel, J.M.: Uncertain Rule-Based Fuzzy Systems: Introduction and New Directions. Springer, Cham, Switzerland (2017)
5. Nguyen, H.T., Walker, C.L., Walker, E.A.: A First Course in Fuzzy Logic. Chapman and Hall/CRC, Boca Raton, Florida (2019)
6. Novák, V., Perfilieva, I., Močkoř, J.: Mathematical Principles of Fuzzy Logic. Kluwer, Boston, Dordrecht (1999)
7. Zadeh, L.A.: Fuzzy sets. *Information and Control* **8**, 338–353 (1965)

Localized Learning: A Possible Alternative to Current Deep Learning Techniques



Javier Viaña, Kelly Cohen, Anca Ralescu, Stephan Ralescu,
and Vladik Kreinovich

Abstract At present, the most efficient deep learning technique is the use of deep neural networks. However, recent empirical results show that in some situations, it is even more efficient to use “localized” learning—i.e., to divide the domain of inputs into sub-domains, learn the desired dependence separately on each sub-domain, and then “smooth” the resulting dependencies into a single algorithm. In this paper, we provide theoretical explanation for these empirical successes.

1 Formulation of the Problem

Deep learning is successful but it is not a panacea: sometimes it cannot be used. In many problems, deep learning techniques—see, e.g., [4]—provide the most efficient learning: they lead to the most accurate approximation to the real-life phenomena.

This does not mean, of course, that no other machine learning techniques are needed: in some other situations, alternative machine learning techniques are needed. For example, it is known that deep learning requires a large amount of data and a lot of computation time. So, if we do not have enough data and/or we do not have enough time to train the network, we have to use other machine learning tools.

J. Viaña · K. Cohen · A. Ralescu · S. Ralescu
University of Cincinnati, Cincinnati, Ohio 45219, USA
e-mail: vianajr@mail.uc.edu

K. Cohen
e-mail: cohenky@ucmail.uc.edu

A. Ralescu
e-mail: ralescal@ucmail.uc.edu

S. Ralescu
e-mail: ralescs@mail.uc.edu

V. Kreinovich (✉)
Department of Computer Science, University of Texas at El Paso, 500 W. University,
El Paso, TX 79968, USA
e-mail: vladik@utep.edu

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2022
O. Castillo and P. Melin (eds.), *New Perspectives on Hybrid Intelligent System Design based on Fuzzy Logic, Neural Networks and Metaheuristics*, Studies in Computational Intelligence 1050, https://doi.org/10.1007/978-3-031-08266-5_29

Even when deep learning can be used, other methods are sometimes better.

Interestingly, it turns out that in some situations, even when there is enough data and enough time to use deep learning, alternative methods still lead to more accurate results; see, e.g., [12, 13]. Specifically, these papers use the following “localized” learning idea:

- we divide the area of possible values of the input into several sub-domains,
- we learn the dependence “locally”—i.e., separately on each sub-domain, and then
- we combine the resulting dependencies by making smooth transitions between them—this can be naturally done, e.g., by using fuzzy techniques (see, e.g., [1, 5, 7, 9, 10, 14]).

Comment. Of course, it is important to make sure that the comparison between different techniques is fair. For each machine learning technique, the more parameters we use, the more accurate results we get. So, the only way to claim that one technique is more accurate is:

- either to compare variants of these two methods that use the same number of parameters,
- or, alternatively, to show one of the techniques requires fewer parameters to reach the same approximation accuracy.

The second alternative is exactly how the comparison was performed in [12, 13].

A natural question. Empirically, the results from [12, 13] are interesting, but in view of the current successes of deep learning, these results are somewhat unexpected. So, a question naturally arises:

Why are these localized techniques so much more accurate than deep learning?

What we do in this paper. To answer this question, we first ask a related question:

Why are deep learning techniques so successful?

To answer this auxiliary question, we first go even further and ask:

Why are neural networks so successful in the first place?

Once we answer these two questions and find out what are the strong points of neural networks (including deep ones), it will also become clear what are the limitations of neural networks, and why shallow localized networks can, in some important practical situations, overcome these limitations.

In line with this plan:

- in Sect. 2, we analyze why neural networks are successful in the first place,
- in Sect. 3, we focus on successes of deep learning, and
- finally, in Sect. 4, we provide a possible explanation of why localized methods are sometimes better.

Comment. Most main ideas described in Sects. 2 and 3 first appeared in [6]; ideas described in Sect. 4 are completely new.

2 Why Neural Networks: A Theoretical Explanation

Why do we need a theoretical explanation. Neural networks appeared as a way to simulate how we humans solve problems: in our brains, signals are processed by neurons. The fact that we are the result of billions of years of improving evolution makes researchers believe that many biological processes are optimal (or at least close to optimal), so simulating them makes perfect sense.

On the other hand, there is a difference between computers and us: computers are built from different materials than our brains, they operate on a different size and time scales, so an optimal solution for a computer may be different from the optimal solution for a brain. For example, birds have wings to fly, and airplanes—that, to some extent, simulate the birds—also have wings, but while for the birds, flapping the wings is the optimal flying strategy, airplane wings do not flap at all.

To design an airplane, it is not enough to copy the birds, we also need to perform some theoretical analysis. Similarly, to decide which features should be used in computing, it is desirable to provide a theoretical analysis.

The main objective of the original neural network: computation speed. Artificial neural networks appeared when the computation speed of computers was several orders of magnitude lower than now. This relatively slow speed was the main bottleneck, preventing computers from solving many practical problems. So, the question arose: how can we make computers faster?

Main idea: parallelism. If a person has a task that takes too long—e.g., cleaning several offices, then to speed it up, a natural idea is to ask for help. If several people work on the same task, this task gets performed much faster. Similarly, if it takes too long for one processor to solve a problem, a natural idea is to have many processors working in parallel:

- on the first stage, all the processors perform some tasks,
- after that, on the second stage, processors use the results of the first stage to perform additional computations, etc.

To decrease the overall computation time, we need to minimize the number of stages (also called *layers*), and to make each stage as fast as possible.

Linear versus nonlinear functions. We consider deterministic computers, where the result is uniquely determined by the inputs. In mathematical terms, this means that on each stage, what each processor computes is a function of the inputs. In these terms, to select fast stages, we need to decide which function are the fastest to compute.

Functions can be linear or nonlinear. Of course, linear functions are easier—and thus, faster—to compute. However, we cannot have processor computing only linear functions—because in this case, all the computer will compute will be compositions of linear functions, and such compositions are themselves linear. On the other hand, many real-life dependencies are nonlinear.

Thus, in addition to processors computing linear functions, we also need processors computing nonlinear functions.

Which non-linear functions are the fastest to compute? In general, the fewer variables the function has, the faster it is to compute. Thus, the fastest to compute are nonlinear functions with the smallest possible number of inputs: namely, a single input.

Resulting computation scheme. We thus arrive at a scheme at which at each stage, processors compute:

- either a linear function $y = a_0 + \sum_{i=1}^n a_i \cdot x_i$
- or a nonlinear function of one variable $y = s(x)$; such a function is known as an *activation function*.

From the viewpoint of minimizing computation time, it makes no sense to have two linear stages one after another, since the composition of two linear functions is also linear—so we can replace these two stages by a single stage. Similarly, it makes no sense to have two nonlinear stages one after another, since a composition $s_1(s_2(x))$ of two functions of one variable is also a function of one variable. Thus, linear and nonlinear stages must interleave.

It turns out that both 2-stage schemes:

- a linear (L) stage followed by a nonlinear (NL) stage, i.e., a sequence L–NL, and
- a nonlinear stage followed by a linear stage: NL–L

cannot accurately represent general continuous functions on a bounded domain—neither of them can even represent the function $f(x_1, x_2) = x_1 \cdot x_2$ with sufficient accuracy. However, both 3-stage schemes L–NL–L and NL–L–NL can approximate any function. Since NL takes longer than L, the scheme L–NL–L is clearly the fastest.

In this scheme:

- First, each processor k ($k = 1, \dots, K$) transforms the inputs x_i into their linear combination $y_k = w_{k1} \cdot x_1 + \dots + w_{kn} \cdot x_n - w_{k0}$.
- On the next stage, a nonlinear transformation is applied to each y_k , so we compute the values $z_k = s_k(y_k)$.
- Finally, in the final third stage, we compute a linear combination

$$y = W_1 \cdot z_1 + \dots + W_K \cdot z_K - W_0,$$

i.e.,

$$y = \sum_{k=1}^K W_k \cdot s_k \left(\sum_{i=1}^n w_{ki} \cdot x_i - w_{k0} \right) - W_0. \quad (1)$$

This is exactly the formula for the traditional 3-layer neural network.

Usually, in this network, all the processors (called *neurons*) use the same activation function: $s_1(x) = \dots = s_K(x) = s(x)$. Then, the formula (1) takes the following form:

$$y = \sum_{k=1}^K W_k \cdot s \left(\sum_{i=1}^n w_{ki} \cdot x_i - w_{k0} \right) - W_0. \quad (2)$$

Comment. In most cases, traditional neural networks use a so-called sigmoid activation function

$$s(x) = \frac{1}{1 + \exp(-x)}.$$

3 Need to Go Beyond Traditional Neural Networks and Deep Learning

At present, computation speed is no longer the major concern, accuracy is. While several decades ago, when neural networks were invented, computational speed was the main bottleneck, at present, computers are much faster.

The main concern now is not the speed, but how accurately we can perform the computations—how accurately we can predict weather, how accurately we can estimate the amount of oil in a given oilfield, etc.

How to increase accuracy. The more parameters we use, the better we can fit the data and thus, the more accurate the model.

From this viewpoint, the larger the number K of the neurons, the more parameters we have in the formula (2), and thus, the more accurately we can represent any given function.

Limitation. However, there is a serious limitation in this increase of number of options—caused by the fact that any perturbation of K neurons does not change the expression (2).

There are $K!$ such permutations—which, for large K , is a huge number. So, while we have many possible combinations of the coefficients w_{ki} and W_k , there are much fewer ($K!$ times fewer) different functions represented by these combinations.

How to overcome this limitation. To overcome this limitation, a natural idea is to decrease the number K of neurons in each layer. To preserve the same number of

parameters, we therefore need to place some neurons in other layers. Thus, instead of the original 3-layer configuration L–NL–L, we get a multi-layer configuration

$$L-NL-L-NL-\dots$$

This is what a deep neural network is about. In a nutshell, this multi-layer scheme is exactly what is known as a deep neural network.

There are also some other differences: e.g., deep learning mostly uses a different activation function $s(x) = \max(x, 0)$ known as *rectified linear unit* (ReLU, for short).

4 Beyond Deep Learning, Towards Localization

Let us get back to estimating accuracy. As we have mentioned, at present, the main objective in most computational problems is accuracy. In the previous section, we explained how transition to deep neural networks helps increase accuracy. But maybe there are other ways to do it?

To answer this question, let us analyze the problem of decreasing accuracy somewhat more deeply.

How to estimate accuracy with which we know the parameters: general reminder. Suppose that to “train” the system, i.e., to find the values of the corresponding parameters, we can use M measurement results, with an average accuracy ε .

Let us denote by P the overall number of parameters in the model. Each measurement result means one equation in which the parameters are unknowns. So, if we take all measurement results into account, we have M equations with P unknowns. In general, if we have fewer equations than unknowns, then we cannot uniquely determine all the unknowns, some of them may be set arbitrarily—so we do not need all P parameters. Thus, we must have $M \geq P$ —and usually, we have $M \gg P$.

If we had exactly P observations, then we could determine each parameters with accuracy proportional to ε . Since we have a duplication—i.e., we have more observations than unknowns—we can use this duplication to make the results more accurate. In general, according to statistics, if we have d measurements to determine a parameter, the accuracy increases by a factor $\sqrt{d}$; see, e.g., [11].

In our case, we have M measurements for P parameters, so, on average, we have $d = M/P$ measurements per parameter. Thus, we can determine each parameter with accuracy proportional to

$$\delta = \frac{\varepsilon}{\sqrt{M/P}} = \sqrt{\frac{P}{M}} \cdot \varepsilon.$$

What is the accuracy of the result of using this model. The value δ describes the accuracy with which we know each parameter. These accuracies affect the accu-

racy of prediction. In general, each predicted values depends on all P parameters. each parameters contributes accuracy $\sim \delta$ to the prediction result. It is reasonable to assume that these P contributions are independent. Thus, according to statistics [11], the overall effect of all these contributions is proportional to $\sqrt{P} \cdot \delta$.

How can we make predictions more accurate. To make the results more accurate, we need to decrease the number of parameters on which each predicted value depends.

If each predicted value depend only on $P' \ll P$ parameters, then the overall inaccuracy of the prediction is proportional to $\sqrt{P'} \cdot \delta$, which is much smaller than $\sqrt{P} \cdot \delta$.

In other words, we need localization. To make sure that $P' \ll P$, we need to make sure that each predicted value depends only on a few of the parameters. Thus, each predicted value $f(x_1, \dots, x_n)$ depends only a few parameters. In other words, the list of all P parameters should be divided into sublists (maybe intersecting), so that in each sub-domain of the domain of all possible inputs $x = (x_1, \dots, x_n)$ we have an expression depending only on a few parameters.

In other words, we have separate few-parametric expressions describing the desired dependence on each sub-domain—this is exactly what is usually mean by *localization*—that the value of the function in each sub-domain is determined only by parameters corresponding to this sub-domain. Such a localization is exactly what is used in [12, 13].

Comments.

- In terms of the function (1), this would mean that instead of using the same activation function $s_k(x) = s(x)$ for all the neurons, as in traditional and in deep neural networks, we use, in effect, different activation functions $s_k(x) \neq s_{k'}(x)$ each of which corresponding to a certain sub-domain of the original domain. When the activation functions are different, there is no $K!$ duplication and thus, no decrease in accuracy—even when we use a traditional (“shallow”) scheme.
- There are additional advantages in a localized approach:
 - it is faster to find the parameters: we need to solve a system with fewer unknowns $P' \ll P$, and the computations corresponding to different sub-domains can be performed in parallel, and
 - it is easier to modify the solution when the values change in some sub-domain.
- Similar arguments explain why an approximation by splines—where we have polynomial approximation on each sub-domain and then smooth them out—leads, in general, to a much better accuracy than a “global” (on the whole domain) approximation by a polynomial; see, e.g., [2, 3, 8].

Corollary: shallow or deep. According to the localization idea, each value $f(x)$ depends only on the parameters corresponding to this point x and neighboring points $x' \approx x$. In a multi-layer scheme, this means that:

- the signal produced by the last layer depends only on the values from the previous layer which are close to x ;

- these values, in turn, depend only on coefficients of the pre-previous layer which correspond to locations x'' which are close to x' , etc.

We have $x' \approx x$, $x'' \approx x'$, etc., i.e., the differences $x' - x$, $x'' - x'$, etc., are small. However:

- the difference $x'' - x$ between x'' and x is the sum of two small differences:

$$x'' - x = (x'' - x') + (x' - x);$$

- if we go back one more layer, then the difference $x''' - x$ is the sum of three small differences, etc.

Thus, the more layers we have, the less localized our system, and therefore, the more parameters we need to take into account to predict each value $f(x)$.

So, to achieve the best accuracy, we need to use the smallest possible number of layers—i.e., one non-linear layer, as in traditional neural networks. This is exactly what is used in [12, 13].

Comment. Interestingly, there is some rudimentary localization effect in deep neural networks as well.

Indeed, since the corresponding activation function $s(x) = \max(x, 0)$ is equal to 0 for half of the inputs—namely, for all negative inputs—this means that, on average, half of the value from the previous layer do not affect the next layer. So, the final value produced by the last layer is determined only by half of the neurons in the previous layer. These values, in turn, depend only on the one half of neurons in the previous layer, etc. So, at least half of the parameters are not used when estimating each value—and this fact decreases the approximation error in comparison with our generic estimates.

Acknowledgements This work was supported in part by the National Science Foundation grants: 1623190 (A Model of Change for Preparing a New Generation for Professional Practice in Computer Science).

HRD-1834620 and HRD-2034030 (CAHSI Includes).

It was also supported by the program of the development of the Scientific-Educational Mathematical Center of Volga Federal District No. 075-02-2020-1478.

The authors are thankful to all the participants of the International Seminar on Computational Intelligence ISCI'2021 (Tijuana, Mexico, August 17–19, 2021), especially to Oscar Castillo and Patricia Melin, for valuable discussions.

References

1. Belohlavek, R., Dauben, J.W., Klir, G.J.: Fuzzy Logic and Mathematics: A Historical Perspective. Oxford University Press, New York (2017)
2. de Boor, C.: A Practical Guide to Splines. Springer, New York (2001)
3. Eilers, P.H.C., Marx, B.D.: Practical Smoothing: The Joys of P-splines. Cambridge University Press, Cambridge, UK (2021)

4. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press, Cambridge, Massachusetts (2016)
5. Klir, G., Yuan, B.: Fuzzy Sets and Fuzzy Logic. Prentice Hall, Upper Saddle River, New Jersey (1995)
6. Kreinovich, V., Kosheleva, O.: Optimization under uncertainty explains empirical success of deep learning heuristics. Machine Learning and No-Free Lunch Theorems. In: Pardalos, P., Rasskazova, V., Vrahatis, M.N. (eds.) Black Box Optimization, pp. 195–220. Springer, Cham, Switzerland (2021)
7. Mendel, J.M.: Uncertain Rule-Based Fuzzy Systems: Introduction and New Directions. Springer, Cham, Switzerland (2017)
8. Micula, G., Micula, S.: Handbook of Splines. Springer, Dordrecht (2008)
9. Nguyen, H.T., Walker, C.L., Walker, E.A.: A First Course in Fuzzy Logic. Chapman and Hall/CRC, Boca Raton, Florida (2019)
10. Novák, V., Perfilieva, I., Močkoř, J.: Mathematical Principles of Fuzzy Logic. Kluwer, Boston, Dordrecht (1999)
11. Sheskin, D.J.: Handbook of Parametric and Non-Parametric Statistical Procedures. Chapman & Hall/CRC, London, UK (2011)
12. Viaña, J., Cohen, K.: Fuzzy-based, noise-resilient, explainable algorithm for regression. In: *Proceedings of the 2021 Annual Conference of the North American Fuzzy Information Processing Society NAFIPS'2021*. West Lafayette, Indiana, June 7–9, (2021)
13. Viaña, J., Raslecu, S., Cohen, K., Ralescu, A., Kreinovich, V.: Extension to multidimensional problems of a fuzzy-based explainable & noise-resilient algorithm. In: *Proceedings of the 2021 International Workshop on Constraint Programming and Decision Making CoProD'2021*, Szeged, Hungary, September 12, (2021)
14. Zadeh, L.A.: Fuzzy sets. Inform. Control **8**, 338–353 (1965)

What is a Reasonable Way to Make Predictions?



Leonardo Orea Amador and Vladik Kreinovich

Abstract Predictions are usually based on what is called laws of nature: many times, we observe the same relation between the states at different moments of time, and we conclude that the same relation will occur in the future. The more times the relation repeats, the more confident we are that the same phenomenon will be repeated again. This is how Newton's laws and other laws came into being. This is what is called inductive reasoning. However, there are other reasonable approaches. For example, assume that a person speeds and is not caught. This may be repeated two times, three times—but here, the more times this phenomenon is repeated, the more confident we become that next time, he/she will be caught. Let us call this anti-inductive reasoning. So which of the two approaches shall we use? This is an example of a question that we study in this paper.

1 Formulation of the Problem

1.1 *Making Predictions is Important*

One of the main objectives of science is to predict what will happen in the future. Another important objective is to make the future more beneficial. This objective also requires predicting how different strategies will affect the future of the world. Prediction is one of the main objectives of science. So in long run, this is one of the main objectives of all the tools that science uses—including AI tools. So, to make these tools more efficient, it is important to understand:

- how we make predictions, and
- how we *should* make predictions.

L. O. Amador · V. Kreinovich (✉)

Department of Computer Science, University of Texas at El Paso, 500 W. University, El Paso, Texas 79968, USA

e-mail: vladik@utep.edu

L. O. Amador

e-mail: lorea@miners.utep.edu

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2022

O. Castillo and P. Melin (eds.), *New Perspectives on Hybrid Intelligent System Design based on Fuzzy Logic, Neural Networks and Metaheuristics*, Studies in Computational Intelligence 1050, https://doi.org/10.1007/978-3-031-08266-5_30

1.2 *At First Glance, the Answer to These Questions is Straightforward*

Predictions are usually based on what is called *laws of nature*:

- many times, we observe the same relation between the states at different moments of time, and
- we conclude that the same relation will occur in the future.

The more times the relation repeats, the more confident we are that the phenomenon will repeat again. This is how Newton's laws and other laws came into being. This is what is called *inductive reasoning*; see, e.g., [1–3].

Comment. It is important not to confuse:

- inductive reasoning—where we make a prediction based on a finite number of observations, and
- *mathematical induction*, when we prove a statement $\forall n P(n)$ by proving that $P(0)$ is true and for every n , $P(n)$ implies $P(n + 1)$.

1.3 *Situation is Not so Simple*

However, there are other reasonable approaches. For example, assume that a person speeds and is not caught. This may be repeated two times, three times. Here:

- the more times this phenomenon is repeated,
- the more confident we become that next time, he will be caught.

This is why gamblers continue to gamble after losing. This is why entrepreneurs try again after failing several times. Let us call this *anti-inductive reasoning*; see, e.g., [3].

So which of the two approaches shall we use?

1.4 *This Should be Decided by an Experiment*

We are accustomed to the fact that everything is decided by experiments. So, a natural way to select one of these two approaches is to compare them with the experimental data.

But how do we decide, based on this data, which approach is better? For this decision:

- a scientist will use inductive reasoning, while
- another person will use anti-inductive reasoning.

What will happen? This is one of the questions that we analyze in this paper.

2 Analysis of the Problem on a Simplified Case

2.1 *Simplified Case: A Description*

For simplicity, let us fix some natural number n , and consider the following simplified versions of the two approaches.

- The first approach is that:
 - if something repeats n times (or its negation repeats n times),
 - we predict that this will happen the next time.
- The second approach is that if something happens n times, the opposite will happen the next time.

2.2 *Case Study*

Suppose that we have a phenomenon—e.g., Sun rising in the morning—that holds for $2n + 1$ moments of time.

In the first approach:

- After the first n cases, we predict that the Sun will rise again—and it does.
- We do a similar prediction for moment $n + 2$ —and again, our prediction turns out to be correct.
- For n moments in a row, predictions based on our reasoning are correct.
- So, by applying inductive reasoning to these n cases, we conclude that inductive reasoning is a valid approach.

But what if we use the second approach?

- We predict that at the moment $n + 1$, the Sun will *not* rise—but it rises!
- This repeats n times, so n times, are predictions are wrong.
- We are then applying the same anti-induction to select the approach.
- Since our approach failed n times, we conclude that next time, it will work.

2.3 *Surprising Conclusion*

So, in this case, no matter how many experiments we perform:

- the proponents of both approaches will remain convinced
- that their approach will work the next time around.

2.4 What We Discuss in this Paper

In this paper, we describe this situation in precise terms.

This is still the beginning of this research. We will present more challenges than results. However, we will formulate these important challenges in precise terms, so these challenges become:

- not just vague philosophical ideas,
- but precisely formulated mathematical questions.

3 General Case

3.1 Let us Describe the Situation in Precise Terms

We want to predict whether some property P will hold. To make this prediction, we used previous observations. Let us assume that we observed similar situations N times. For each i from 1 to N , we define s_i as follows:

- if the property P was satisfied in the i -th observation, we take $s_i = T$;
- if the property P was not satisfied in the i -th observation, we take $s_i = F$;
- if it is unknown whether P was satisfied, we take $s_i = U$.

The set of all such sequences will be denoted by $\{T, F, U\}^*$.

Prediction rule $M(s)$ means that, for each such sequence s , we predict:

- either that P will be satisfied at the next moment of time: $M(s) = T$,
- or that P will not be satisfied: $M(s) = F$,
- or we do not have enough data for predictions: $M(s) = U$.

So, a prediction rule M is a mapping $M : \{T, F, U\}^* \mapsto \{T, F, U\}$.

3.2 Prediction Rule Must be Fair

A priori, we have no reason to prefer P or its negation $\neg P$. So, we should make the same prediction whether we consider P or $\neg P$. We call this absence of a priori preference *fairness*. So, for observations of $\neg P$, we should get the same conclusion.

Let us describe fairness in precise terms.

- For each observation $s = (s_1, \dots, s_n)$ of P , the observation of $\neg P$ is $\neg s = (\neg s_1, \dots, \neg s_n)$, where $\neg u \stackrel{\text{def}}{=} u$.
- Similarly, prediction $M(s)$ for P means predicting $\neg M(s)$ for $\neg P$.

Thus, fairness means that $M(\neg s) = \neg M(s)$ for all s .

3.3 *Meta-Analysis: Using Prediction Rule to Select Prediction Rule*

Let $s = (s_1, \dots, s_N)$ be a sequence of observations.

- Based on s_1 , we form a prediction $M(s_1)$ for s_2 .
- Based on (s_1, s_2) , we form a prediction $M(s_1, s_2)$ for s_3 .
- In general, based on $(s_1, \dots, s_i)$, we form a prediction $M(s_1, \dots, s_i)$ for s_{i+1} .
- Finally, based on $(s_1, \dots, s_{N-1})$, we form a prediction $M(s_1, \dots, s_{N-1})$ for s_N .

For each i , we check whether predictions were correct:

- if s_{i+1} and/or $M(s_1, \dots, s_i)$ are unknown, we take $c_i = U$;
- otherwise, we take $c_i = T$ if $M(s_1, \dots, s_i) = s_{i+1}$ and $c_i = F$ if $M(s_1, \dots, s_i) \neq s_{i+1}$.

This way, we get a sequence $c = (c_1, \dots, c_{N-1})$ of truth values describing how well prediction rule M worked. We can now apply the rule M to the sequence c to predict whether M will work the next time.

- If $M(c) = F$, this means that our own induction rule requires us to reject this rule. So, if $M(c) = F$, we say that M is *inconsistent* with the observations s .
- Otherwise, we say that M is *consistent* with s .

3.4 *Induction Versus Anti-induction Revisited*

Induction rule M_I means that:

- if the last n elements of s are T , then $M_I(s) = T$;
- if the last n elements of s are F , we take $M_I(s) = F$;
- otherwise, $M_I(s) = U$.

Anti-induction rule M_A means that:

- if the last n elements of s are T , then $M_A(s) = F$;
- if the last n elements of s are false (F), we take $M_A(s) = T$;
- otherwise, we take $M_A(s) = U$.

Here, for all s , we have $M_A(s) = \neg M_I(s)$, i.e., $M_A = \neg M_I$.

3.5 *General Result*

We had an example of a sequence s with which both M_I and M_A were consistent. What happens in the general case? Was it a weird example or is it a general phenomenon?

We prove that this is a general phenomenon.

Proposition 1 *For each fair prediction rule M and for each sequence s :*

$$M \text{ is consistent with } s \Leftrightarrow \neg M \text{ is consistent with } s.$$

Proof Let us denote sequences c corresponding to M and $\neg M$ by c^M and $c^{\neg M}$. Here, $c_i^M = T$ if $M(s_1, \dots, s_i) = s_{i+1}$. This is exactly when we have $\neg M(s_1, \dots, s_i) \neq s_{i+1}$, i.e., when $c_i^{\neg M} = F$. Thus, $c_i^{\neg M} = \neg c_i^M$.

Based on the sequence $c^{\neg M}$, the rule $\neg M$ will predict

$$\neg M(c^{\neg M}) = \neg M(\neg c^M).$$

Since M is fair, we have $M(\neg c^M) = \neg M(c^M)$. Thus:

$$\neg M(c^{\neg M}) = \neg M(\neg c^M) = \neg \neg M(c^M) = M(c^M).$$

So, indeed, M and $\neg M$ are consistent or inconsistent simultaneously.

The proposition is proven.

4 Rules Must be Falsifiable

4.1 An Example Where a Reasonable Prediction Rule is Inconsistent

The fact that both induction and anti-induction rules are consistent with the same observations makes one think that maybe all reasonable rules are always consistent with all the observations. That would be bad, because if something cannot be disproved by experiment, this does not sound very scientific.

Let us show that this is not the case.

Indeed, a natural rule M_m is to go by majority:

- if in s , we had more T than F , we predict T ;
- if in s , we had more F than T , we predict F ;
- otherwise, we predict U .

What happens if we apply this rule to a periodic sequence $s = (T, F, T, F, \dots)$ for which:

- we have $s_{2k} = F$ for all k , and
- we have $s_{2k+1} = T$ for all k .

Here:

- For even $i = 2k$, we have equally many T s and F s, so $M(s_1, \dots, s_i) = U$, thus $c_i = U$.

- For odd $i = 2k + 1$, we have more T s than F s, so $M(s_1, \dots, s_i) = T$. For $i = 2k + 1$, we have $s_{i+1} = s_{2k+2} = F$, so $c_{2k+1} = F$.

So, the sequence c_i has only F s and U s. Thus $M(c) = F$.

In other words, the majority rule is inconsistent with this sequence.

4.2 A Problem with Simple Induction

Let us show that, somewhat unexpectedly, the simple induction M_I —as described in the previous sections—cannot be falsified and is, thus, not a very scientific approach.

Proposition 2 *For $n > 1$, no sequence s is inconsistent with the prediction rule M_I*

Proof The only way to show that the observation sequence s is inconsistent with M_I is when the corresponding sequence c contains n false values in a row, i.e., if n times in a row, the prediction rule M_I did not work: $c_N = \dots = c_{N+n-1} = F$.

When it did not work the first time, this means that we have

$$s_{N+1} \neq M^I(s_{N-n+1}, \dots, s_N).$$

By definition of the simple induction rule M^I , this can happen in two possible situations:

- either we have $s_{N+1} = F$ and $s_{N-n+1} = \dots = s_N = T$,
- or we have $s_{N+1} = T$ and $s_{N-n+1} = \dots = s_N = F$.

Let us first consider the first situation. In this case, at the moment $N + 1$, the last n values of the sequence s are:

- several (namely, $n - 1$) T -values $s_{N-n+2} = \dots = s_N = T$,
- followed by an F -value $s_{N+1} = F$.

In this situation, the simple induction rule M^I does not predict anything at all, so we have $c_{N+1} = U$ (“unknown”), and we cannot have $c_{N+1} = F$.

Similarly, in the second situation, the last n values of the sequence s are:

- several (namely, $n - 1$) F -values $s_{N-n+2} = \dots = s_N = F$,
- followed by a T -value $s_{N+1} = T$.

In this situation, the simple induction rule M^I also does not predict anything at all, so we have $c_{N+1} = U$ (“unknown”), and we cannot have $c_{N+1} = F$.

In both situations, we cannot have $c_N = c_{N+1} = \dots = F$ and thus, the simple prediction rule indeed cannot be falsified. The proposition is proven.

The situation is not better for the simple anti-induction principle M_A either:

Corollary 1 *For $n > 1$, no sequence s is inconsistent with M_A .*

Proof This immediately follows from Proposition 2 if we take into account Proposition 1, according to which any sequence s is consistent with the prediction rule M_I if and only if it is consistent with its negation $M_A = \neg M_I$.

5 Conclusions and Future Work

5.1 Predictions: Naive Idea

How do we make predictions? At first glance, the situation sounds straightforward: if we observe some phenomenon sufficiently many (n) times, then we naturally conclude that the same phenomenon (e.g., rising of the sun) will happen again the next time. This argument is known as inductive reasoning.

5.2 What We Show: Situation is More Complex That it May Appear

However, in principle, we can consider the opposite rule: if something happens sufficiently many times, then we expect that the opposite will happen the next time. For example, if someone was speeding many times and never got caught, we expect that he/she will eventually get caught by the police.

So, which principle should we use for prediction: inductive reasoning or the above-described “anti-inductive” reasoning? A natural idea is to use the same principle to select the prediction principle itself. For example, if we believe in inductive reasoning, then if this principle led to good predictions n times, we expect it to be working the next time as well. Similarly, if we believe in anti-inductive reasoning, then:

- if this principle does not lead to good predictions n times in a row, we expect it to be working next time—and,
- vice versa, if anti-induction reasoning led to good predictions n times in a row, we expect this principle to fail next time.

This seems to provide an experimental way to test which principle better suits the observations.

Somewhat unexpectedly, we show that it is not possible to experimentally distinguish between the two principle: each sequence of observations which is consistent with induction is also consistent with anti-induction, and vice versa. Moreover, we show that neither of these two principle can be falsified at all—so both principles are dubious from the scientific viewpoint, according to which scientific laws and techniques must be, in principle, falsifiable by experiments.

5.3 *Future Work*

The above results are just the beginning. We need to analyze more realistic formulations of the induction rule this way, as well as other possible rules. We need some experiments: what will happen if we apply different rules to different sequences of observations? From the more theoretical viewpoint: can we algorithmically (and feasibly algorithmically) check whether a given prediction rule is falsifiable?

We hope that our work will inspire others analyze to these important methodological questions.

Acknowledgements This work was supported in part by the National Science Foundation grants:

- 1623190 (A Model of Change for Preparing a New Generation for Professional Practice in Computer Science);
- HRD-1834620 and HRD-2034030 (CAHSI Includes).

It was also supported by the program of the development of the Scientific-Educational Mathematical Center of Volga Federal District No. 075-02-2020-1478.

The authors are thankful to all the participants of the International Seminar on Computational Intelligence ISCI'2021 (Tijuana, Mexico, August 17–19, 2021), especially to Oscar Castillo and Patricia Melin, for valuable discussions.

References

1. Holland, J.H., Holyoak, K.J., Nisbett, R.E., Thagard, P.R.: *Induction: Processes of Inference, Learning, and Discovery*. MIT Press, Cambridge (1989)
2. Holyoak, K., Morrison, R.: *The Cambridge Handbook of Thinking and Reasoning*. Cambridge University Press, New York (2005)
3. Kosheleva, O.M., Kreinovich, V.Y., Zakharevich, M.I.: Why induction, not antiinduction? *Notices Am. Math. Soc.* **26**(7), A-619 (1979)